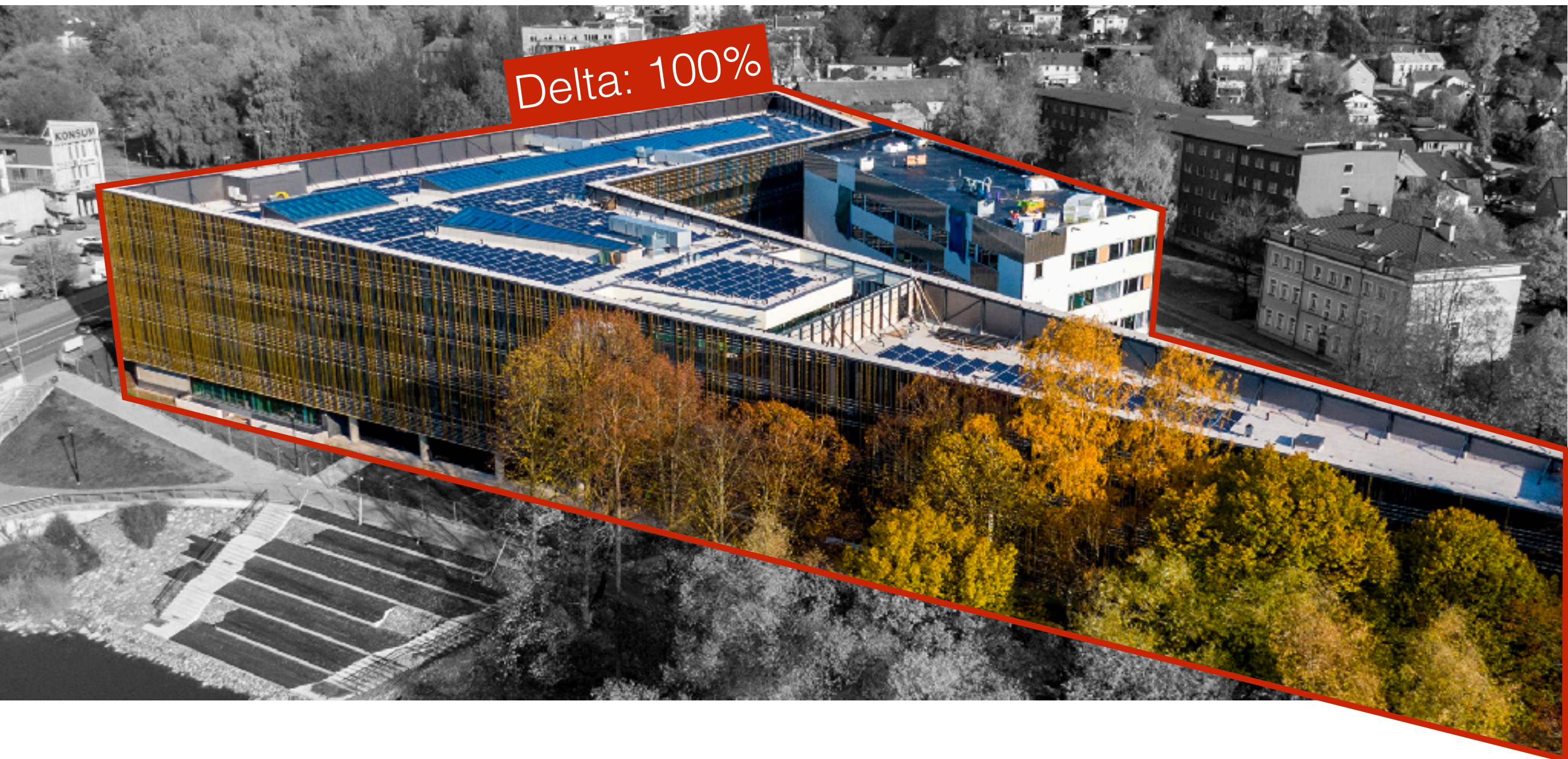


Deep Learning

Dmytro Fishman (dmytro.fishman@ut.ee)



UNIVERSITY OF TARTU
Institute of Computer Science

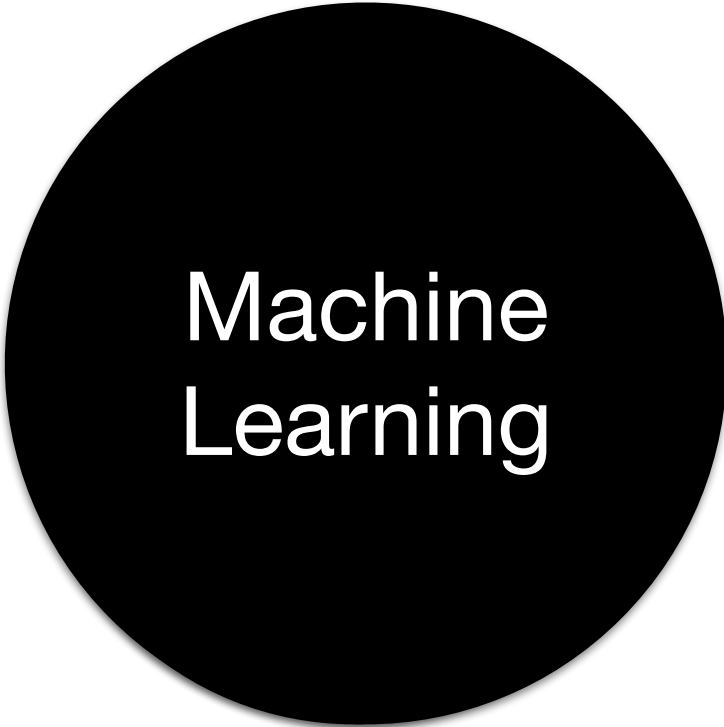
BIIT

October 4, 2021

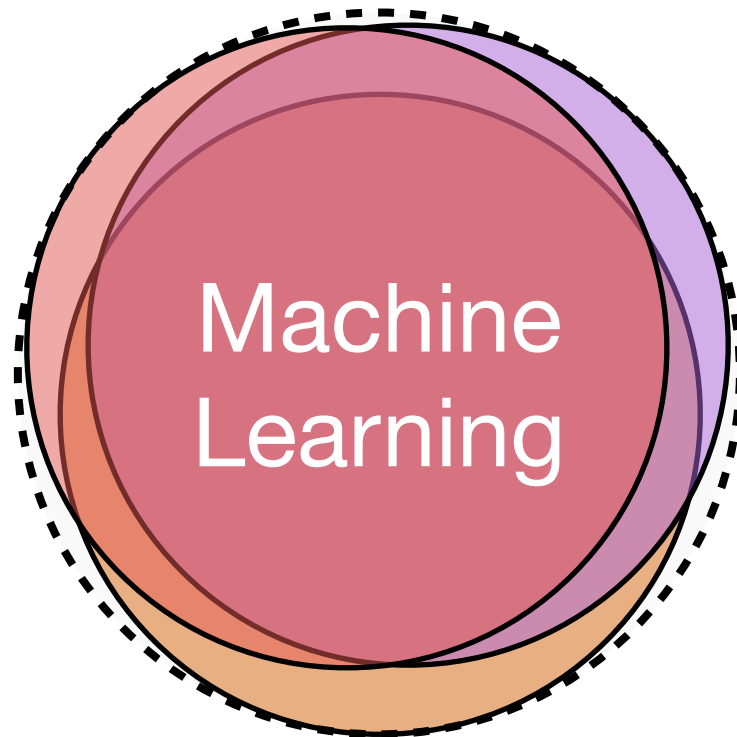
Deadlines

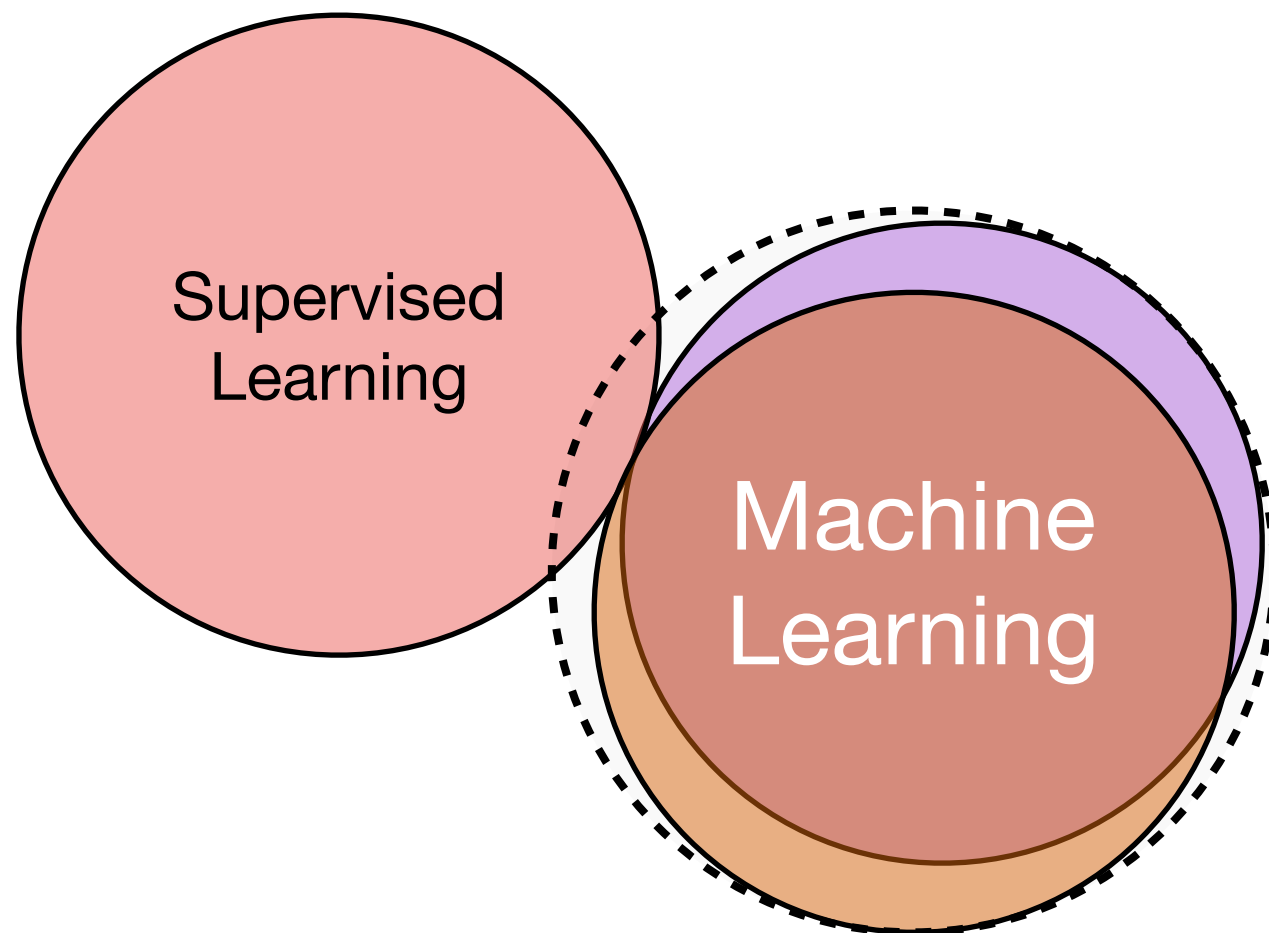
Assignment	Date of assignment	Deadline (midnight 23:59)
HW1	Sep 6	Sep 19
HW2	Sep 20	Oct 3
HW3	Oct 4	Oct 17
Paper summary	Oct 11	Oct 24
HW4	Oct 18	Oct 31
HW5	Nov 1	Nov 14
HW6	Nov 22	Dec 5
Project	Projects are collected	Dec 13 - 15

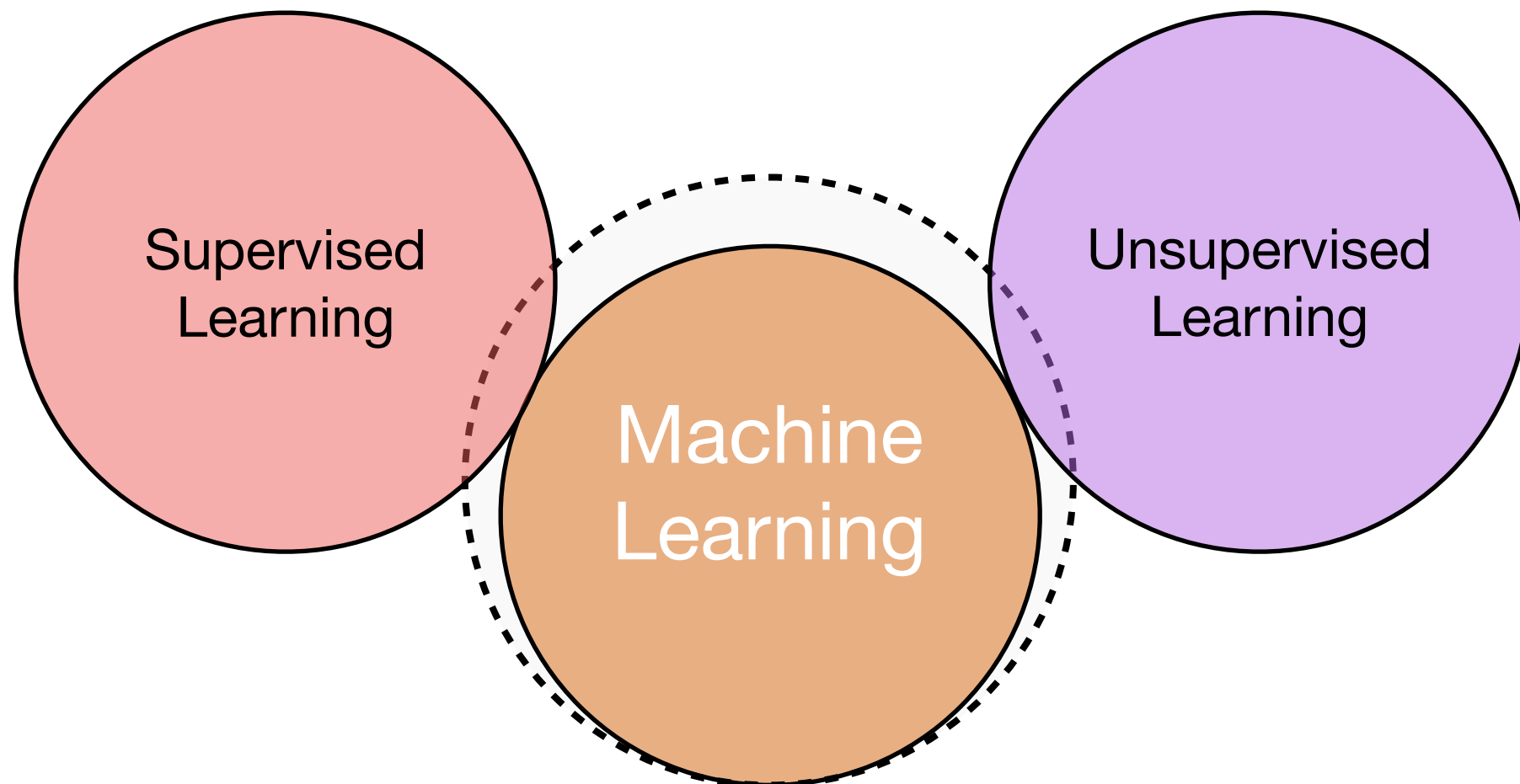
***All deadlines are subject to change, check out CampusWire and website for updates**

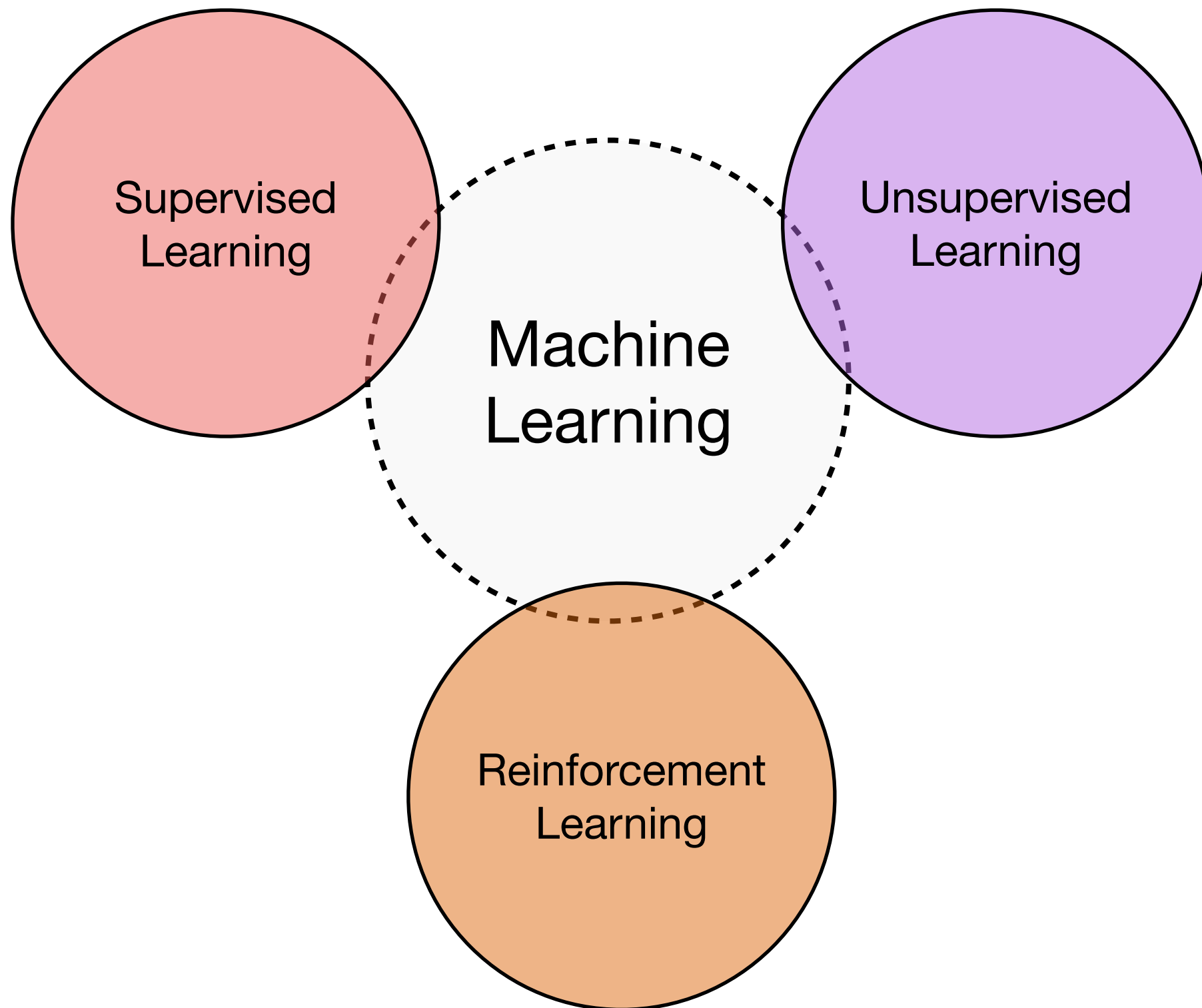


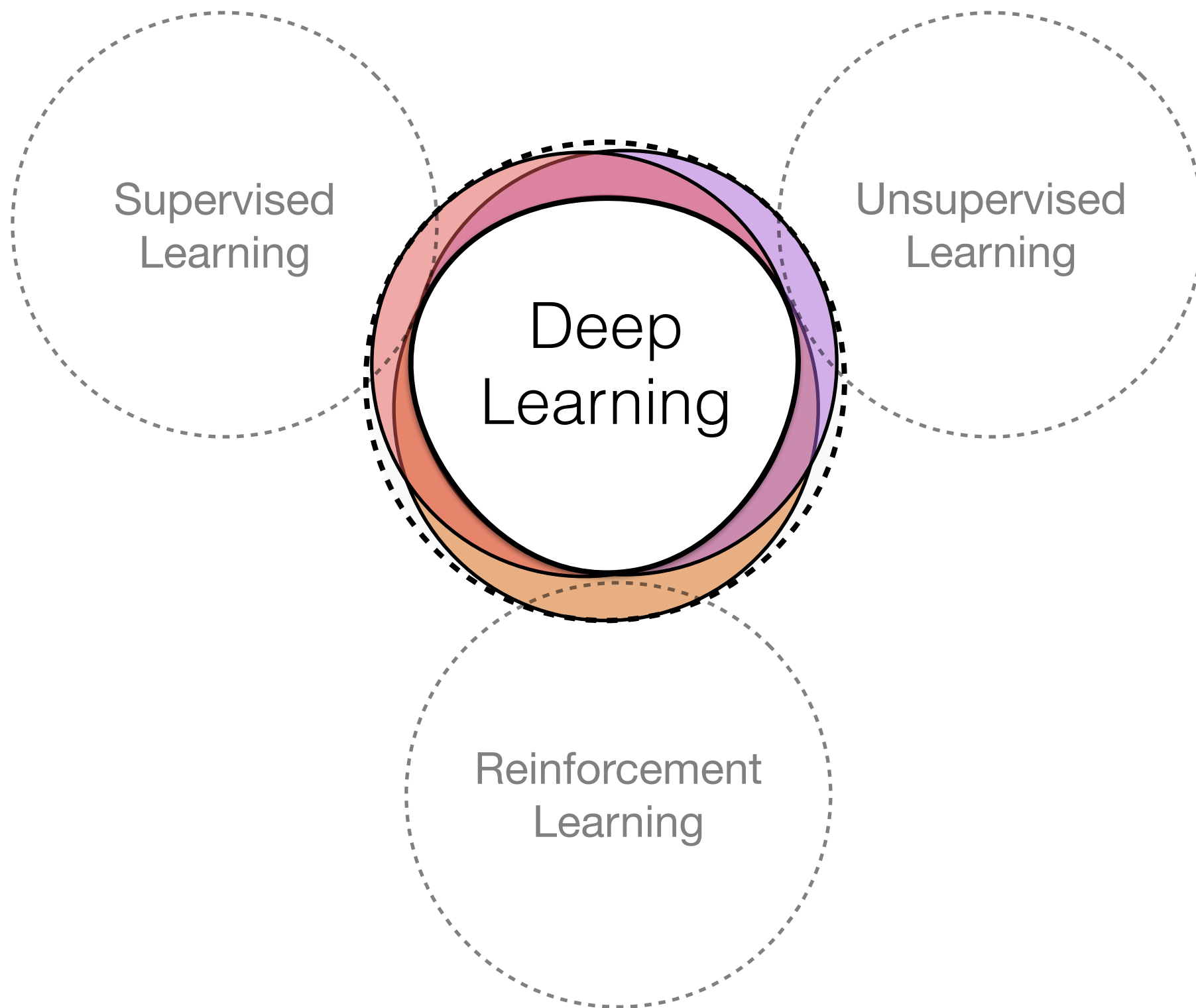
Machine
Learning











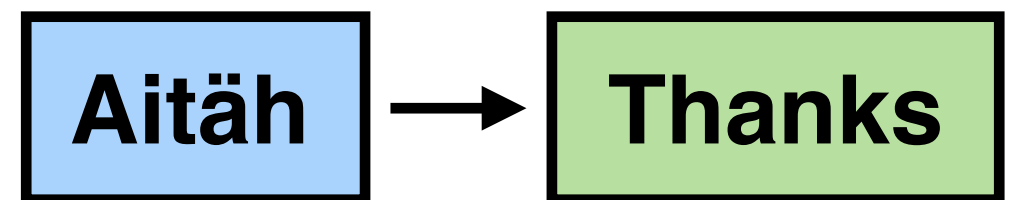
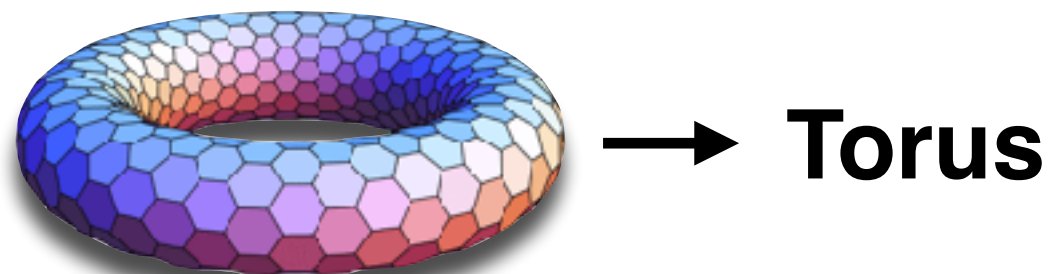
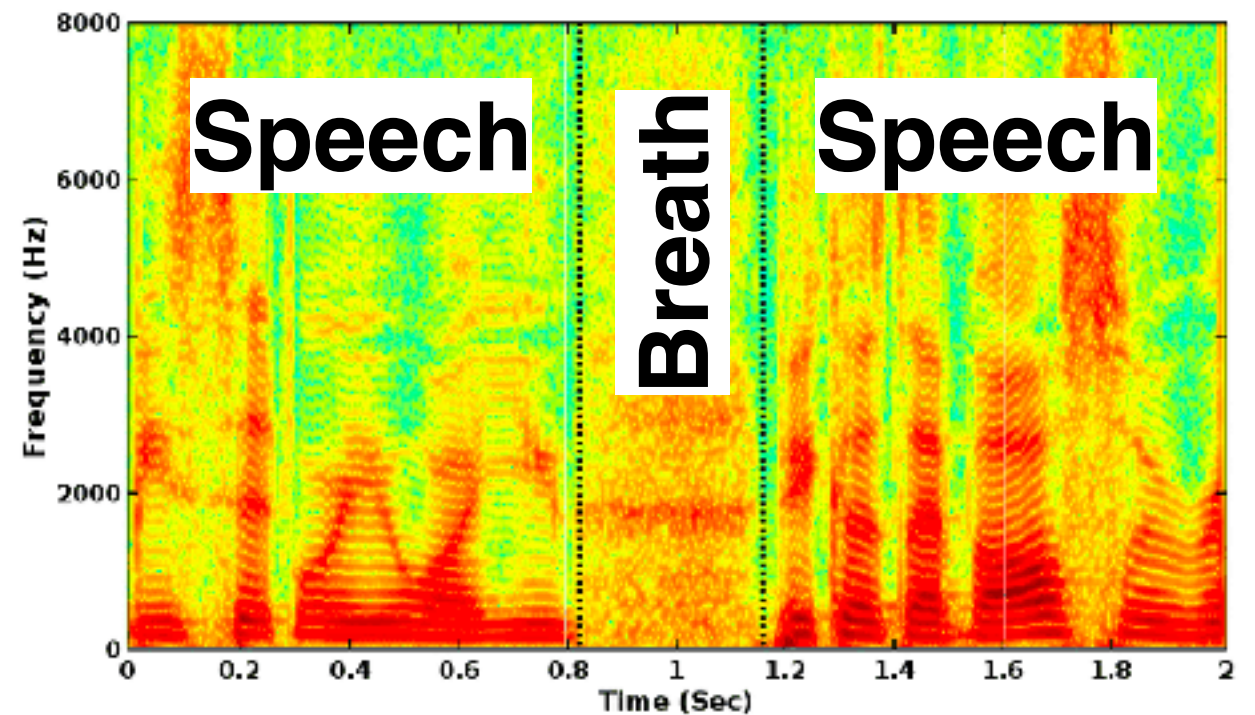
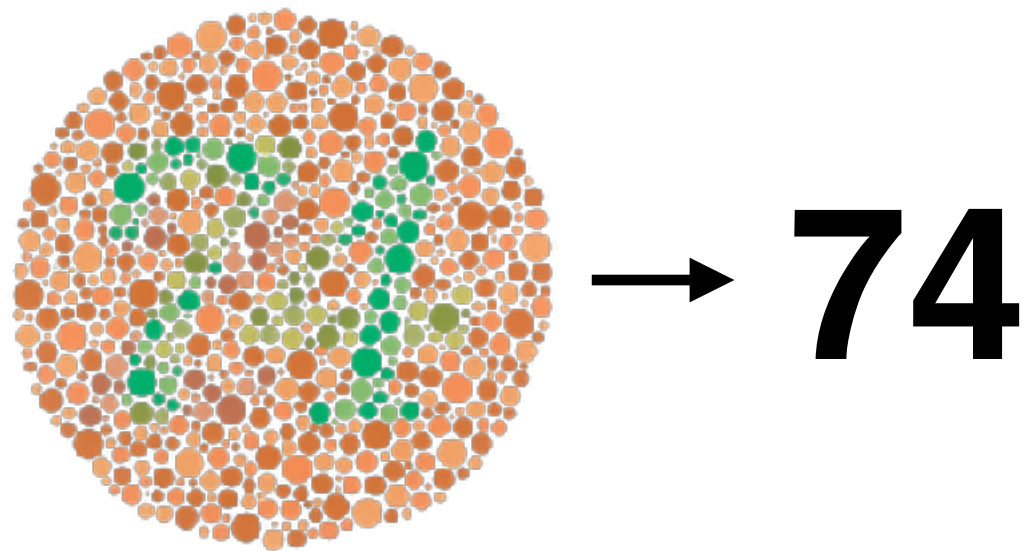


This handsome man will
talk to you real **Deep
Learning** (soon)

I am here to scratch
the surface :)

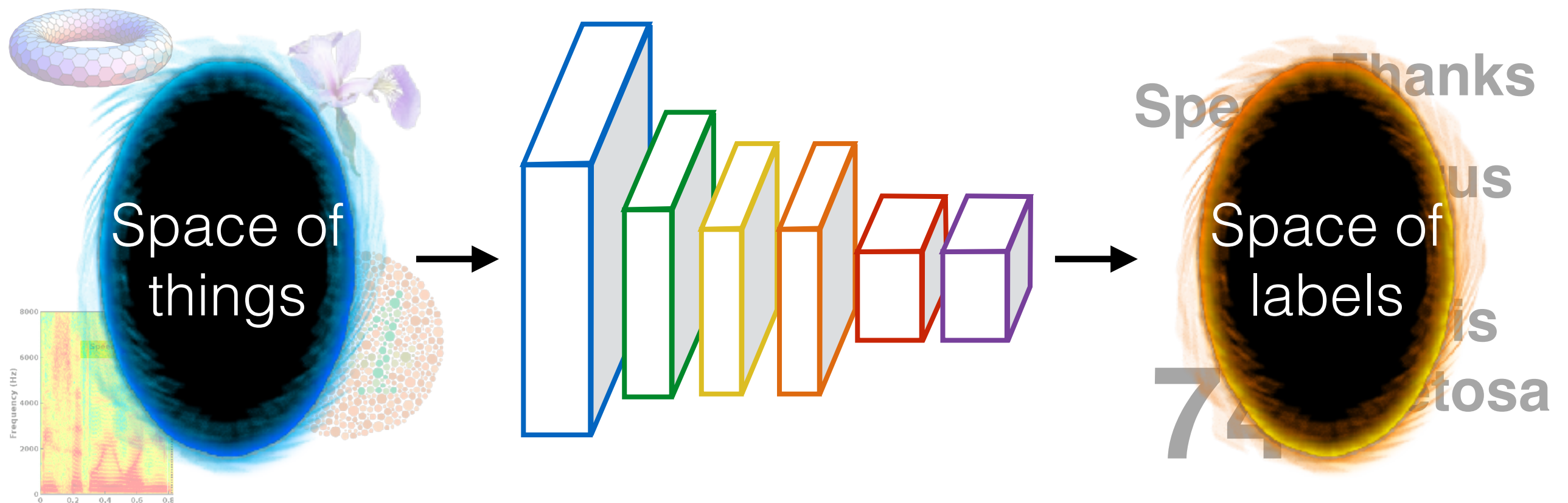
What is deep learning?

What is deep learning?



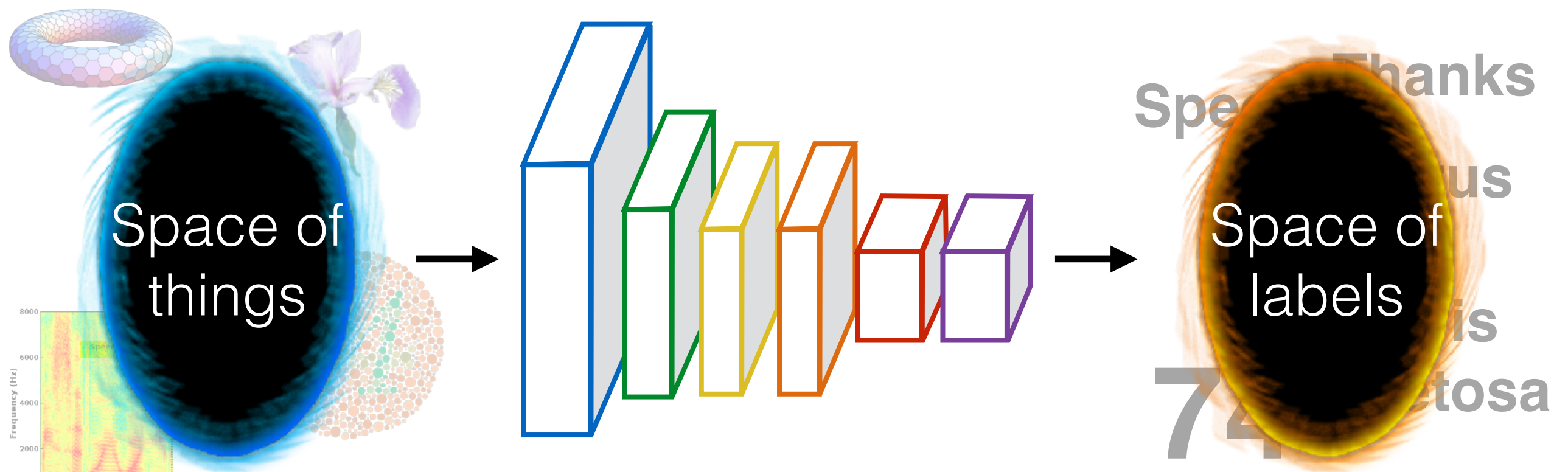
What is deep learning?

An adaptive **non-linear mapping** from one **space** to **another**



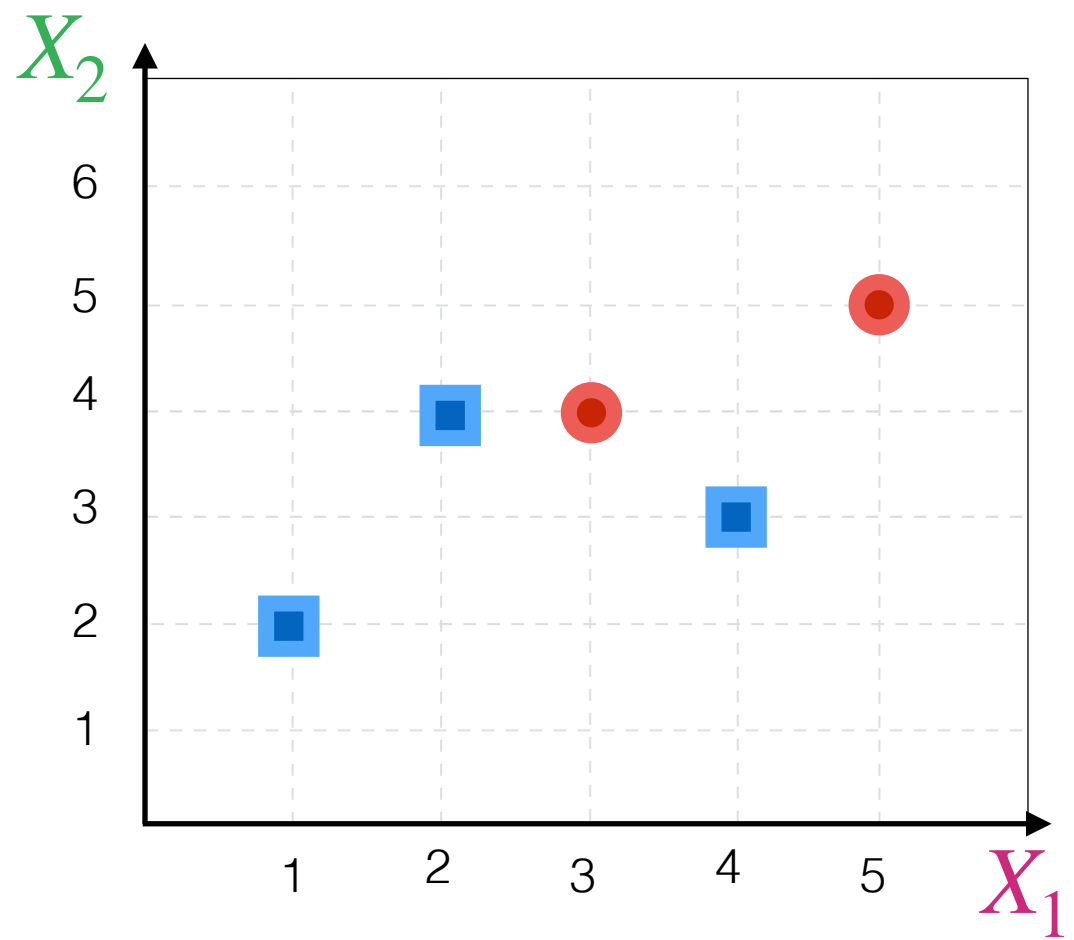
What is deep learning?

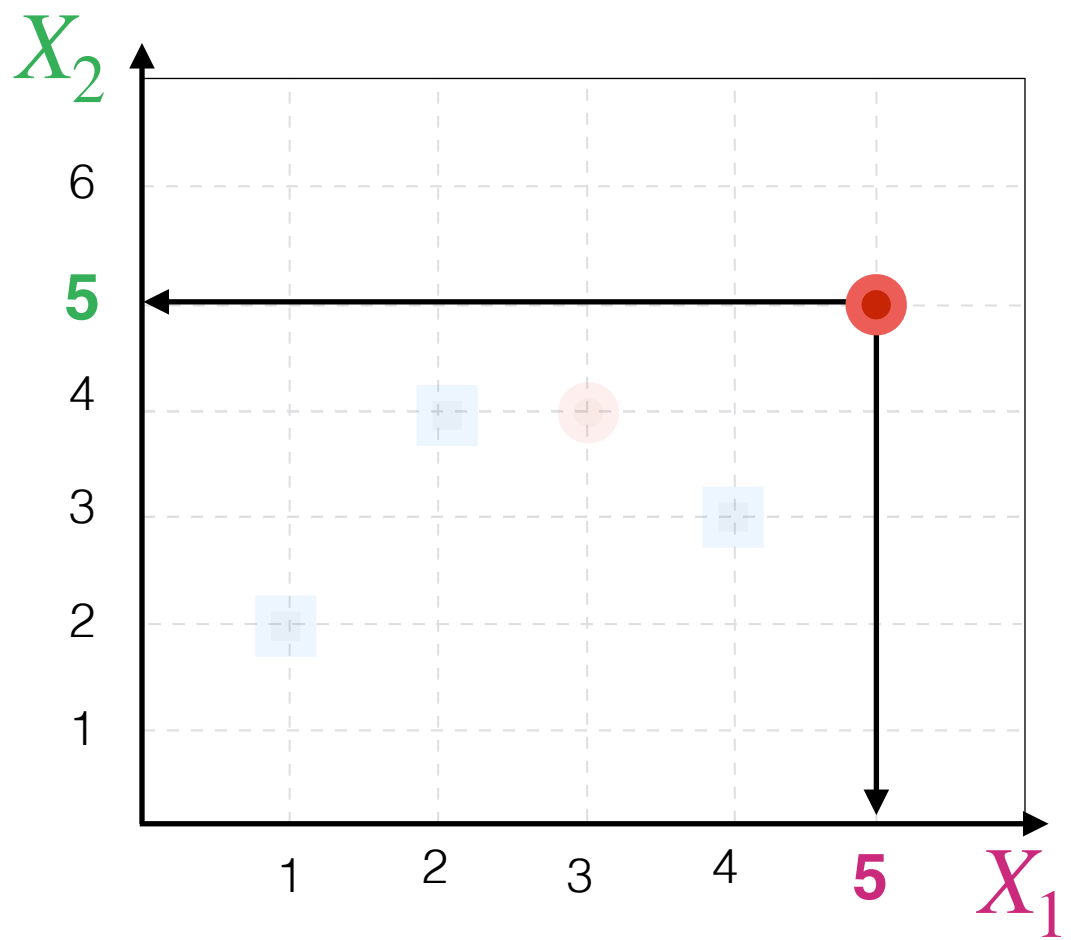
An adaptive **non-linear mapping** from one **space** to **another**

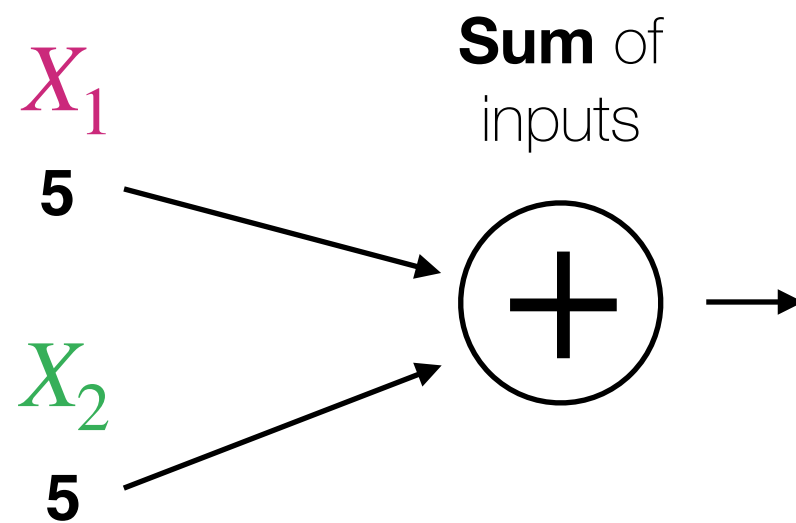
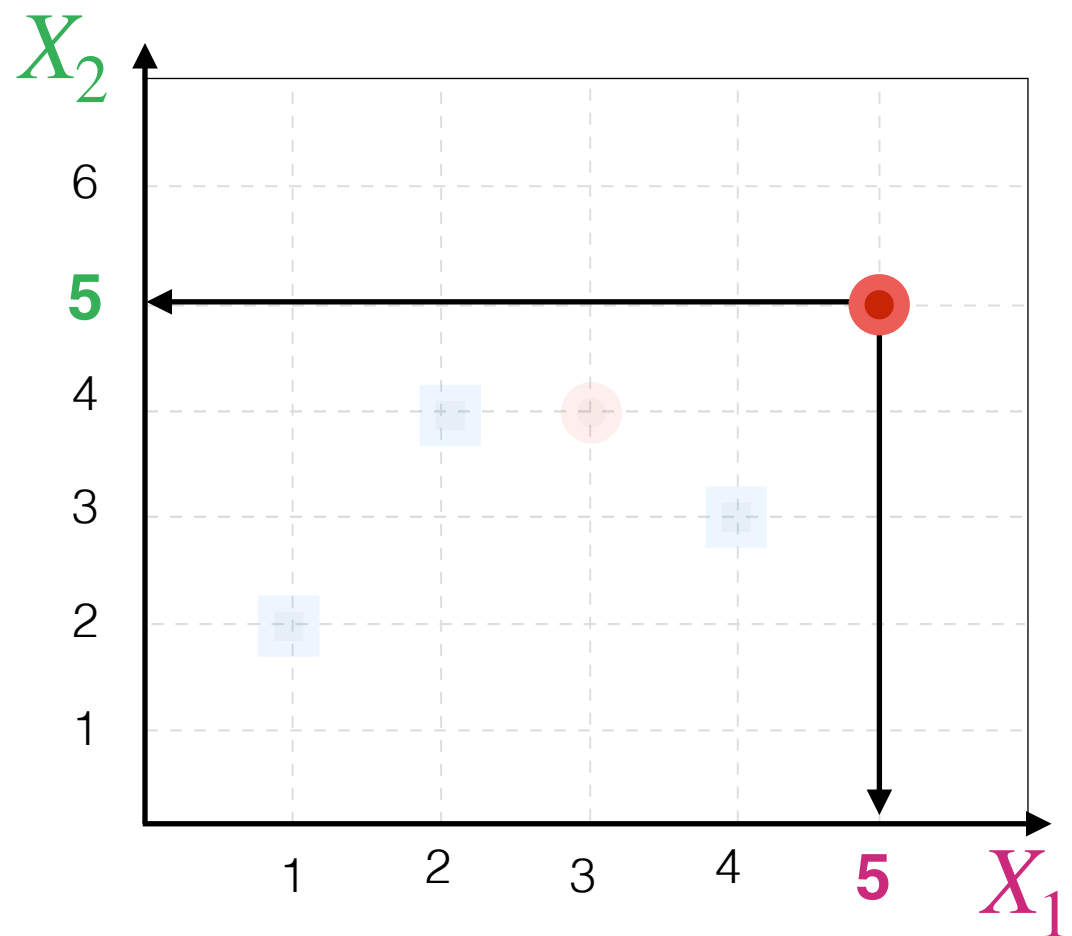


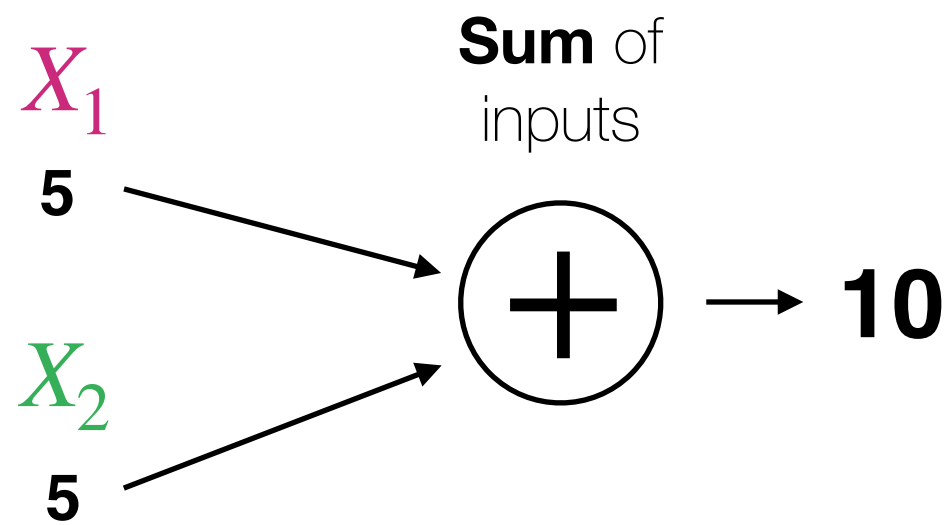
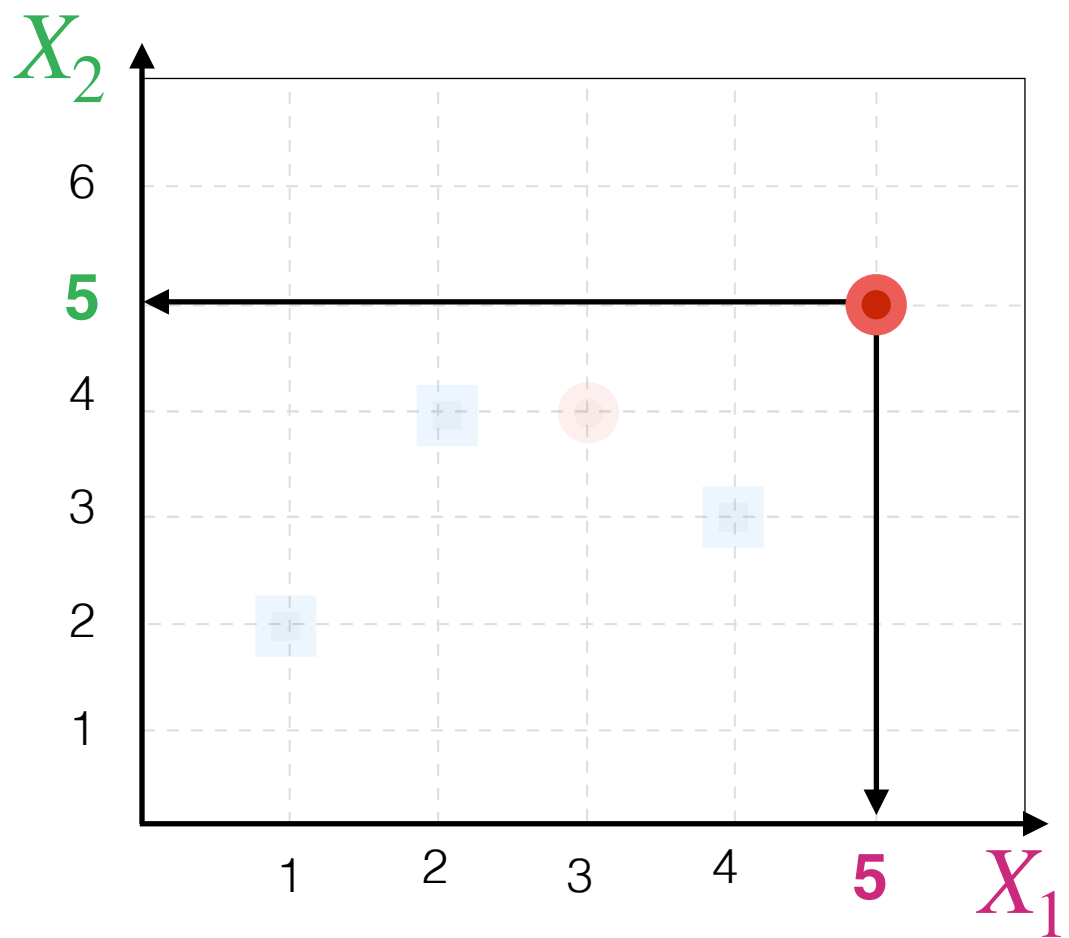
Artificial Neural

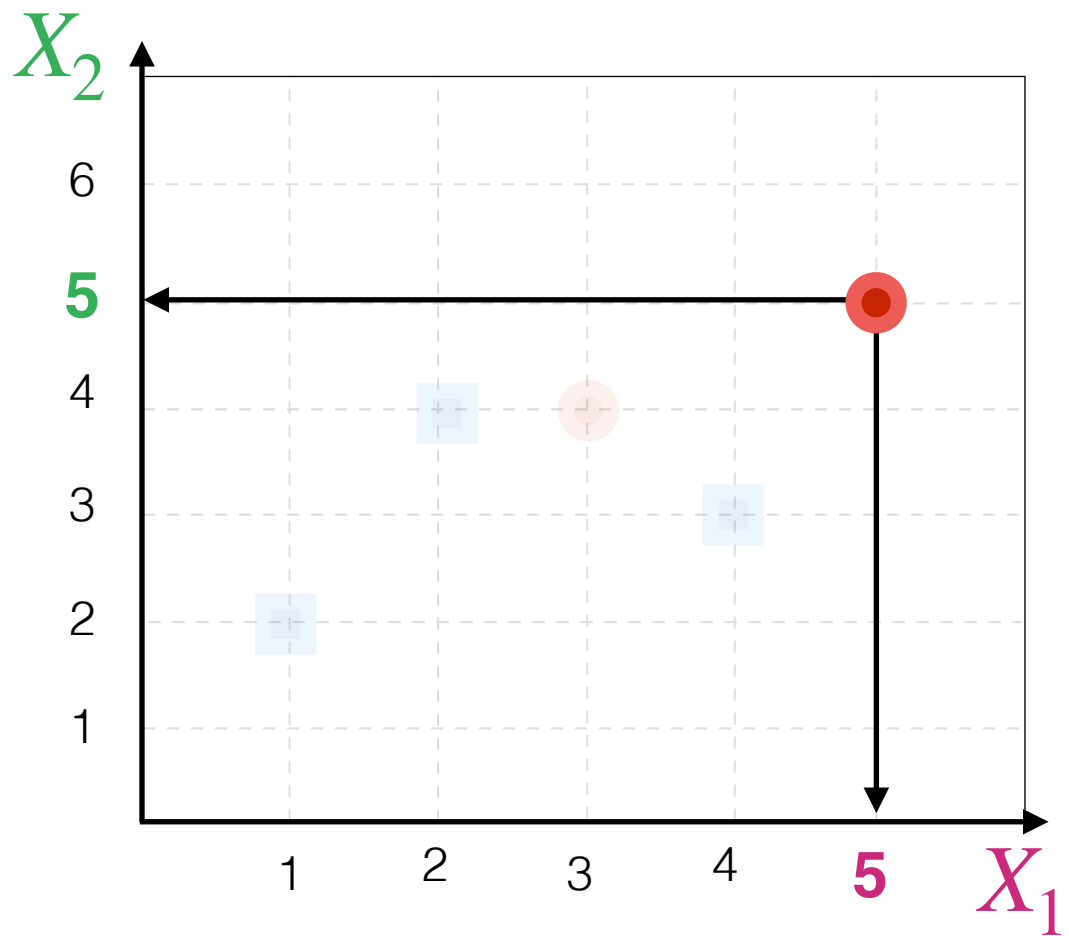
DL = Networks with many layers





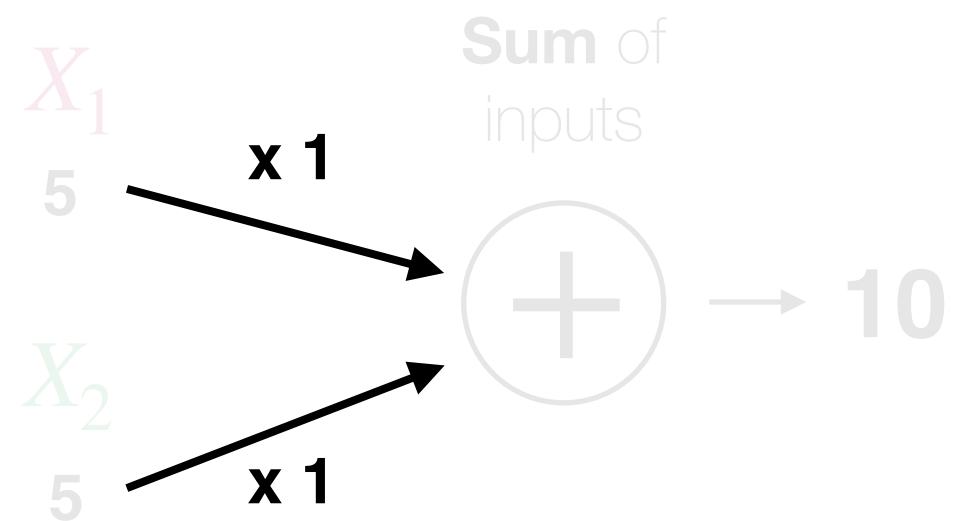


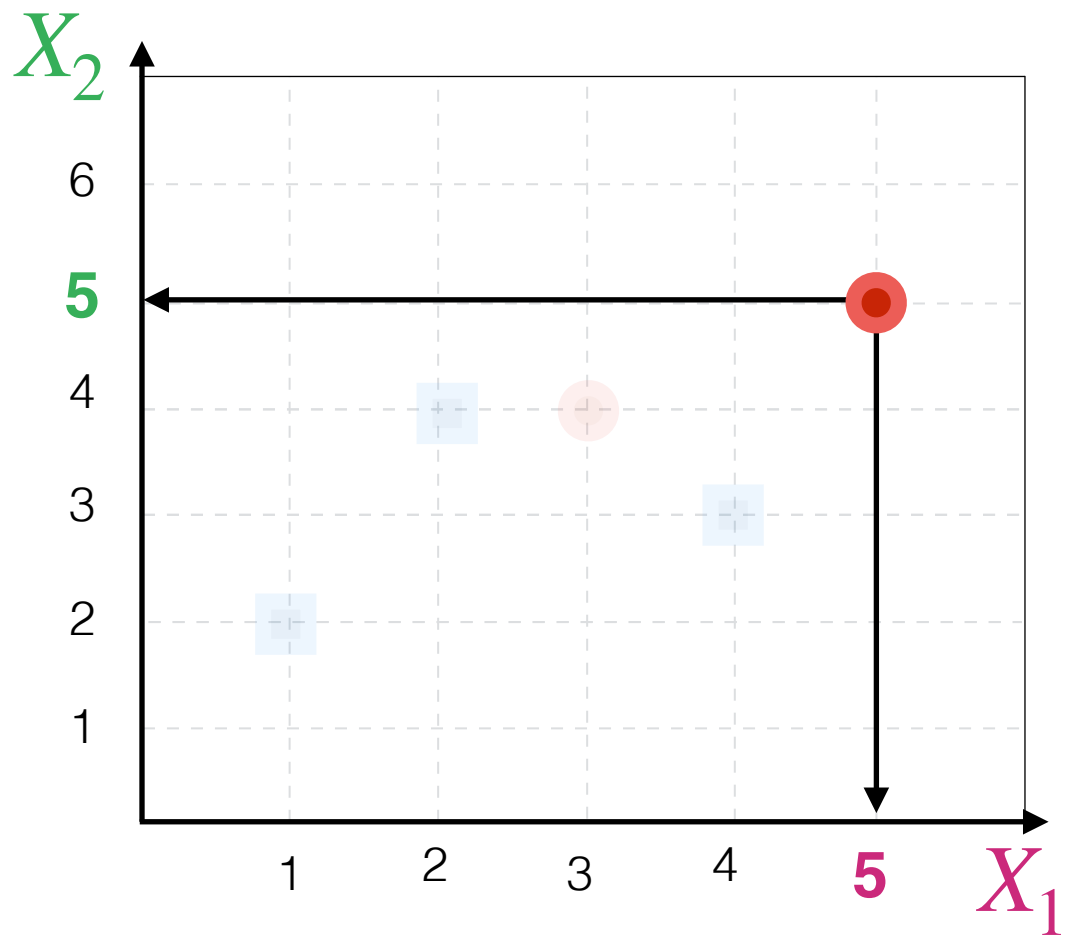




Weights

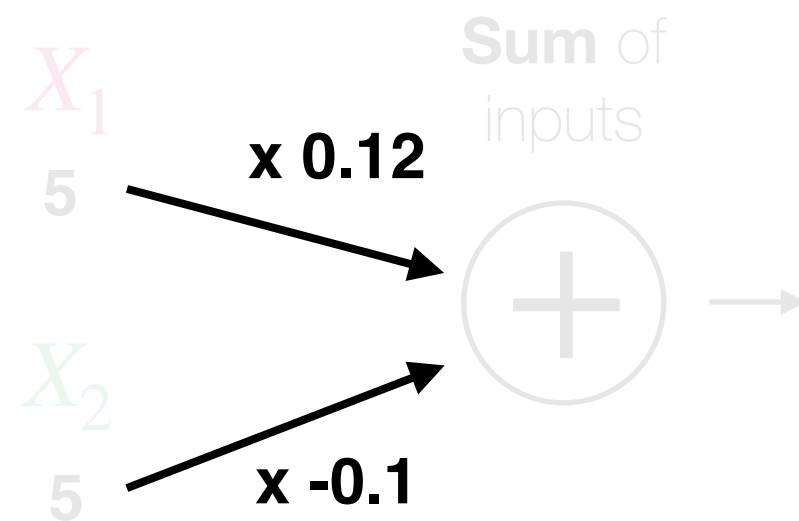
Weights are connections between nodes of the network



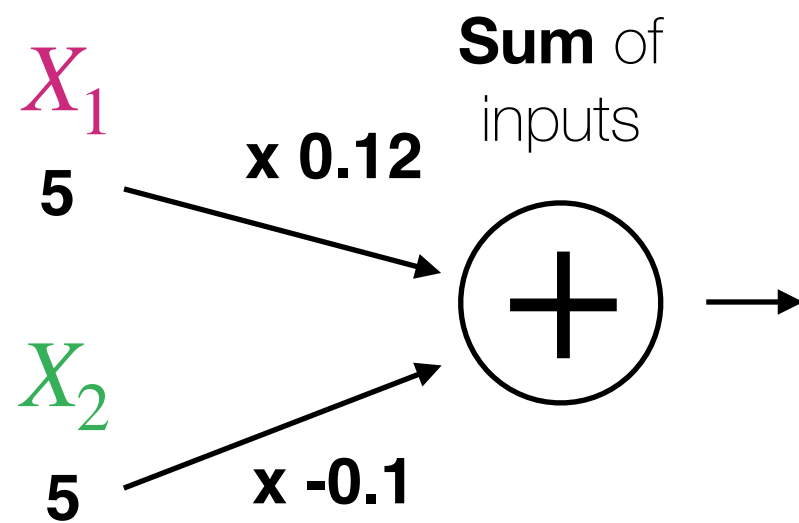
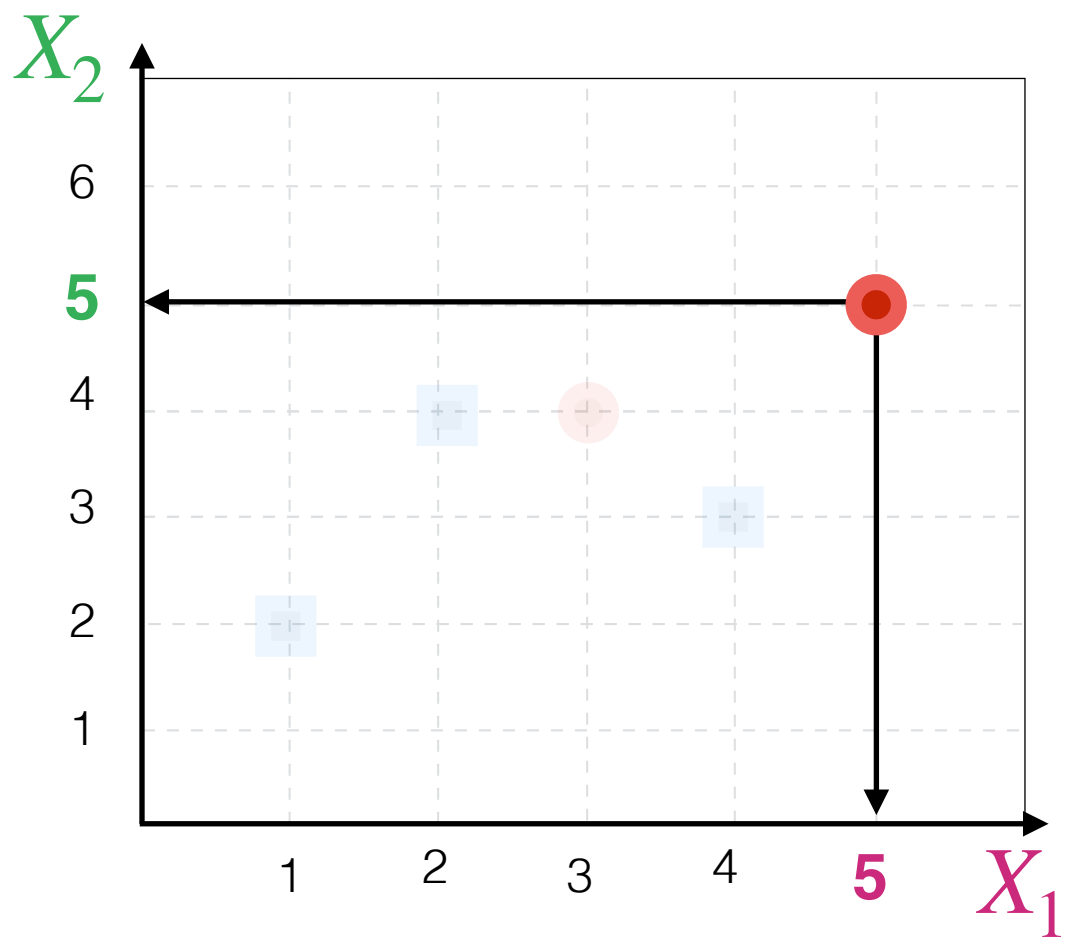


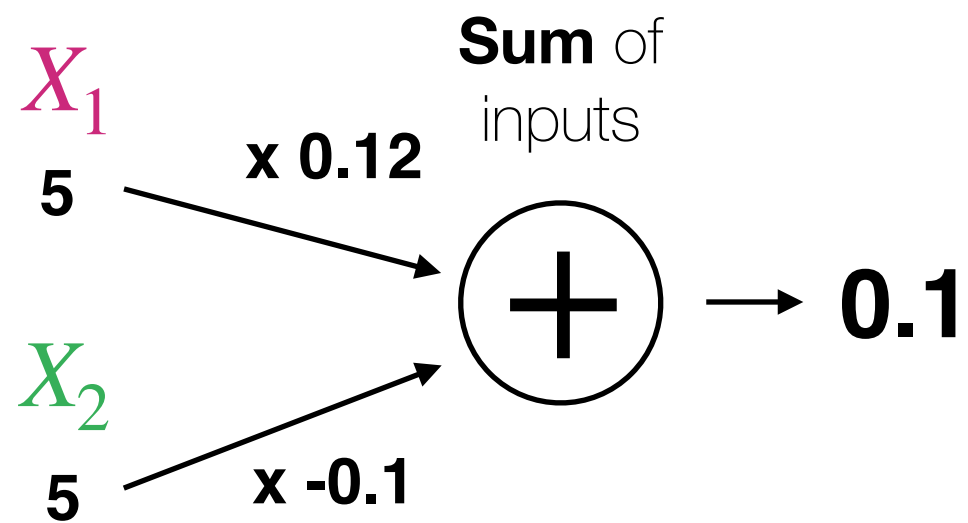
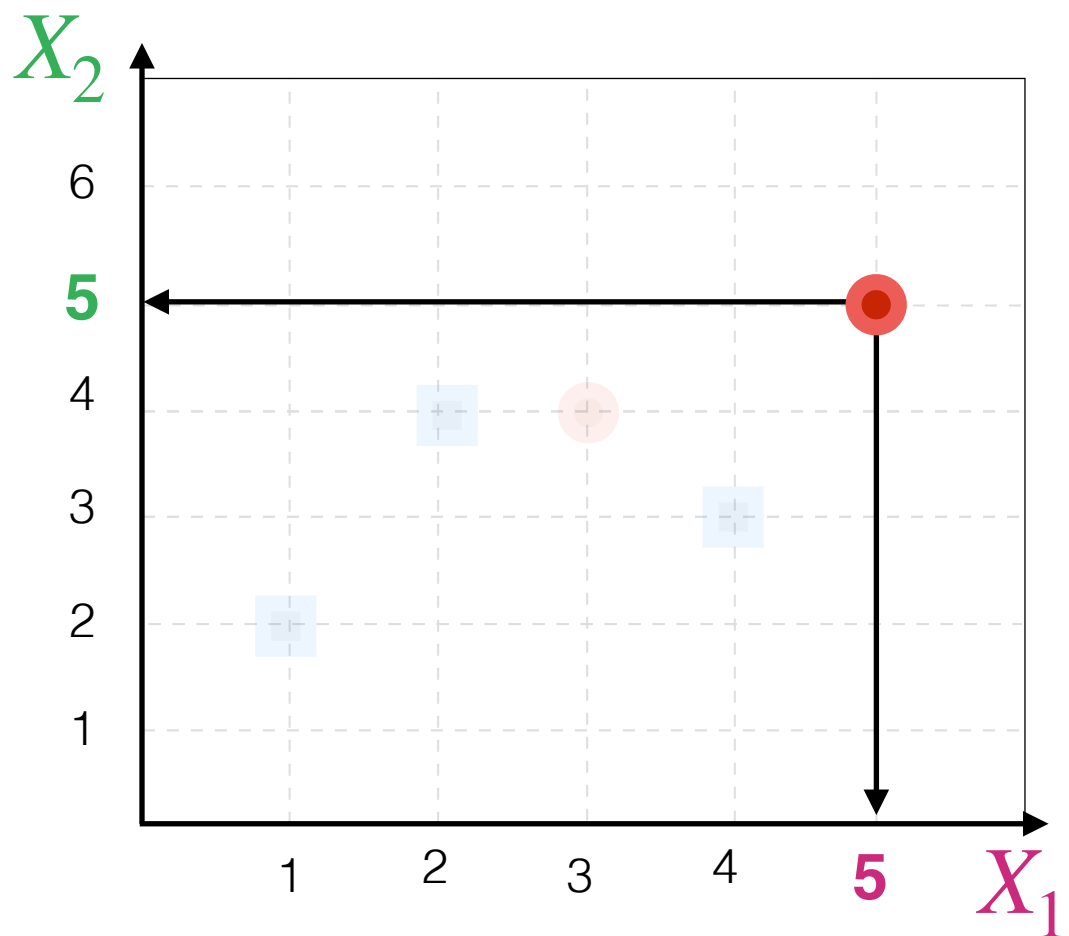
Weights

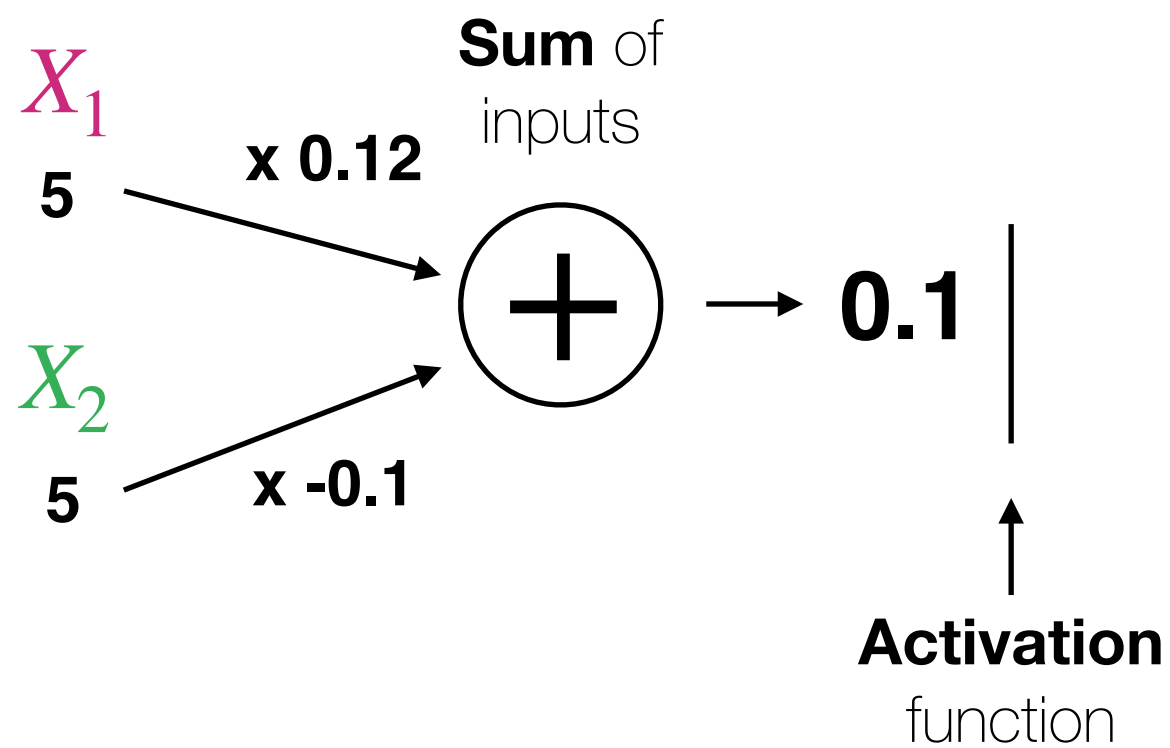
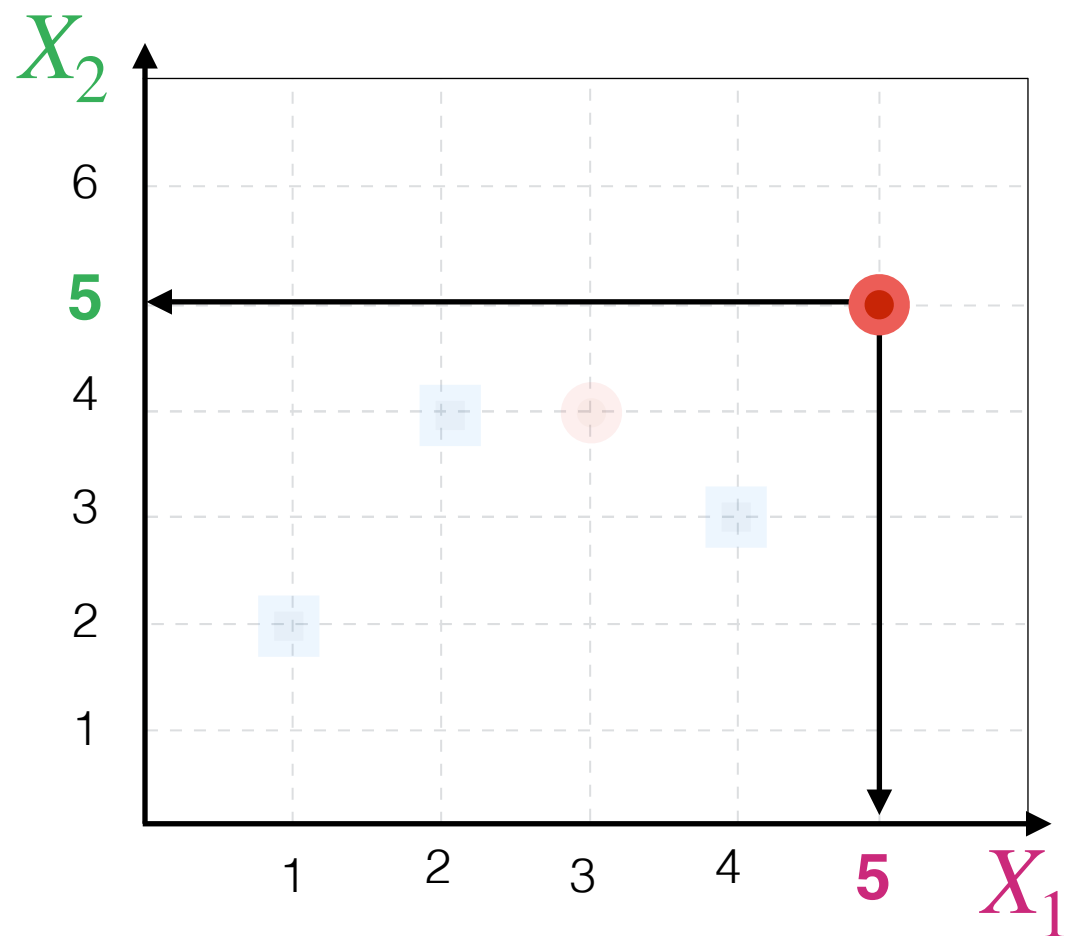
Weights are connections between nodes of the network

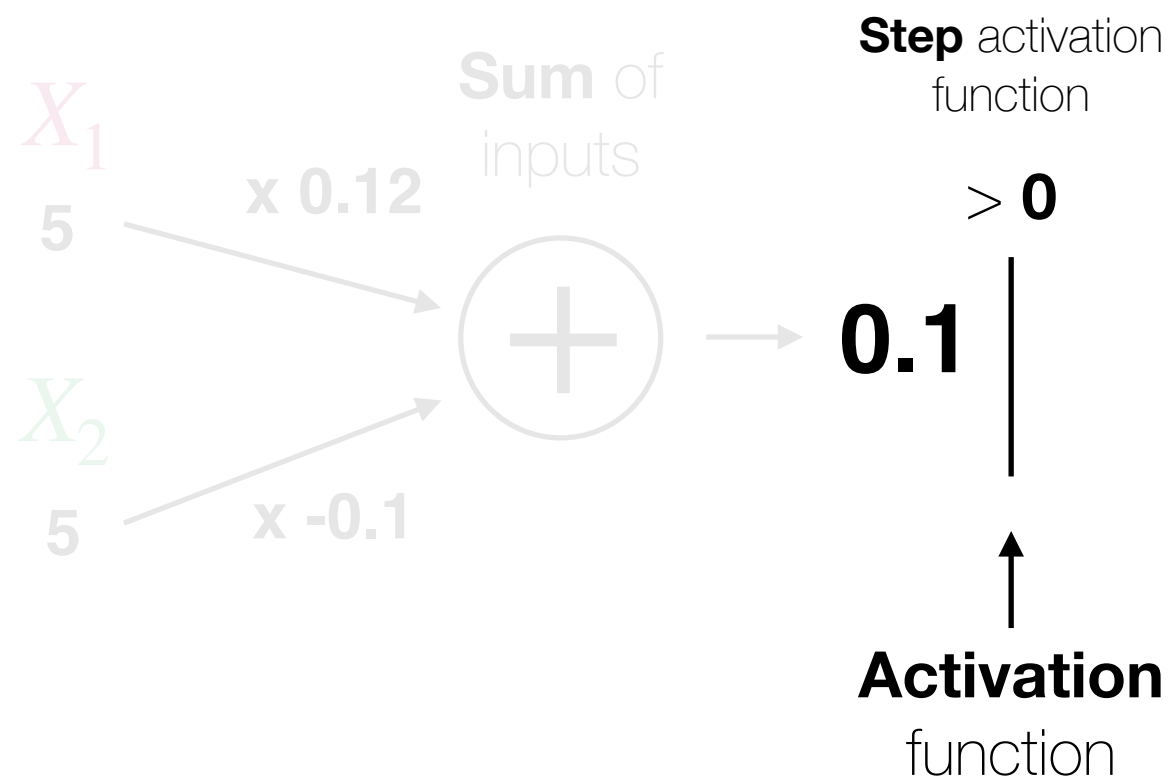
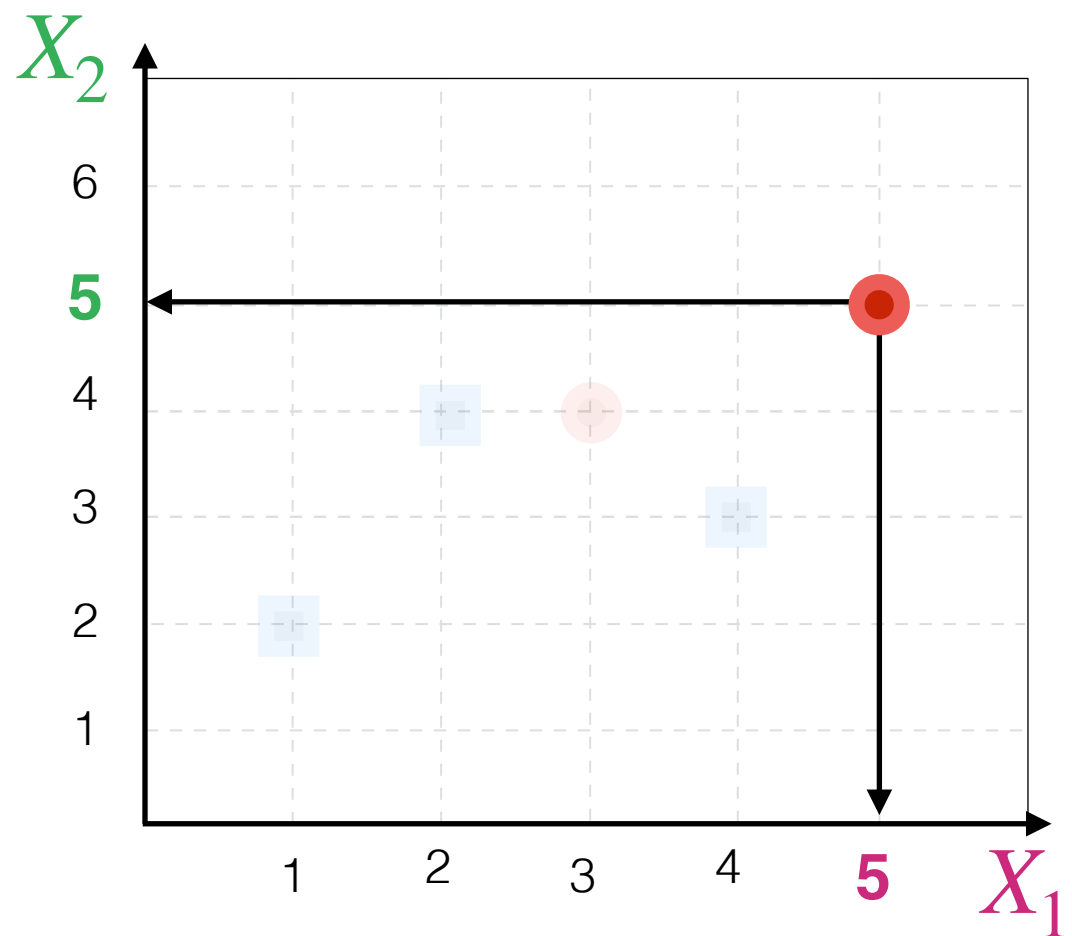


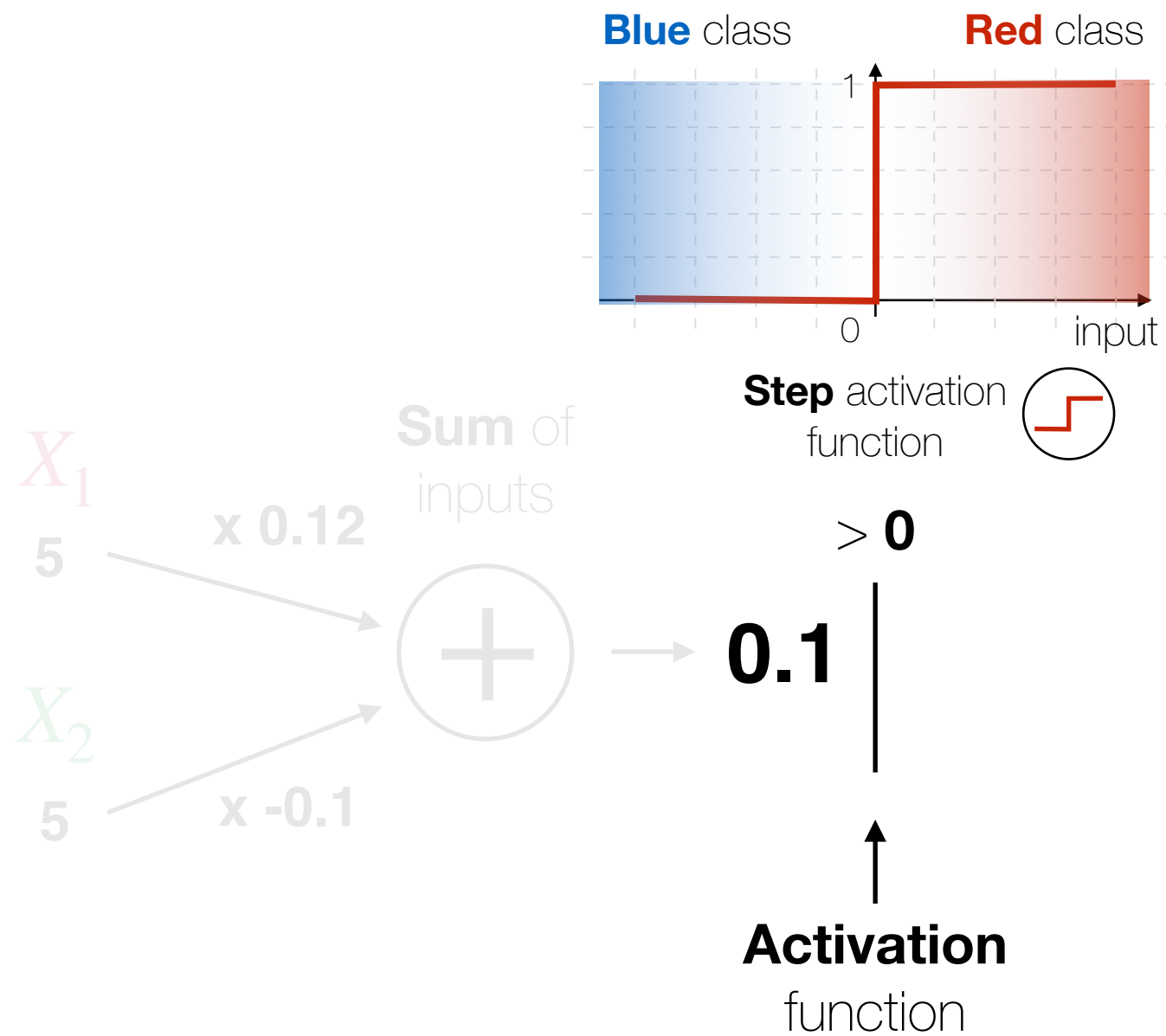
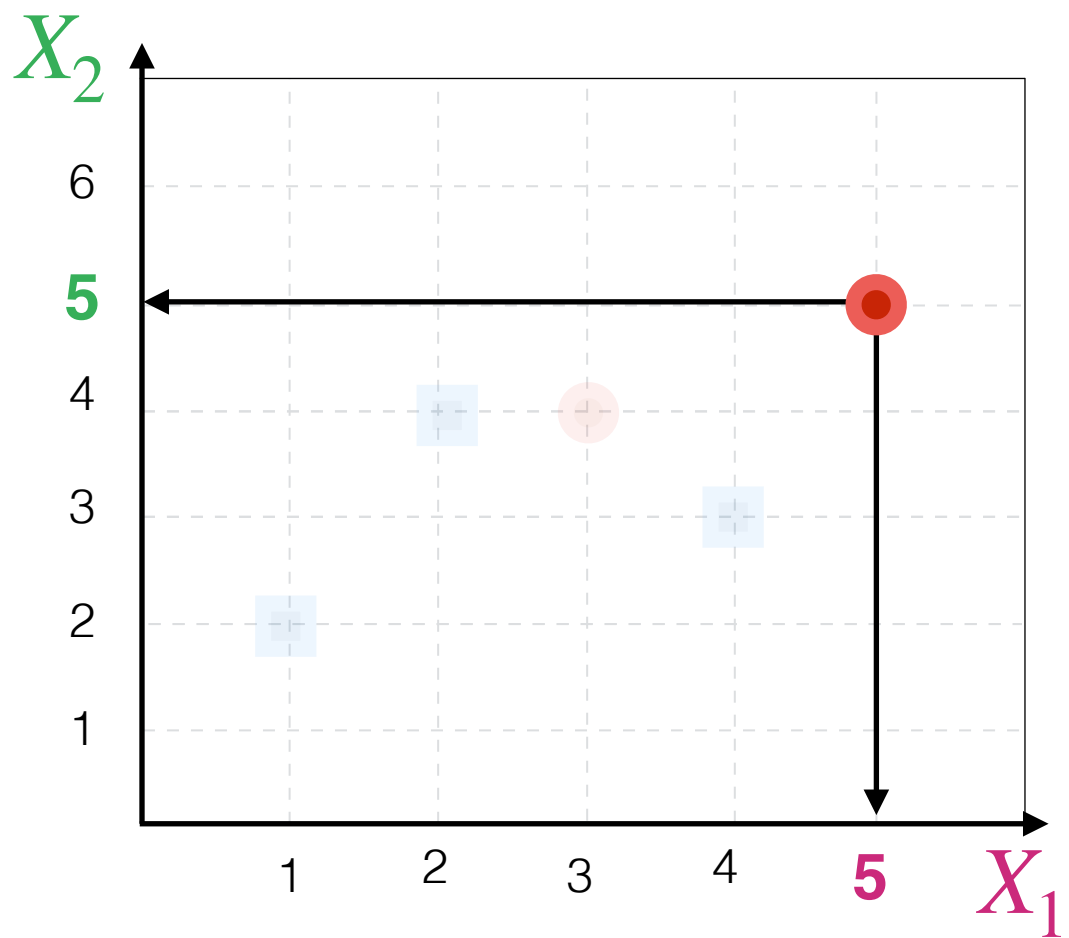
Weights are not necessarily positive integers

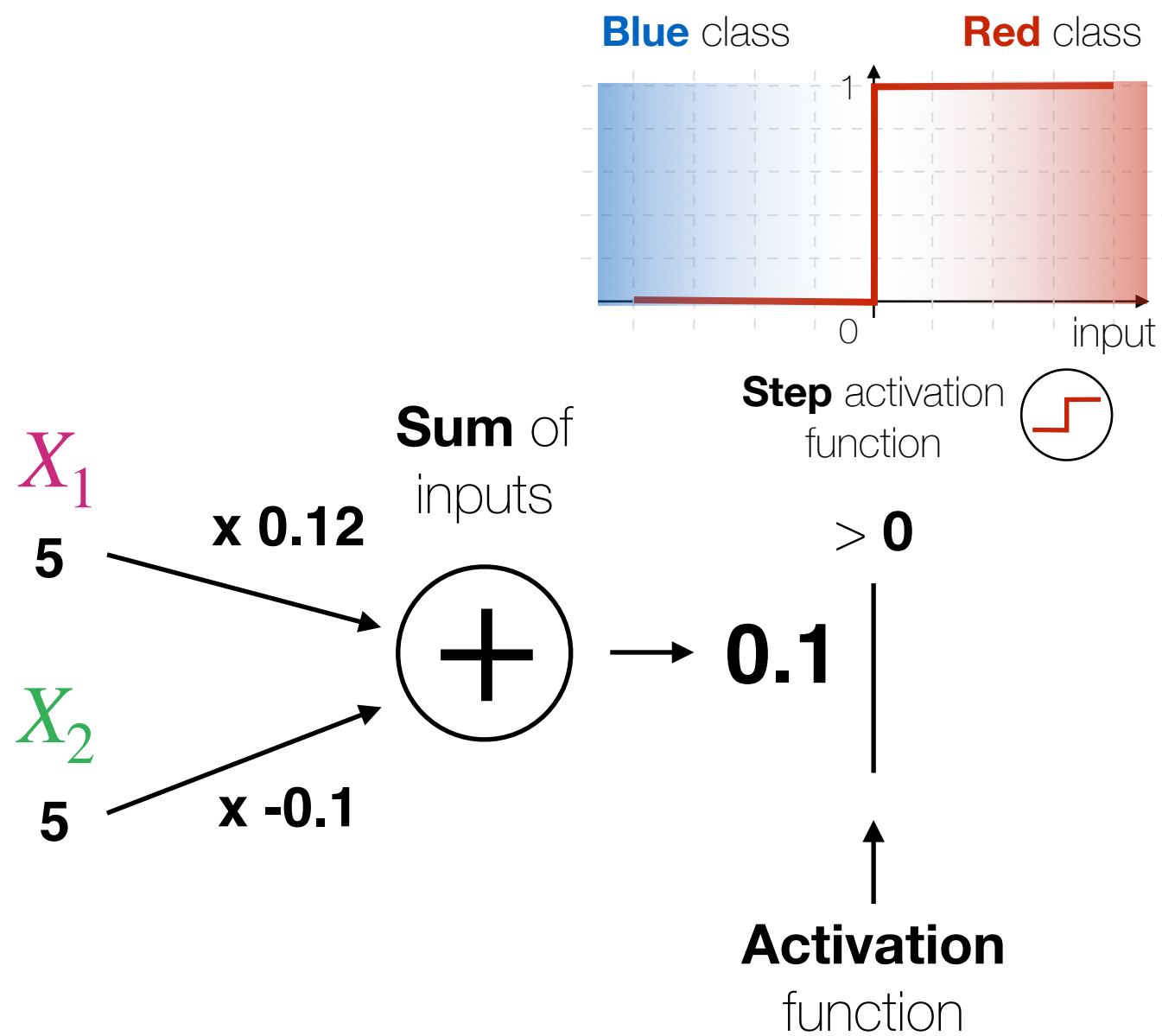
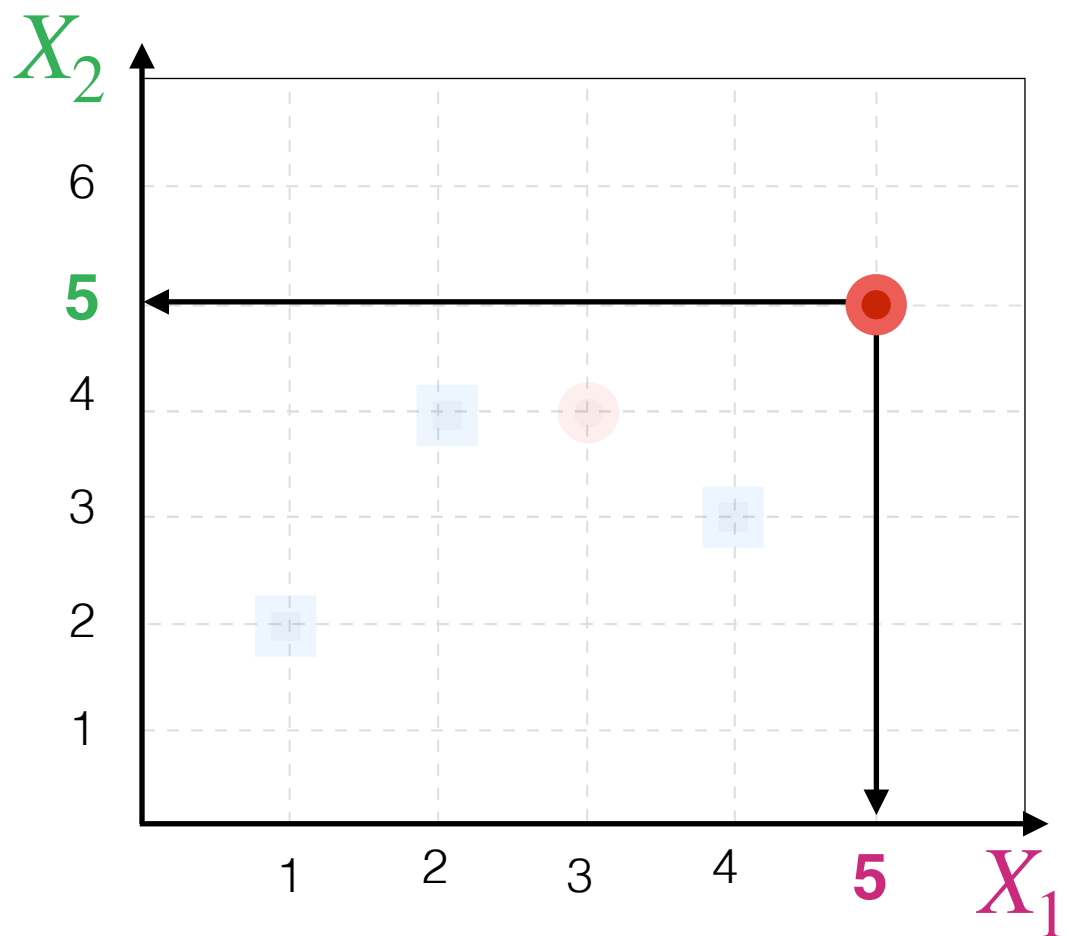


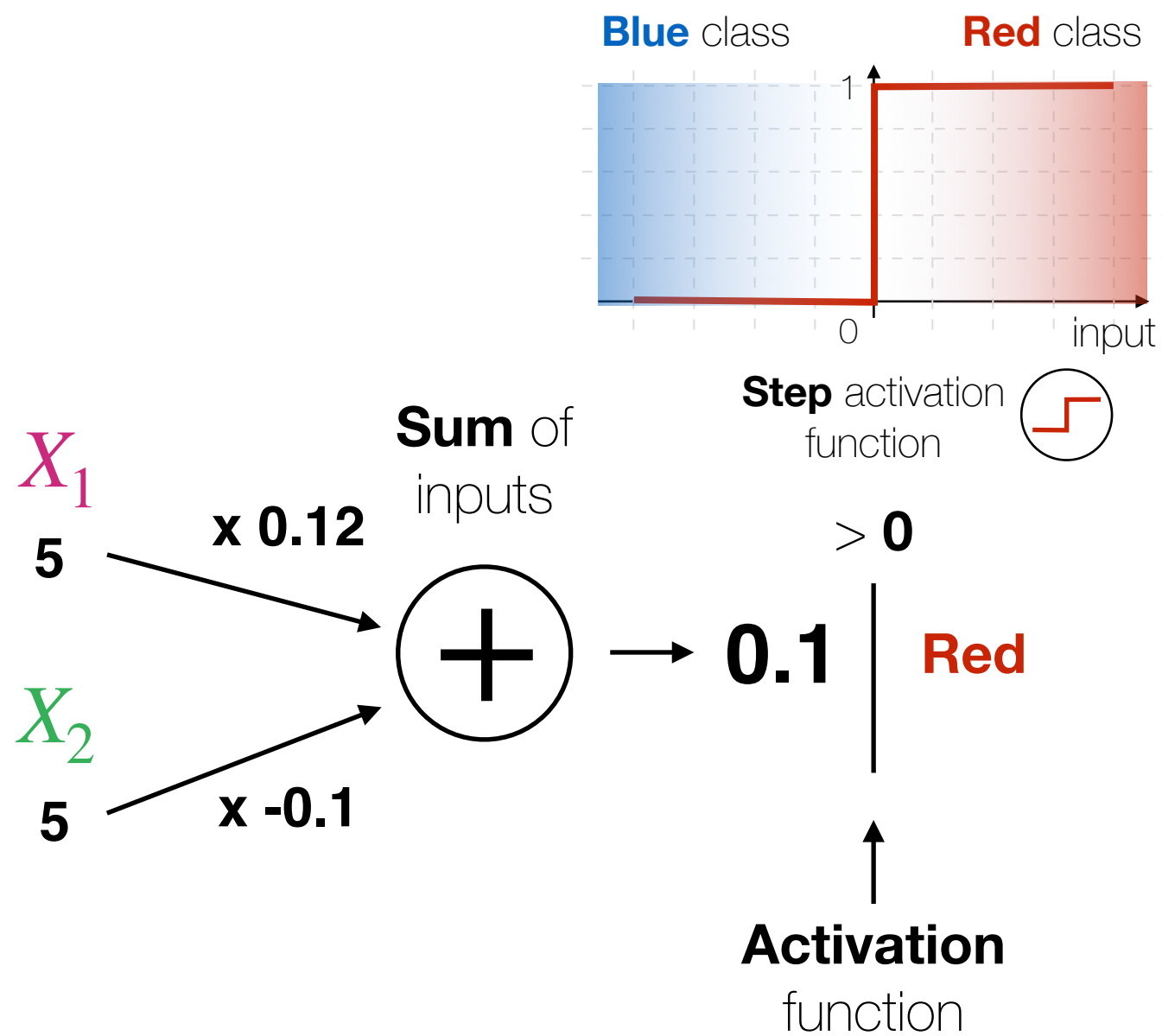
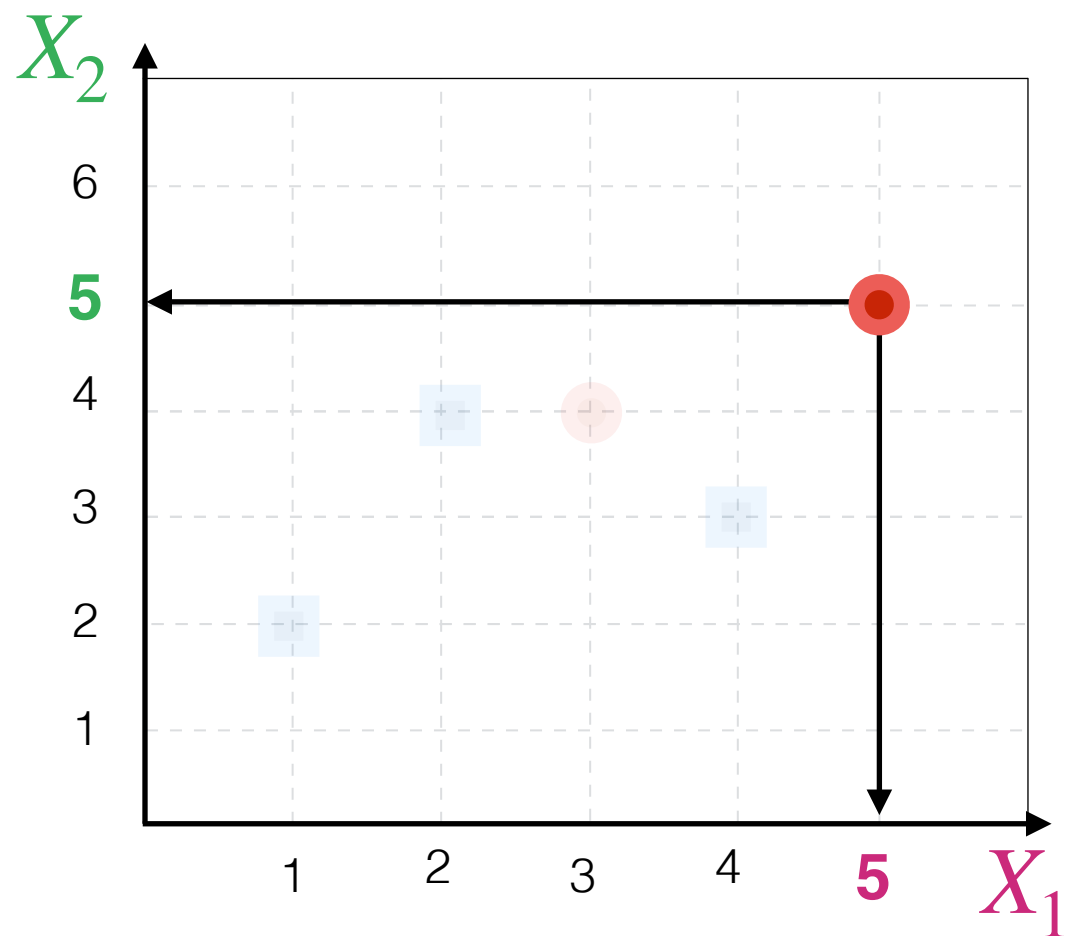


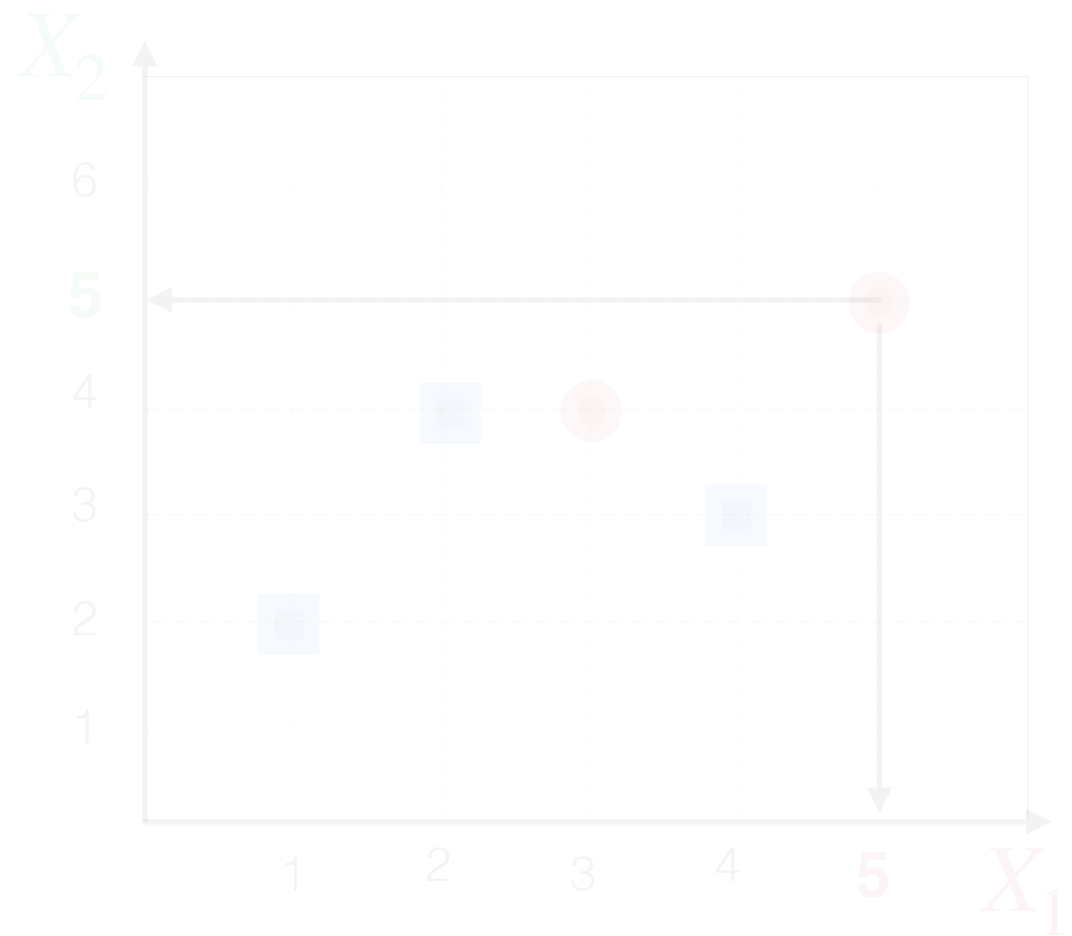




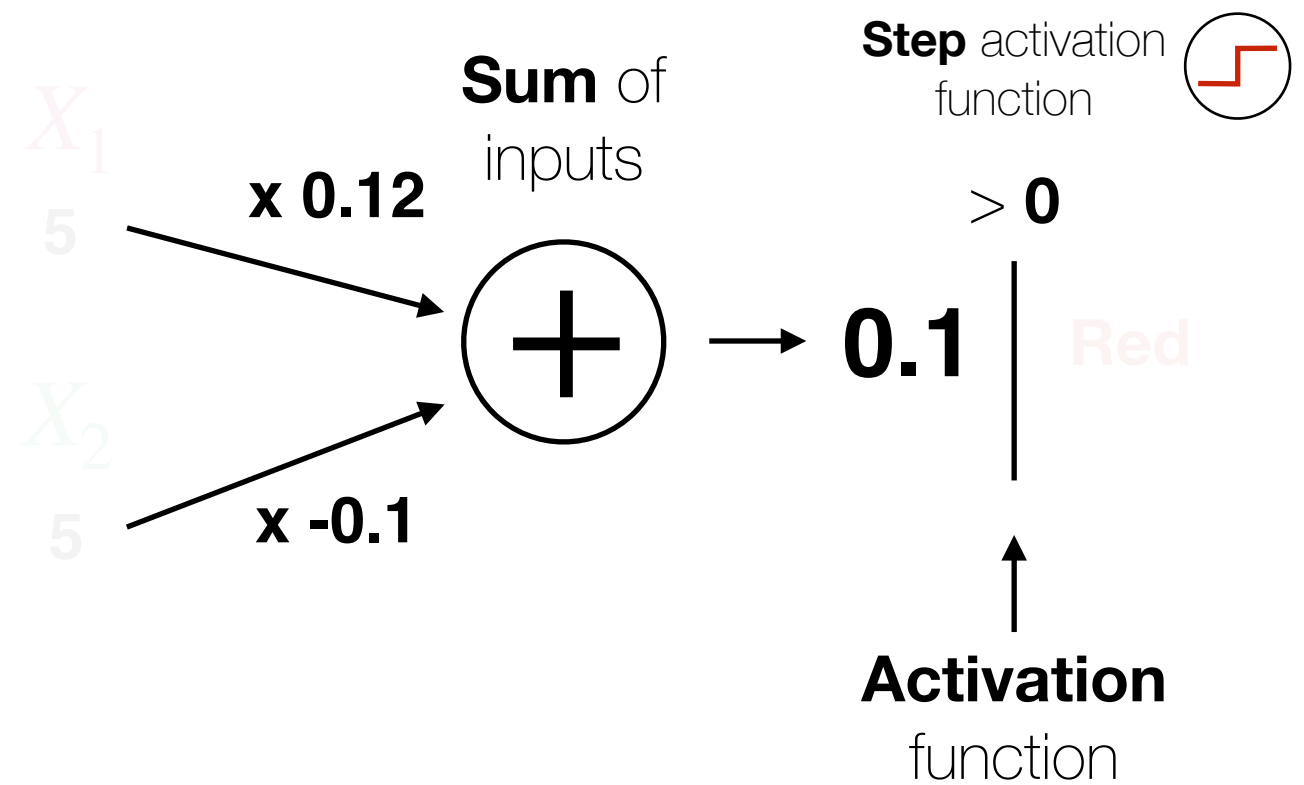




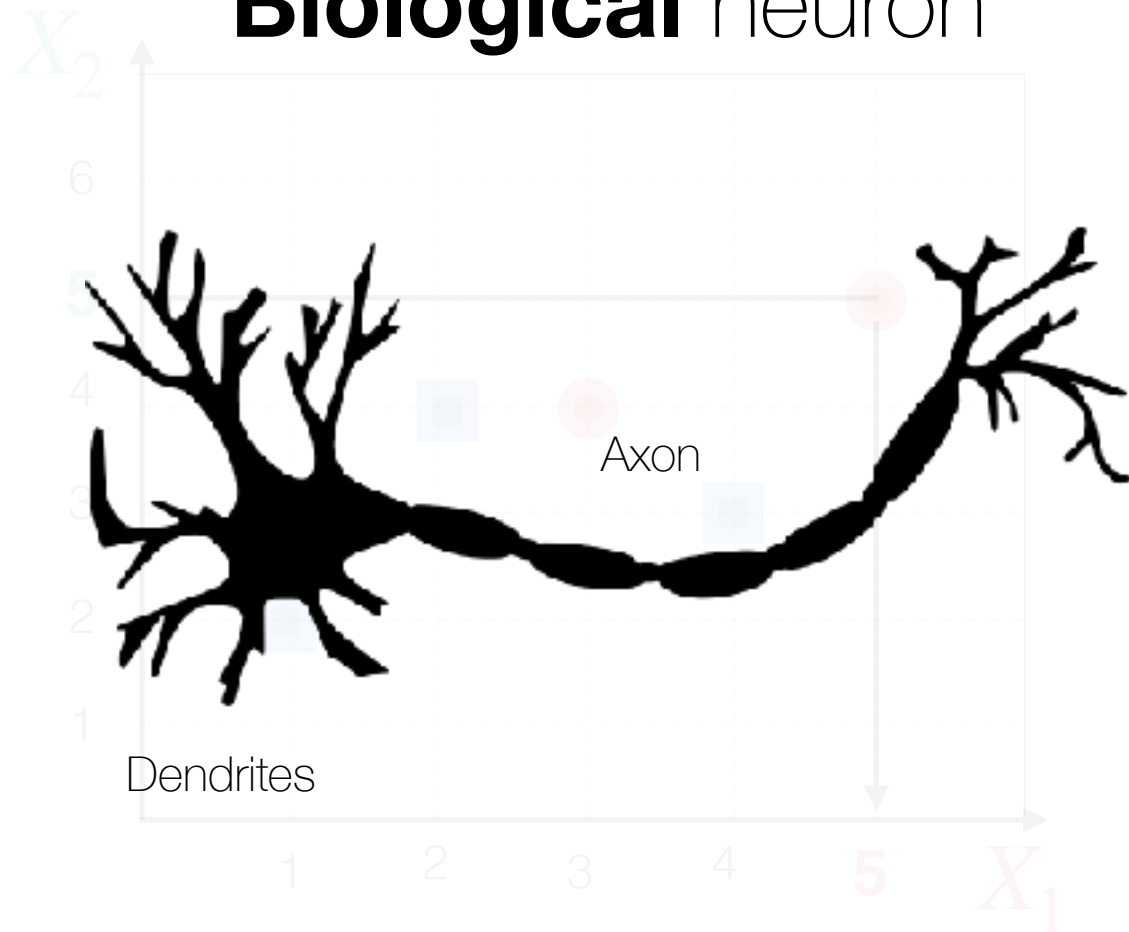




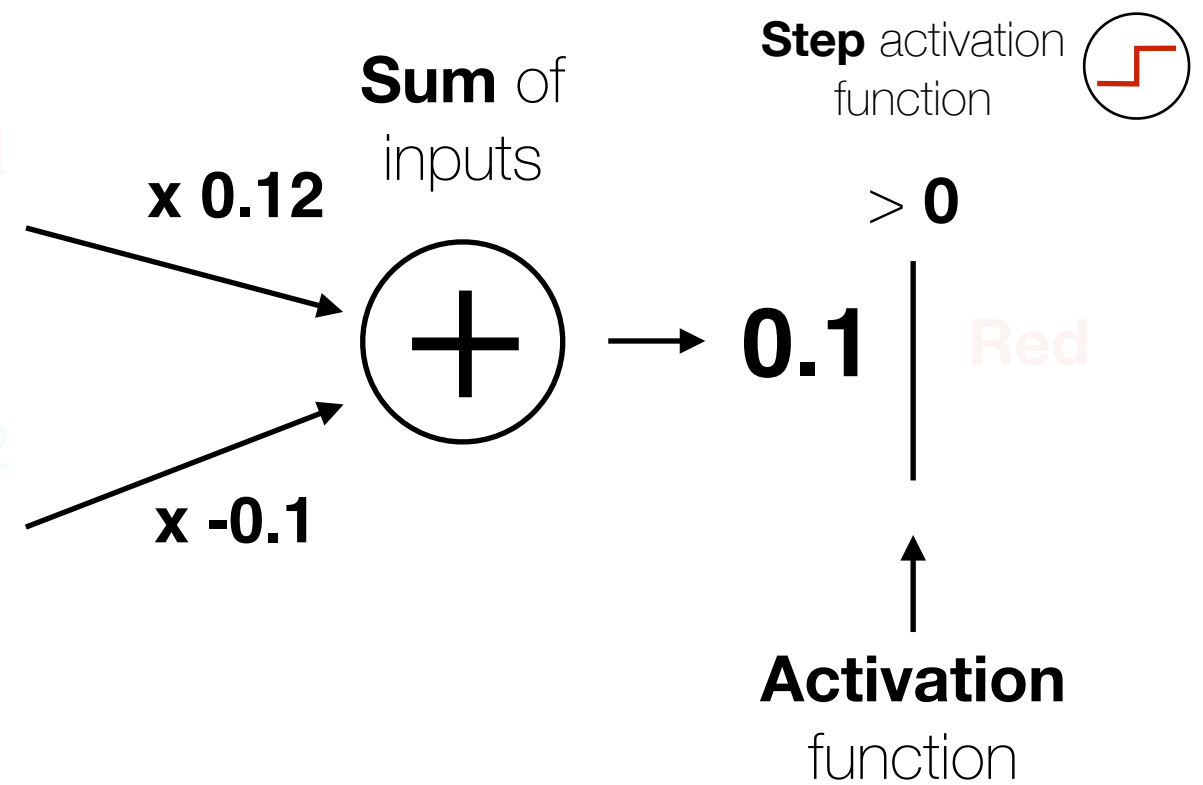
Artificial neuron (perceptron)

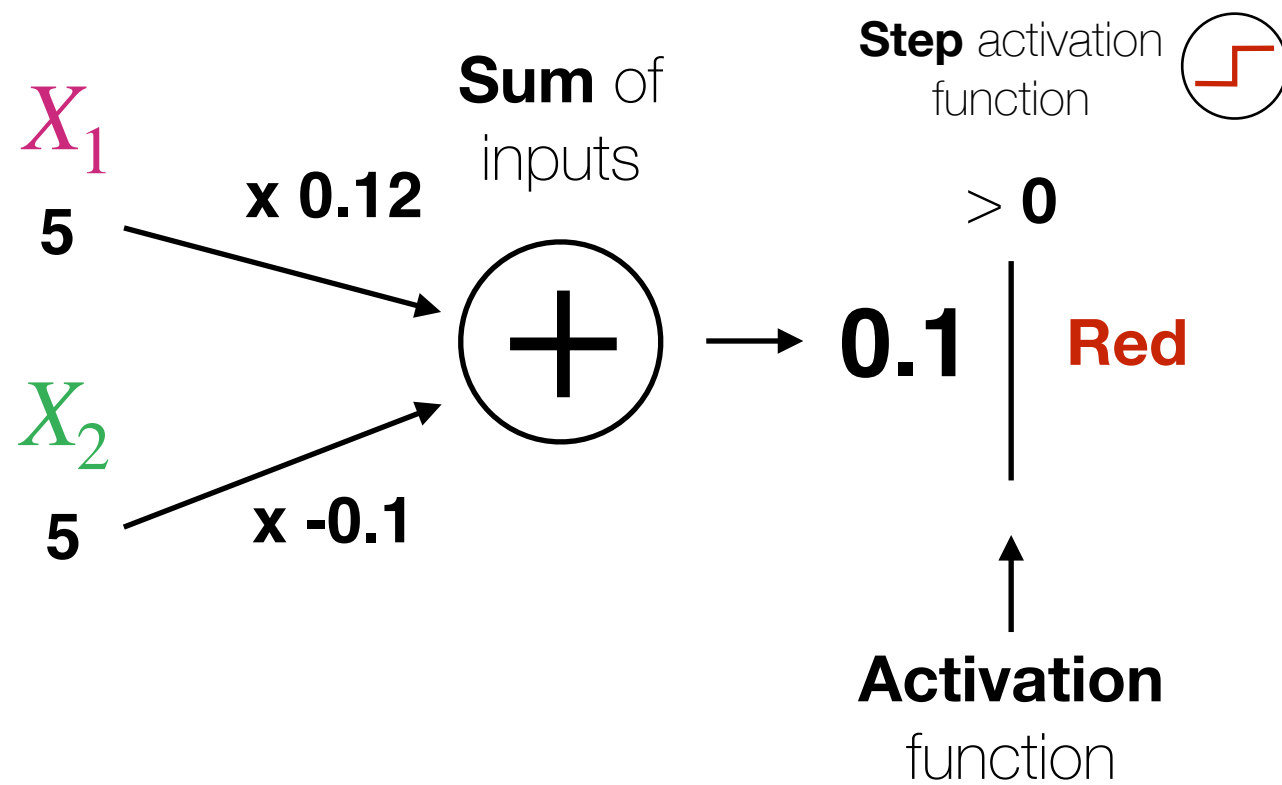
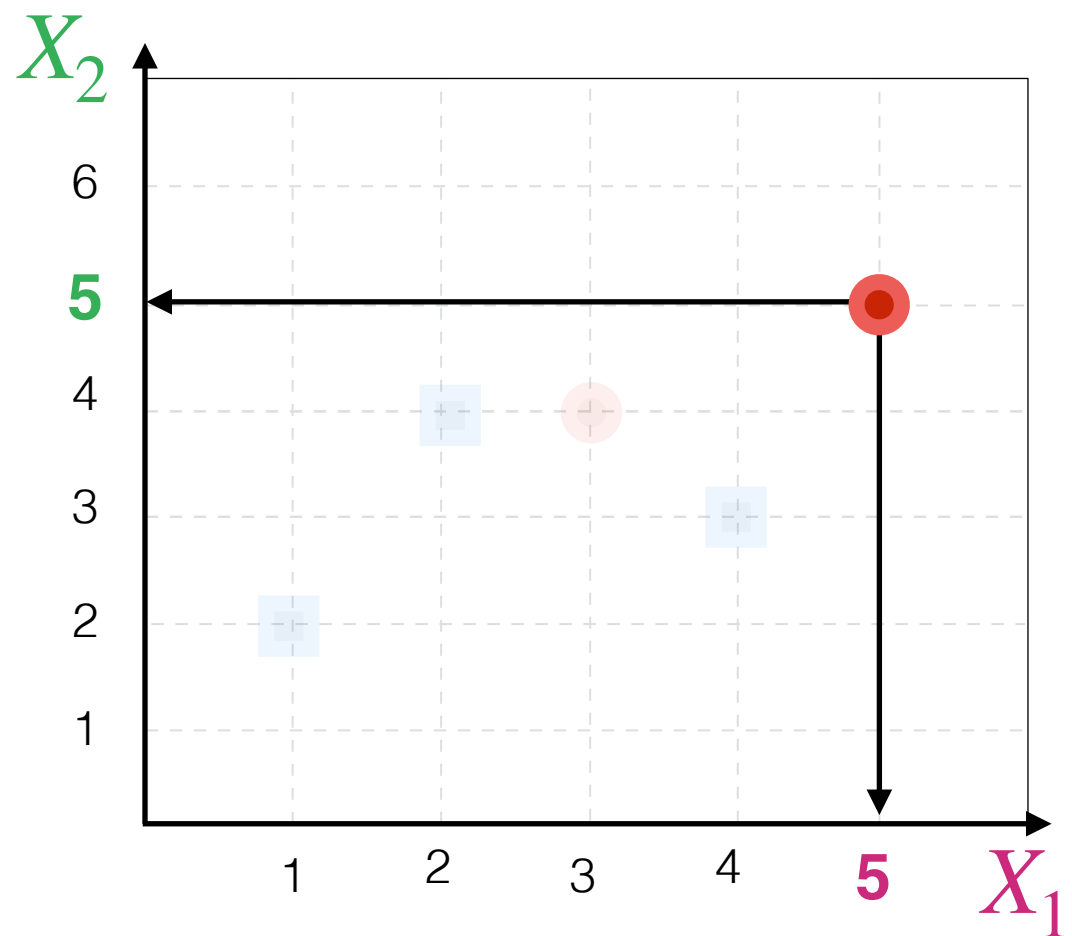


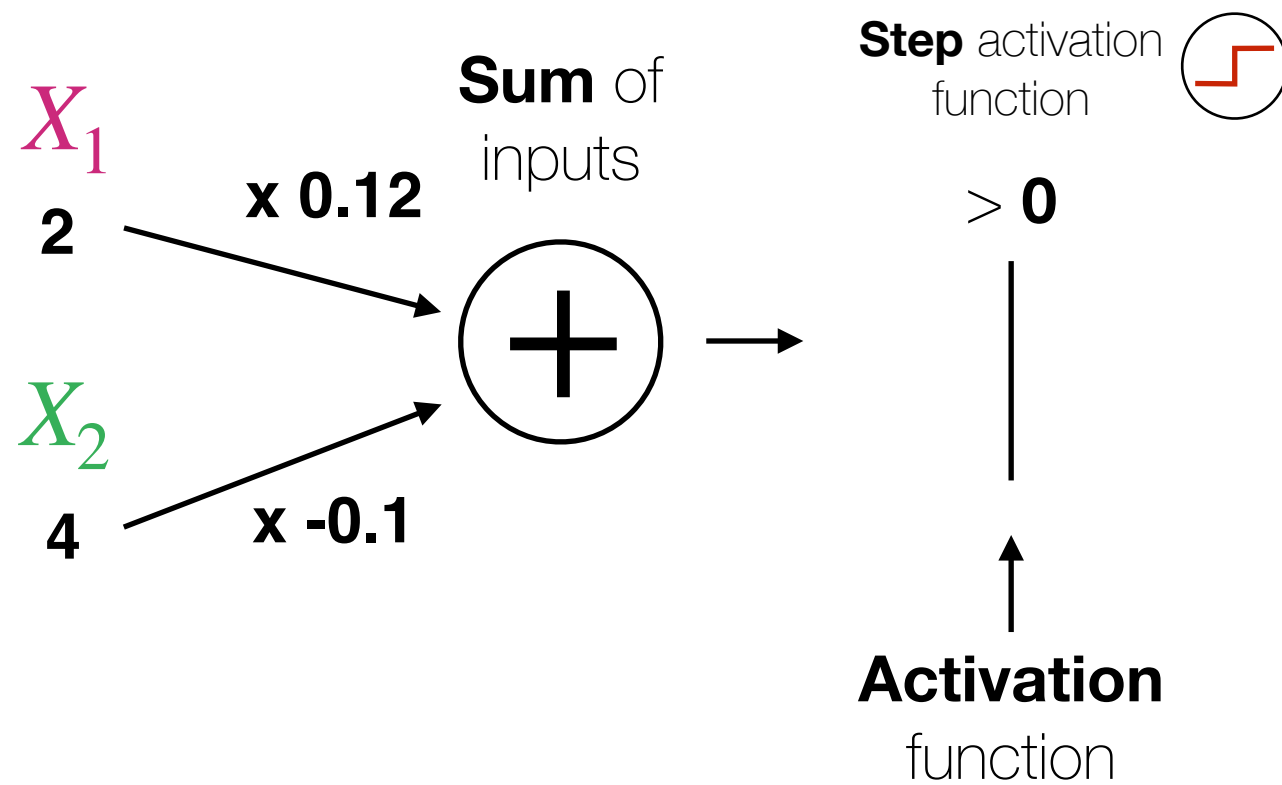
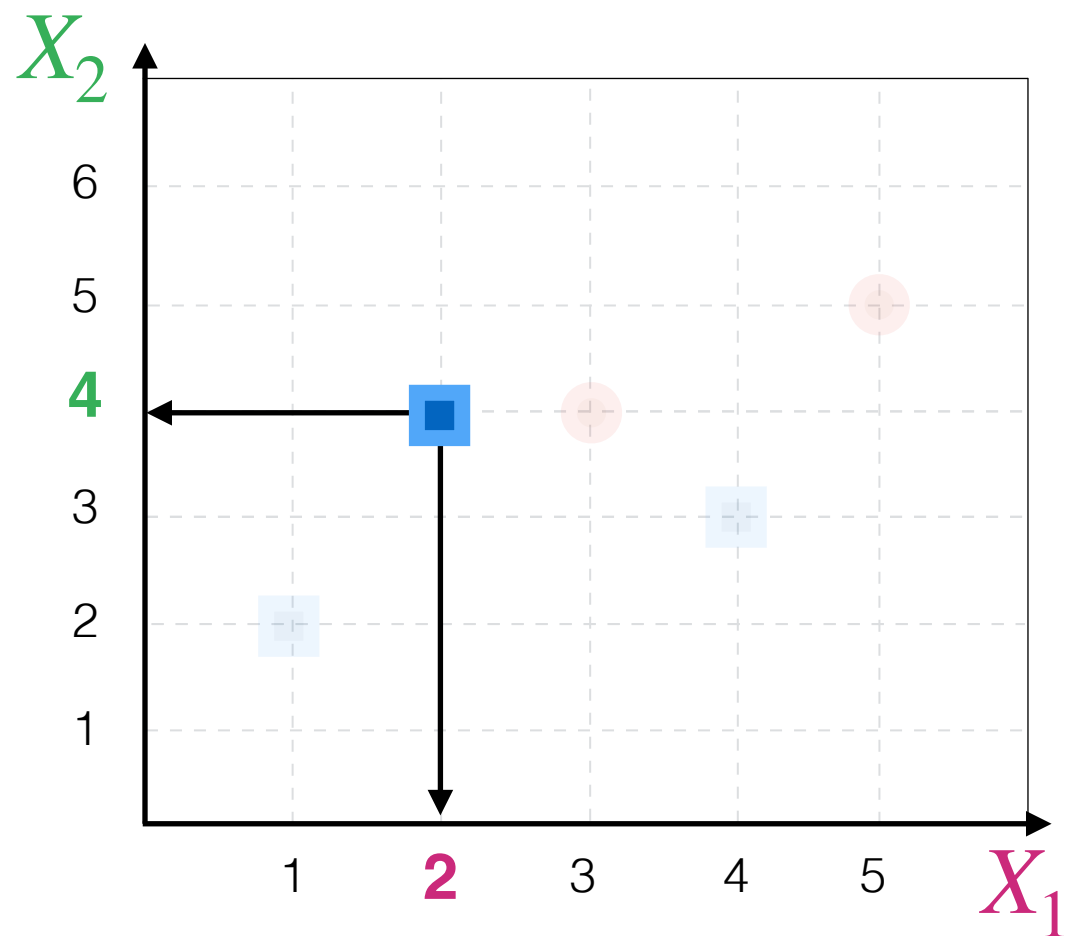
Biological neuron

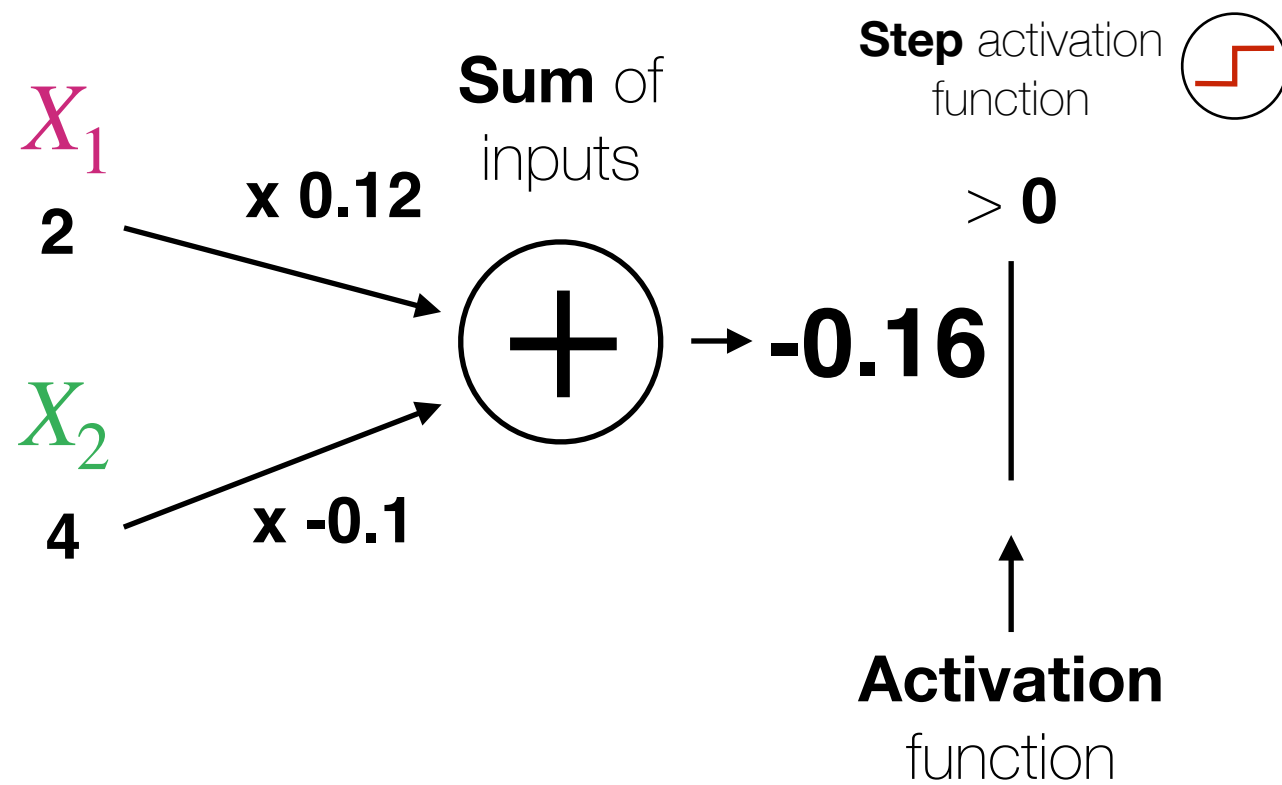
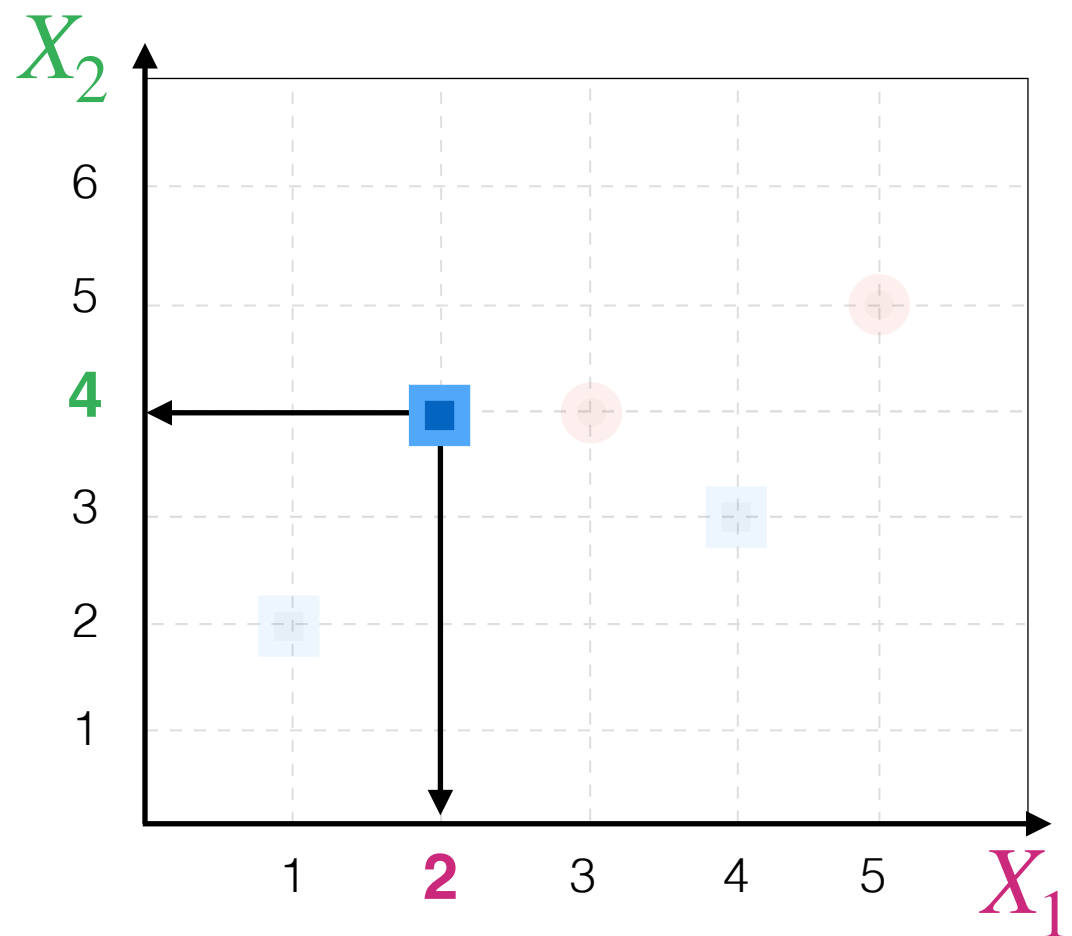


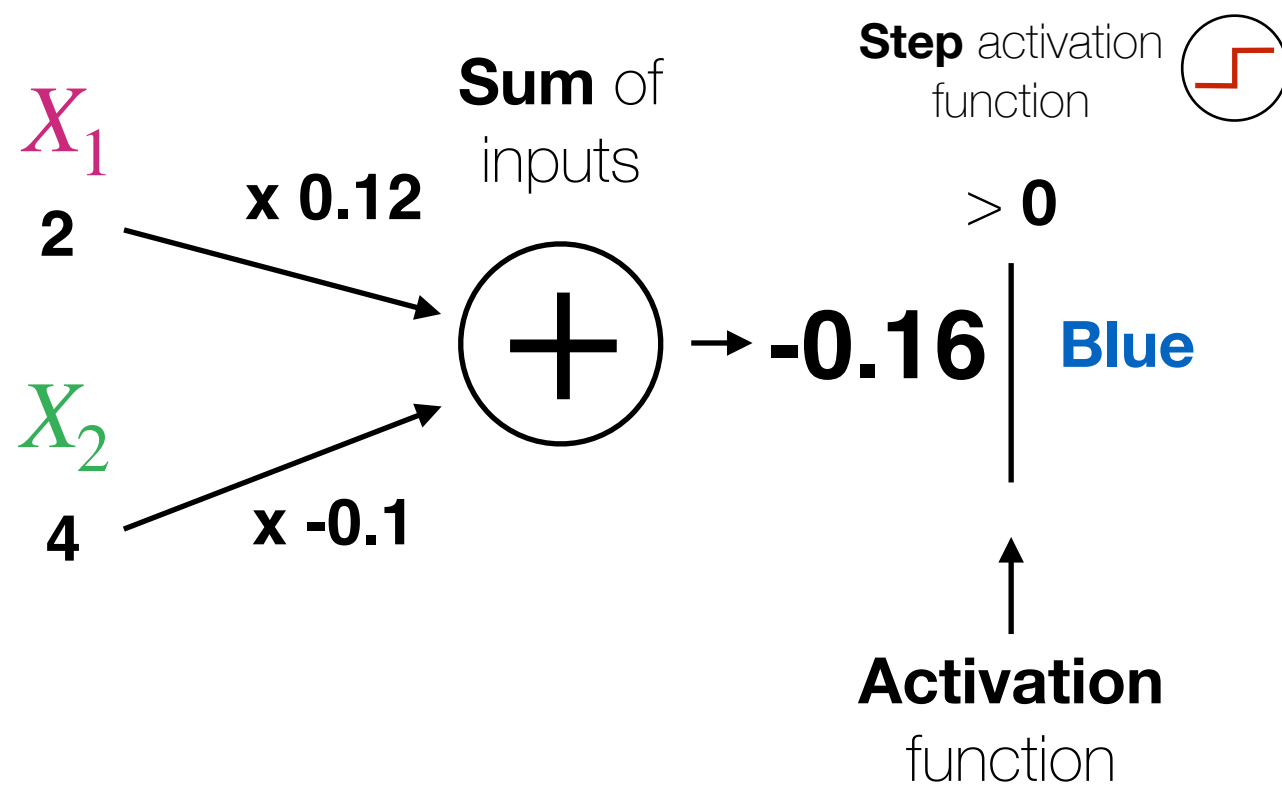
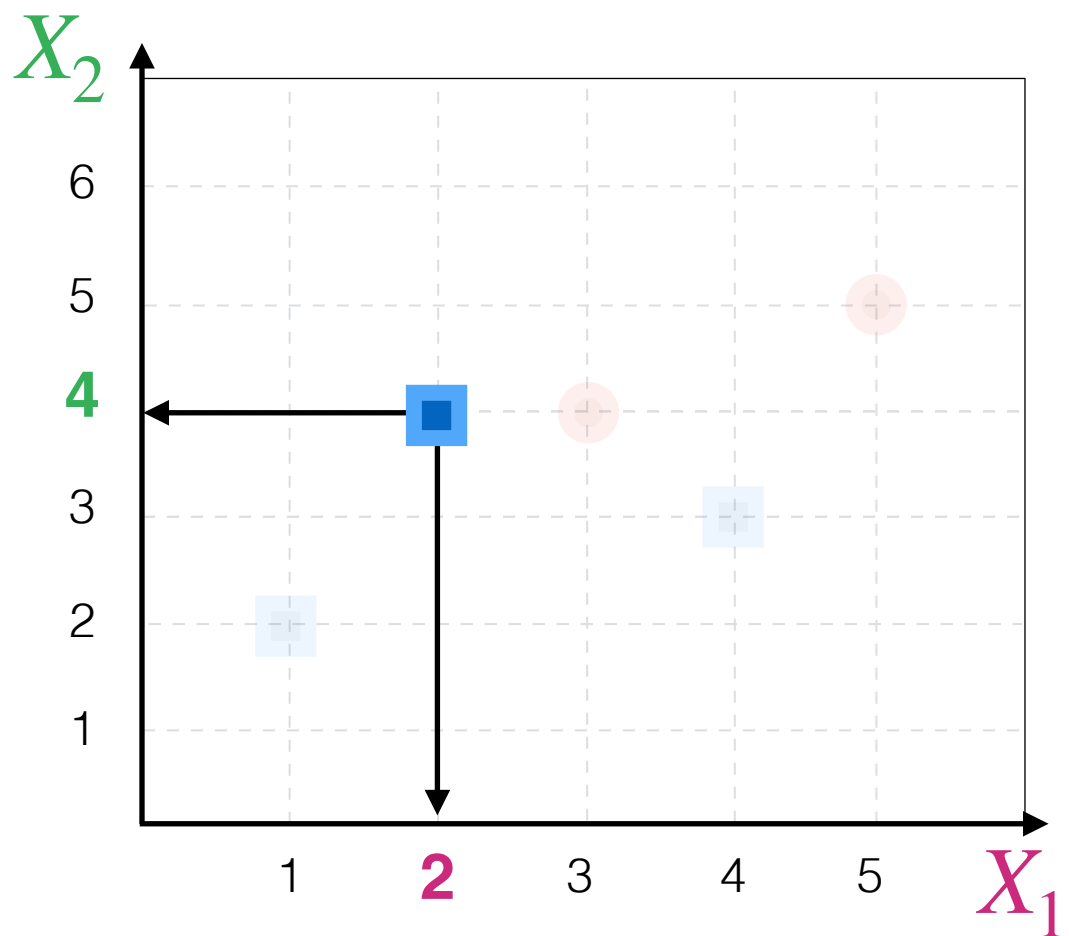
Artificial neuron (perceptron)



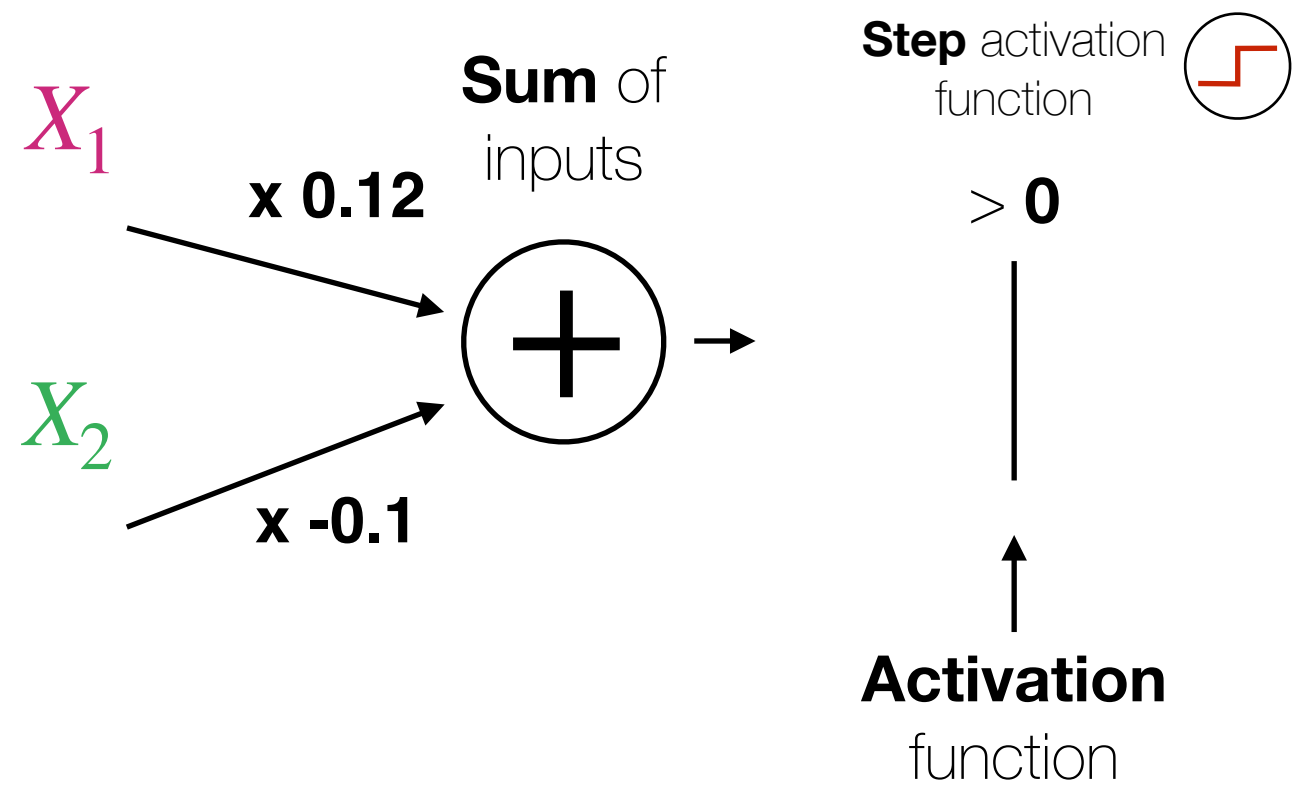
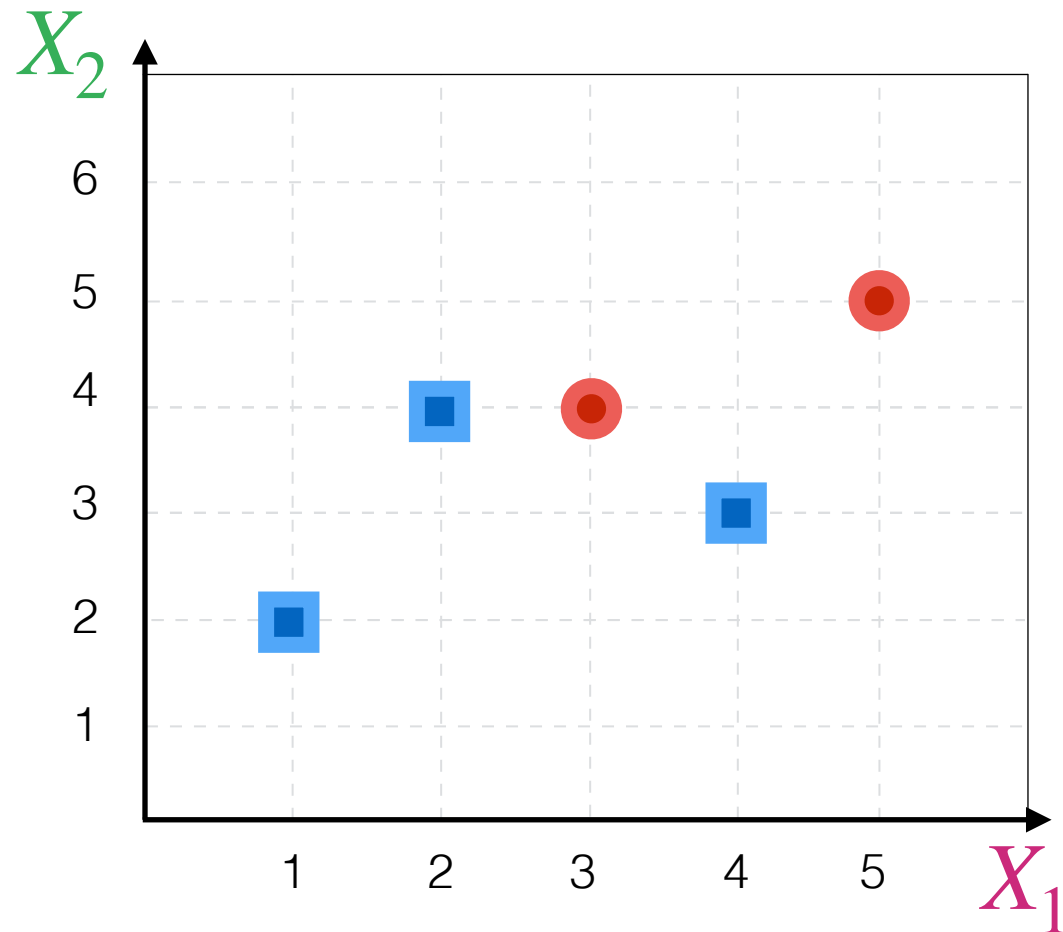




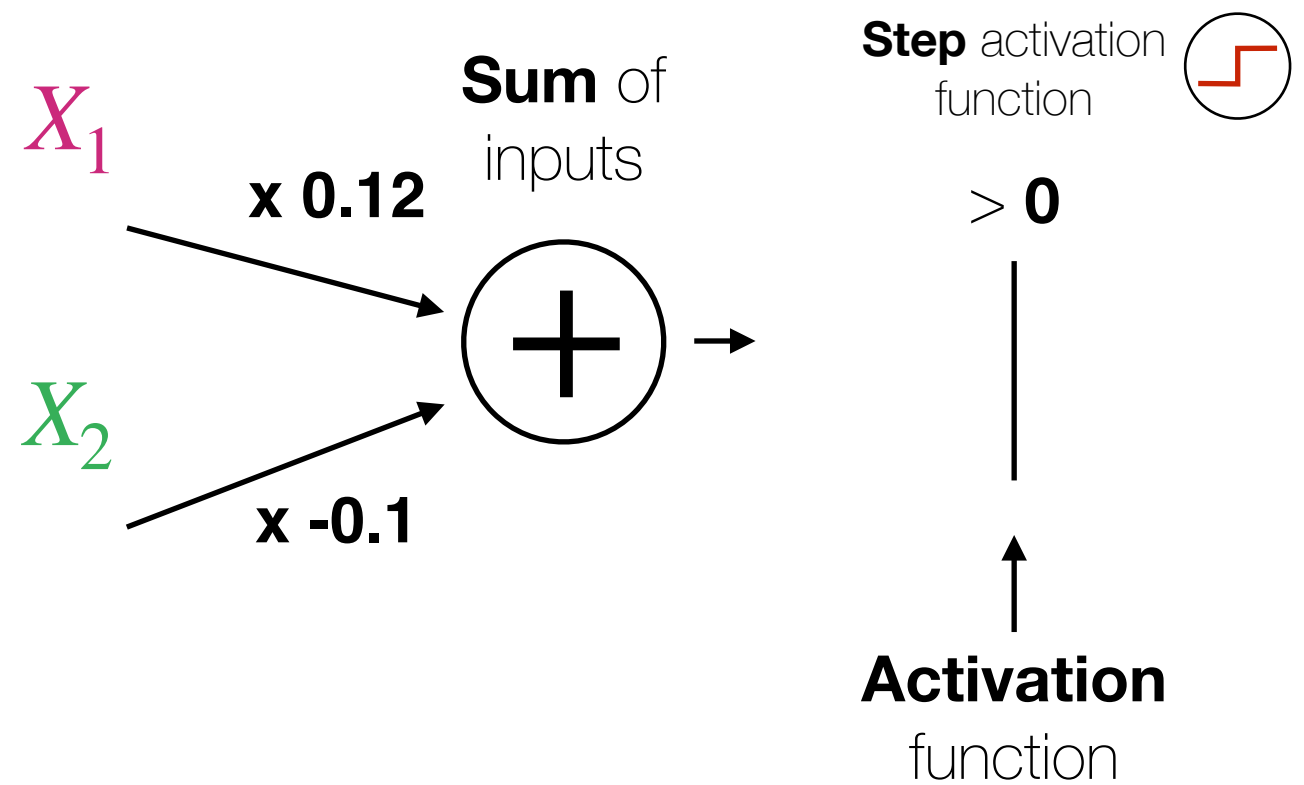
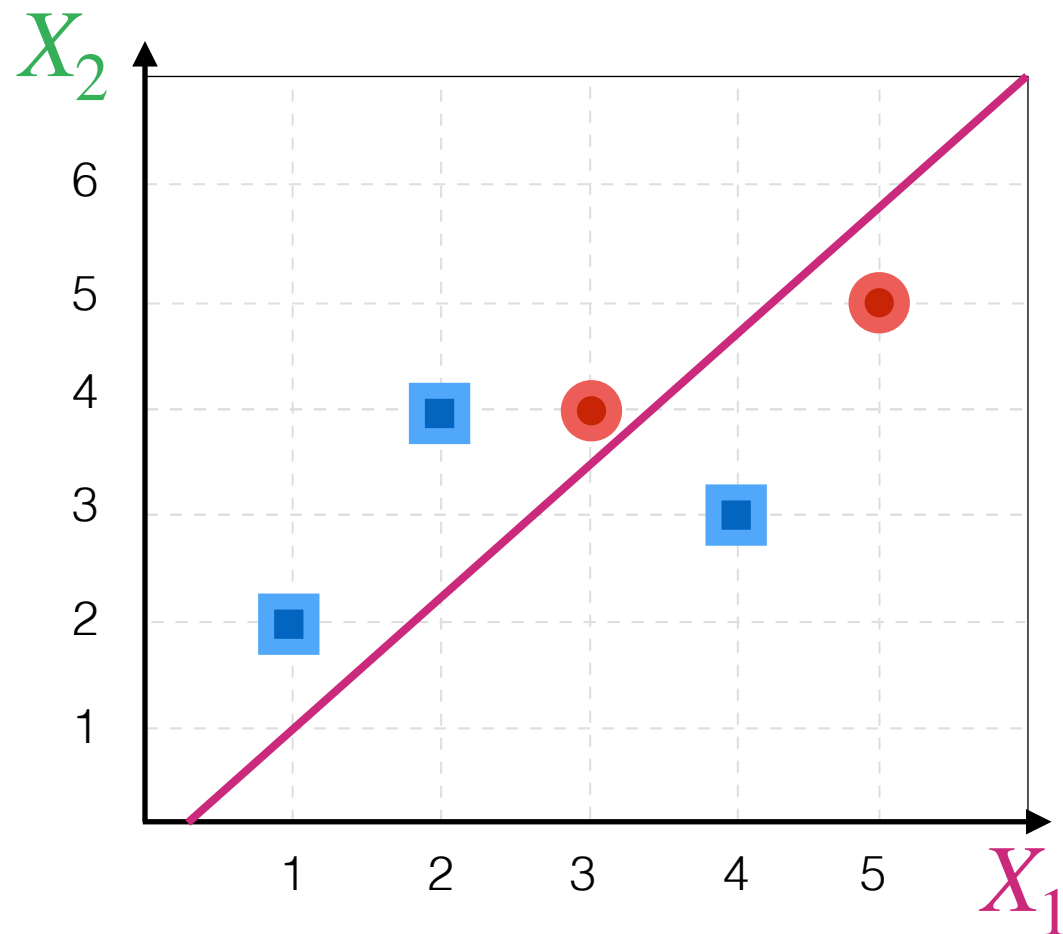




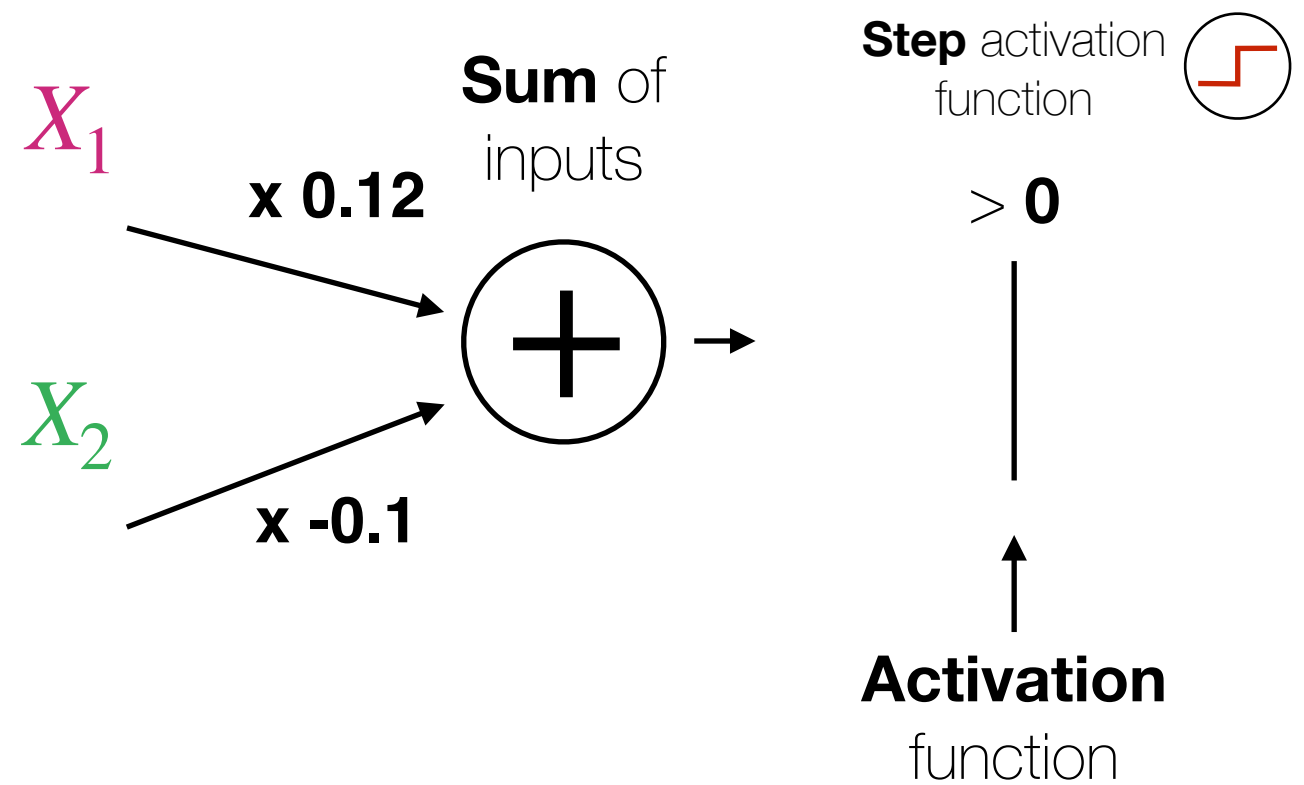
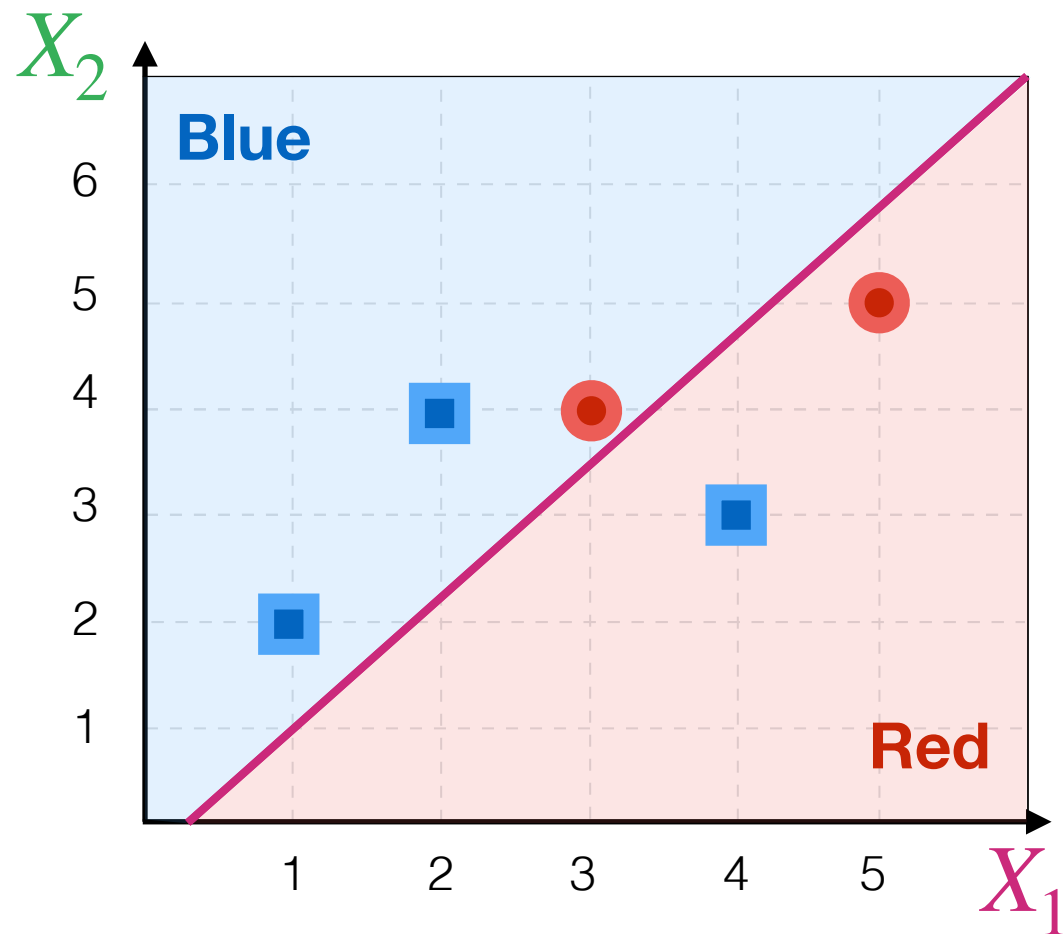
Same procedure should be performed for **all data** points

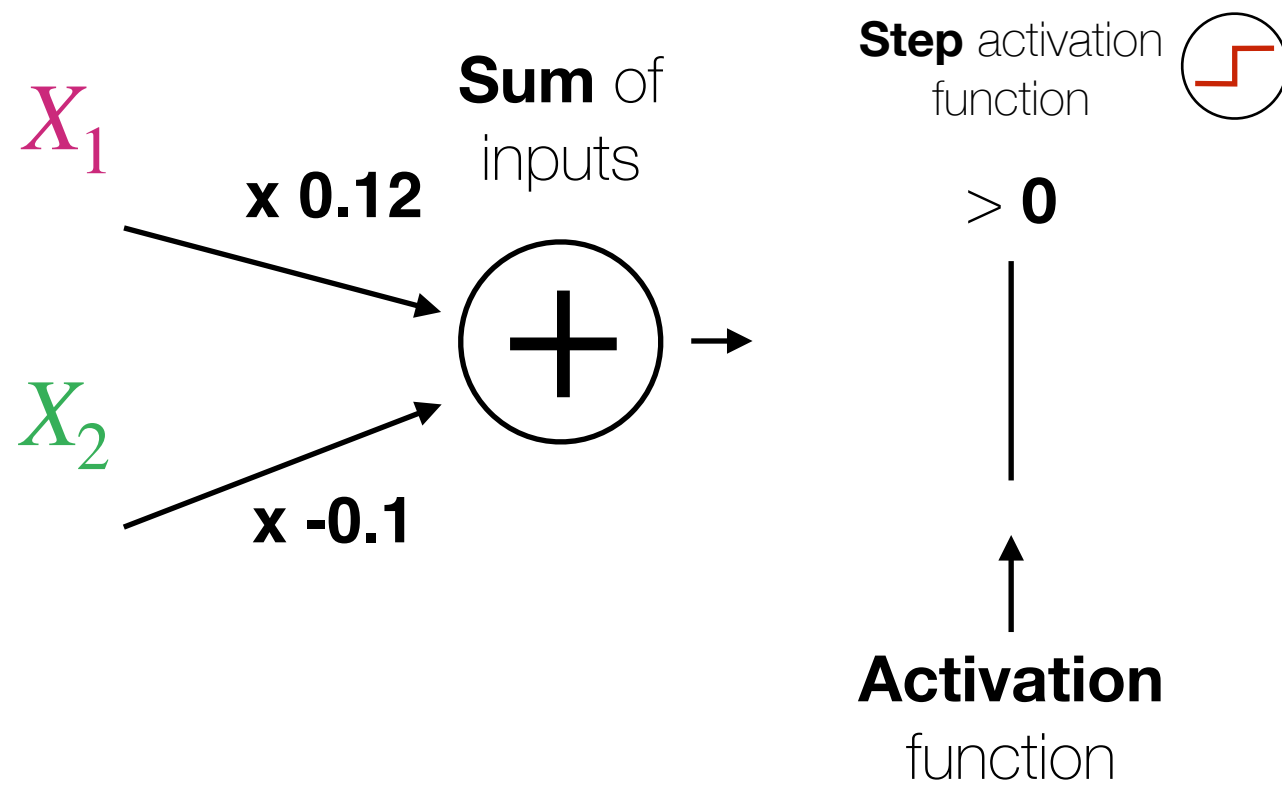
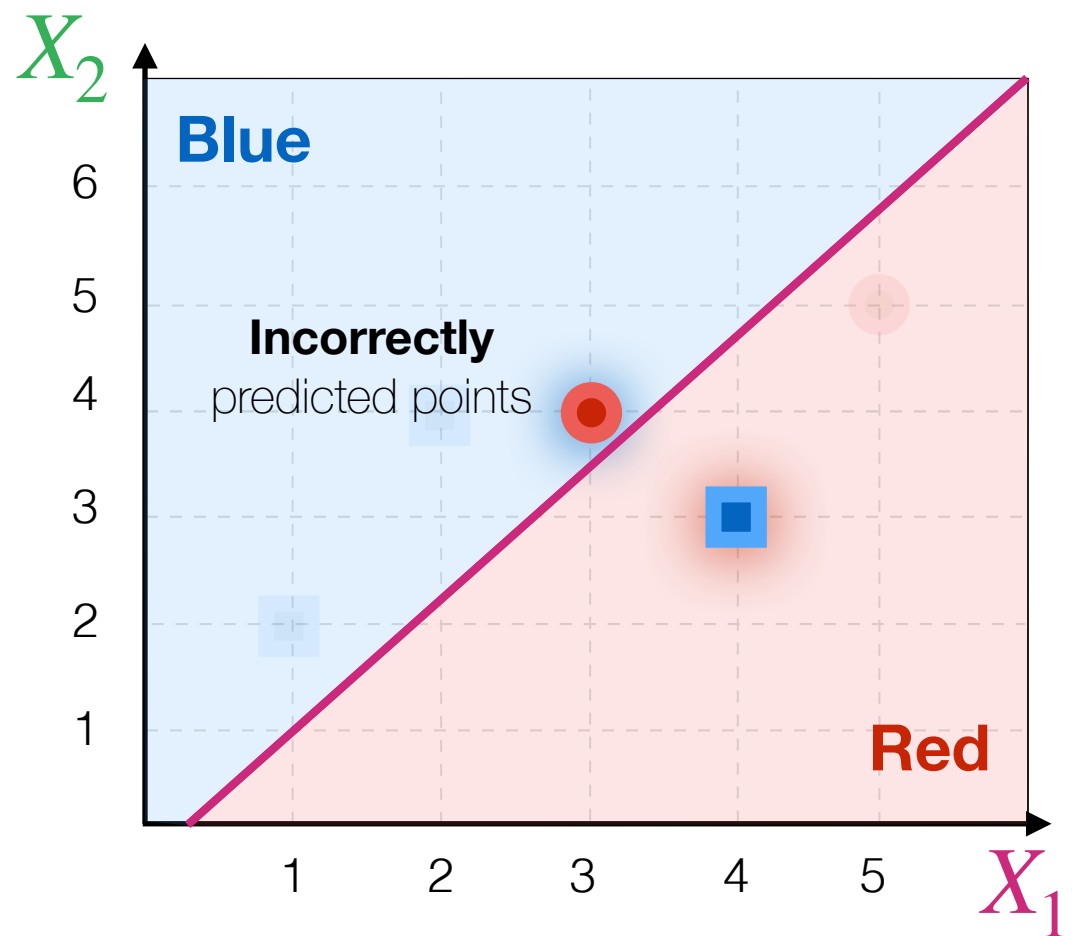


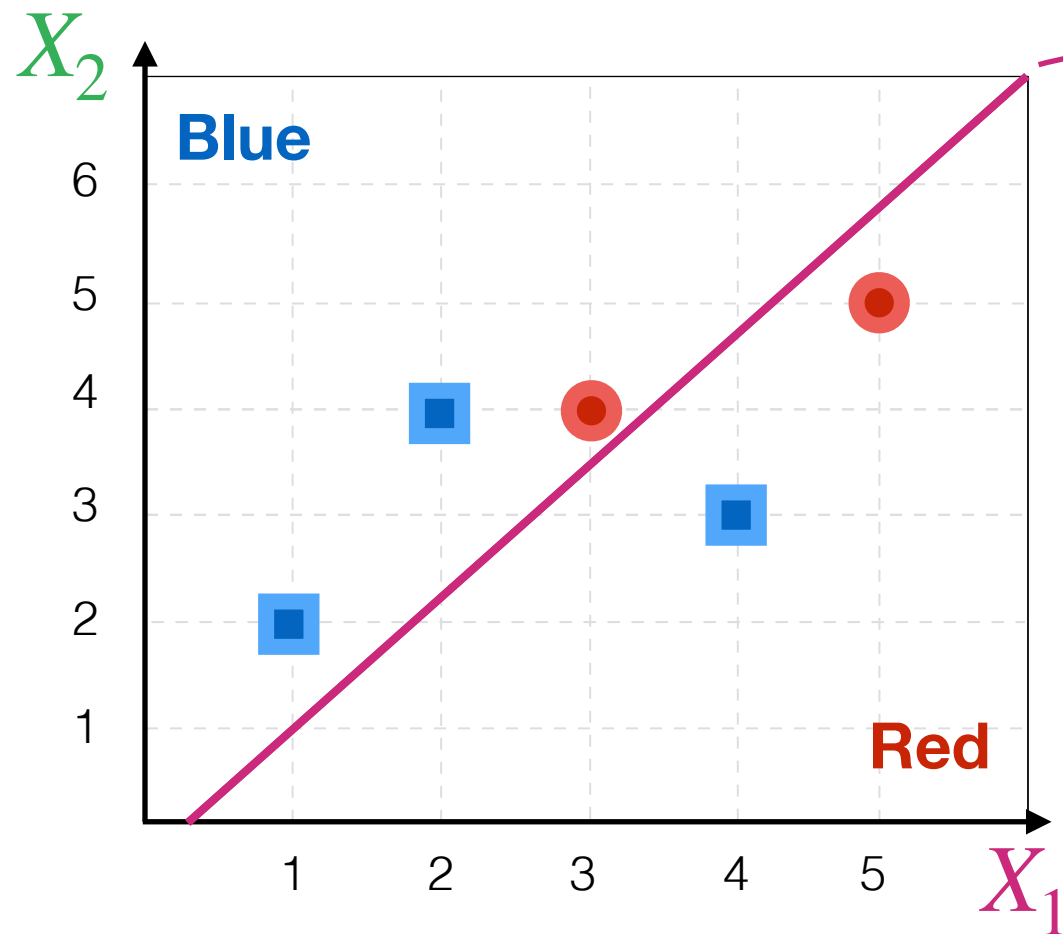
Same procedure should be performed for **all data** points



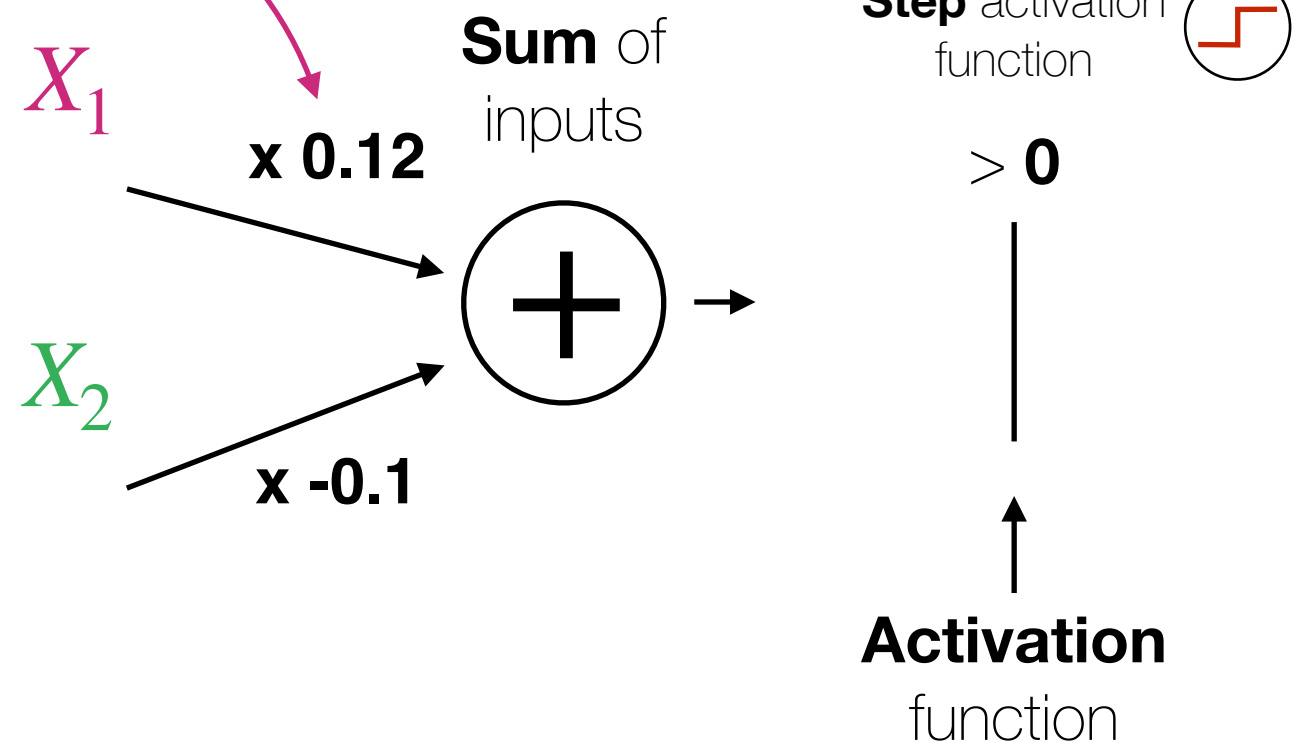
Same procedure should be performed for **all data** points



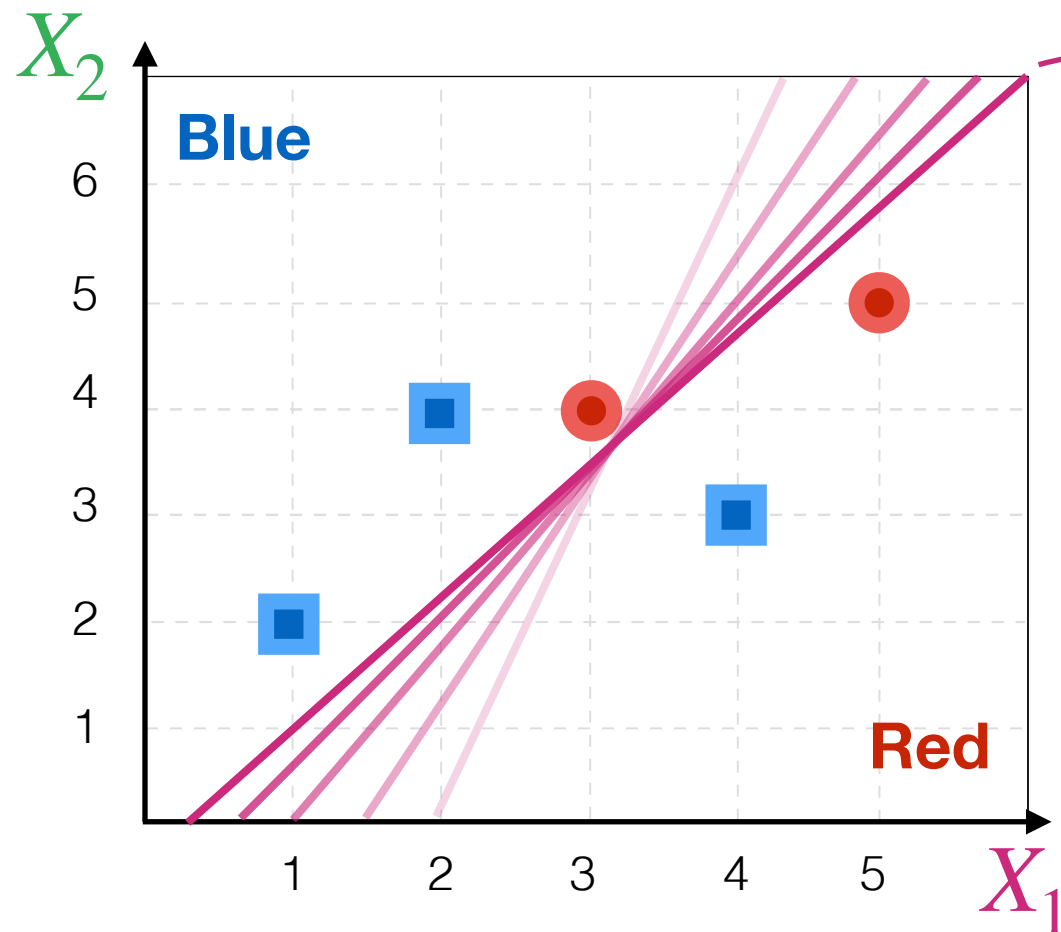




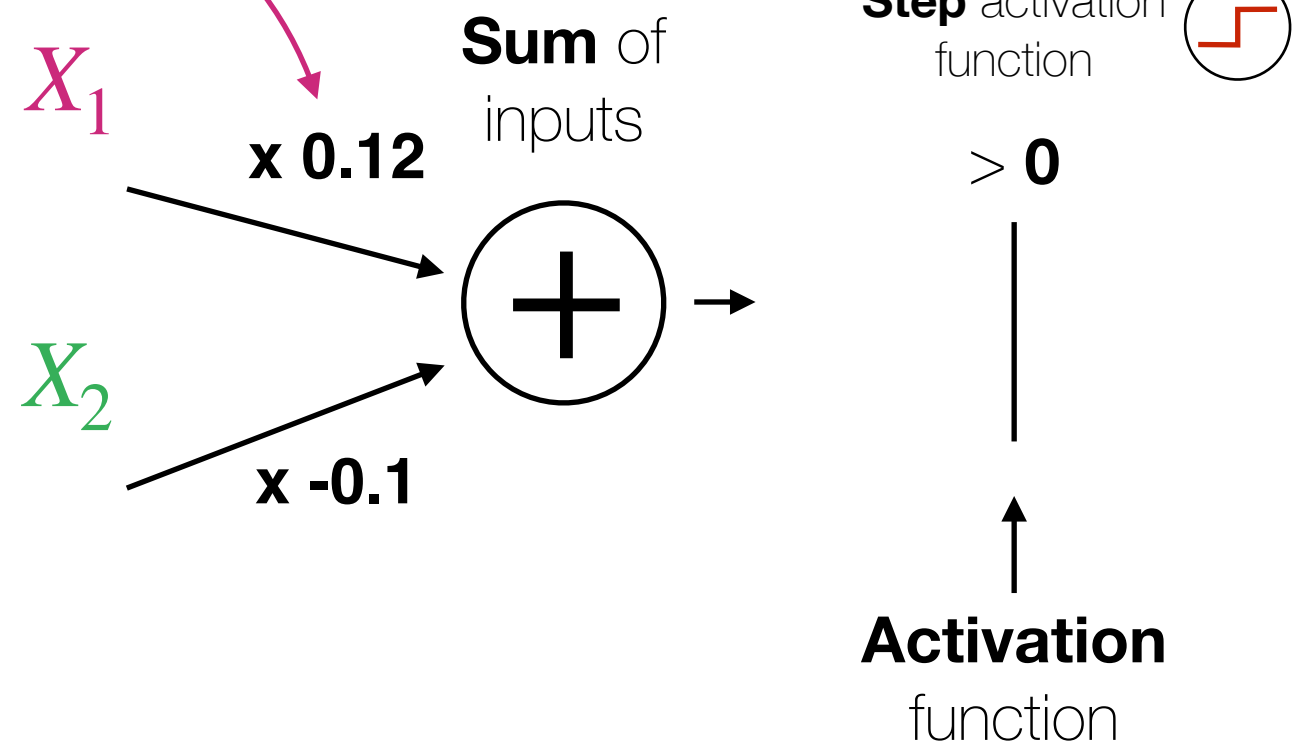
Decision boundary
separating classes is
controlled by weights



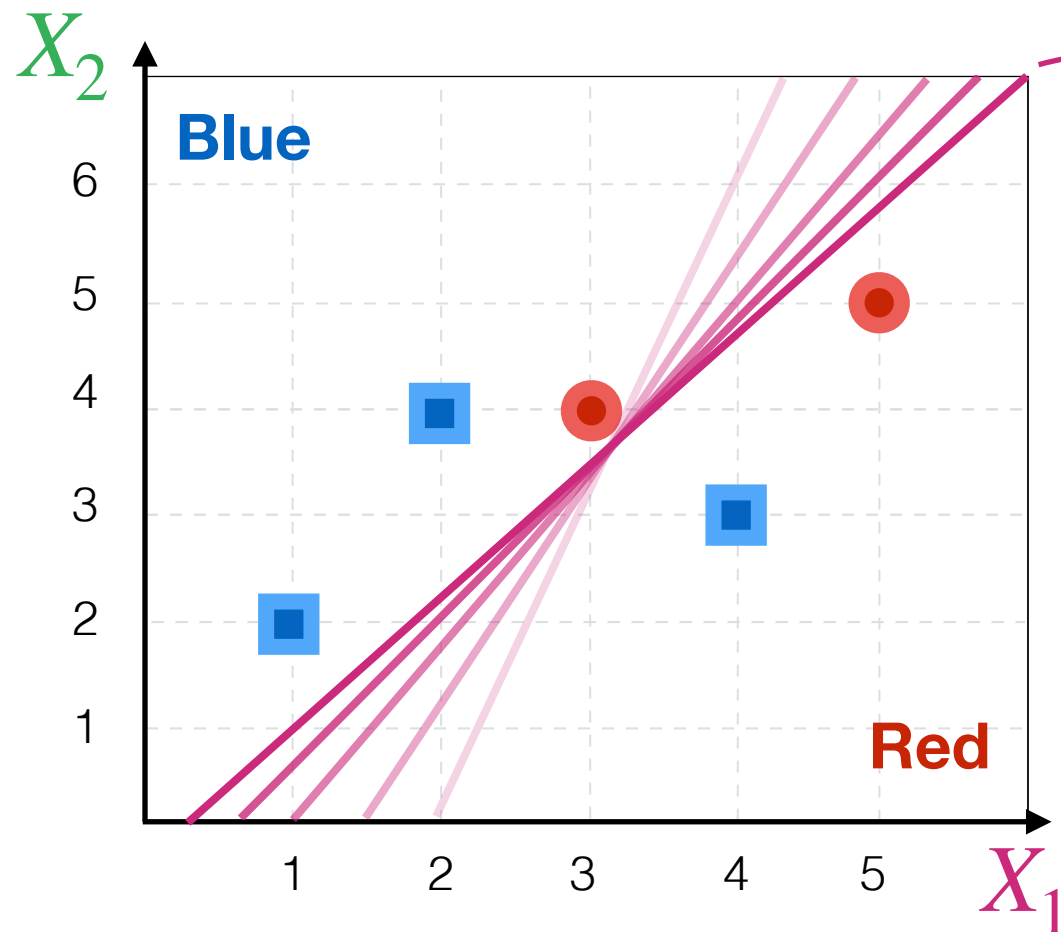
By changing weights we can **increase** or **decrease** the influence of input features, hence **rotate** the **decision boundary**



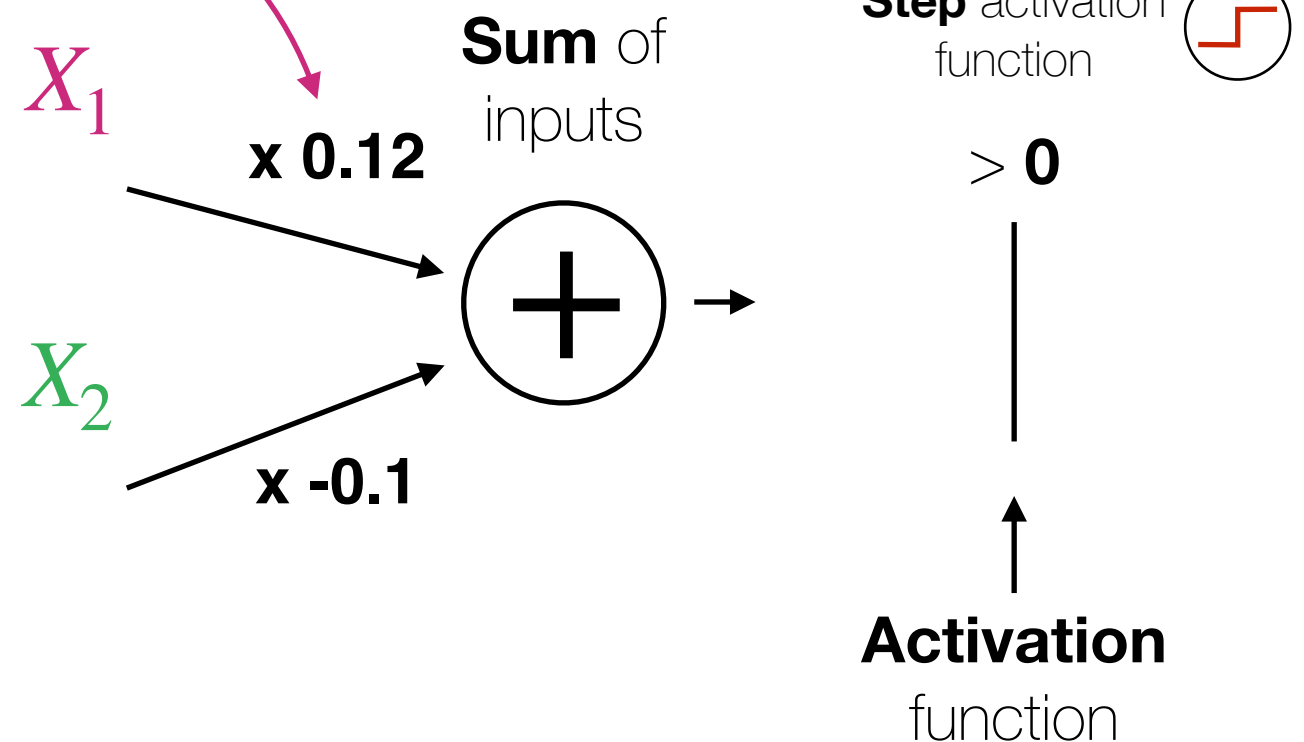
Decision boundary separating classes is controlled by weights



By changing weights we can **increase** or **decrease** the influence of input features, hence **rotate** the **decision boundary**

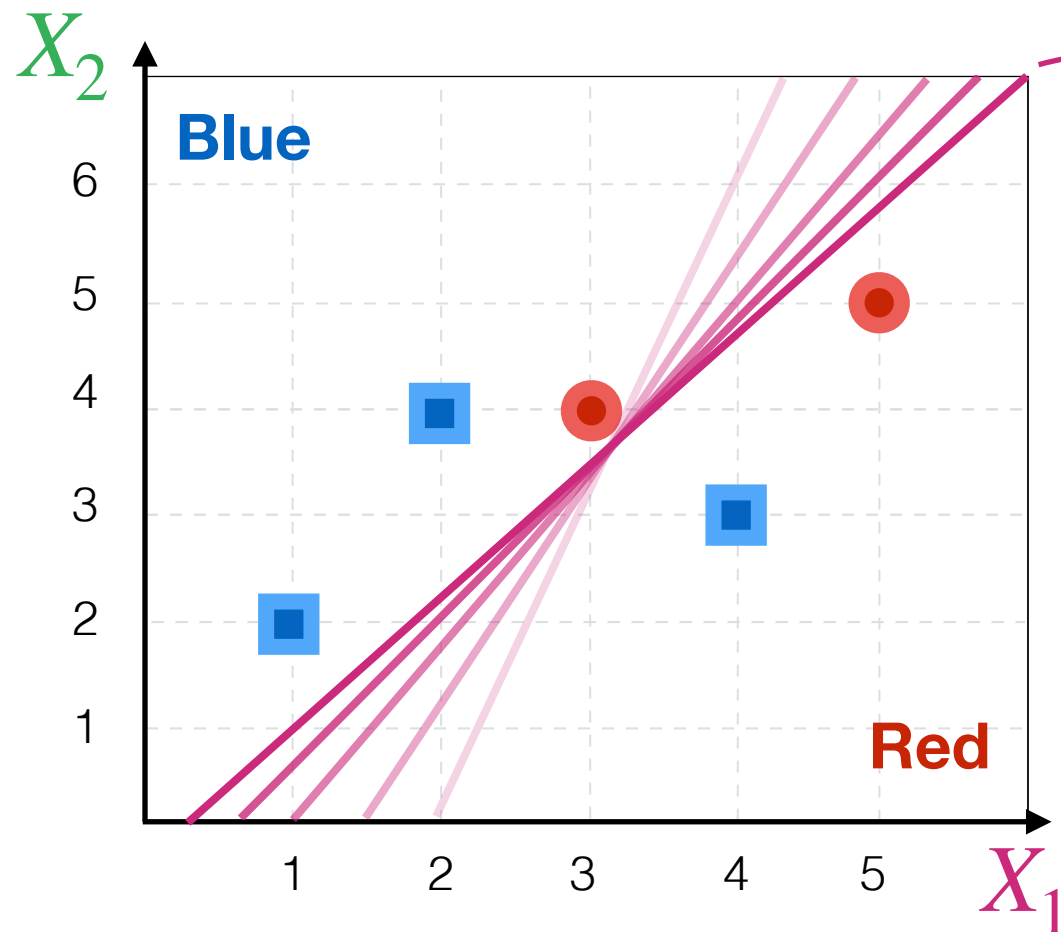


Decision boundary separating classes is controlled by weights

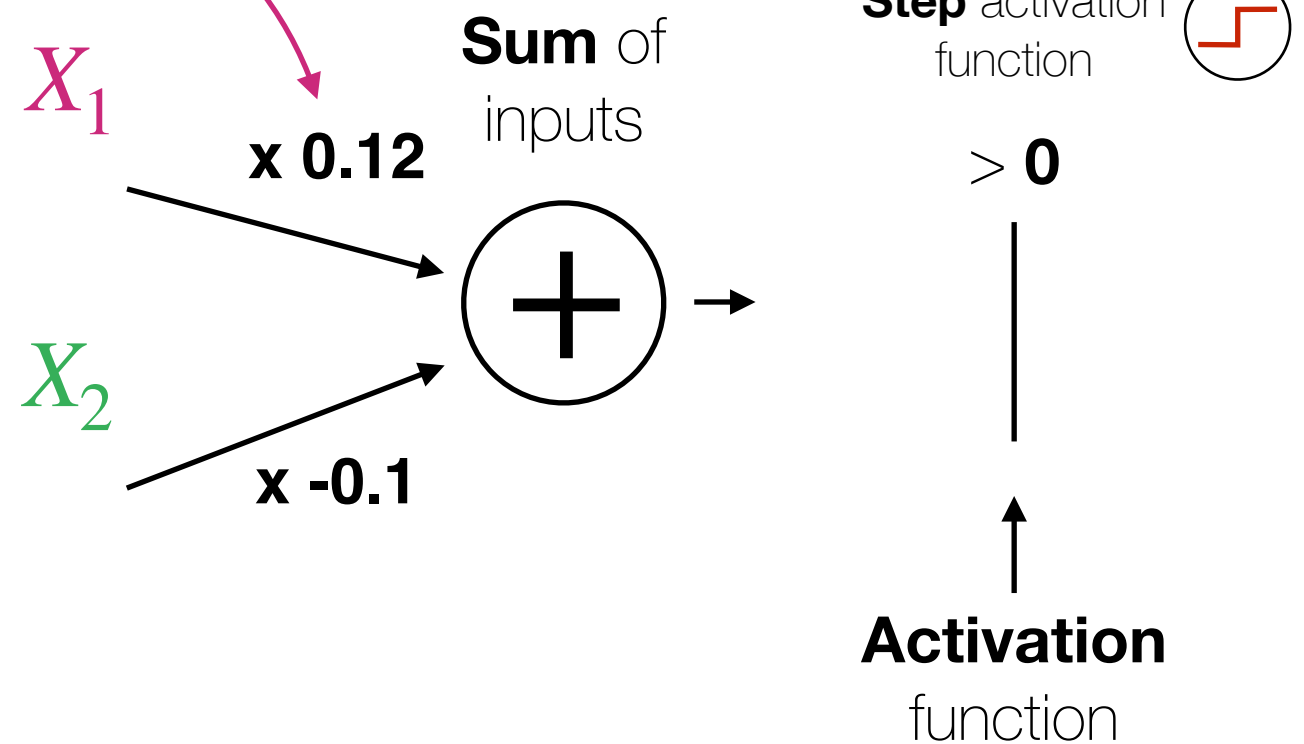


Good weights are found using **backpropagation** algorithm, which we will examine further

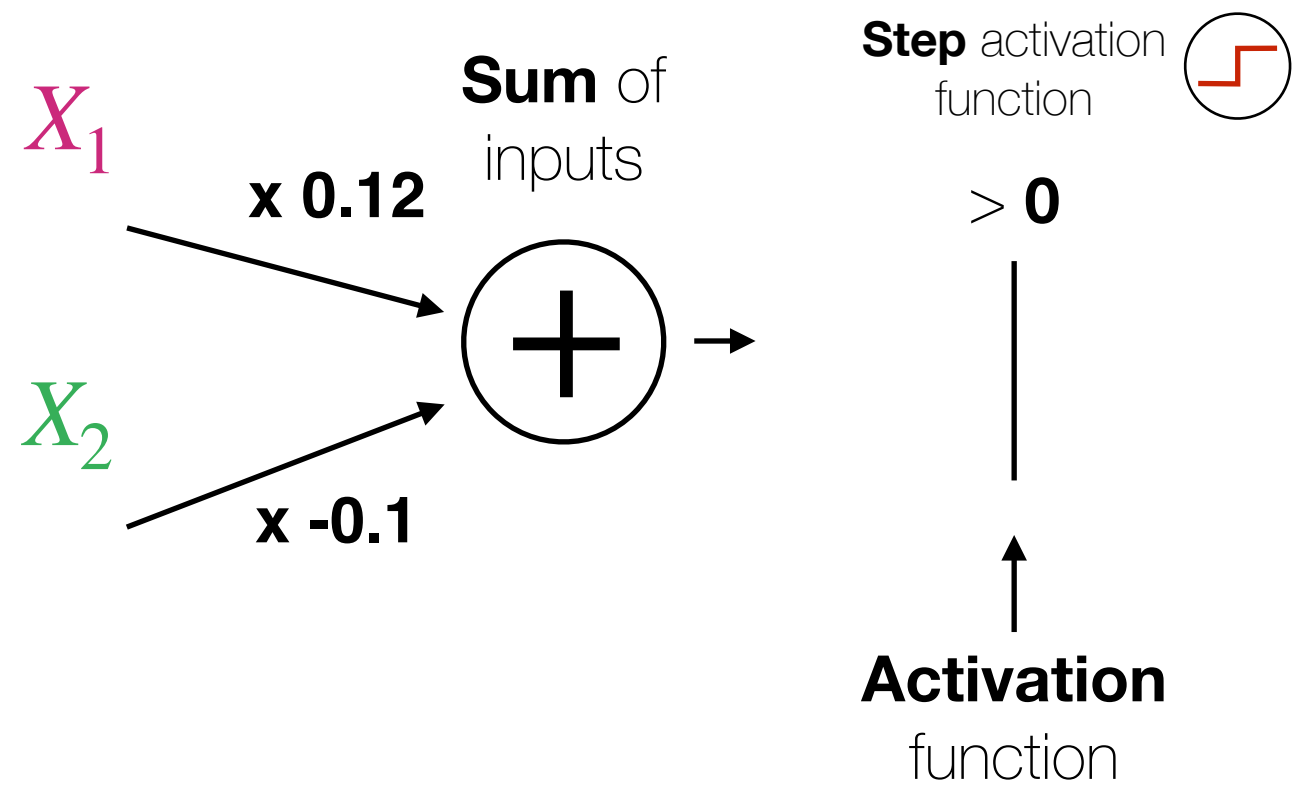
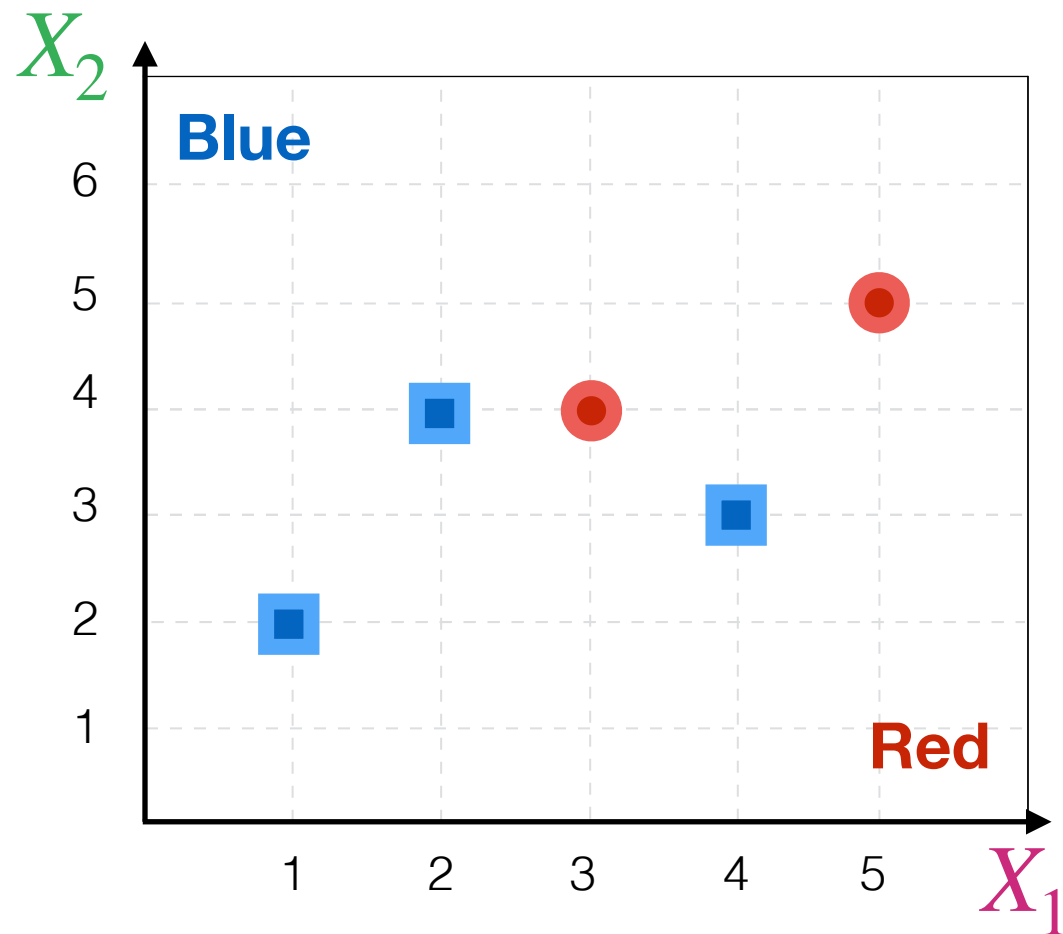
In this case **linear decision boundary** won't help.



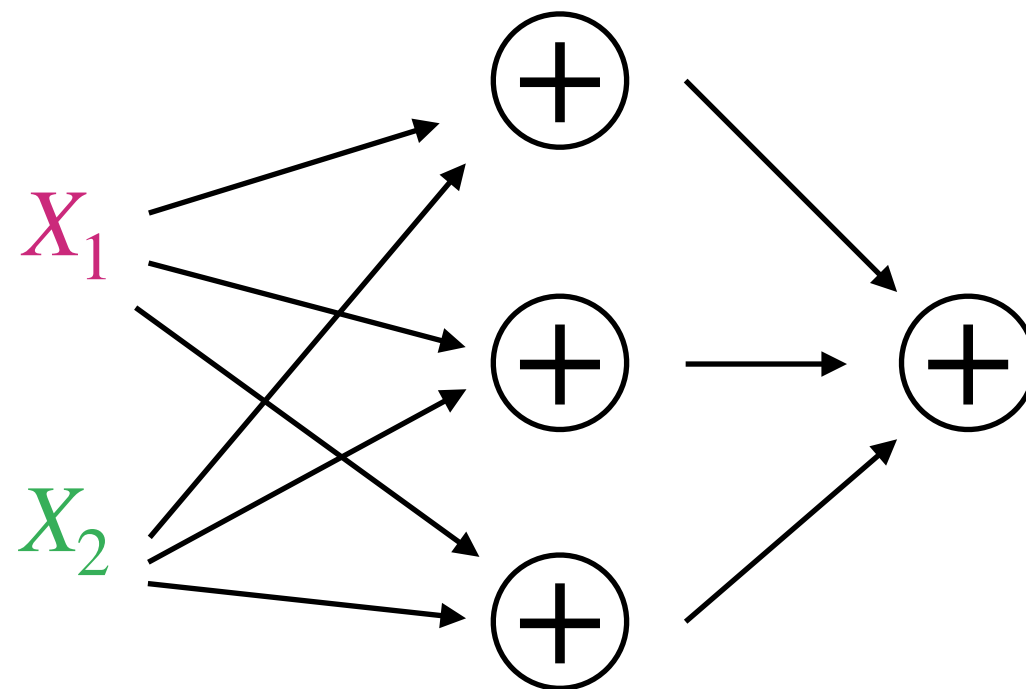
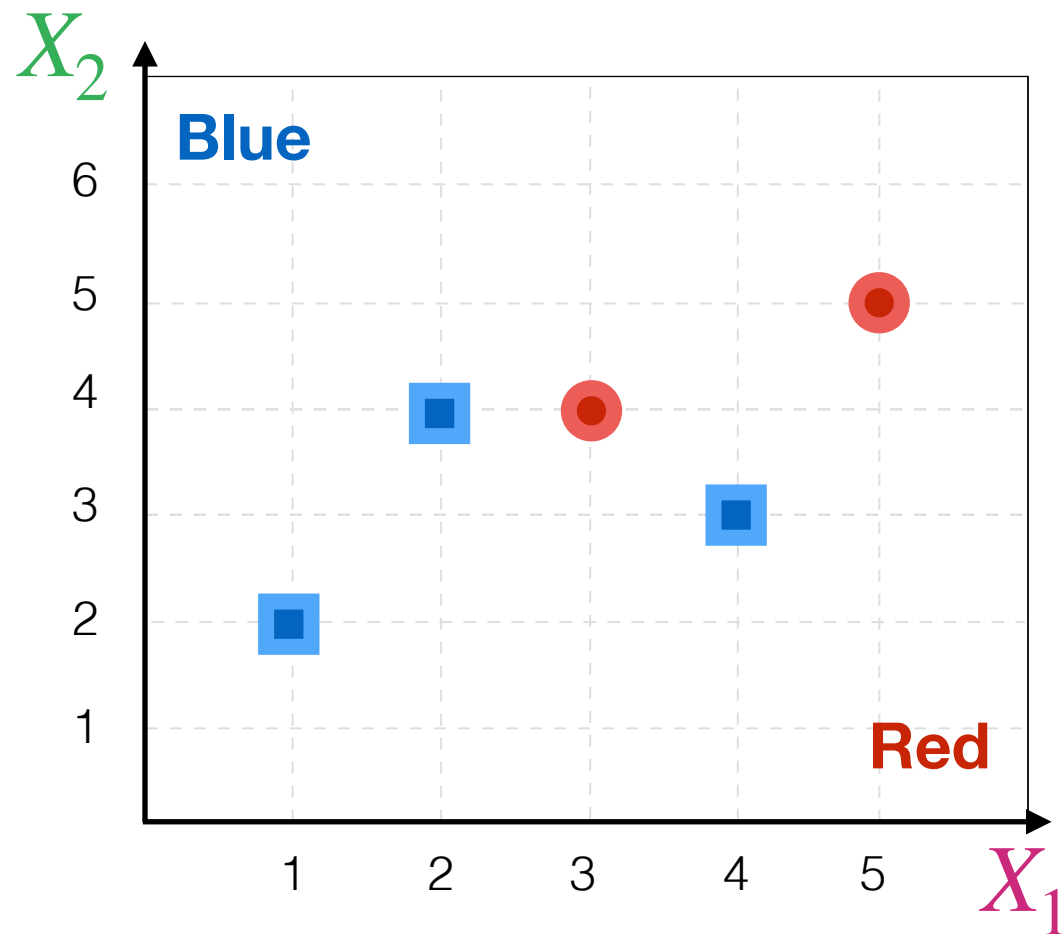
Decision boundary separating classes is controlled by weights



In this case **linear decision boundary** won't help.

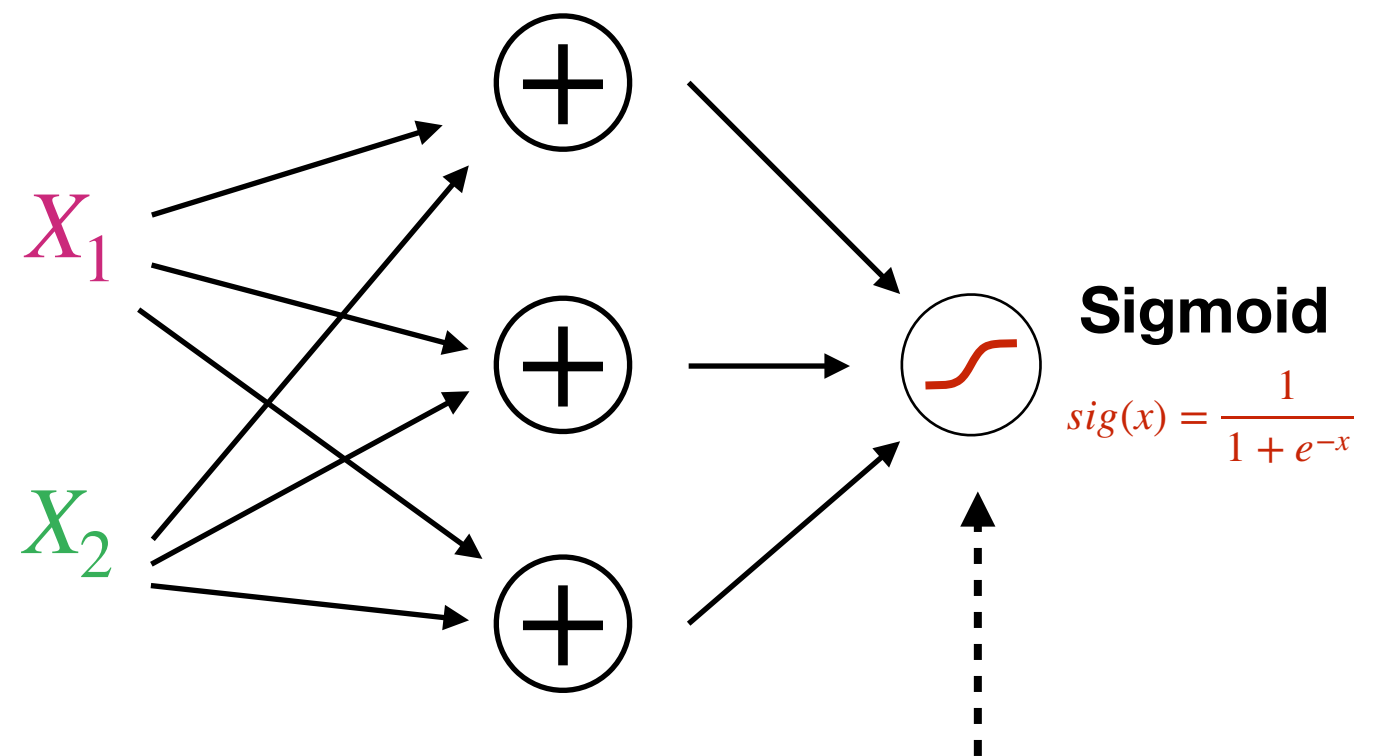
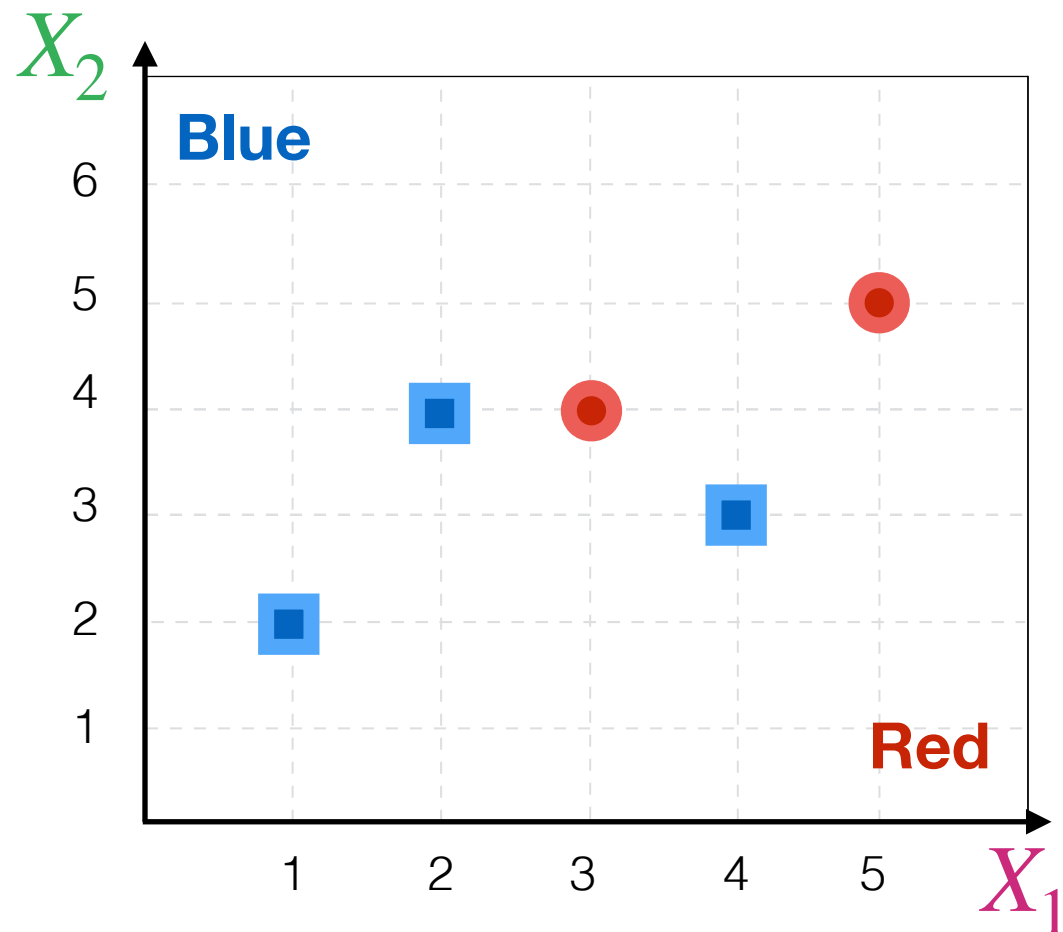


In this case **linear decision boundary** won't help.

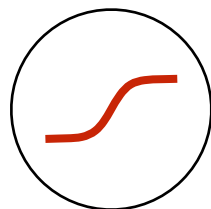


Using more **neurons** and ...

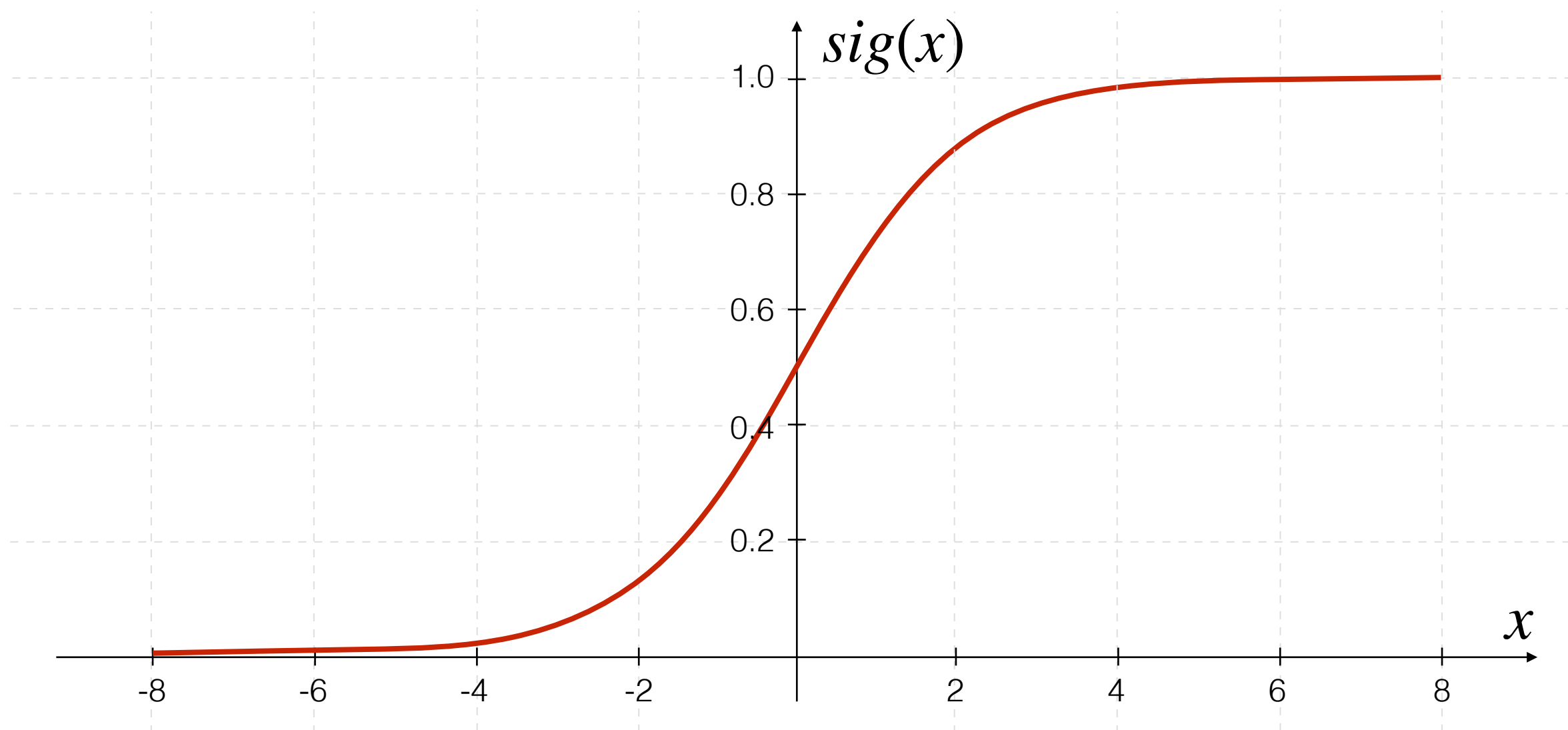
In this case **linear decision boundary** won't help.



Using more **neurons** and **non-linear** activation function

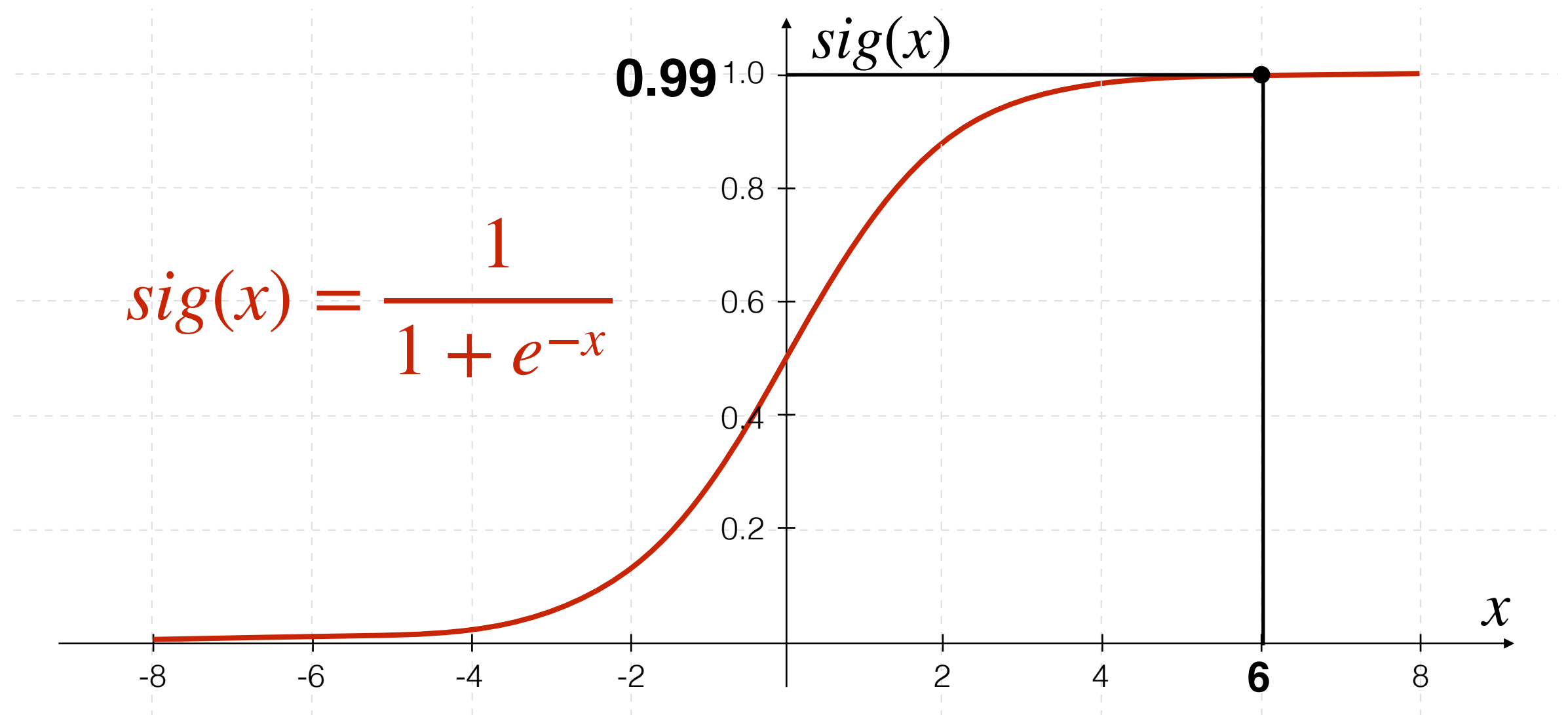


Sigmoid activation function



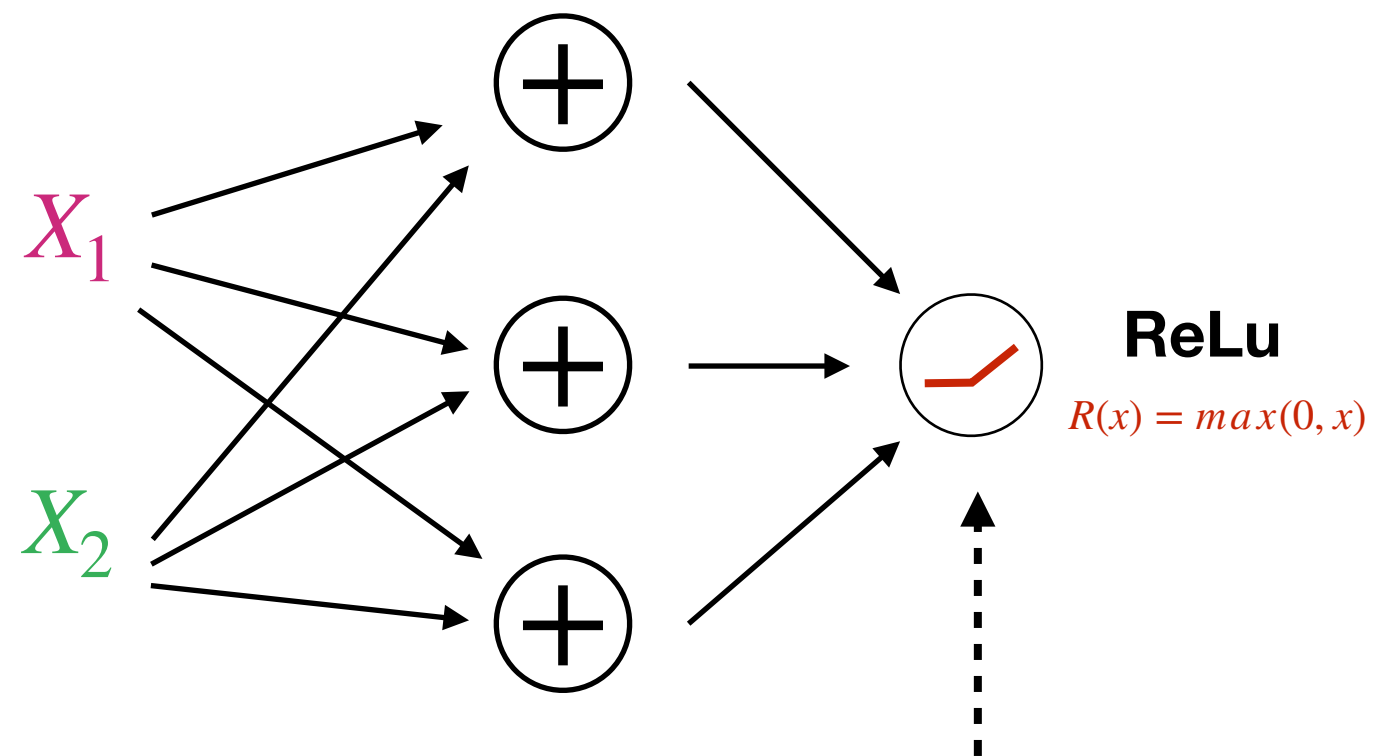
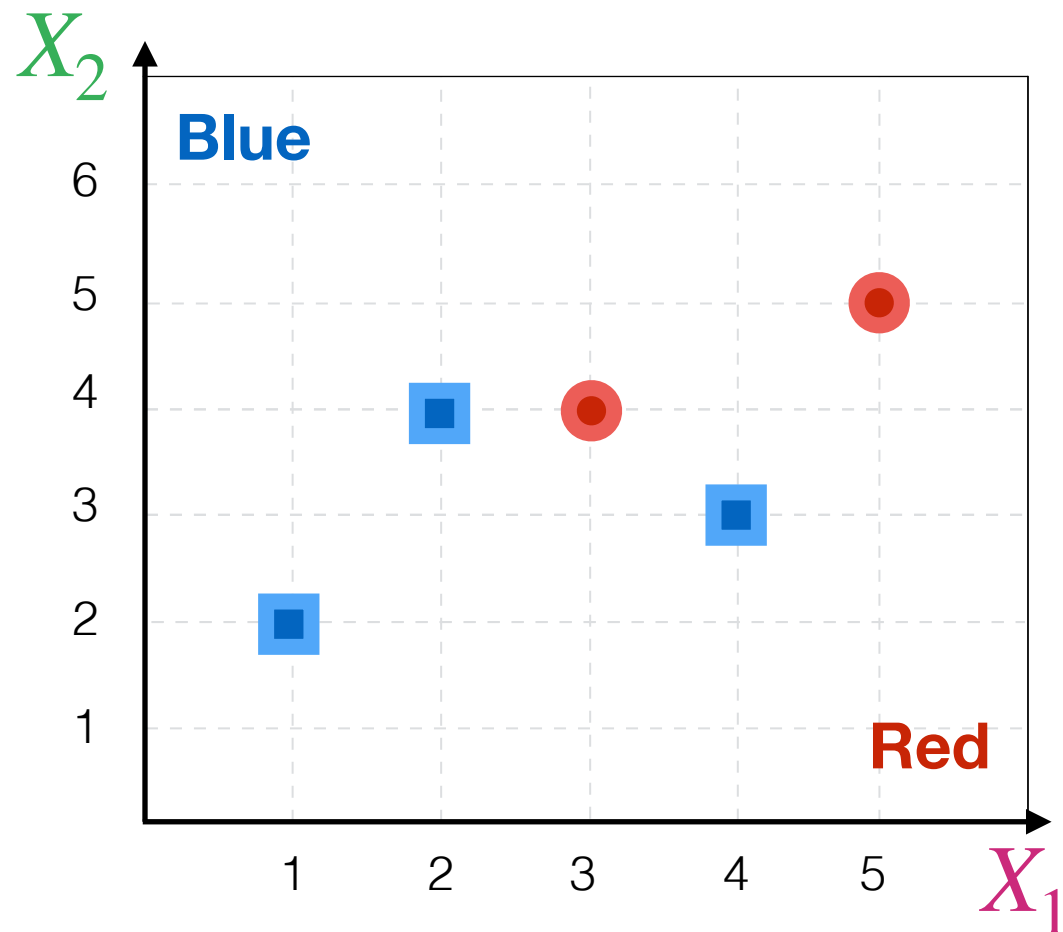


Sigmoid activation function

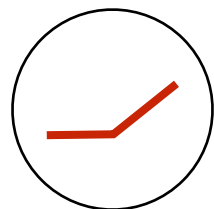


No matter how big your **x** can grow, your **sig(x)** will always be between **0** and **1**

In this case **linear decision boundary** won't help.

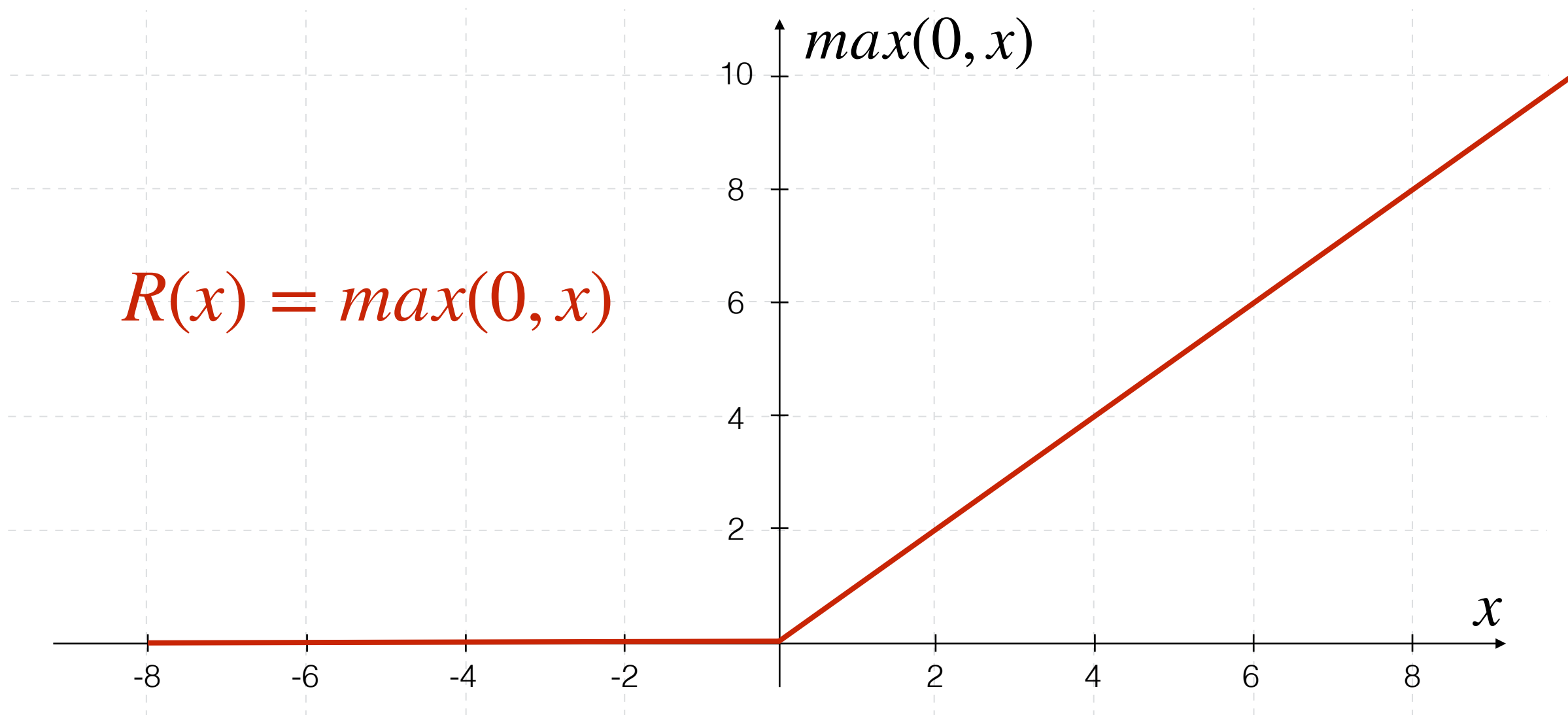


Using more **neurons** and **non-linear** activation function

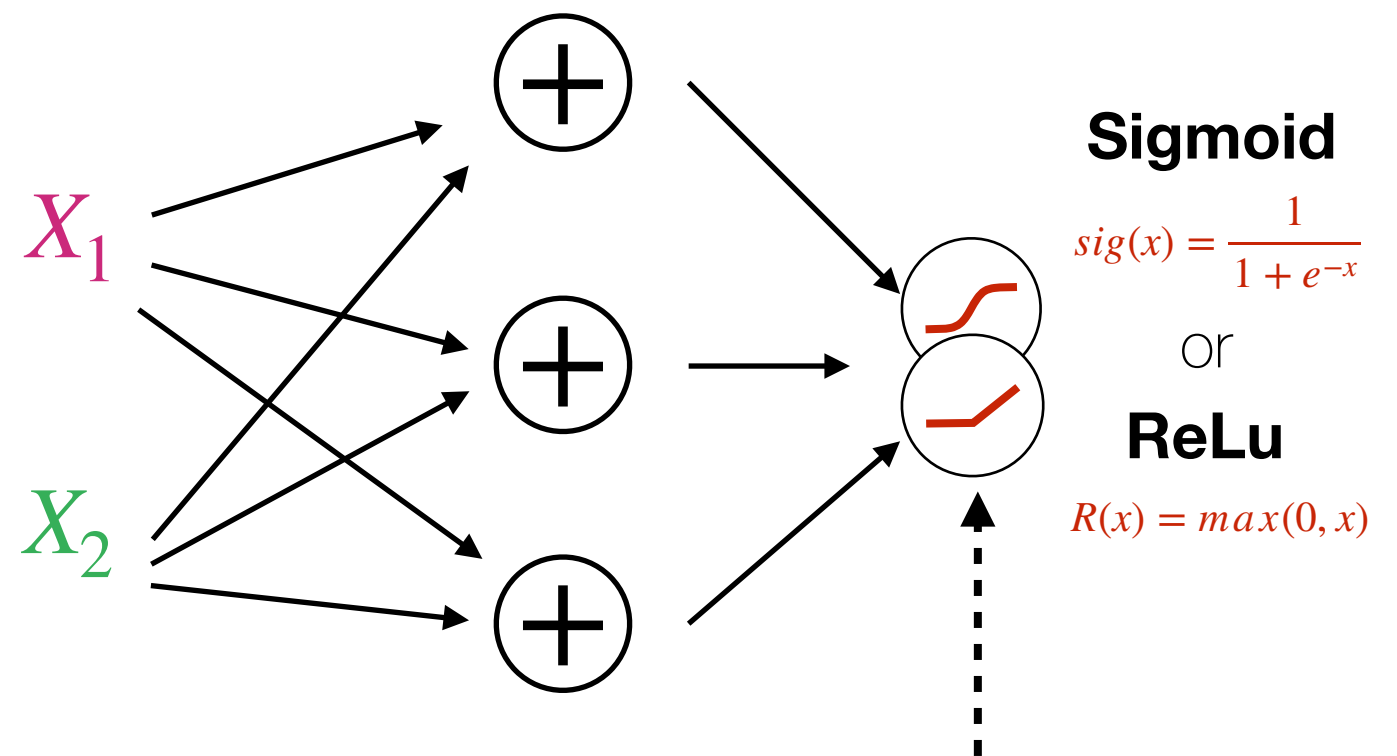
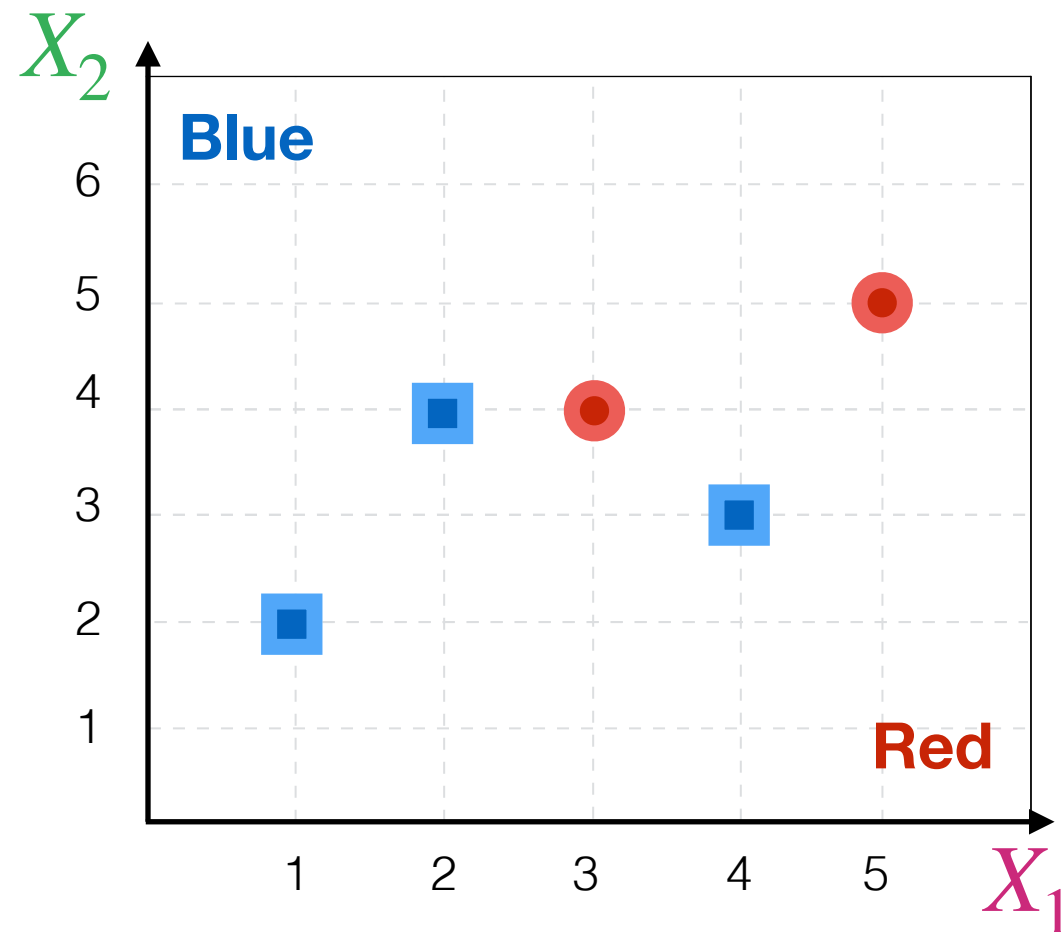


Rectified Linear Units (**ReLU**)

$$R(x) = \max(0, x)$$

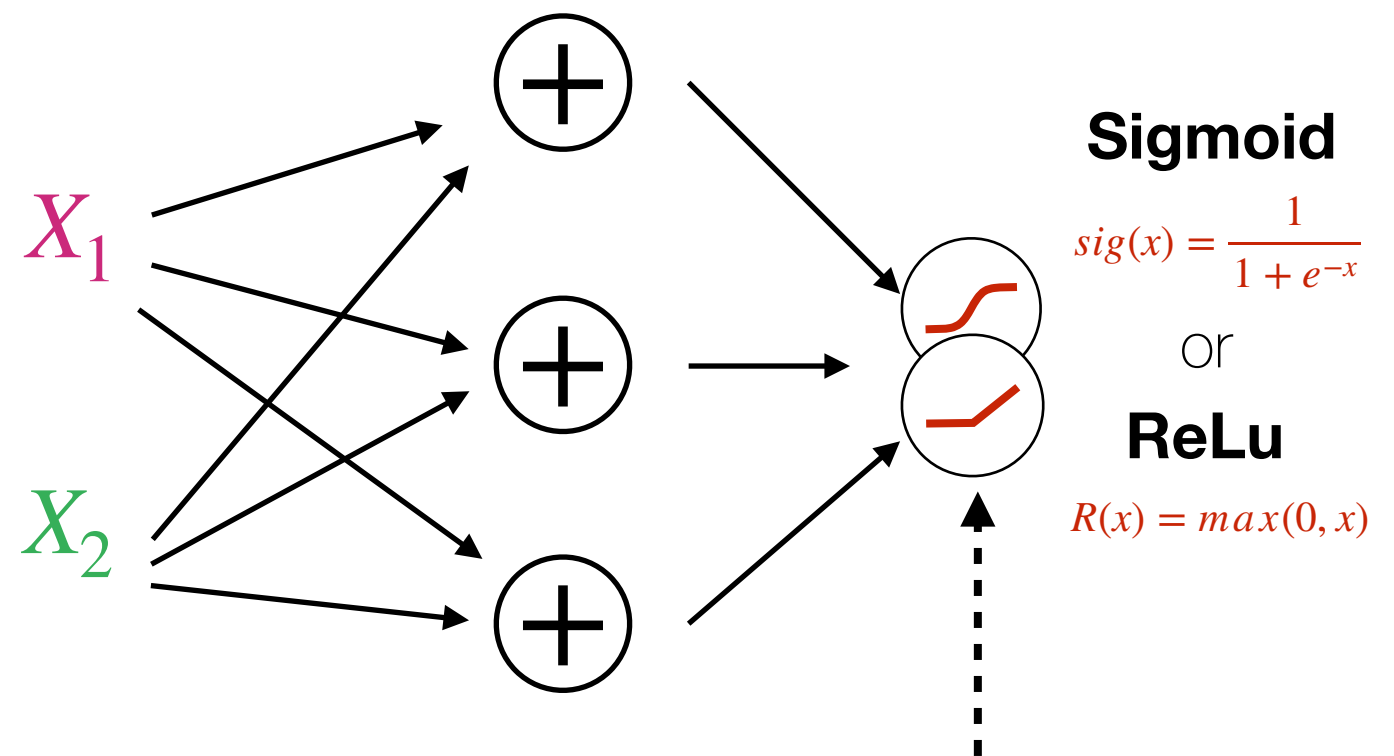
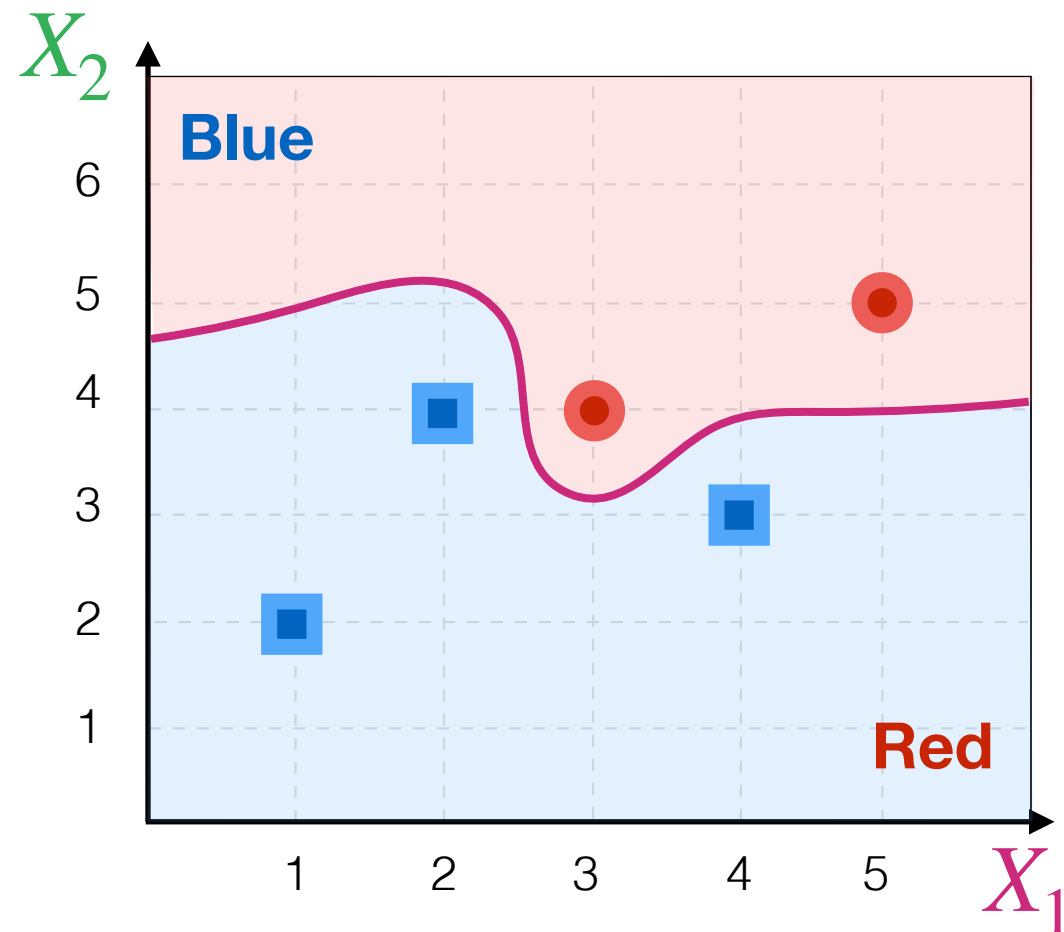


In this case **linear decision boundary** won't help.



Using more **neurons** and **non-linear** activation function

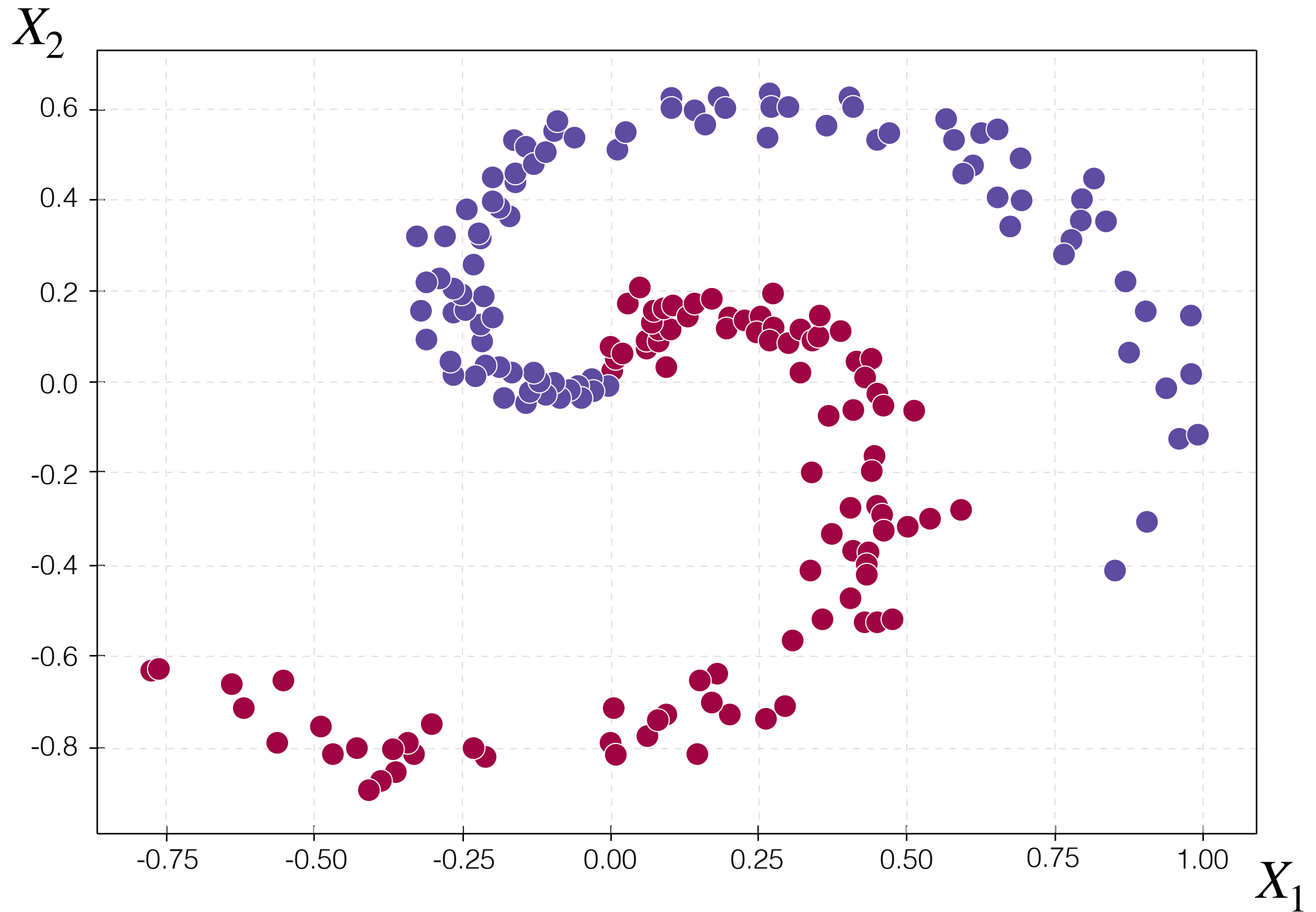
In this case **linear decision boundary** won't help.



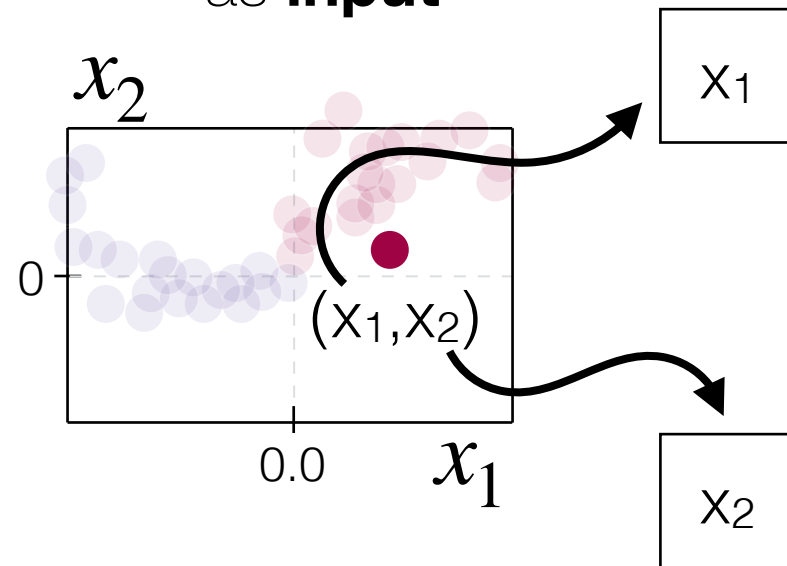
Using more **neurons** and **non-linear** activation function

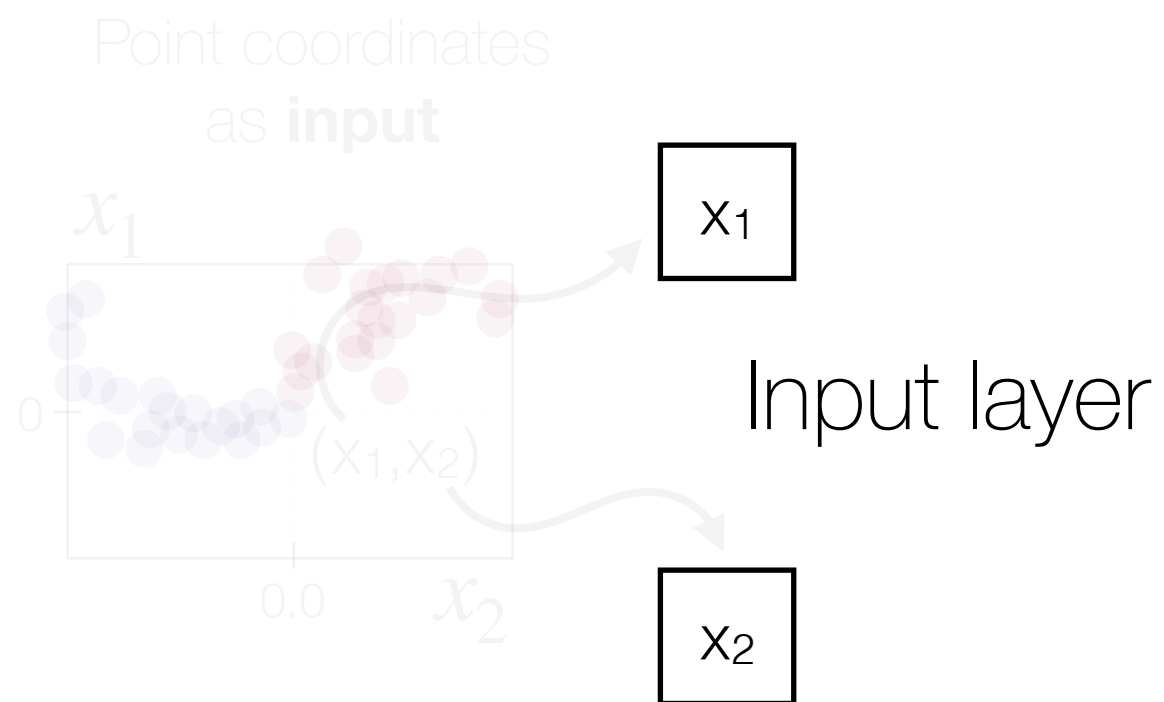
It is possible to obtain non-linear **decision boundary**

Consider this **2D** example

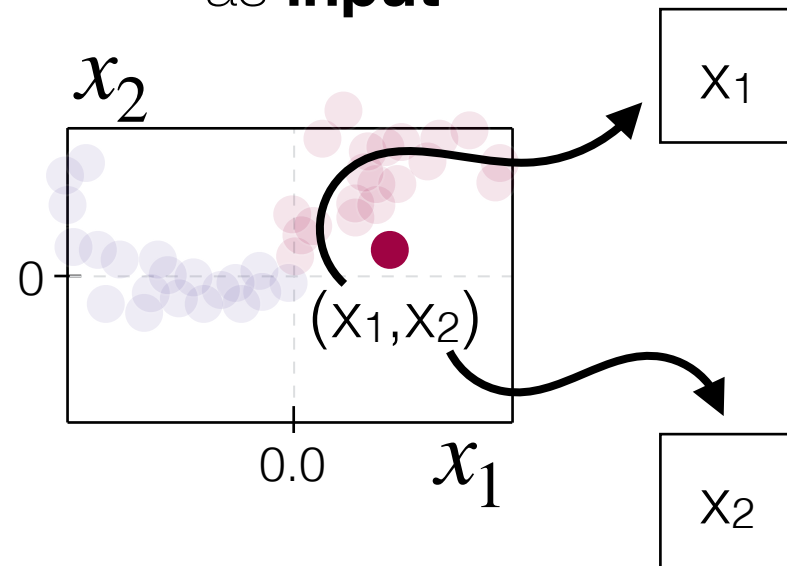


Point coordinates
as **input**

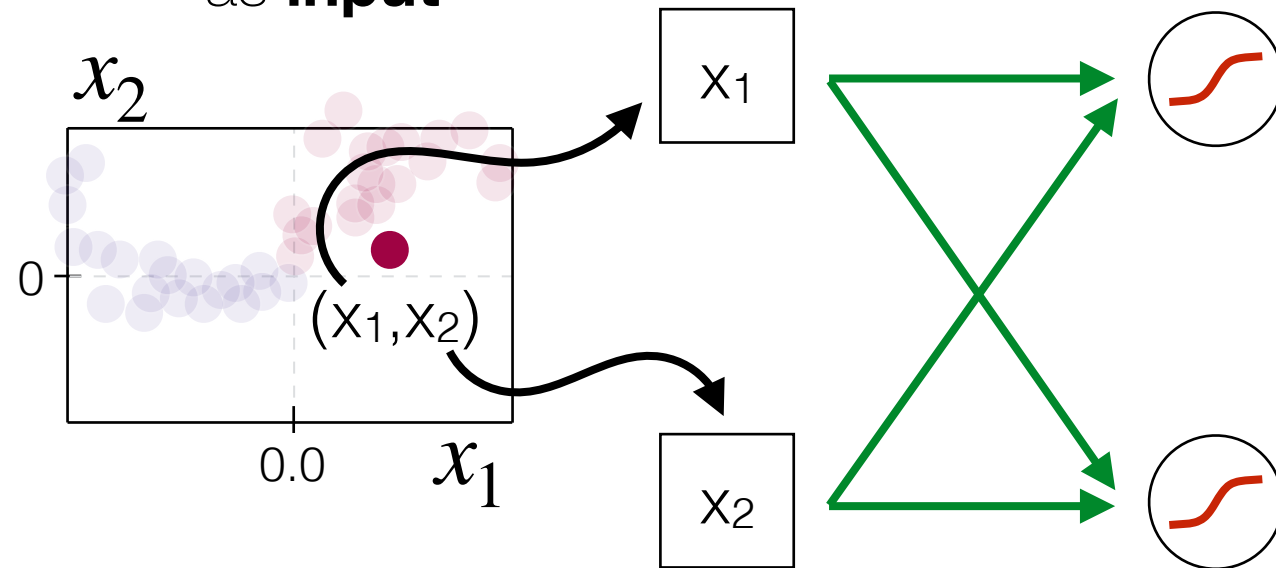


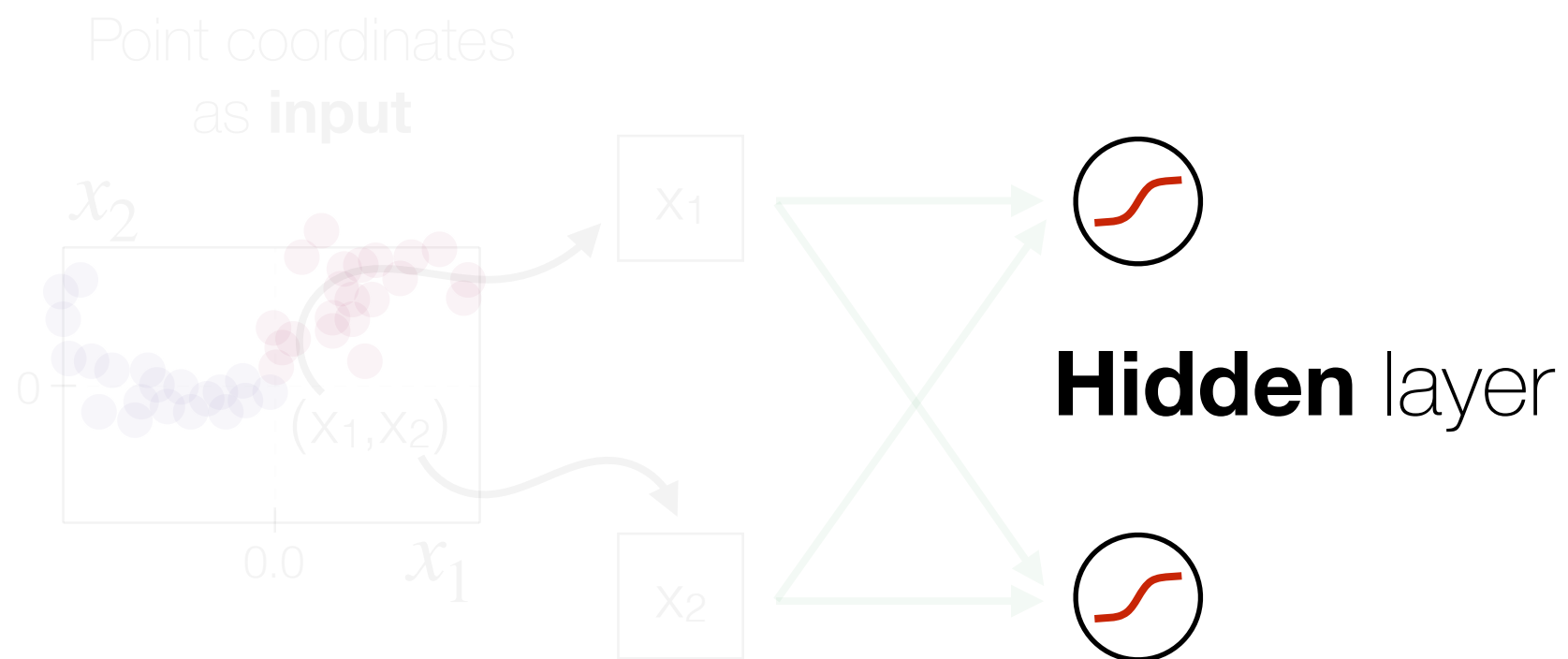


Point coordinates
as **input**

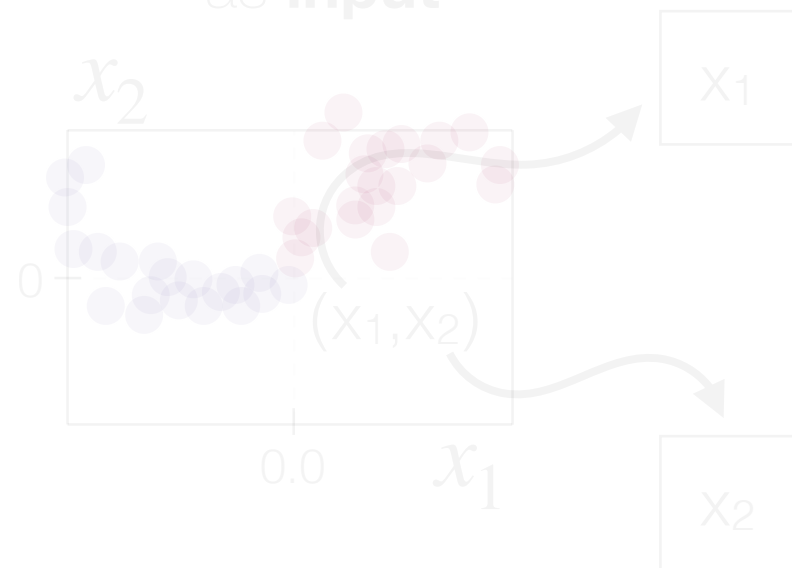


Point coordinates
as **input**



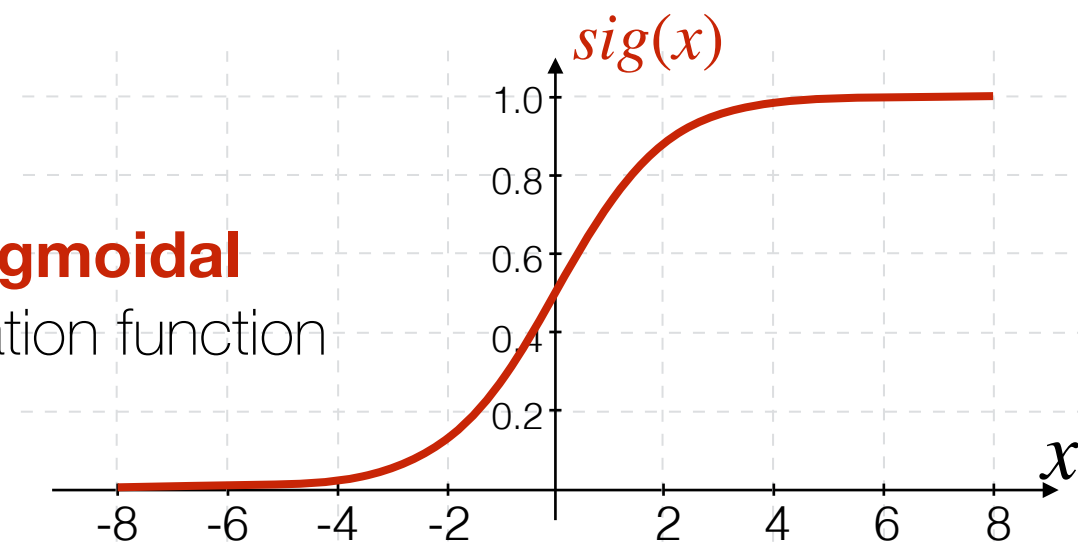


Point coordinates
as **input**

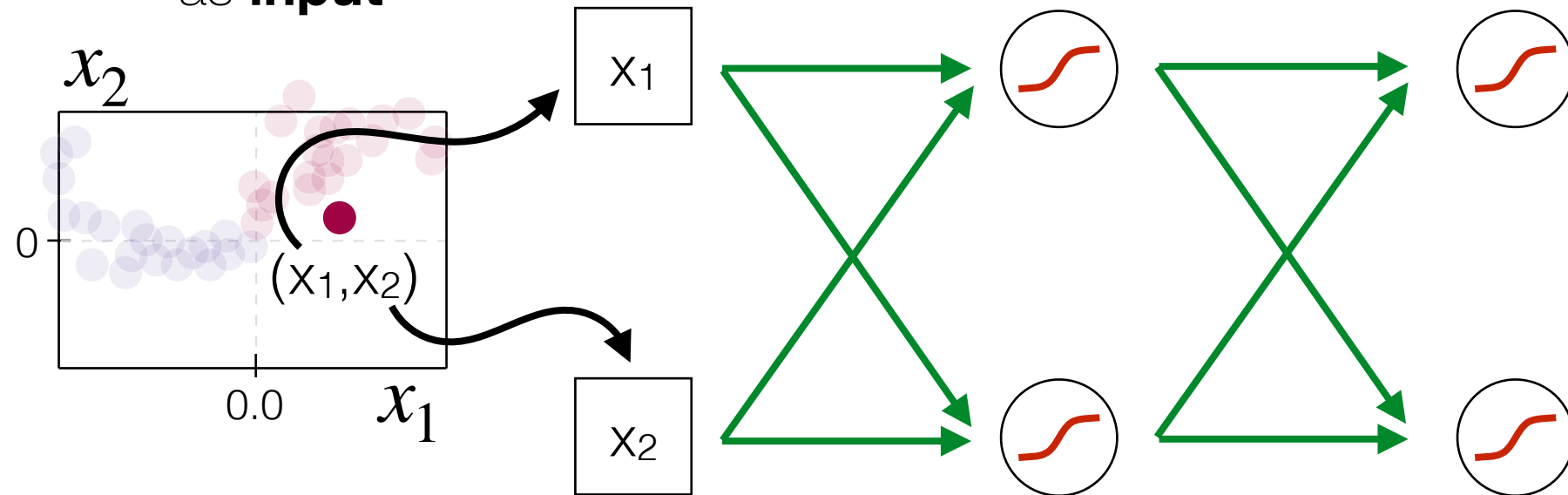


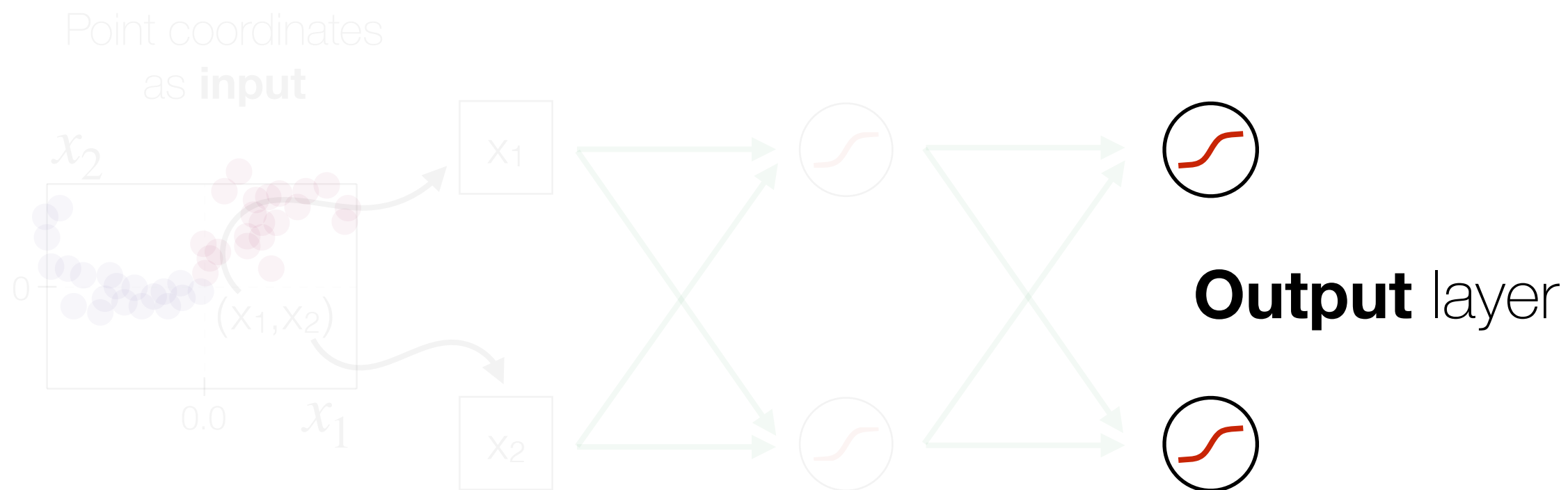
Hidden layer

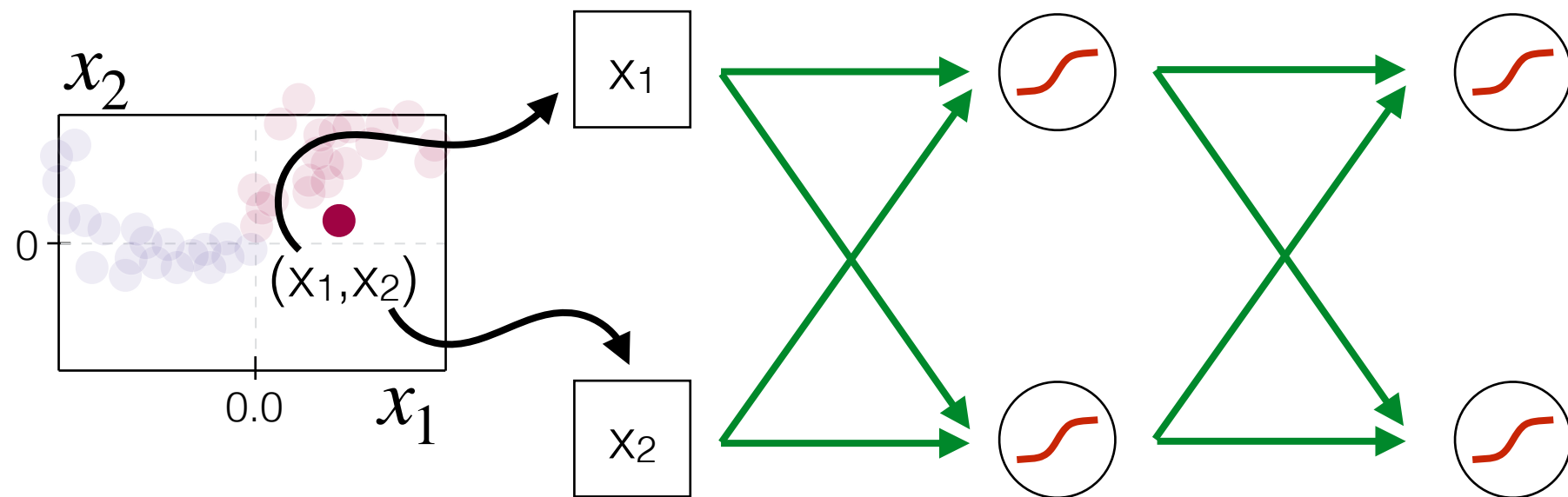
Sigmoidal
activation function

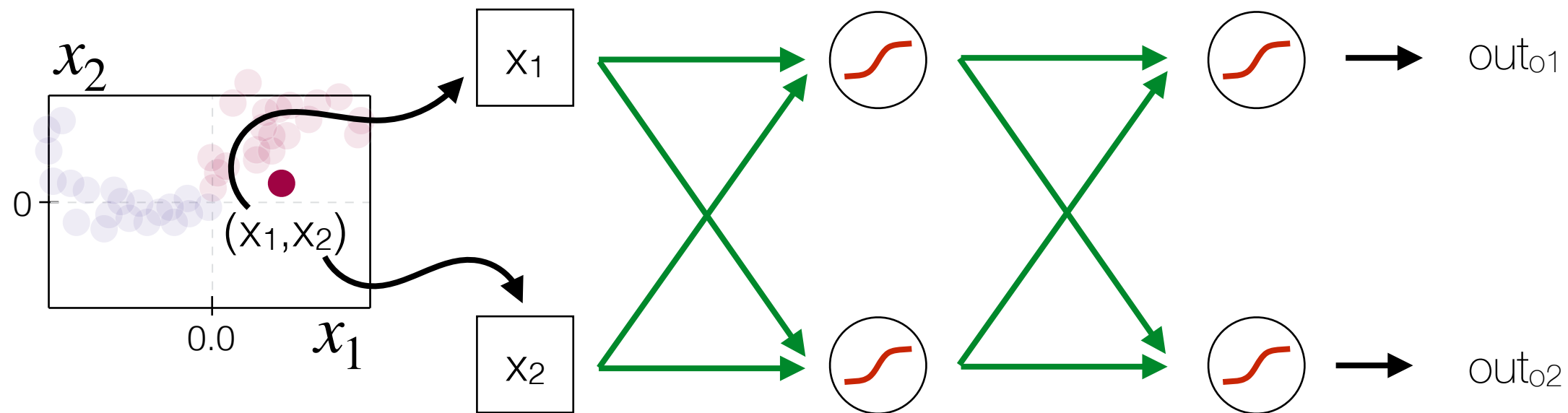


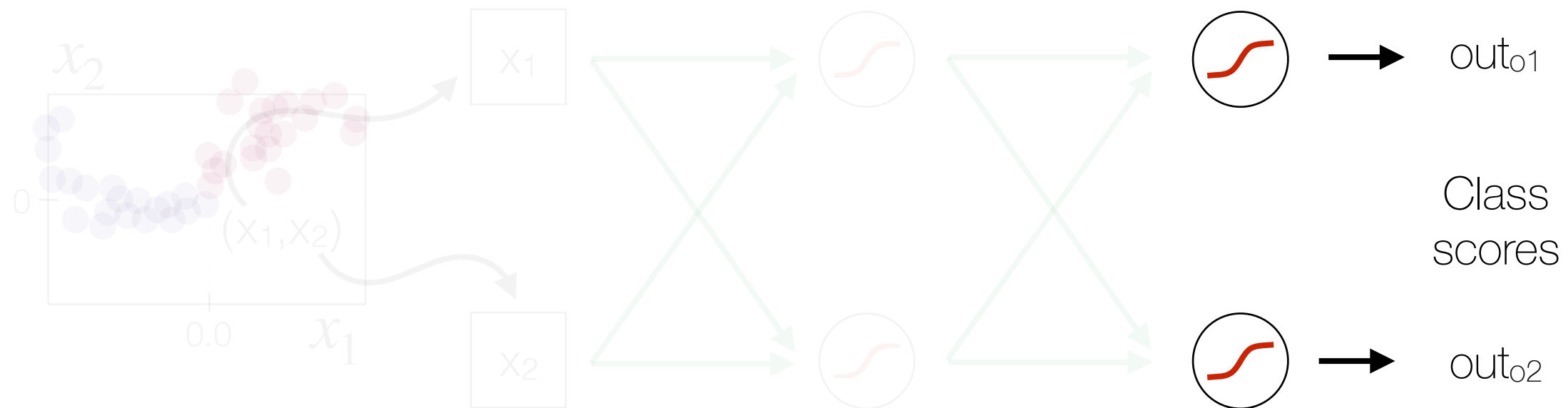
Point coordinates
as **input**

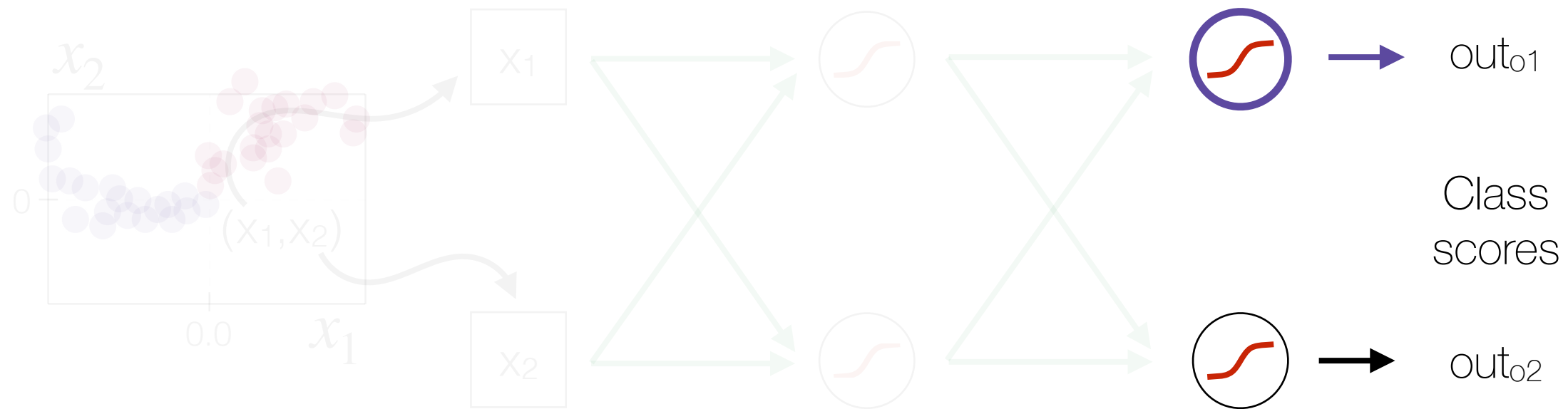


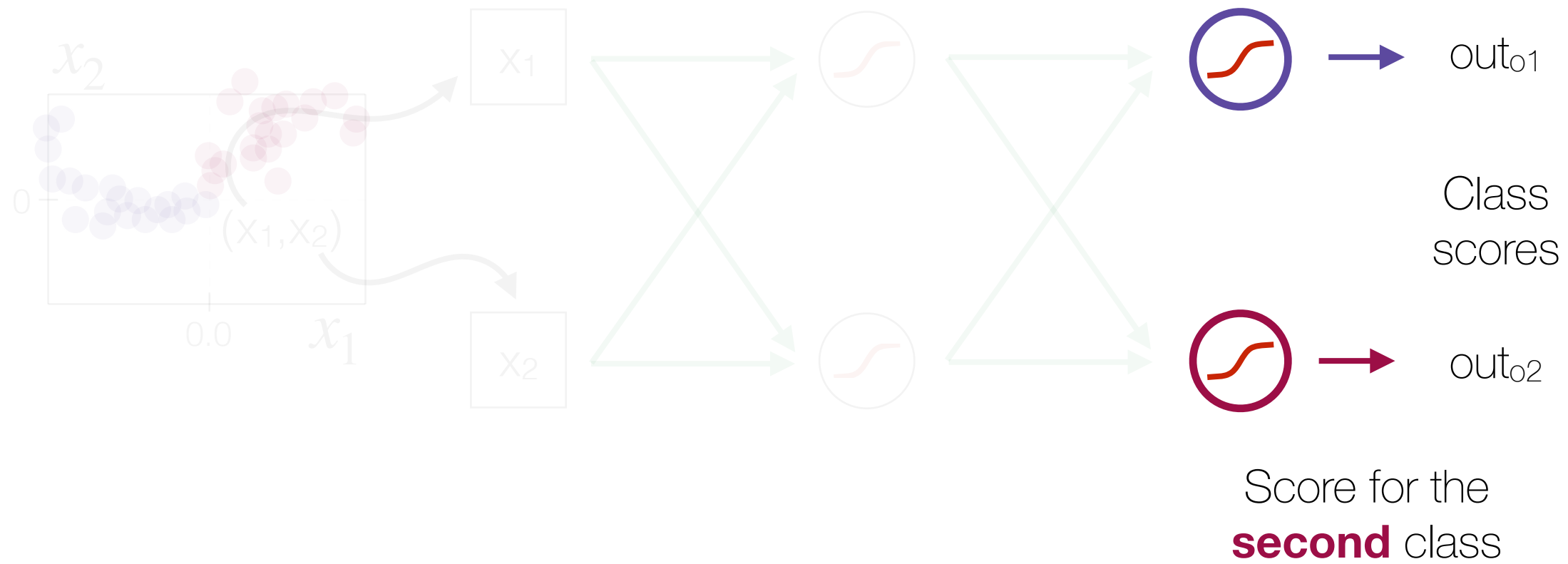




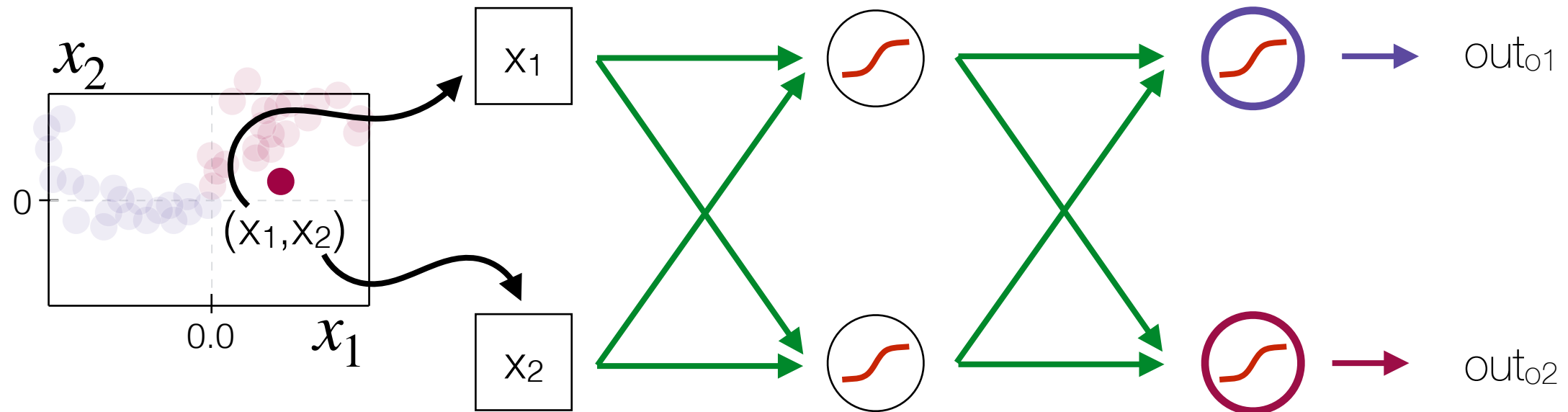






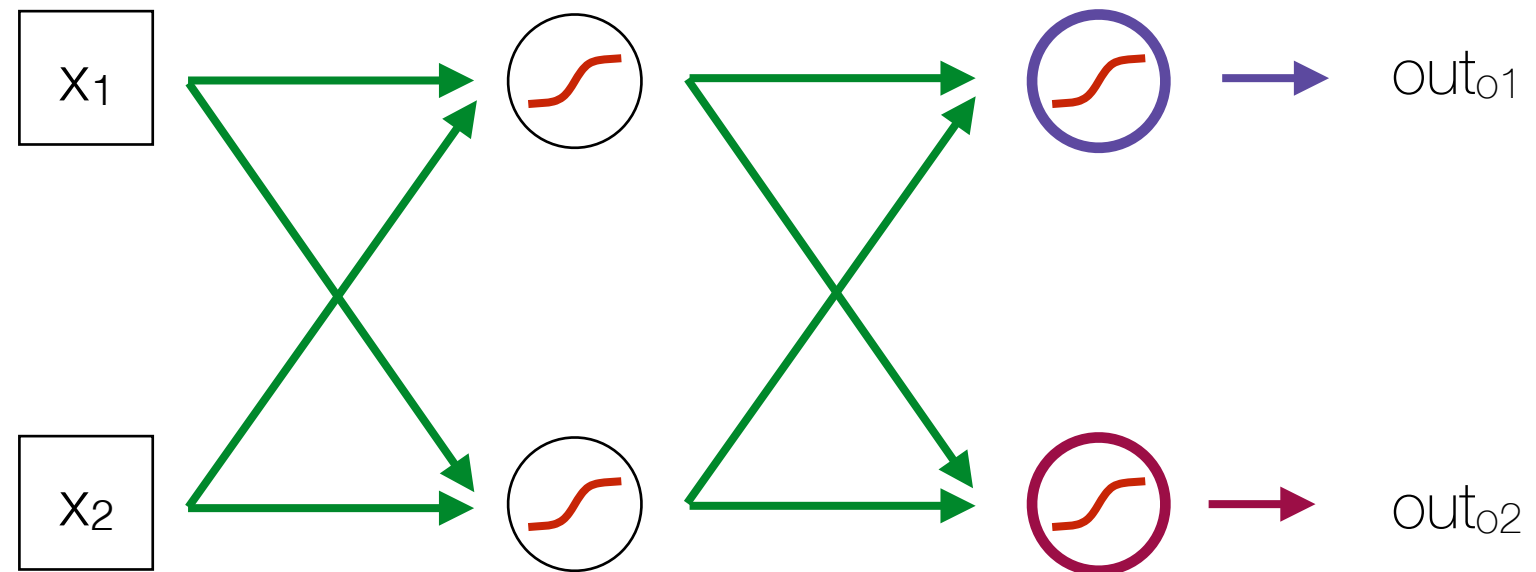
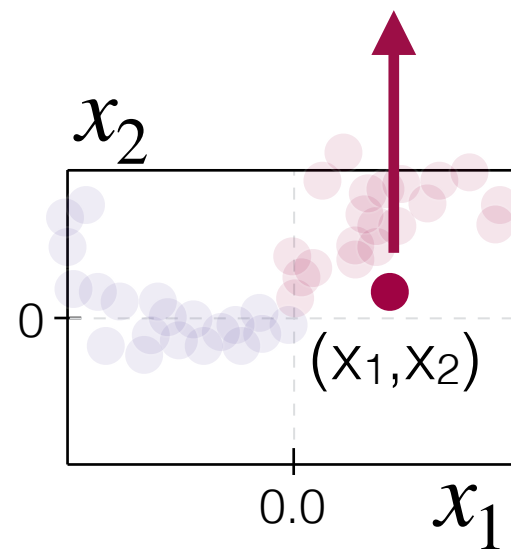


If **out₀₁** > **out₀₂** neural network predicts the **first** class

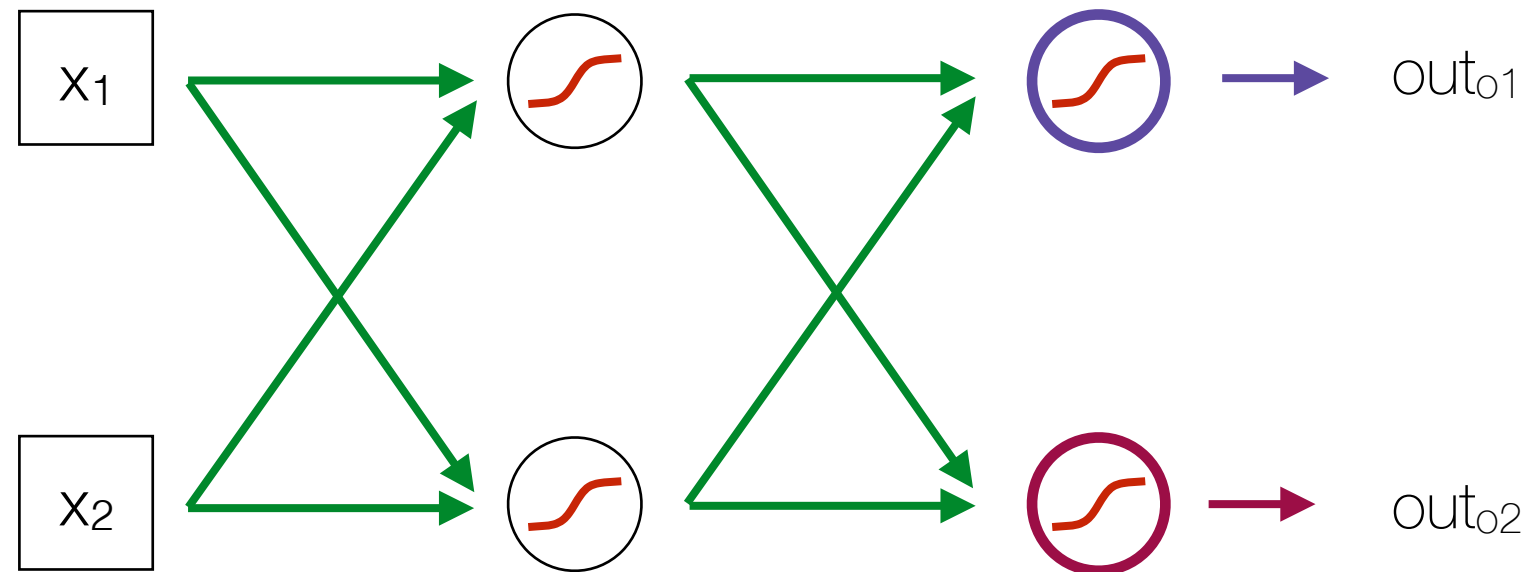
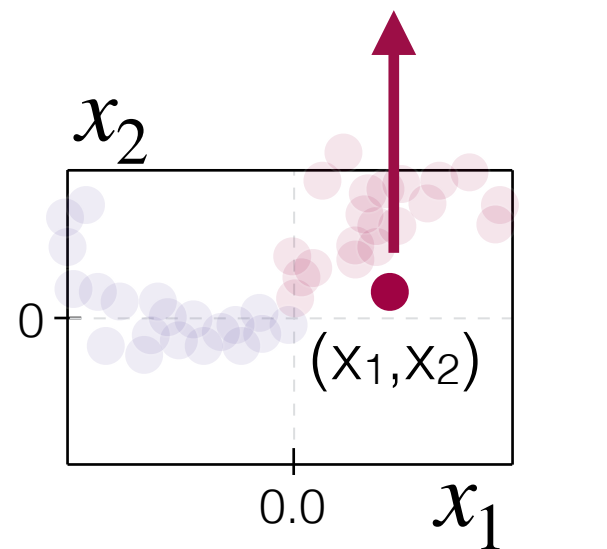


If **out₀₁** > **out₀₂** neural network predicts the **first** class

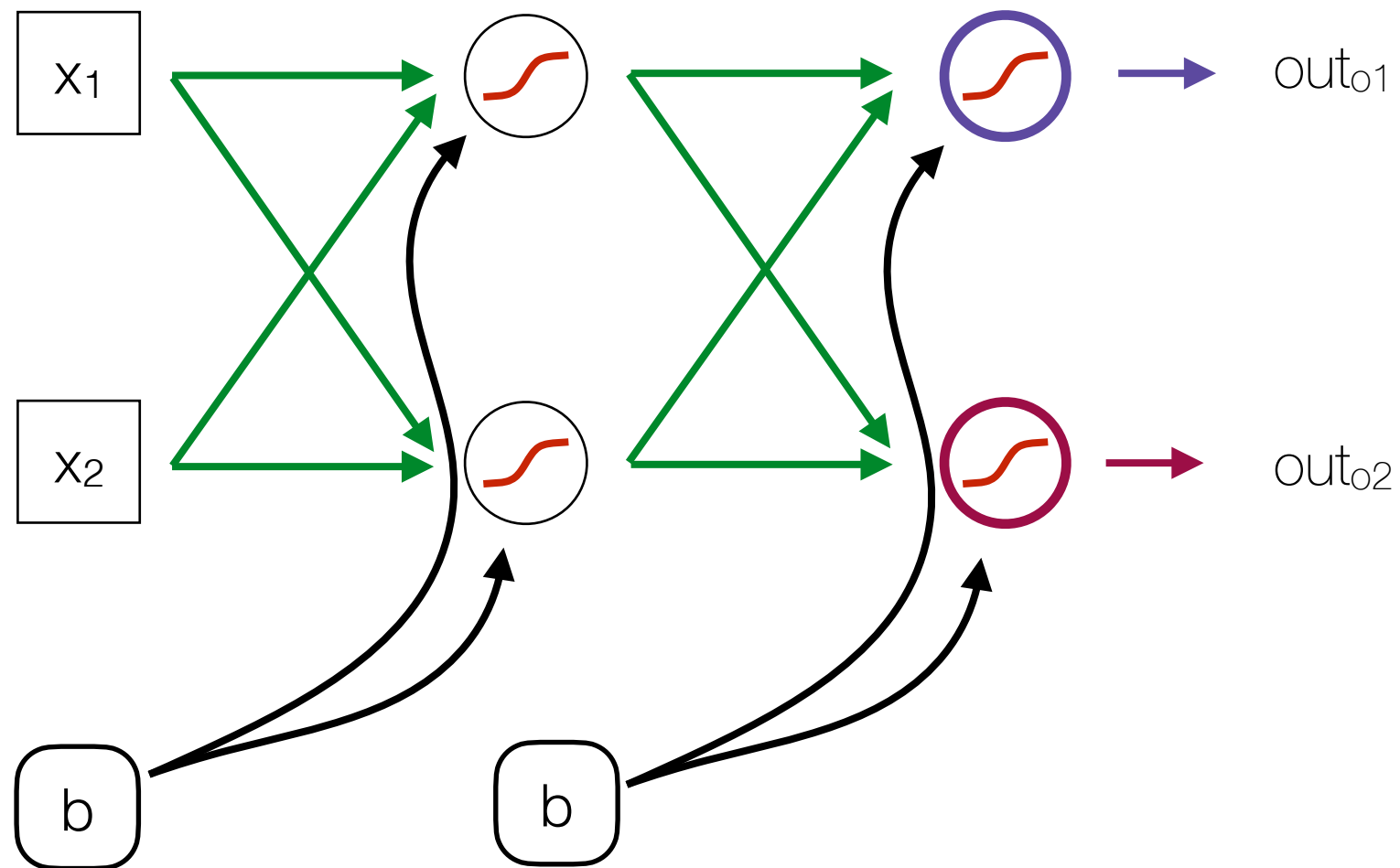
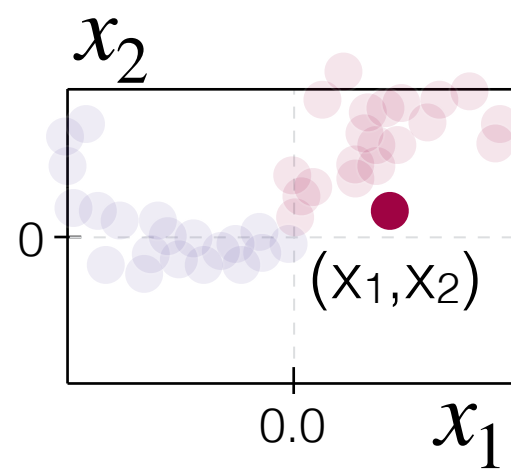
Our **input** point belongs to the **second** class

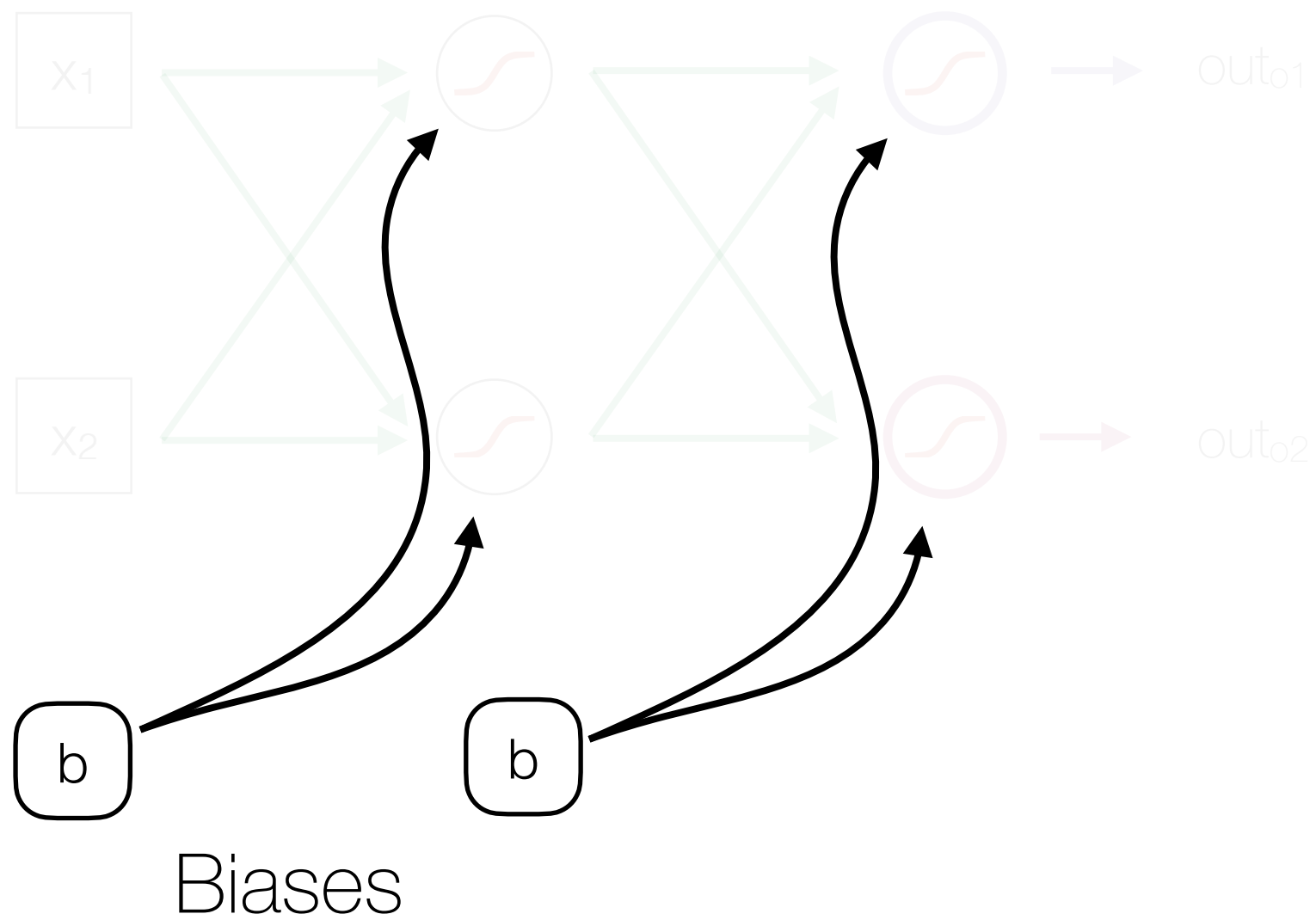
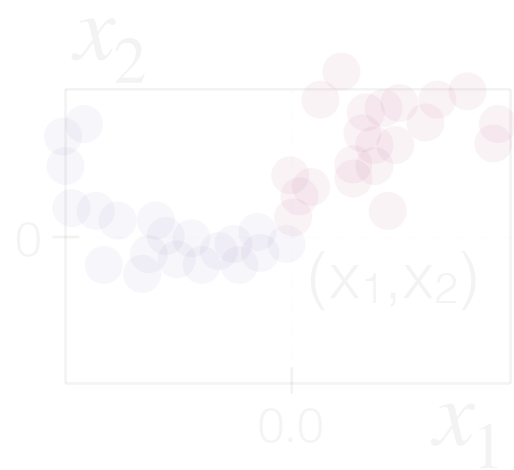


If $\text{out}_1 > \text{out}_2$ neural network predicts the **first** class

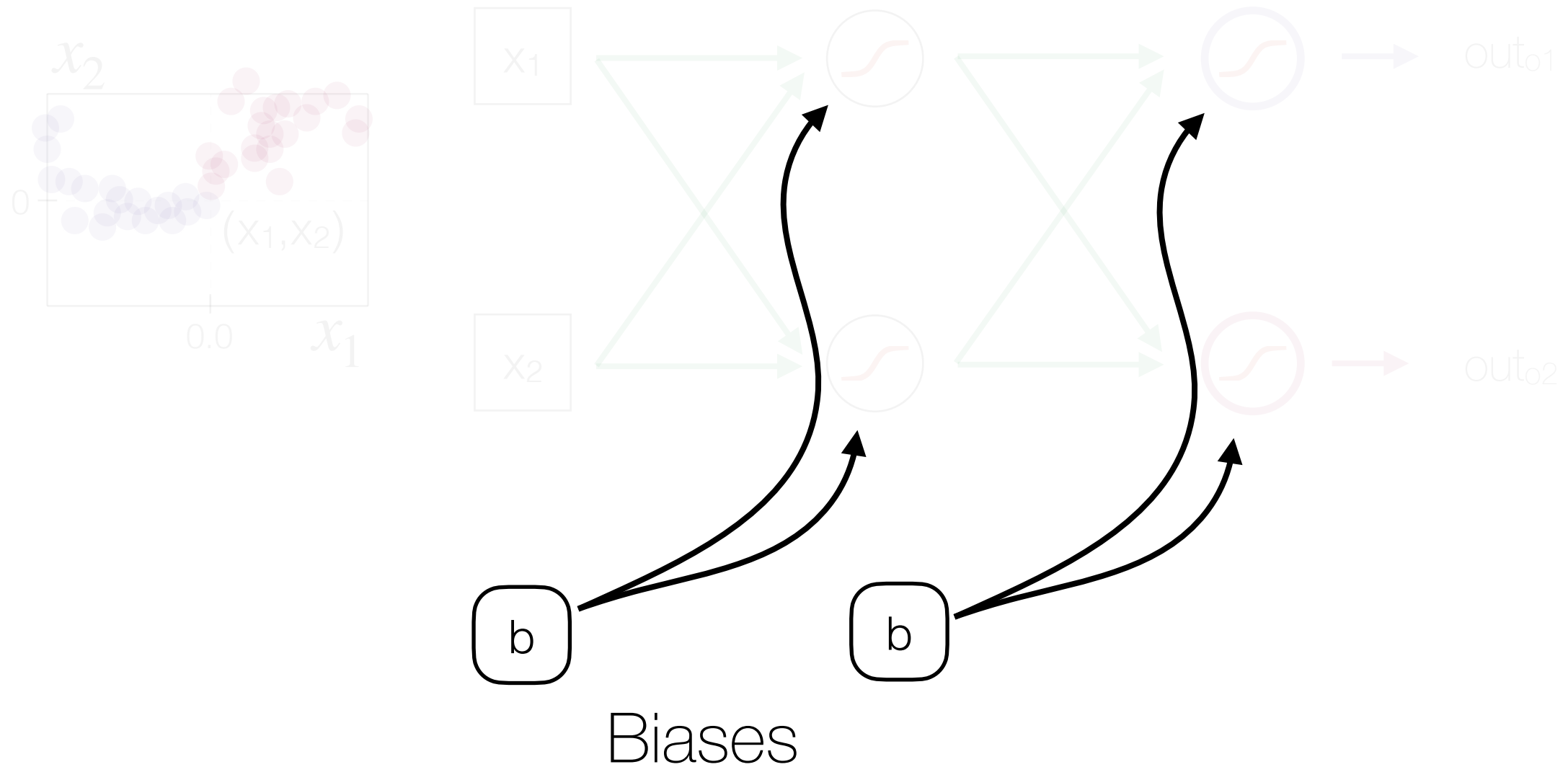


Therefore for this point we expect
 $\text{out}_2 > \text{out}_1$

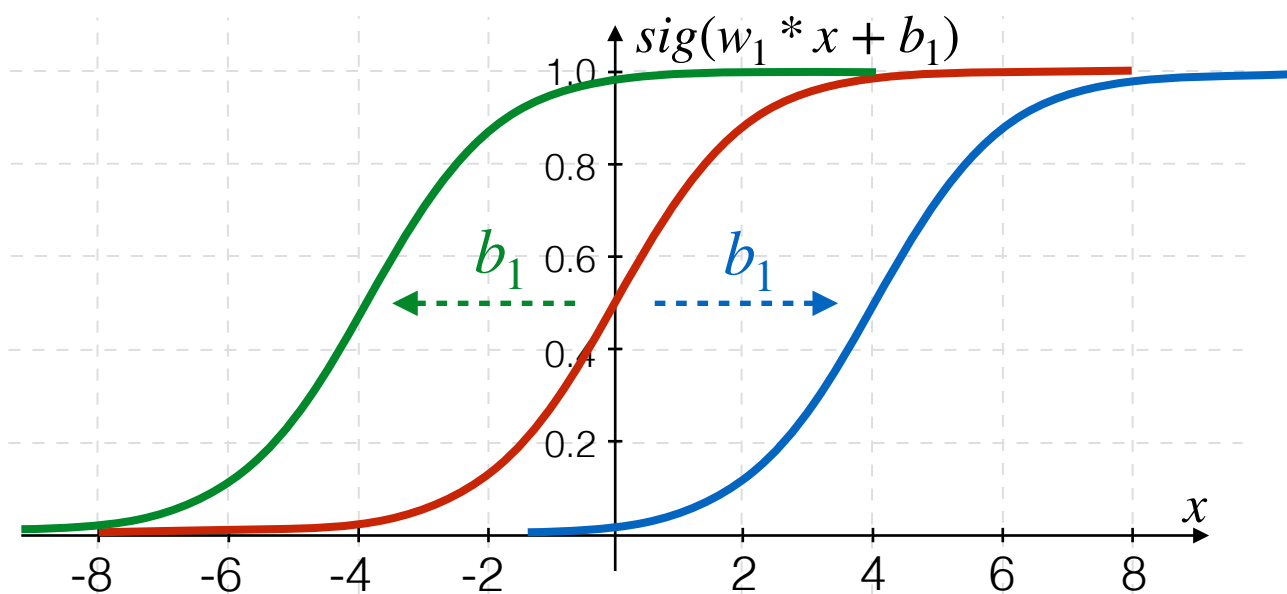
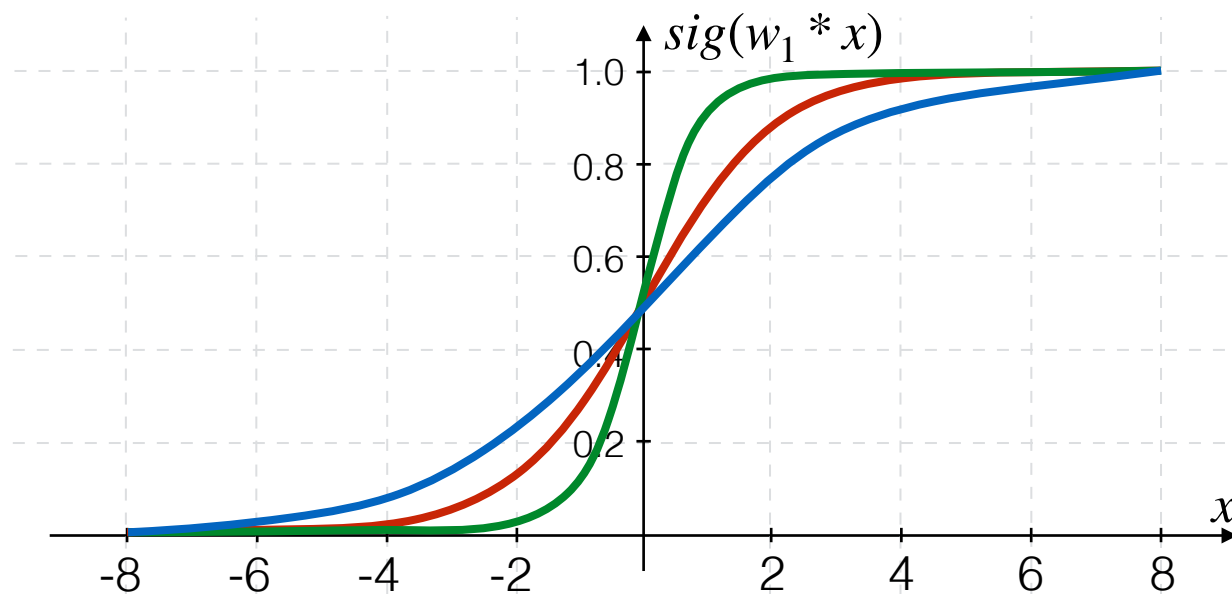
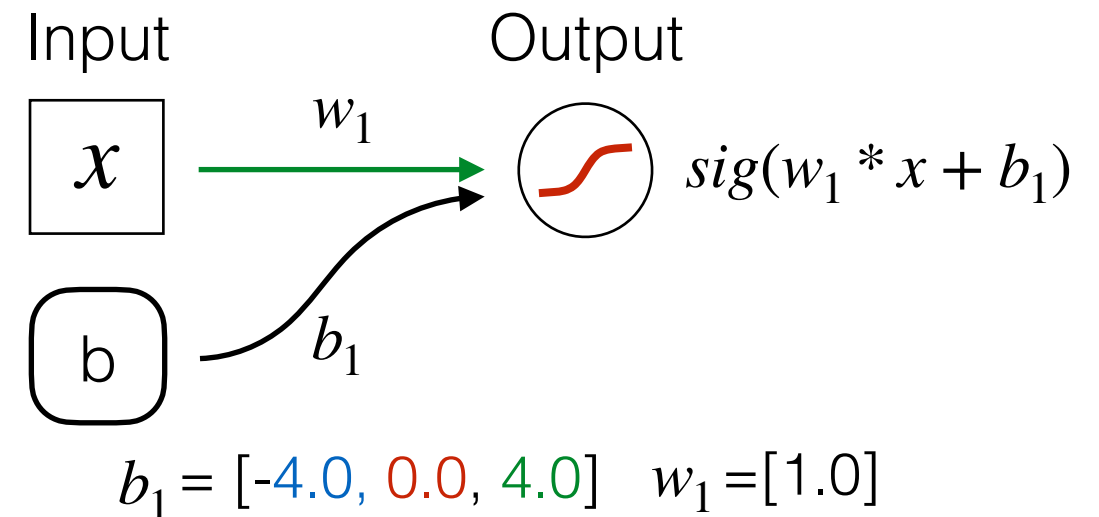
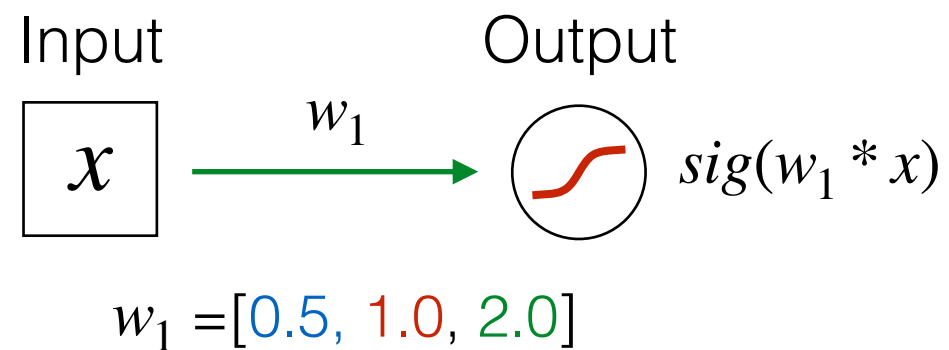




What is the role of **bias** in Neural Networks?

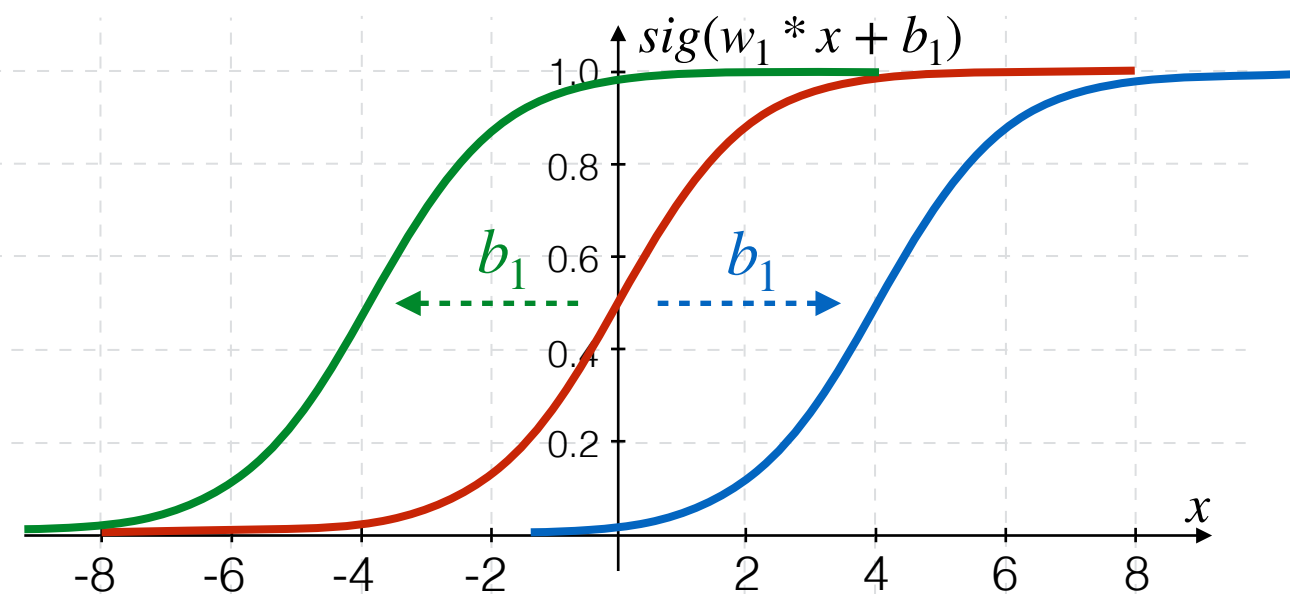
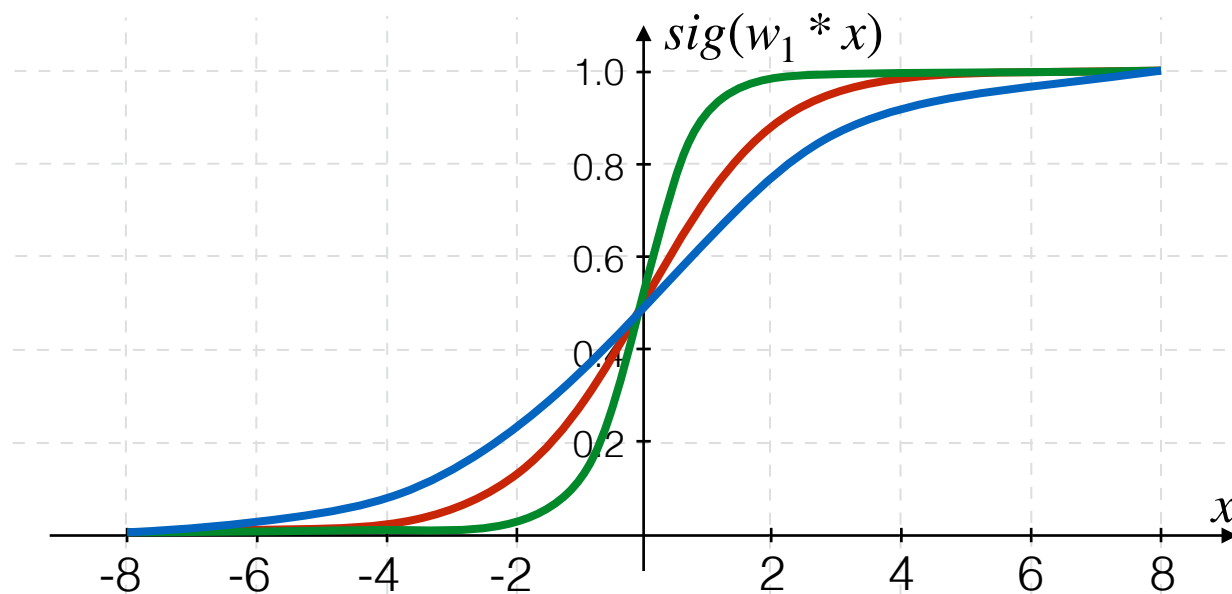
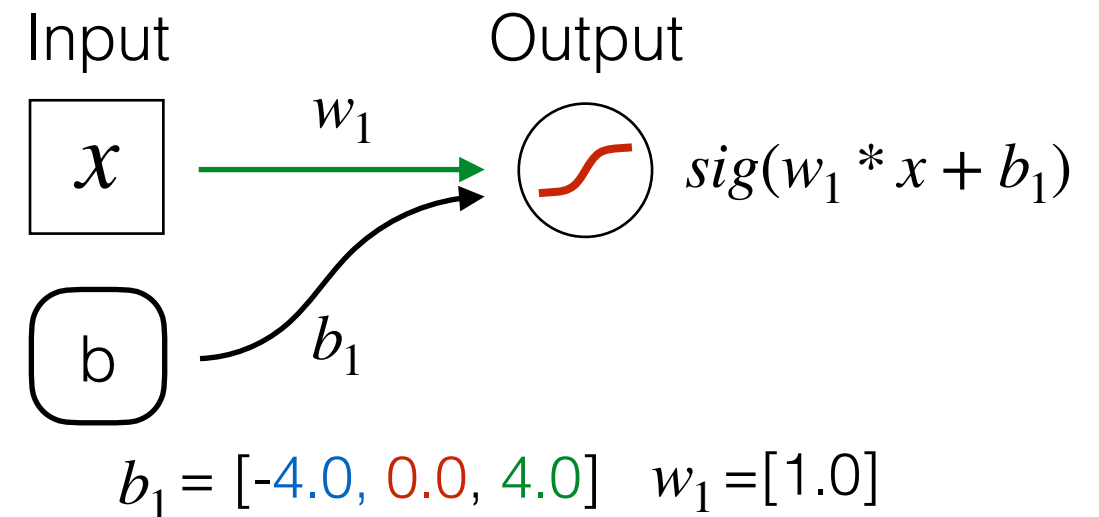
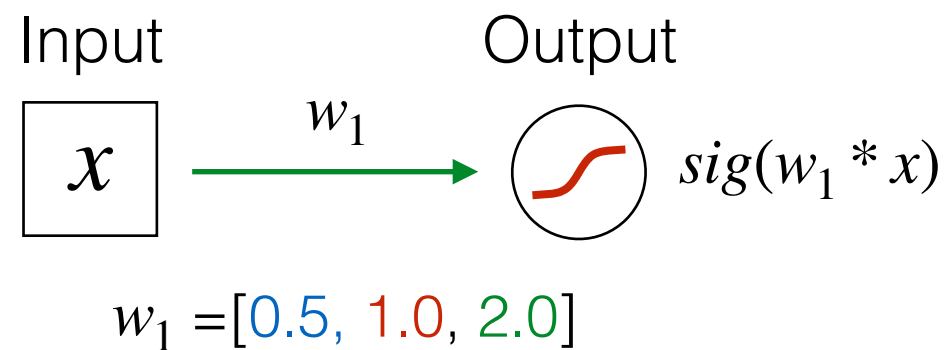


What is the role of **bias** in Neural Networks?



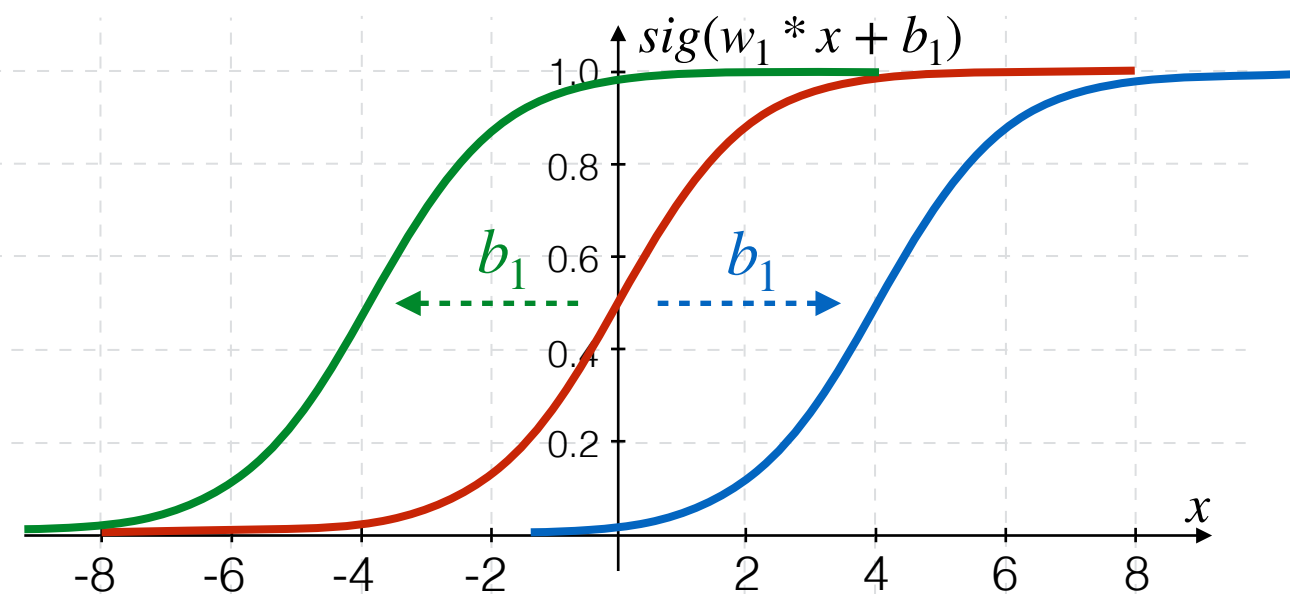
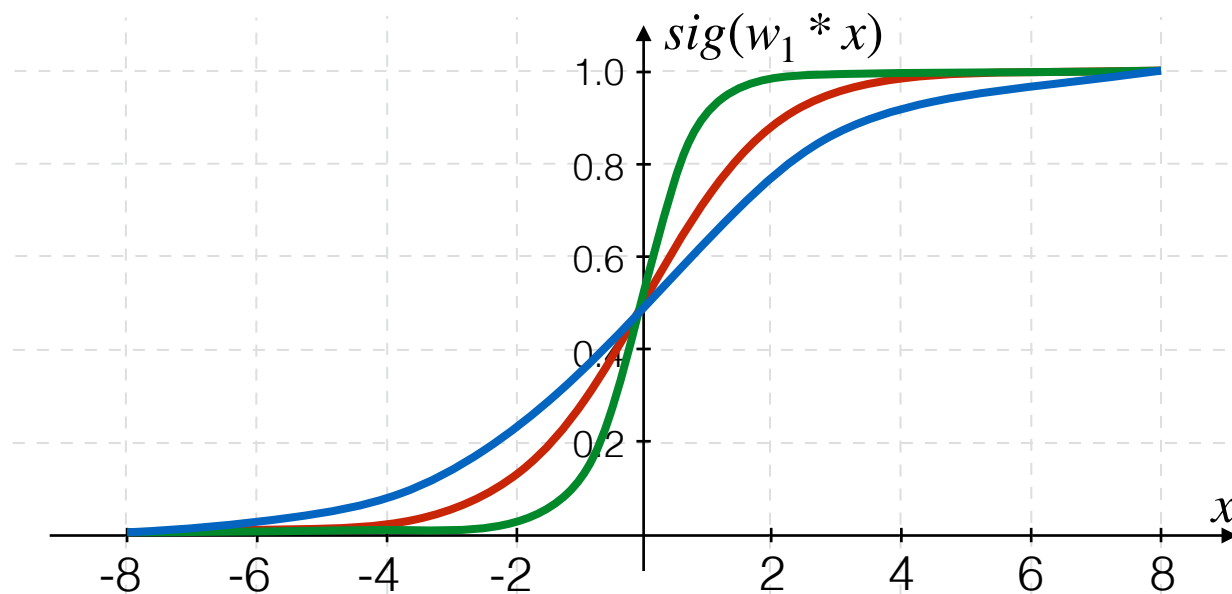
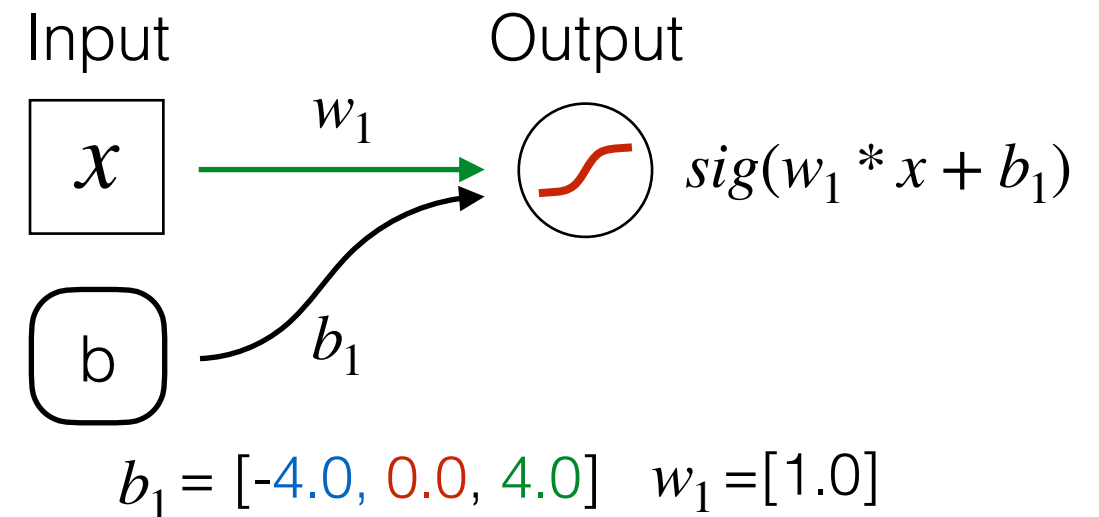
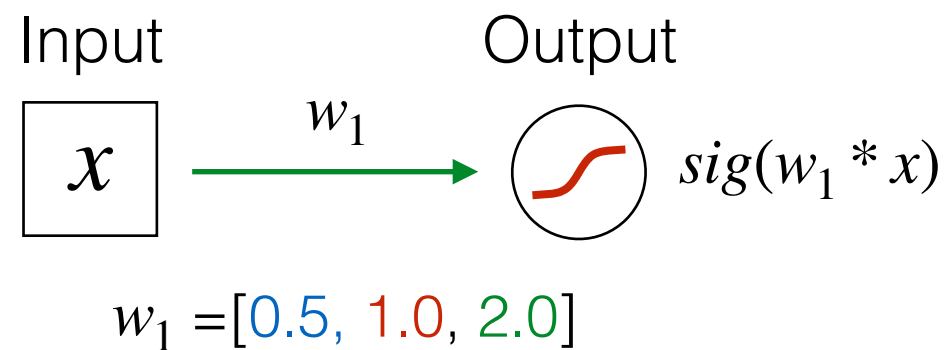
Bias helps to **shift** the resulting curve

What is the role of **bias** in Neural Networks?

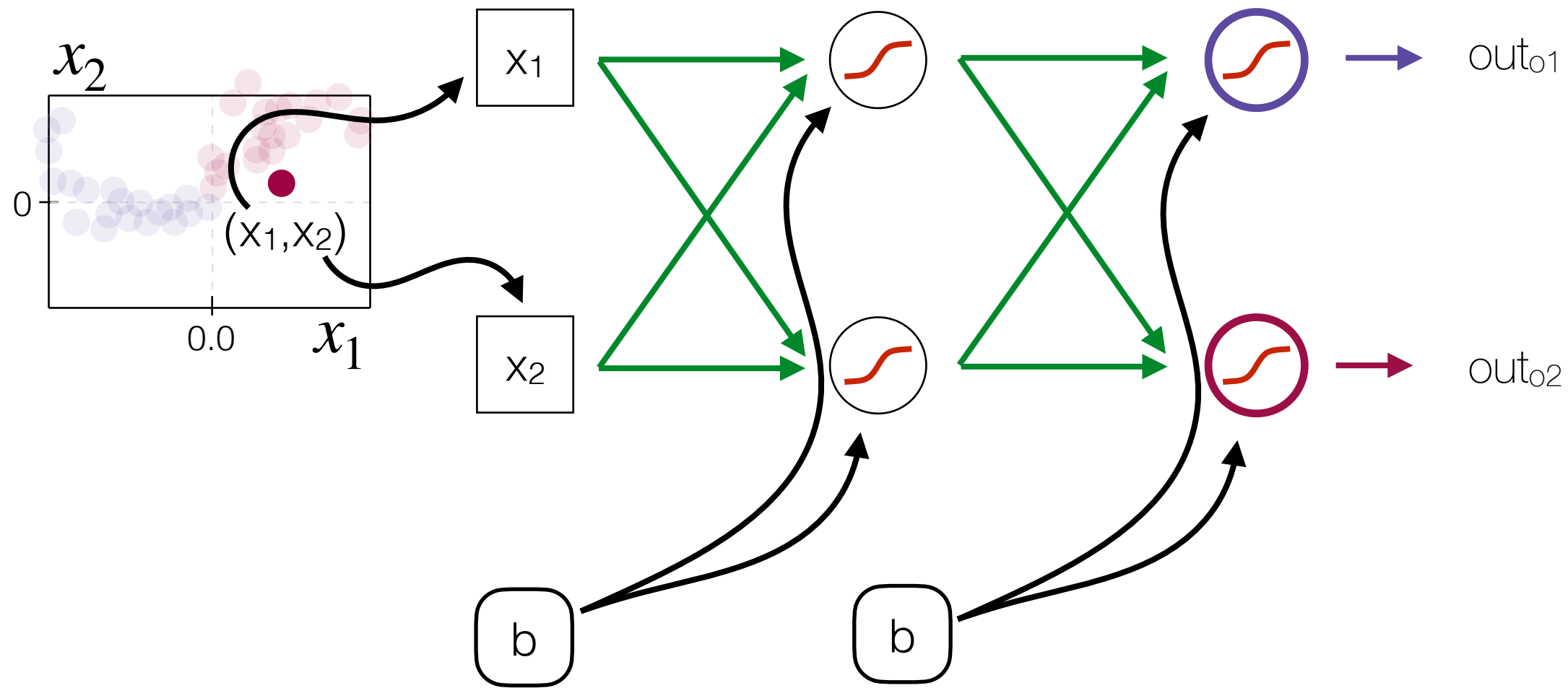


$$\text{sig}(x) = \frac{1}{1 + e^{-x}}$$

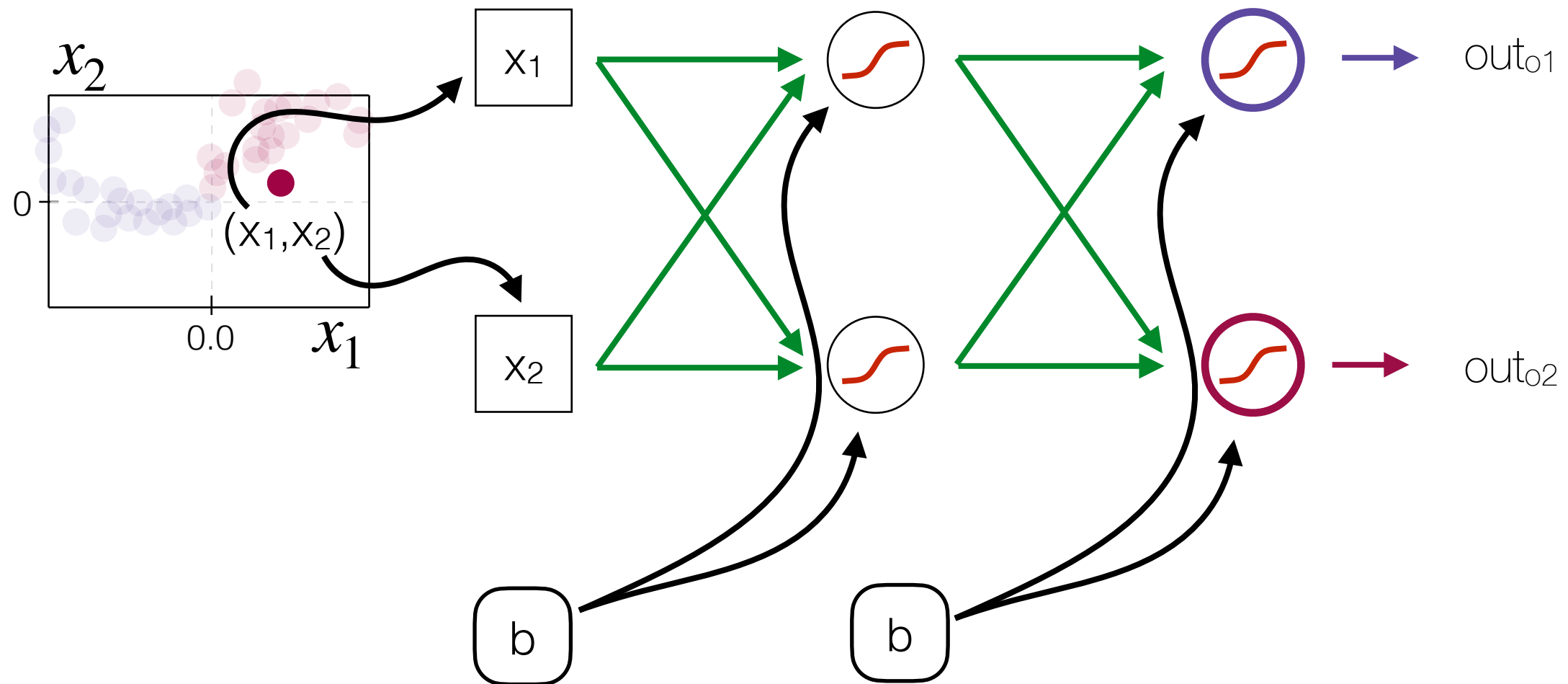
What is the role of **bias** in Neural Networks?



$$\text{sig}(x) = \frac{1}{1 + e^{-(wx+b)}}$$

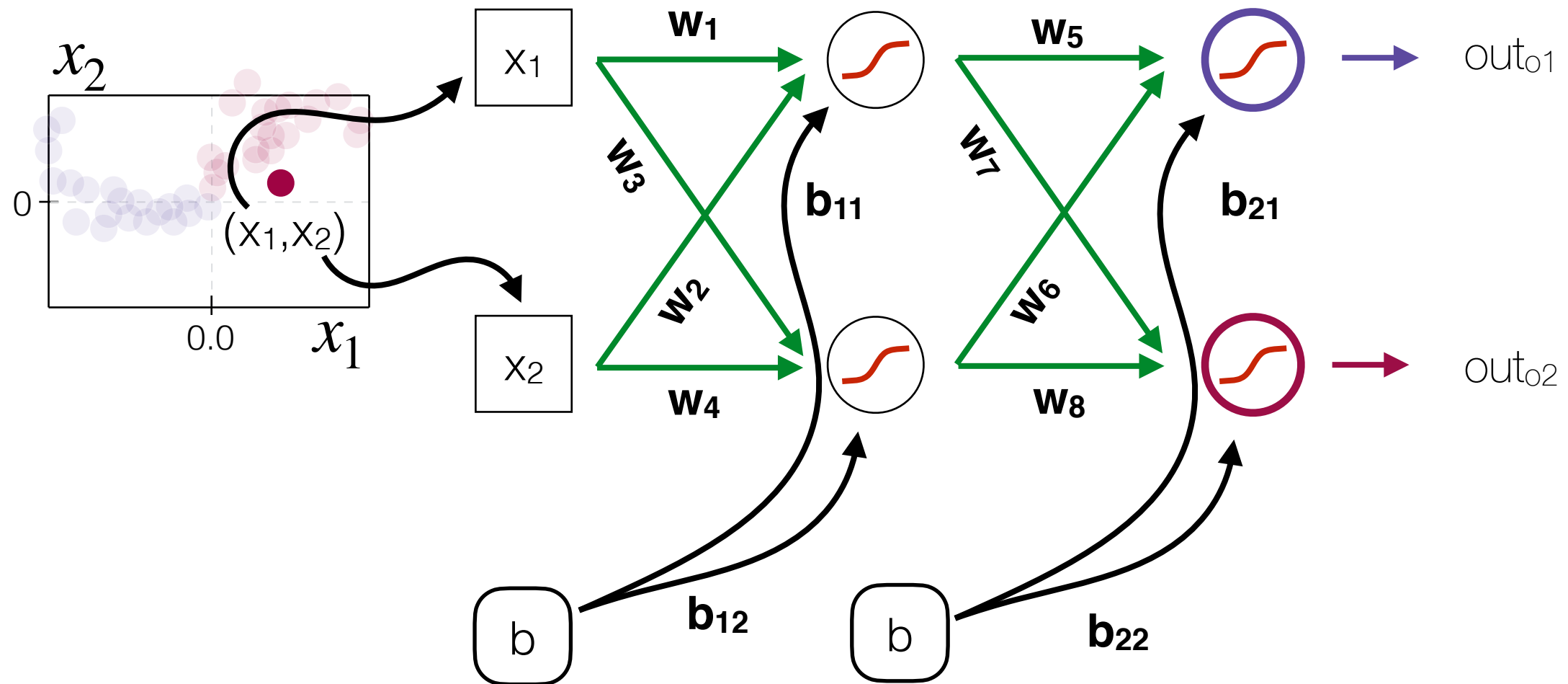


This particular architecture is **arbitrary** and meant as an **example**

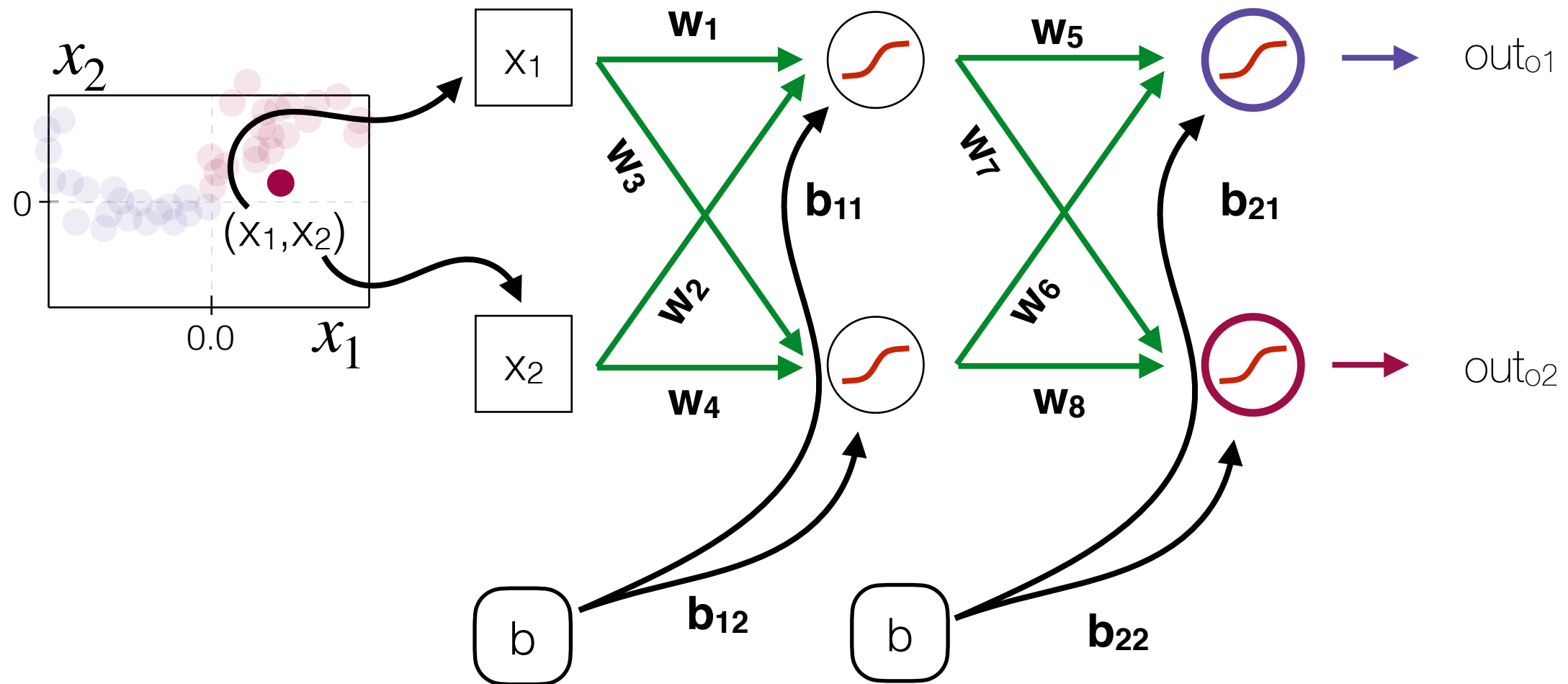


In practice, people use already existing architectures (e.g. **ResNet**, **BERT**, **U-Net** etc.)

When you create a new neural network, first you need to **initialise** weights and biases



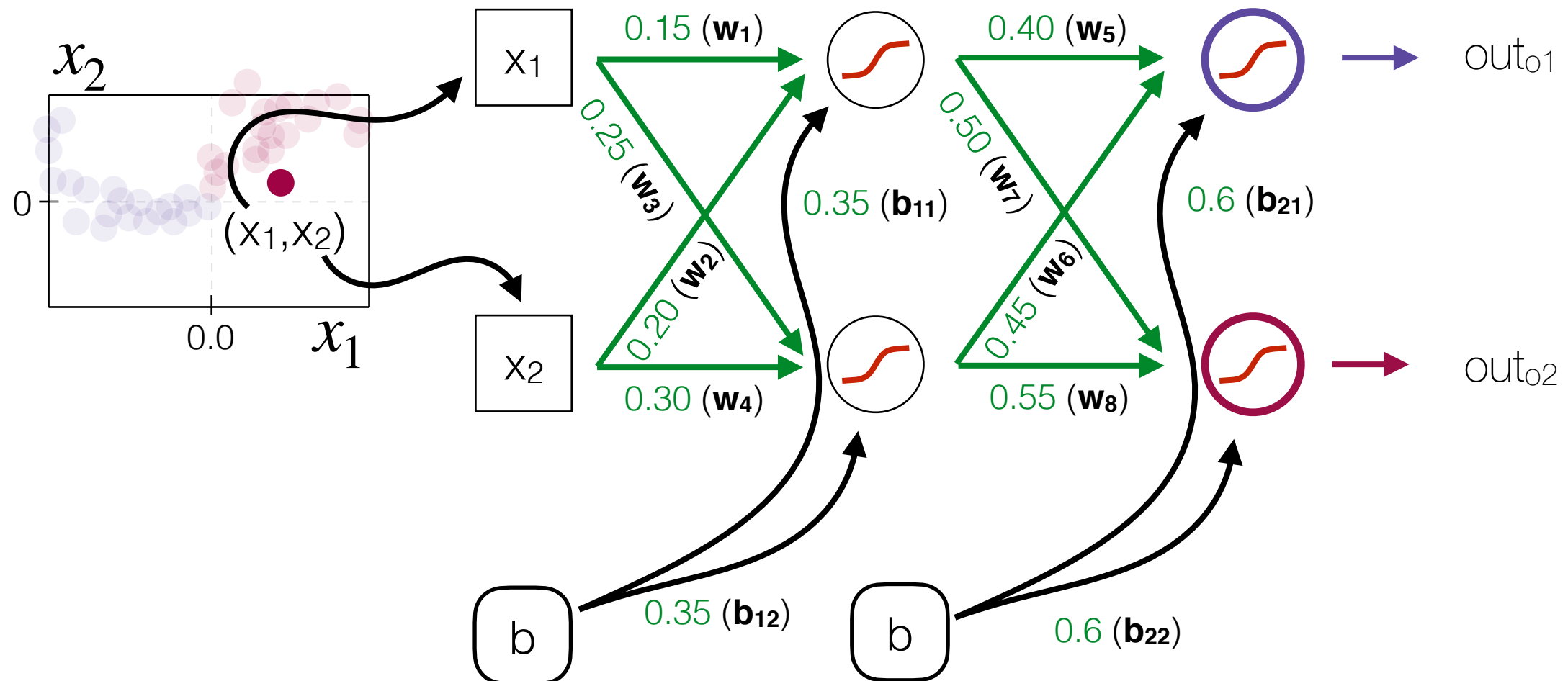
When you create a new neural network, first you need to **initialise** weights and biases



Sample **normal** distribution

```
import numpy as np
W = np.random.randn(2,2)
b = np.random.randn(1,2)
```

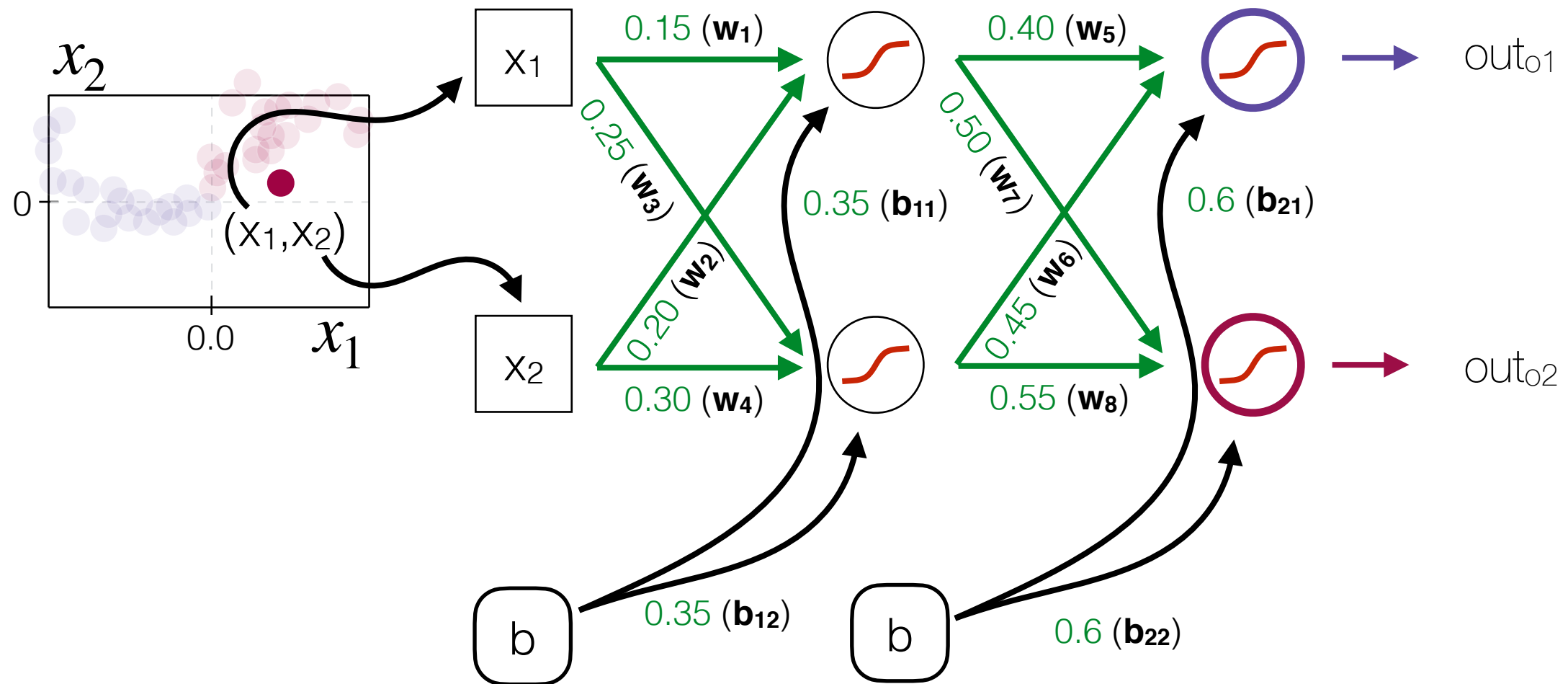
When you create a new neural network, first you need to **initialise** weights and biases



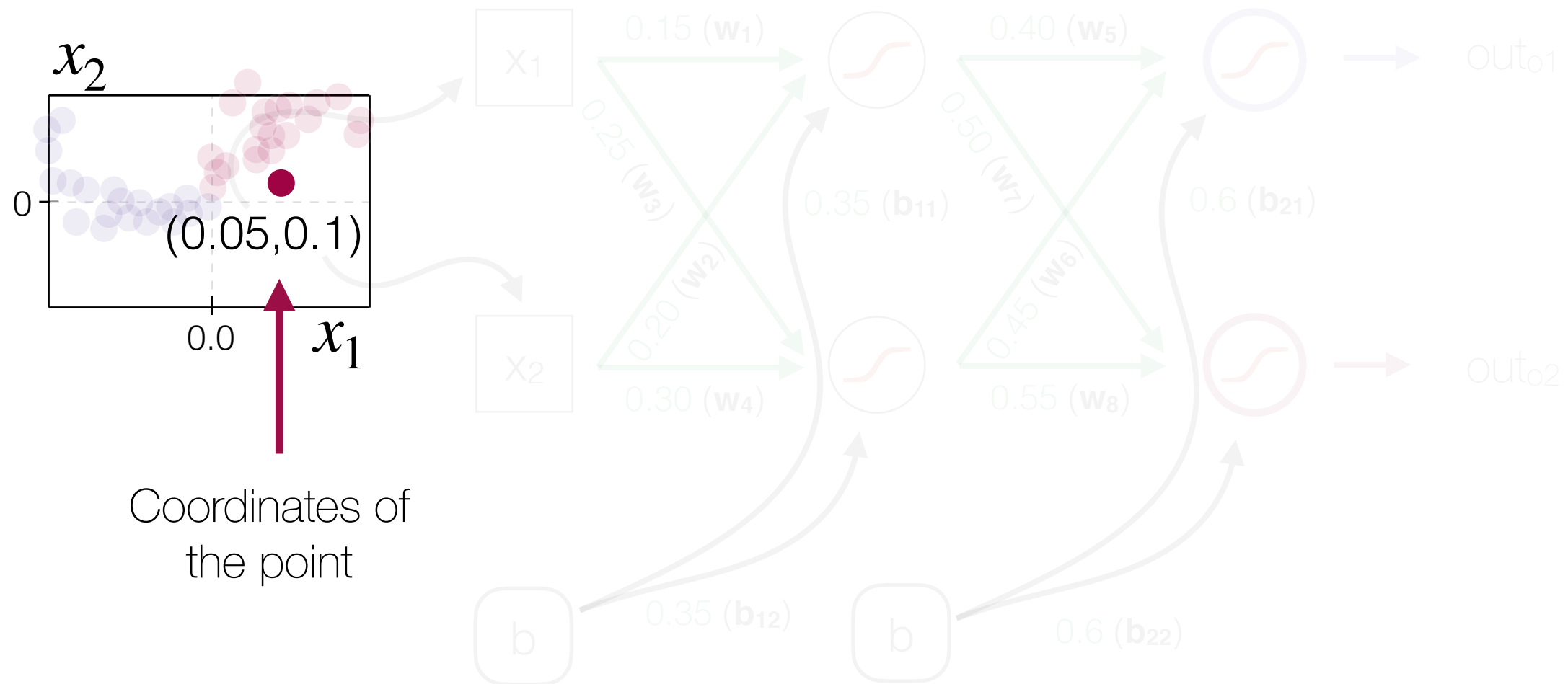
Sample **normal** distribution

```
import numpy as np
W = np.random.randn(2,2)
b = np.random.randn(1,2)
```

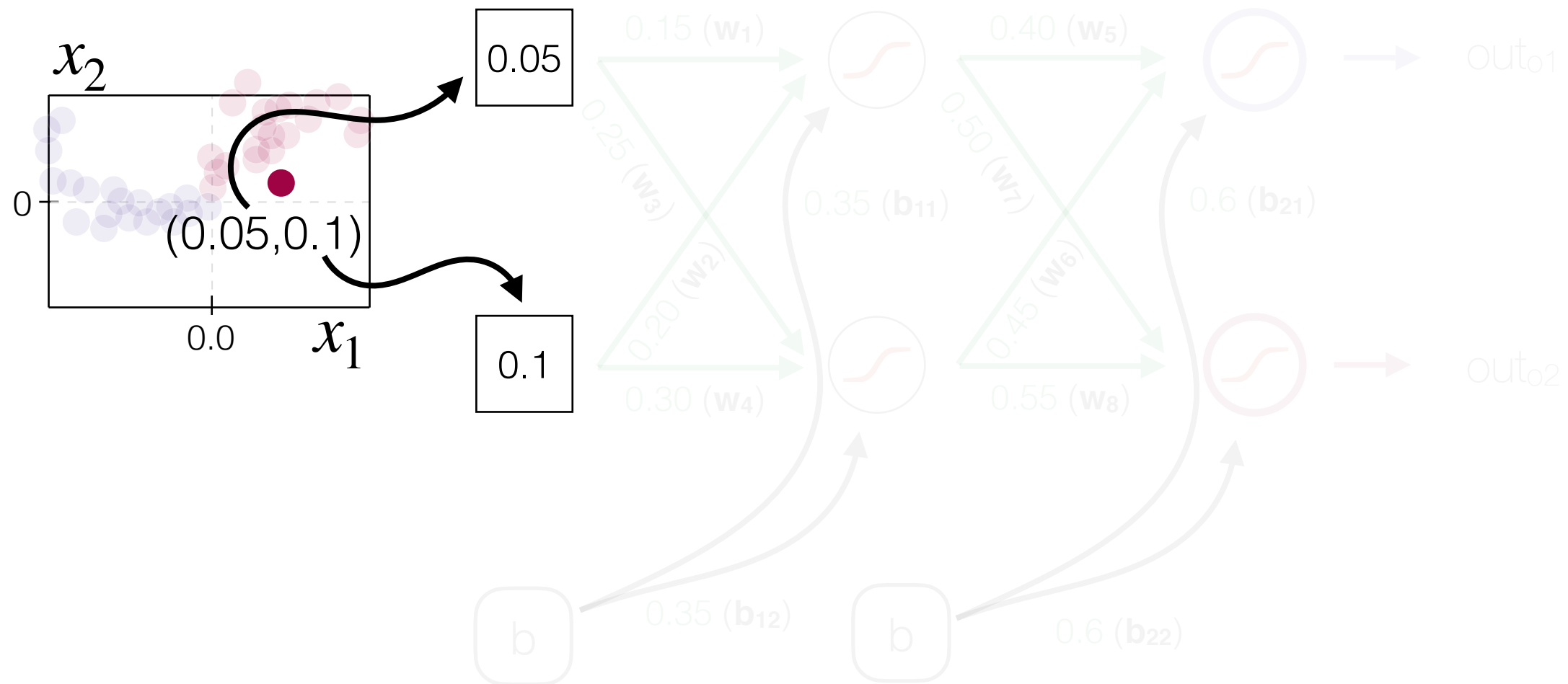
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



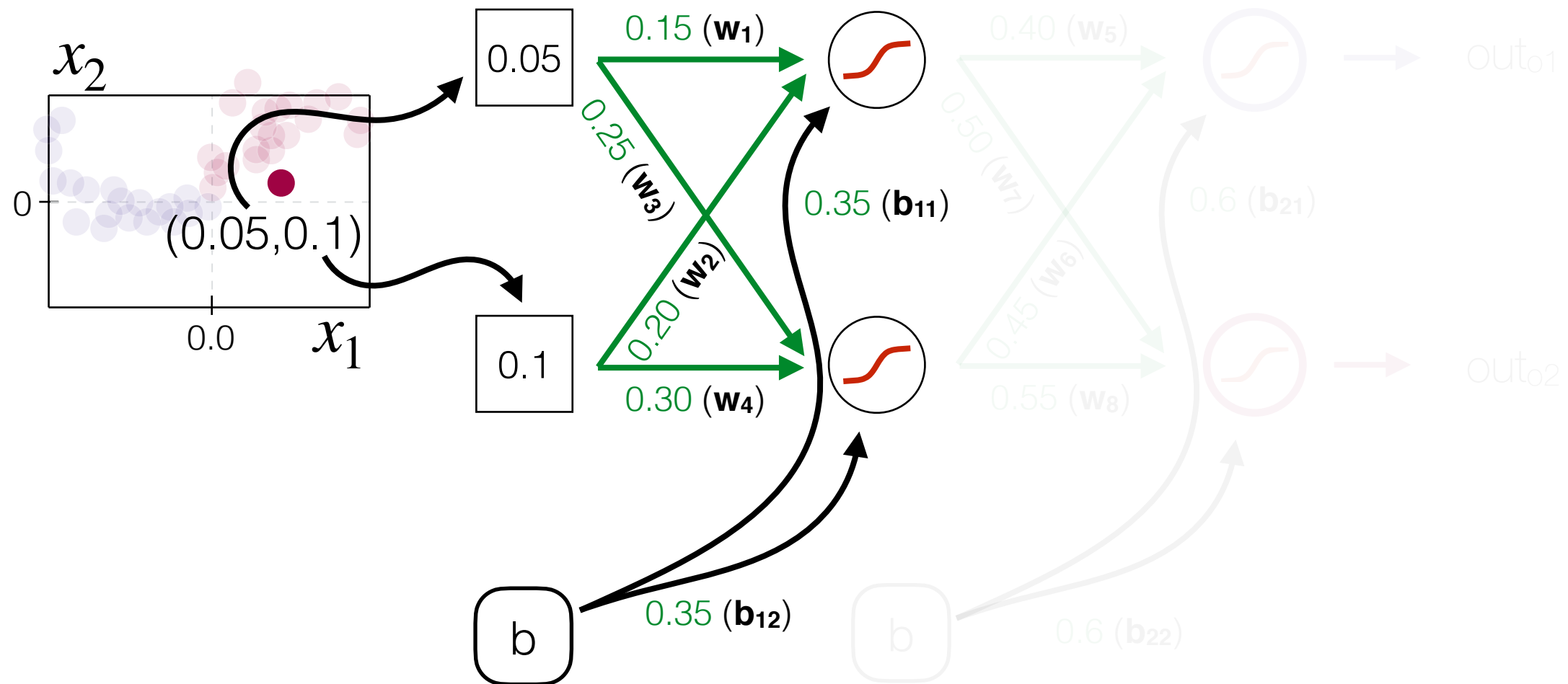
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



Let's calculate **class scores** for each class
using **feed-forward path** algorithm



Let's calculate **class scores** for each class
using **feed-forward path** algorithm

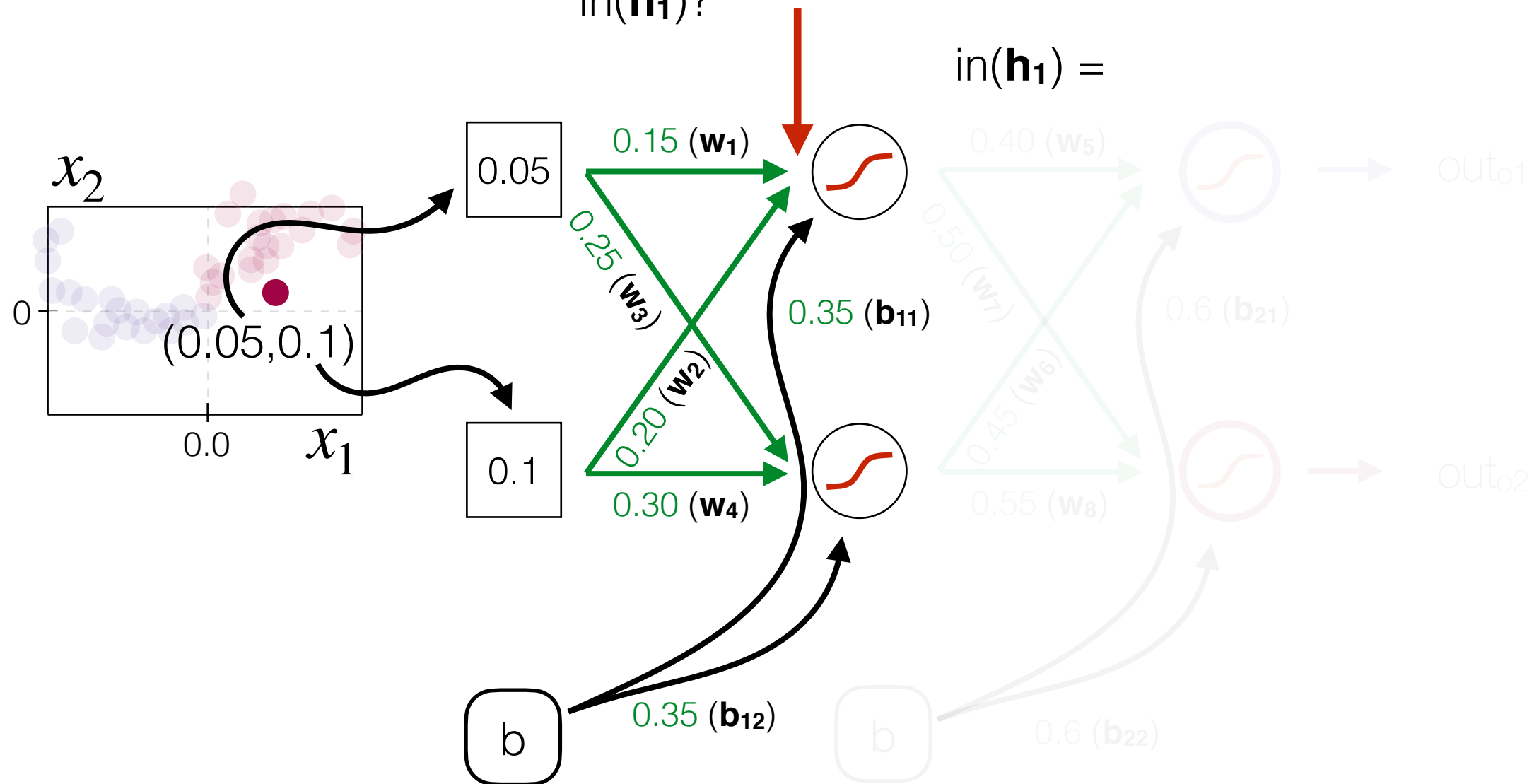


Let's calculate **class scores** for each class
using **feed forward path** algorithm

What is the **input** of the first
neuron of the hidden layer

$\text{in}(\mathbf{h}_1)$?

$\text{in}(\mathbf{h}_1) =$

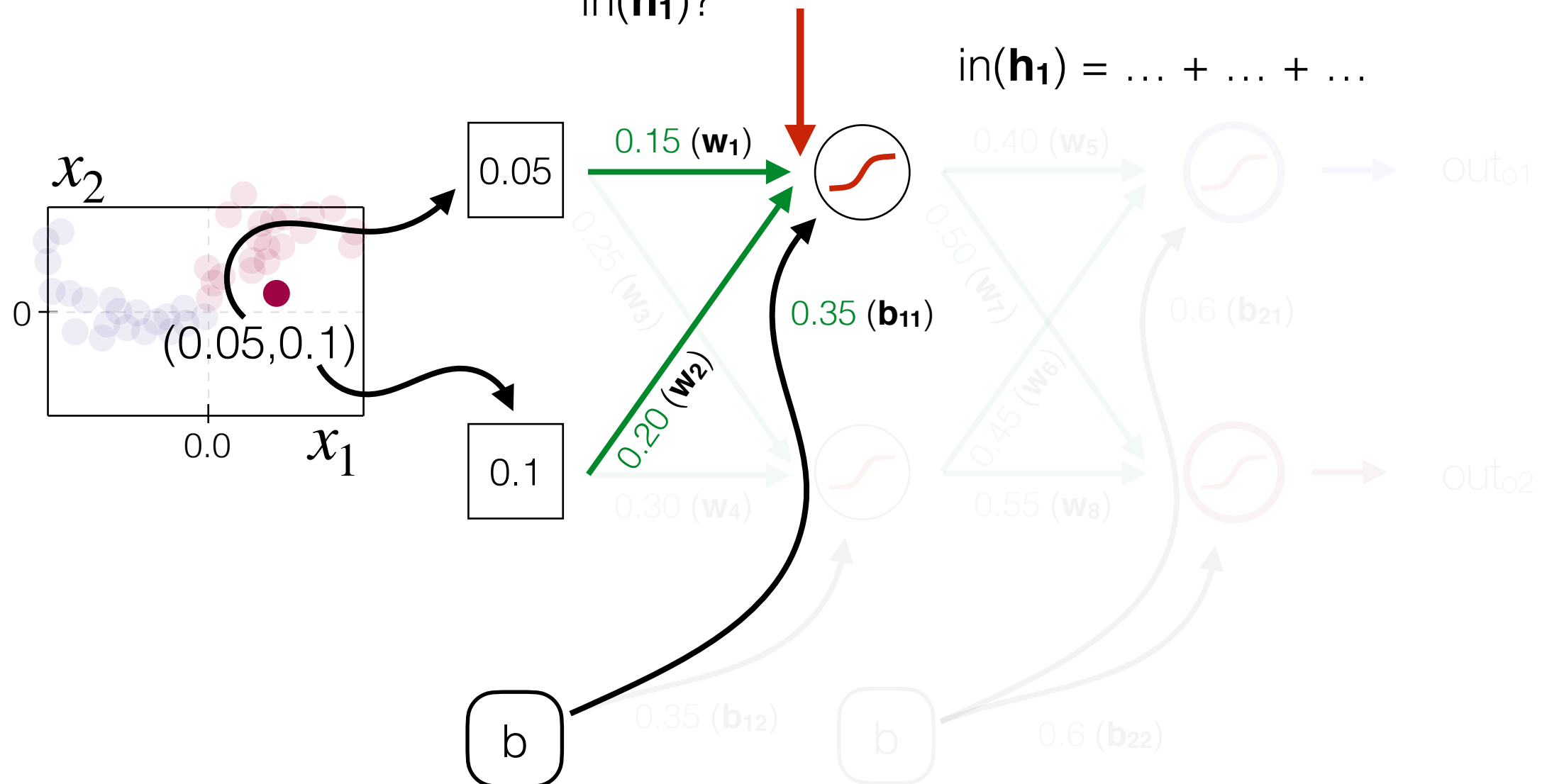


Let's calculate **class scores** for each class
using **feed forward path** algorithm

What is the **input** of the first
neuron of the hidden layer

$\text{in}(\mathbf{h}_1)$?

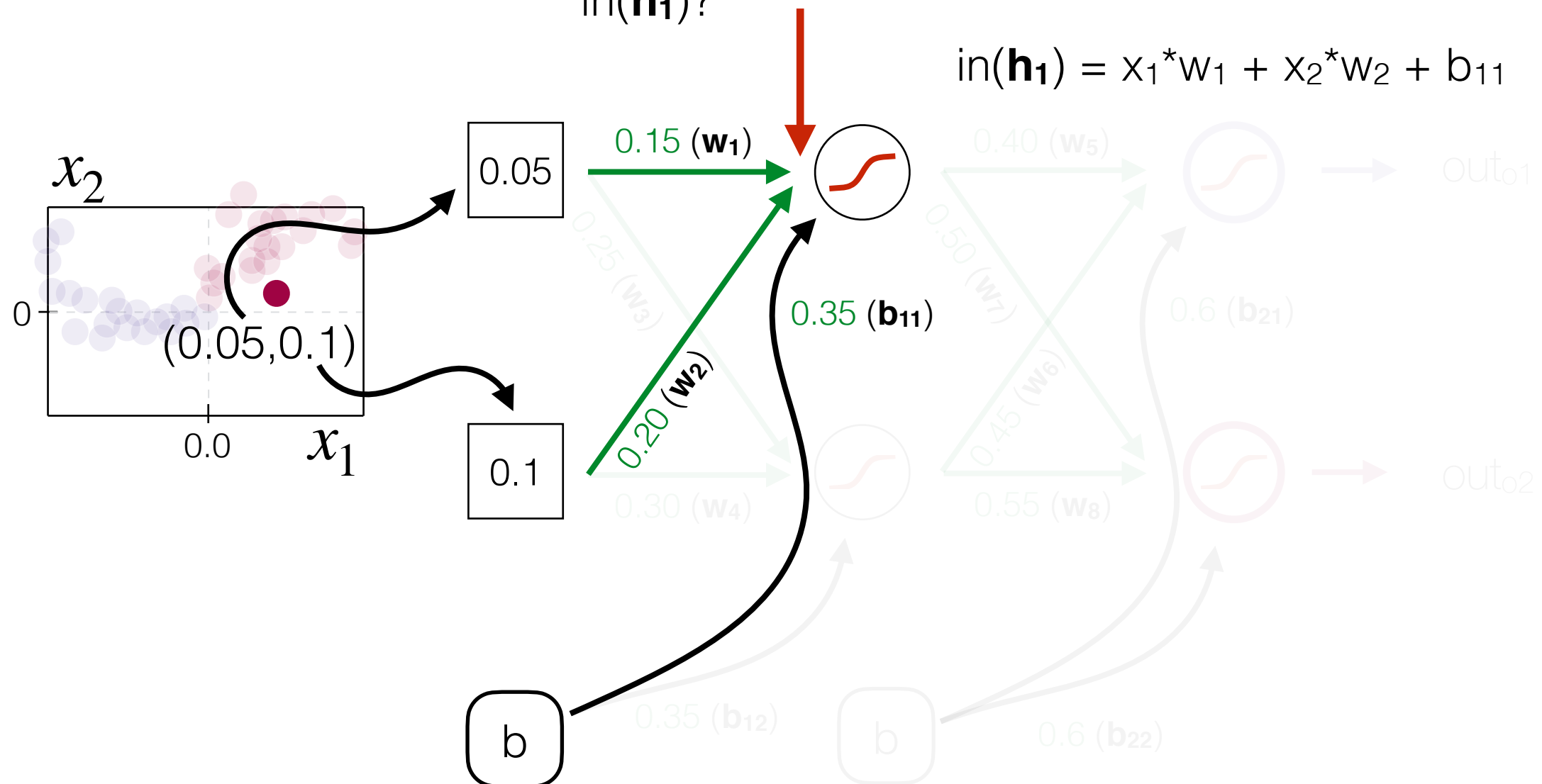
$$\text{in}(\mathbf{h}_1) = \dots + \dots + \dots$$



Let's calculate **class scores** for each class using **feed forward path** algorithm

What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

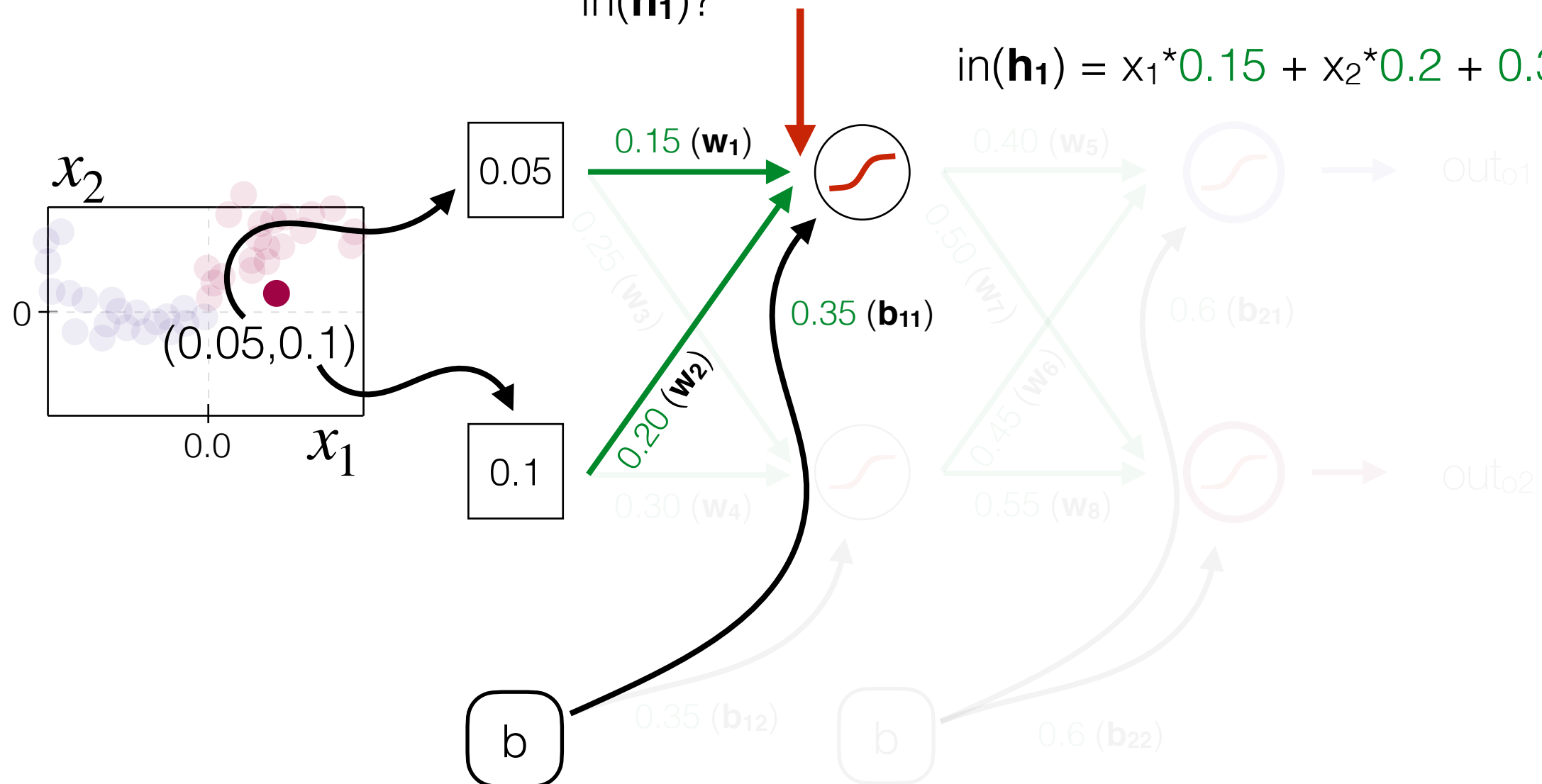
$$\text{in}(\mathbf{h}_1) = x_1 * w_1 + x_2 * w_2 + b_{11}$$



Let's calculate **class scores** for each class
using **feed forward path** algorithm

What is the **input** of the first
neuron of the hidden layer
 $\text{in}(\mathbf{h}_1)$?

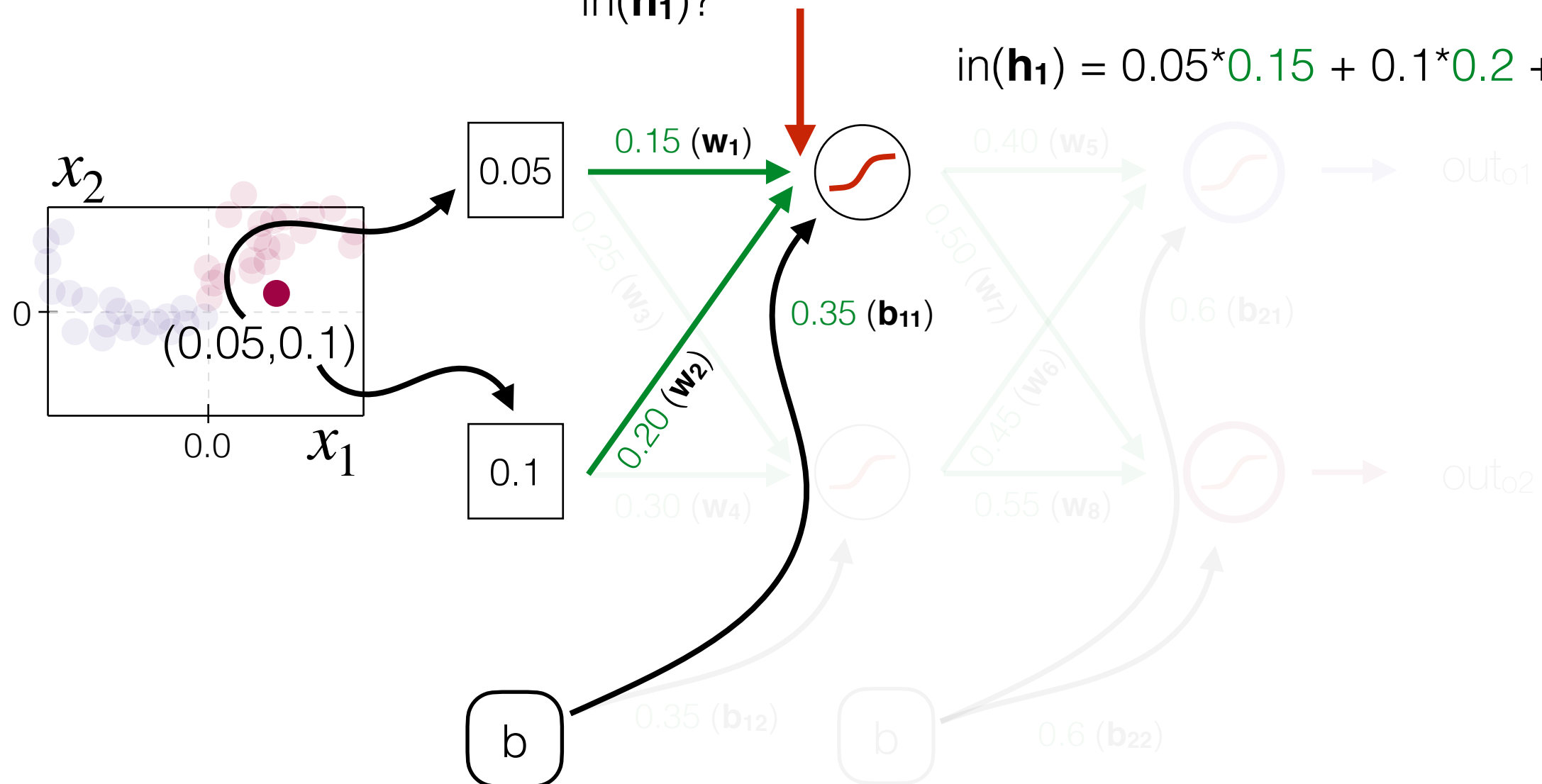
$$\text{in}(\mathbf{h}_1) = x_1 * 0.15 + x_2 * 0.2 + 0.35$$



Let's calculate **class scores** for each class
using **feed forward path** algorithm

What is the **input** of the first
neuron of the hidden layer
 $\text{in}(\mathbf{h}_1)$?

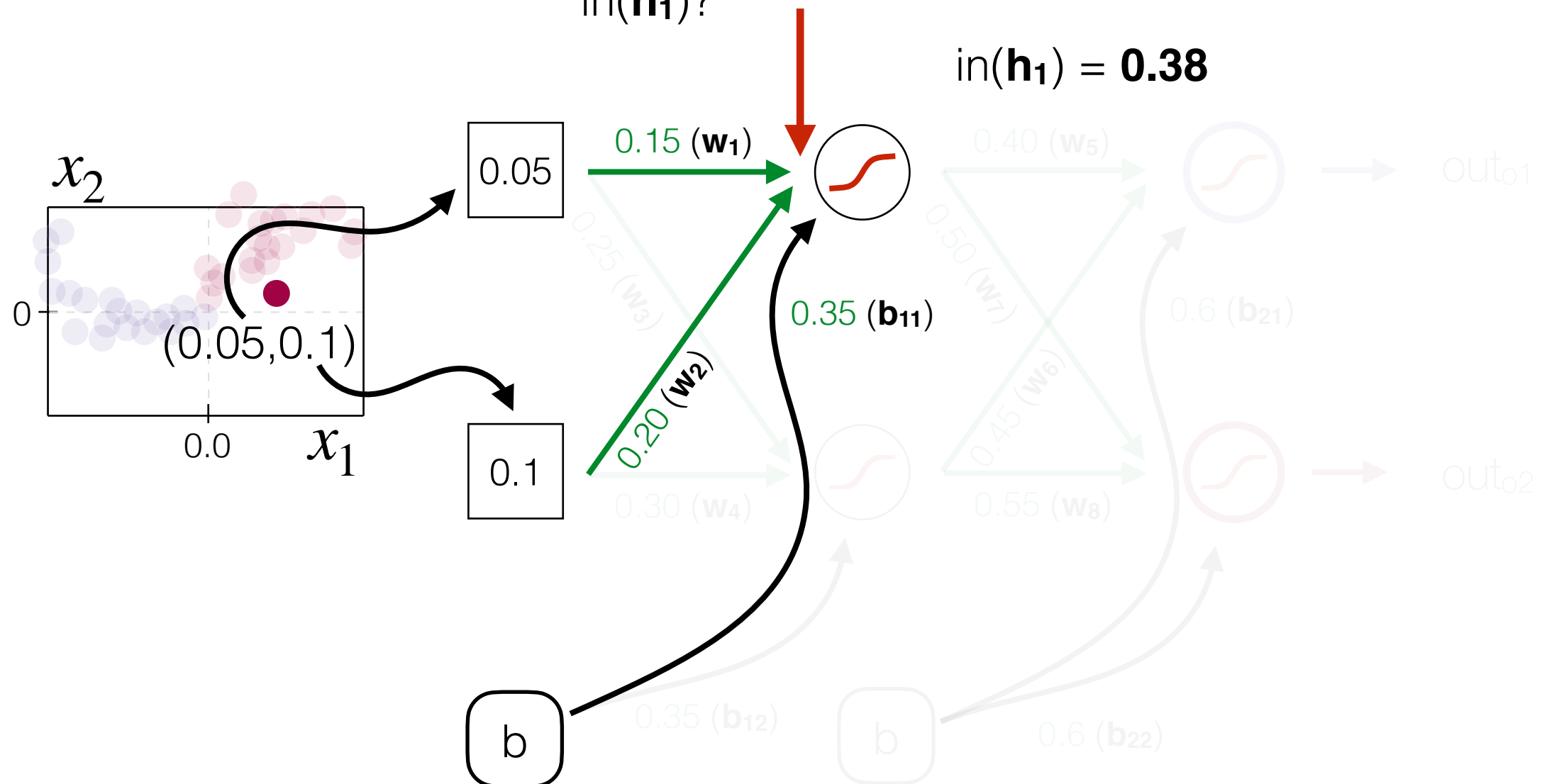
$$\text{in}(\mathbf{h}_1) = 0.05 * 0.15 + 0.1 * 0.2 + 0.35$$



Let's calculate **class scores** for each class using **feed forward path** algorithm

What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

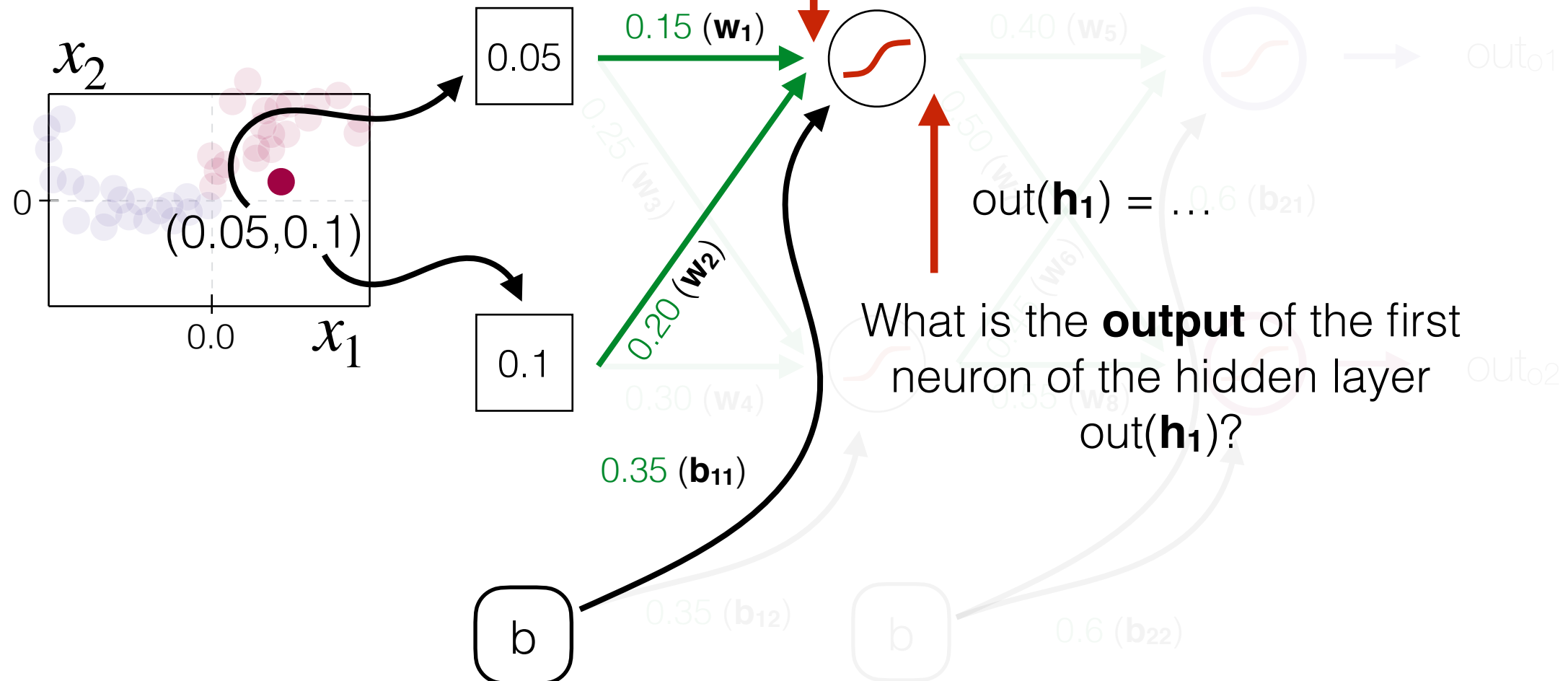
$$\text{in}(\mathbf{h}_1) = \mathbf{0.38}$$



Let's calculate **class scores** for each class using **feed forward path** algorithm

What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

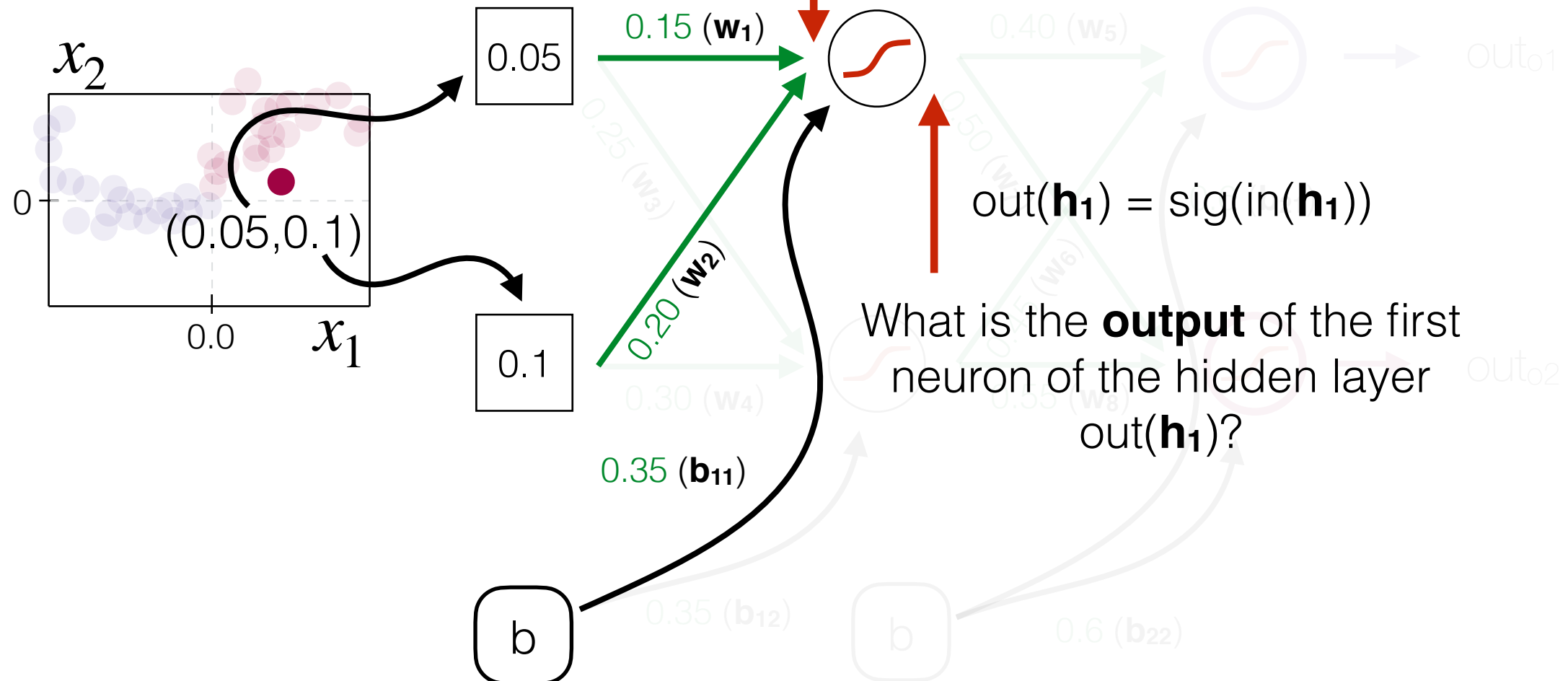
$$\text{in}(\mathbf{h}_1) = \mathbf{0.38}$$



Let's calculate **class scores** for each class using **feed forward path** algorithm

What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

$$\text{in}(\mathbf{h}_1) = \mathbf{0.38}$$



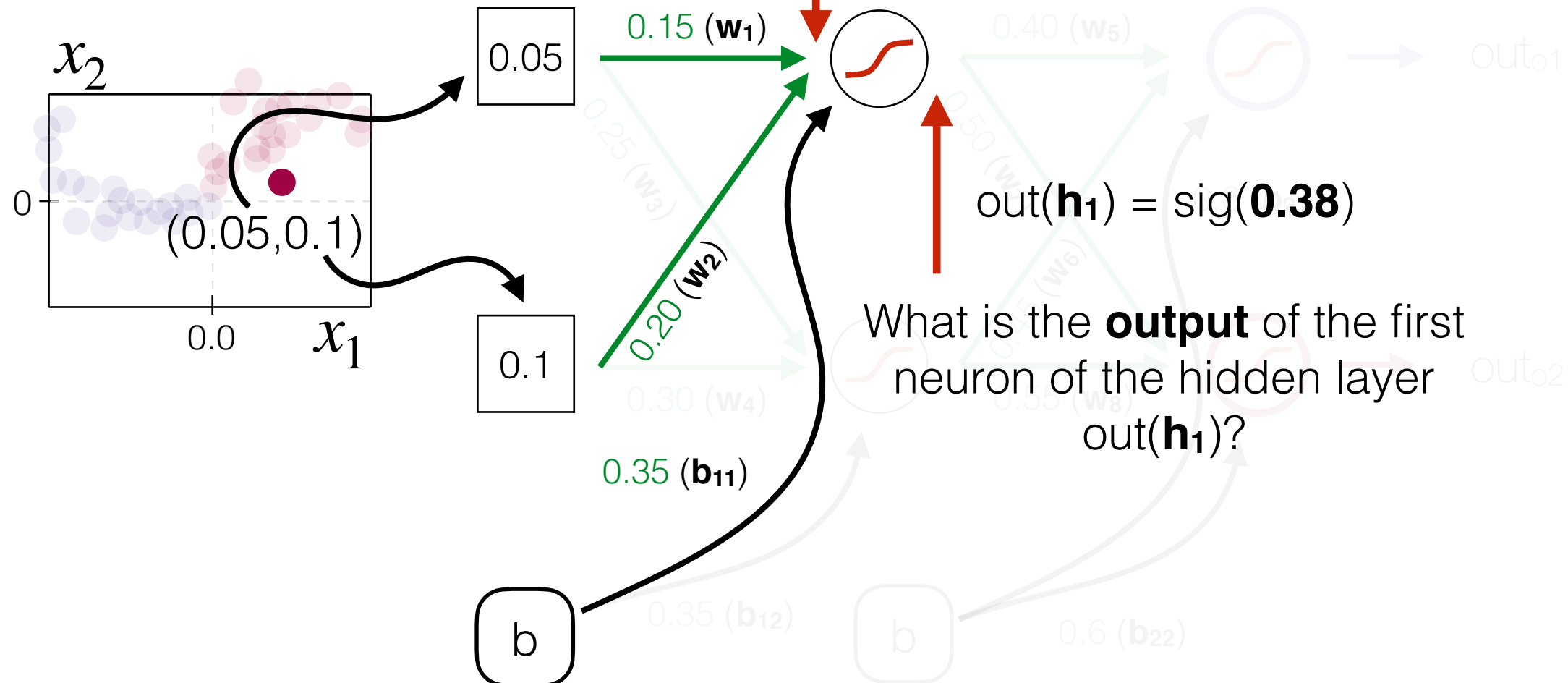
Let's calculate **class scores** for each class using **feed forward path** algorithm

What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

$$\text{in}(\mathbf{h}_1) = \mathbf{0.38}$$

$$\text{out}(\mathbf{h}_1) = \text{sig}(\mathbf{0.38})$$

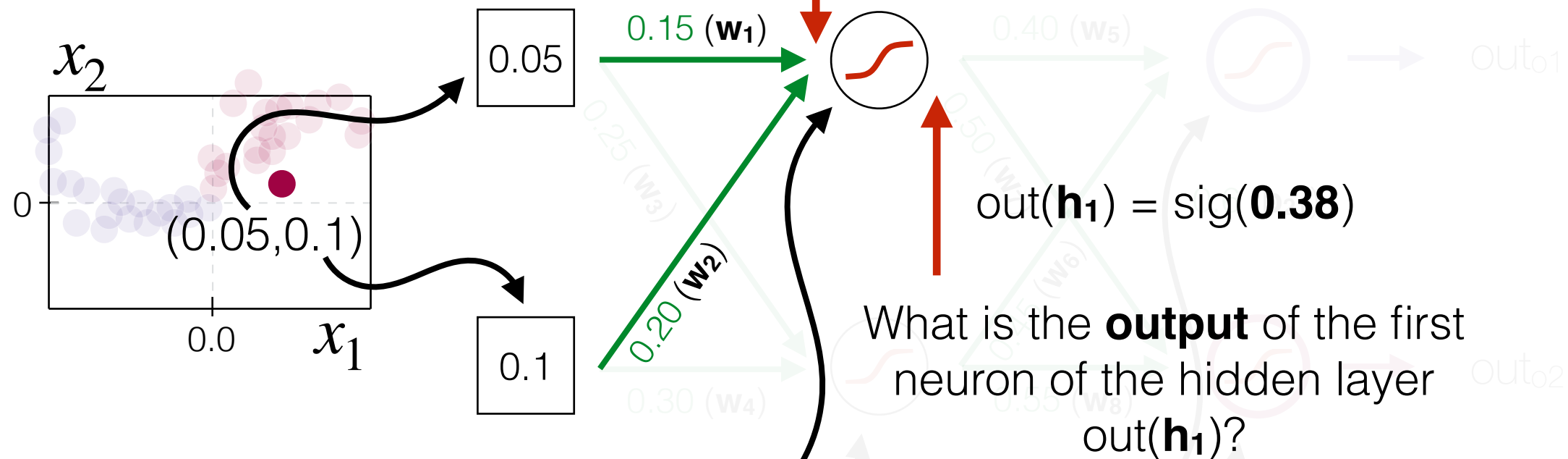
What is the **output** of the first neuron of the hidden layer $\text{out}(\mathbf{h}_1)$?



Let's calculate **class scores** for each class using **feed forward path** algorithm

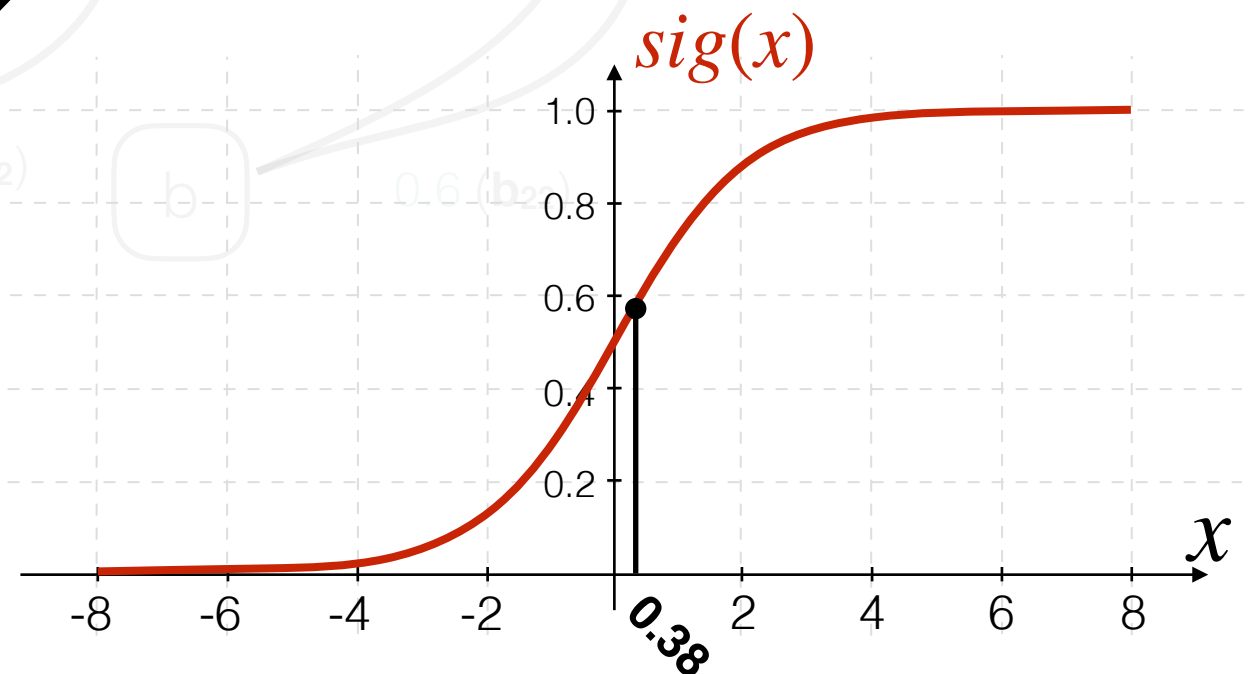
What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?

$$\text{in}(\mathbf{h}_1) = \mathbf{0.38}$$



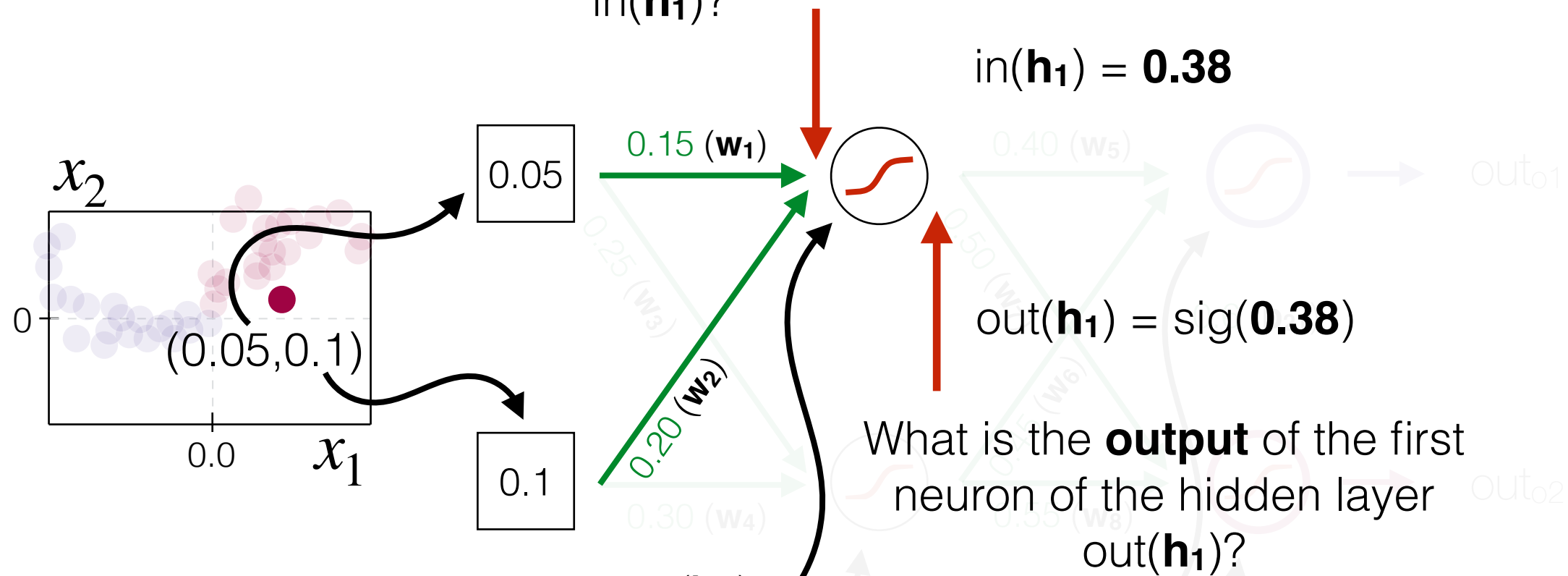
What is the **output** of the first neuron of the hidden layer $\text{out}(\mathbf{h}_1)$?

b



Let's calculate **class scores** for each class using **feed forward path** algorithm

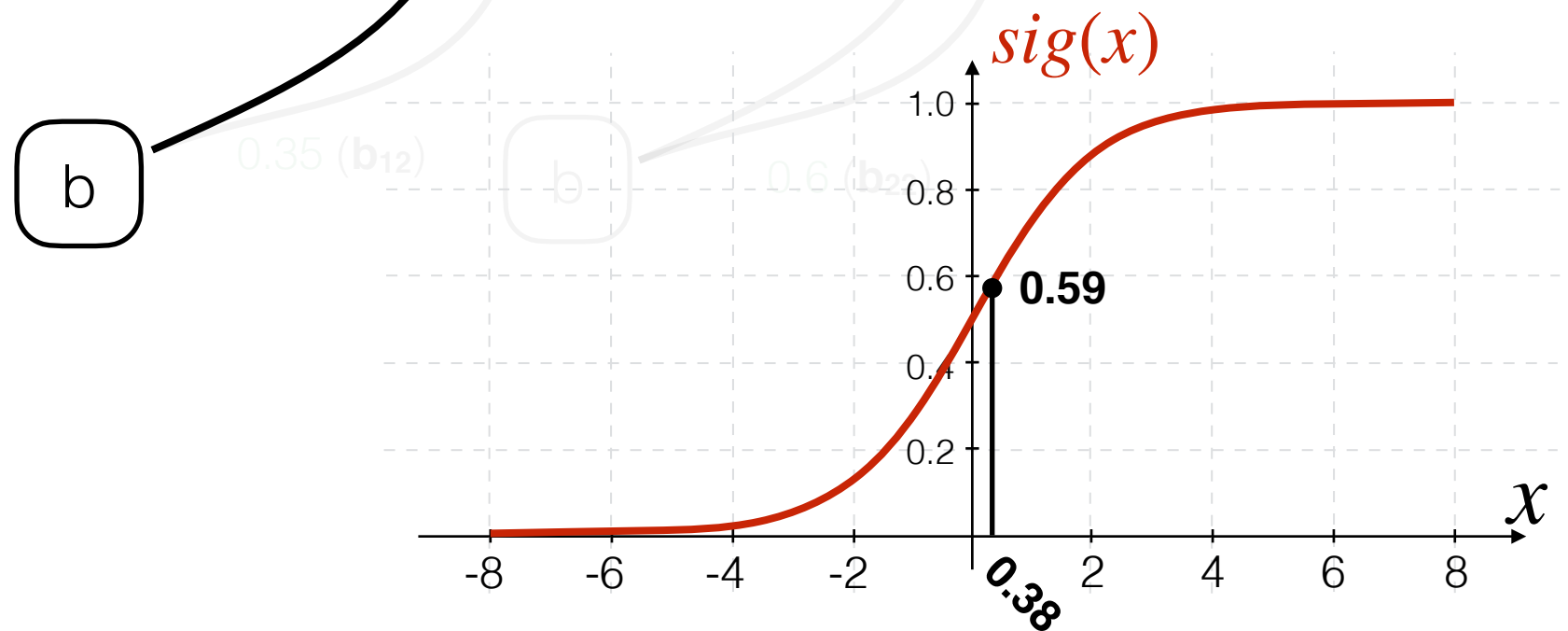
What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?



$$\text{in}(\mathbf{h}_1) = 0.38$$

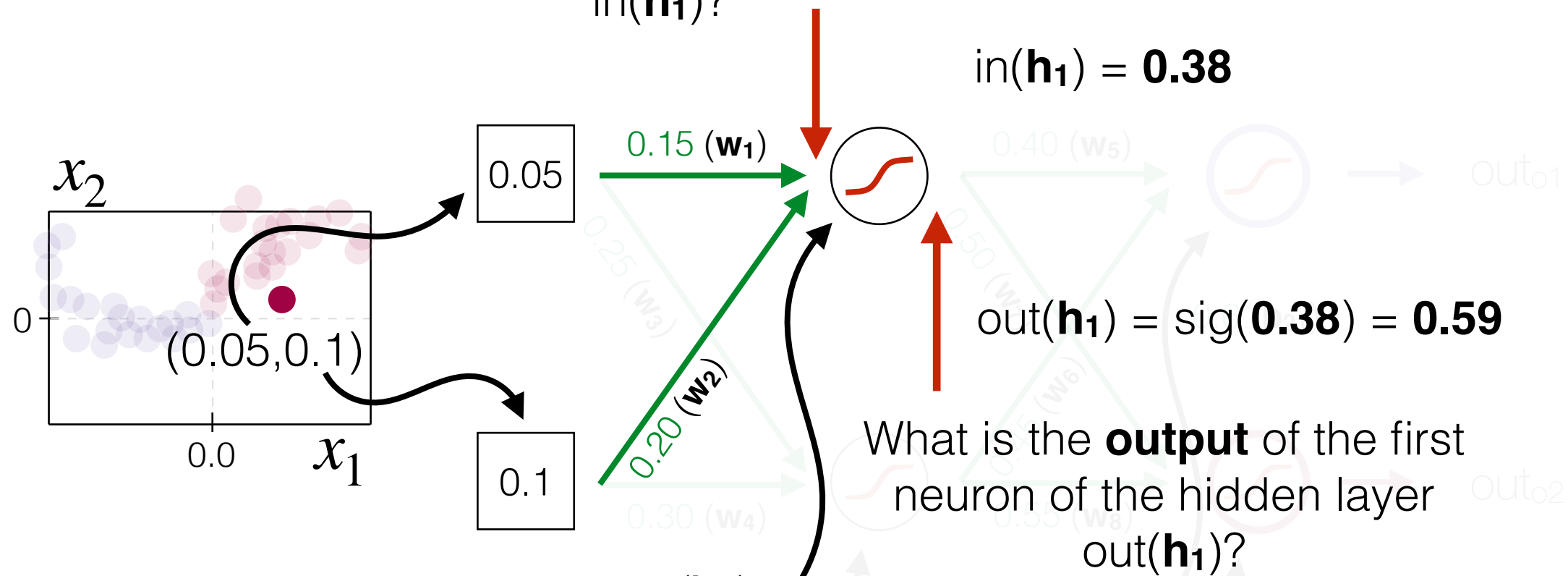
$$\text{out}(\mathbf{h}_1) = \text{sig}(0.38)$$

What is the **output** of the first neuron of the hidden layer $\text{out}(\mathbf{h}_1)$?

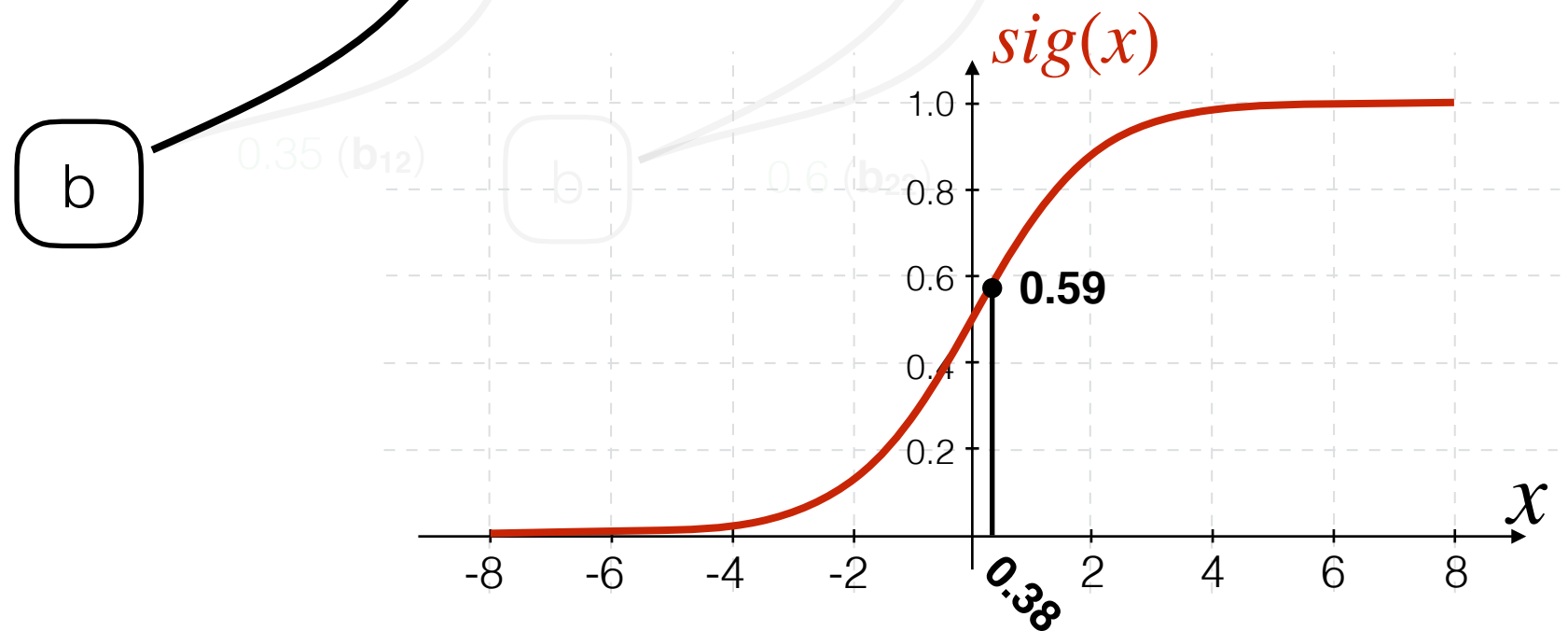


Let's calculate **class scores** for each class using **feed forward path** algorithm

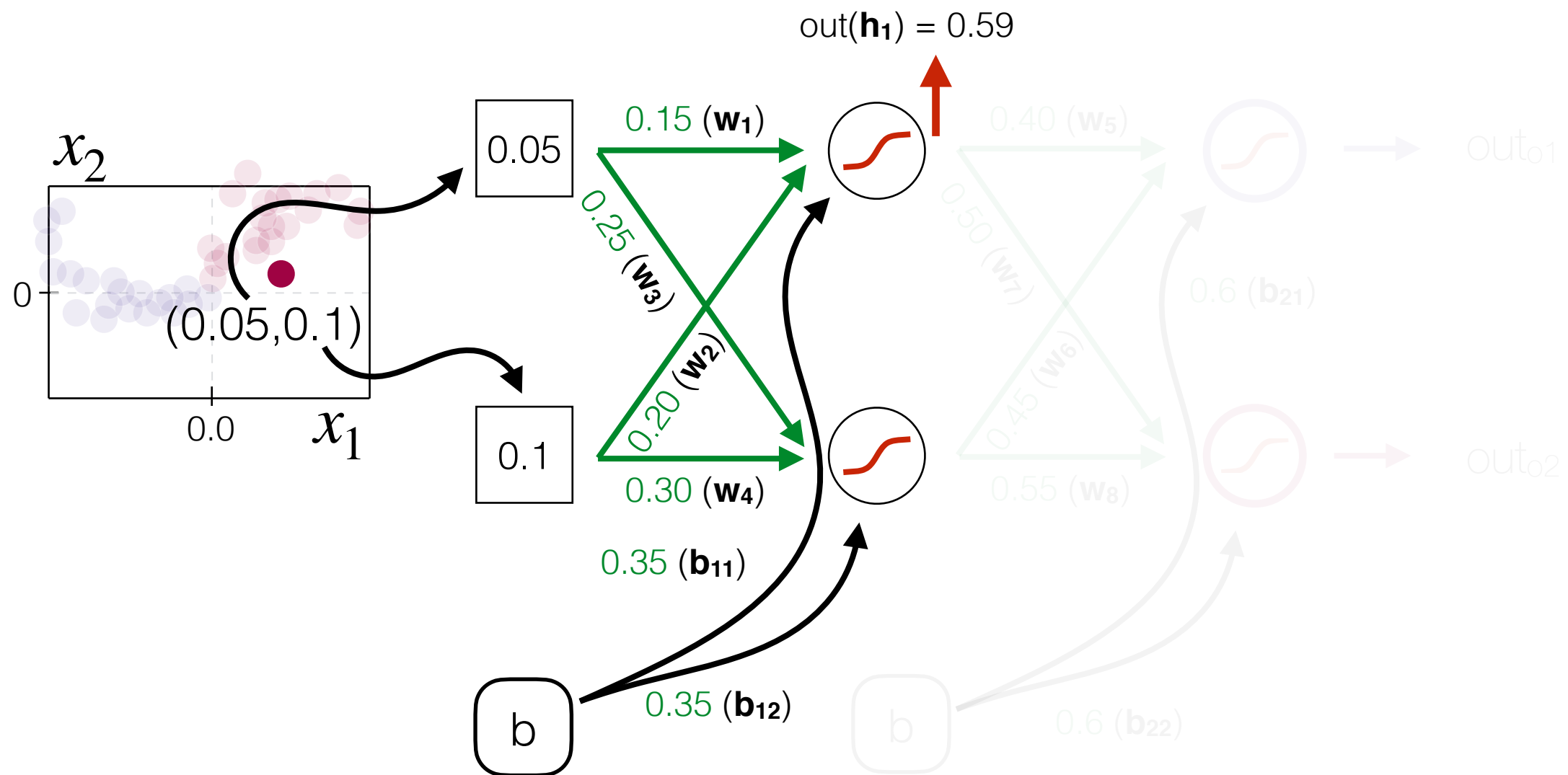
What is the **input** of the first neuron of the hidden layer $\text{in}(\mathbf{h}_1)$?



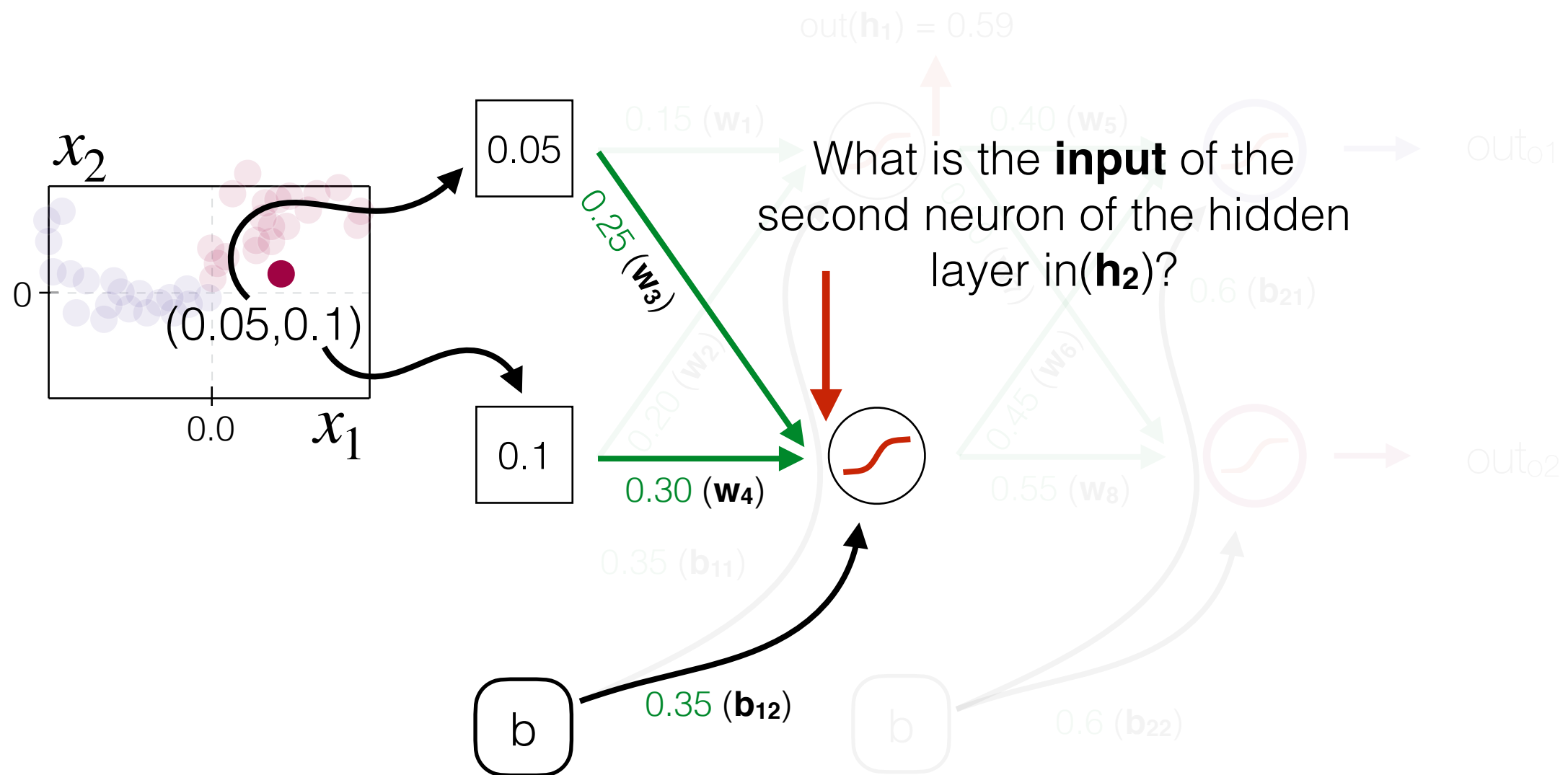
What is the **output** of the first neuron of the hidden layer $\text{out}(\mathbf{h}_1)$?



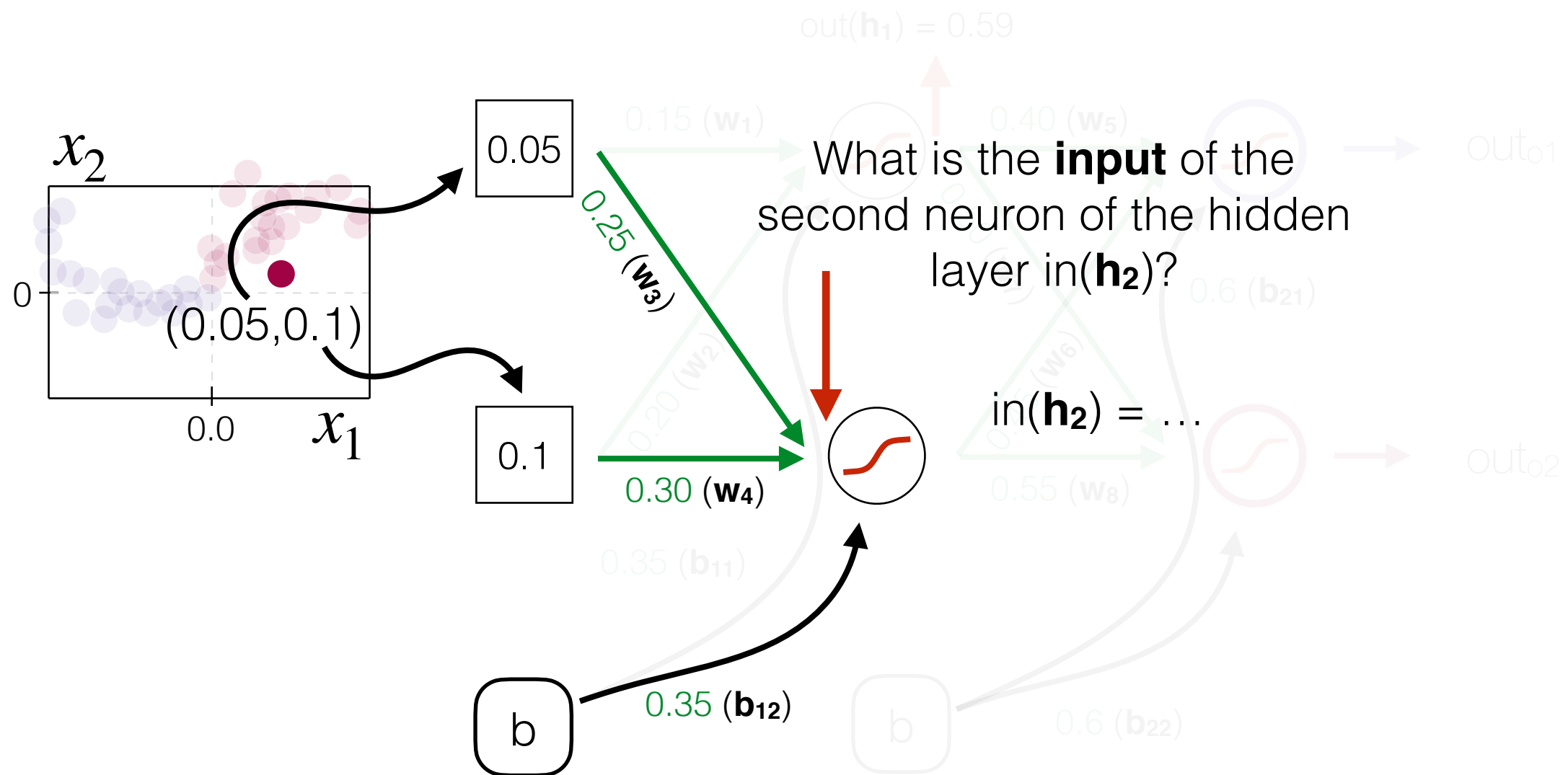
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



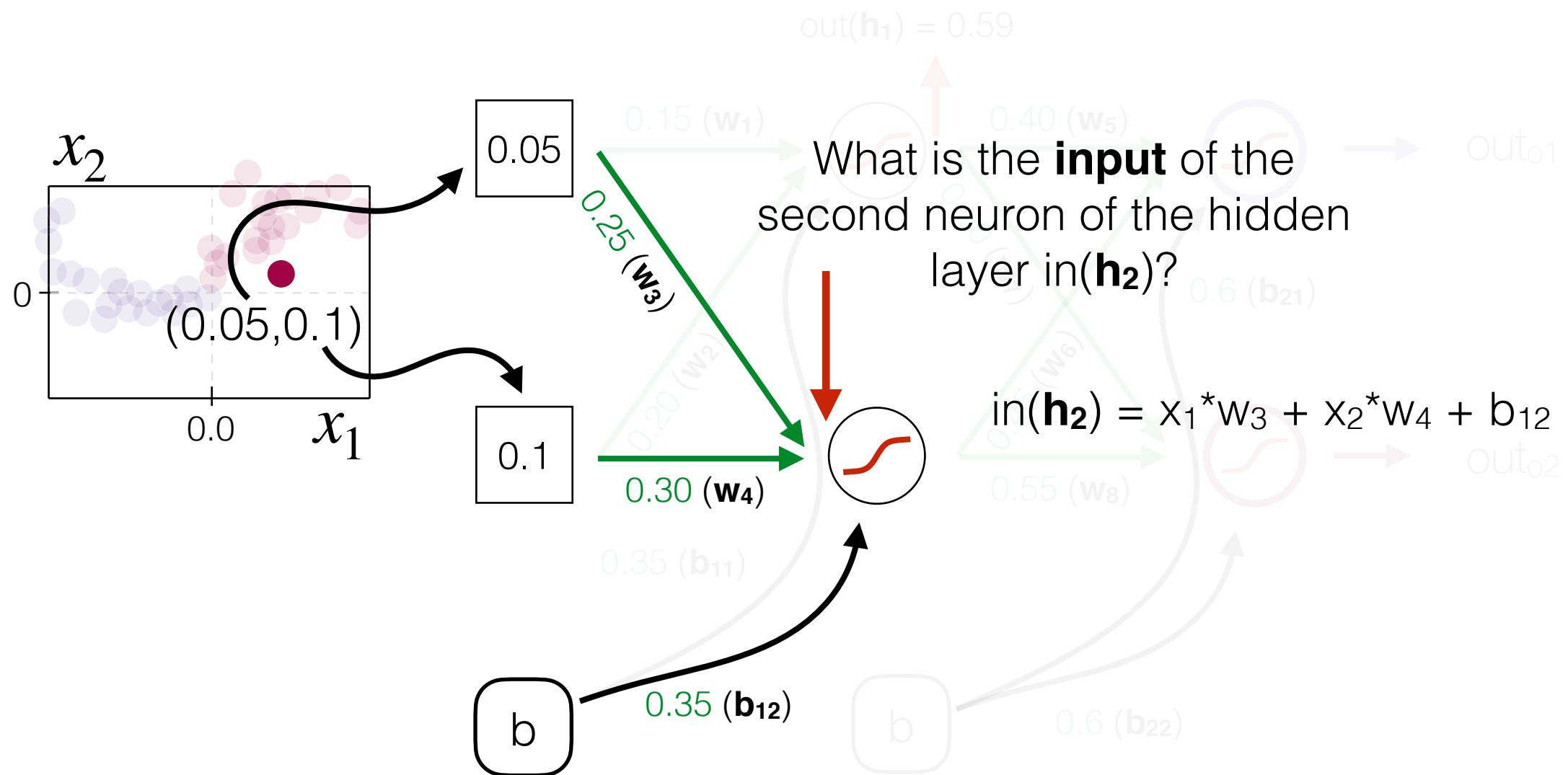
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



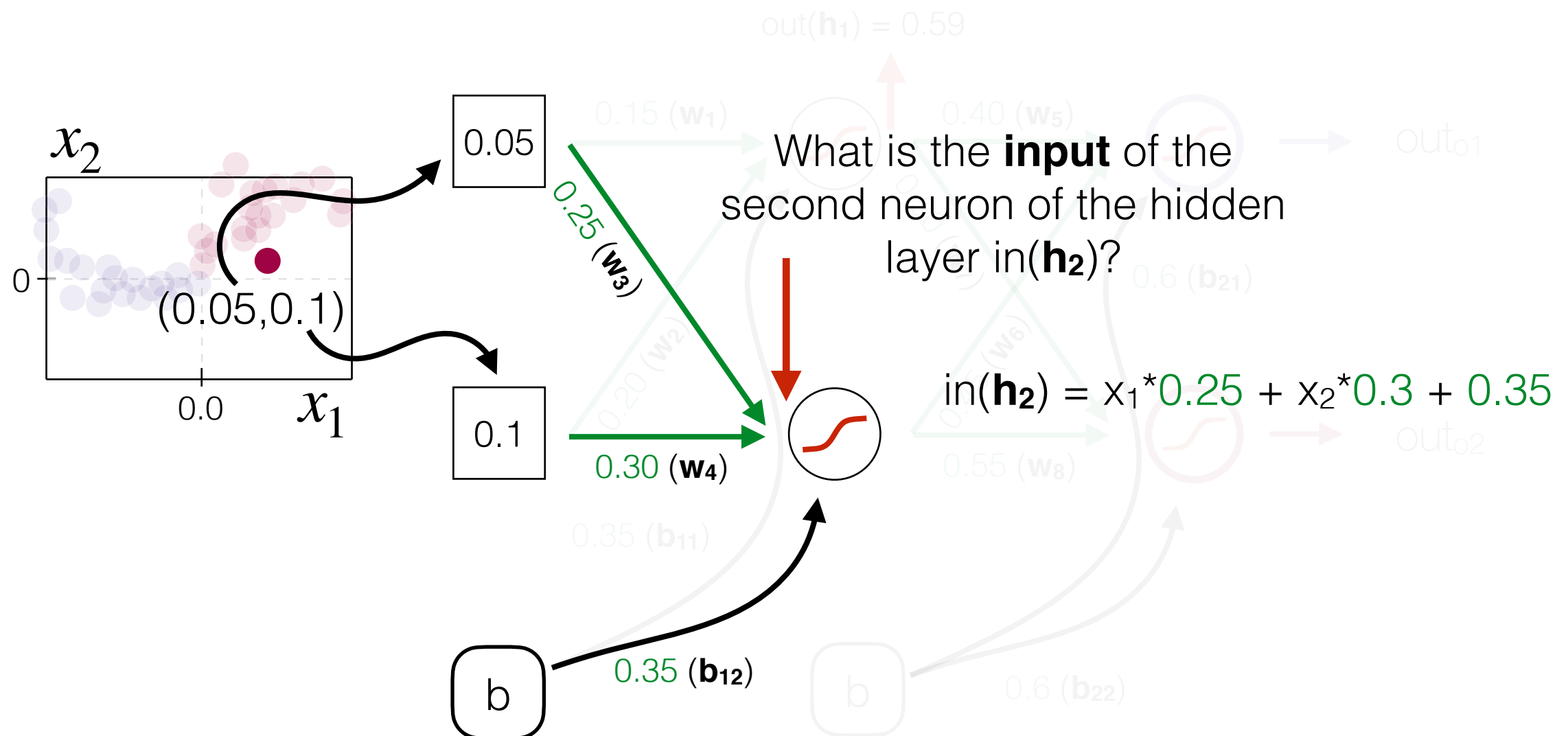
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



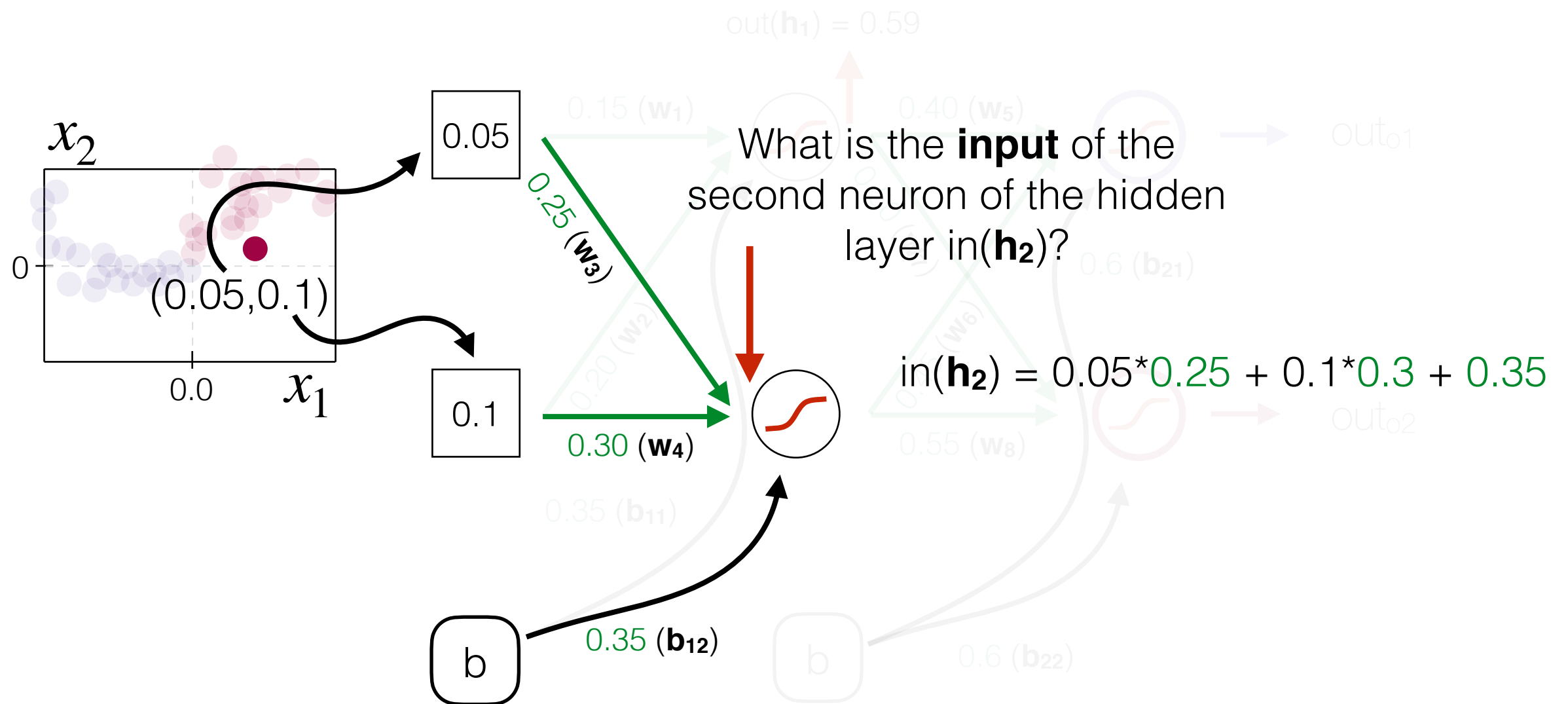
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



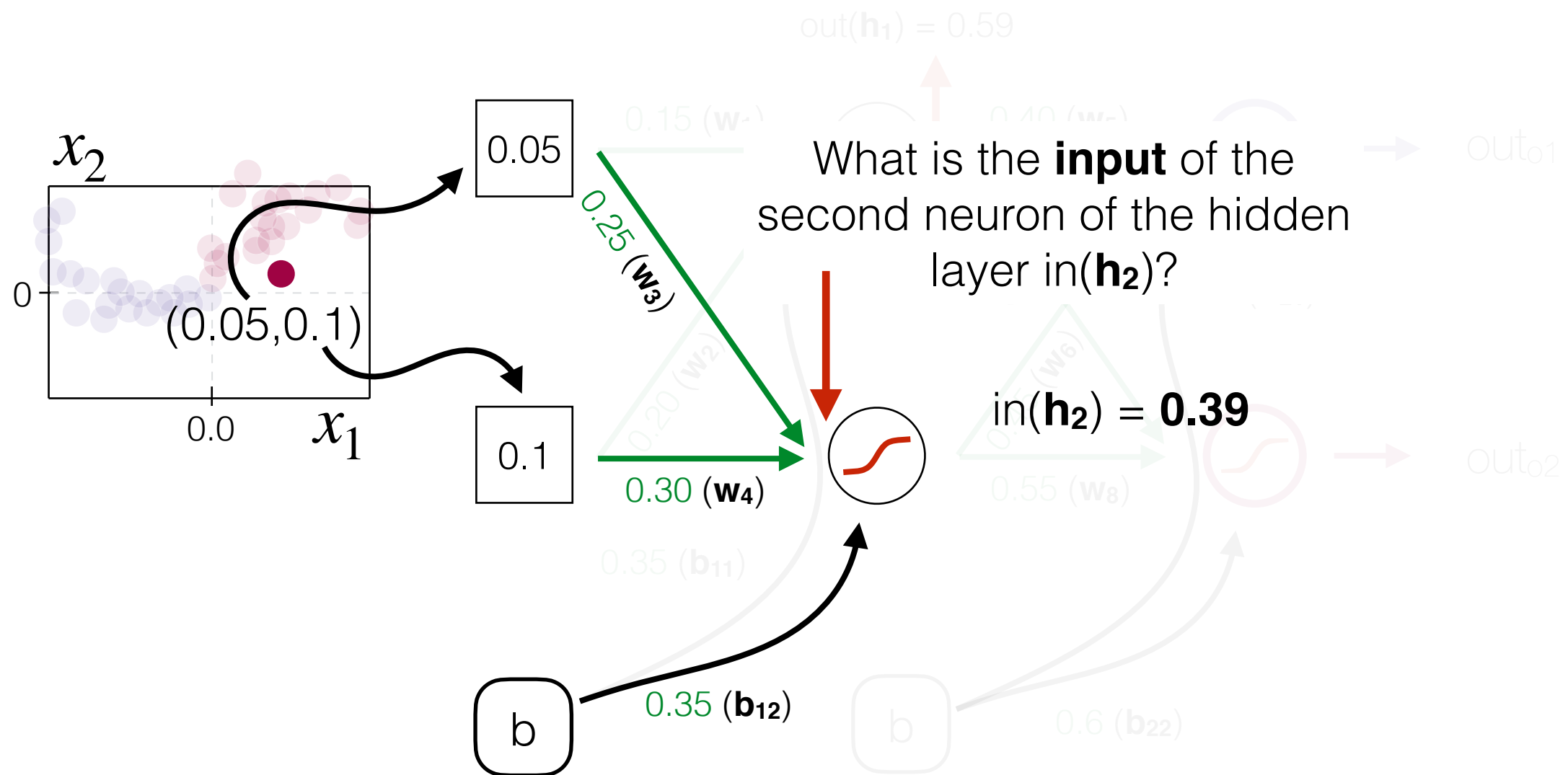
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



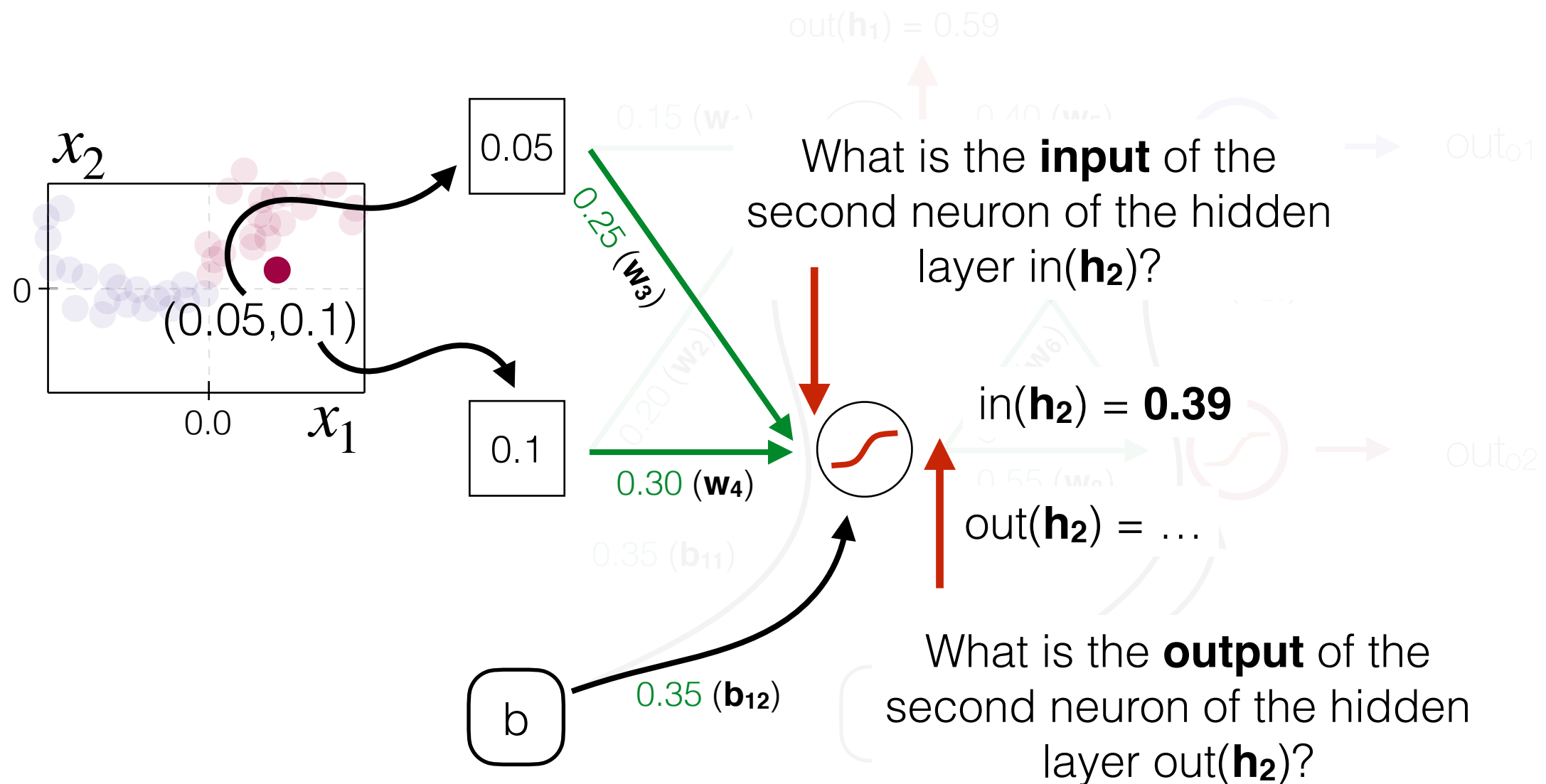
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



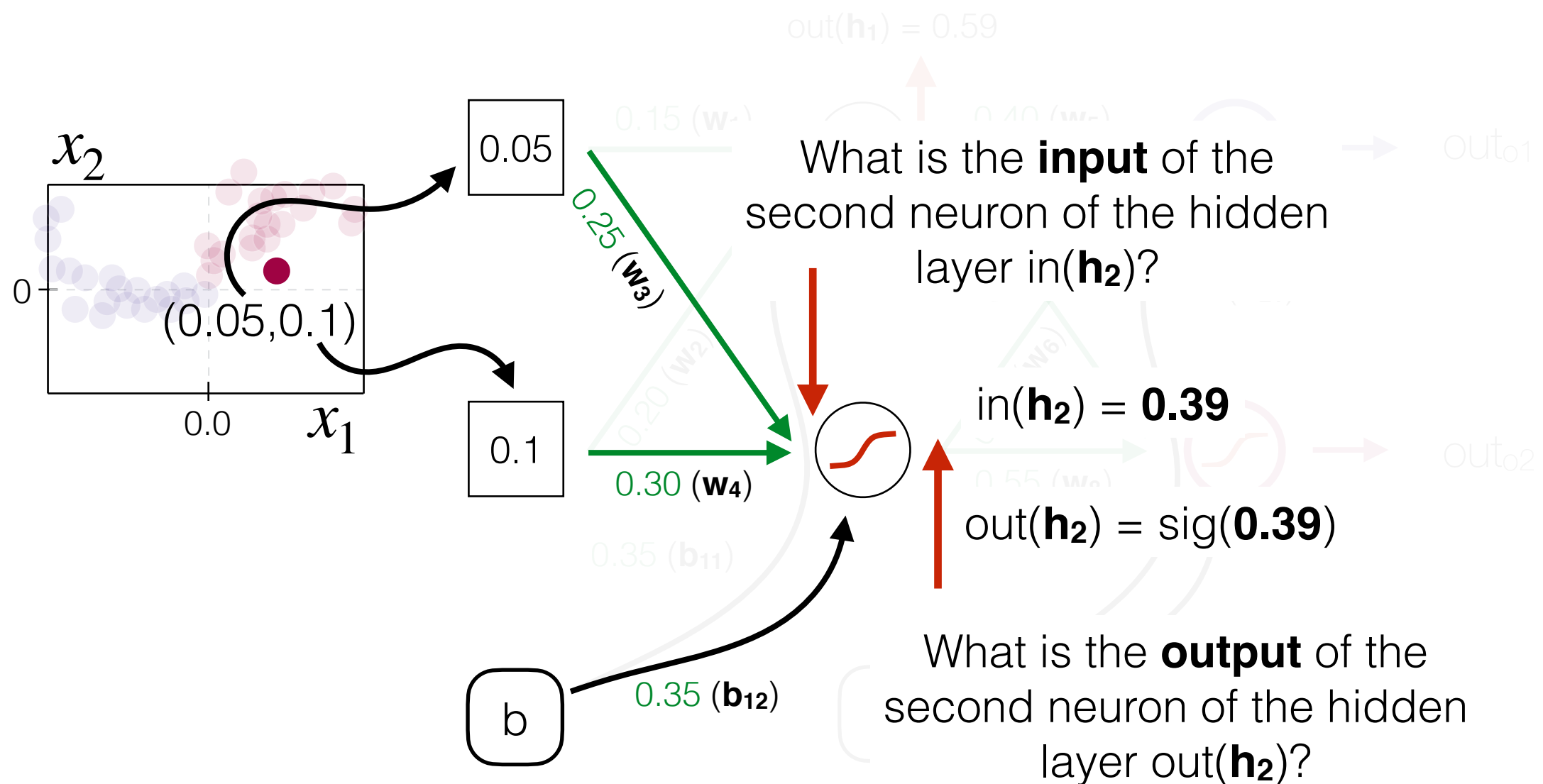
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



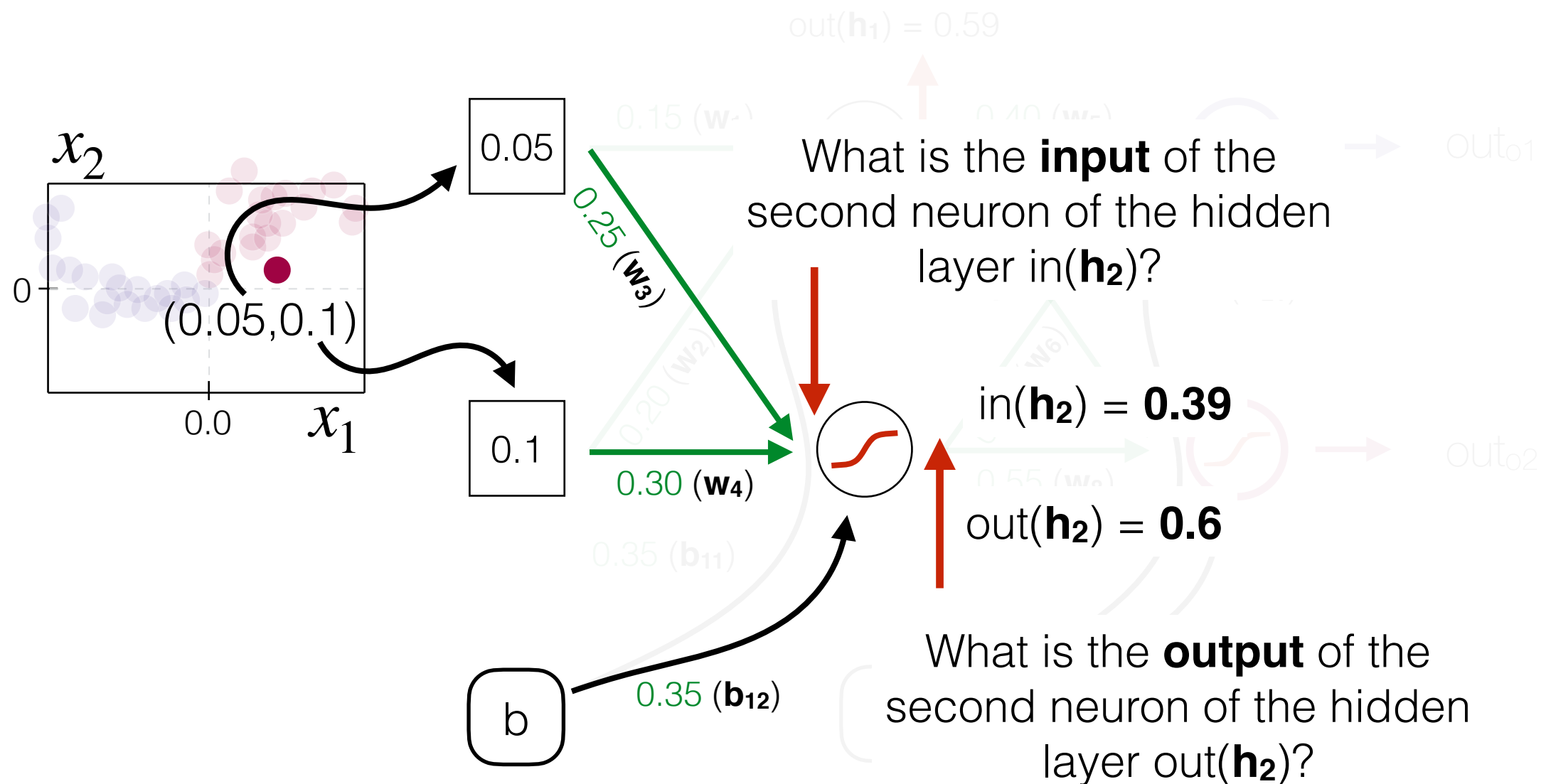
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



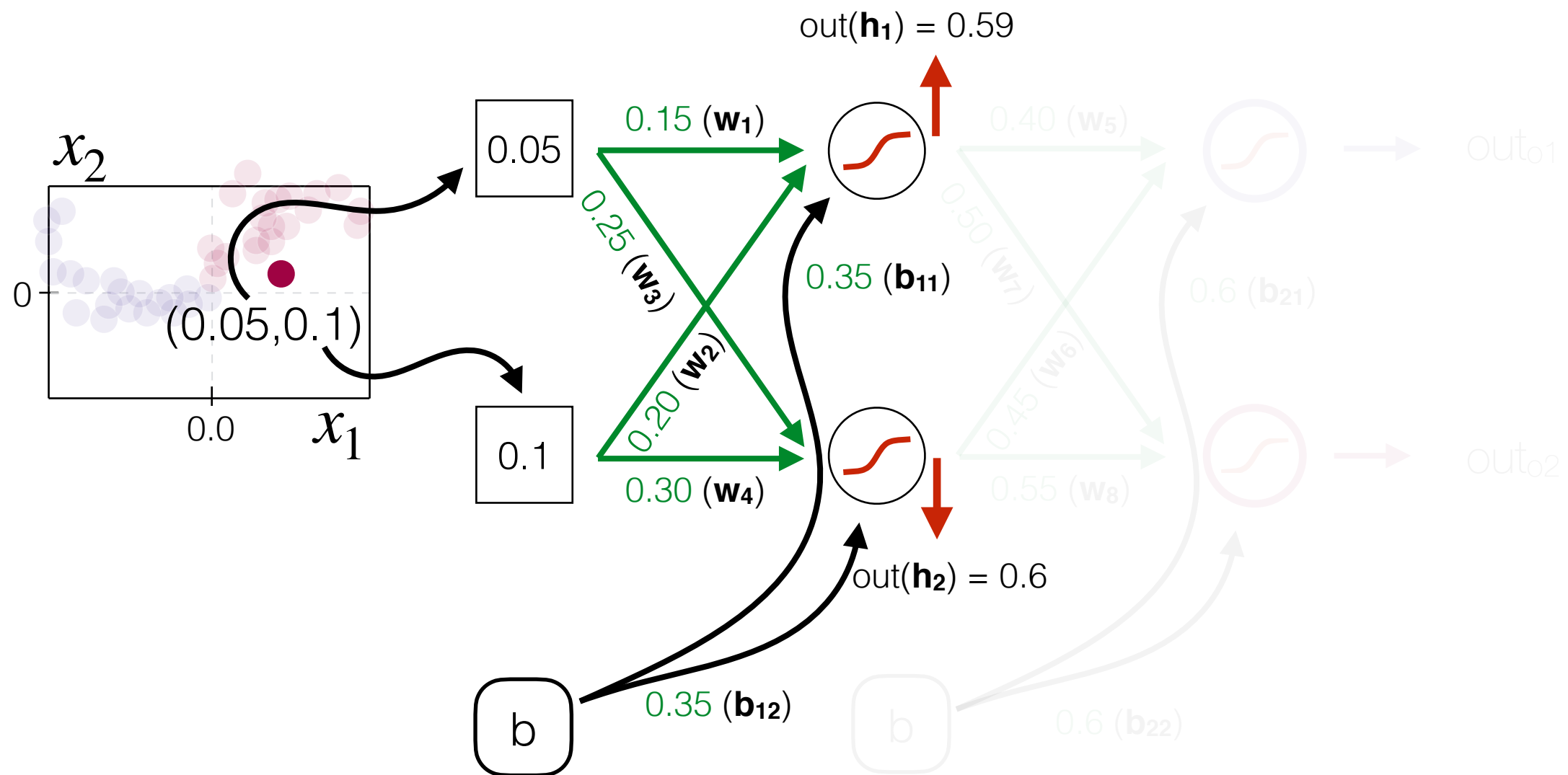
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



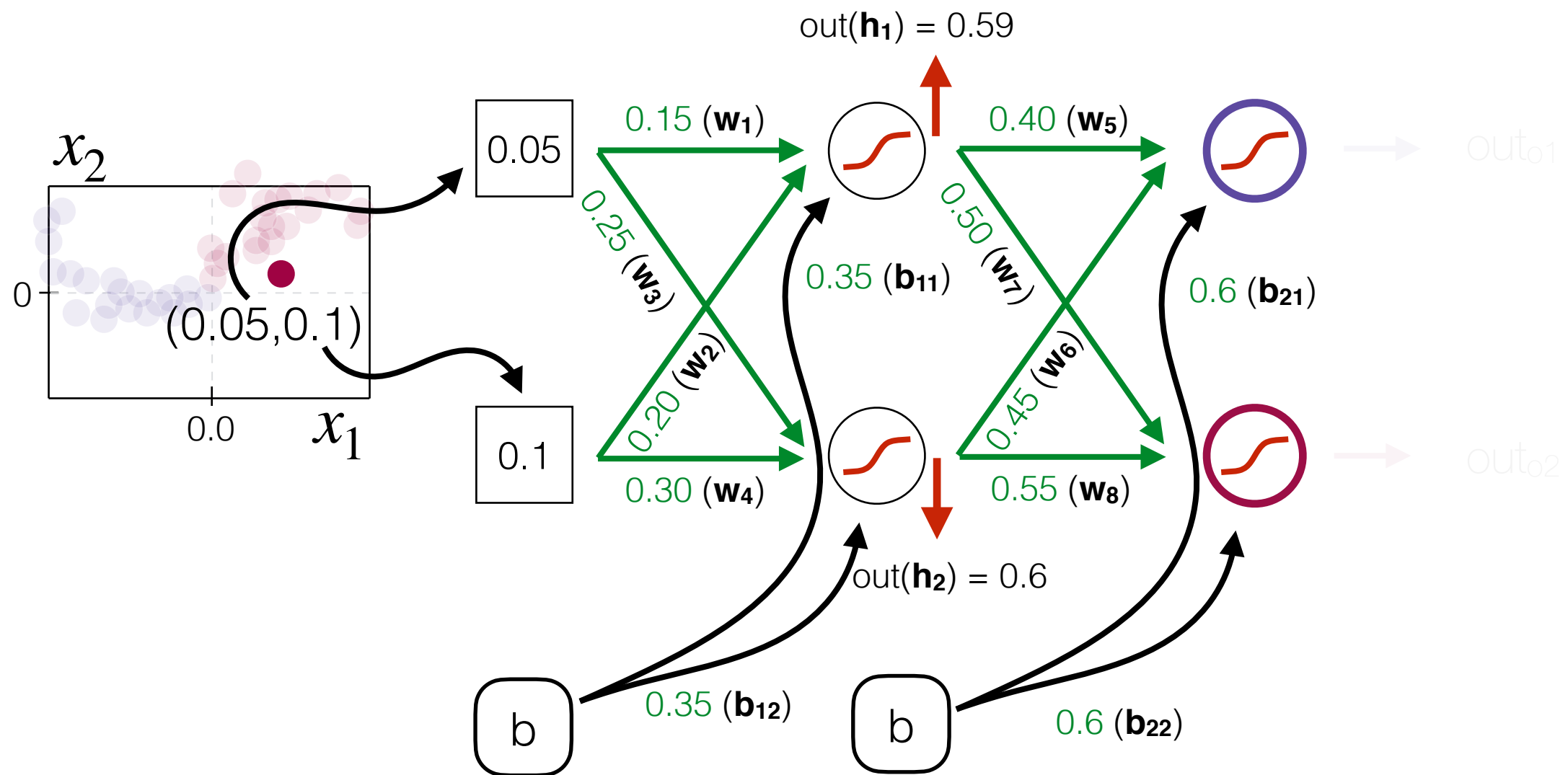
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



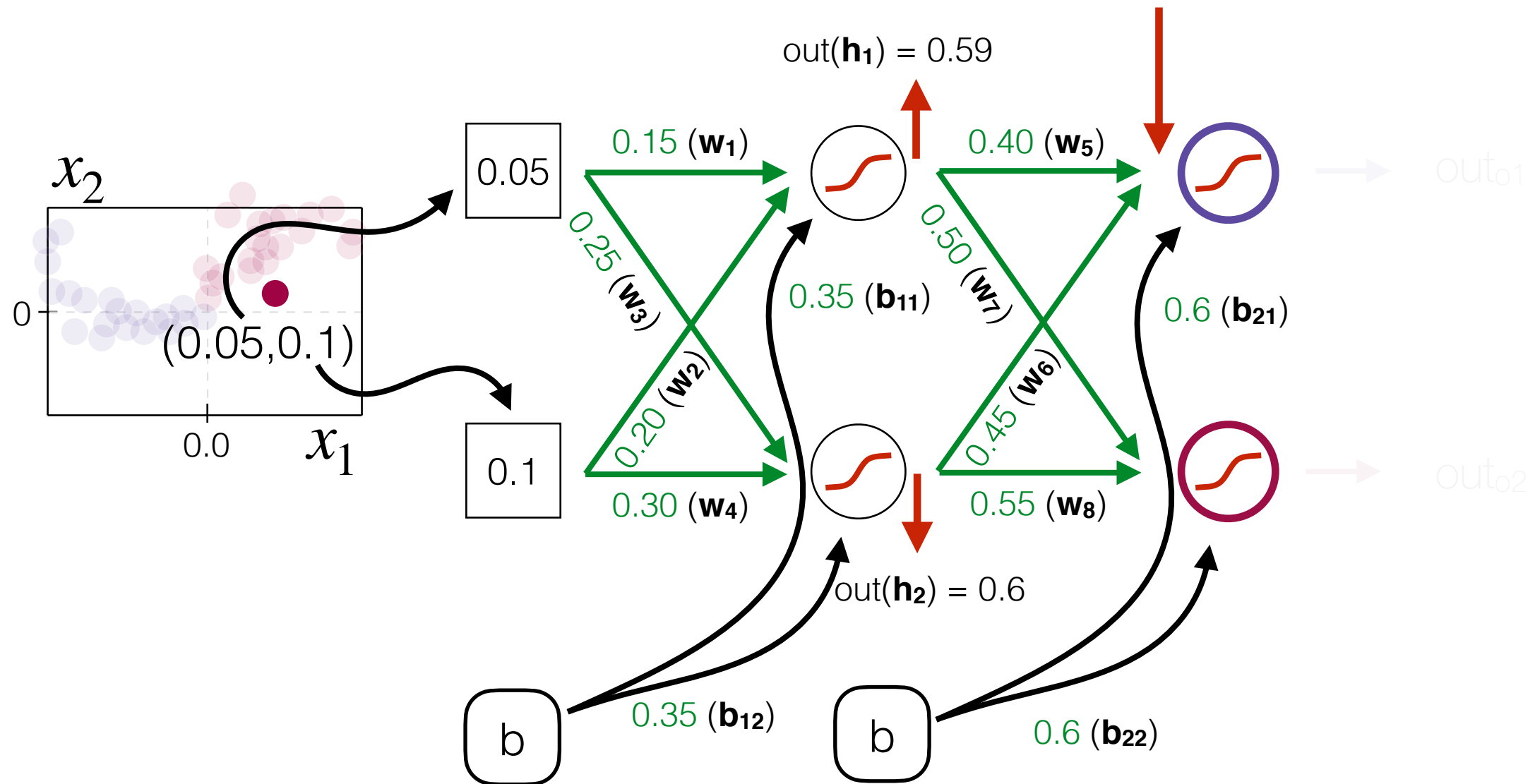
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



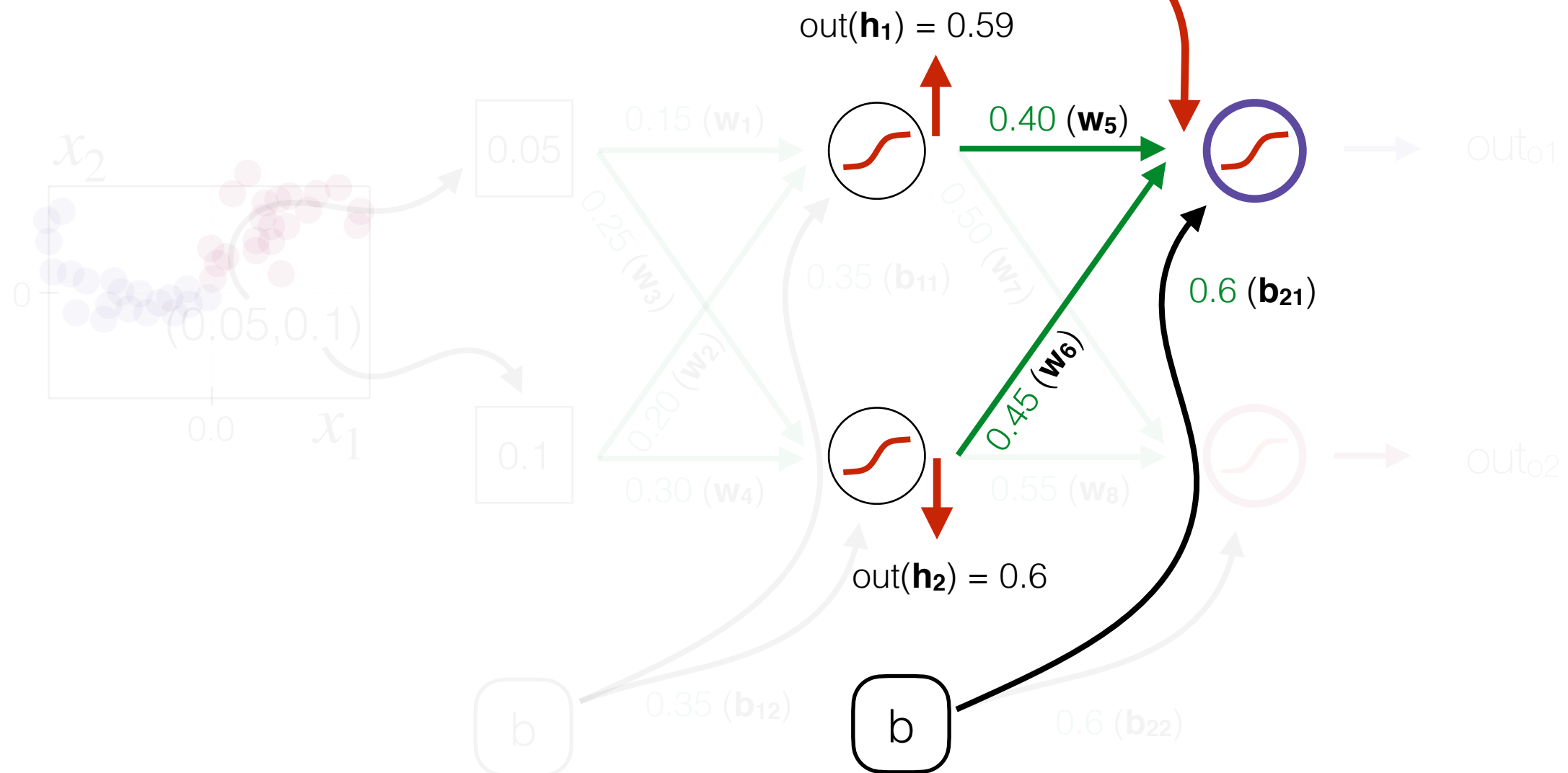
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



Let's calculate **class s** using **feed-forward** What is the **input** of the first neuron of the output layer $\text{in}(\mathbf{o}_1)$?



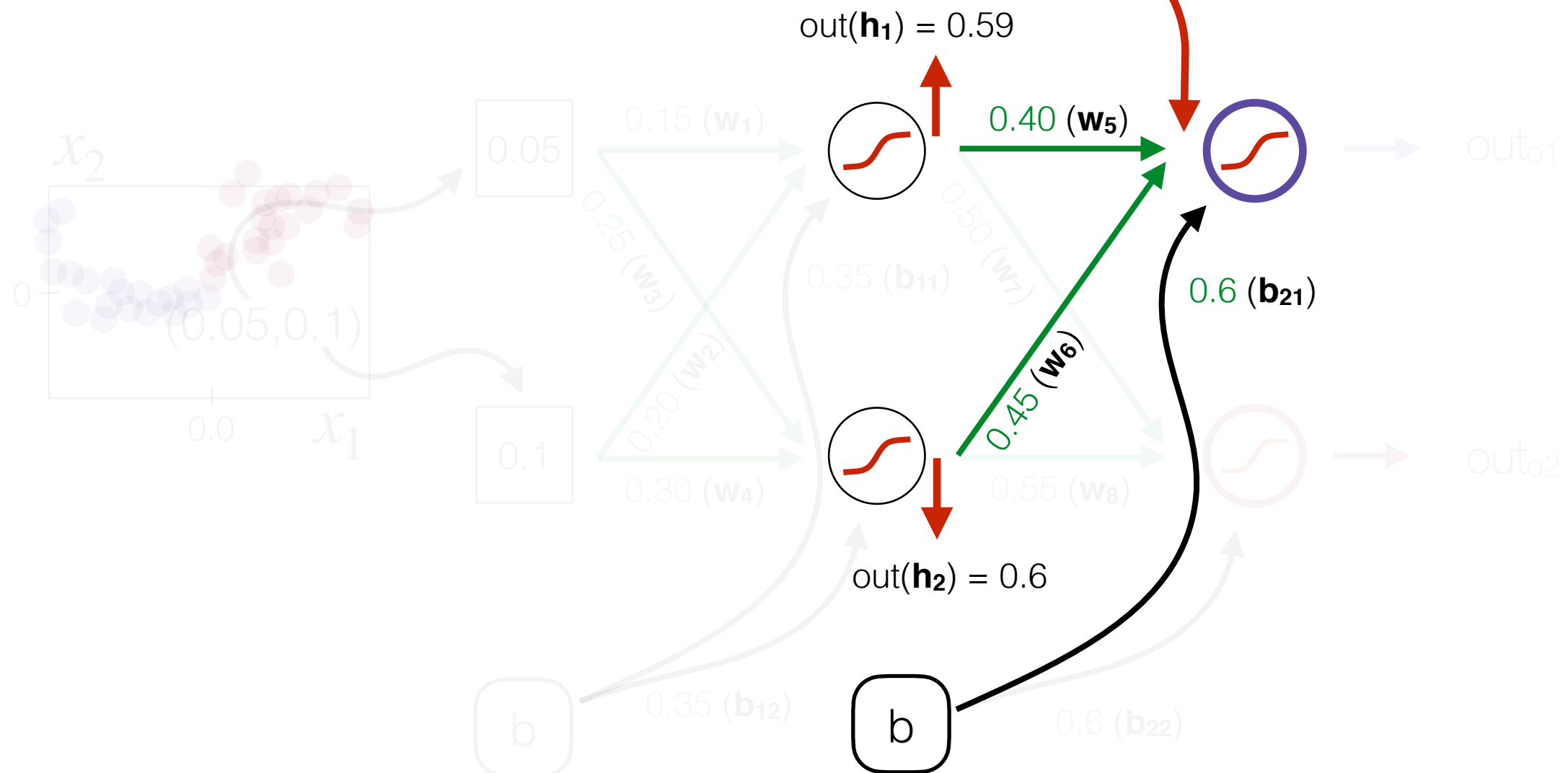
Let's find the **input** of the first neuron of the output layer
neuron of the output layer $\text{in}(\mathbf{o}_1) = \dots$



Let's find the **input** of the first neuron of the output layer

$$\text{in}(\mathbf{o}_1) = \text{out}(\mathbf{h}_1) * w_5 + \text{out}(\mathbf{h}_2) * w_6 + b_{21}$$

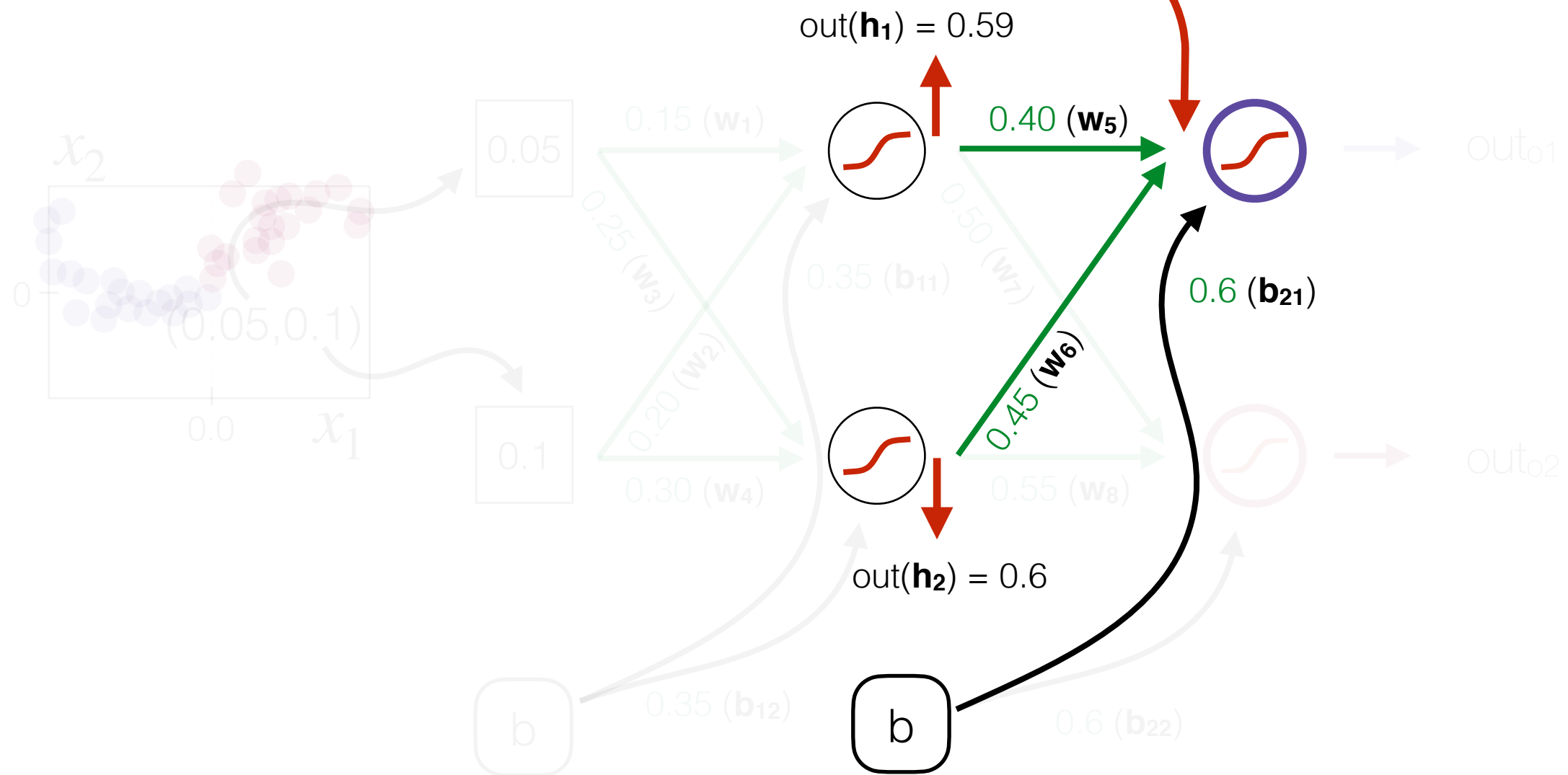
$\text{in}(\mathbf{o}_1)$?



Let's What is the **input** of the first neuron of the output layer
neuron of the output layer
 $\text{in}(\mathbf{o}_1)$?

res for each class
path algorithm

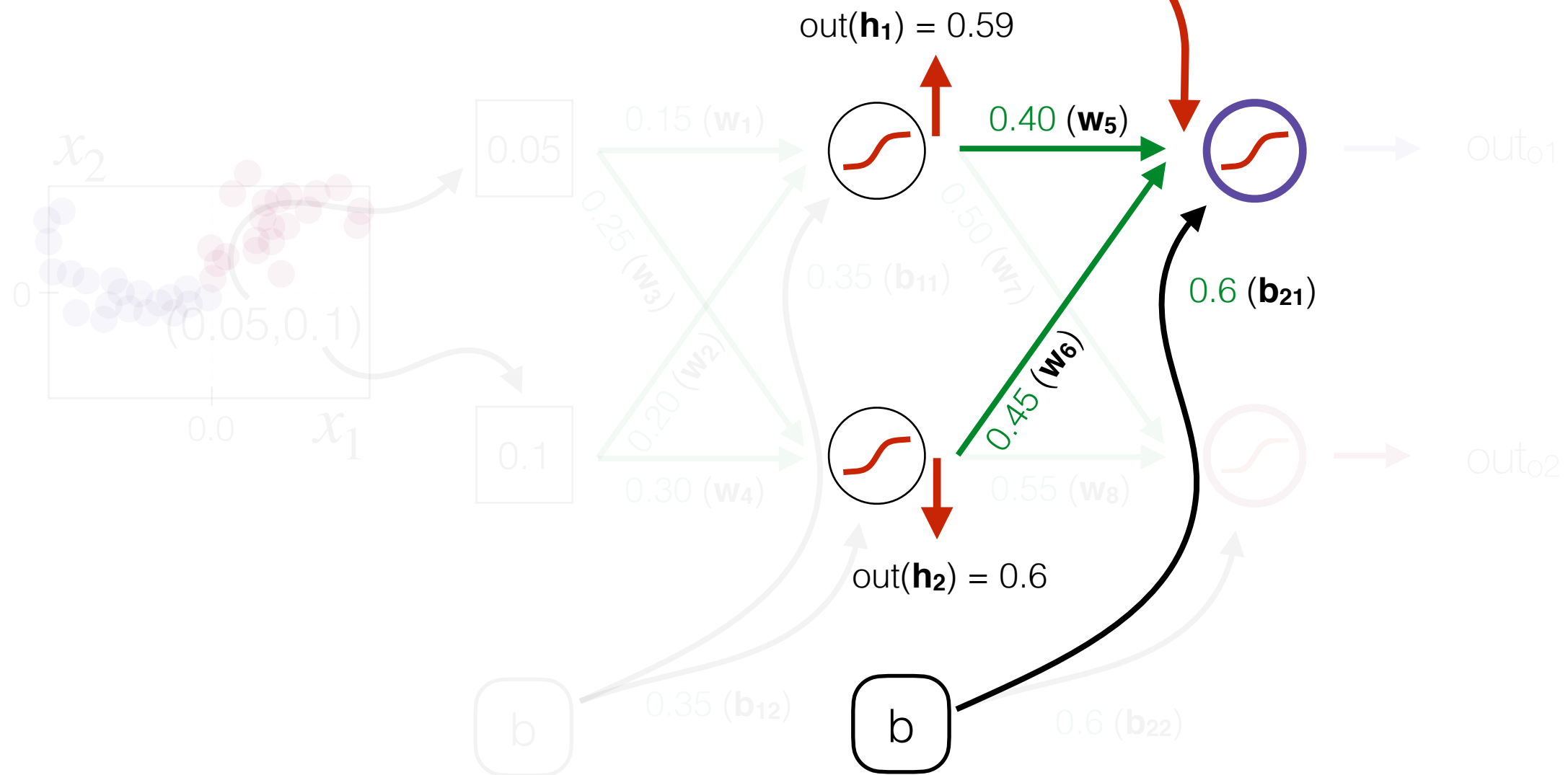
$$\text{in}(\mathbf{o}_1) = \text{out}(\mathbf{h}_1) * 0.4 + \text{out}(\mathbf{h}_2) * 0.45 + 0.6$$



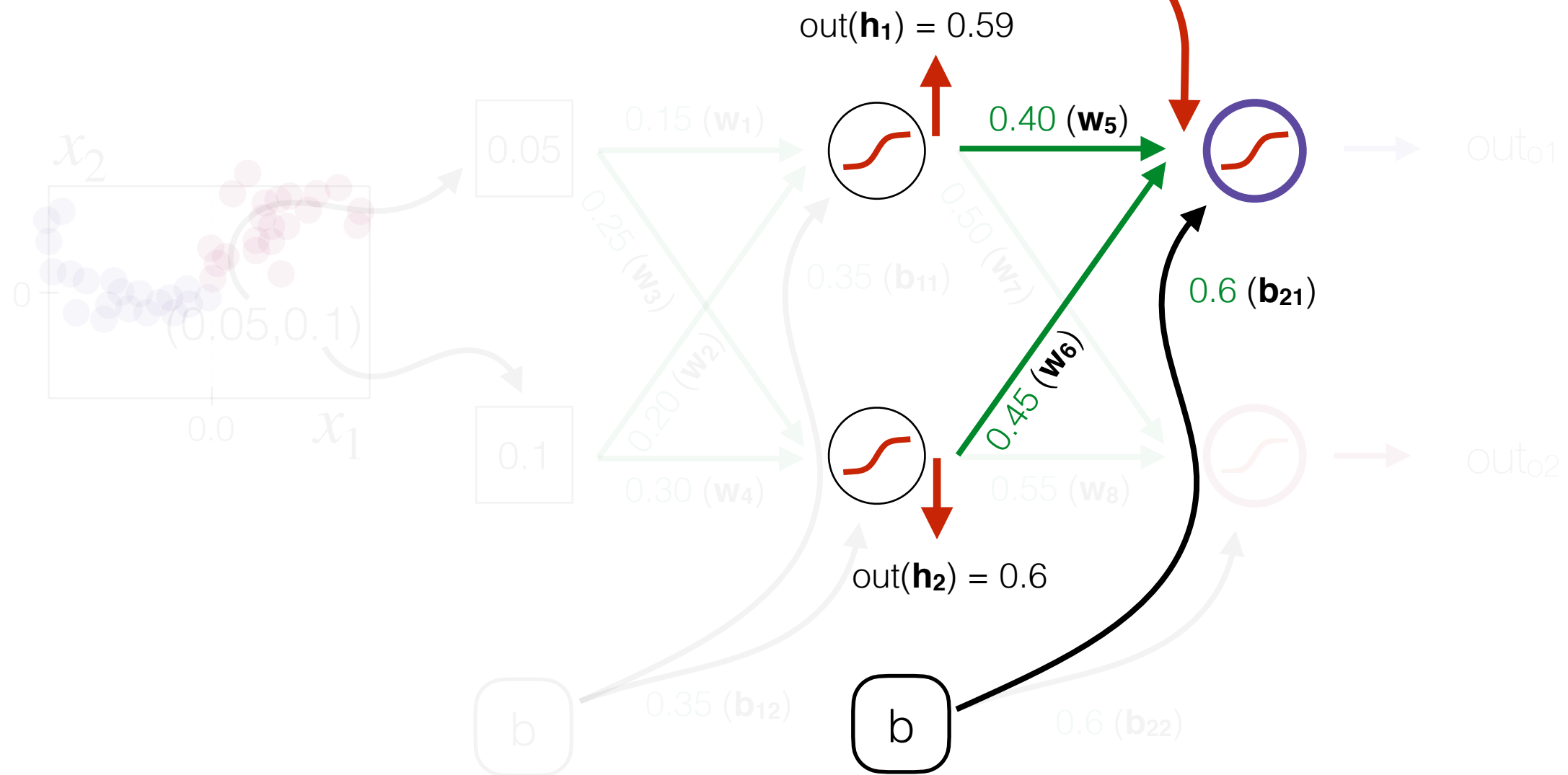
Let's What is the **input** of the first neuron of the output layer
neuron of the output layer
 $\text{in}(\mathbf{o}_1)$?

res for each class
path algorithm

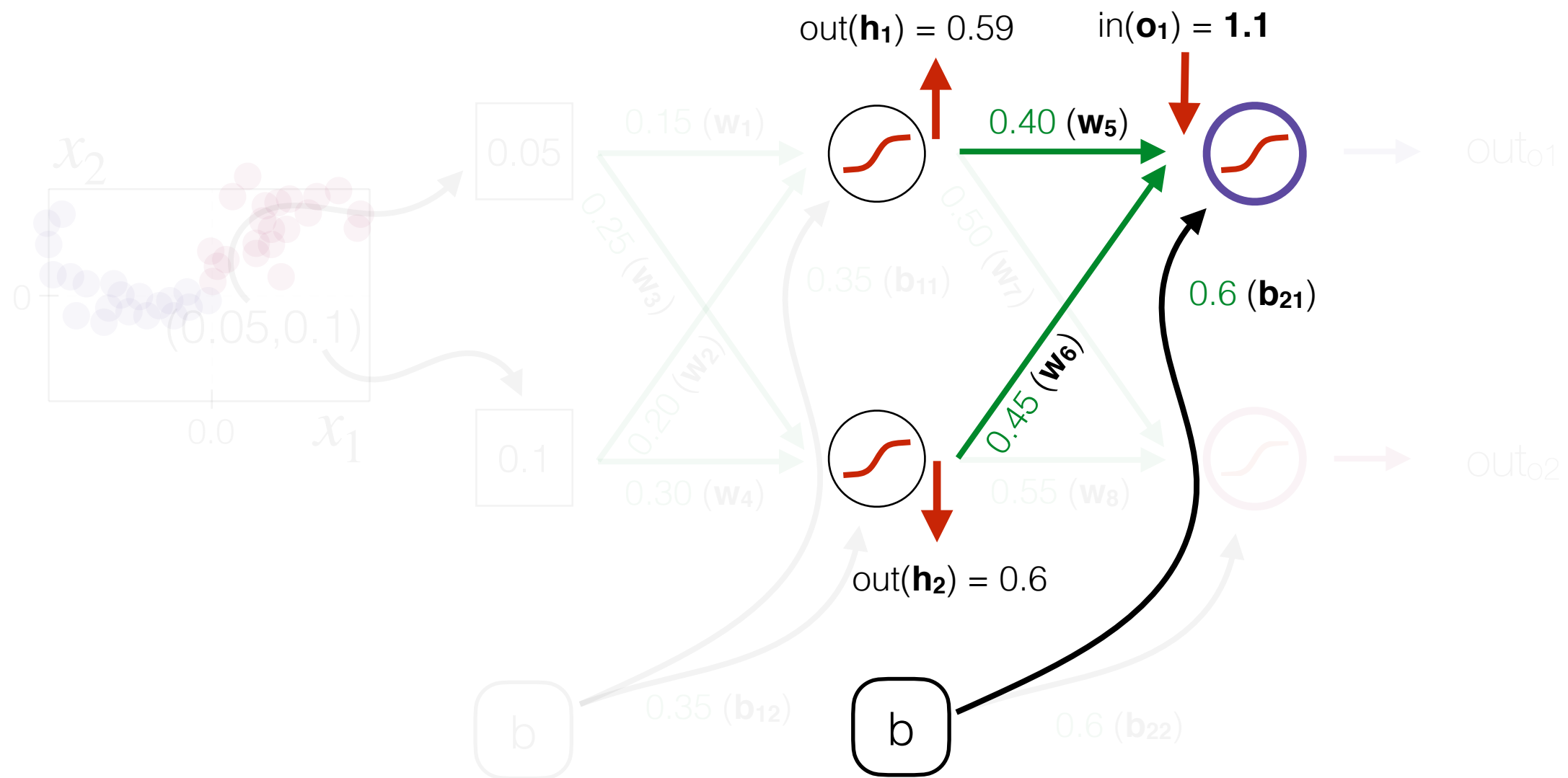
$$\text{in}(\mathbf{o}_1) = 0.59 * 0.4 + 0.6 * 0.45 + 0.6$$



Let's find the **input** of the first neuron of the output layer
 What is the **input** of the first neuron of the output layer
 $\text{in}(\mathbf{o}_1) = 1.1$

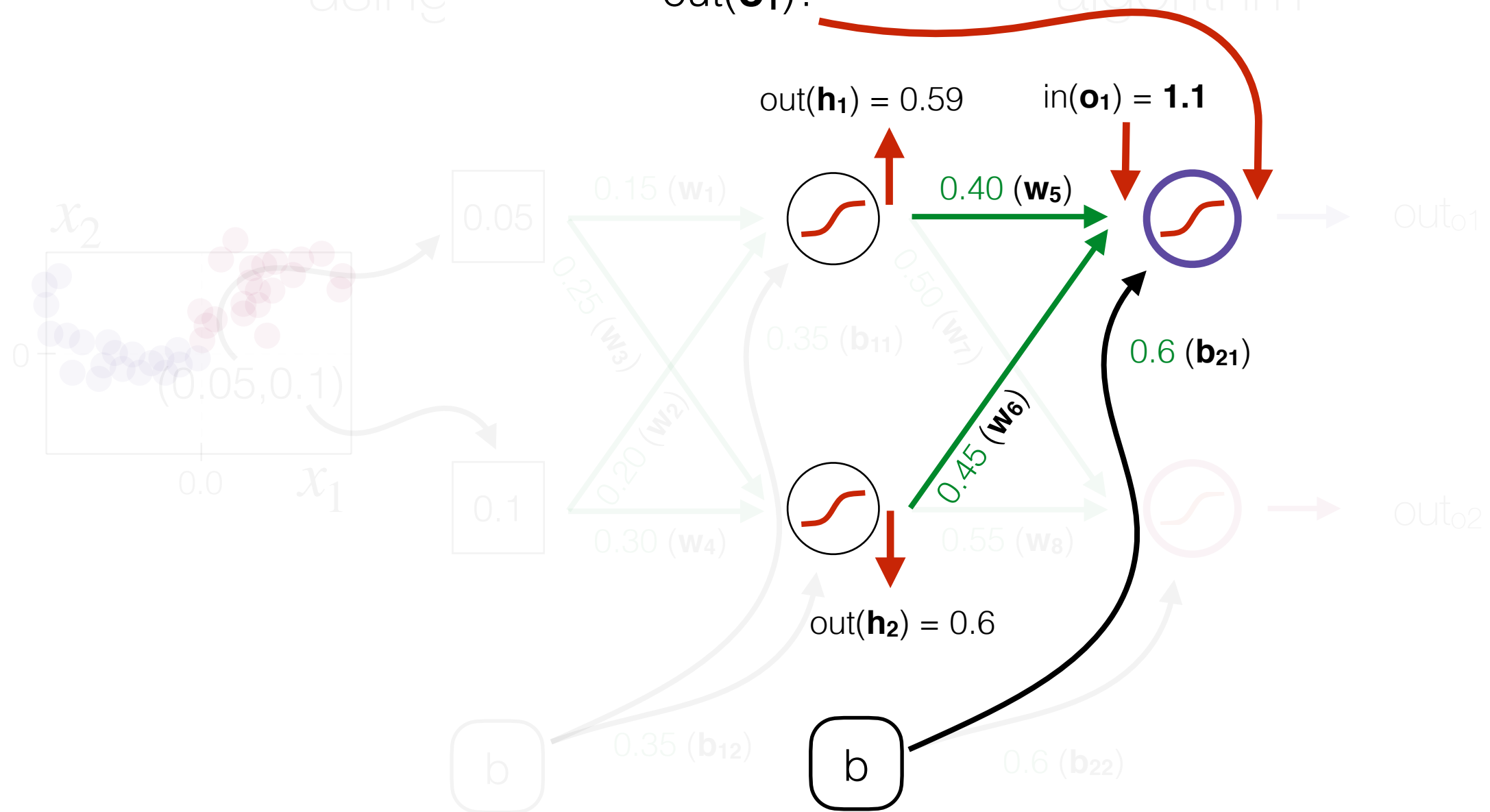


Let's calculate **class scores** for each class
using **feed-forward path** algorithm



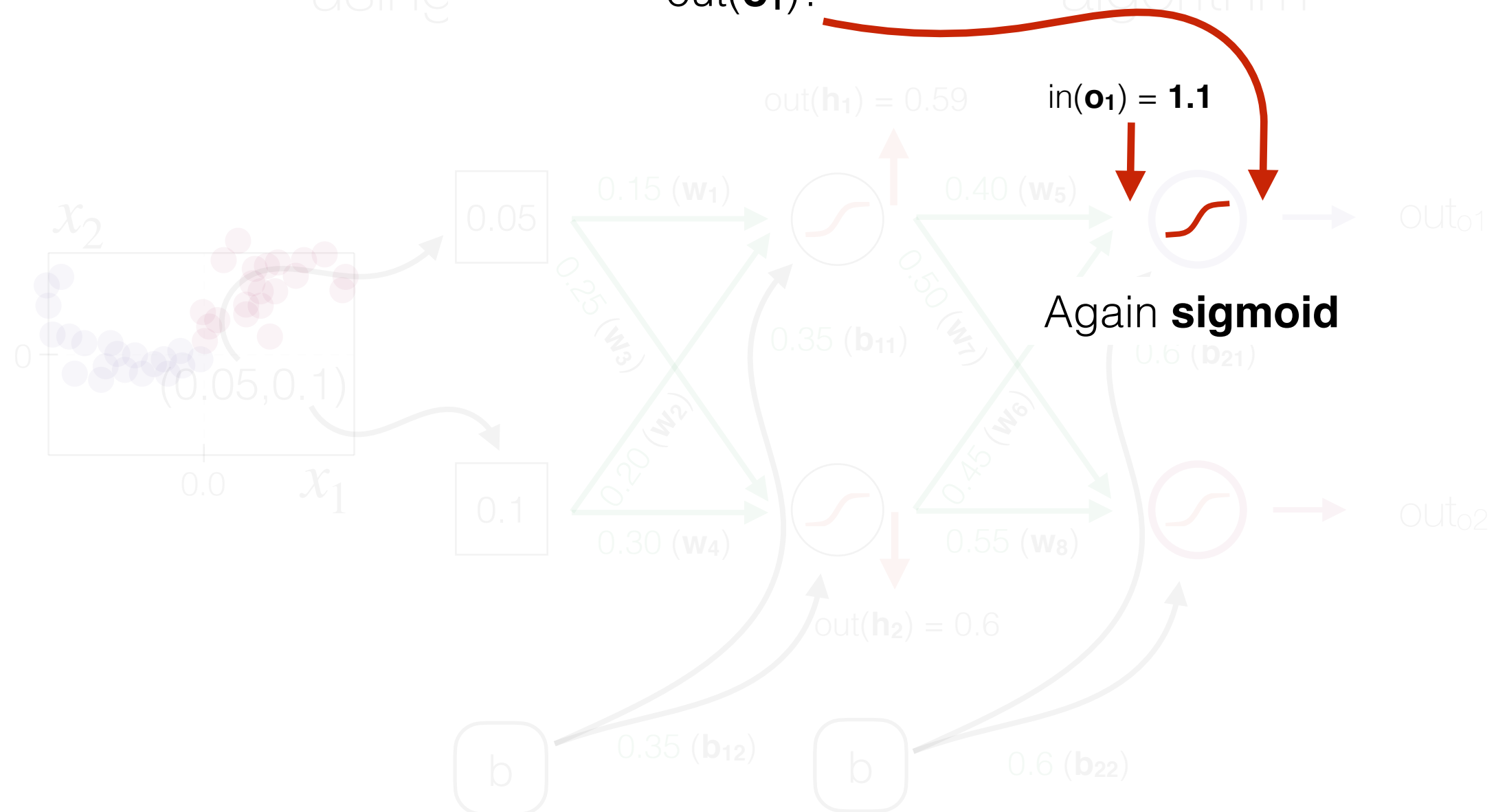
Let's calculate the output for each class using the sigmoid algorithm

What is the **output** of the first neuron of the output layer $\text{out}(\mathbf{o}_1)$?



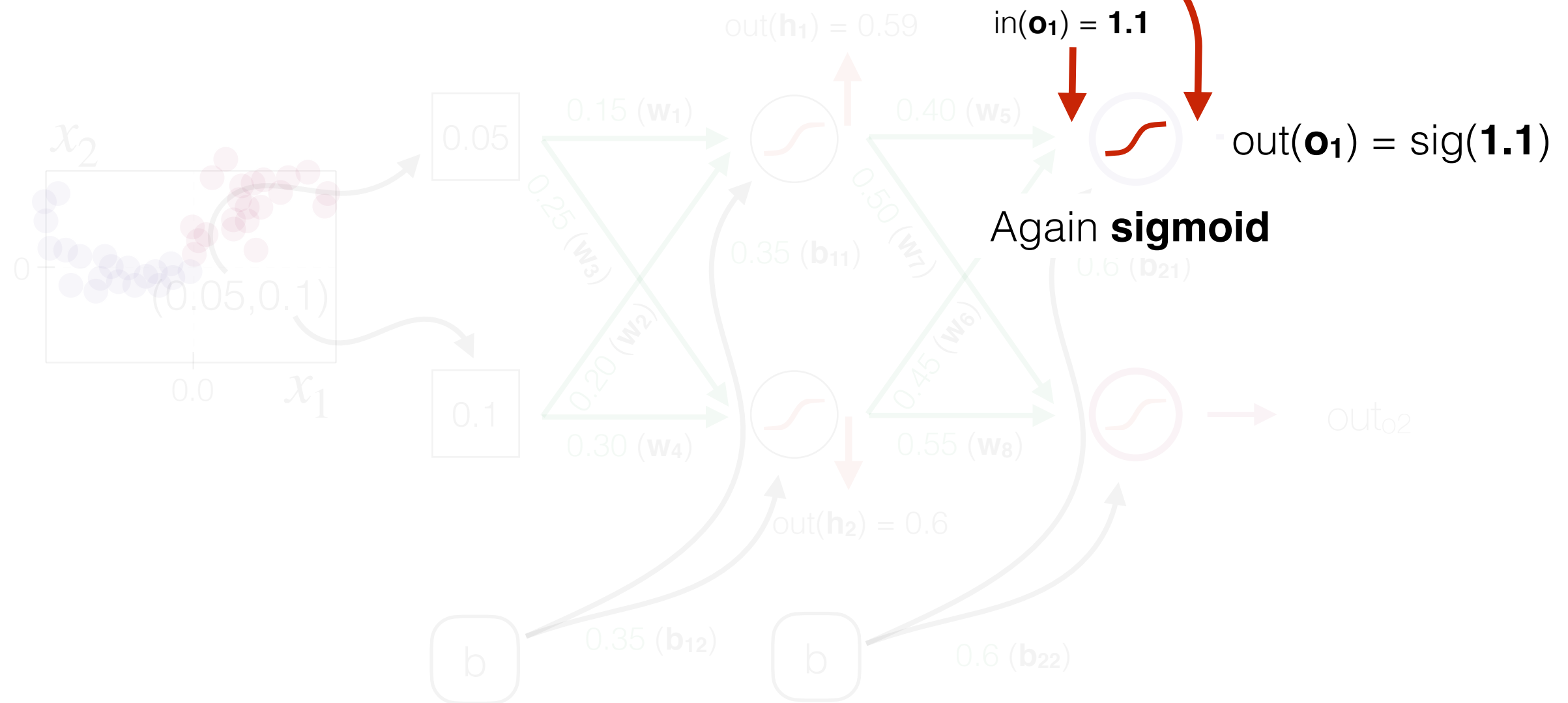
Let's calculate the output for each class using the sigmoid algorithm

What is the **output** of the first neuron of the output layer $out(o_1)$?



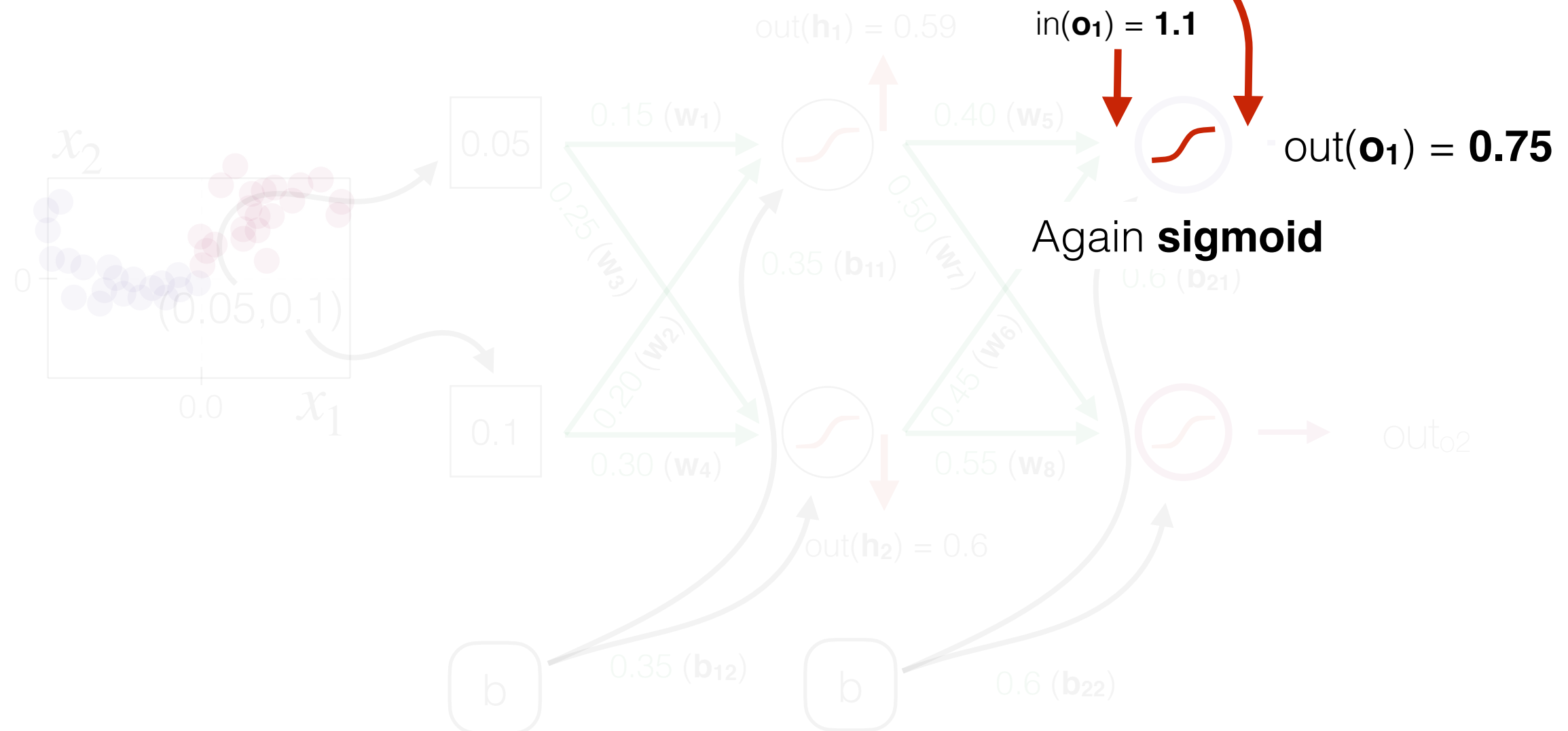
Let's calculate the output for each class using the sigmoid algorithm

What is the **output** of the first neuron of the output layer $\text{out}(\mathbf{o}_1)$?

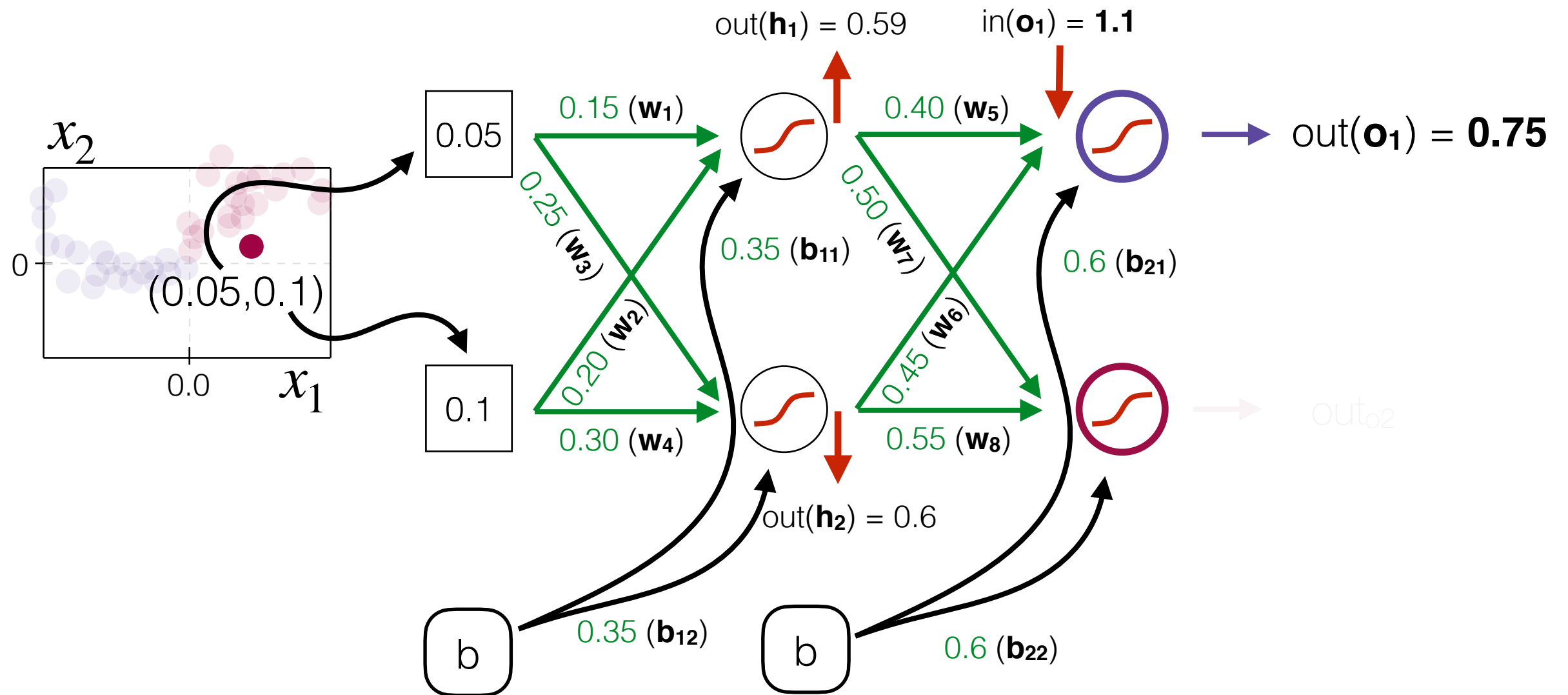


Let's calculate the output for each class using the sigmoid algorithm

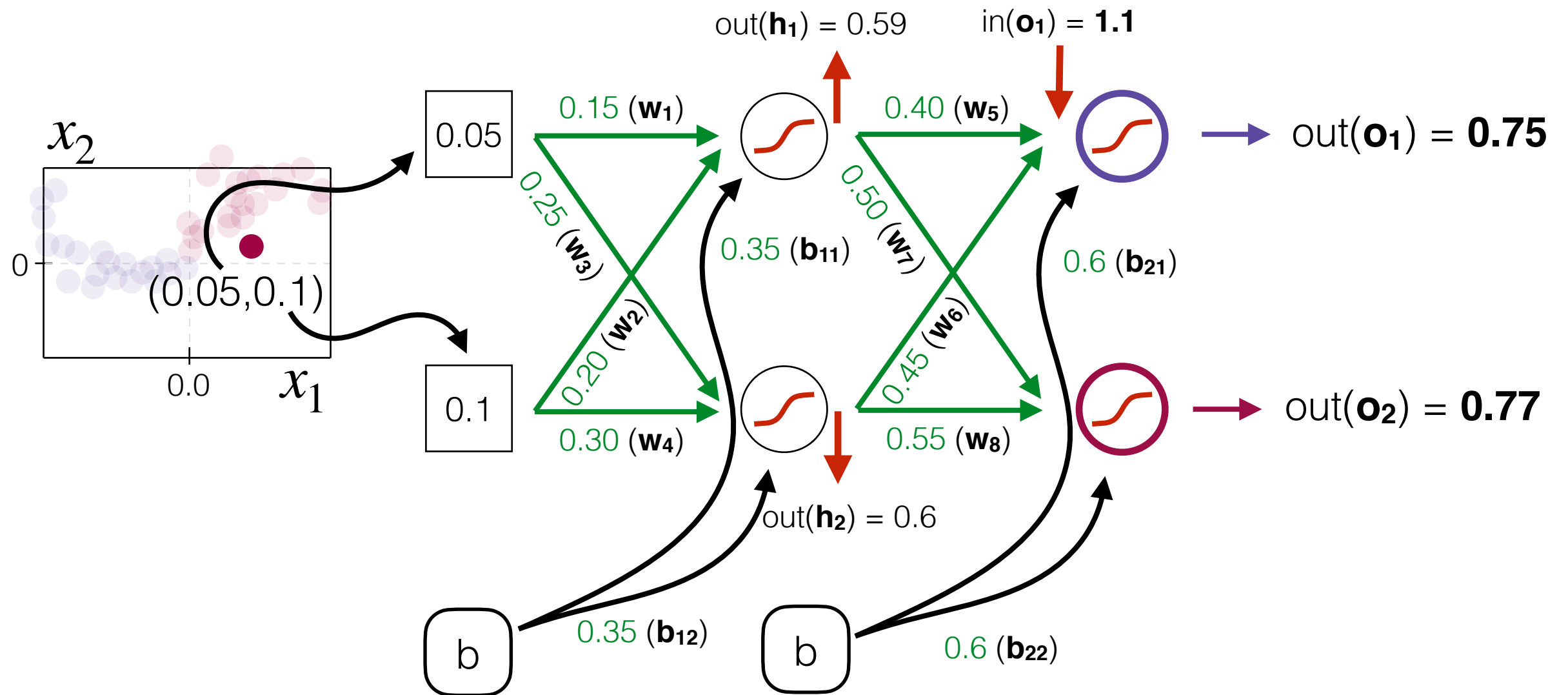
What is the **output** of the first neuron of the output layer $\text{out}(\mathbf{o}_1)$?



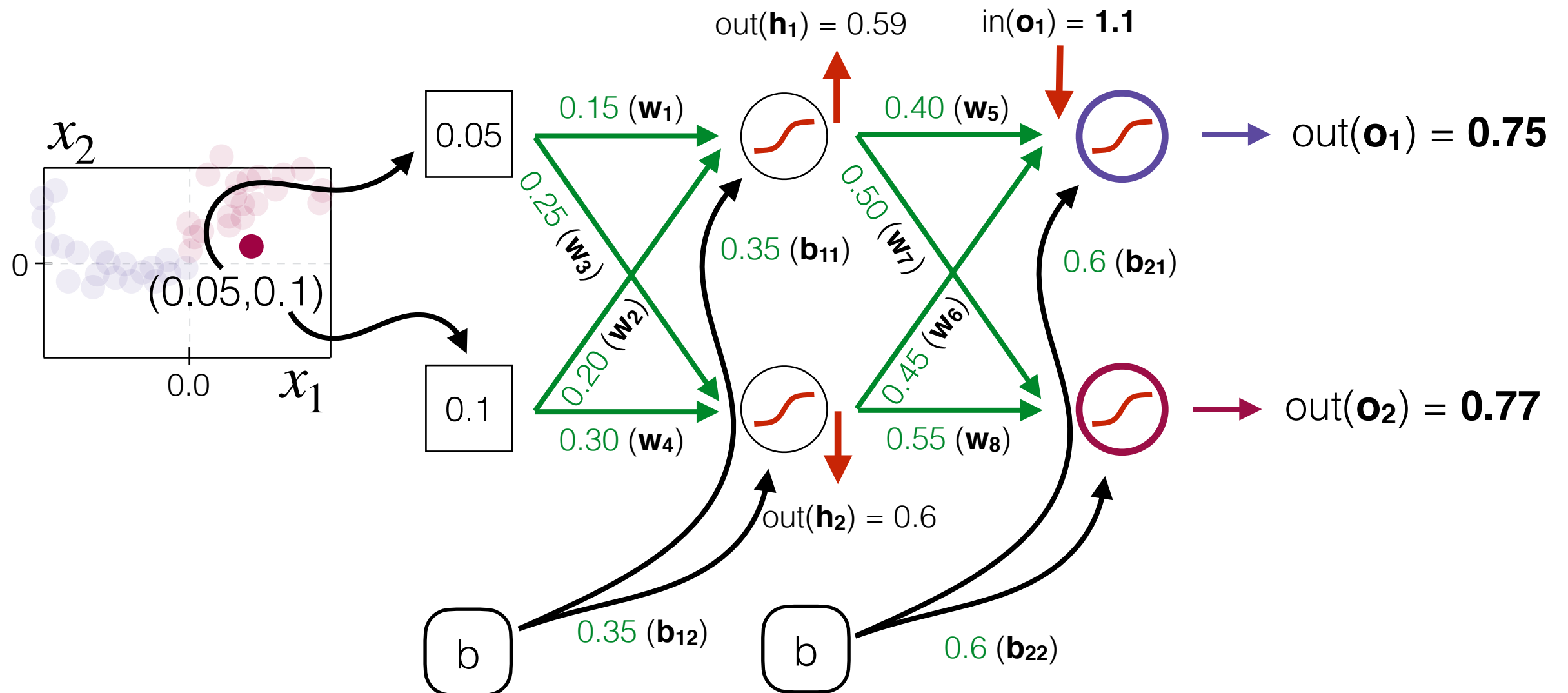
Let's calculate **class scores** for each class
using **feed-forward path** algorithm



Let's calculate **class scores** for each class
using **feed-forward path** algorithm

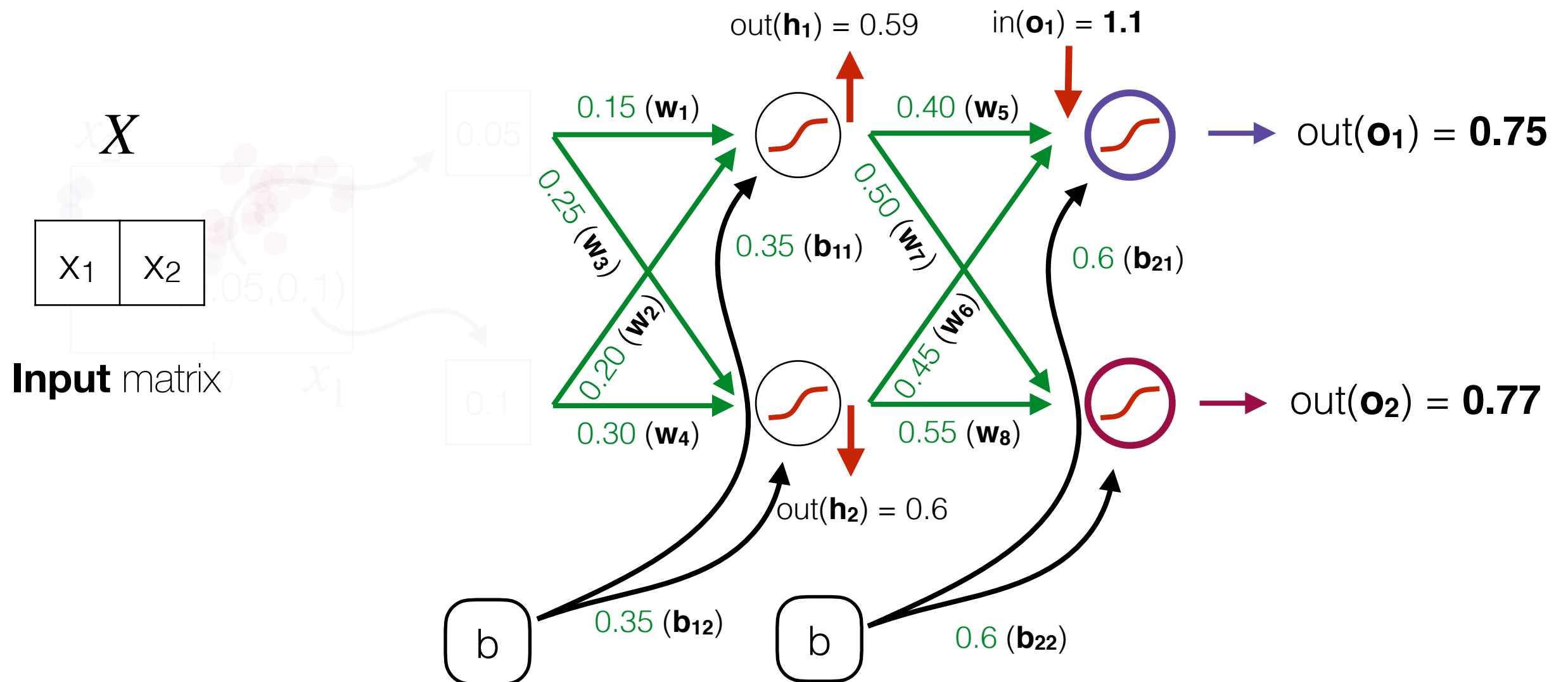


Let's calculate **class scores** for each class
using **feed-forward path** algorithm



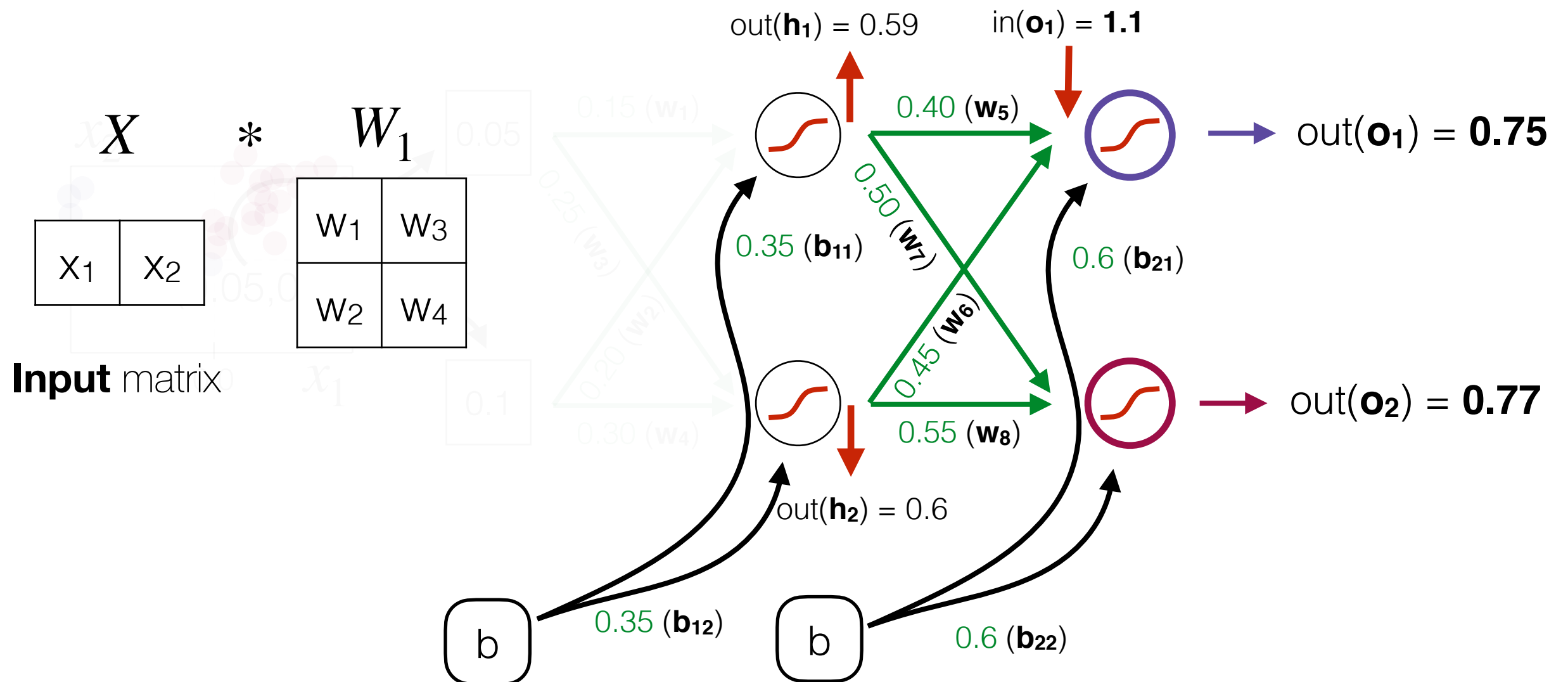
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



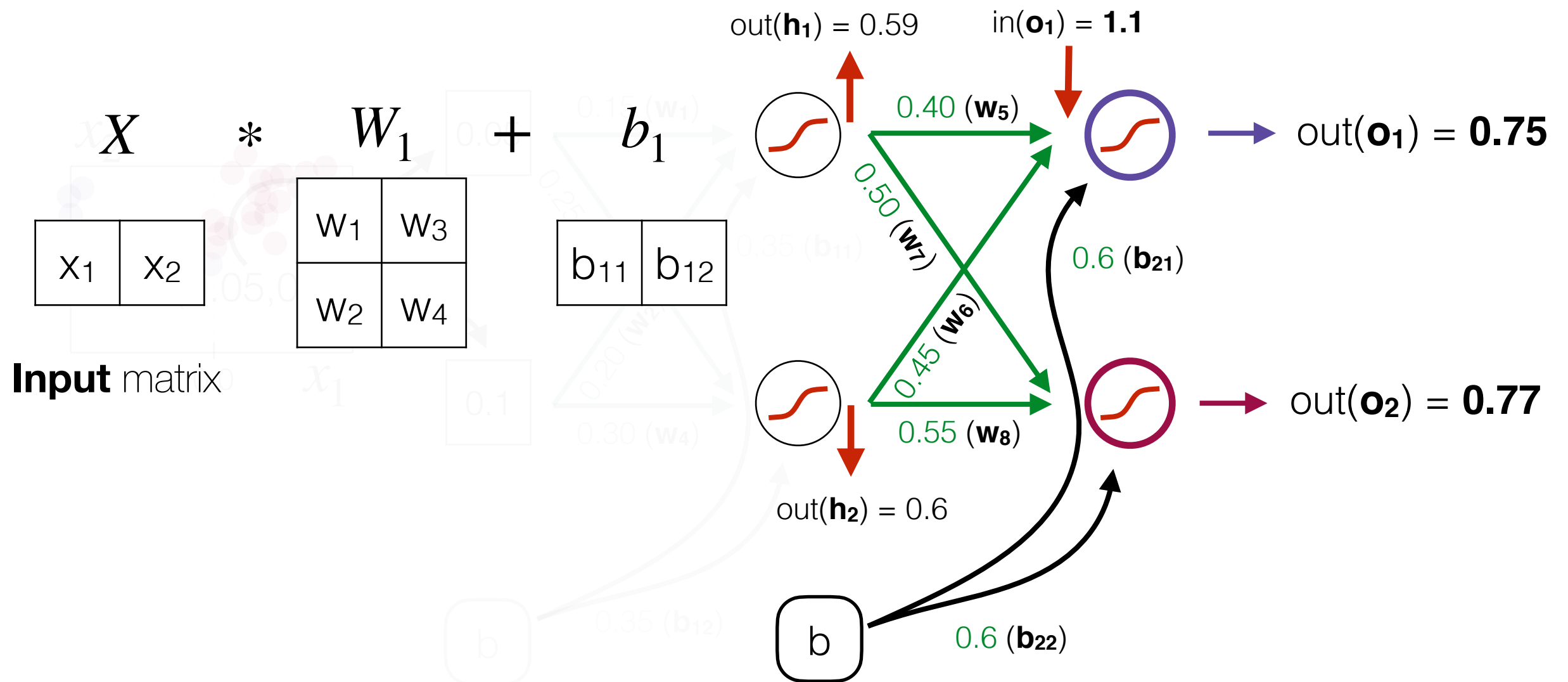
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



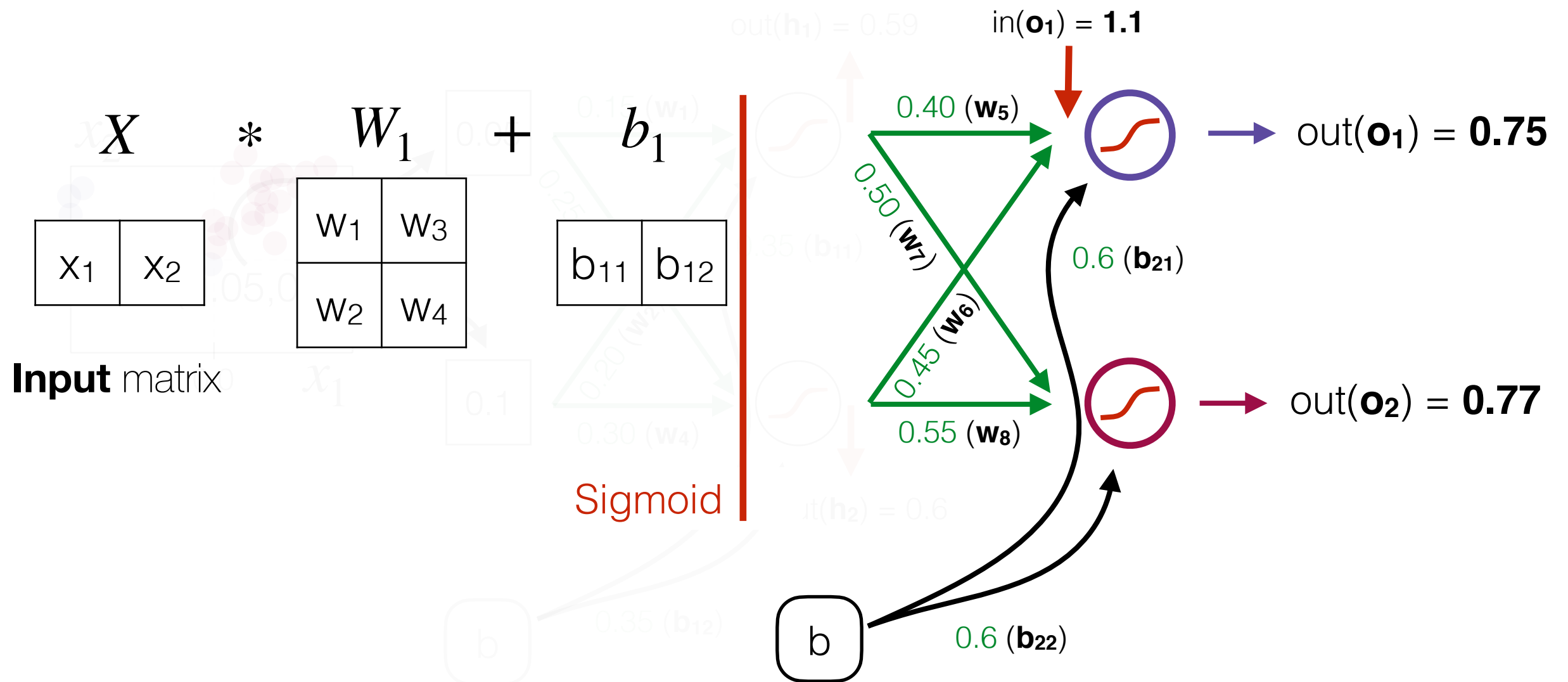
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



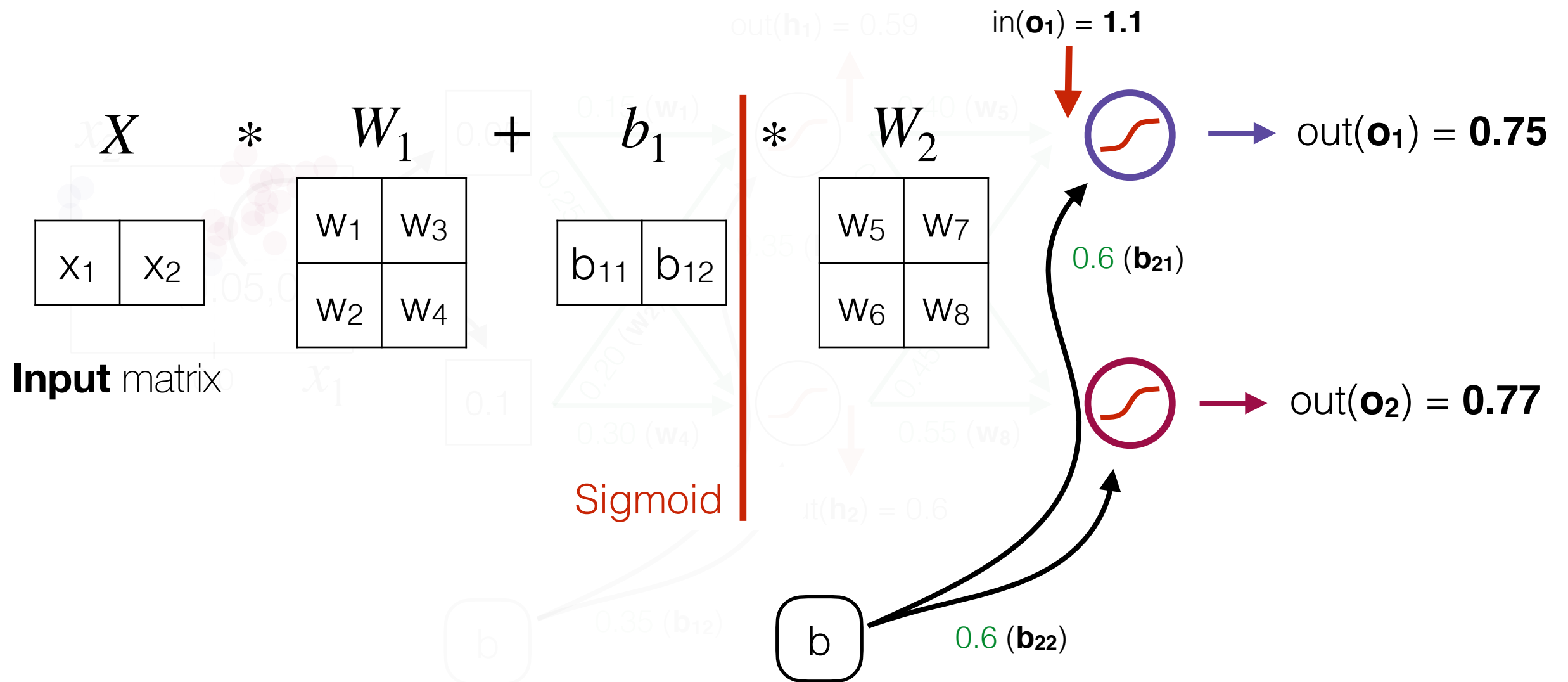
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



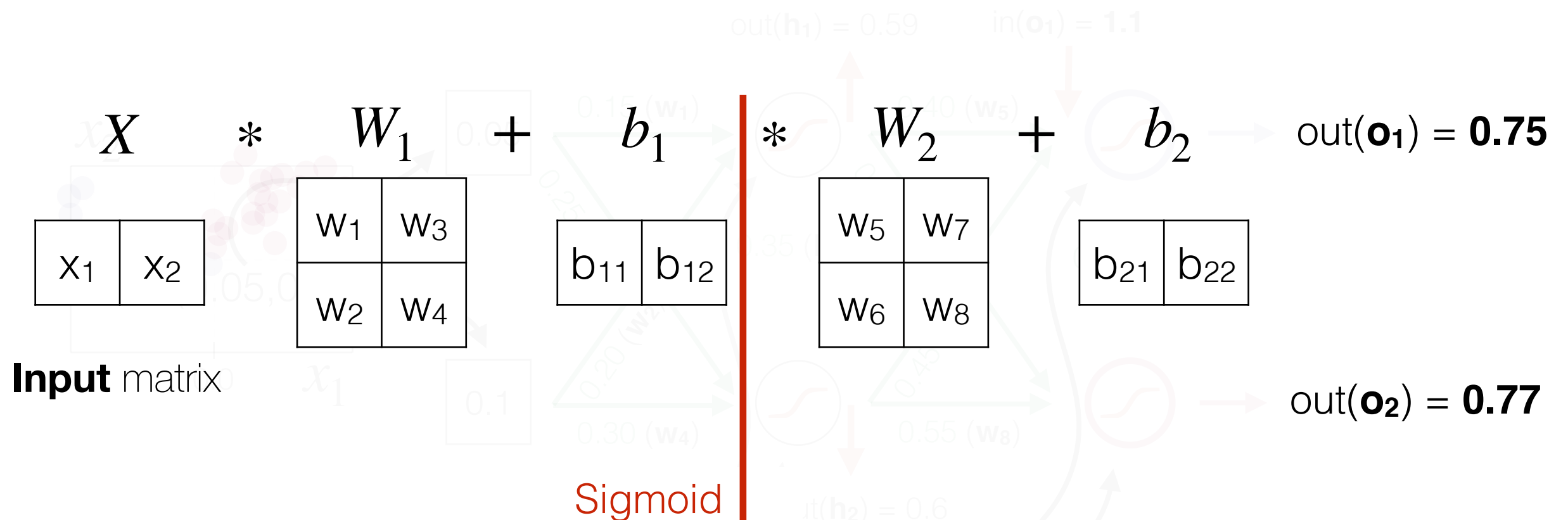
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



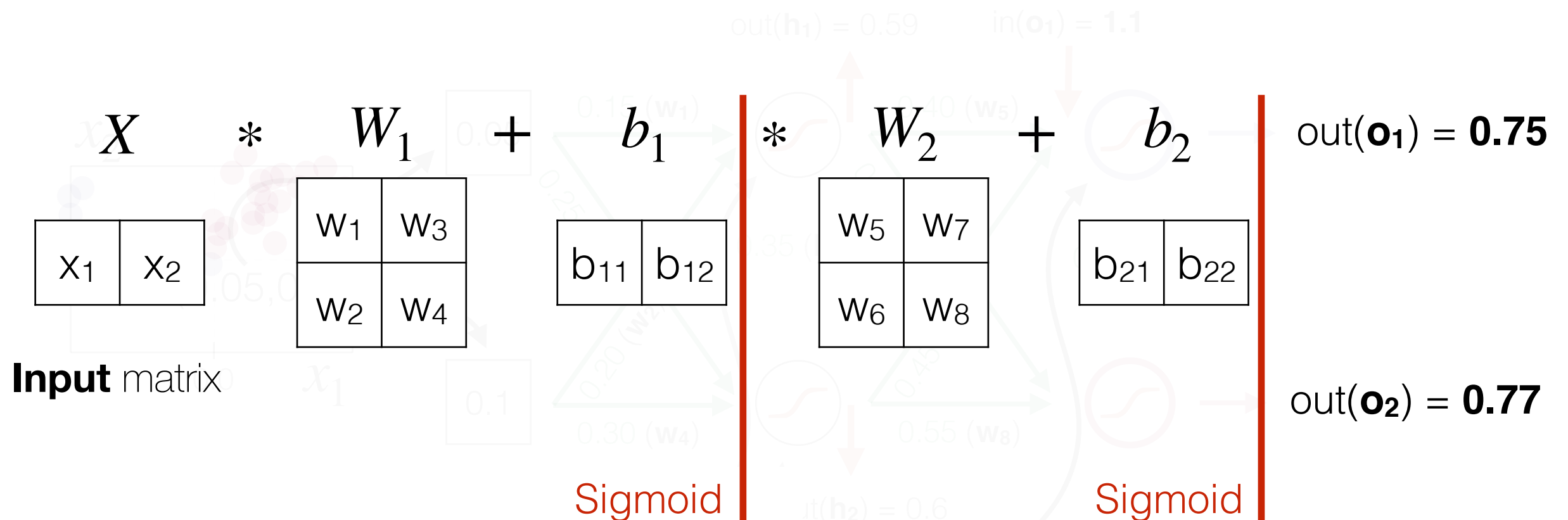
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



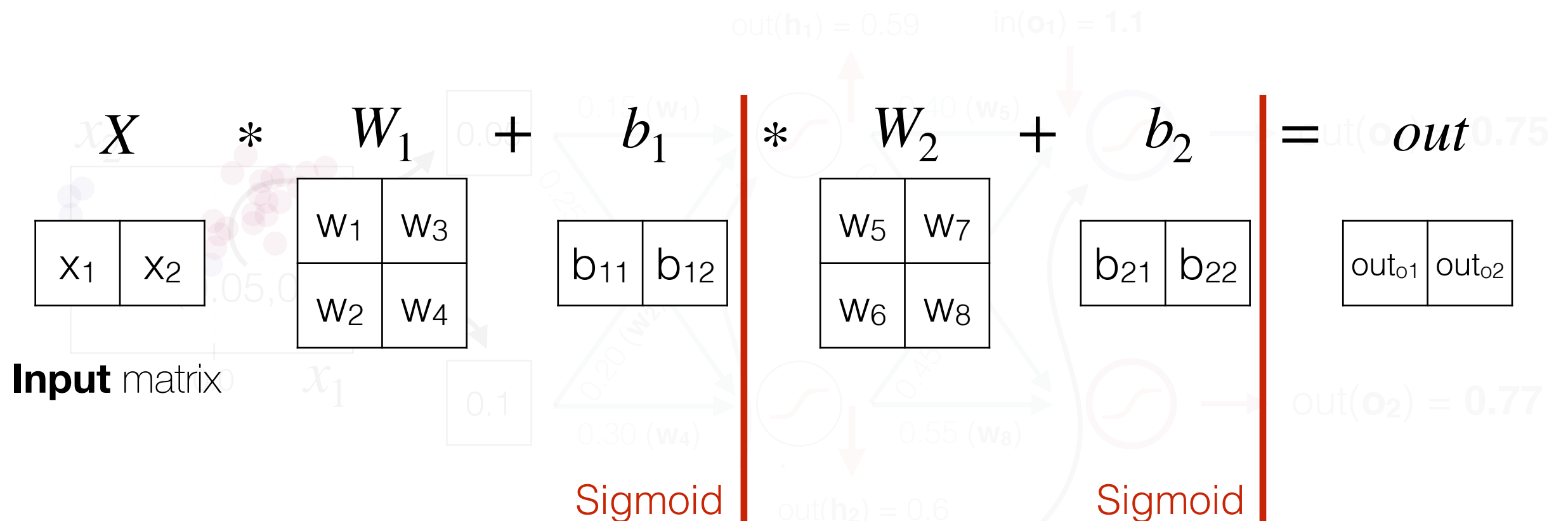
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm



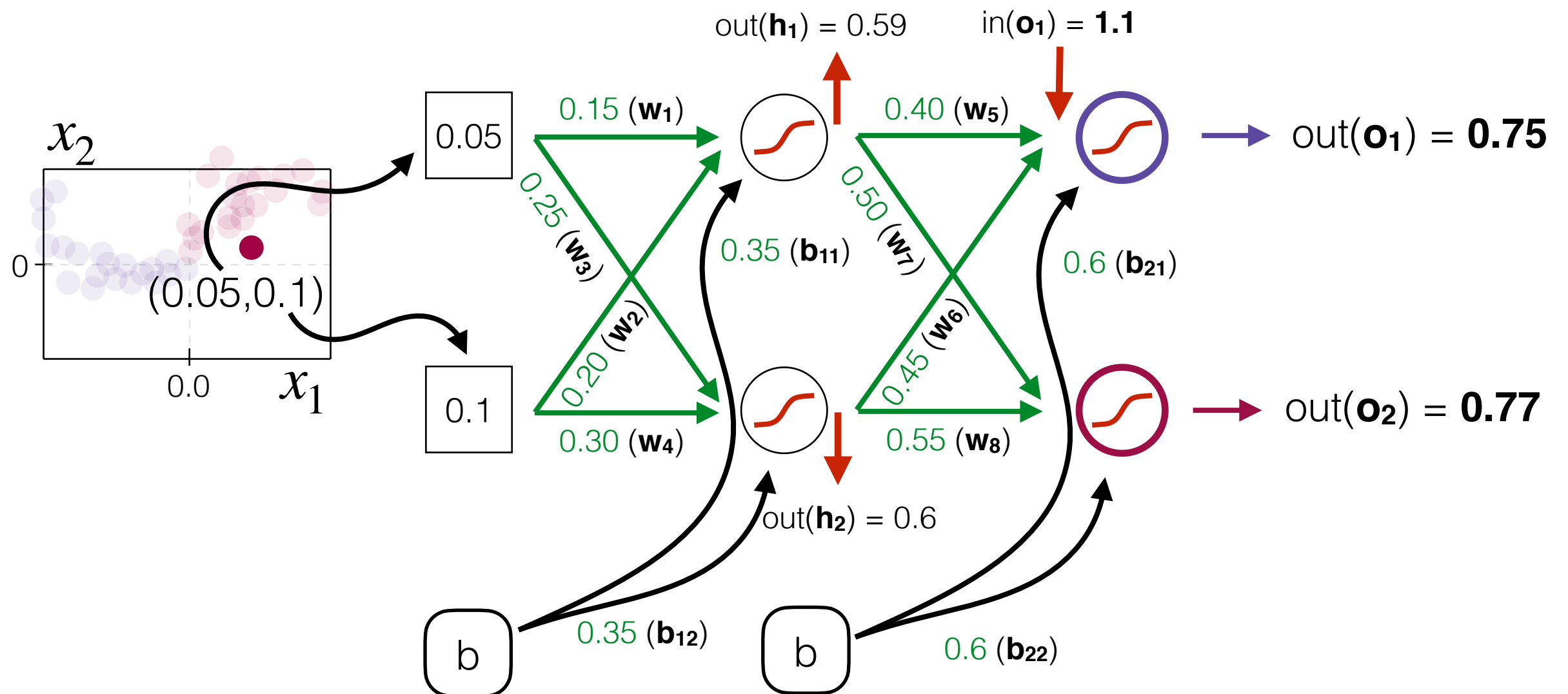
Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm

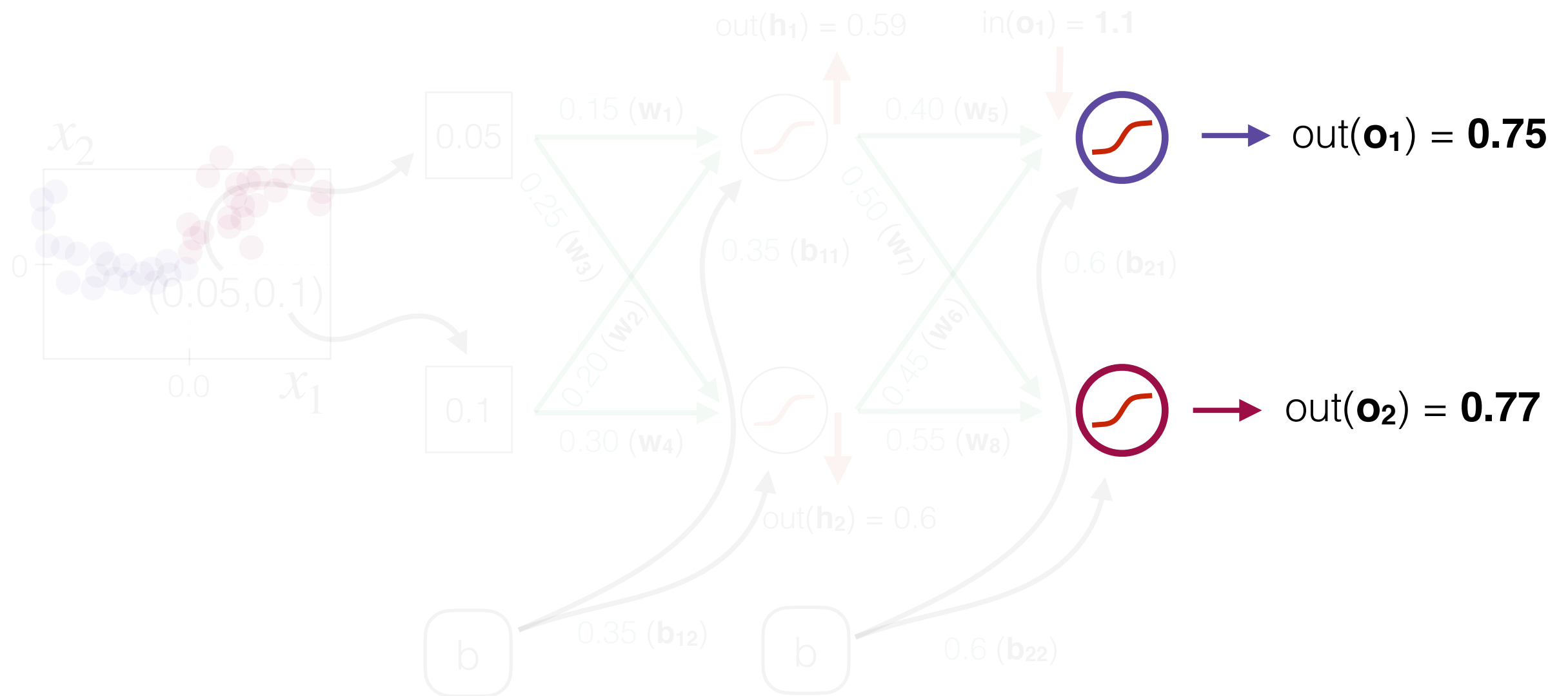


Often in practice, feed-forward path is
performed using **matrices**

Let's calculate **class scores** for each class
using **feed-forward path** algorithm

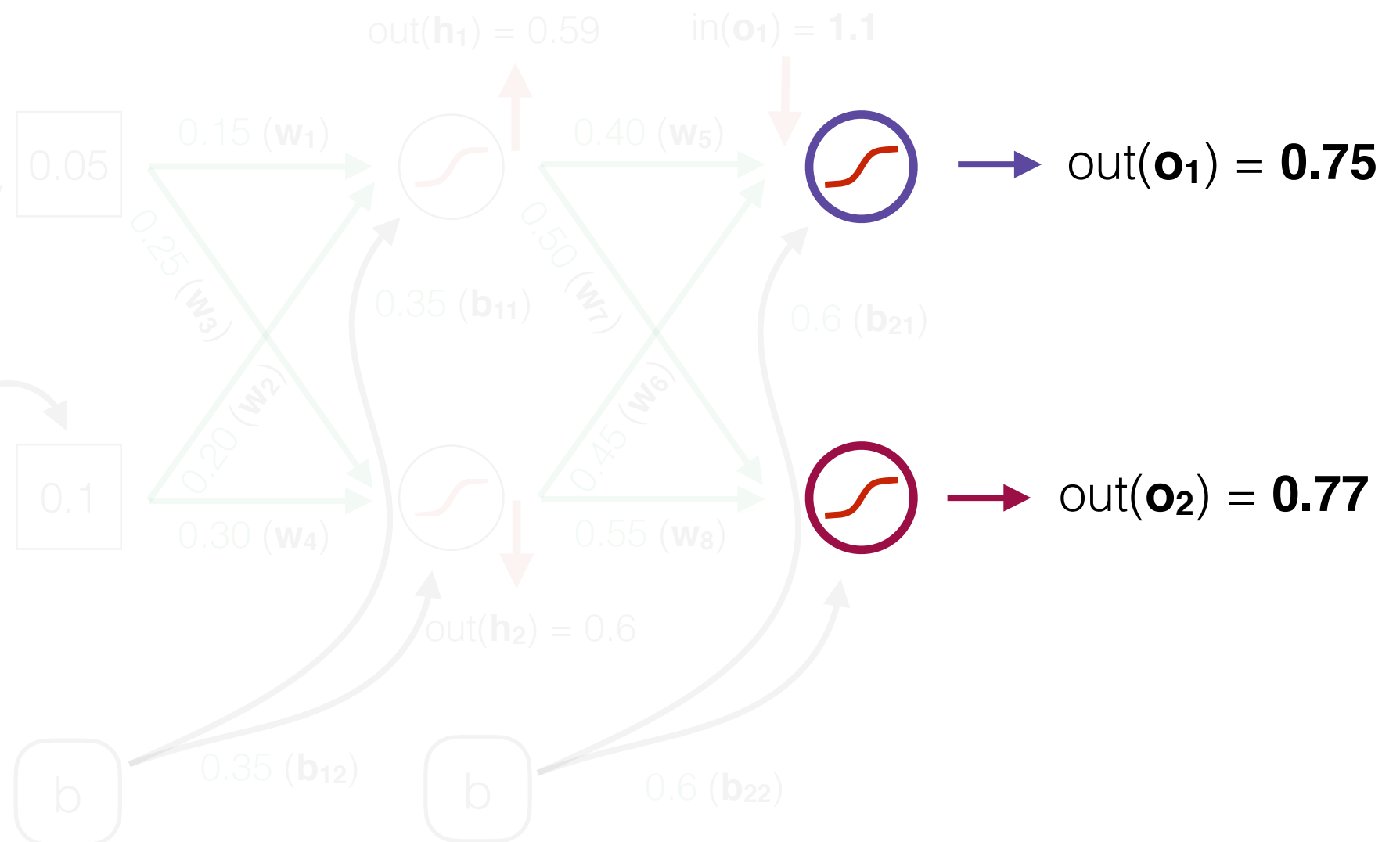
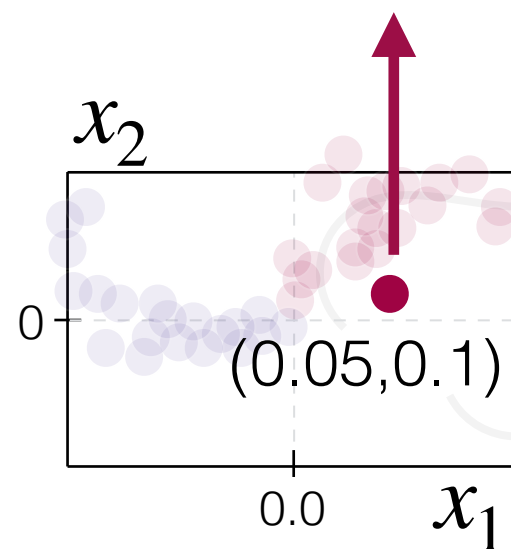


How good are the class **predictions** (scores)?



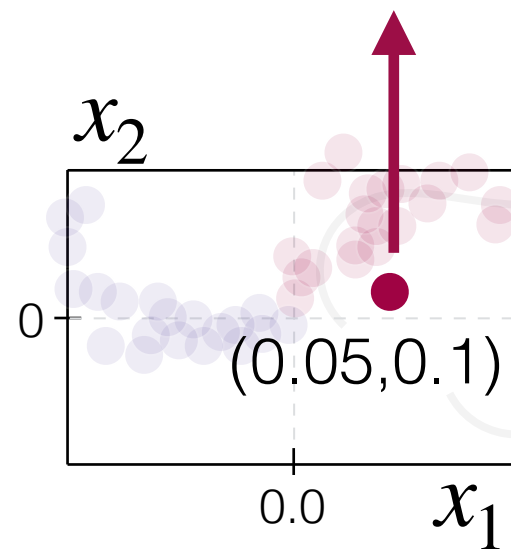
How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



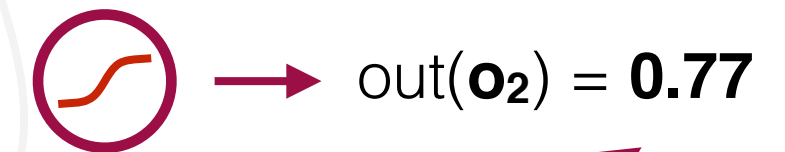
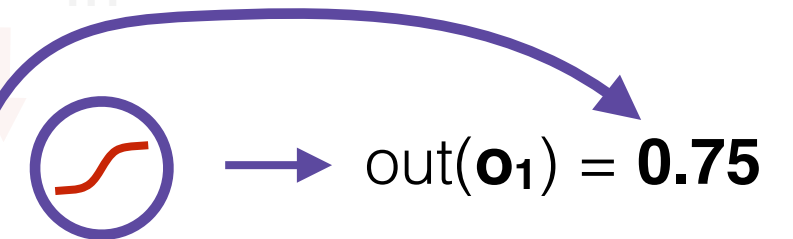
How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Therefore for this
point we **expect**

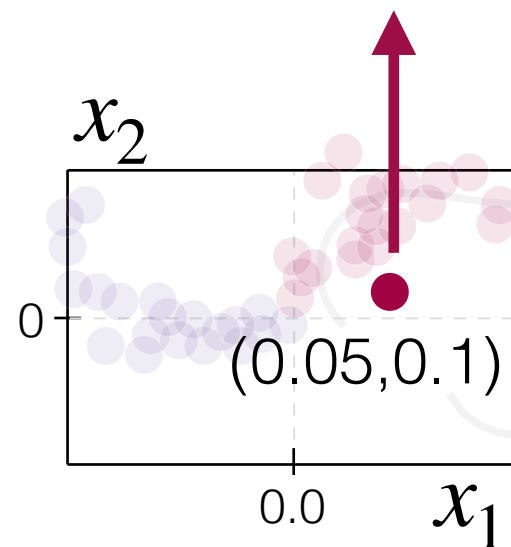
out(**o**₁) to be
very close to **0**



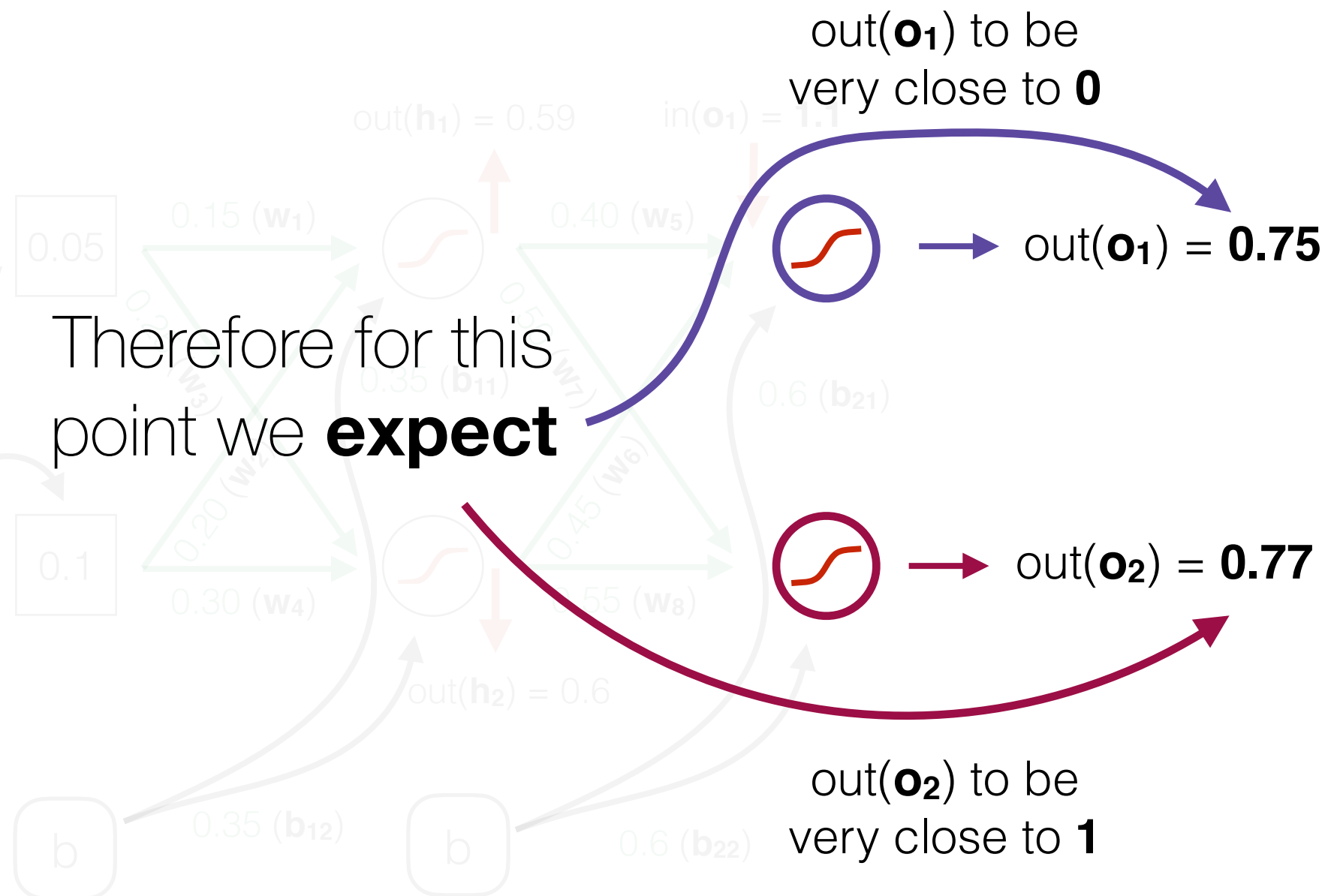
out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Therefore for this
point we **expect**

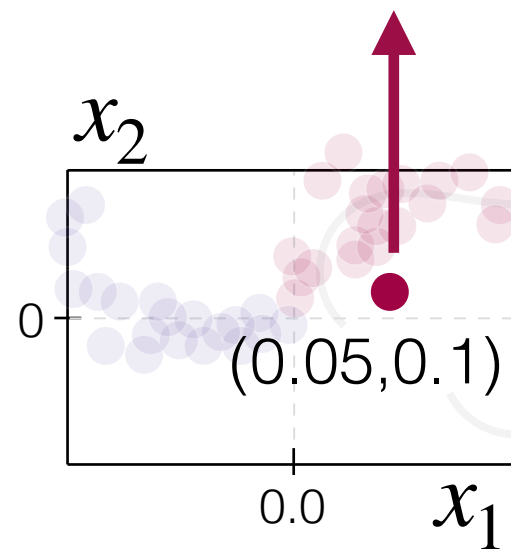


These expected values for each input point are stored in
a variable called **ground truth** or **truth** for short.

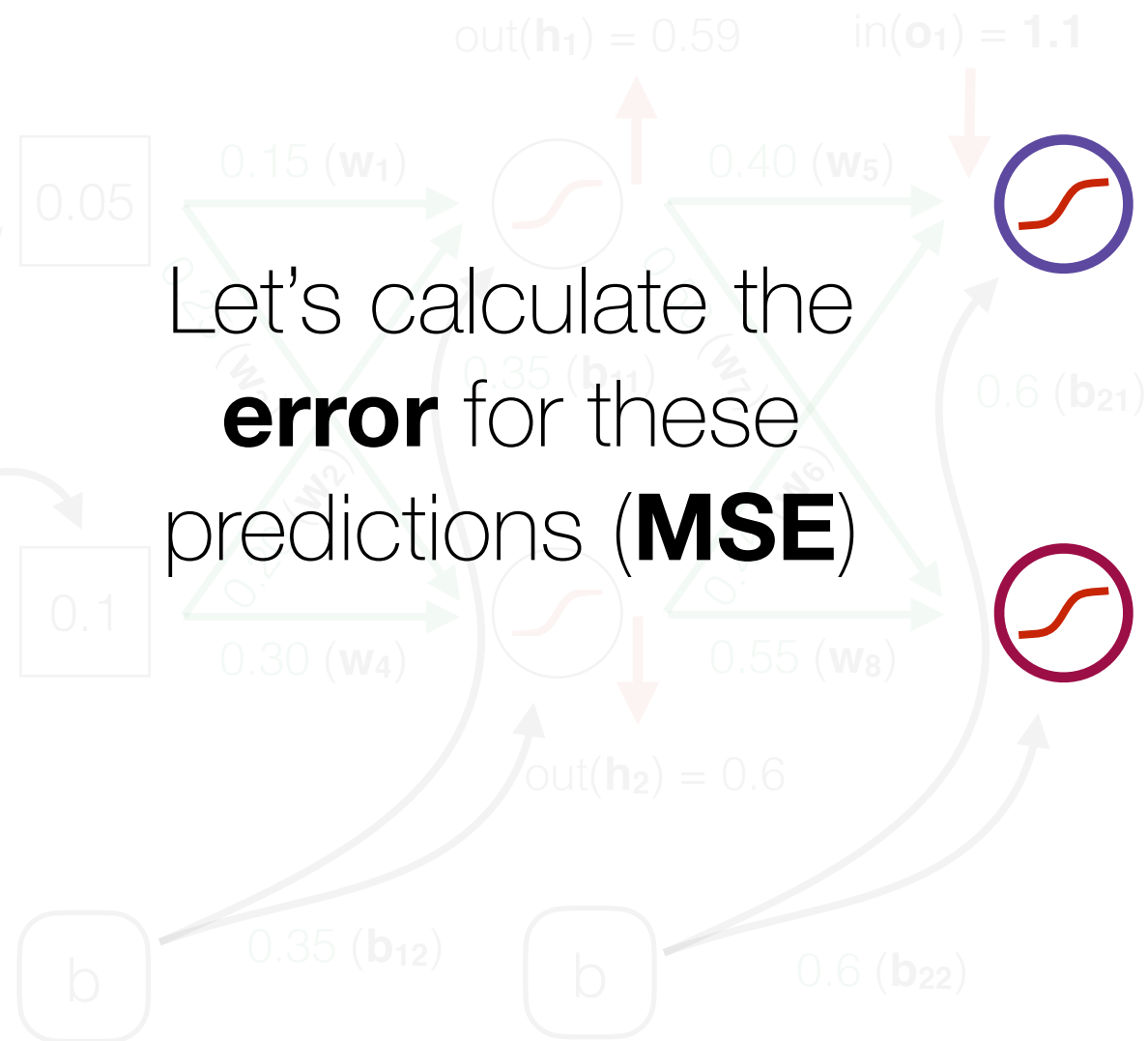
These are **labels** in supervised learning terms.

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)



out(**o**₁) to be
very close to **0**

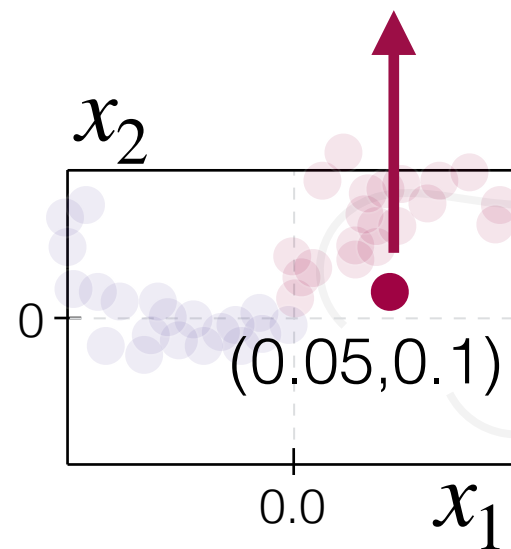
→ out(**o**₁) = **0.75**

→ out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = E_{o_1} + E_{o_2}$$

out(**o**₁) to be
very close to **0**

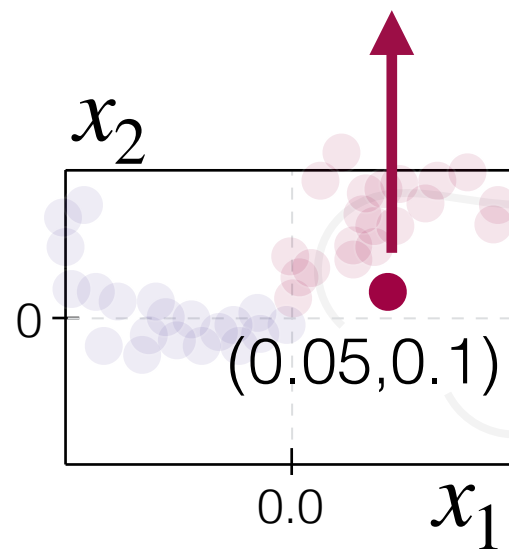
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



out(**o**₁) to be
very close to **0**

Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} =$$

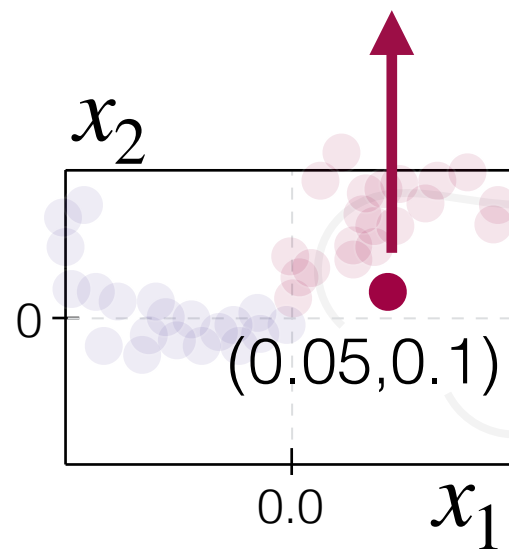
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

out(**o**₁) to be
very close to **0**

out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

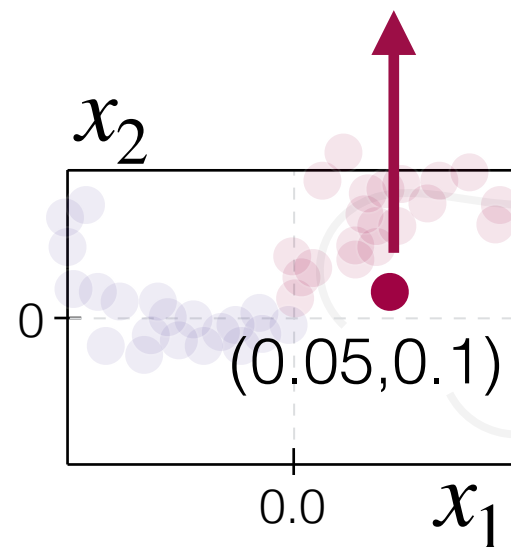
out(**o**₂) to be
very close to **1**

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = \frac{1}{2} (truth - out_{o_1})^2$$

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

out(**o_1**) to be
very close to **0**

out(**o_1**) = **0.75**

out(**o_2**) = **0.77**

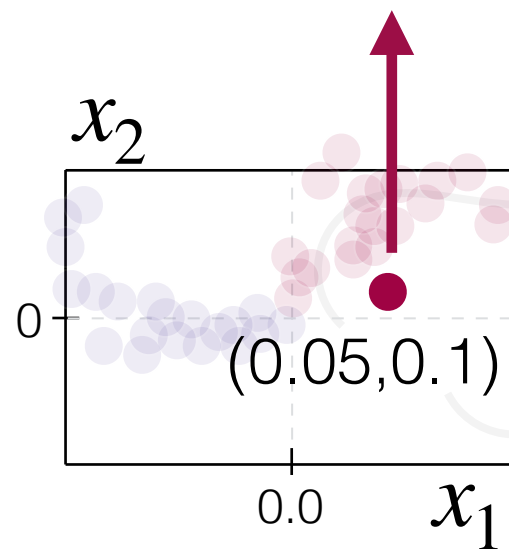
out(**o_2**) to be
very close to **1**

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = \frac{1}{2}(\text{truth} - \text{out}_{o_1})^2$$

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

out(**o**₁) to be
very close to **0**

out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

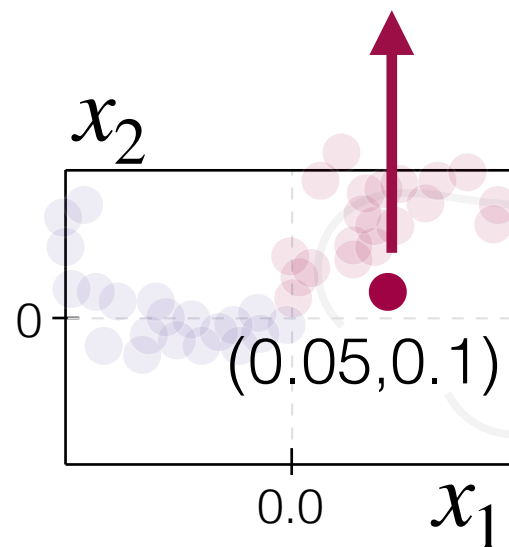
out(**o**₂) to be
very close to **1**

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = \frac{1}{2}(\text{truth} - 0.75)^2$$

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

out(**o**₁) to be
very close to **0**

out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

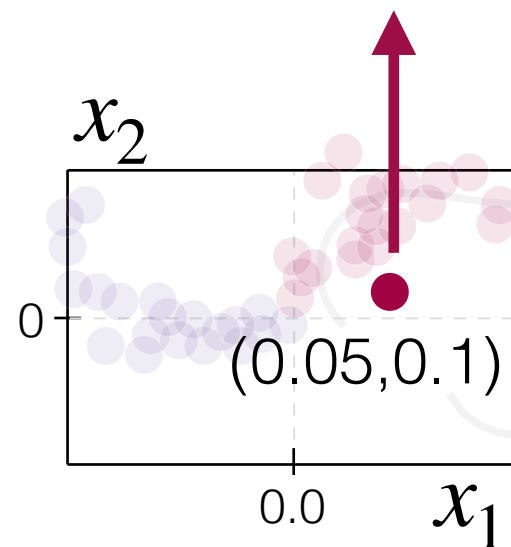
out(**o**₂) to be
very close to **1**

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = \frac{1}{2}(0 - 0.75)^2$$

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = E_{o_1} + E_{o_2}$$

$$E_{o_1} = 0.28$$

out(**o**₁) to be
very close to **0**

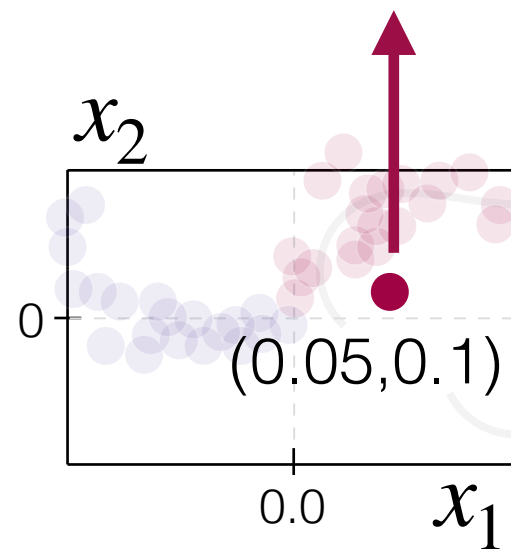
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

out(**o_1**) to be
very close to **0**

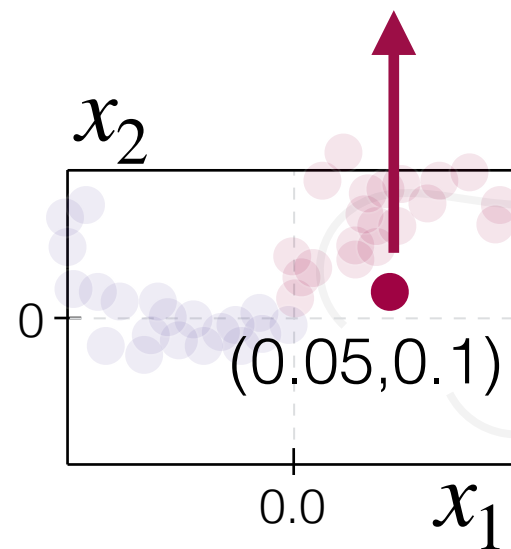
out(**o_1**) = **0.75**

out(**o_2**) = **0.77**

out(**o_2**) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

Error on the **first**
output neuron

out(**o_1**) to be
very close to **0**

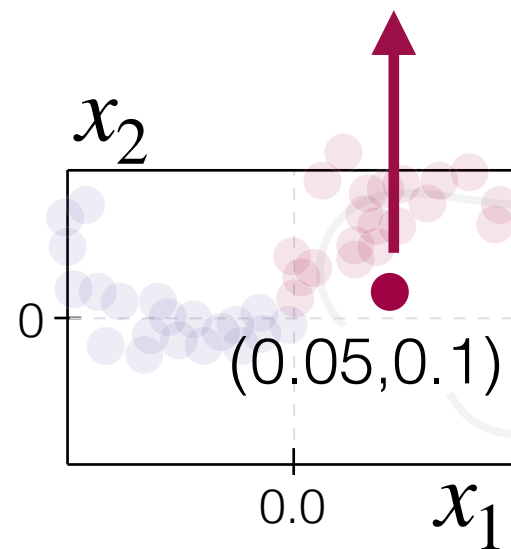
out(**o_1**) = **0.75**

out(**o_2**) = **0.77**

out(**o_2**) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

out(**o**₁) to be
very close to **0**

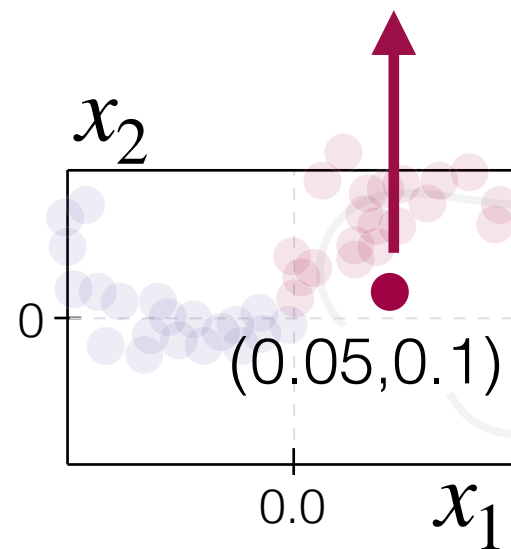
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

$$E_{o_2} = \frac{1}{2}(\text{truth} - \text{out}_{o_2})^2$$

out(**o**₁) to be
very close to **0**

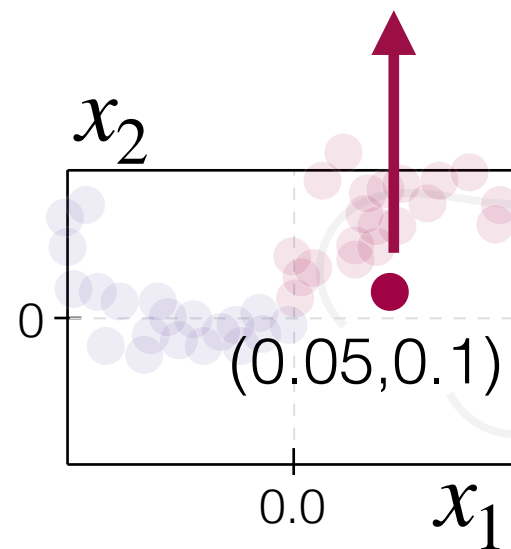
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

$$E_{o_2} = \frac{1}{2}(1 - 0.77)^2$$

out(**o**₁) to be
very close to **0**

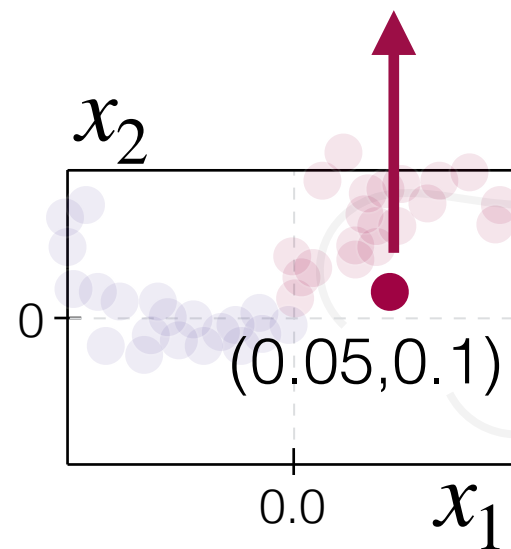
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + E_{o_2}$$

$$E_{o_2} = 0.025$$

out(**o**₁) to be
very close to **0**

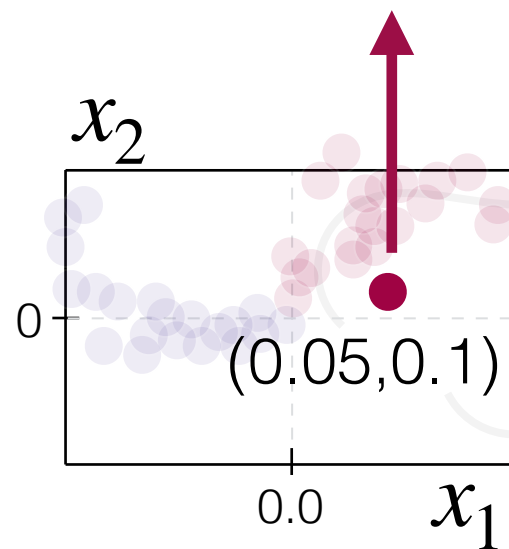
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

$$E_{total} = 0.28 + 0.025$$

out(**o**₁) to be
very close to **0**

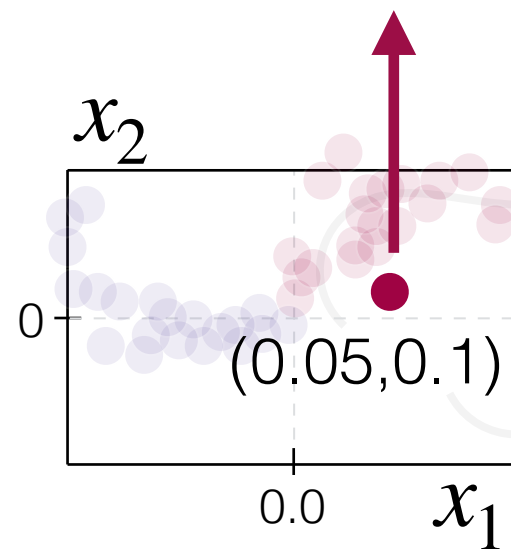
out(**o**₁) = **0.75**

out(**o**₂) = **0.77**

out(**o**₂) to be
very close to **1**

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

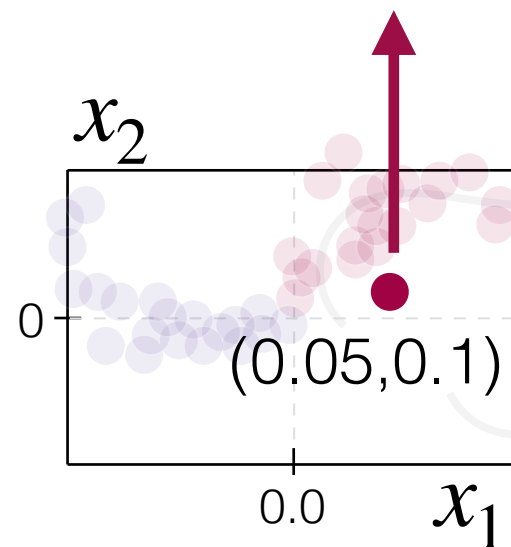
$$E_{total} = 0.28 + 0.025$$

prediction	error
0.75	0.28
truth = 0	

prediction	error
0.77	0.025
truth = 1	

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

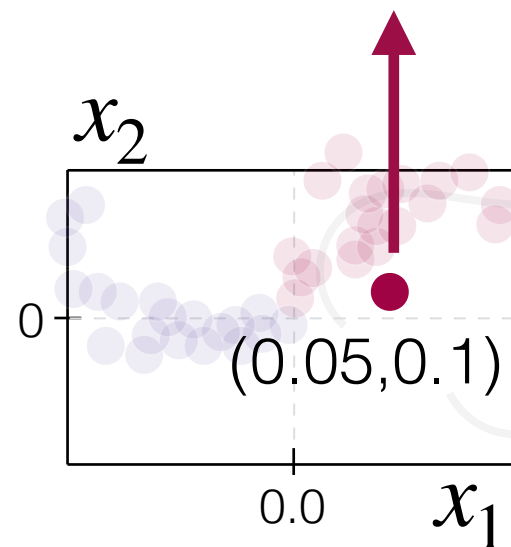
$$E_{total} = 0.305$$

prediction	error
0.75	0.28
truth = 0	

prediction	error
0.77	0.025
truth = 1	

How good are the class **predictions** (scores)?

Our **input** point
belongs to the
second class



Let's calculate the
error for these
predictions (**MSE**)

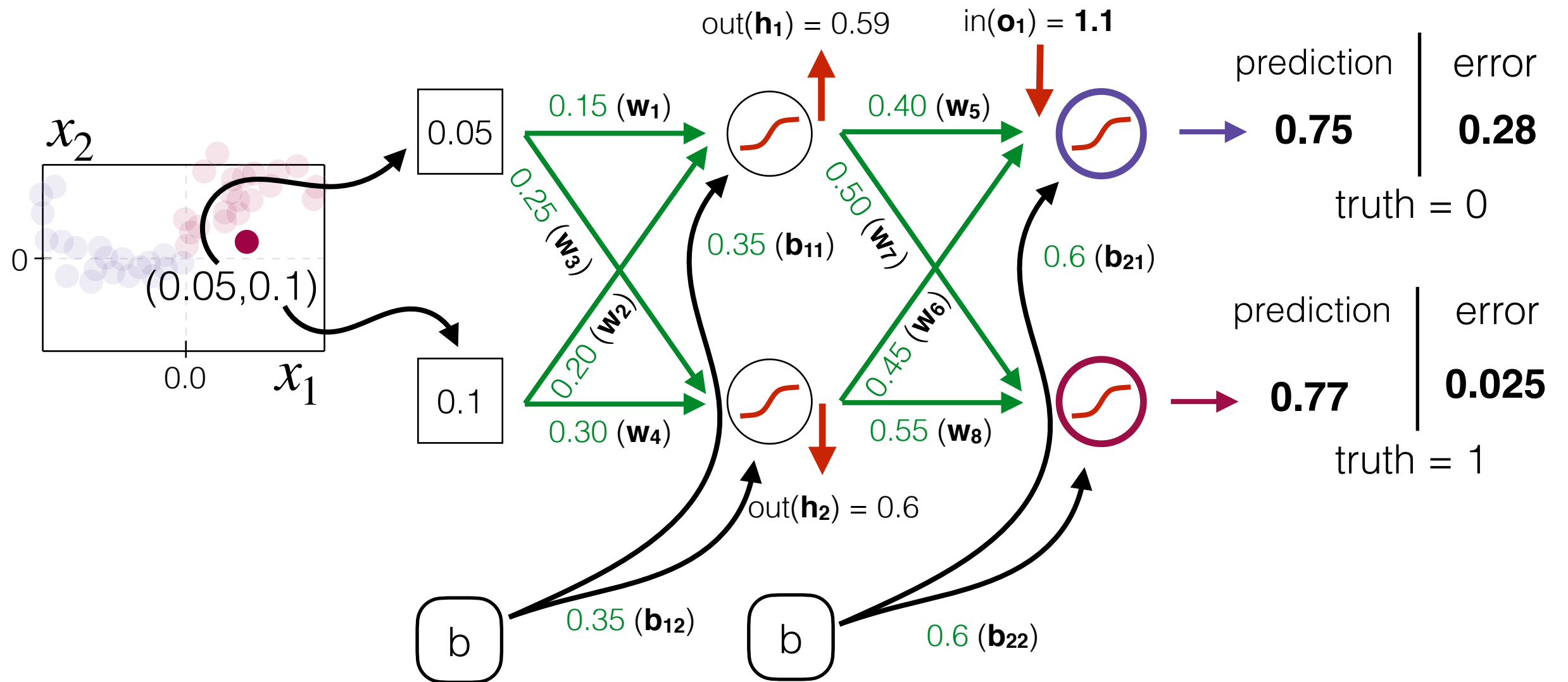
$$E_{total} = 0.305$$

What can we do to make the error **smaller**?

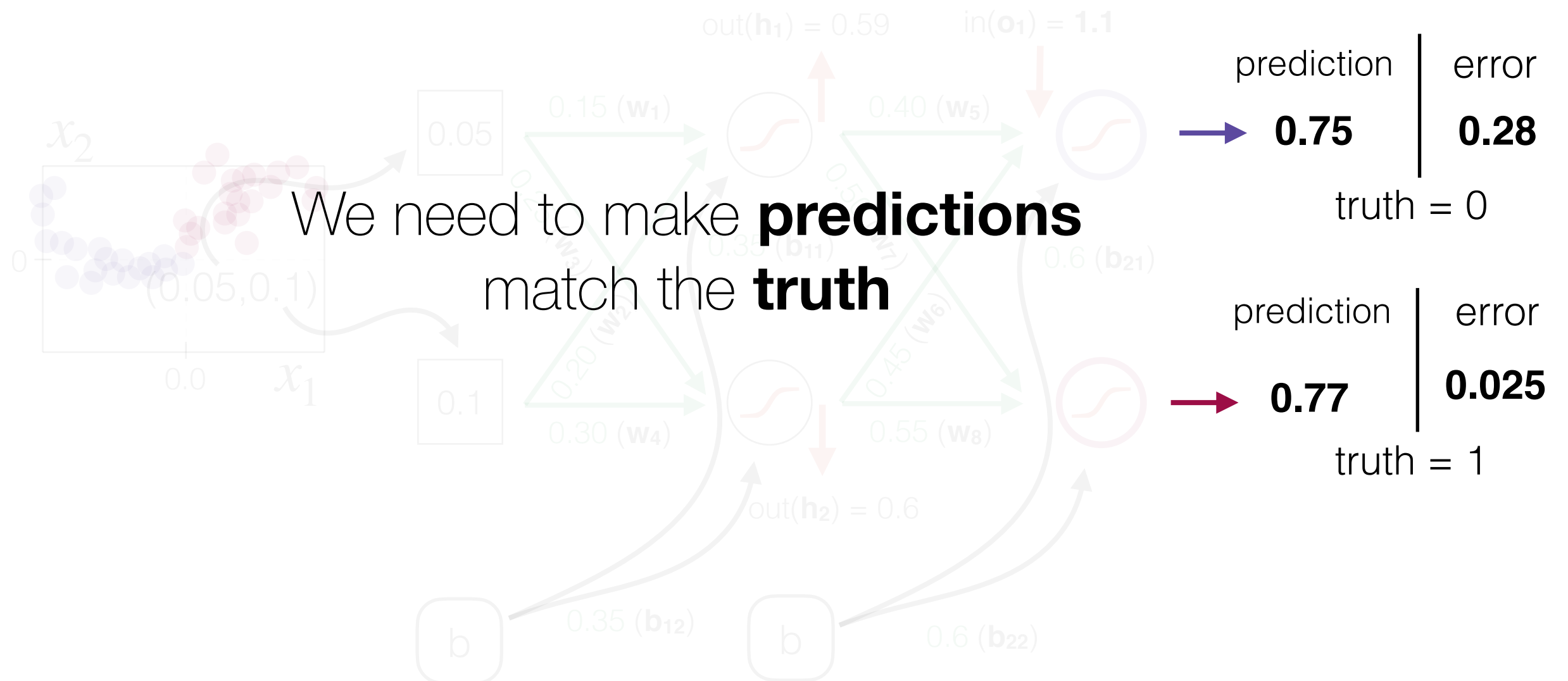
prediction | error
0.75 | **0.28**
truth = 0

prediction | error
0.77 | **0.025**
truth = 1

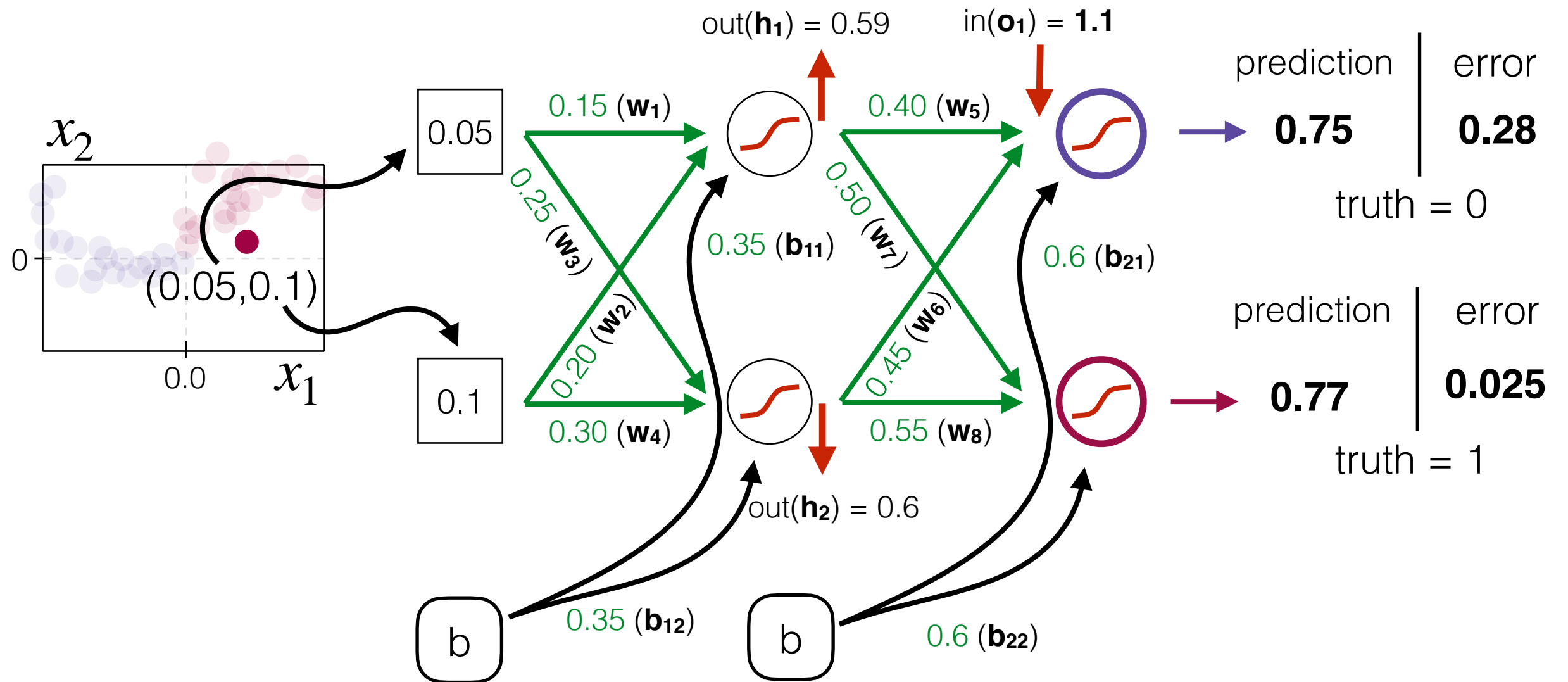
What can we do to make the error **smaller**?



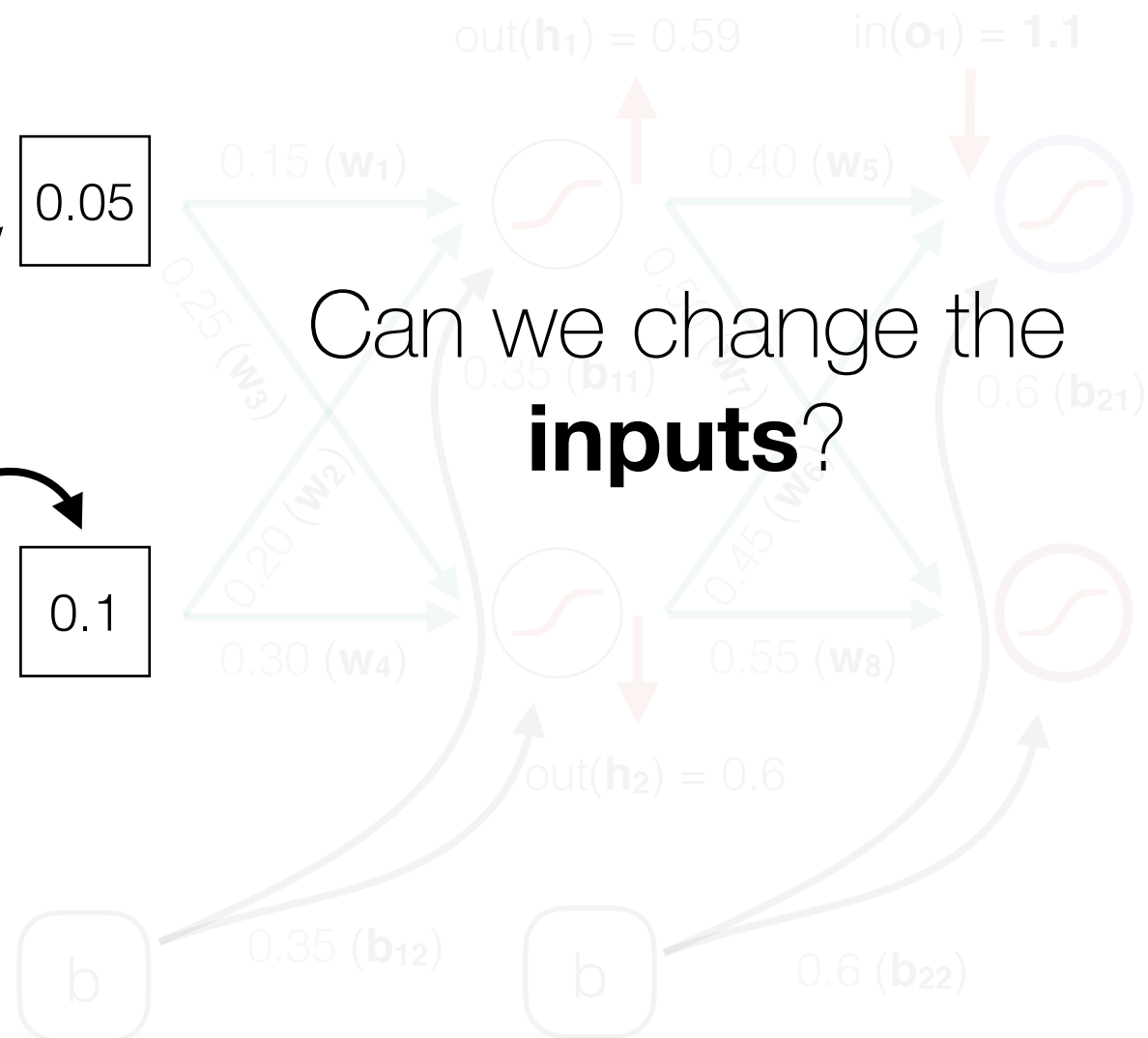
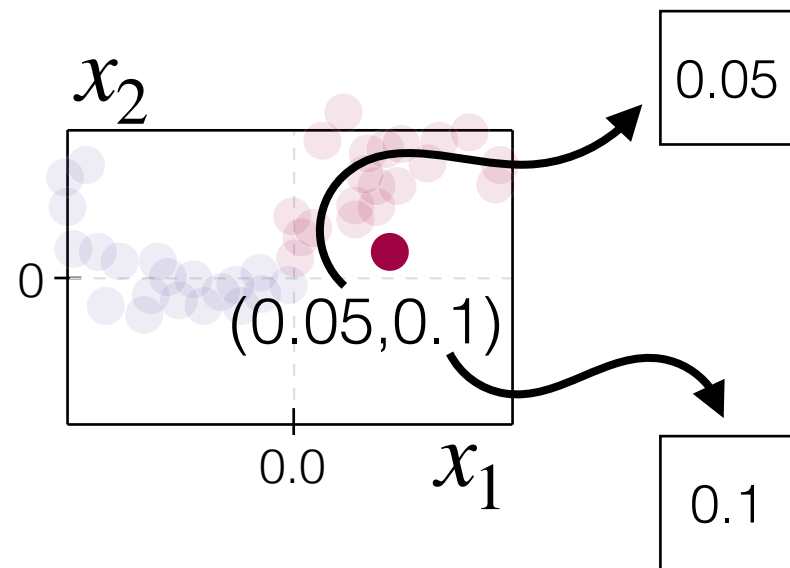
What can we do to make the error **smaller**?



What can we do to make the error **smaller**?



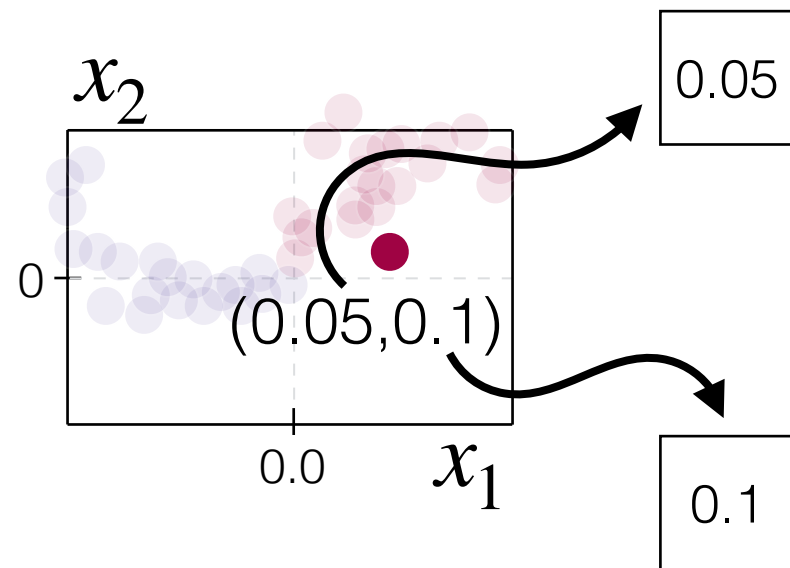
What can we do to make the error **smaller**?



prediction	error
0.75	0.28
truth = 0	

prediction	error
0.77	0.025
truth = 1	

What can we do to make the error **smaller**?



Can we change the **inputs**?

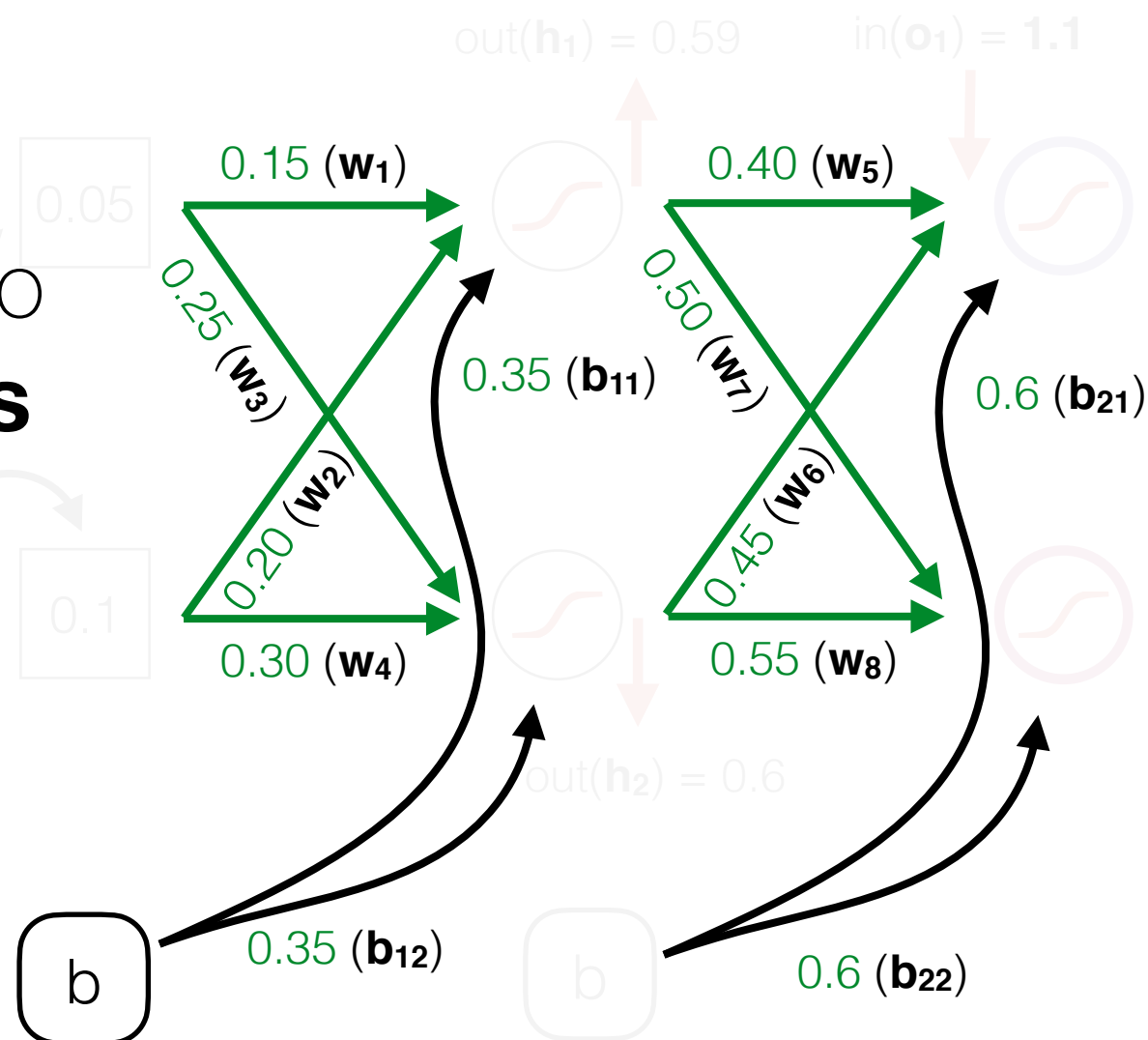
Yes, but this will compromise the data

prediction	error
0.75	0.28
truth = 0	

prediction	error
0.77	0.025
truth = 1	

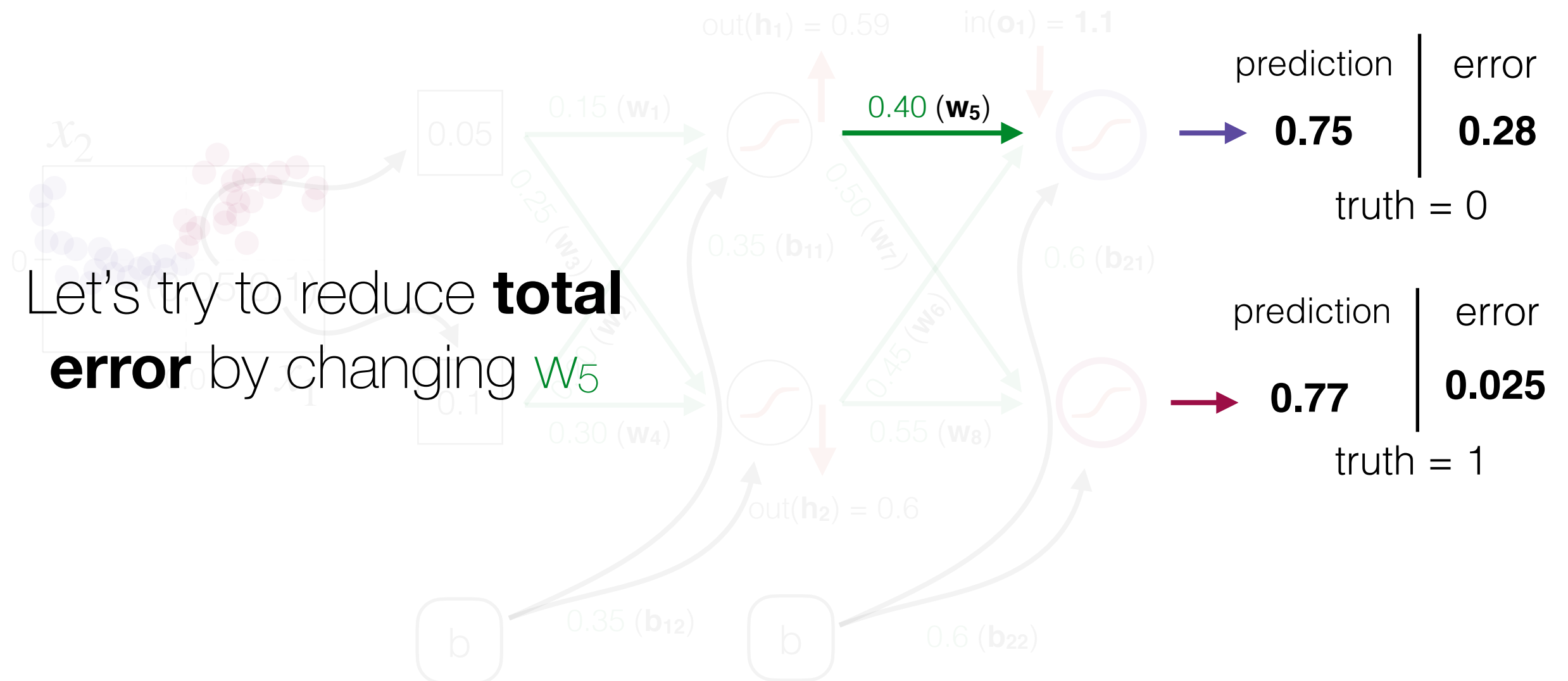
What can we do to make the error **smaller**?

The only way is to
change **weights**
and **biases**

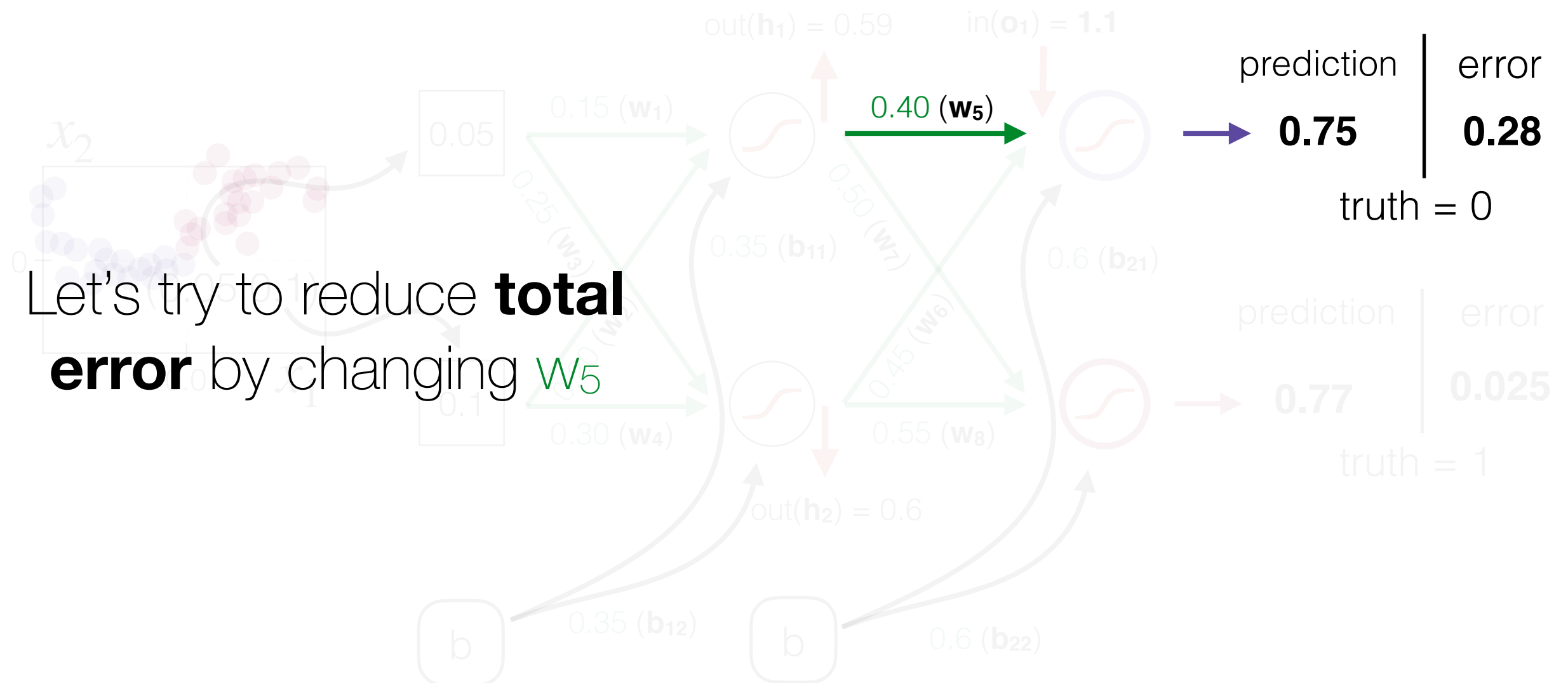


prediction	error
0.75	0.28
truth = 0	
prediction	error
0.77	0.025
truth = 1	

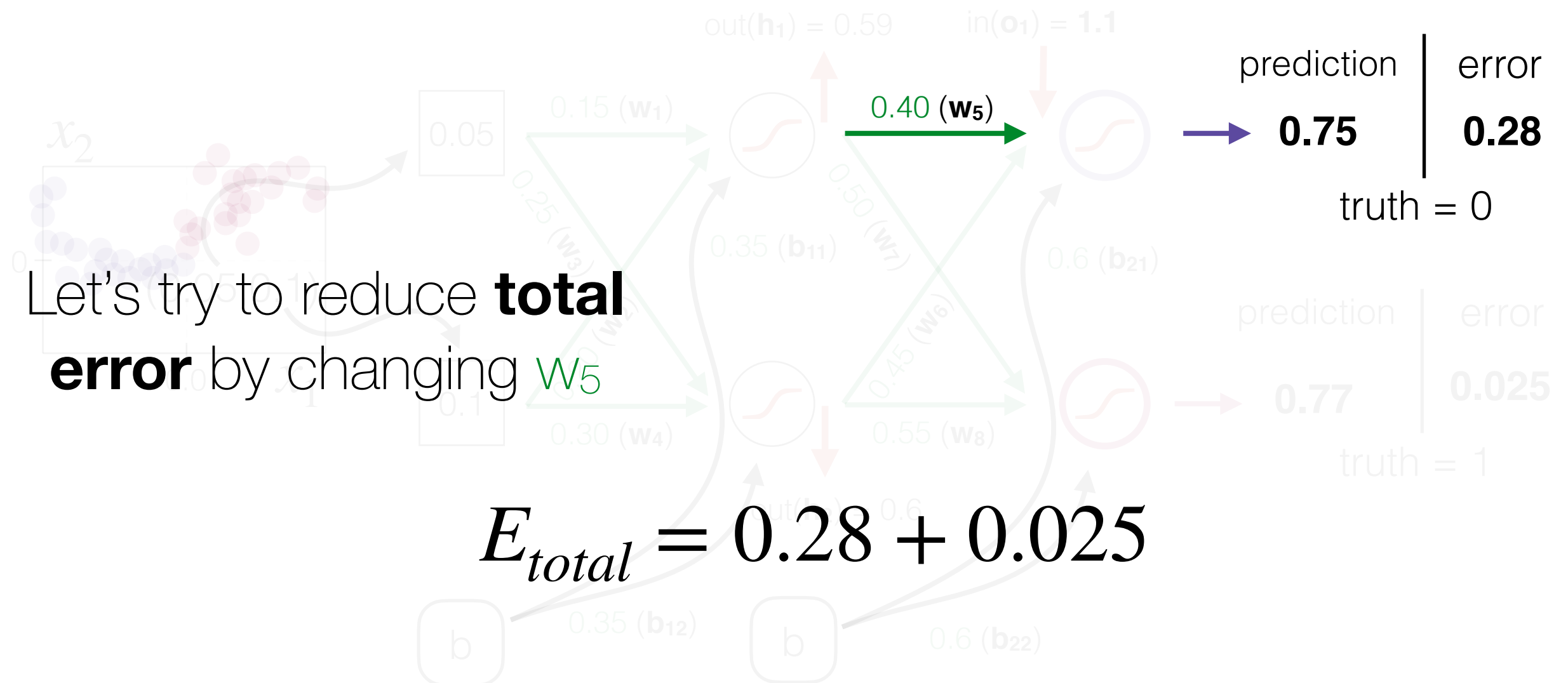
What can we do to make the error **smaller**?



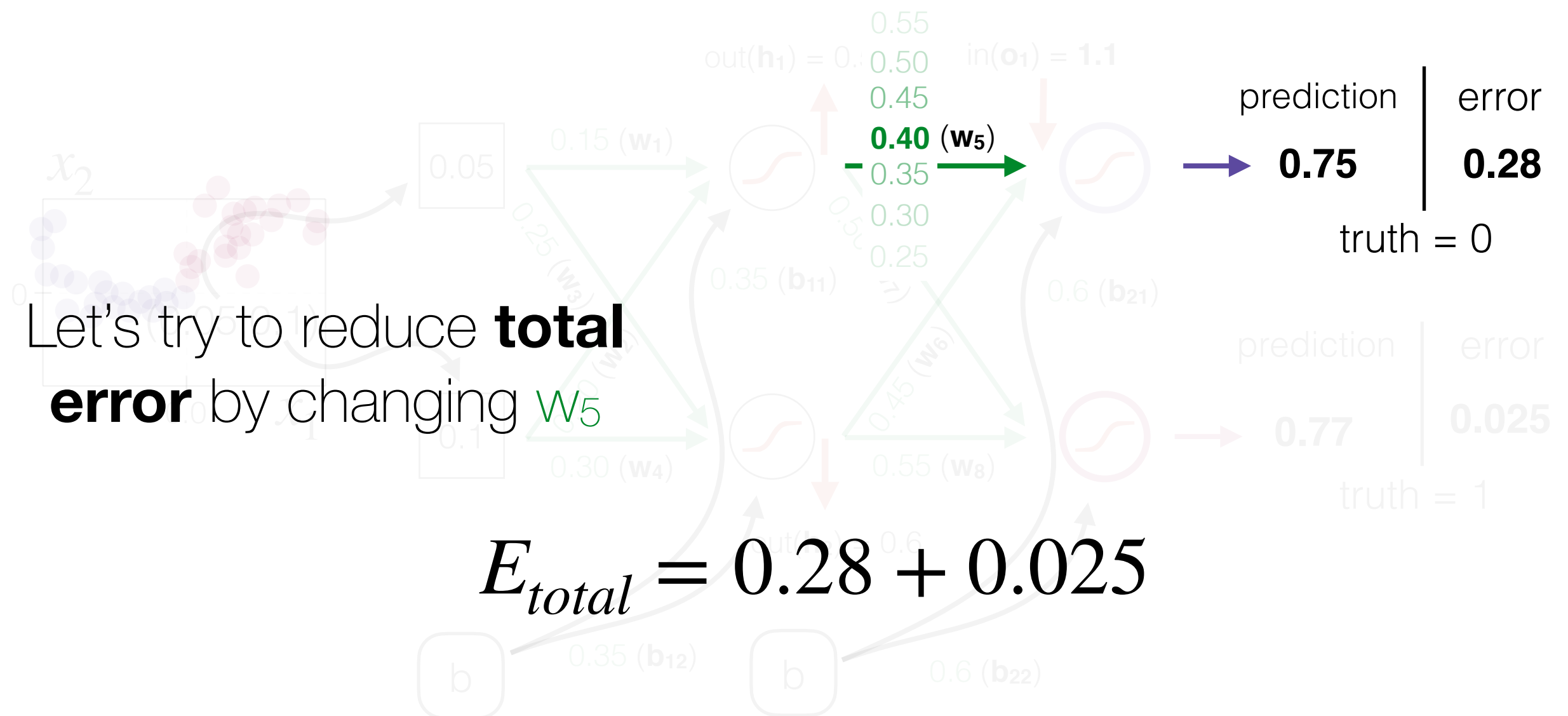
What can we do to make the error **smaller**?



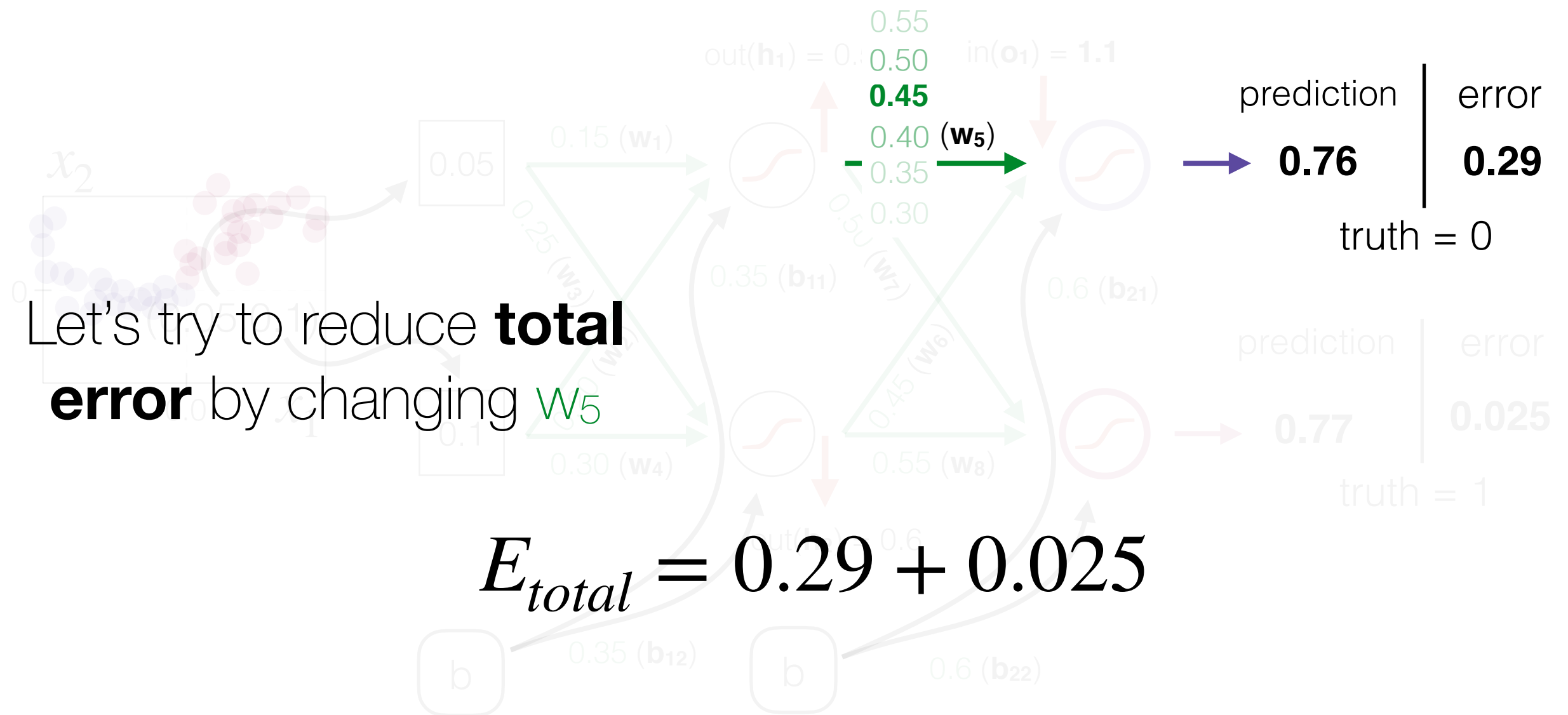
What can we do to make the error **smaller**?



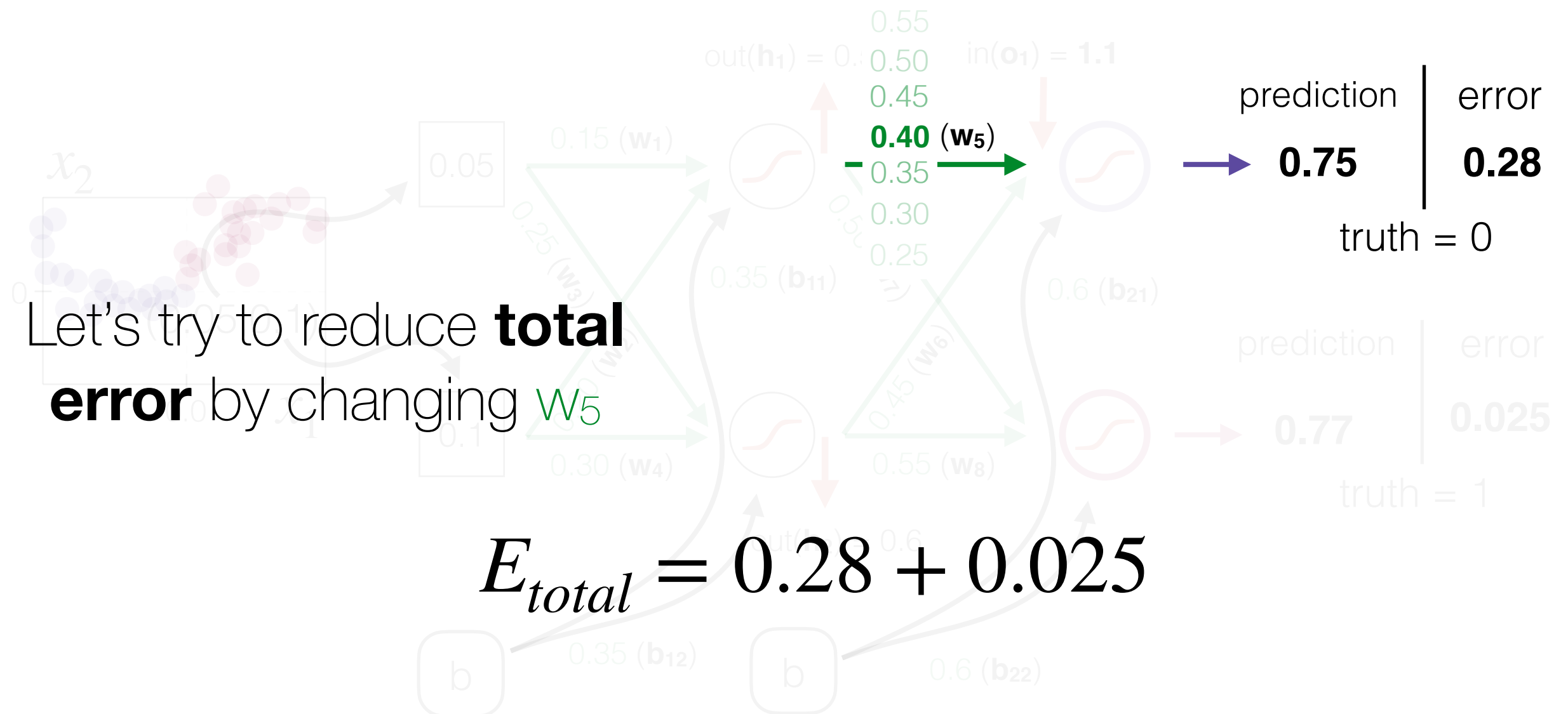
What can we do to make the error **smaller**?



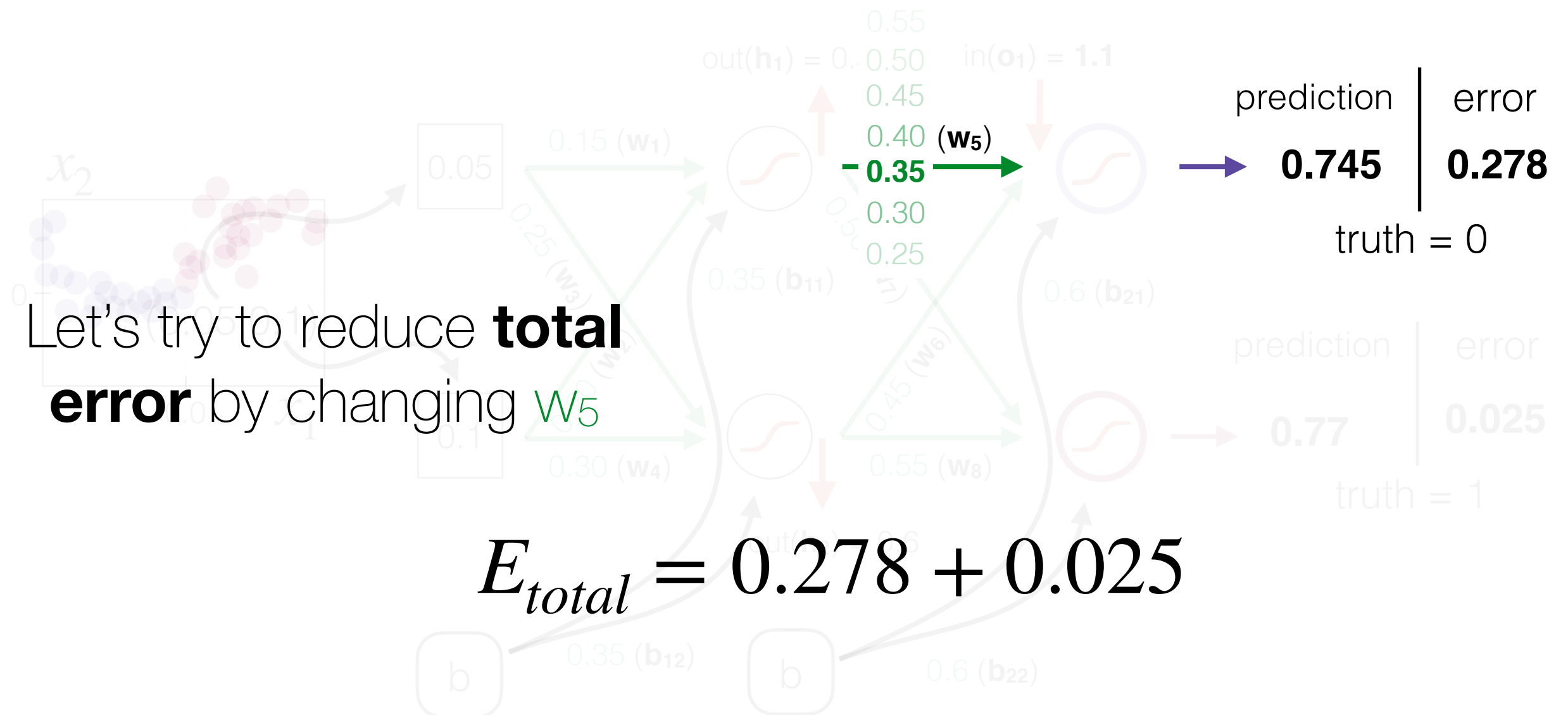
What can we do to make the error **smaller**?



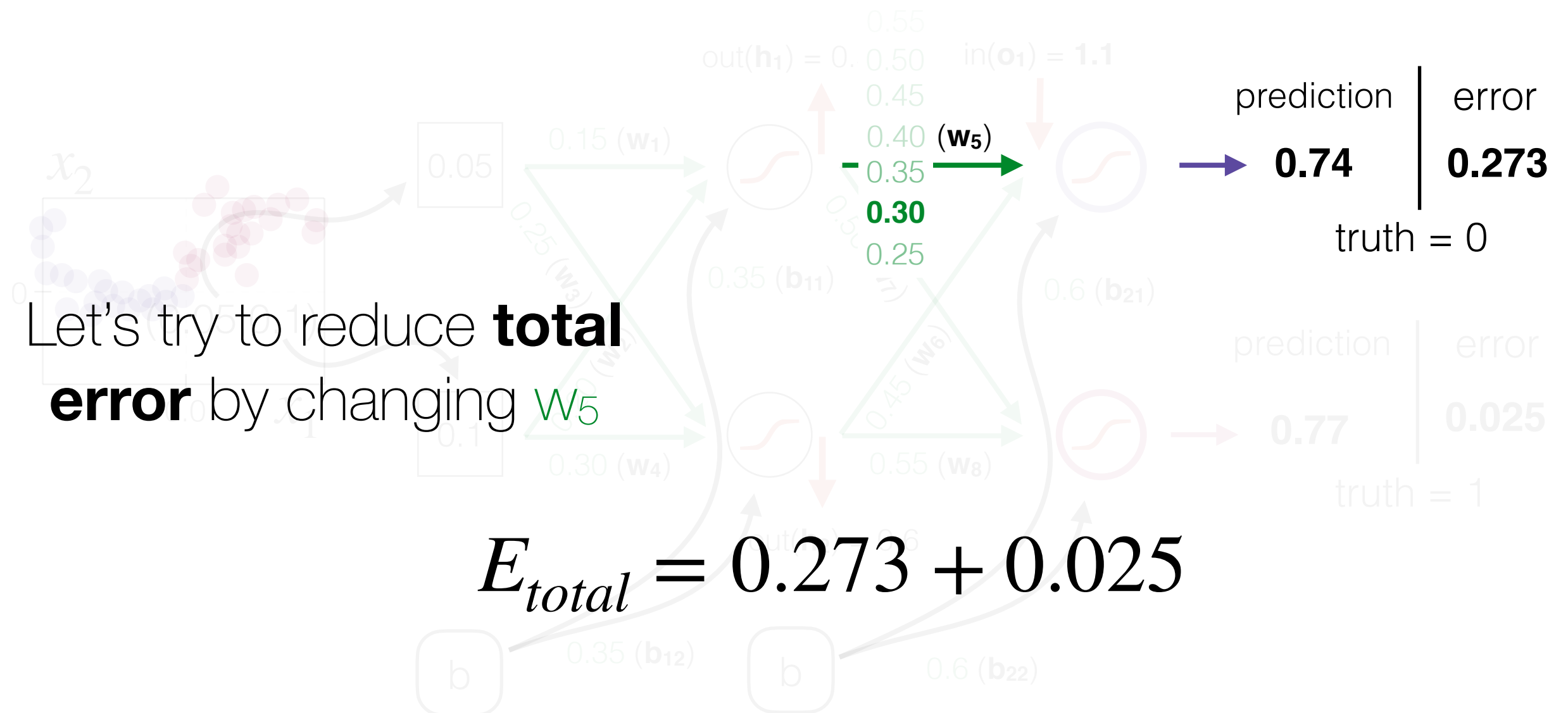
What can we do to make the error **smaller**?



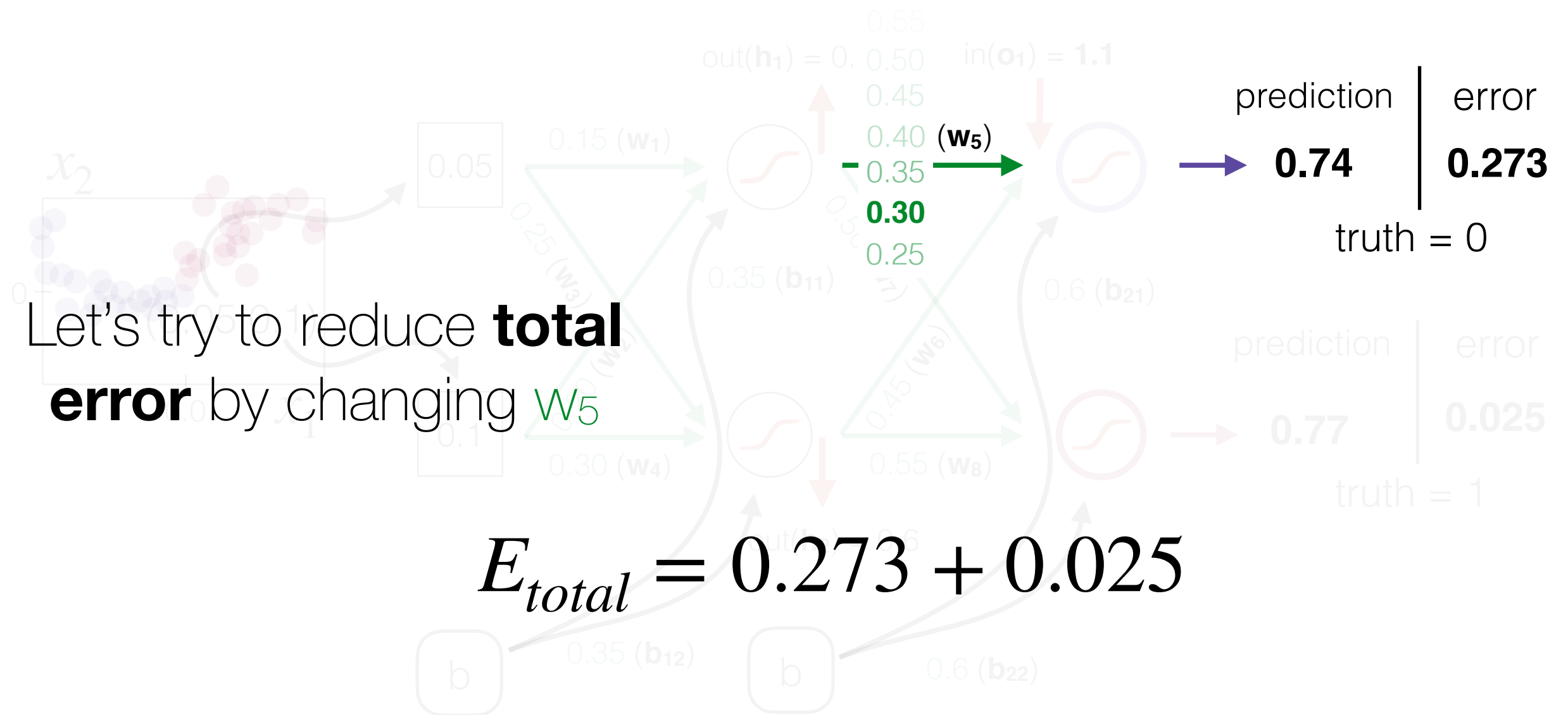
What can we do to make the error **smaller**?



What can we do to make the error **smaller**?

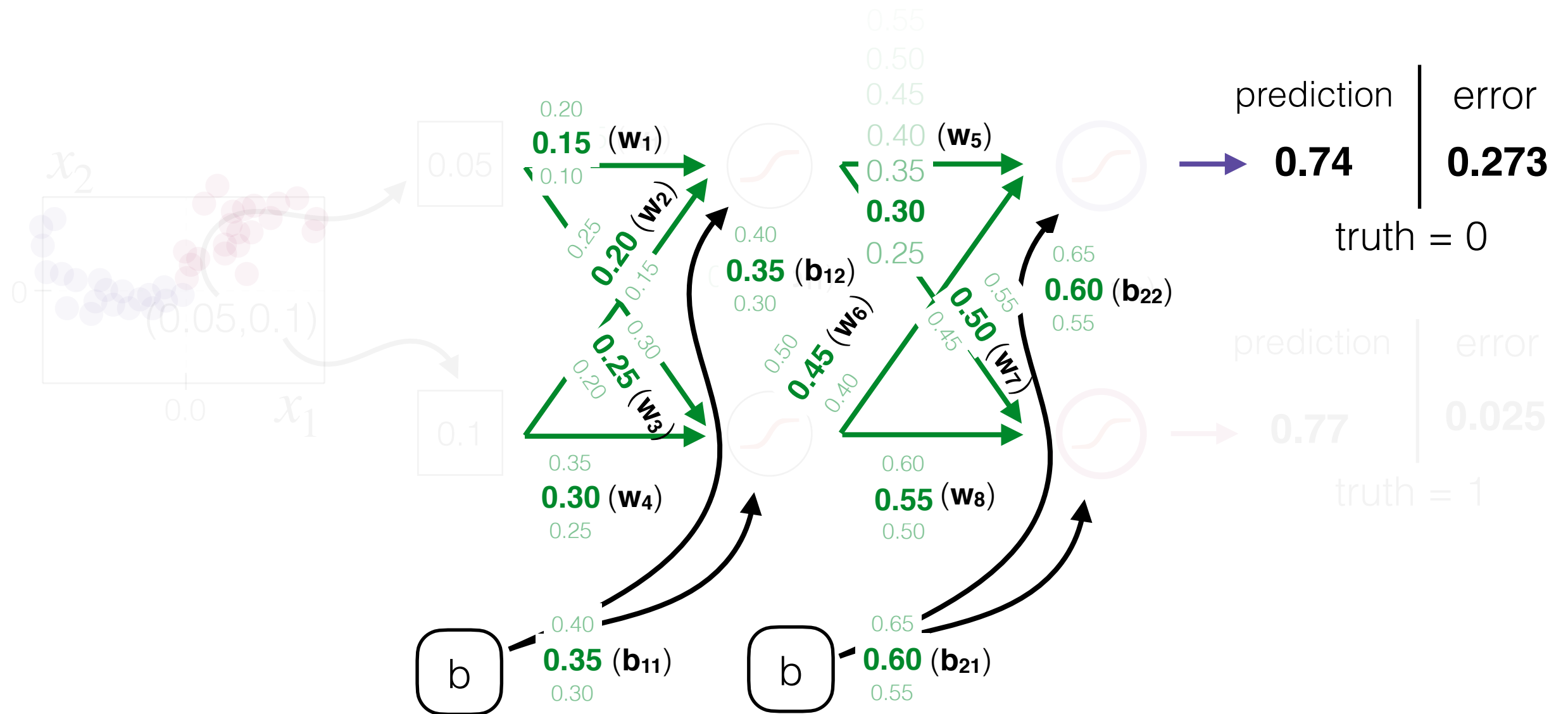


What can we do to make the error **smaller**?



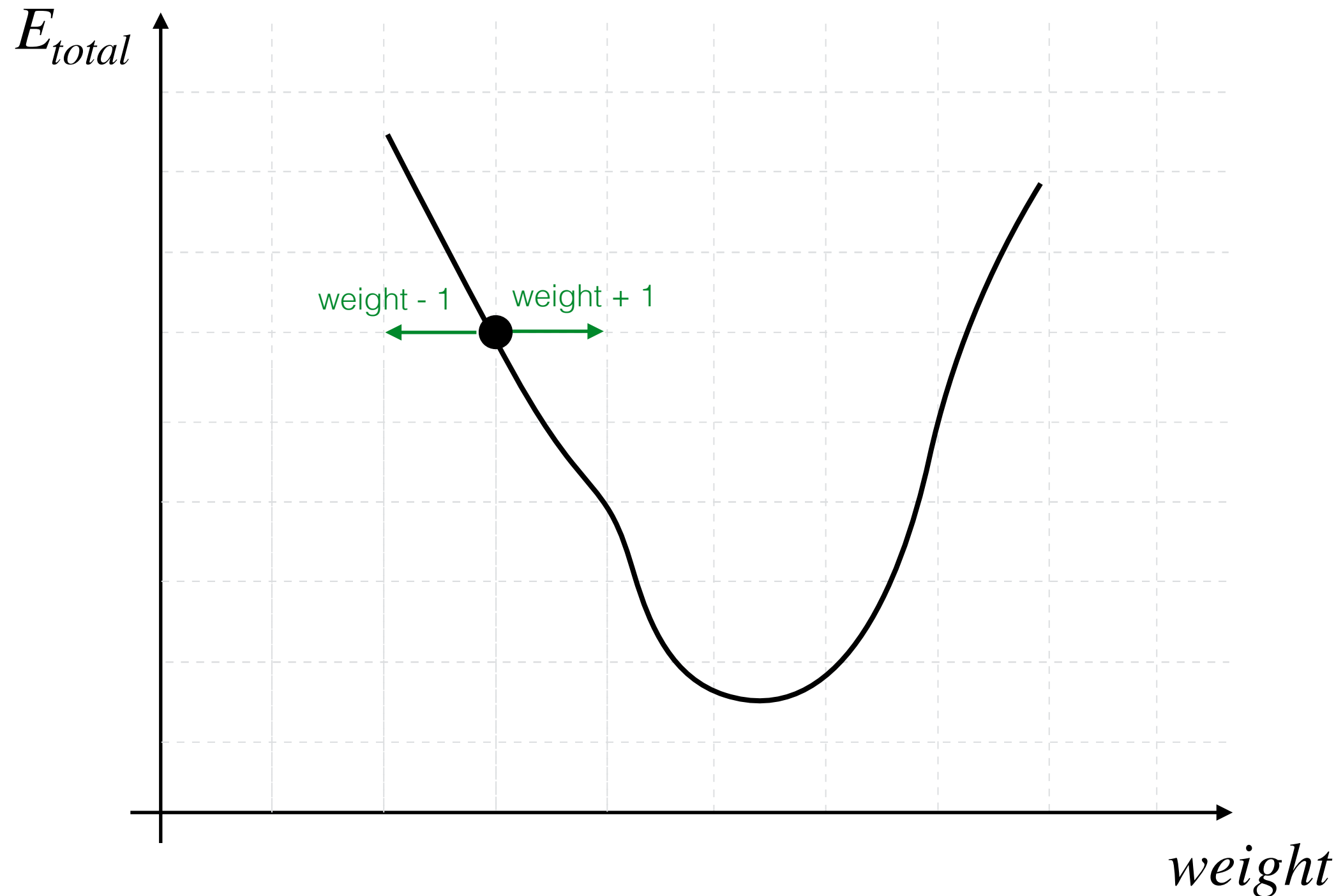
Every time we change a weight, have to recompute the **feed-forward path**

What can we do to make the error **smaller**?

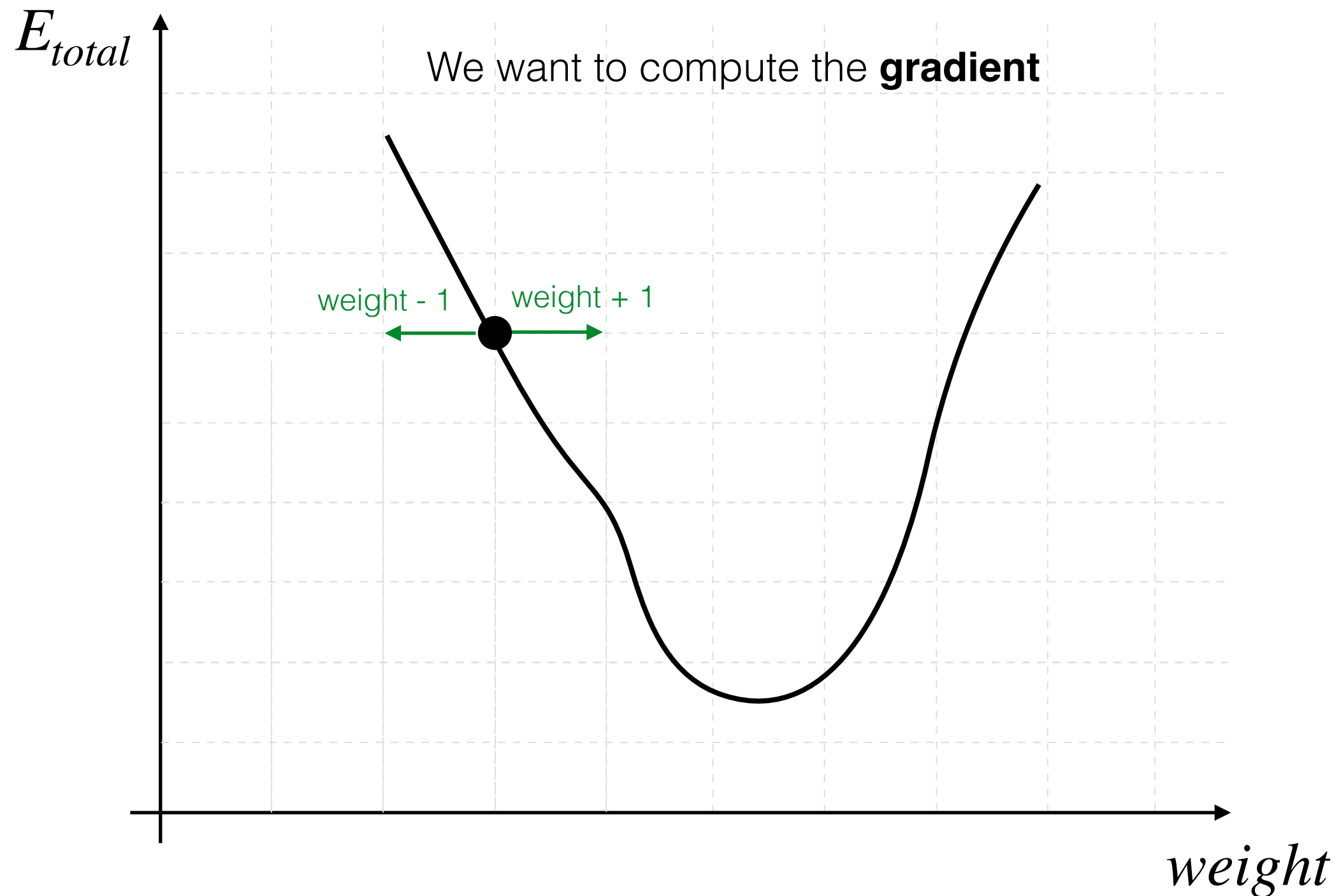


Every time we change a weight, have to recompute the **feed-forward path**

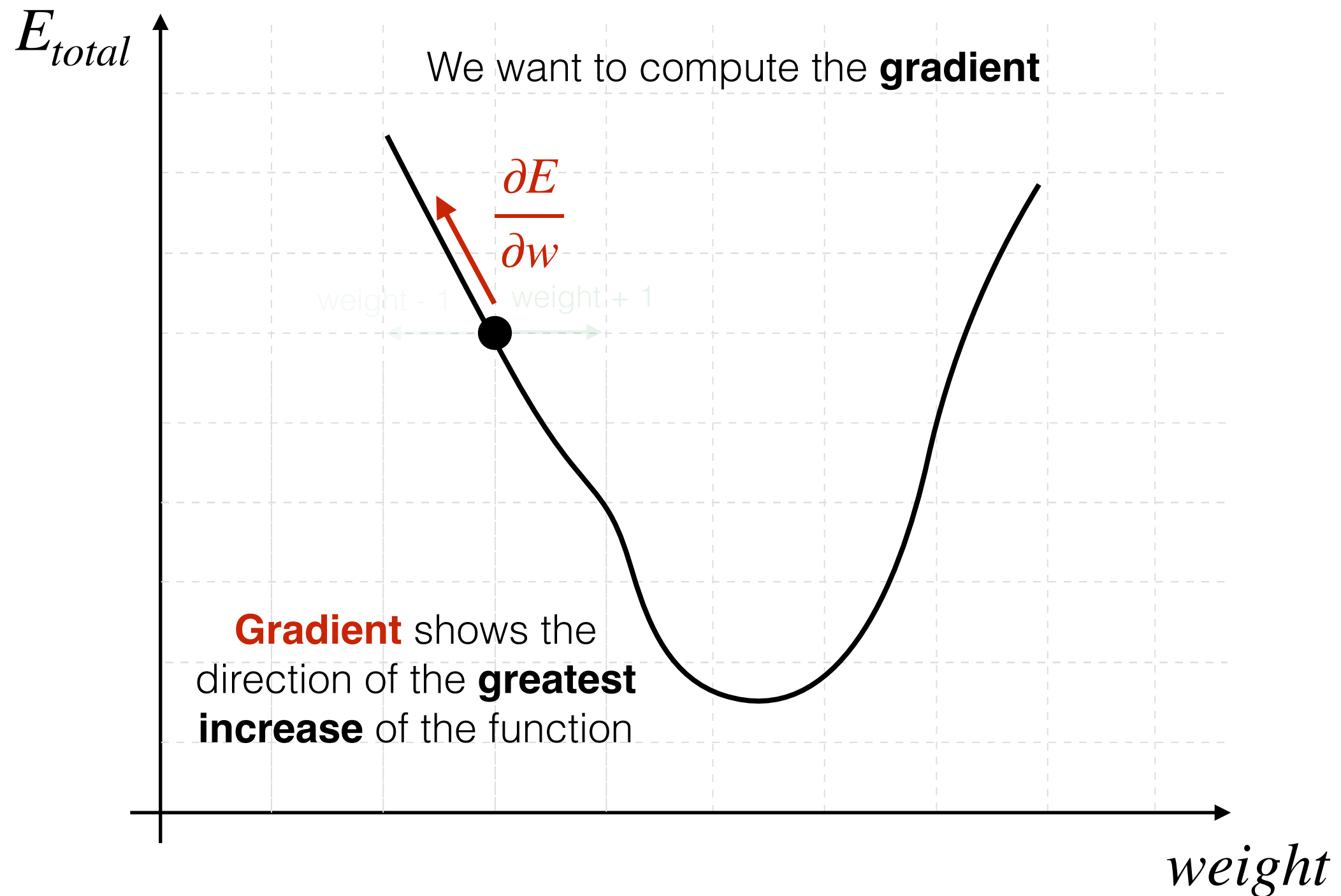
We want a way to **efficiently update all** our **weights** so that **total error decreases** most substantially



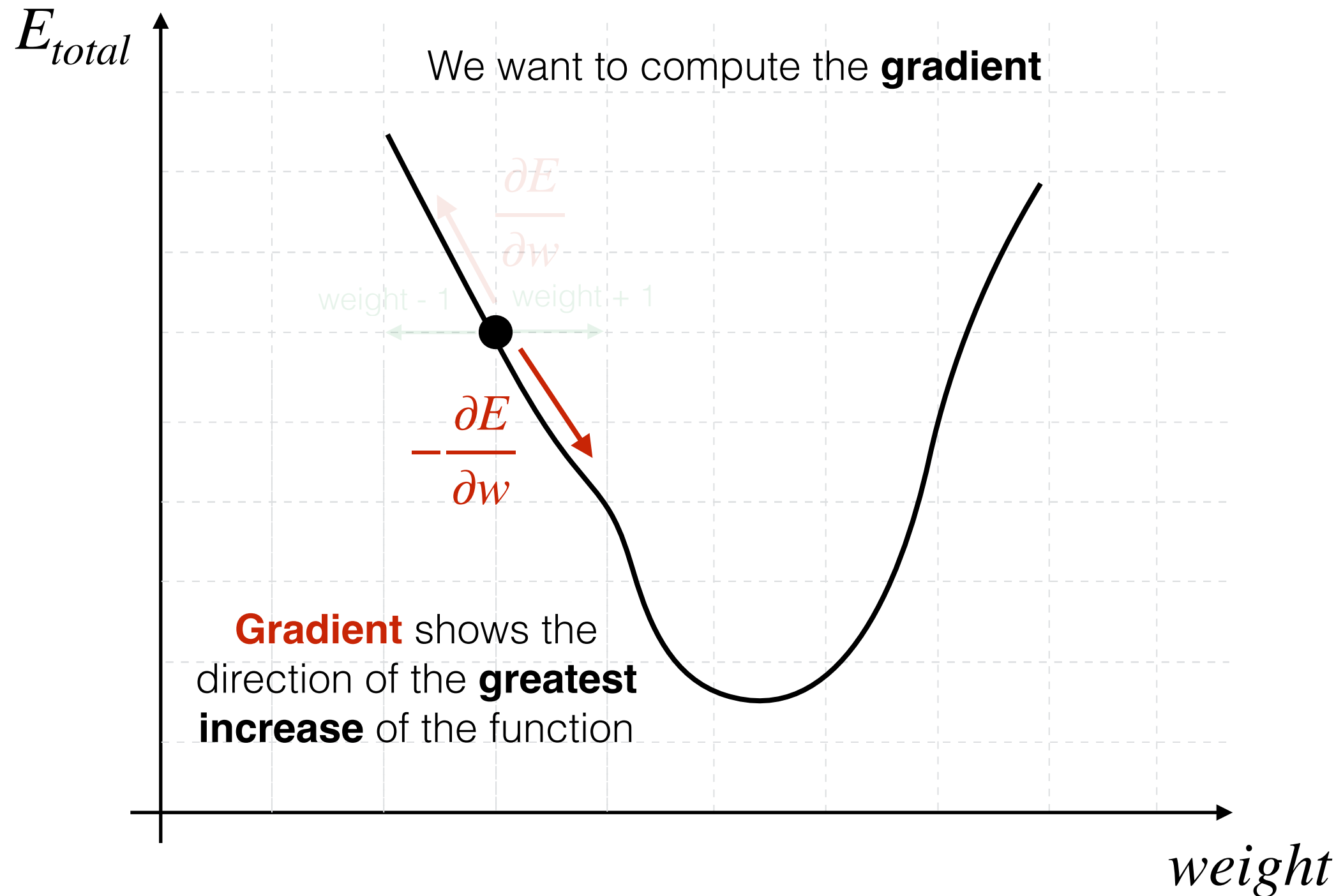
We want a way to **efficiently update all** our **weights** so that **total error decreases** most substantially



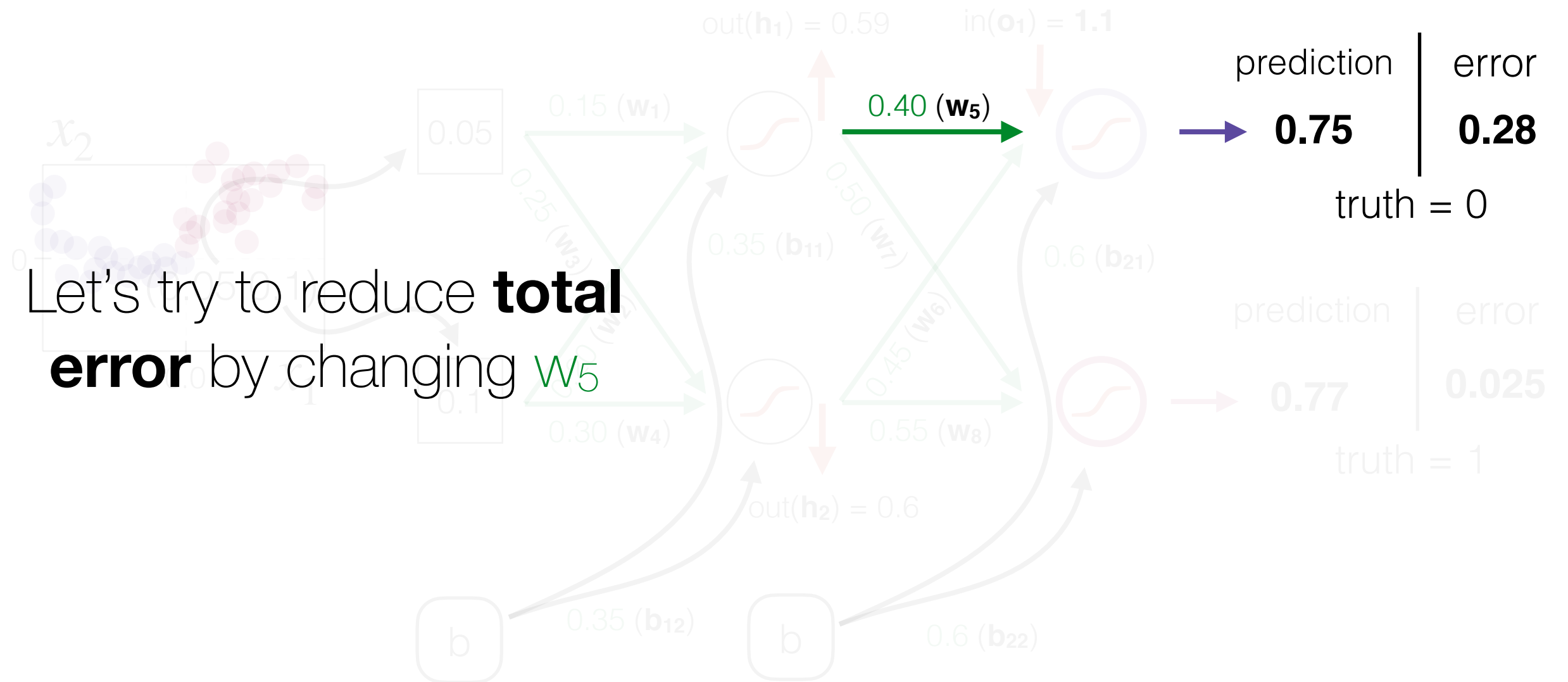
We want a way to **efficiently update all** our **weights** so that **total error decreases** most substantially



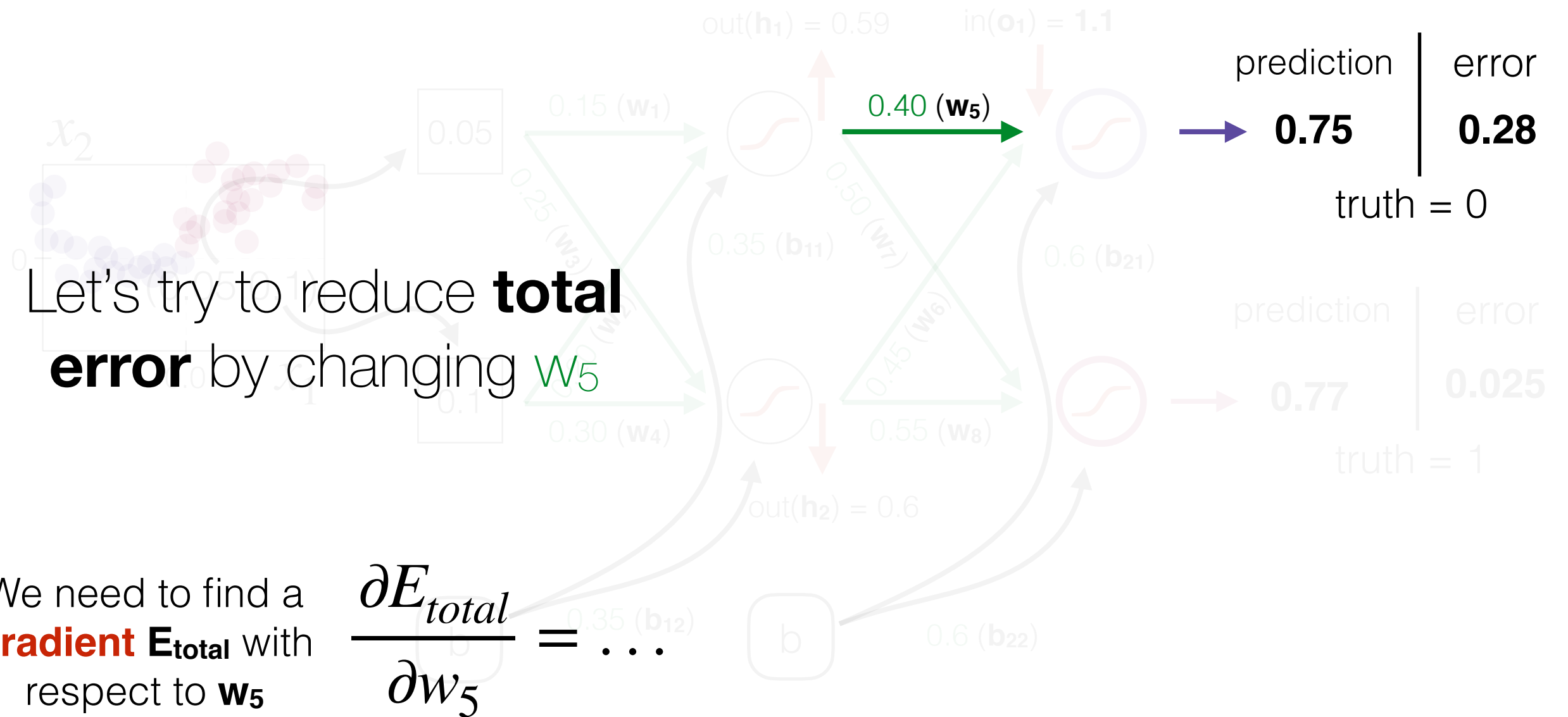
We want a way to **efficiently update all** our **weights** so that **total error decreases** most substantially



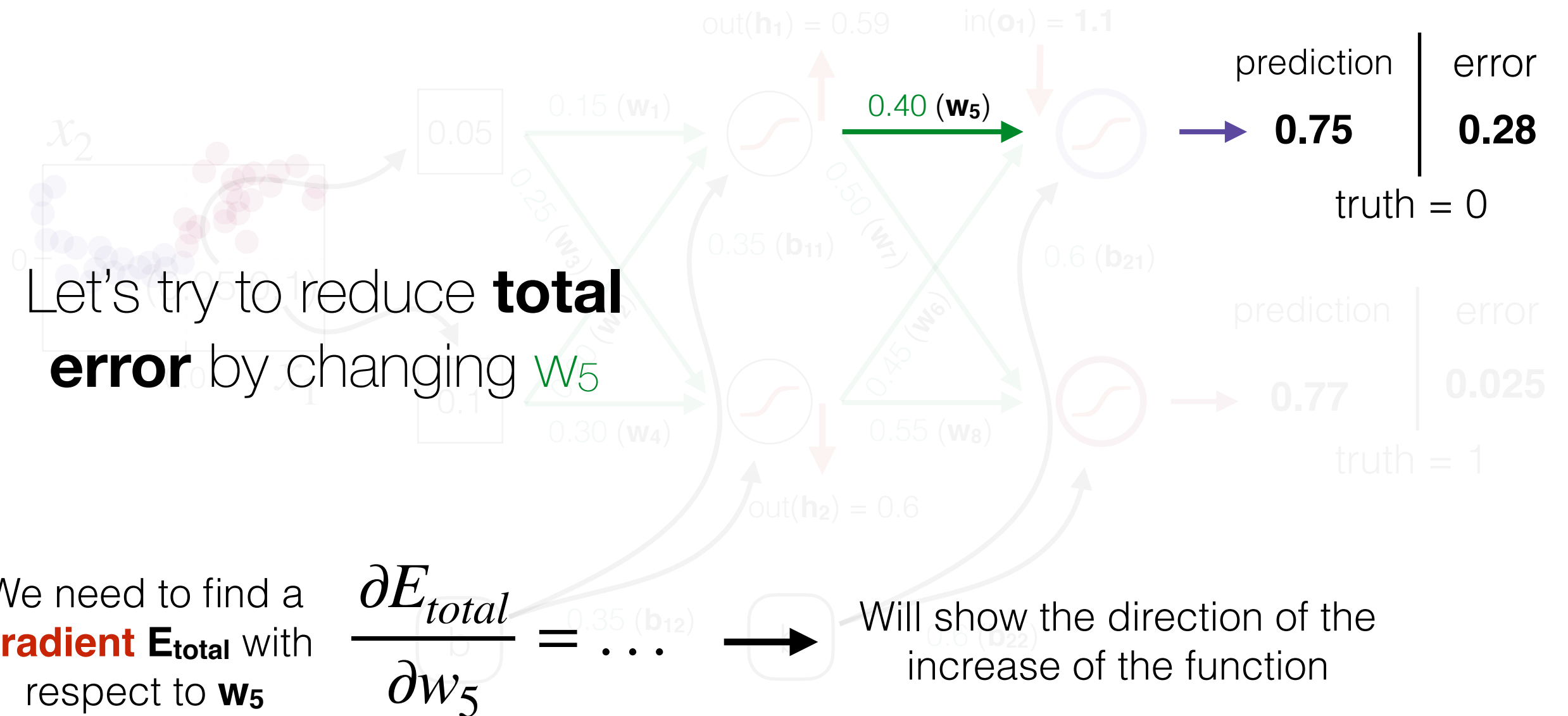
Backpropagation algorithm



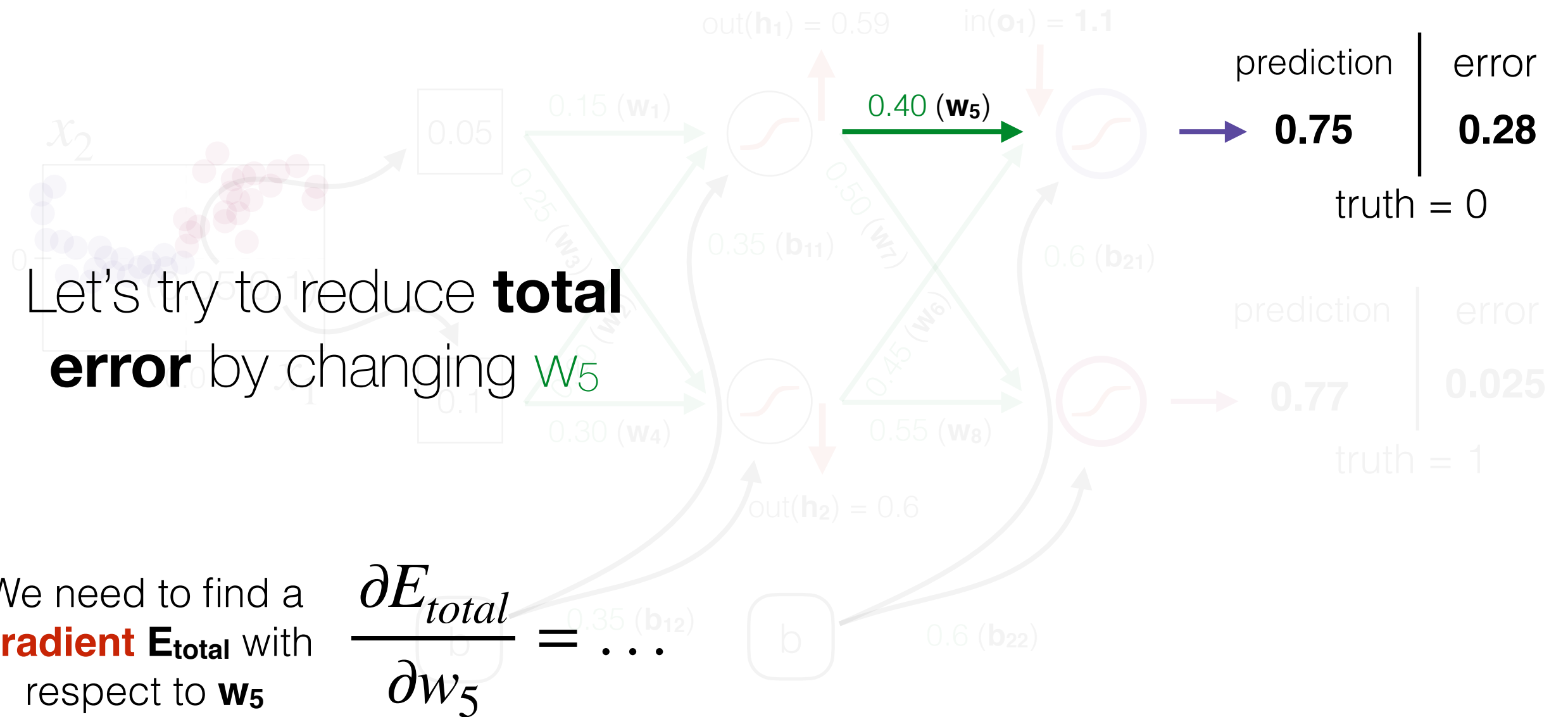
Backpropagation algorithm



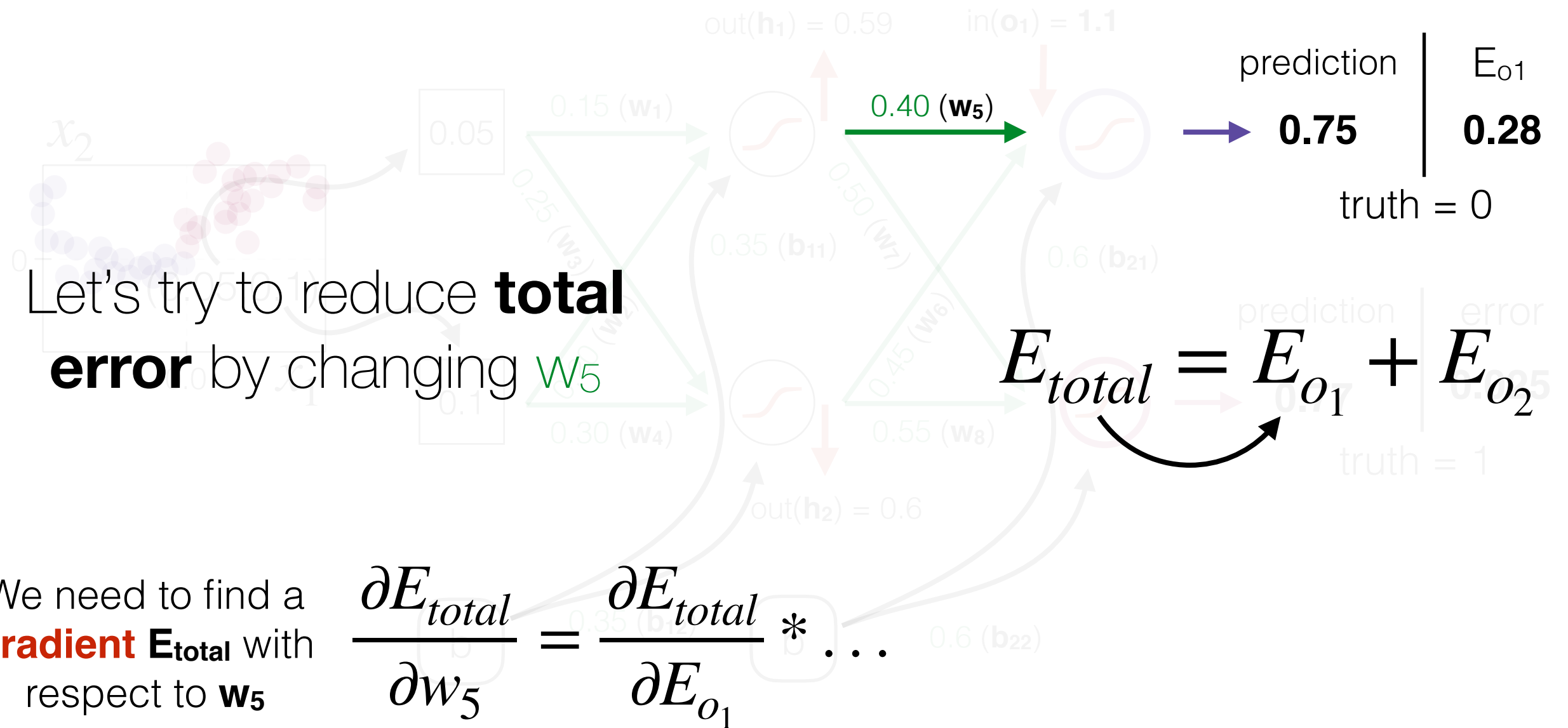
Backpropagation algorithm



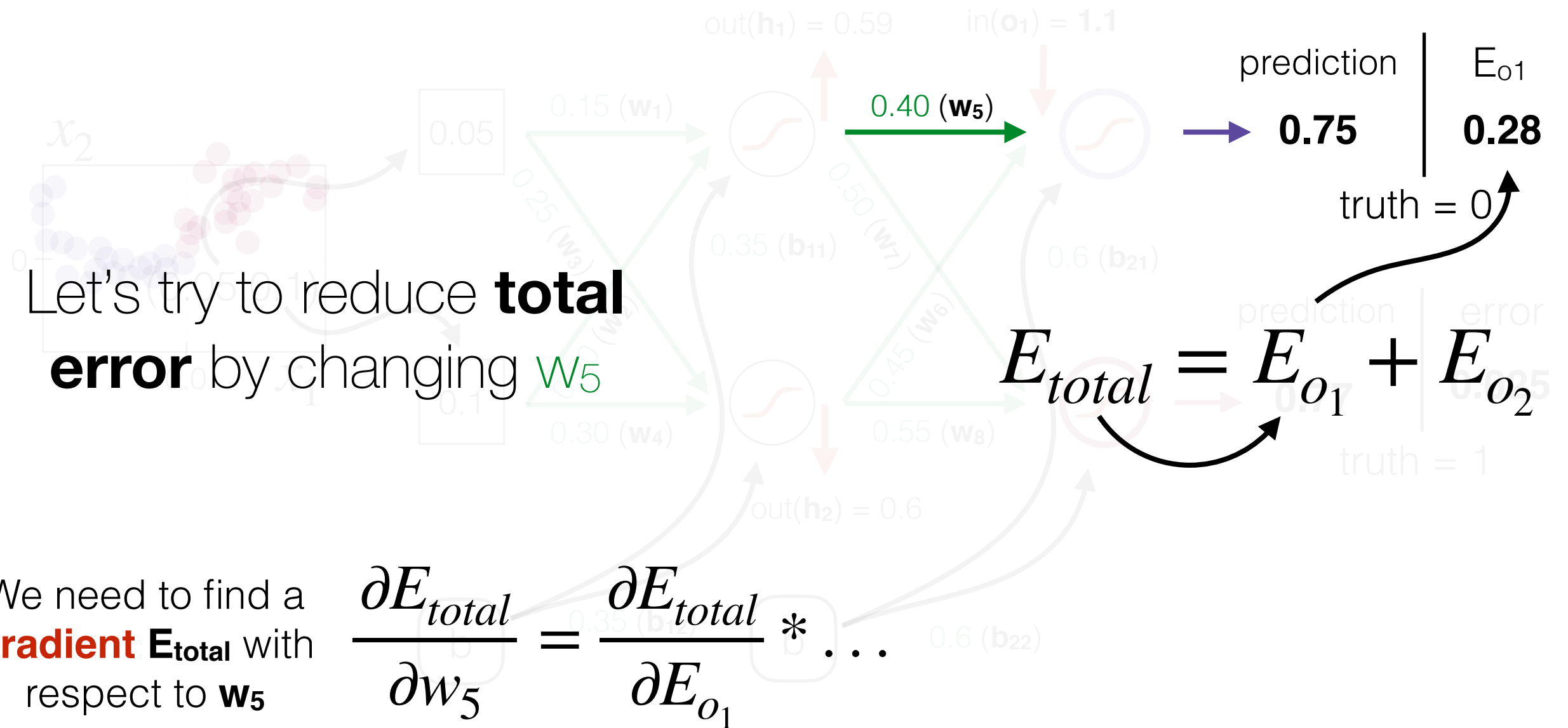
Backpropagation algorithm



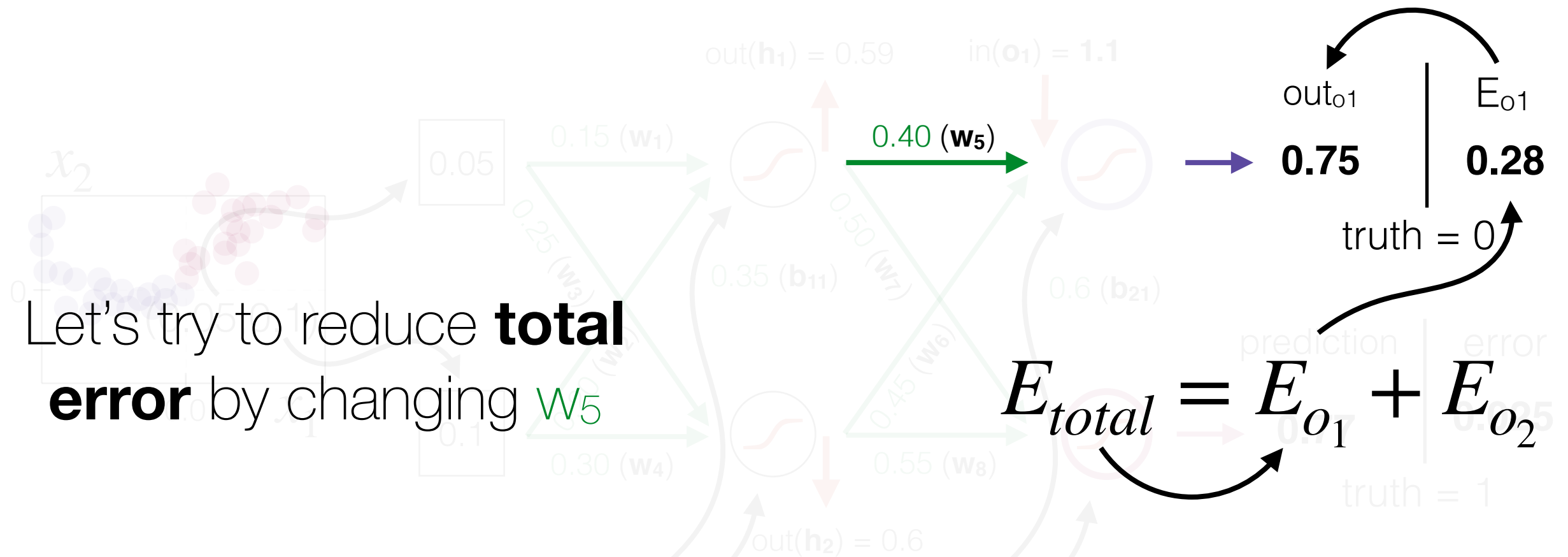
Backpropagation algorithm



Backpropagation algorithm



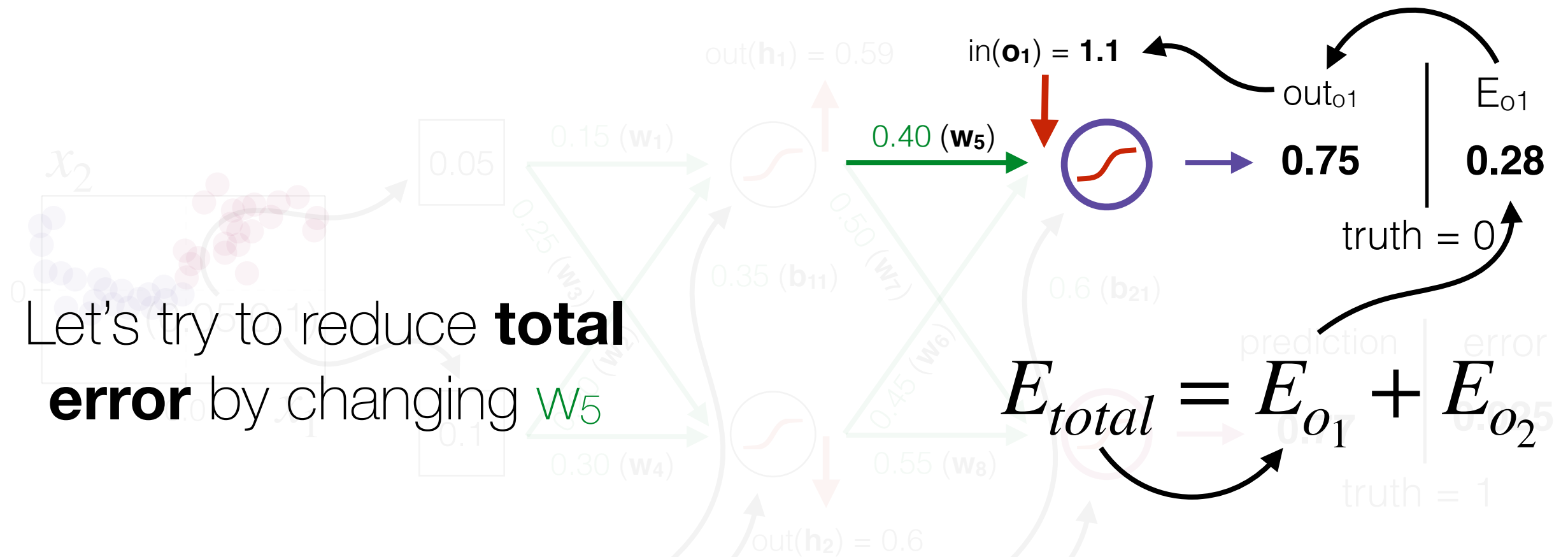
Backpropagation algorithm



We need to find a **gradient** $\mathbf{E}_{total}$ with respect to $\mathbf{w}_5$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \dots$$

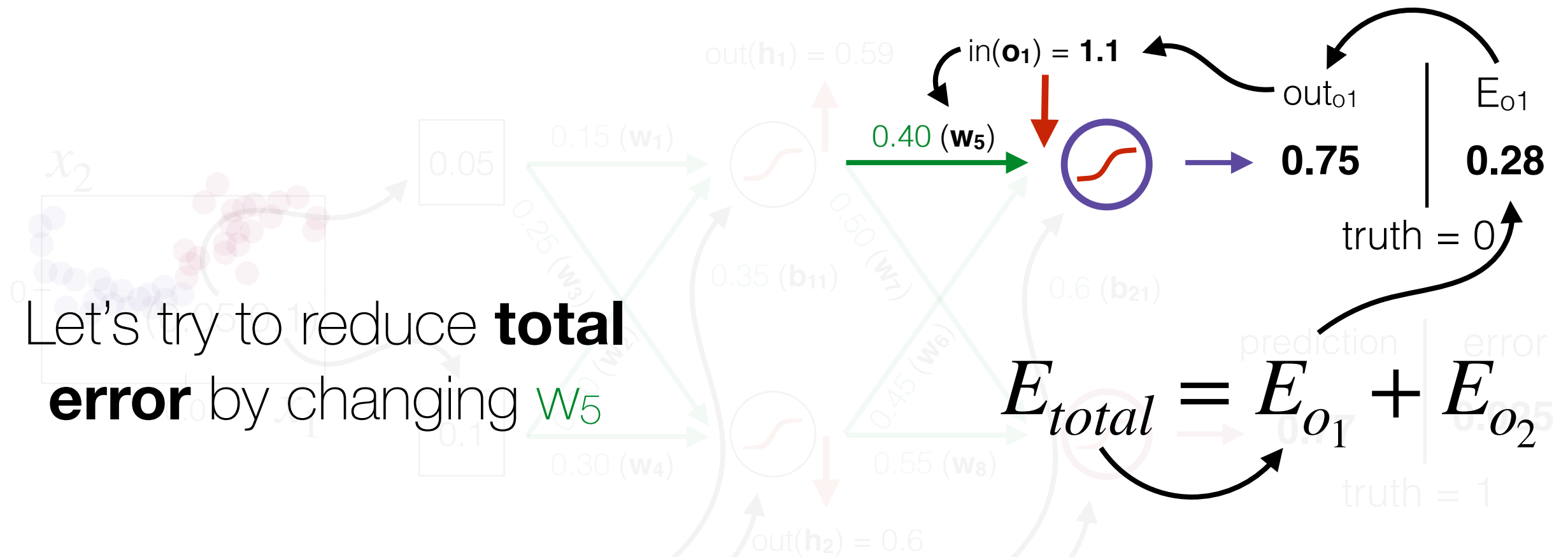
Backpropagation algorithm



We need to find a **gradient** $\mathbf{E}_{total}$ with respect to $\mathbf{w}_5$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \dots$$

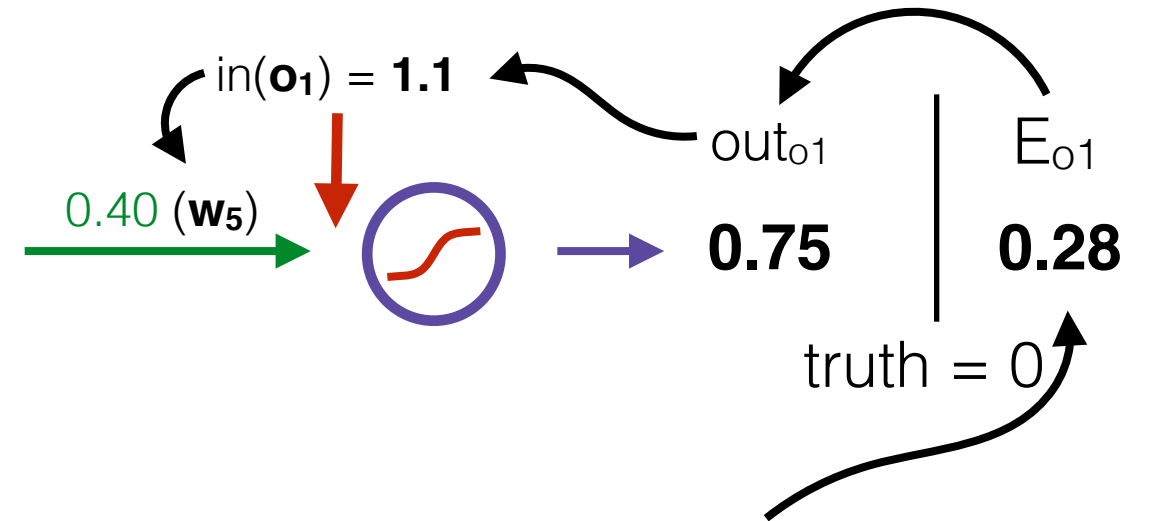
Backpropagation algorithm



We need to find a **gradient** $\mathbf{E}_{total}$ with respect to $\mathbf{w}_5$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

Backpropagation algorithm

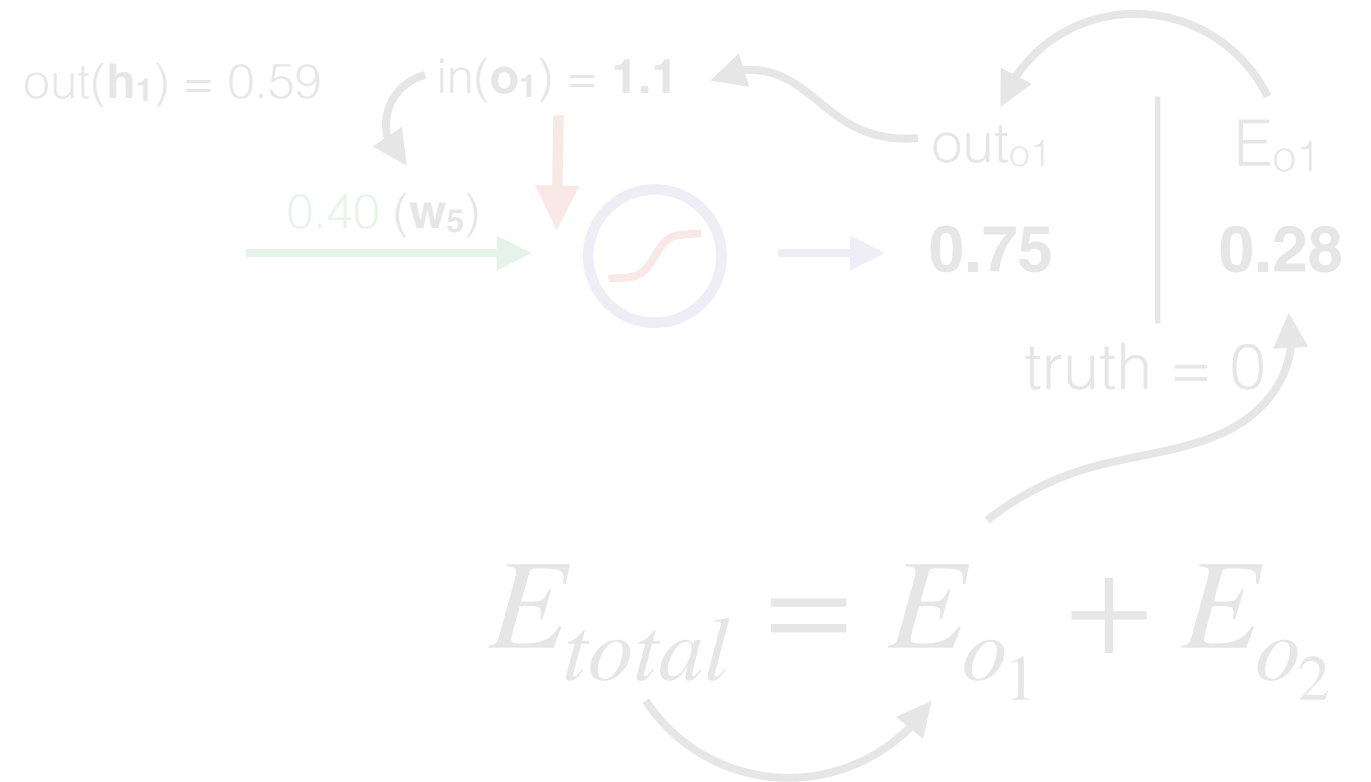


Let's try to reduce **total error** by changing w_5

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

Backpropagation algorithm

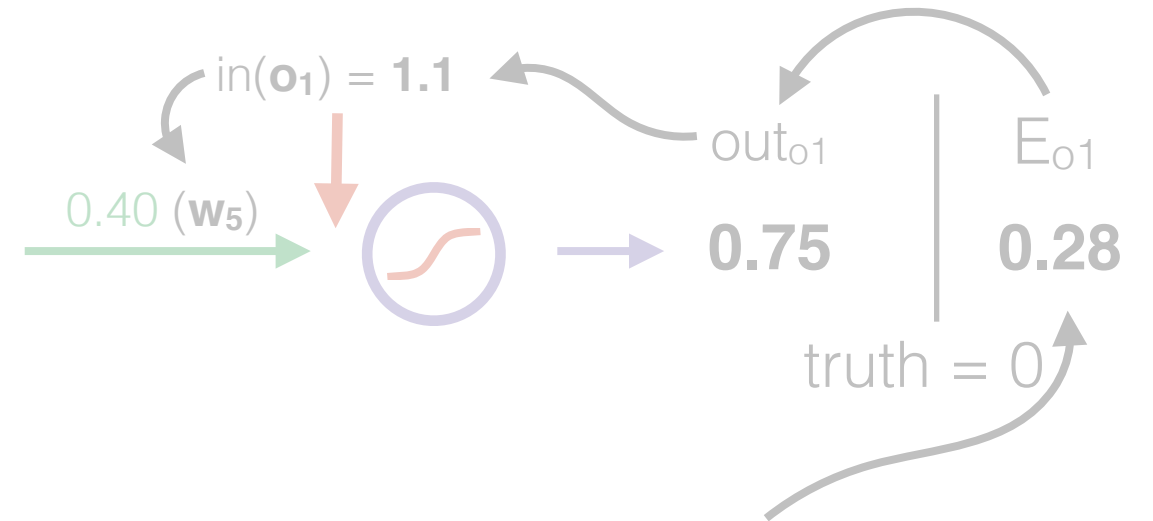


Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o1}} * \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

Chain rule

Backpropagation algorithm



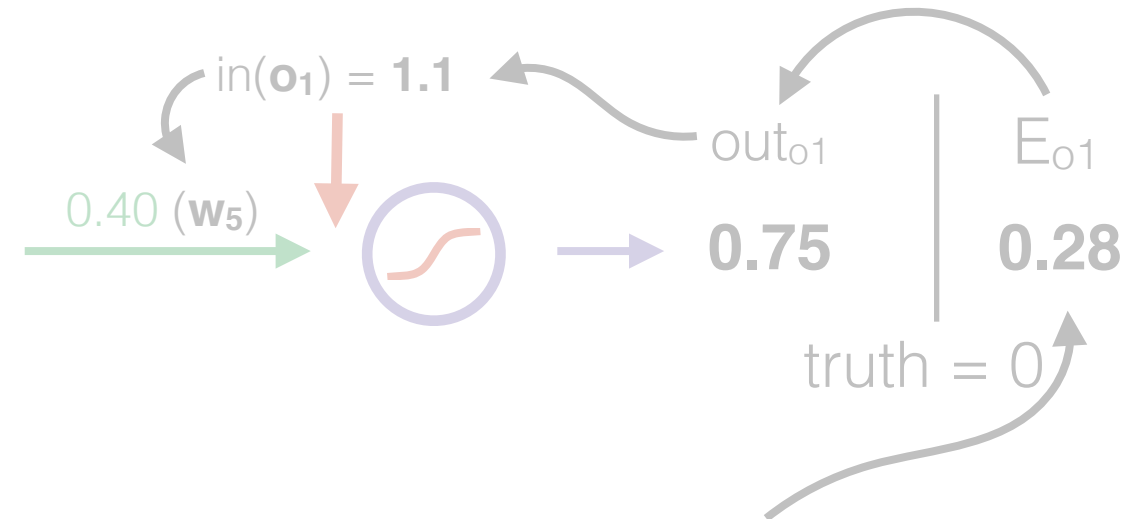
Let's try to reduce **total error** by changing w_5

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

0.6 (b_{22})

Backpropagation algorithm



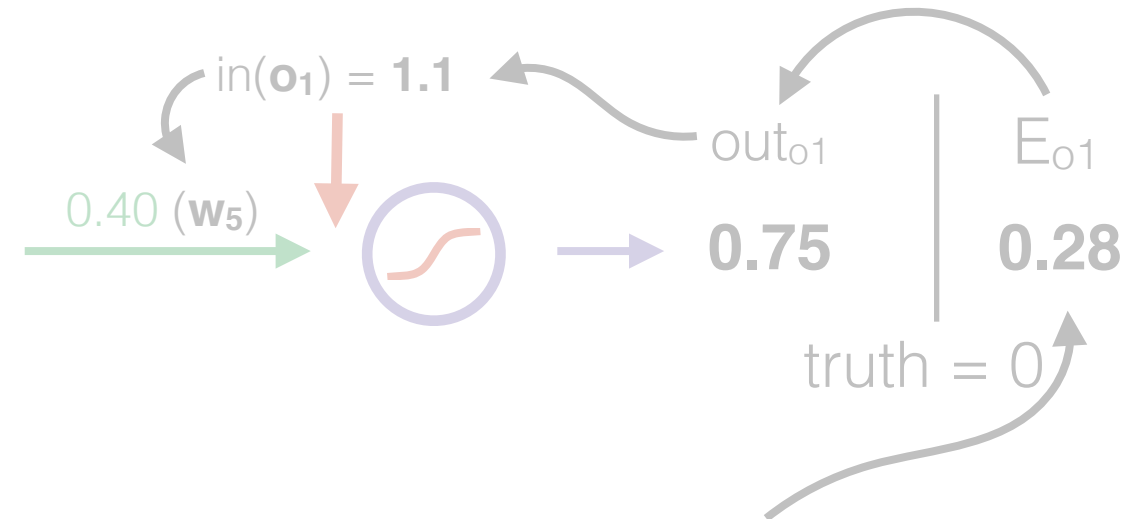
Let's try to reduce **total error** by changing w_5

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{total} = E_{o_1} + E_{o_2}$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

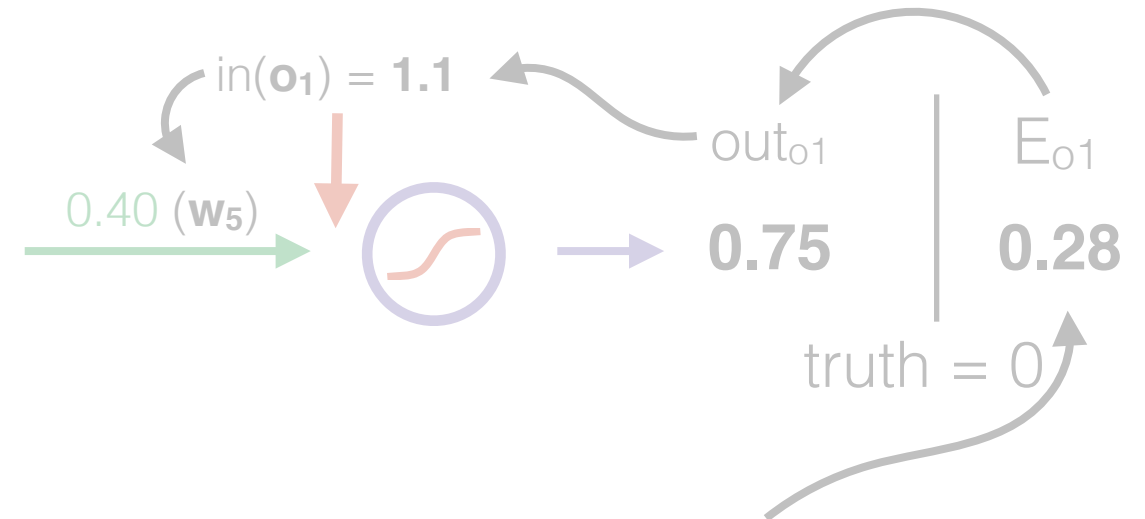
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial \text{out}_{o_1}} * \frac{\partial \text{out}_{o_1}}{\partial \text{in}_{o_1}} * \frac{\partial \text{in}_{o_1}}{\partial w_5}$$

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial E_{o_1}} = \frac{\partial}{\partial E_{o_1}} (E_{o_1} + E_{o_2})$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

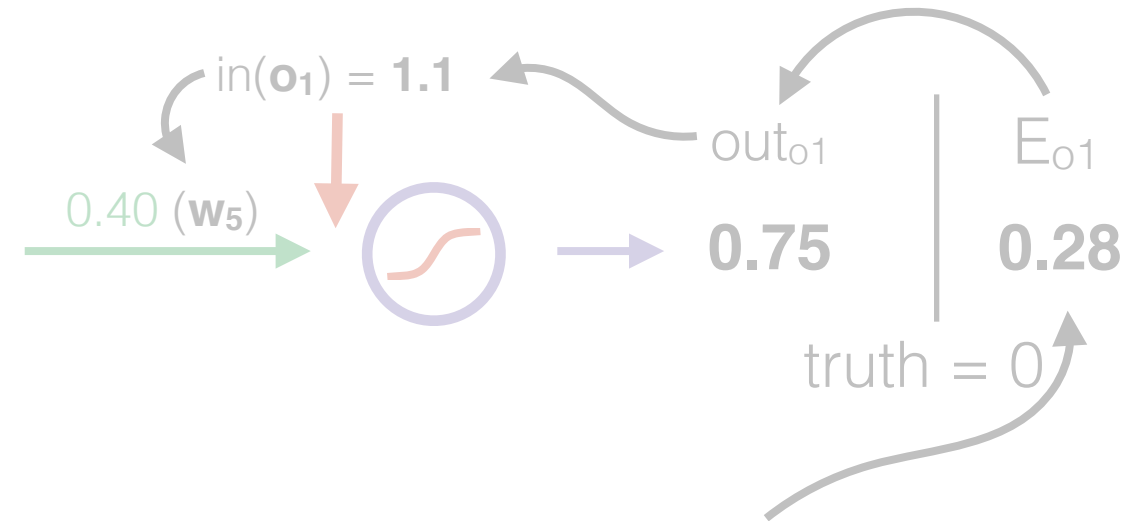
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(\mathbf{x}, \mathbf{y}) \leftarrow E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial E_{o_1}} = \frac{\partial}{\partial E_{o_1}} (E_{o_1} + E_{o_2})$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

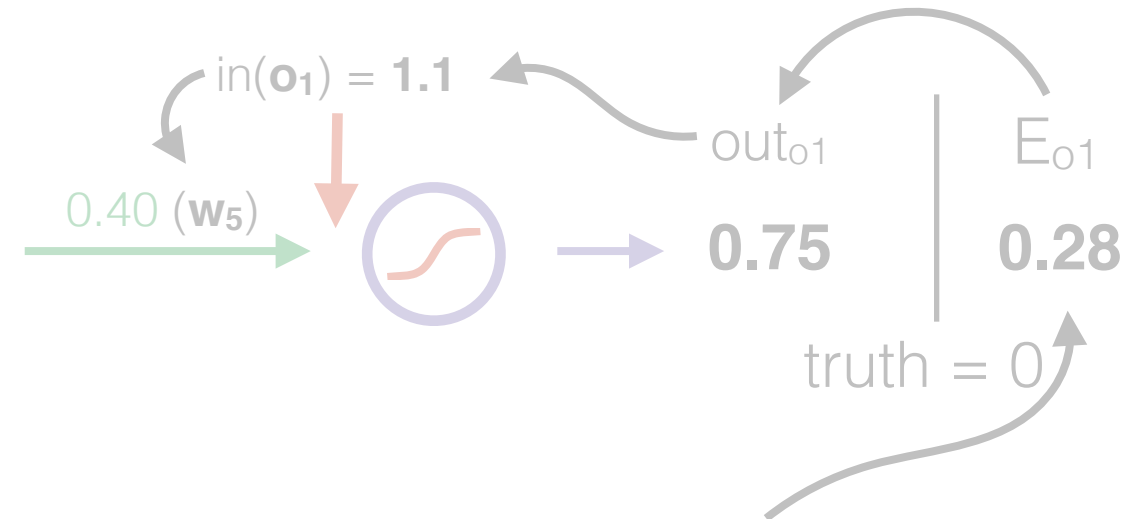
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(x, y) = x + y$$

$$\frac{\partial E_{total}}{\partial E_{o_1}} = \frac{\partial}{\partial E_{o_1}} (E_{o_1} + E_{o_2})$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

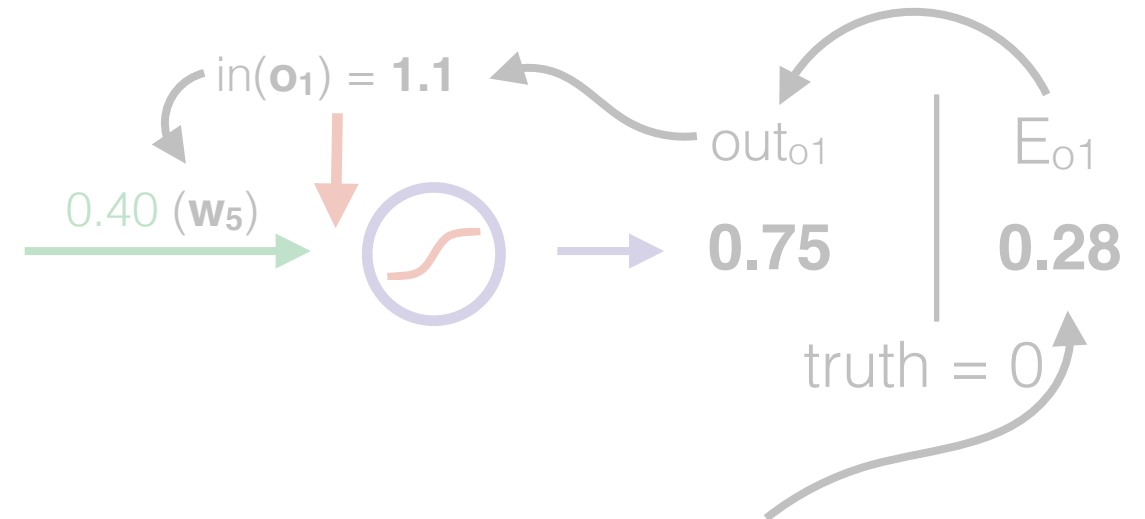
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(x, y) = x + y$$

$$\frac{\partial f(x, y)}{\partial x} = \frac{\partial}{\partial x}(x + y)$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

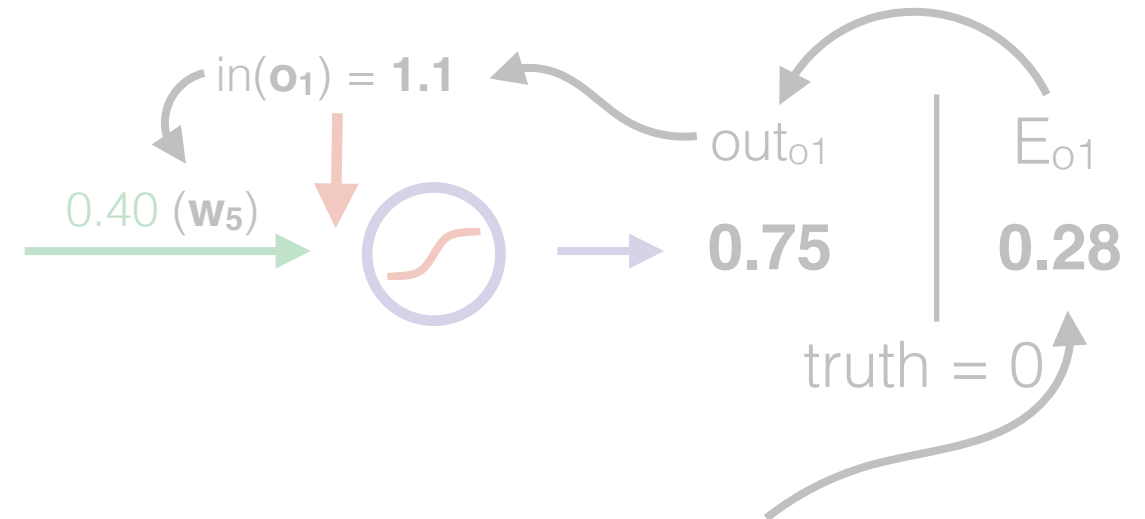
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(x, y) = x + y$$

$$\frac{\partial f(x, y)}{\partial x} = \frac{\partial}{\partial x} x + \frac{\partial}{\partial x} y$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

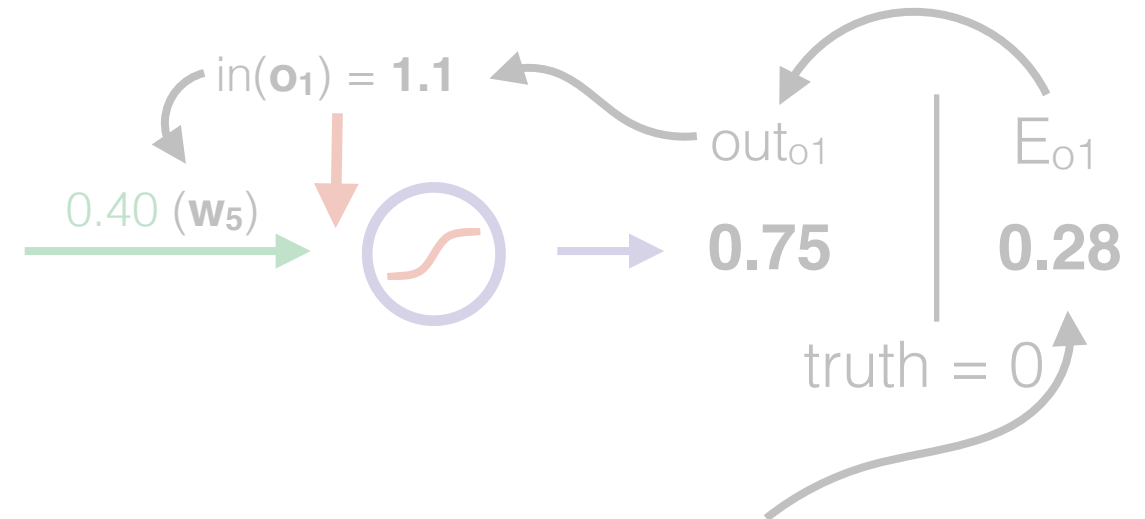
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(x, y) = x + y$$

$$\frac{\partial f(x, y)}{\partial x} = \frac{\partial}{\partial x} x + 0$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

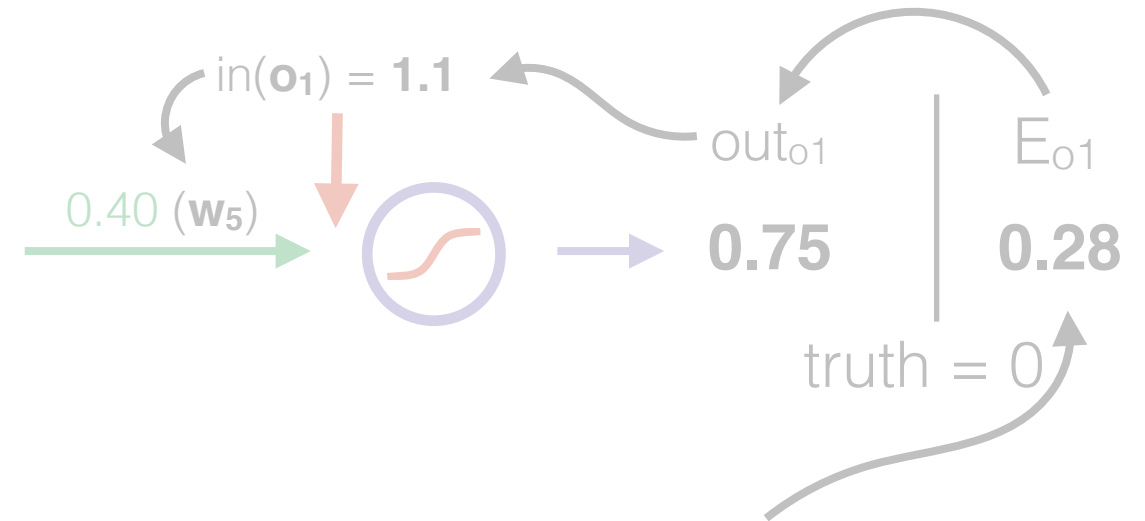
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$f(x, y) = x + y$$

$$\frac{\partial f(x, y)}{\partial x} = 1$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

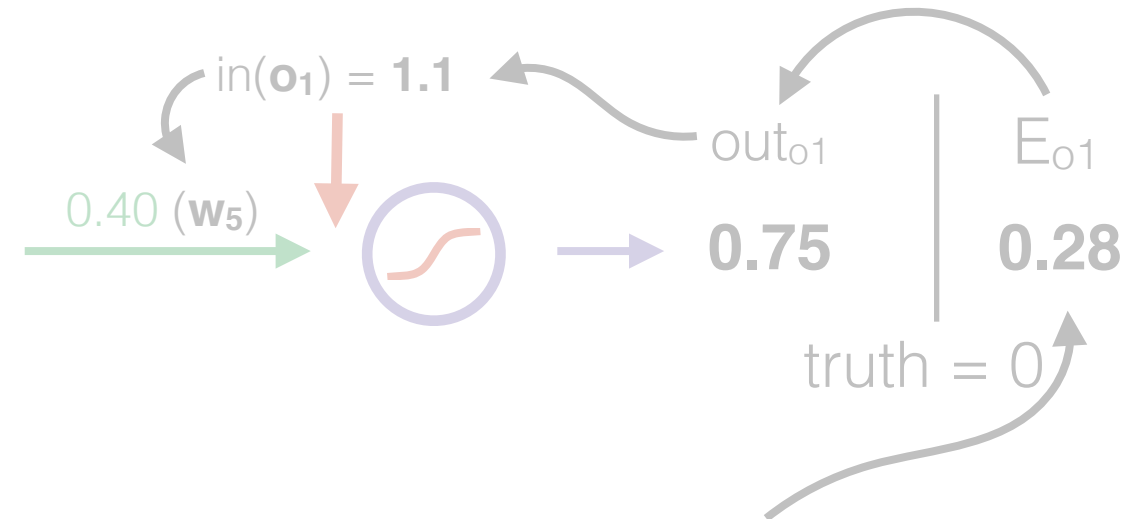
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial E_{o_1}} = 1$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

What does this **actually** say?

$$E_{total} = E_{o_1} + E_{o_2}$$
$$\frac{\partial E_{total}}{\partial E_{o_1}} = 1$$

If we increase E_{o1} by **1**



$$E_{total} = E_{o_1} + E_{o_2}$$

What does this
actually say?

$$\frac{\partial E_{total}}{\partial E_{o_1}} = 1$$

If we increase E_{o_1} by **1**



$$E_{total} = (E_{o_1} + 1) + E_{o_2}$$

What does this
actually say?

$$\frac{\partial E_{total}}{\partial E_{o_1}} = 1$$

If we increase E_{o_1} by **1**



$$E_{total} = (E_{o_1} + E_{o_2}) + 1$$

What does this
actually say?

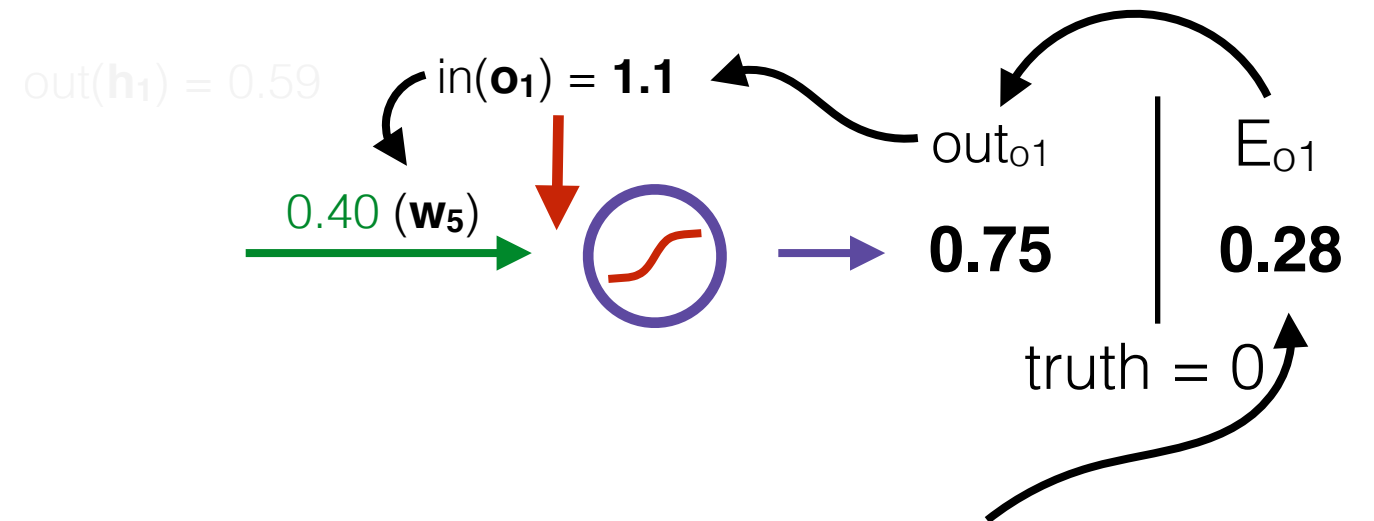
$$\frac{\partial E_{total}}{\partial E_{o_1}} = 1$$

If we increase E_{o1} by **1**

What does this
actually say?

$$E_{total} = E_{total} + 1 \quad \begin{array}{l} E_{total} \text{ would also} \\ \text{increase by } \mathbf{1} \end{array}$$
$$\frac{\partial E_{total}}{\partial E_{o1}} = 1$$

Backpropagation algorithm

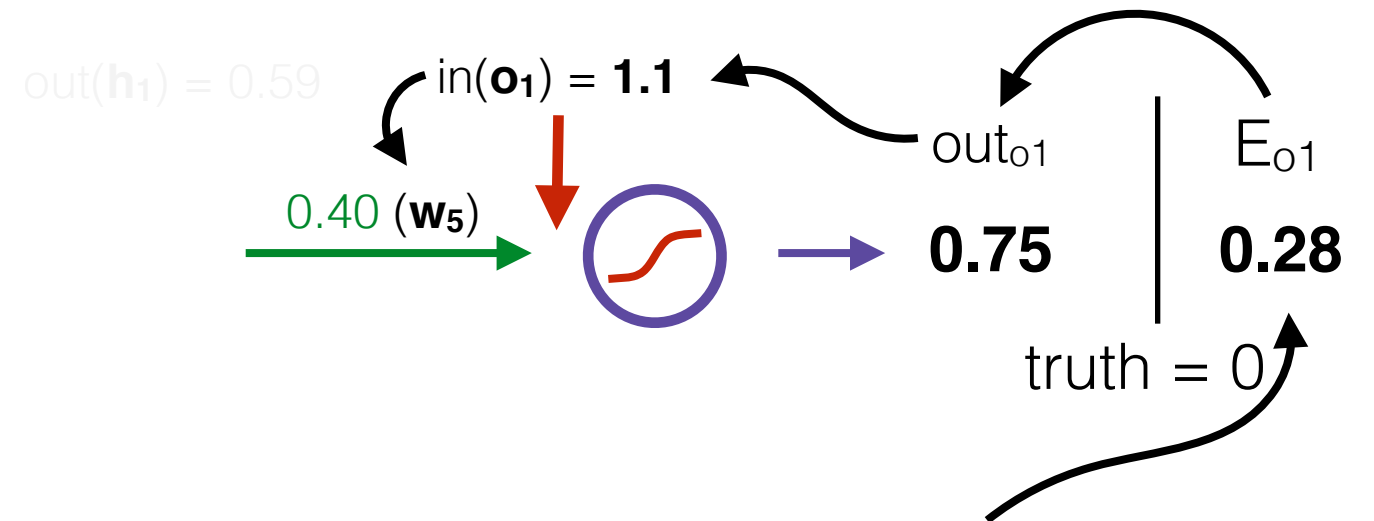


Let's try to reduce **total error** by changing w_5

$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

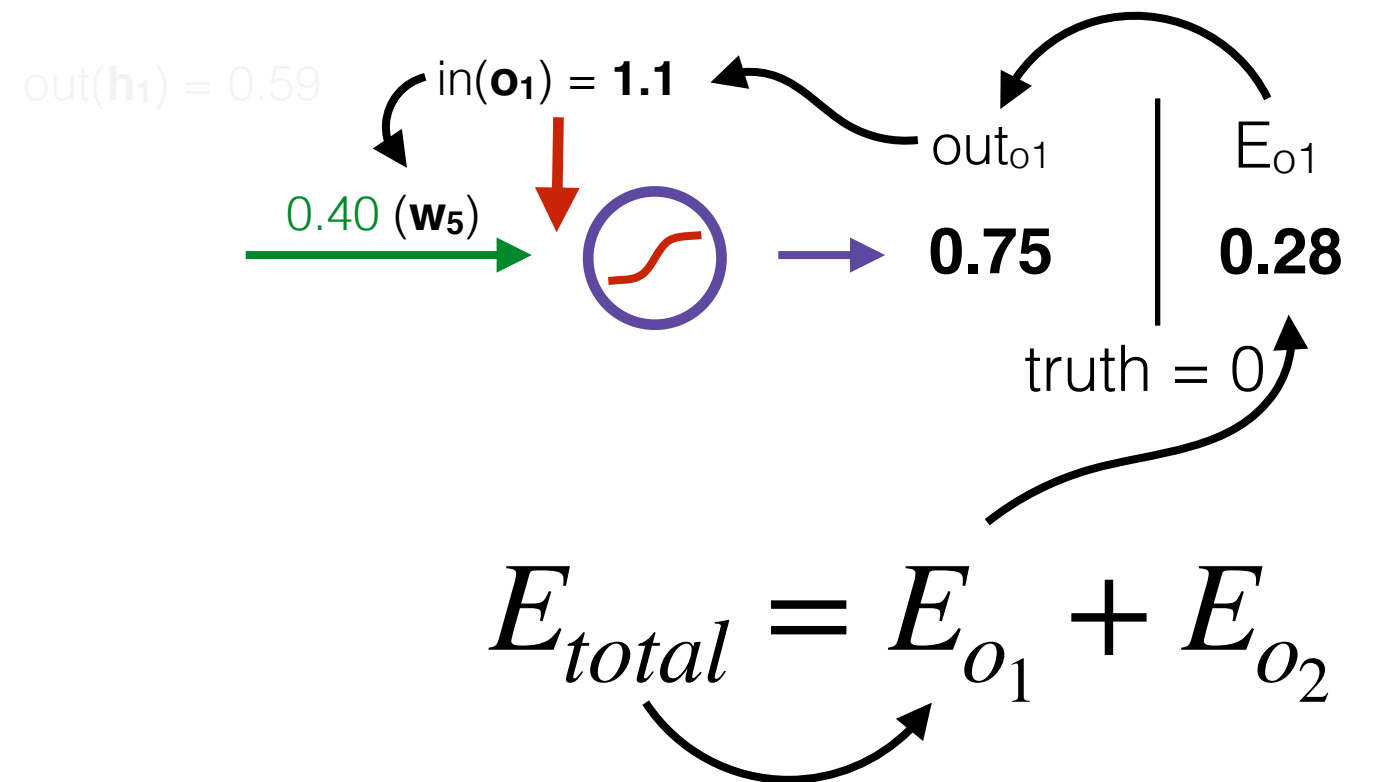
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial E_{o_1}} * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$\uparrow$
 0.6 (b_{22})

We know this value

Backpropagation algorithm

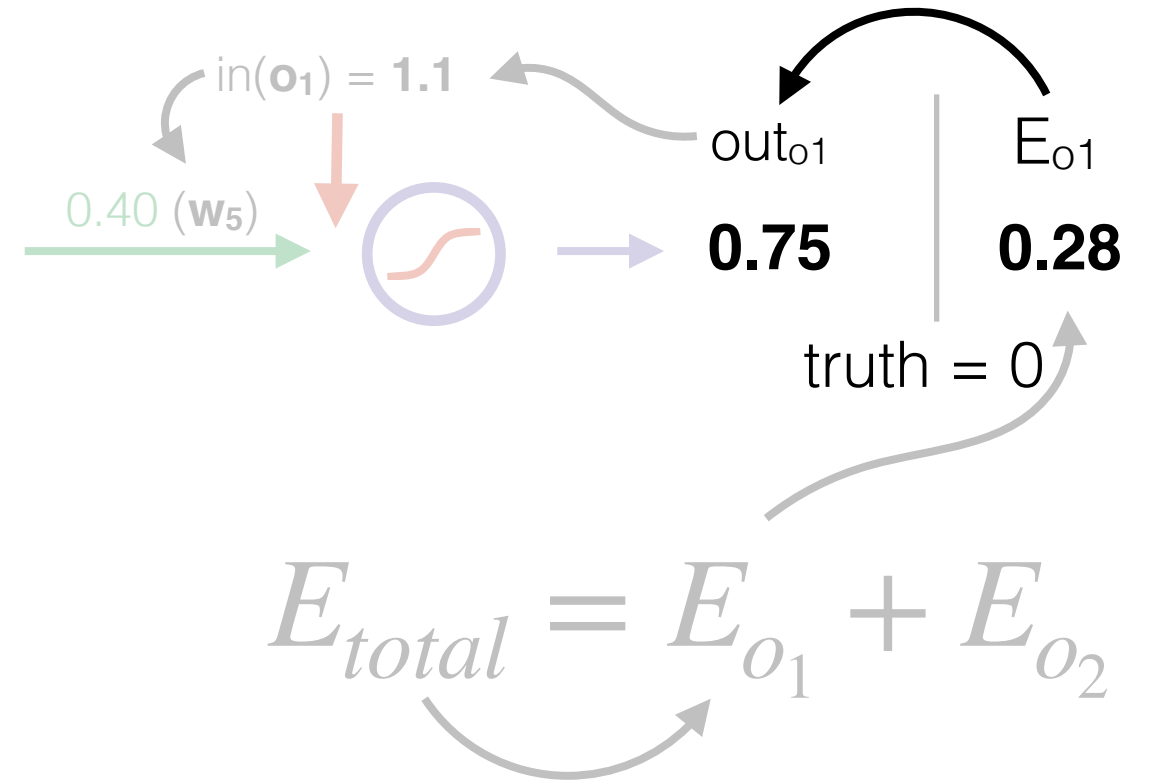


Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

$0.6 (b_{22})$

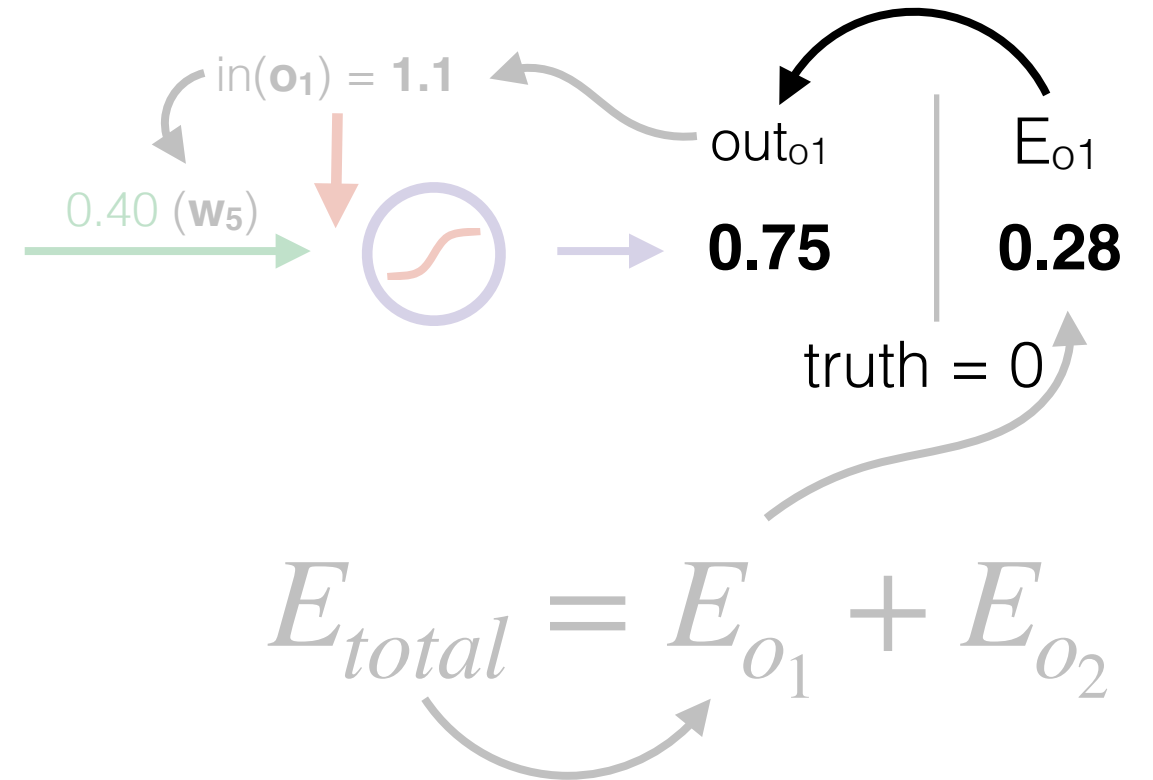
Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

Backpropagation algorithm

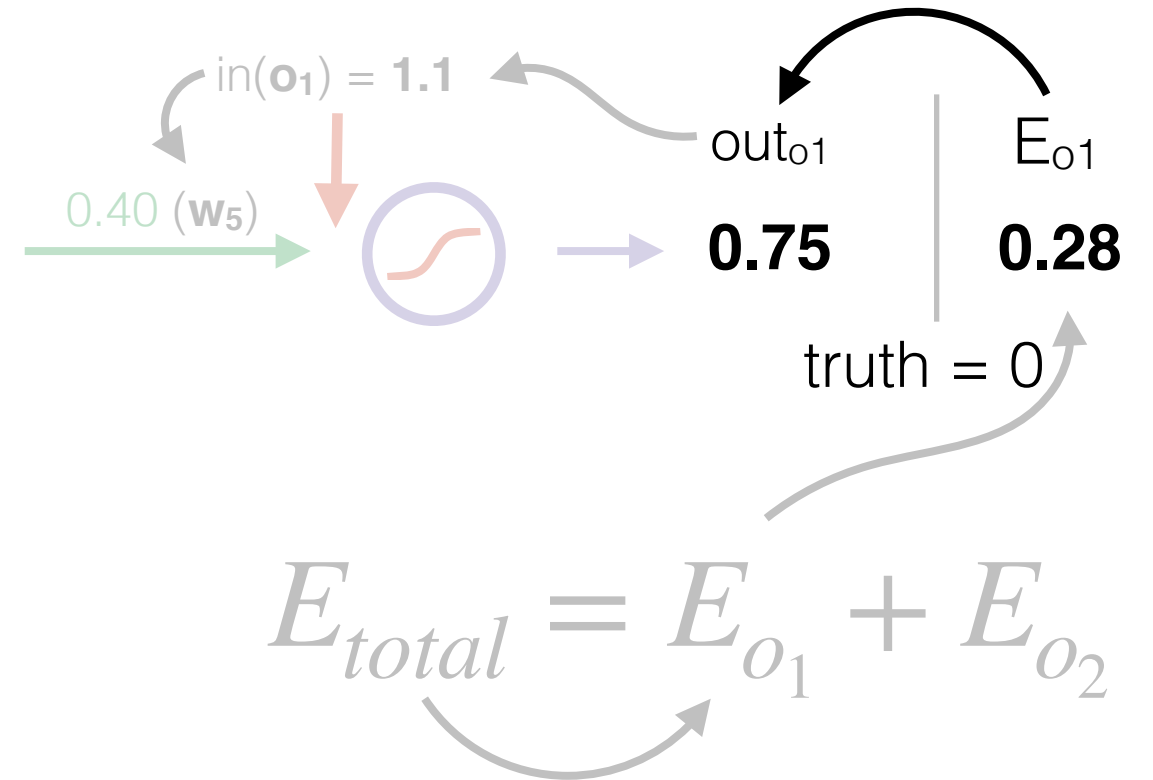


Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(truth - out_{o_1})^2$$

Backpropagation algorithm



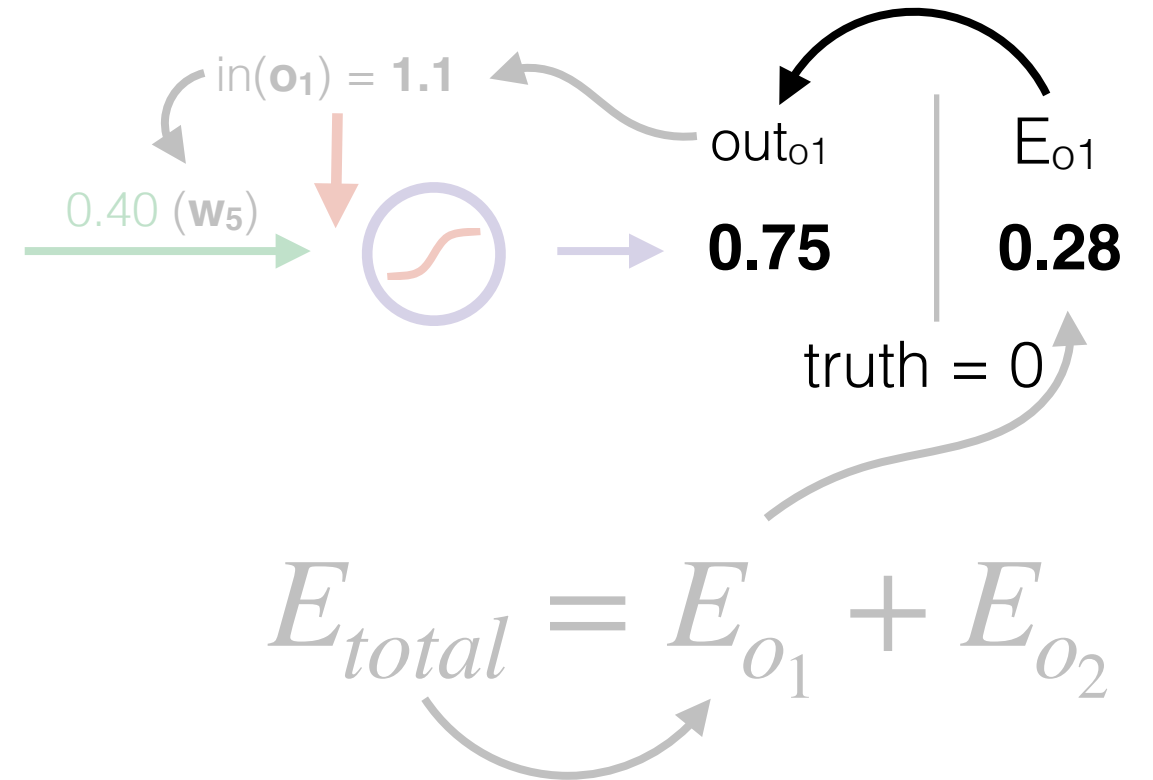
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(truth - out_{o_1})^2$$

$$\frac{\partial E_{o_1}}{\partial out_{o_1}} = \frac{\partial}{\partial out_{o_1}} \left(\frac{1}{2}(truth - out_{o_1})^2 \right)$$

Backpropagation algorithm



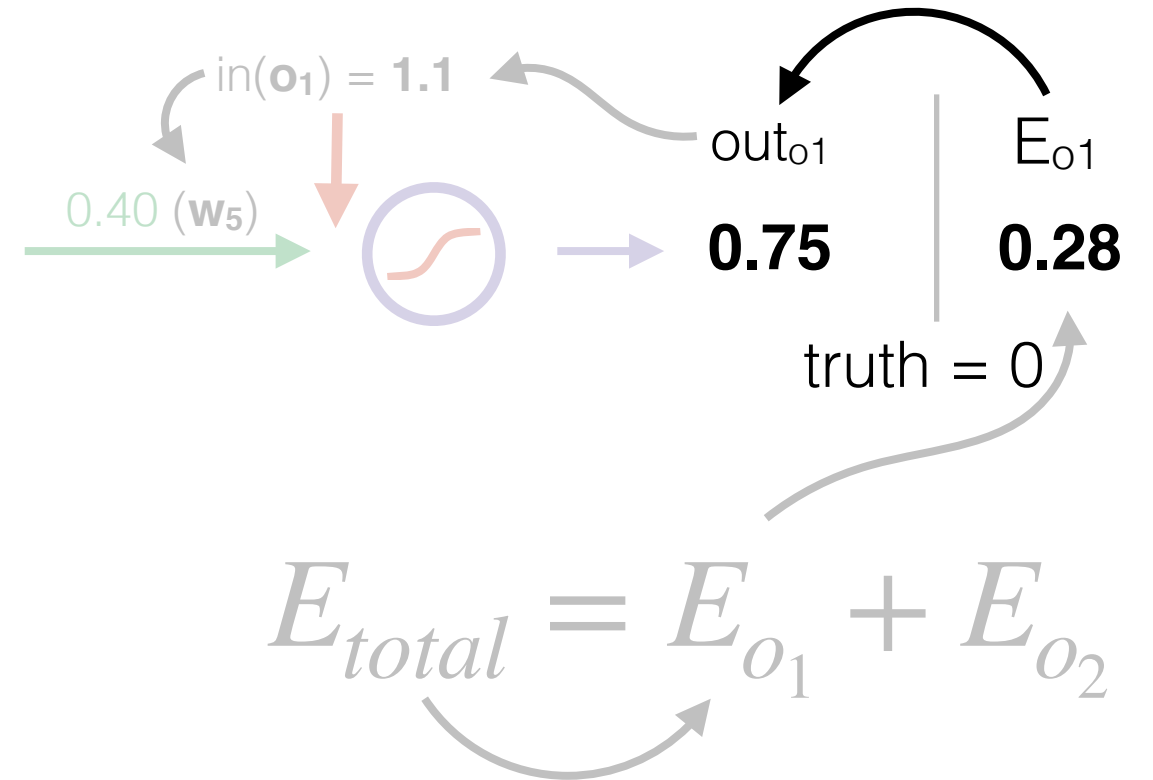
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(x - y)^2$$

$$\frac{\partial E_{o_1}}{\partial y} = \frac{\partial}{\partial y} \left(\frac{1}{2}(x - y)^2 \right)$$

Backpropagation algorithm



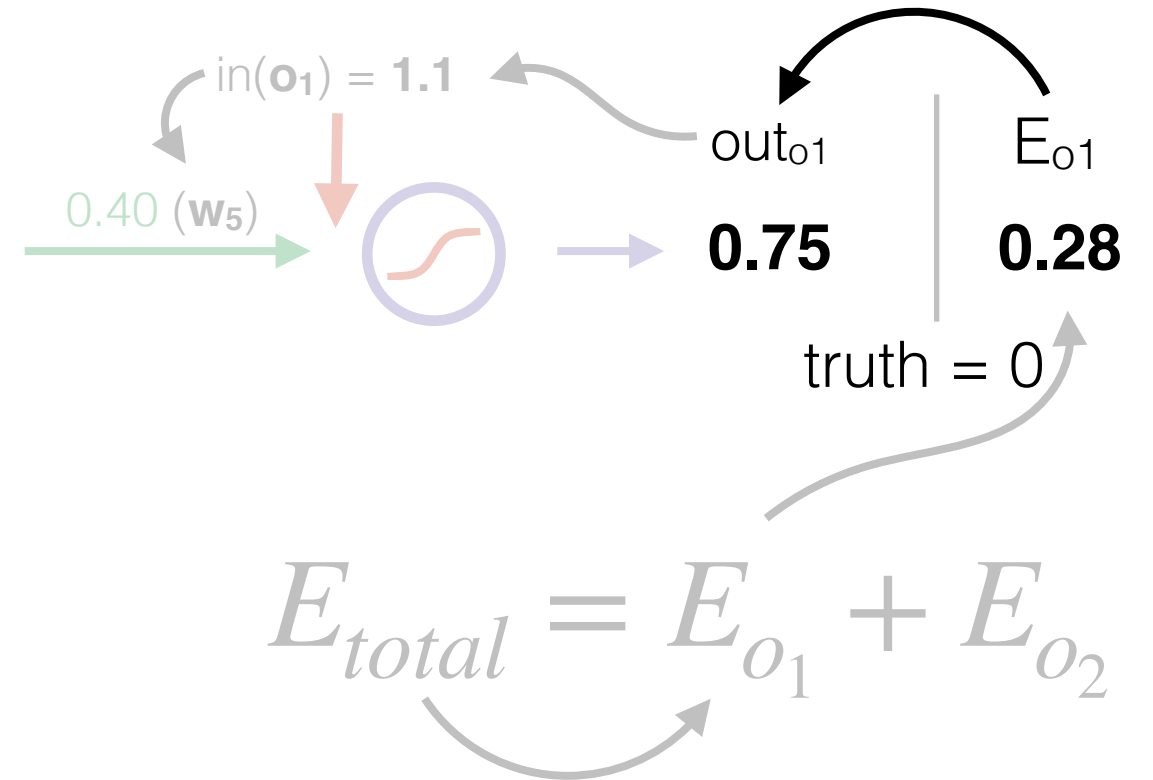
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(x - y)^2$$

$$\frac{\partial E_{o_1}}{\partial y} = \left(\frac{1}{2} * 2(x - y)\right) * \frac{\partial}{\partial y}(x - y)$$

Backpropagation algorithm



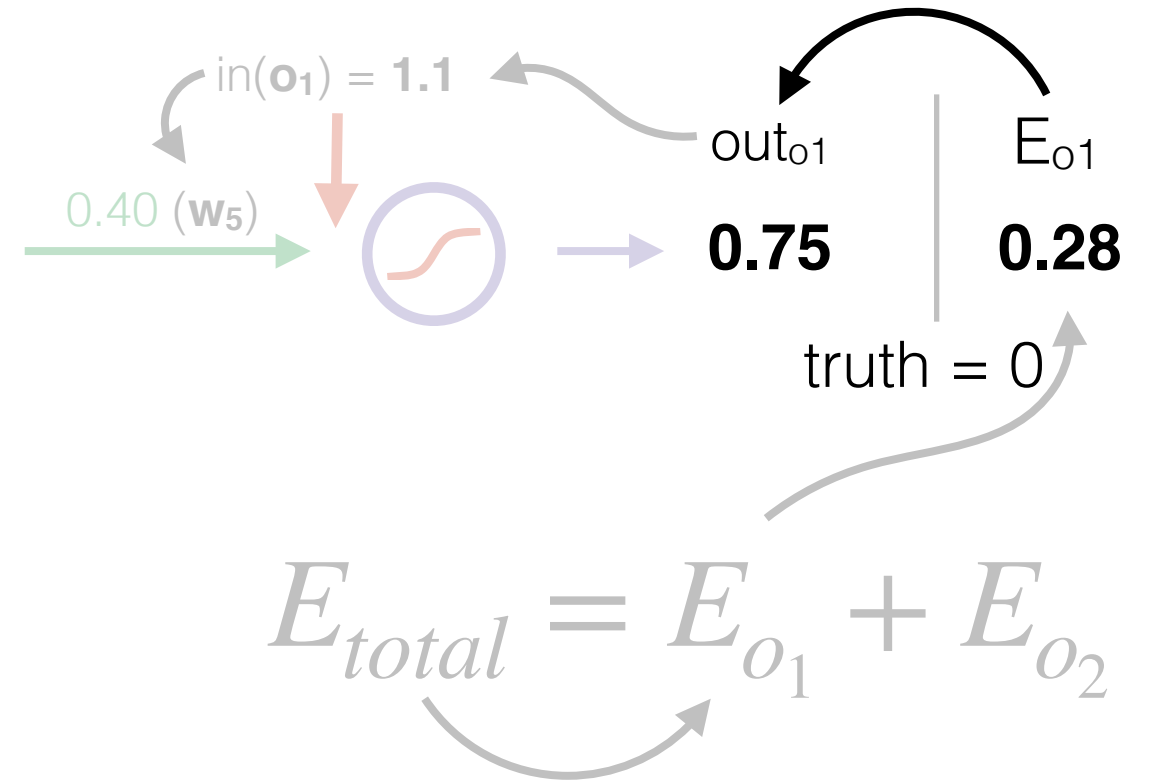
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(x - y)^2$$

$$\frac{\partial E_{o_1}}{\partial y} = \left(\frac{1}{2} * 2(x - y)\right) * (-1)$$

Backpropagation algorithm



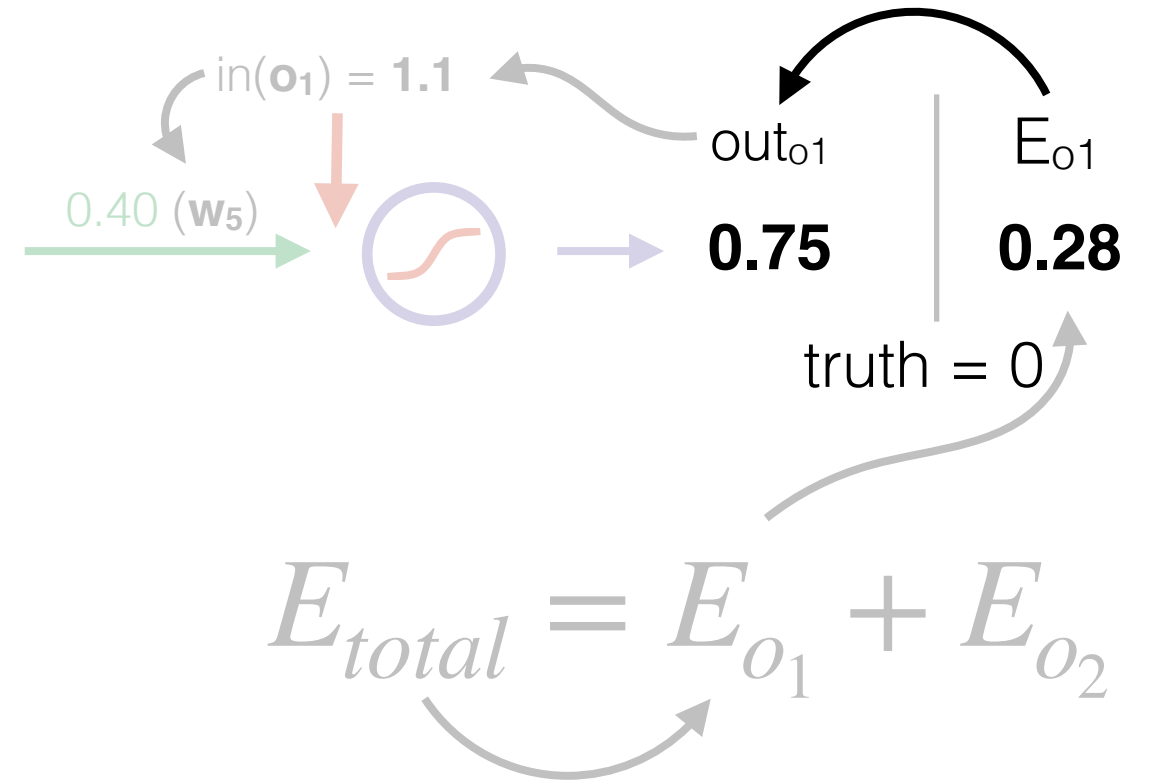
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(x - y)^2$$

$$\frac{\partial E_{o_1}}{\partial y} = (x - y) * (-1)$$

Backpropagation algorithm



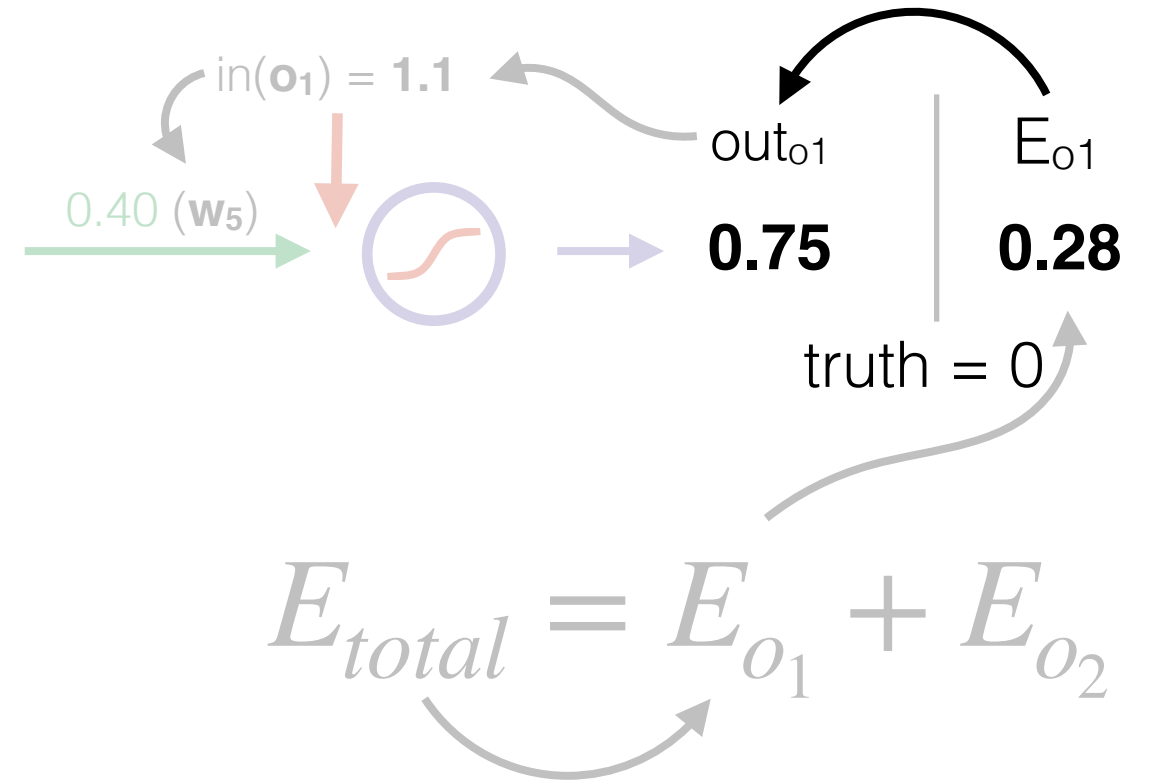
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(x - y)^2$$

$$\frac{\partial E_{o_1}}{\partial y} = y - x$$

Backpropagation algorithm



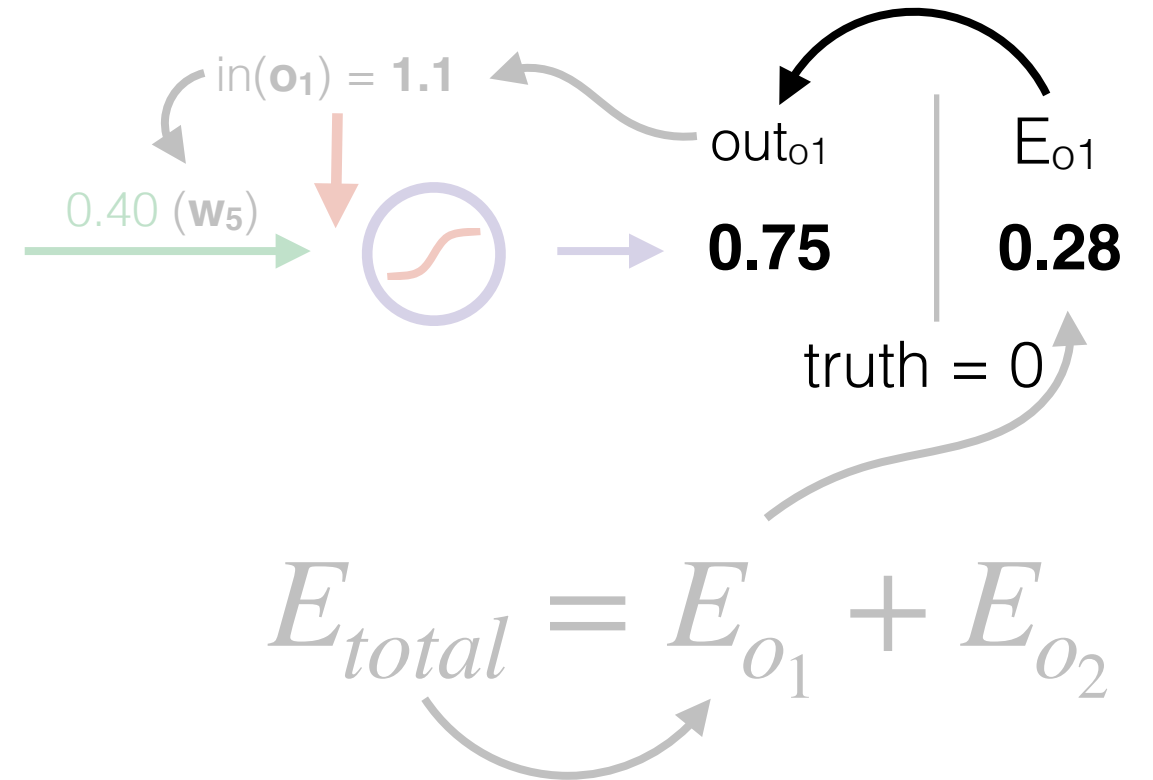
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(truth - out_{o_1})^2$$

$$\frac{\partial E_{o_1}}{\partial y} = out_{o_1} - truth$$

Backpropagation algorithm



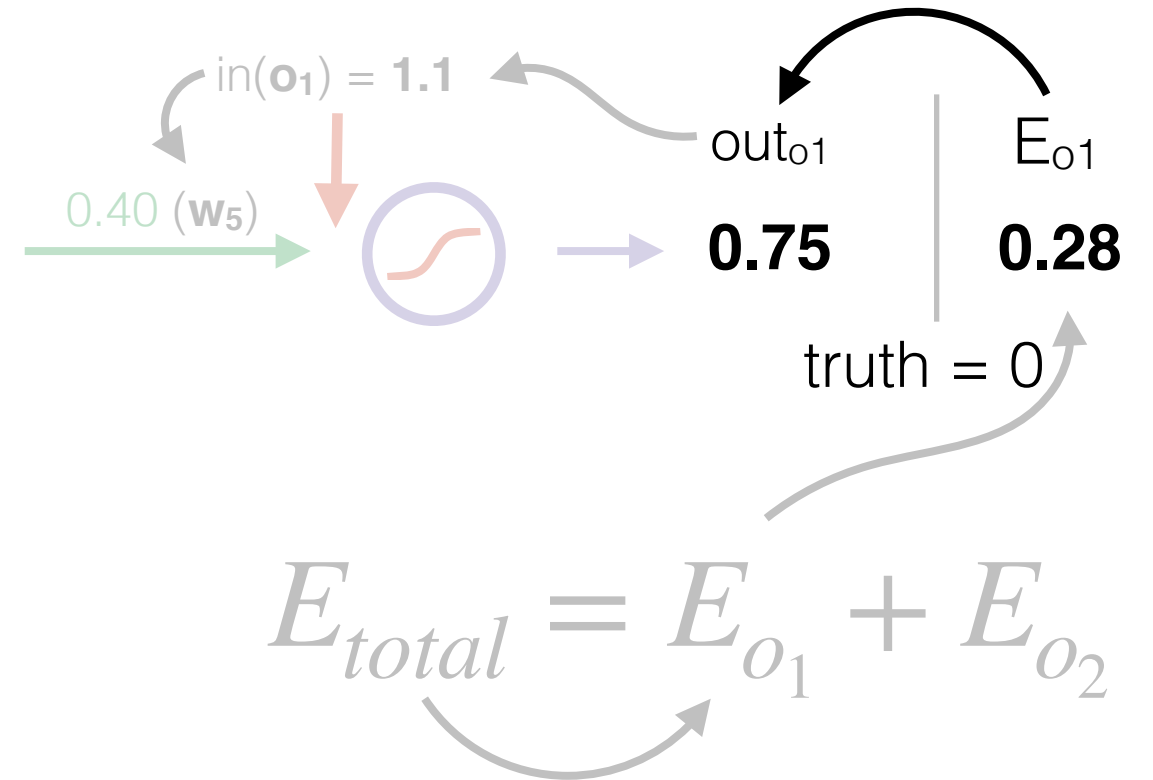
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(truth - out_{o_1})^2$$

$$\frac{\partial E_{o_1}}{\partial y} = 0.75 - 0$$

Backpropagation algorithm



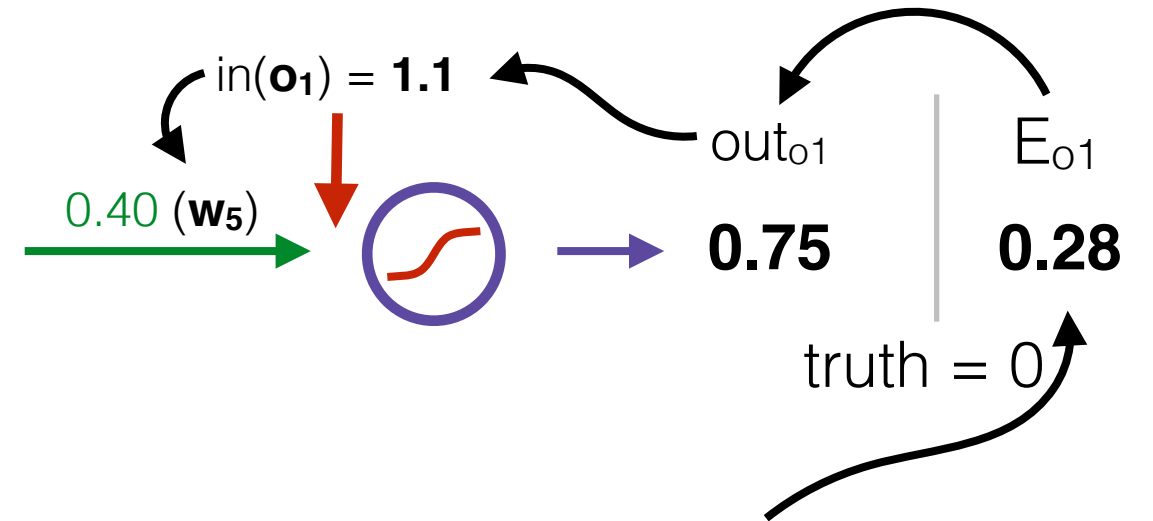
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$

$$E_{o_1} = \frac{1}{2}(truth - out_{o_1})^2$$

$$\frac{\partial E_{o_1}}{\partial y} = 0.75$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

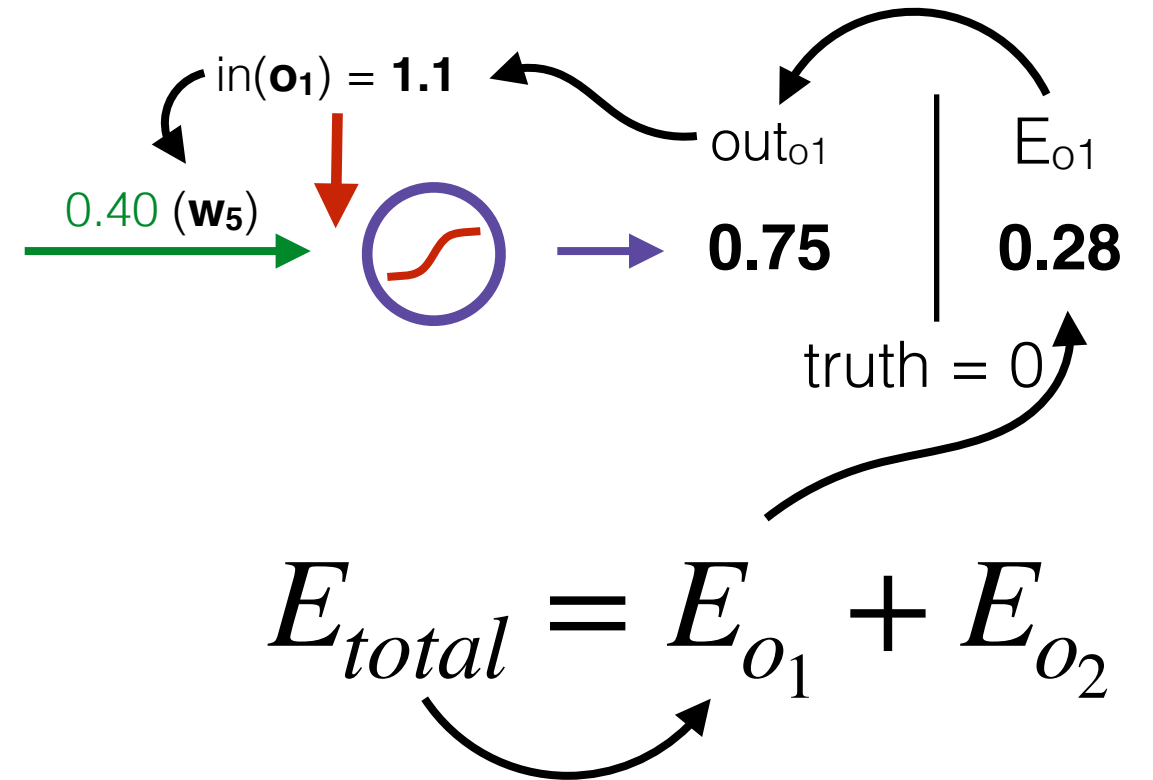
$$E_{total} = E_{o_1} + E_{o_2}$$

$$\frac{\partial E_{total}}{\partial w_5} = 1 * \frac{\partial E_{o_1}}{\partial out_{o_1}} * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$



We know this value

Backpropagation algorithm



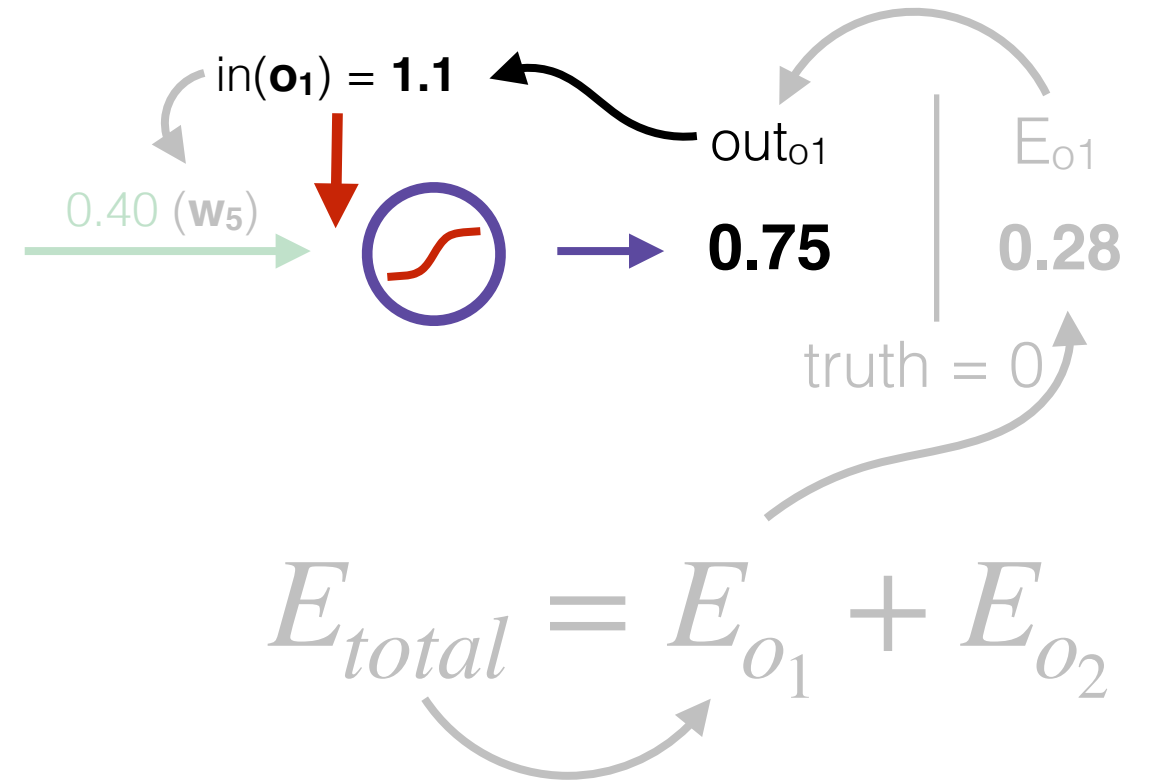
Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o_1}}{\partial in_{o_1}} * \frac{\partial in_{o_1}}{\partial w_5}$$



We know this value

Backpropagation algorithm

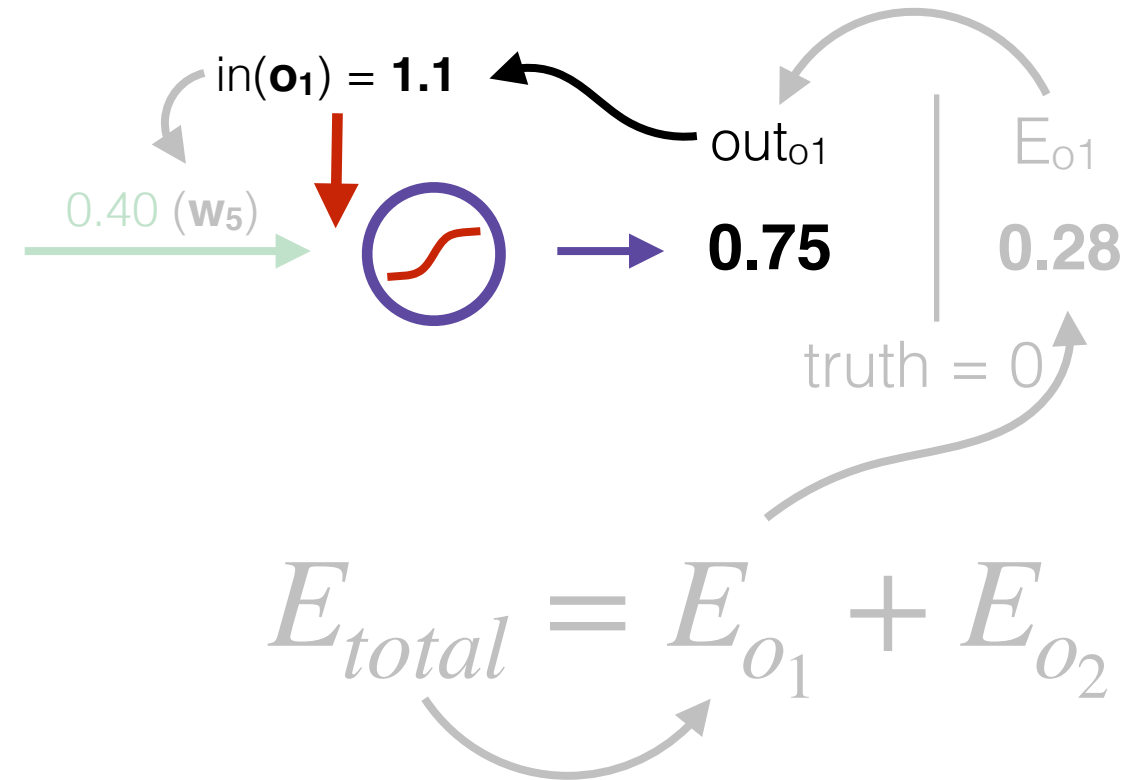


Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



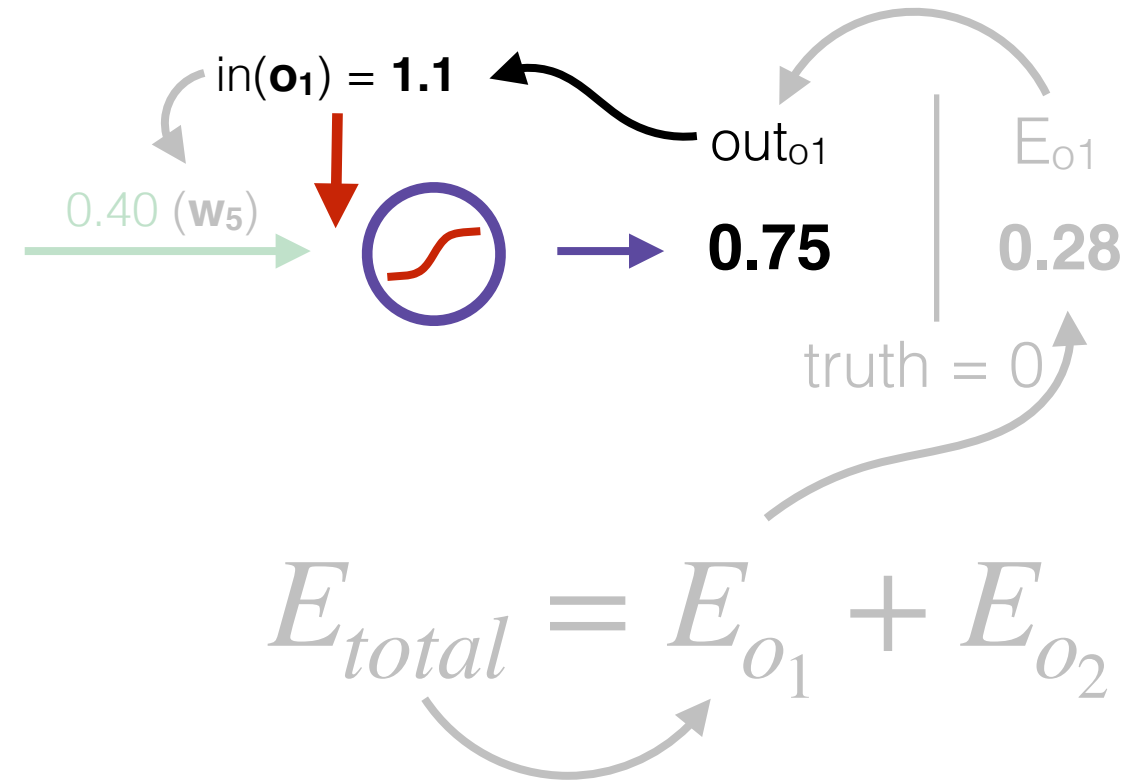
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

$$out_{o1} = \frac{1}{1 + e^{-in_{o1}}}$$

$$\frac{\partial out_{o1}}{\partial in_{o1}} = \frac{\partial}{\partial in_{o1}} \left(\frac{1}{1 + e^{-in_{o1}}} \right)$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

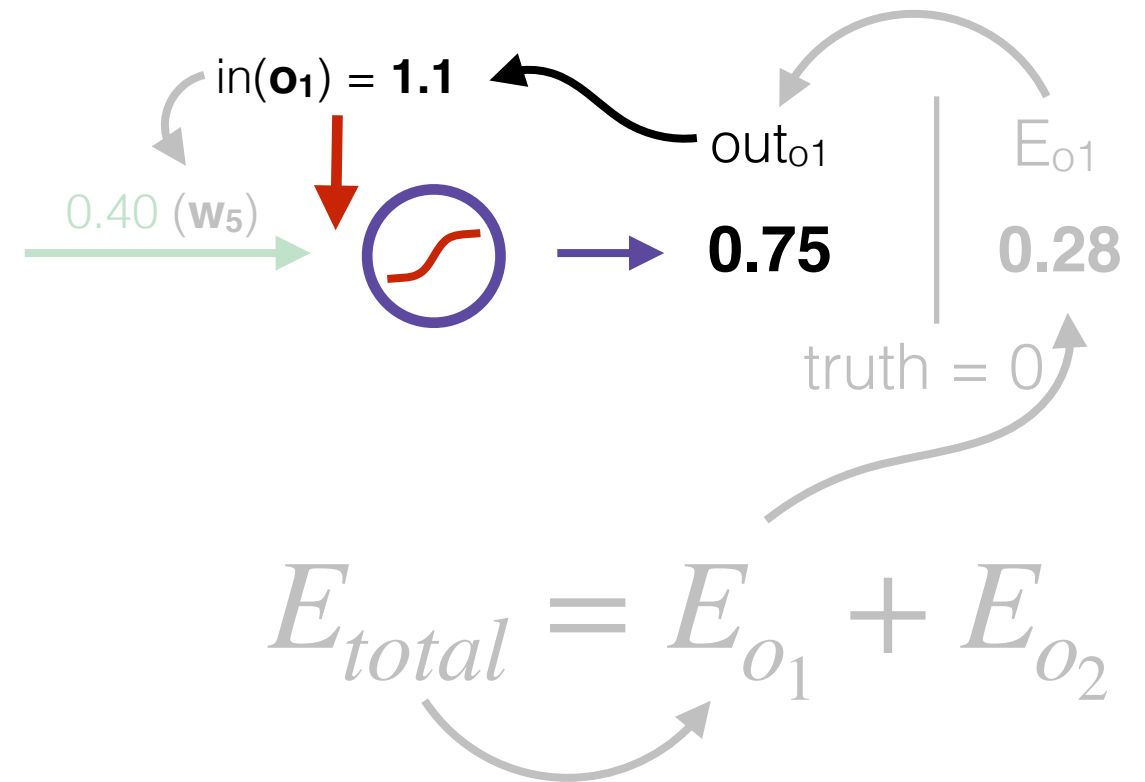
$$out_{o1} = \frac{1}{1 + e^{-in_{o1}}}$$

There is a **shortcut**

$$\frac{\partial out_{o1}}{\partial in_{o1}} = \frac{\partial}{\partial in_{o1}} \left(\frac{1}{1 + e^{-in_{o1}}} \right)$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

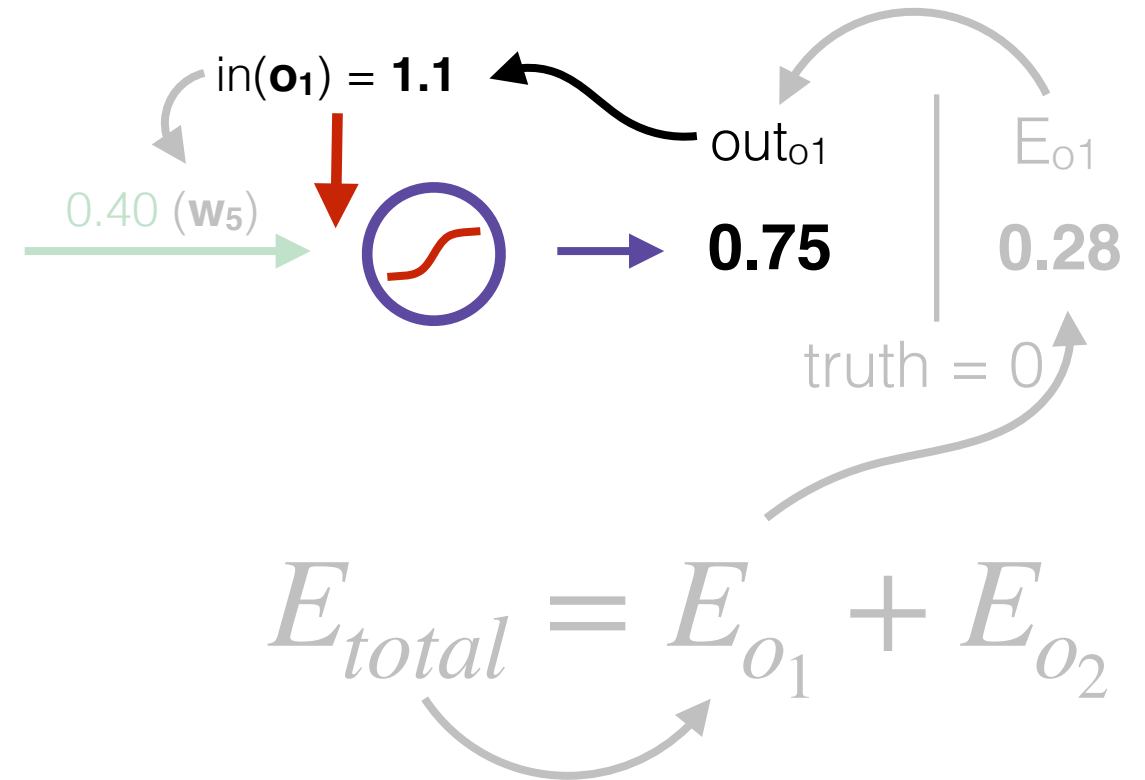
$$out_{o1} = \frac{1}{1 + e^{-in_{o1}}}$$

There is a **shortcut**

$$\frac{\partial out_{o1}}{\partial in_{o1}} = out_{o1} * (1 - out_{o1})$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

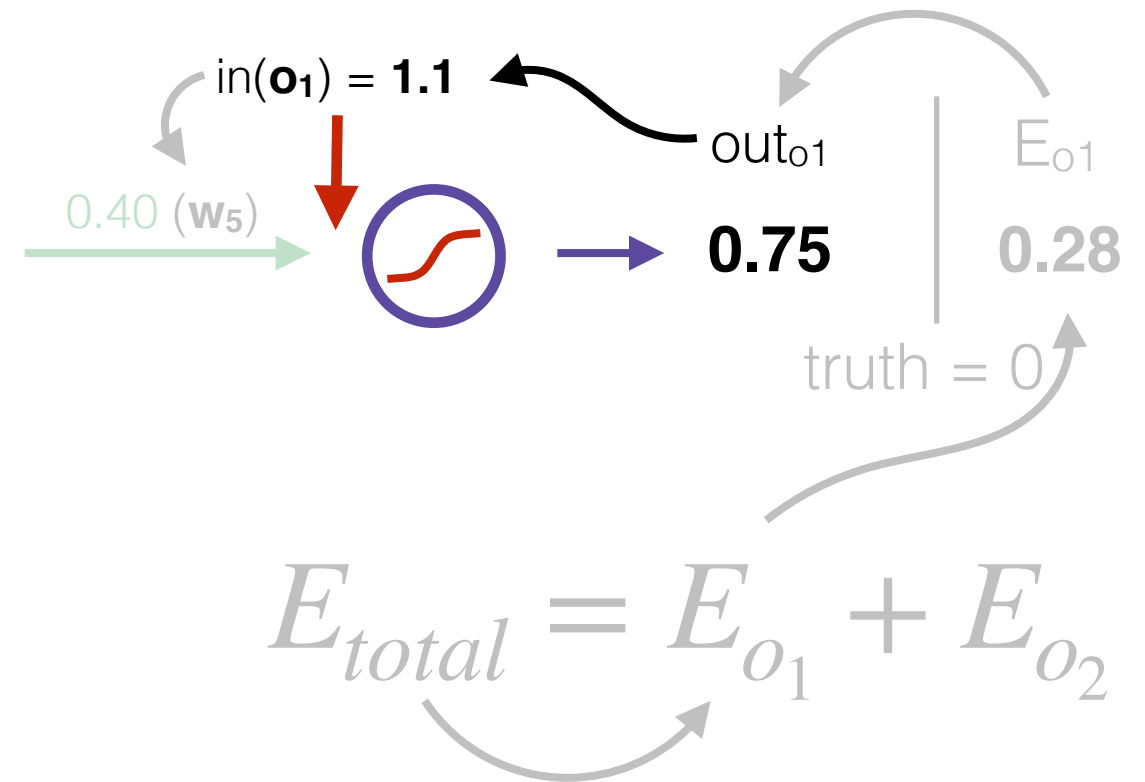
$$out_{o1} = \frac{1}{1 + e^{-in_{o1}}}$$

There is a **shortcut**

$$\frac{\partial out_{o1}}{\partial in_{o1}} = 0.75 * (1 - 0.75)$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



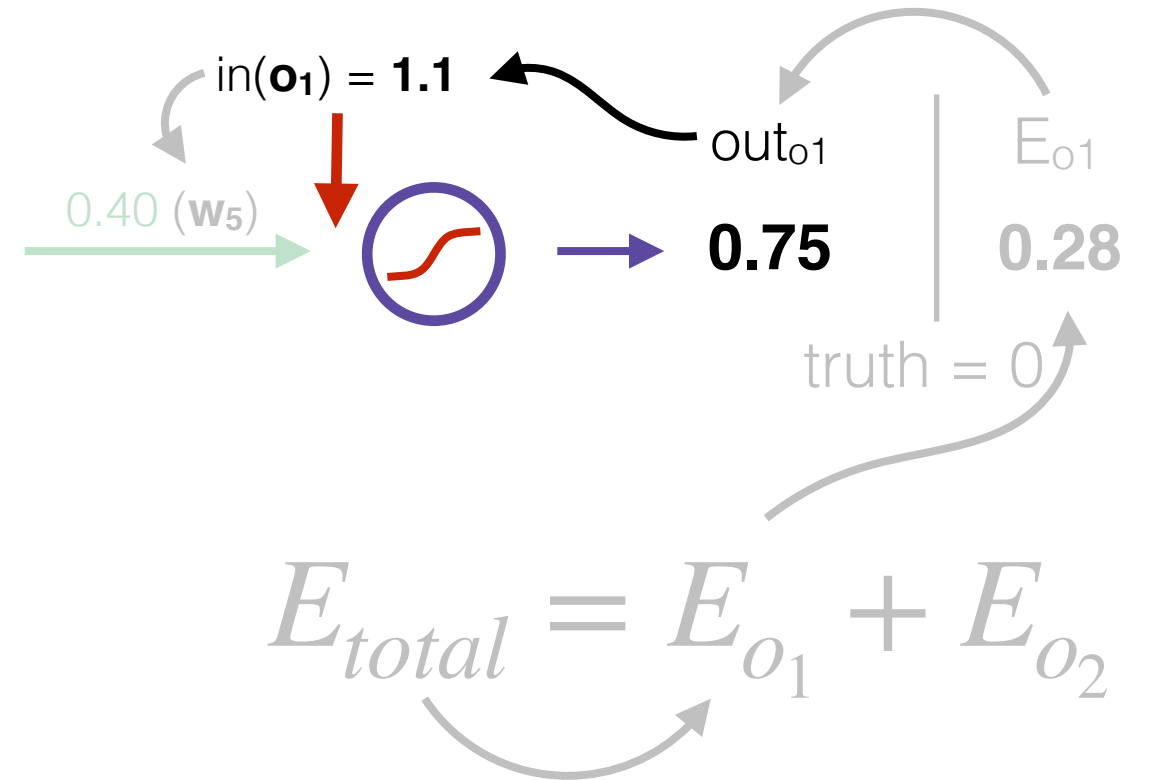
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * \frac{\partial out_{o1}}{\partial in_{o1}} * \frac{\partial in_{o1}}{\partial w_5}$$

$$out_{o1} = \frac{1}{1 + e^{-in_{o1}}}$$

There is a **shortcut**

$$\frac{\partial out_{o1}}{\partial in_{o1}} = 0.186$$

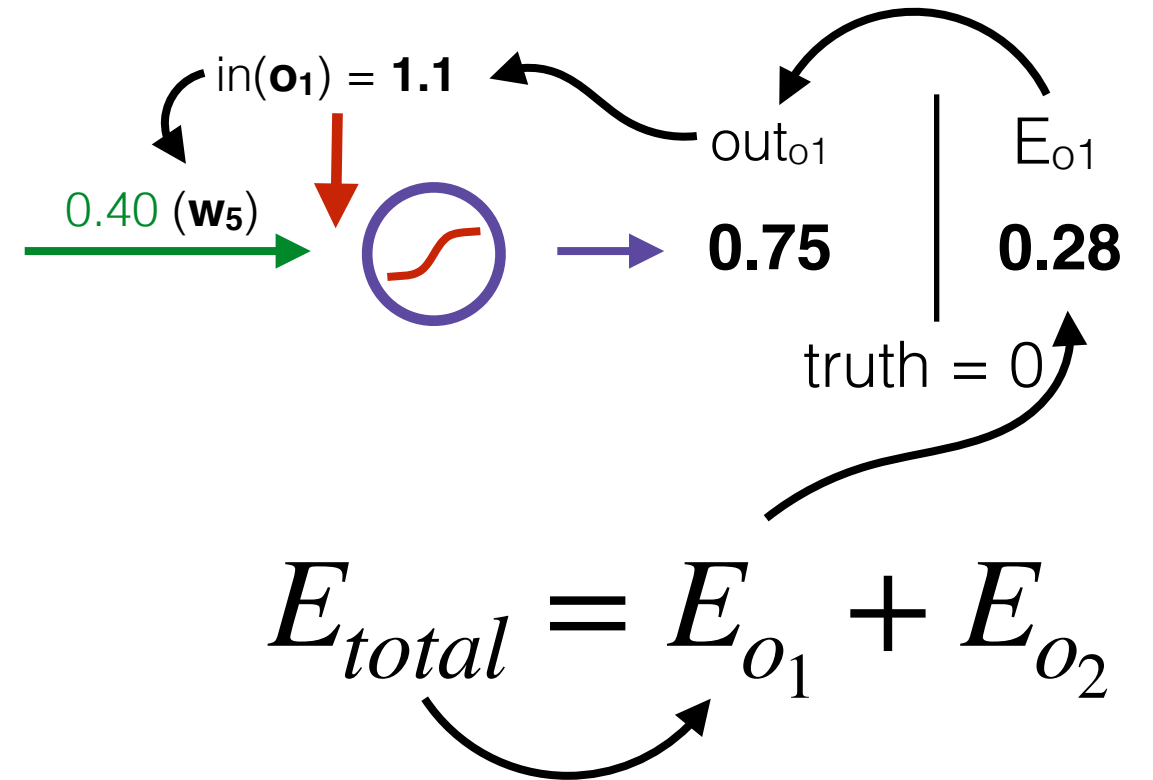
Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o1}}{\partial w_5}$$

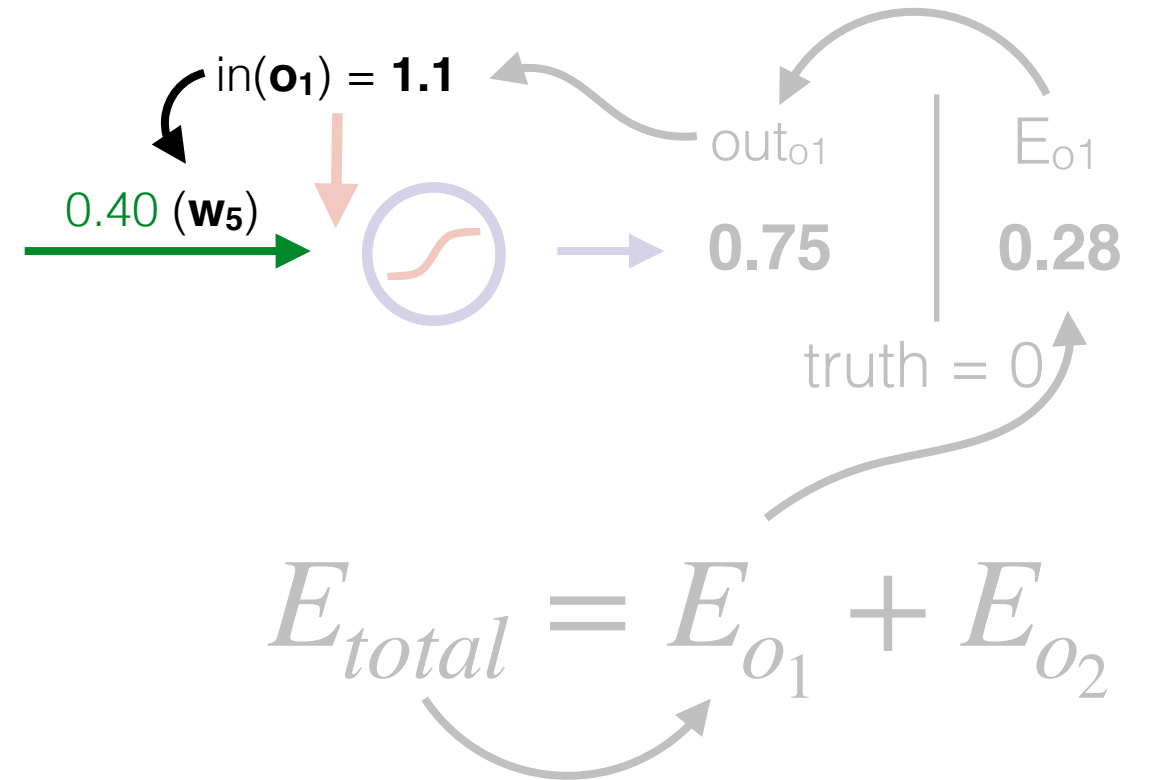
Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o1}}{\partial w_5}$$

Backpropagation algorithm

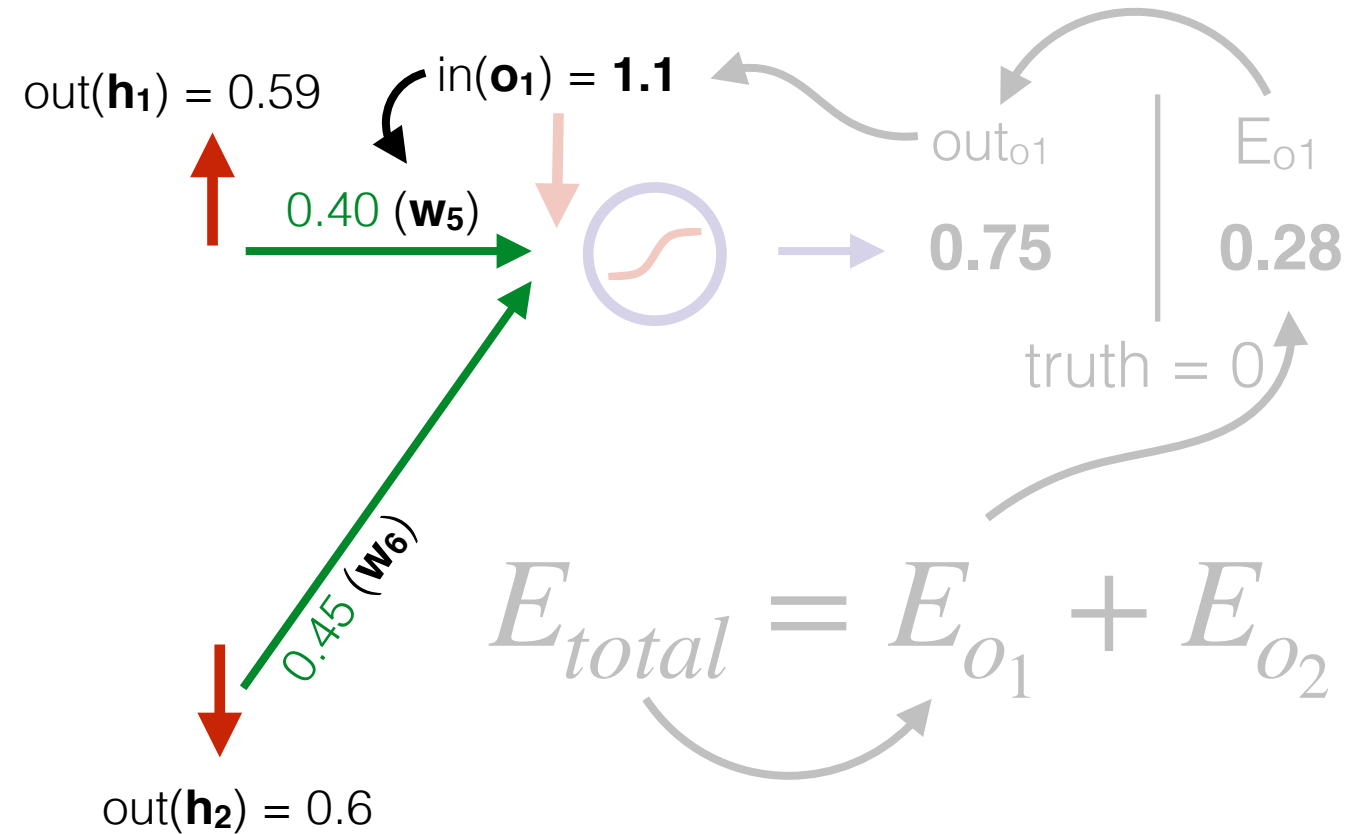


Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5

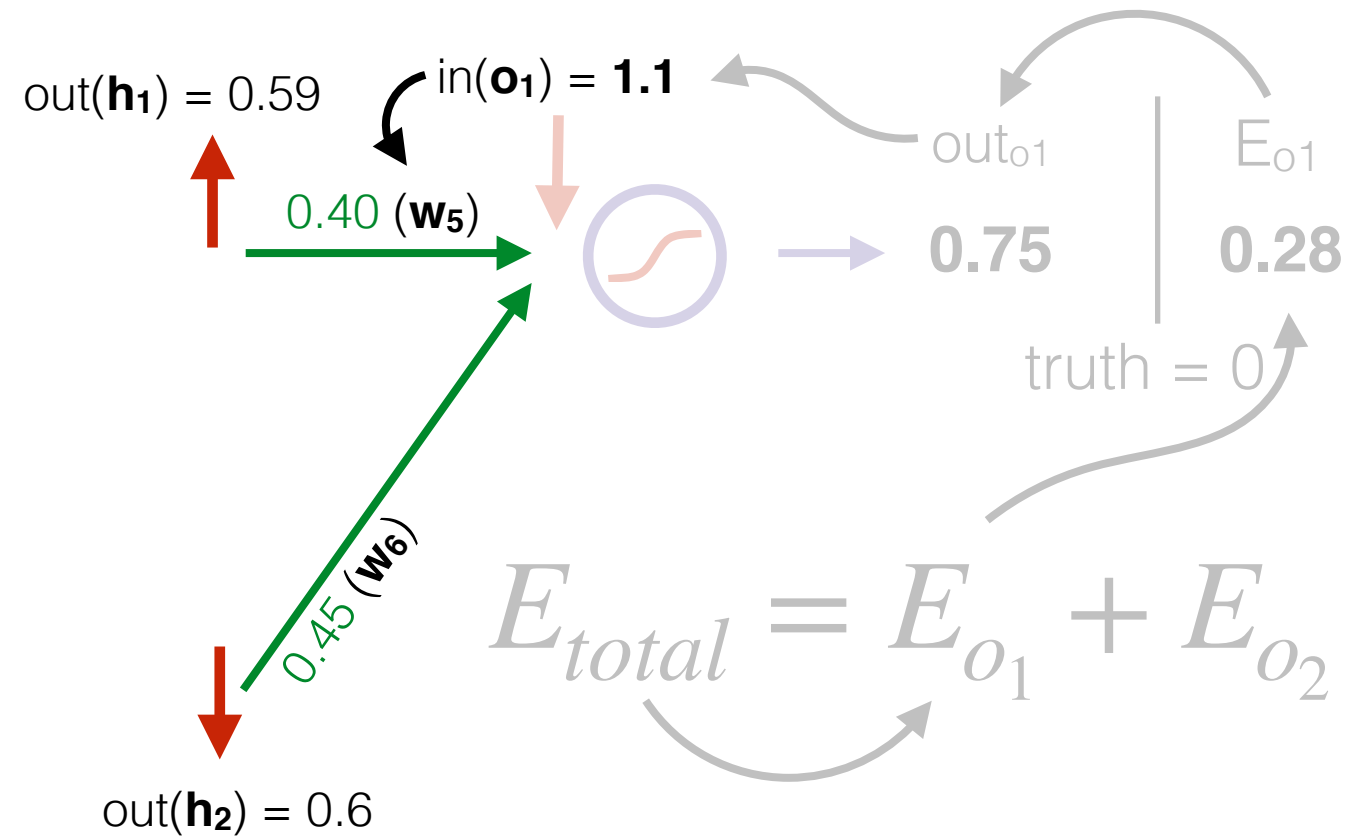


$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o1}}{\partial w_5}$$

$$in_{o1} = out_{h1} * w_5 + out_{h2} * w_6 + b_{21}$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



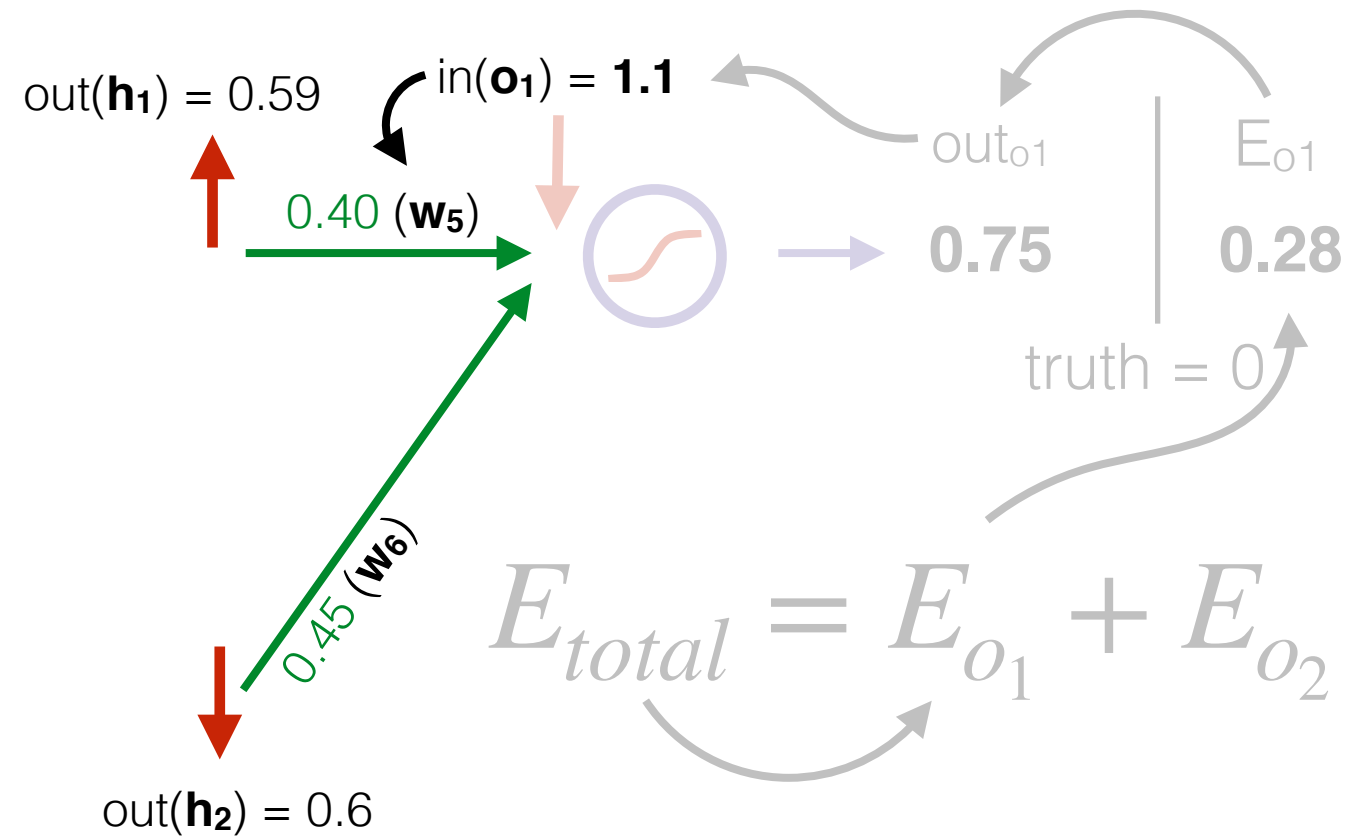
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} =$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



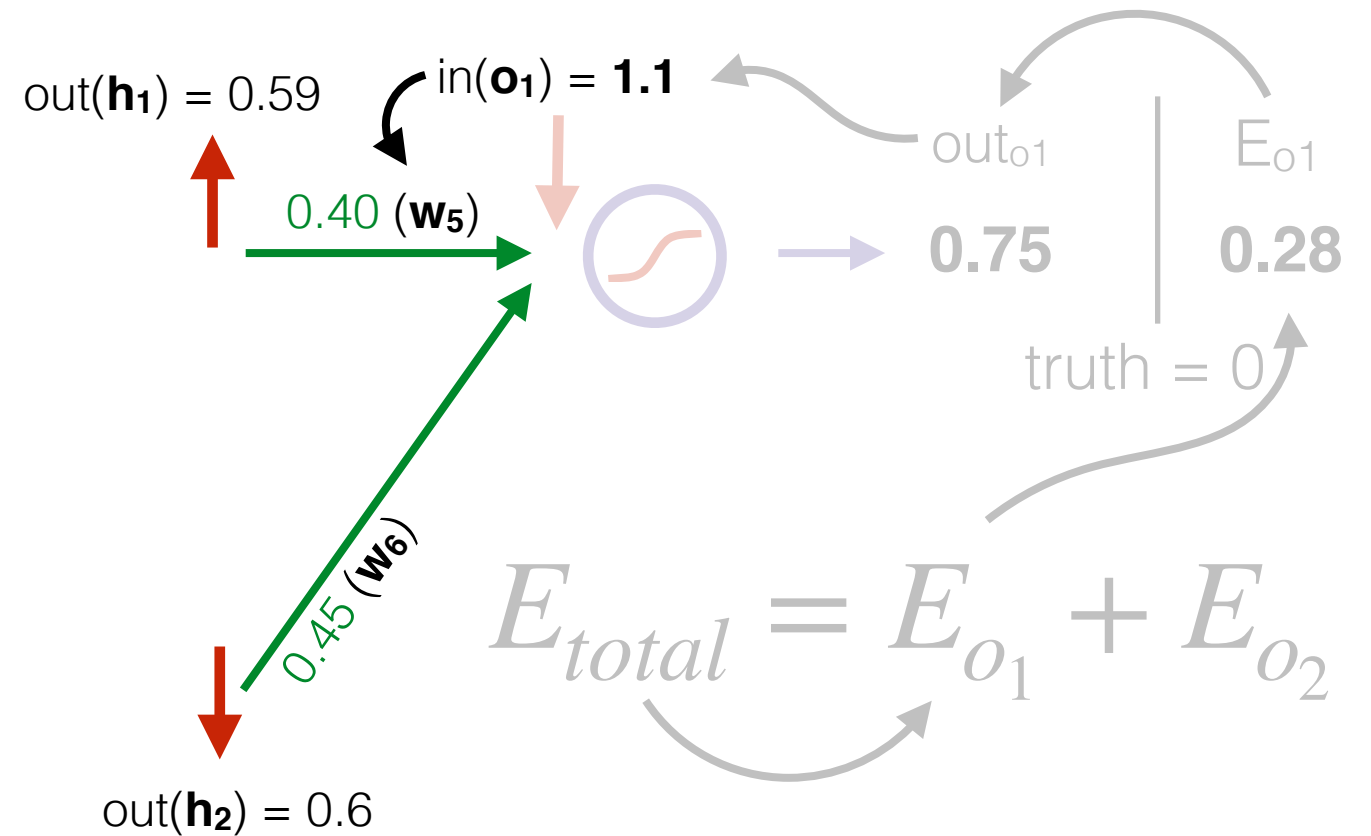
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = \frac{\partial}{\partial w_5} (out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21})$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



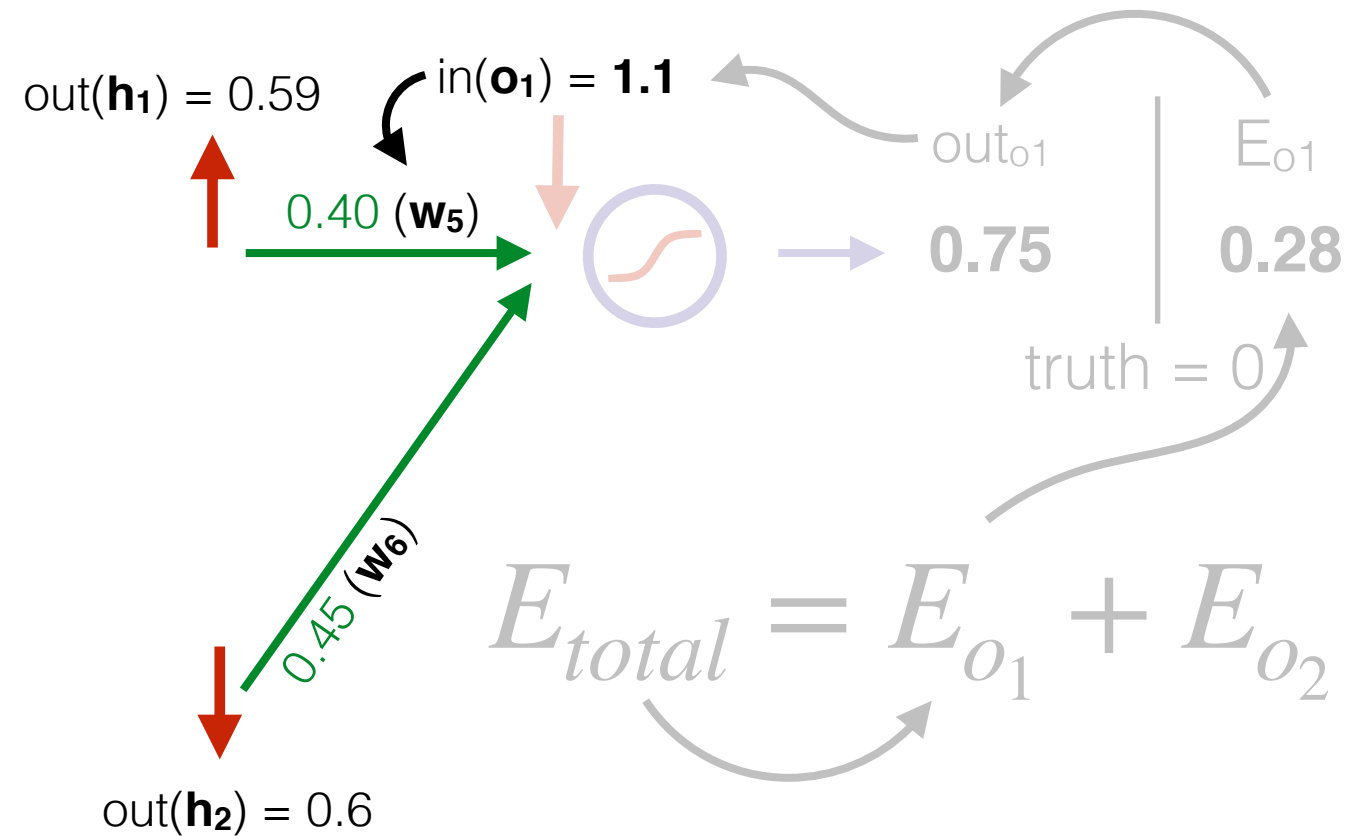
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = \frac{\partial}{\partial w_5}(out_{h_1} * w_5) + \frac{\partial}{\partial w_5}(out_{h_2} * w_6) + \frac{\partial}{\partial w_5}(b_{21})$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



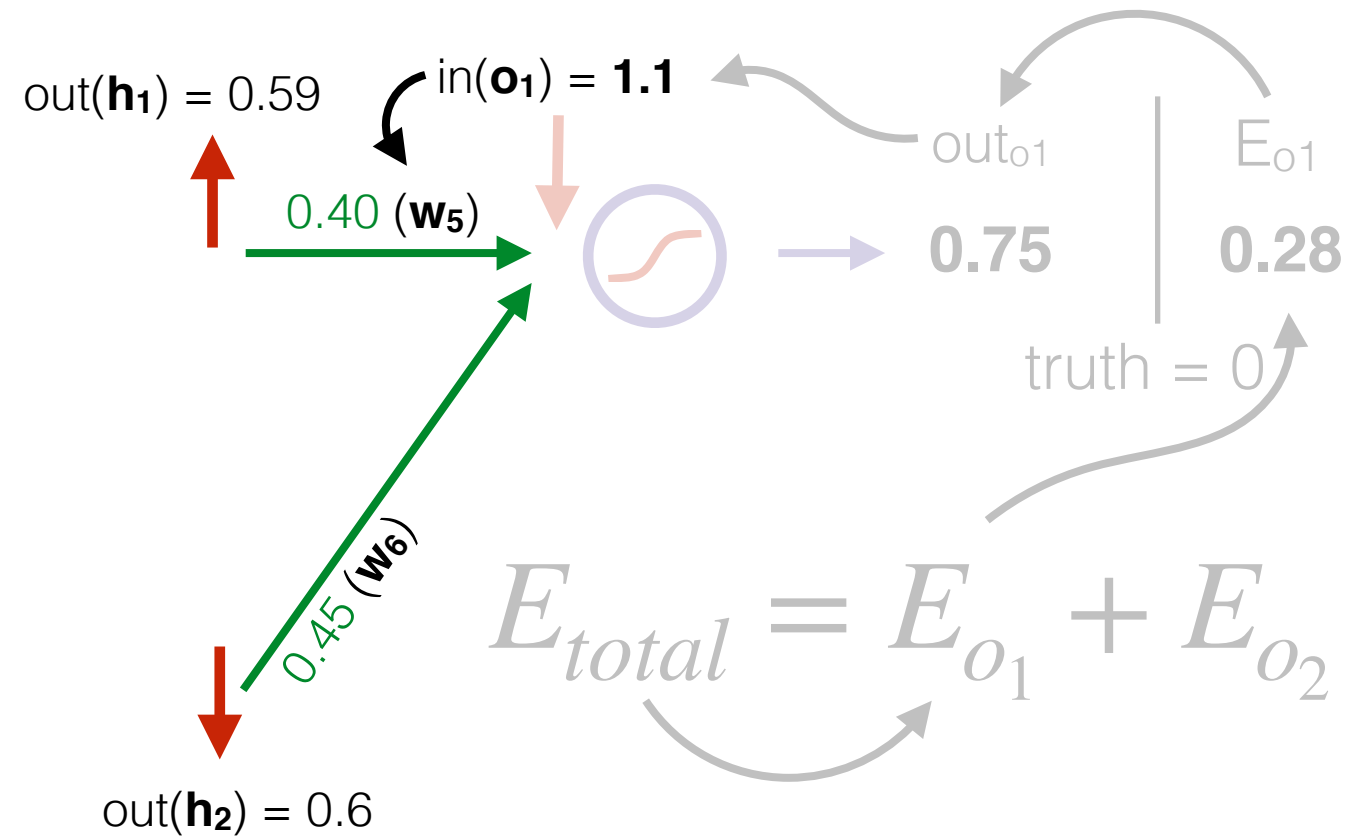
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = \frac{\partial}{\partial w_5}(out_{h_1} * w_5) + \frac{\partial}{\partial w_5}(out_{h_2} * w_6) + 0$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



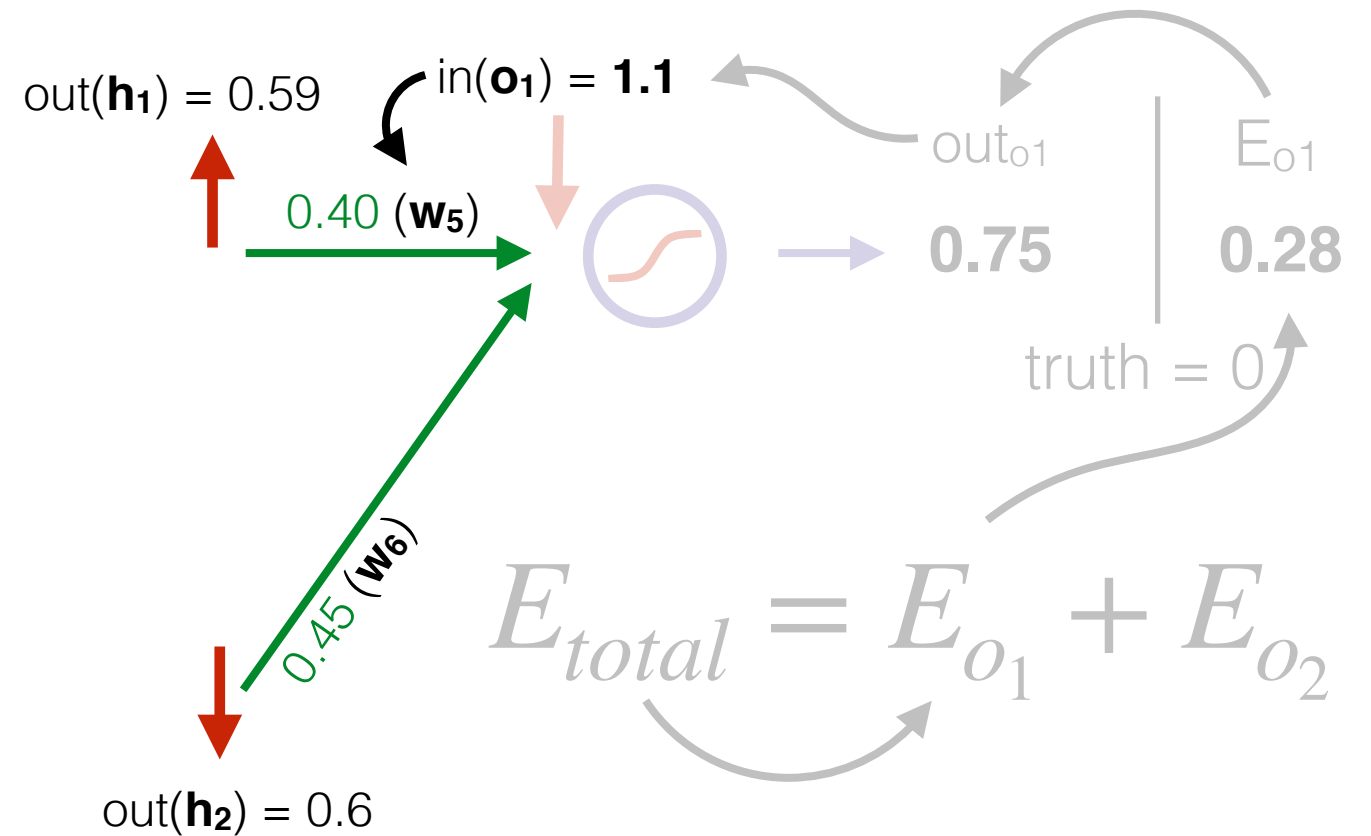
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = \frac{\partial}{\partial w_5} (out_{h_1} * w_5) + 0 + 0$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5



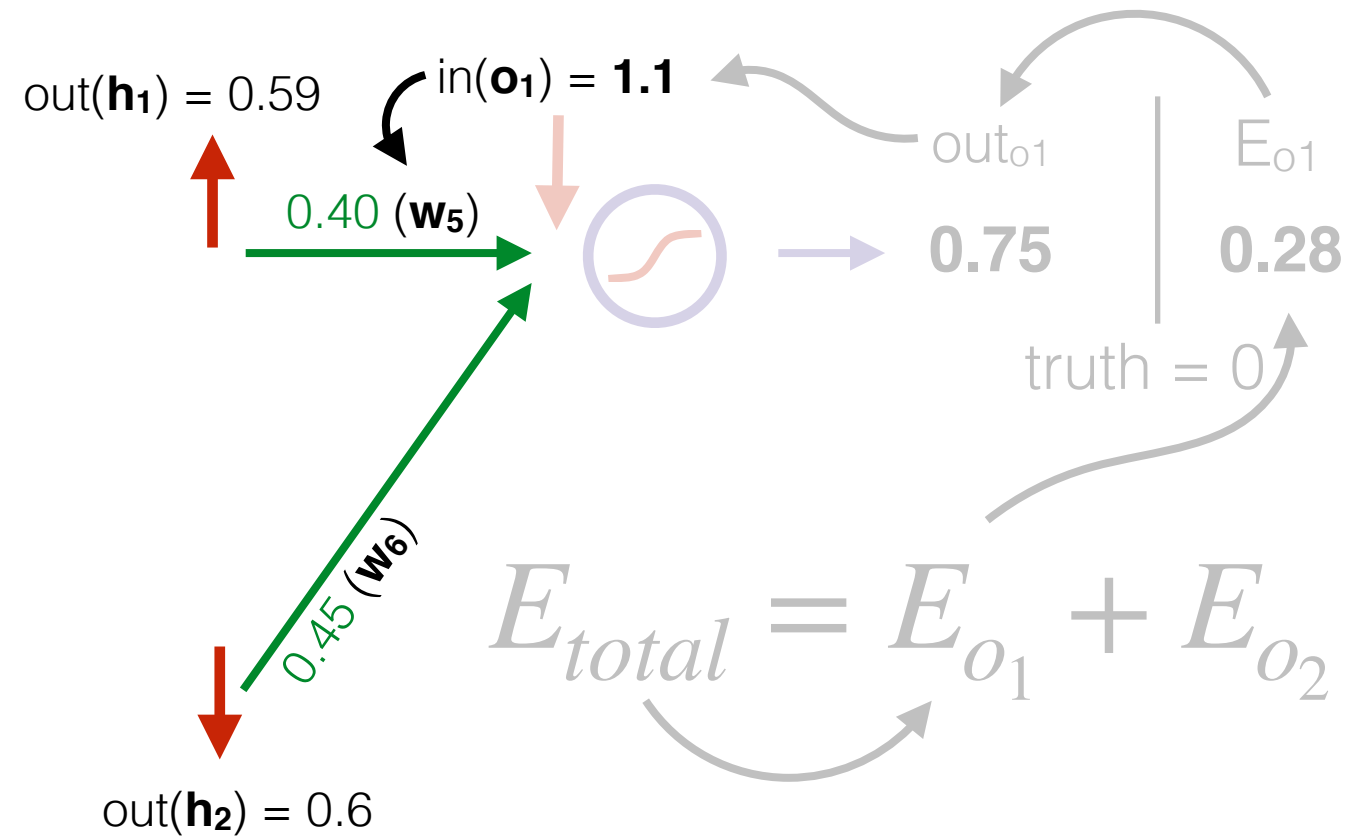
$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = out_{h_1} + 0 + 0$$

Backpropagation algorithm

Let's try to reduce **total error** by changing w_5

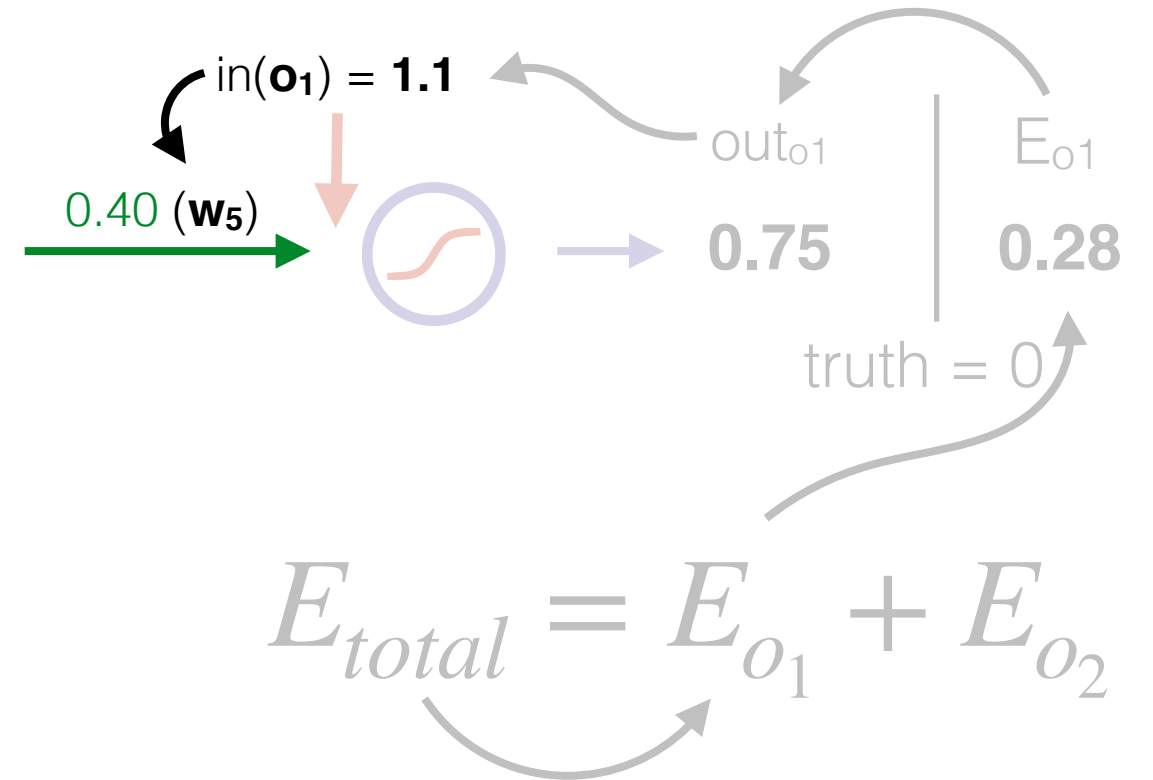


$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * \frac{\partial in_{o_1}}{\partial w_5}$$

$$in_{o_1} = out_{h_1} * w_5 + out_{h_2} * w_6 + b_{21}$$

$$\frac{\partial in_{o_1}}{\partial w_5} = 0.59$$

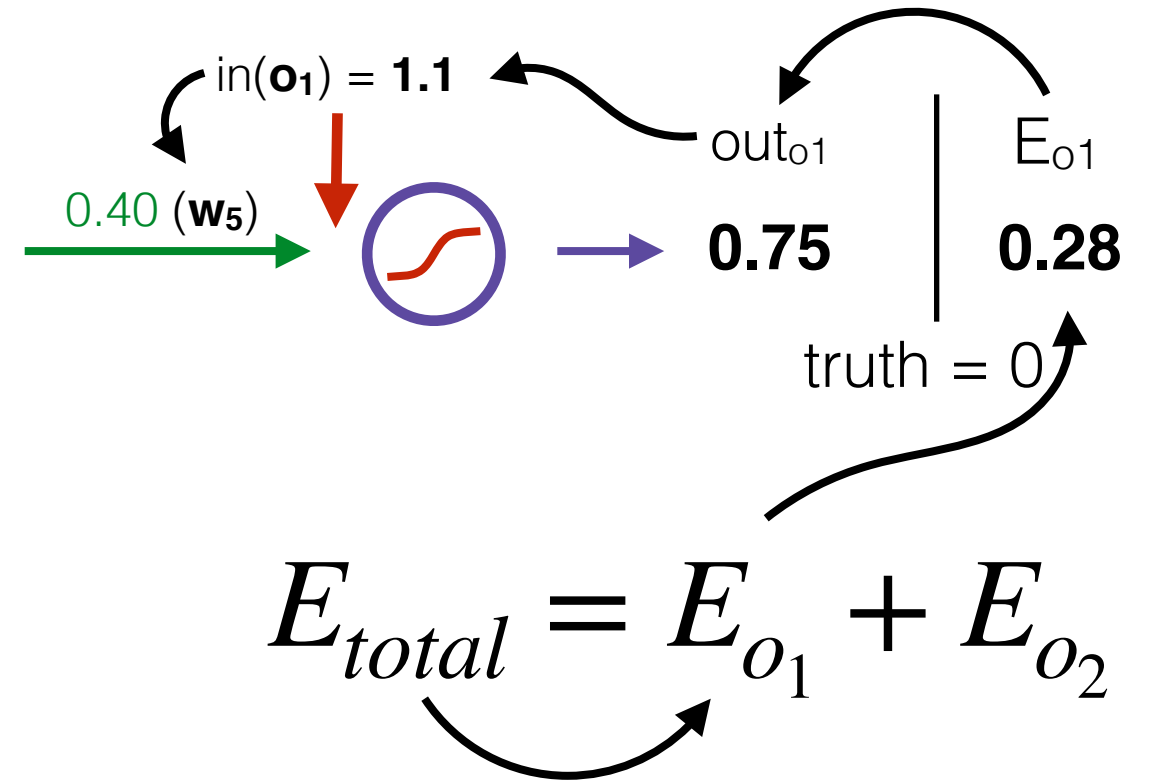
Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59$$

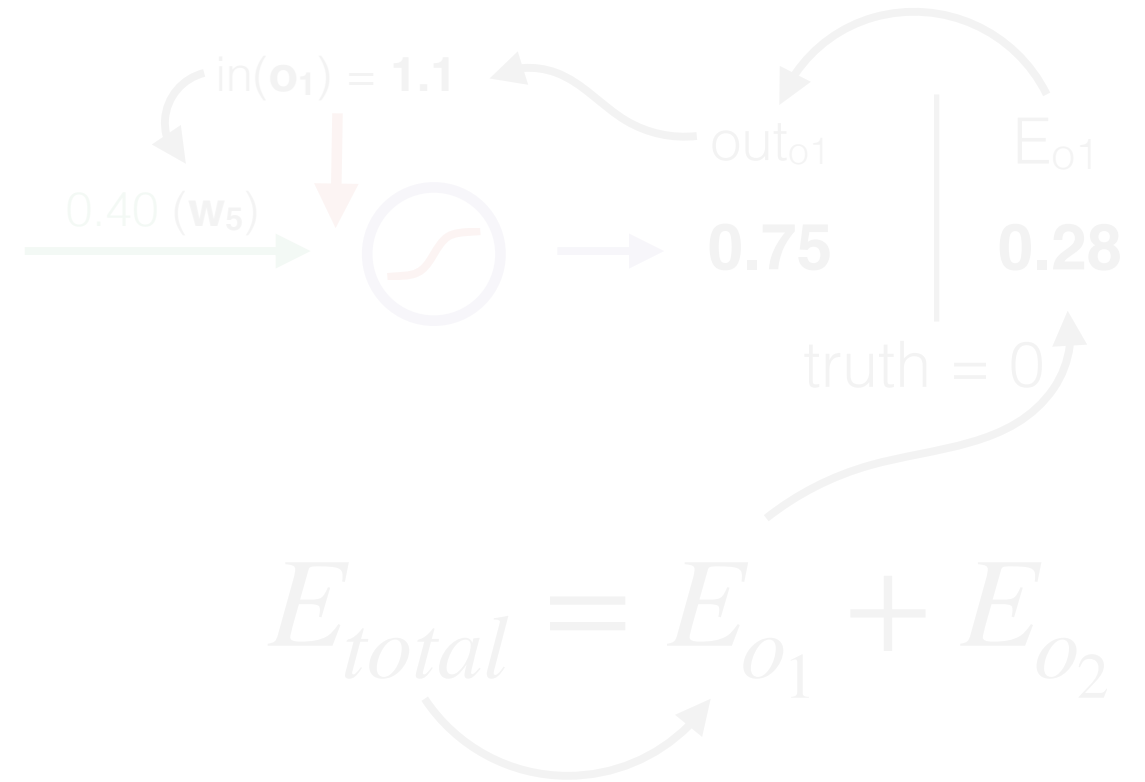
Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59 = 0.082$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

What does this **actually** say?

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59 = 0.082$$

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

What does this **actually** say?

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59 = 0.082$$

If we **increase** w_5 , E_{total} will **increase**

Backpropagation algorithm



Let's try to reduce **total error** by changing w_5

What does this **actually** say?

$$\frac{\partial E_{total}}{\partial w_5} = 1 * 0.75 * 0.186 * 0.59 = 0.082$$

If we **increase** w_5 , E_{total} will **increase**

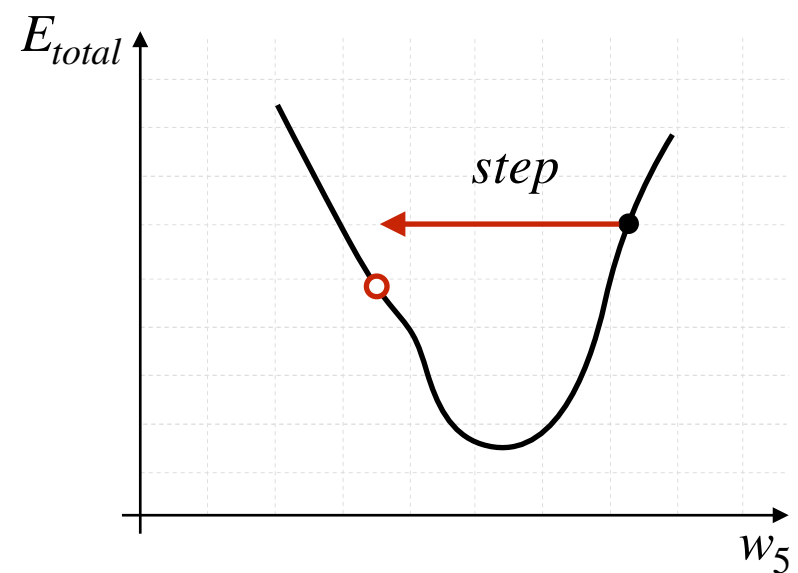
So we need to **decrease** it!

$$w_5 = w_5 - \frac{\partial E_{total}}{\partial w_5}$$

So we need to **decrease** it!

$$w_5 = w_5 - \frac{\partial E_{total}}{\partial w_5}$$

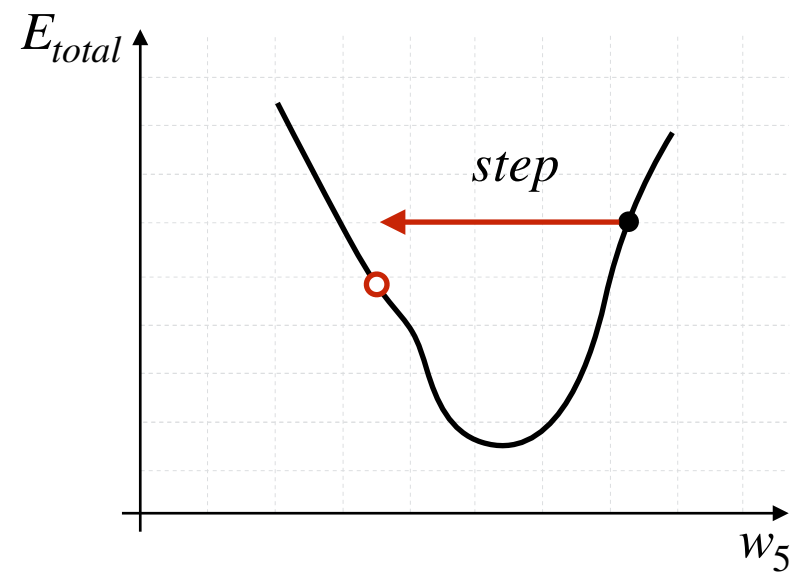
If update by a lot,
we can miss the optimum



So we need to **decrease** it!

$$w_5 = w_5 - \frac{\partial E_{total}}{\partial w_5}$$

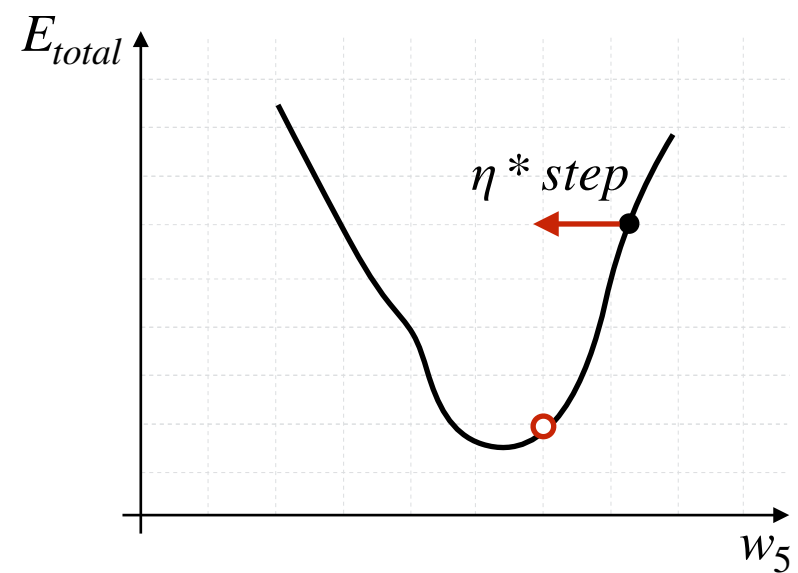
Thus, we make a bit **smaller step**



So we need to **decrease** it!

$$w_5 = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5}$$

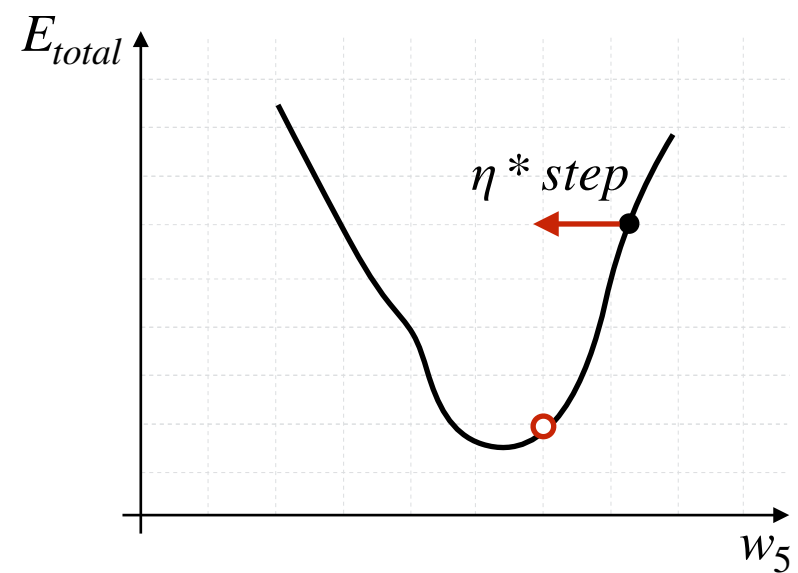
Thus, we make a bit **smaller step**



So we need to **decrease** it!

$$w_5 = w_5 - \eta * 0.082$$

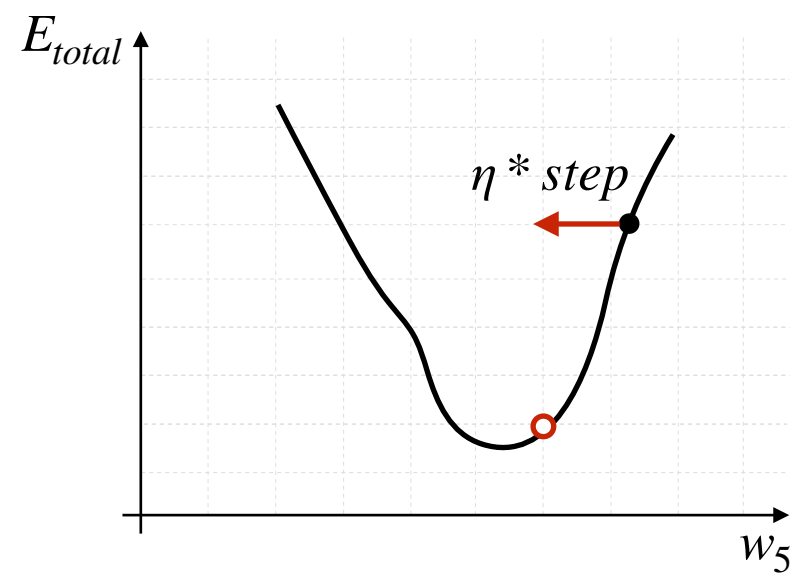
Thus, we make a bit **smaller step**



So we need to **decrease** it!

$$w_5 = 0.4 - \eta * 0.082$$

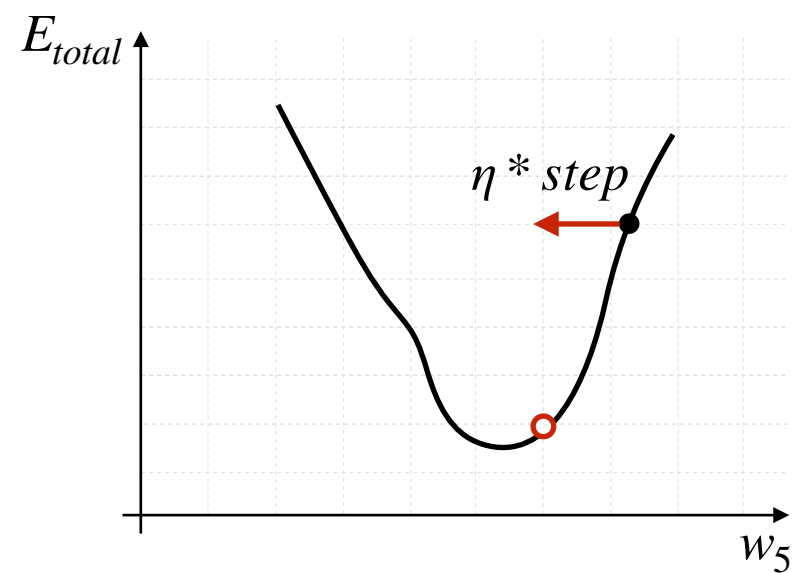
Thus, we make a bit **smaller step**



So we need to **decrease** it!

$$w_5 = 0.4 - 0.5 * 0.082$$

Thus, we make a bit **smaller step**

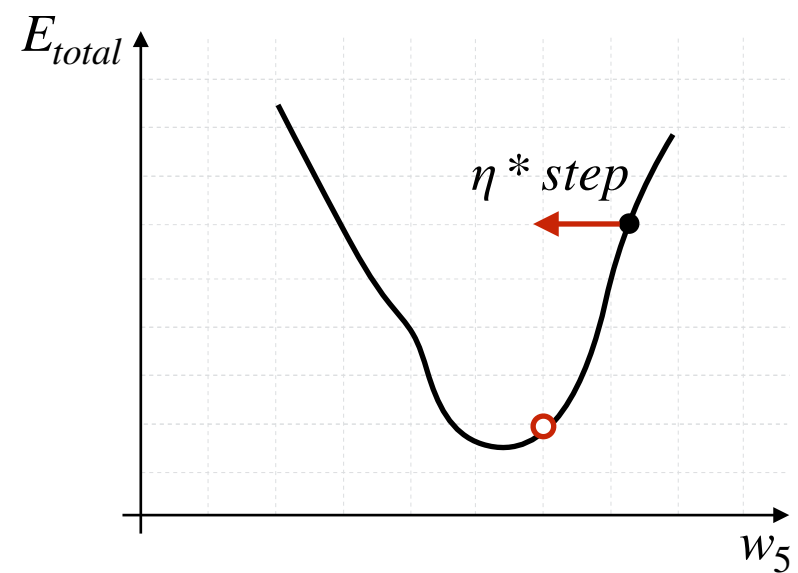


So we need to **decrease** it!

$$w_5 = 0.4 - 0.5 * 0.082$$

Step size is also called **learning rate**, and it can be really small

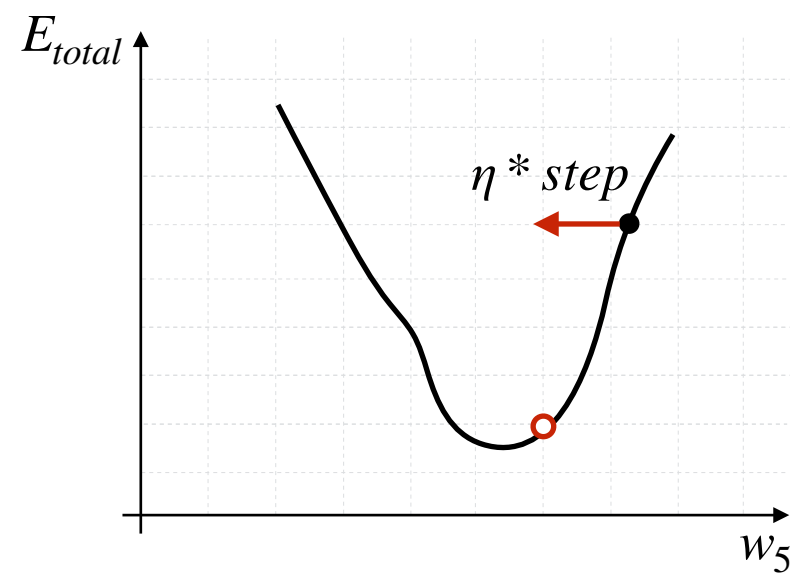
Thus, we make a bit **smaller step**



So we need to **decrease** it!

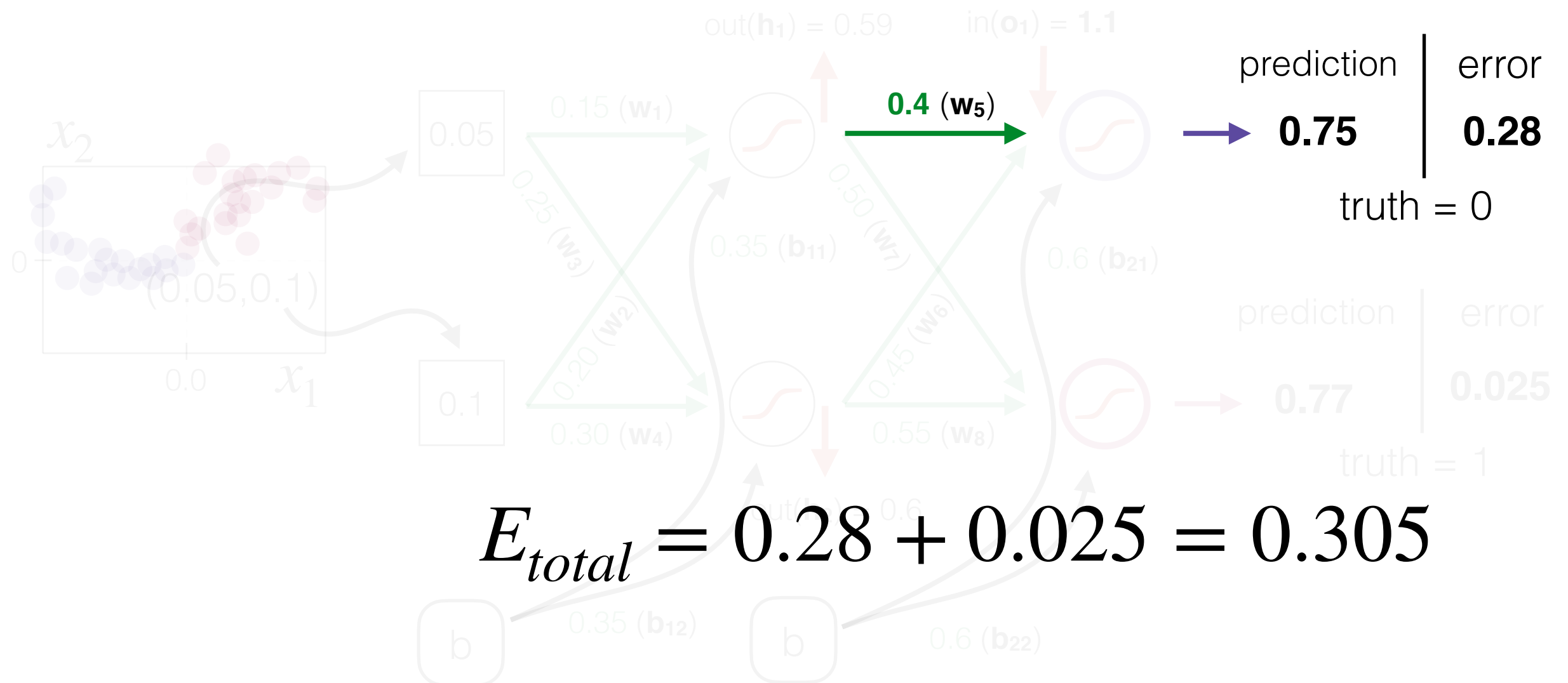
$$w_5 = 0.3585$$

Thus, we make a bit **smaller step**

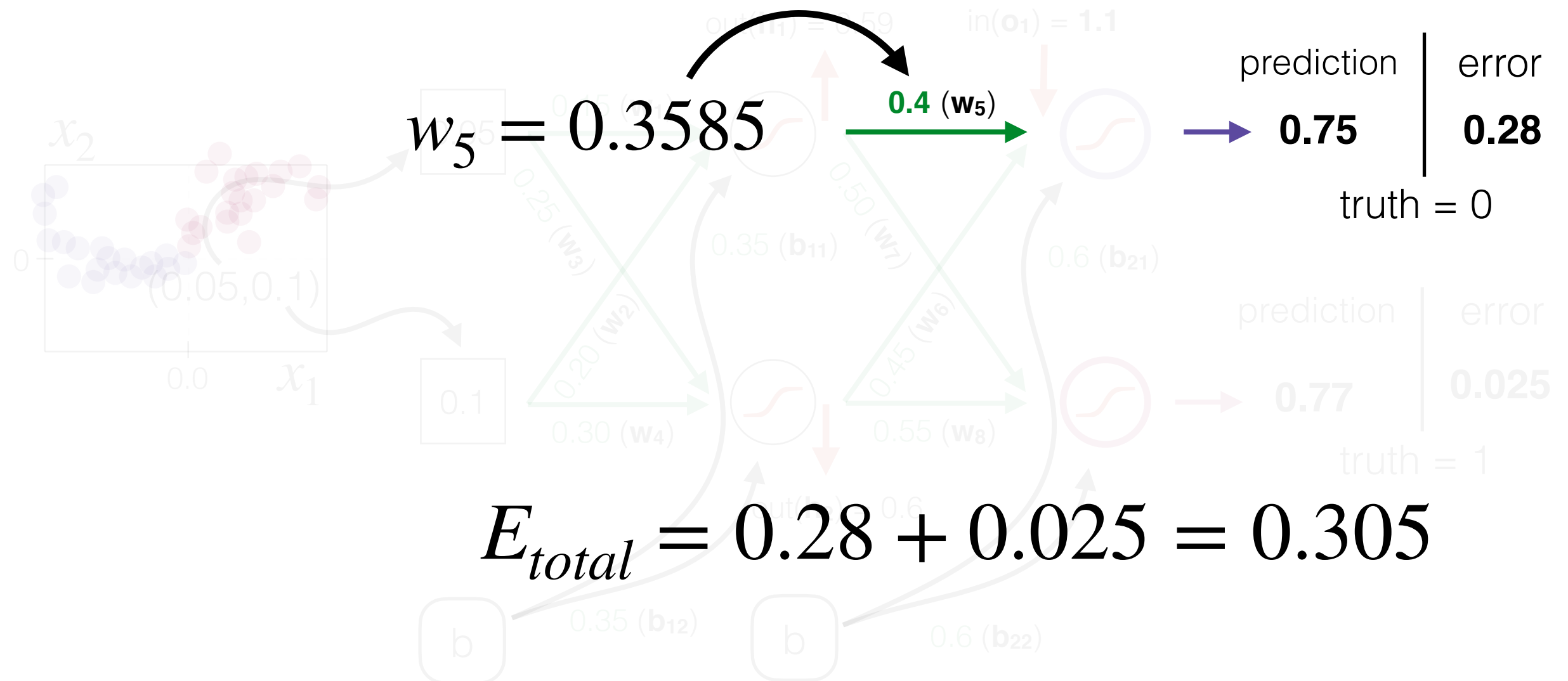


So we need to **decrease** it!

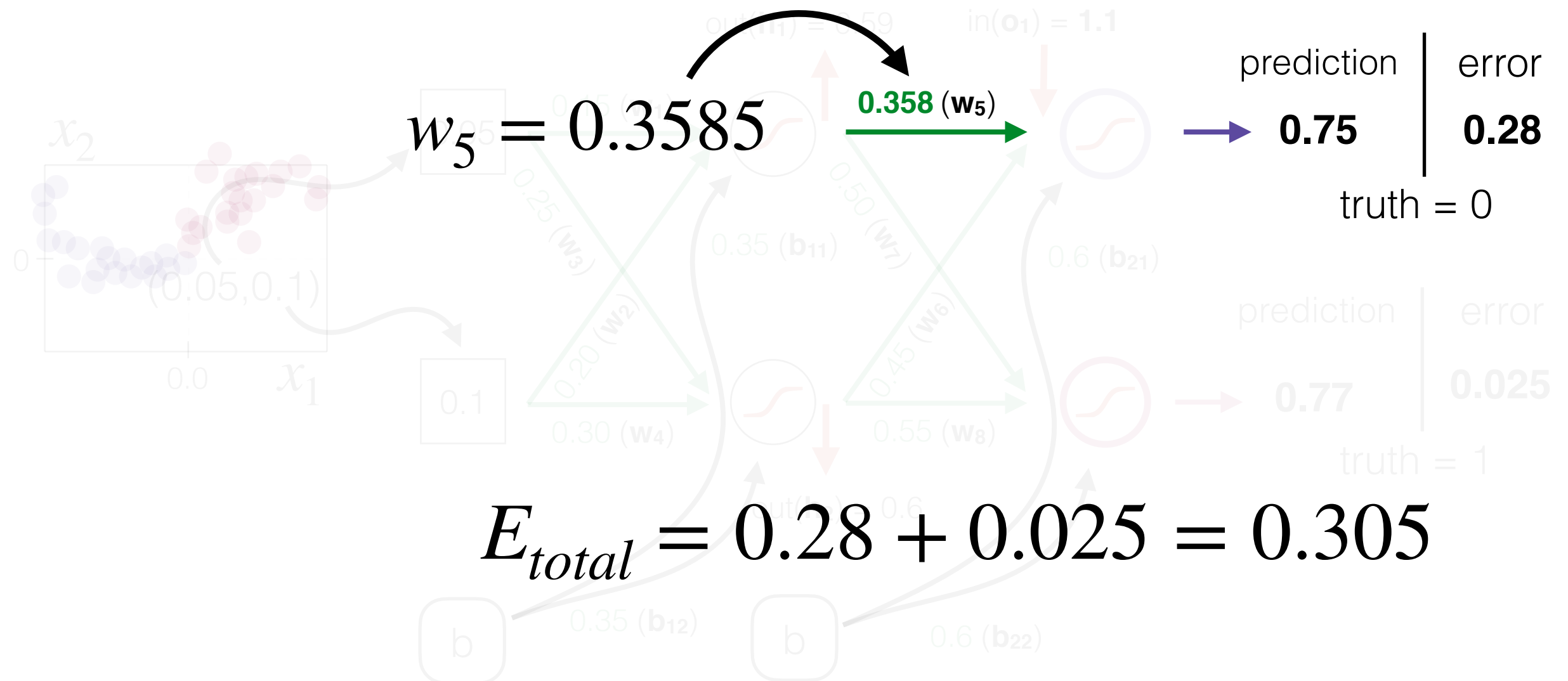
What can we do to make the error **smaller**?



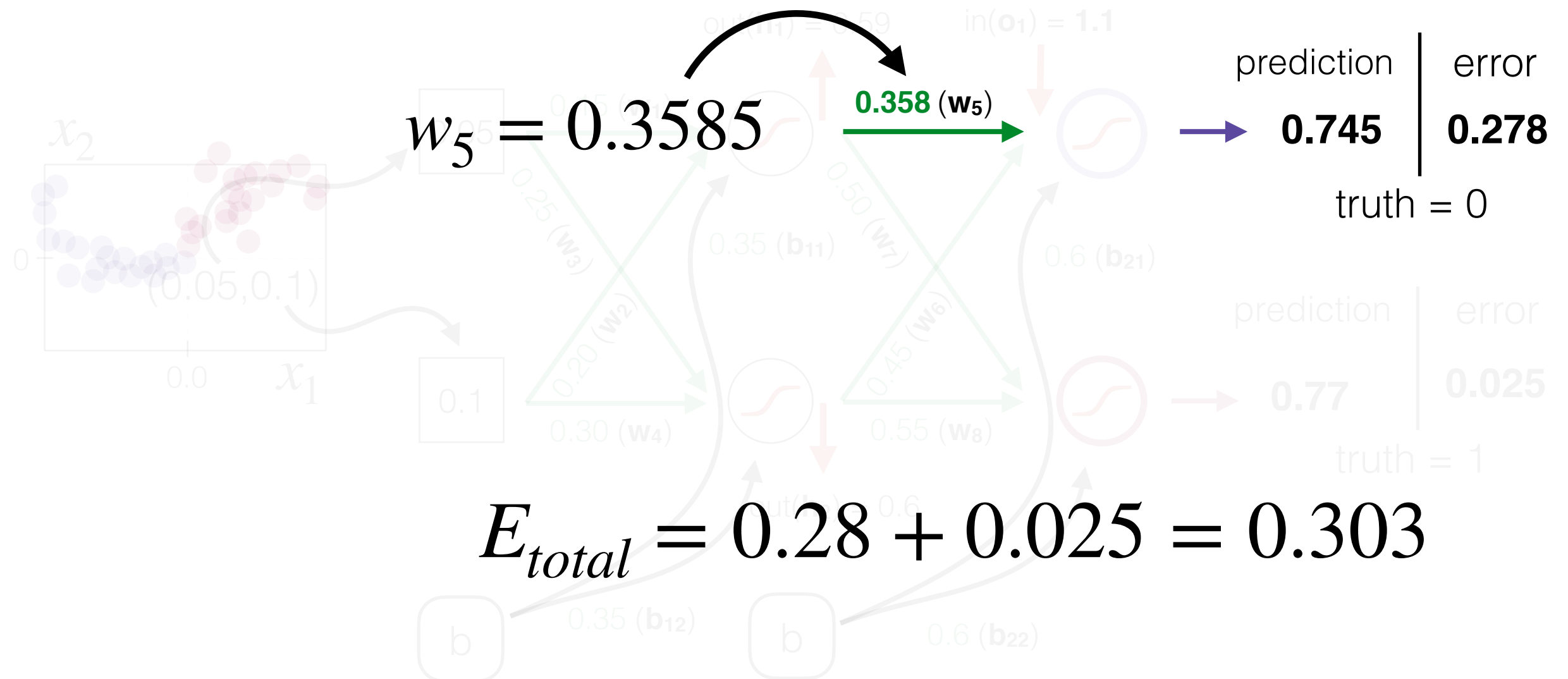
What can we do to make the error **smaller**?



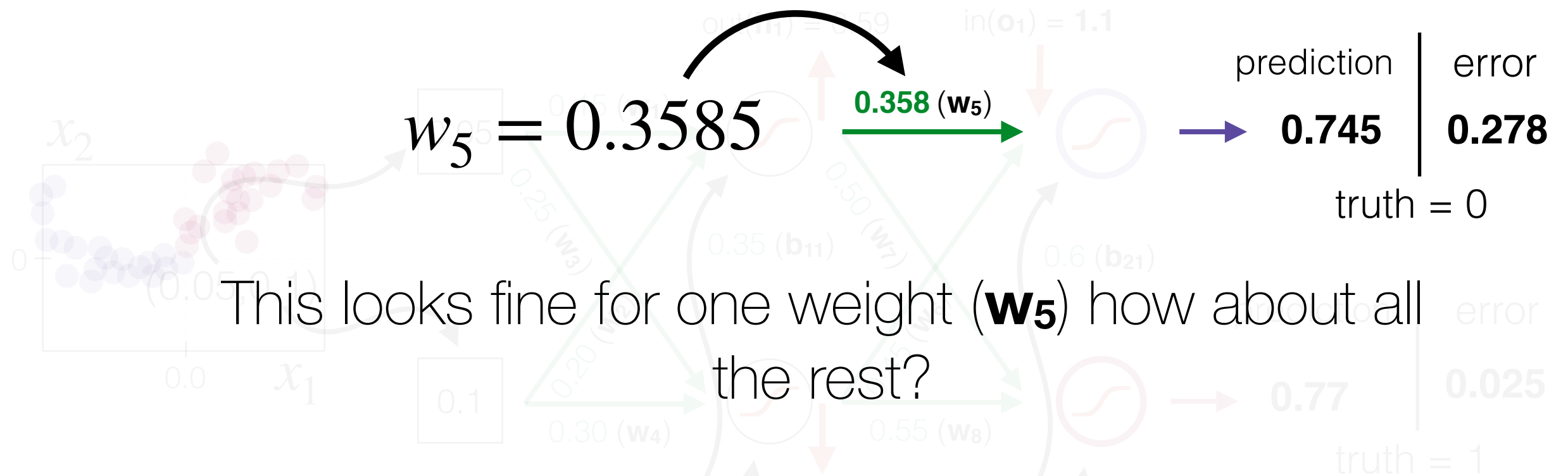
What can we do to make the error **smaller**?



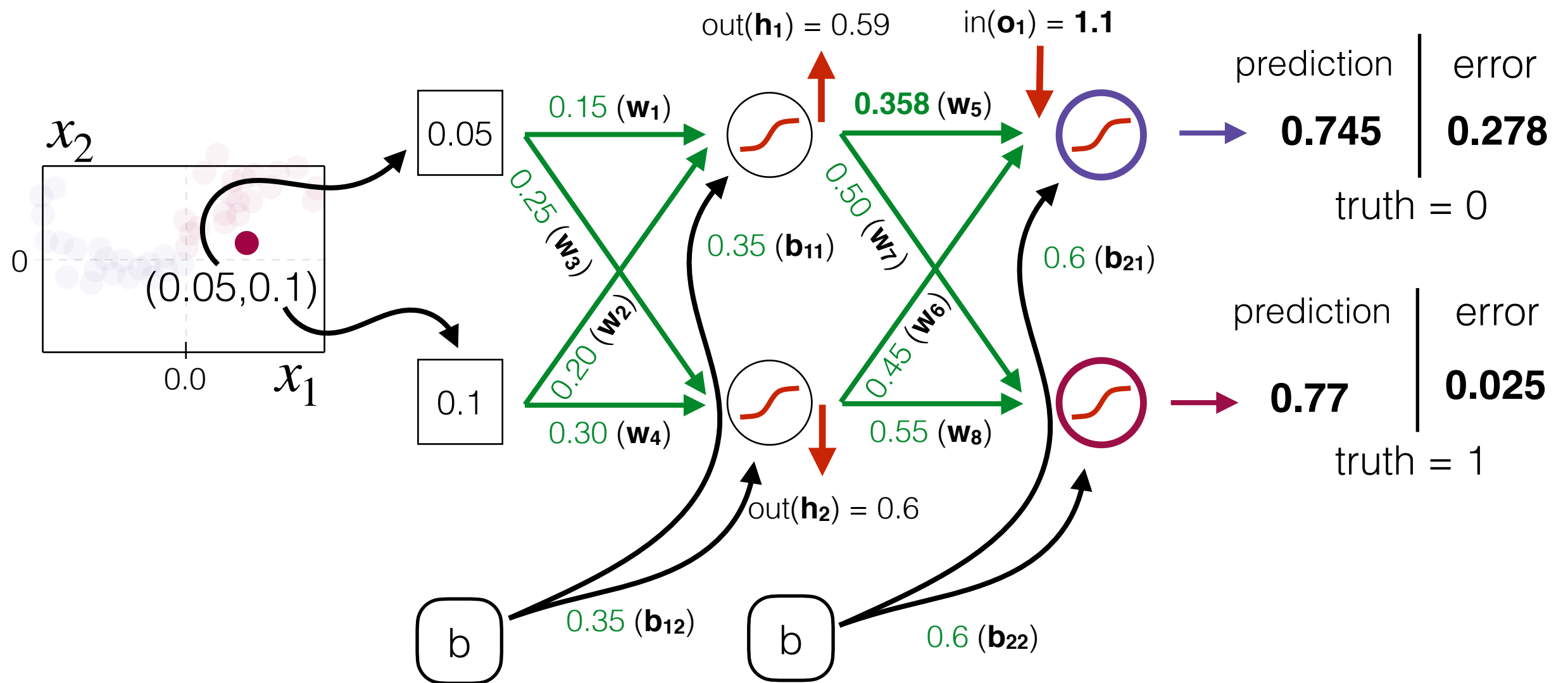
What can we do to make the error **smaller**?

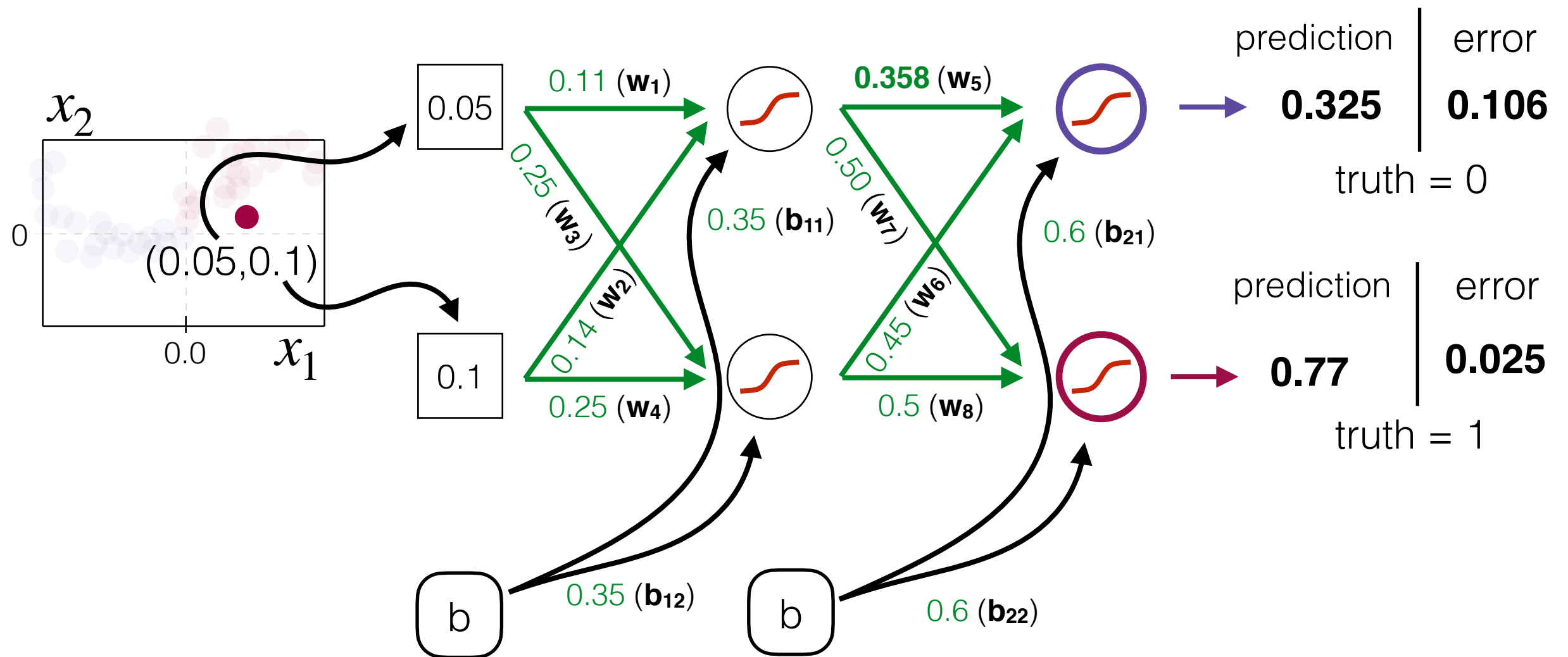


What can we do to make the error **smaller**?

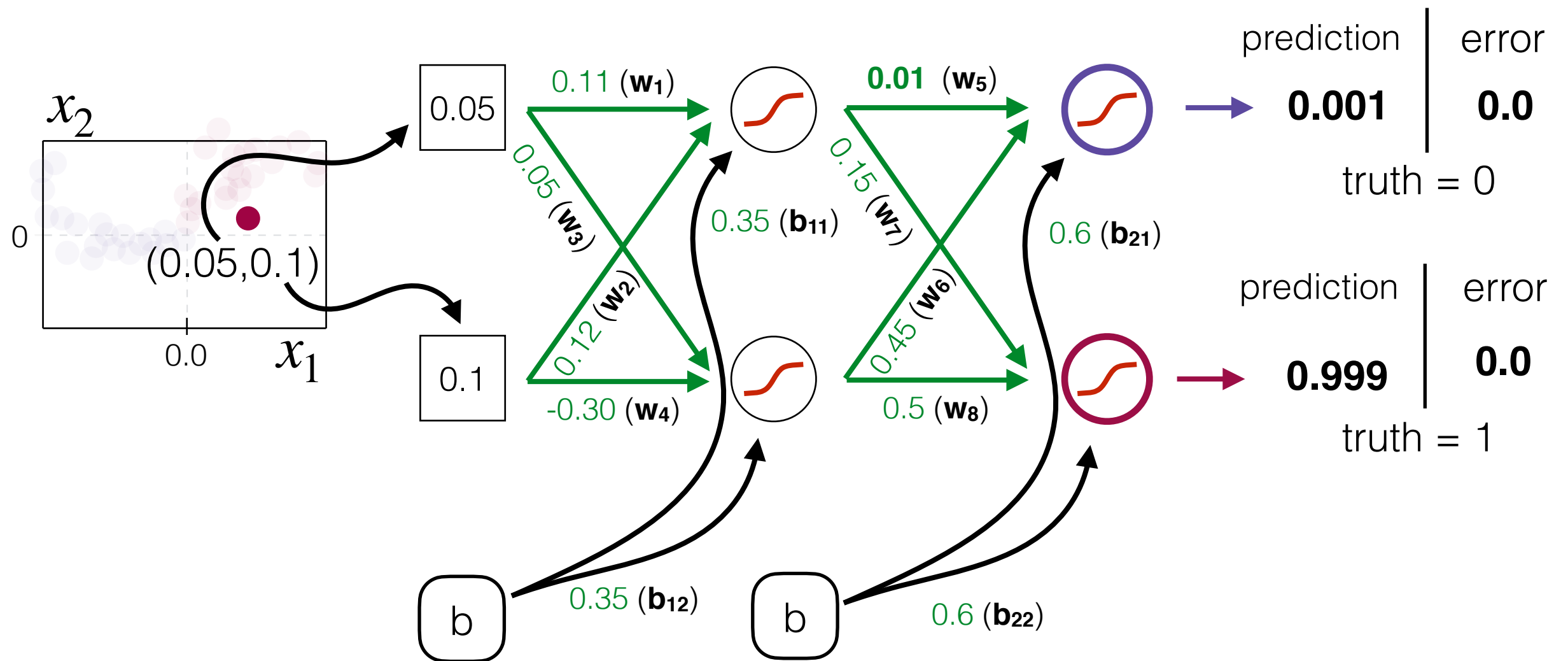


$$E_{total} = 0.28 + 0.025 = 0.303$$





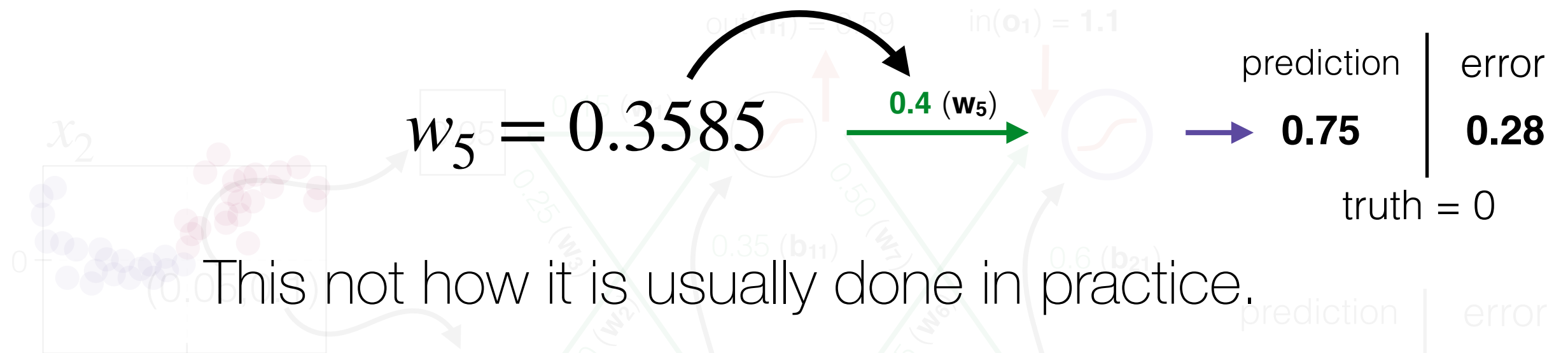
After updating all remaining weights total error = **0.131**



After updating all remaining weights total error = **0.131**

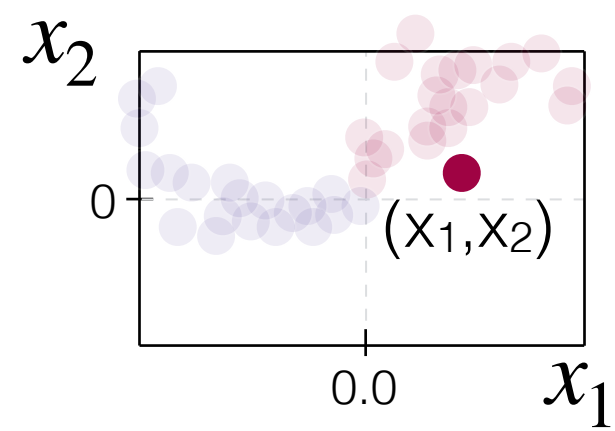
Repeating **1000** times decreases it to **0.001**

In this example we were dealing with only one weight (**w_5**)



This not how it is usually done in practice.
Usually updates for weights in the same layer are computed at the same time using matrix operations.

Let's see how backpropagation works
using **matrix** notation

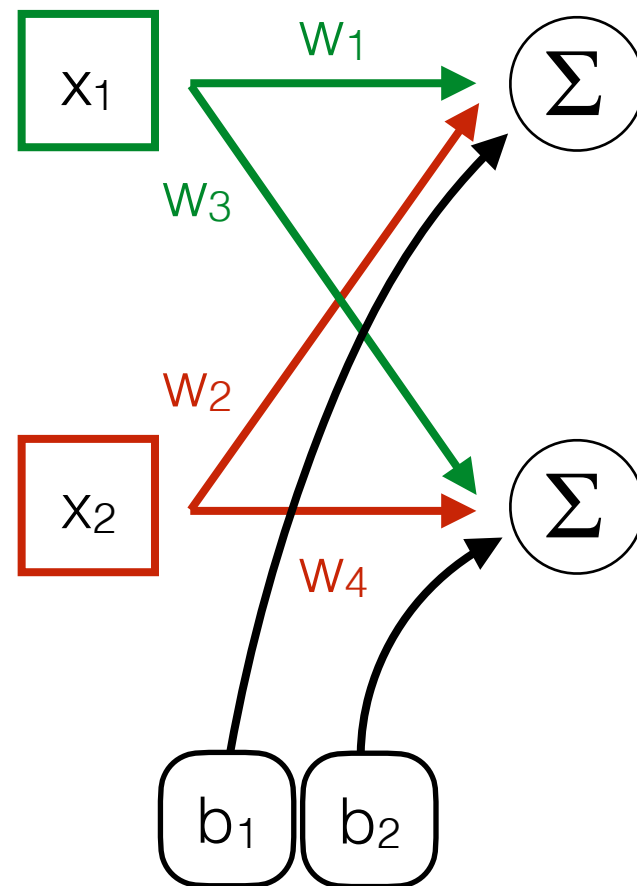
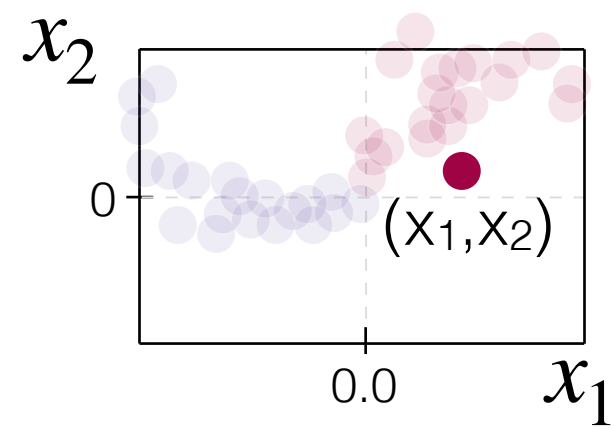


x_1

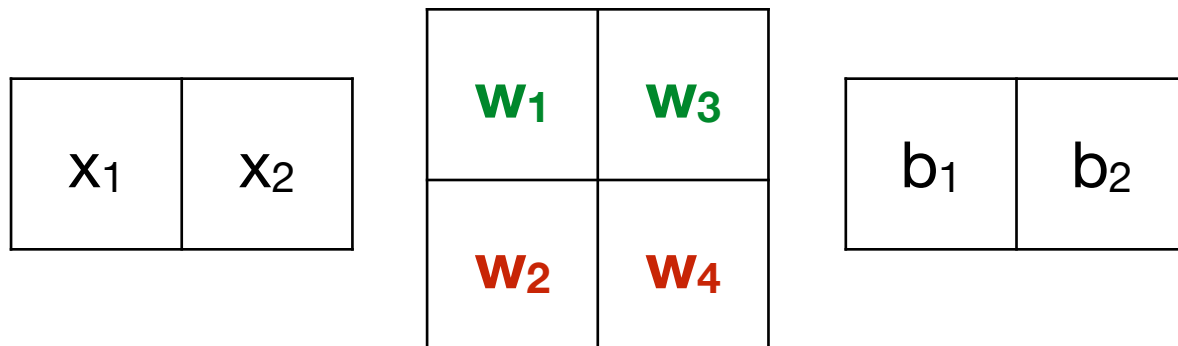
x_2

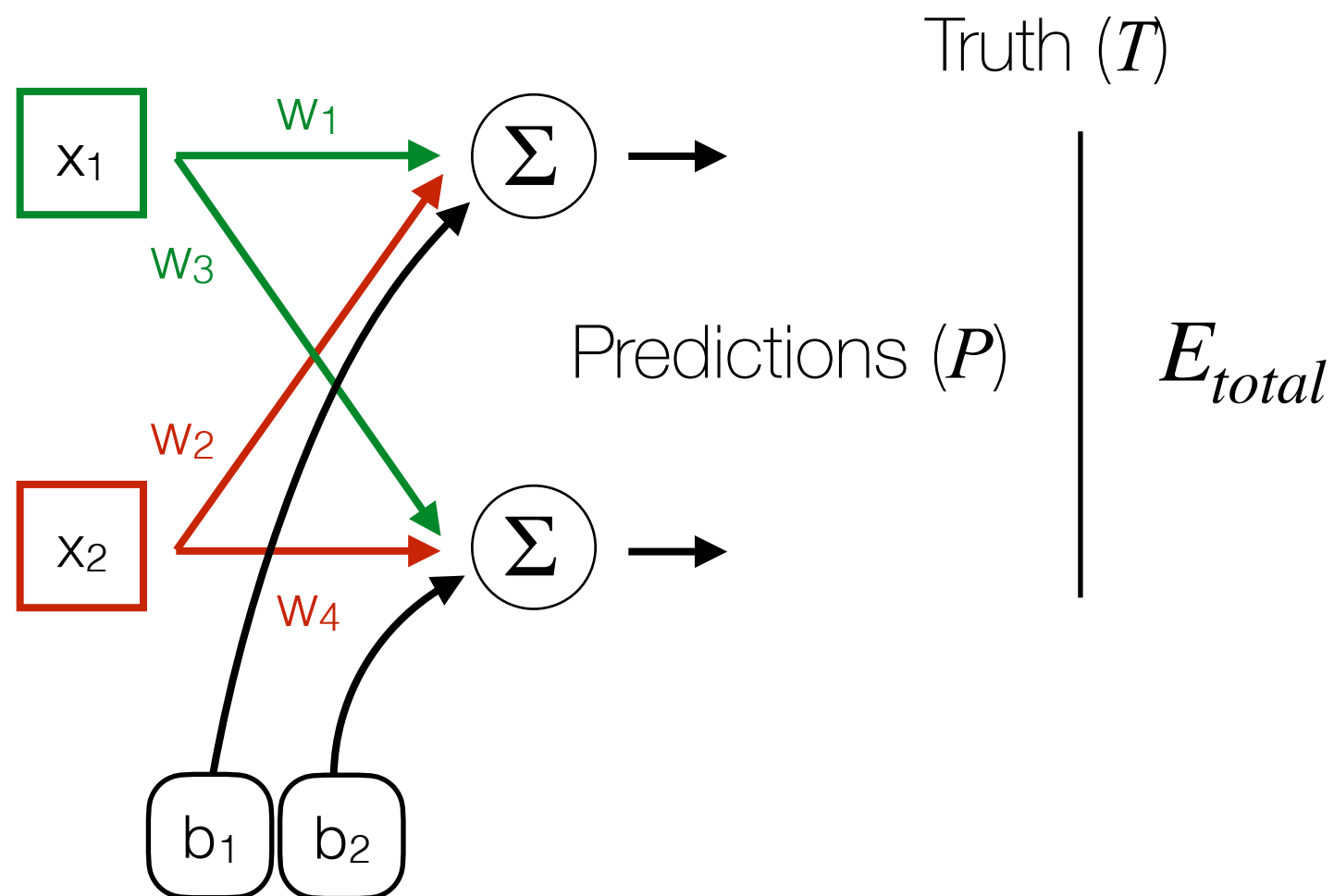
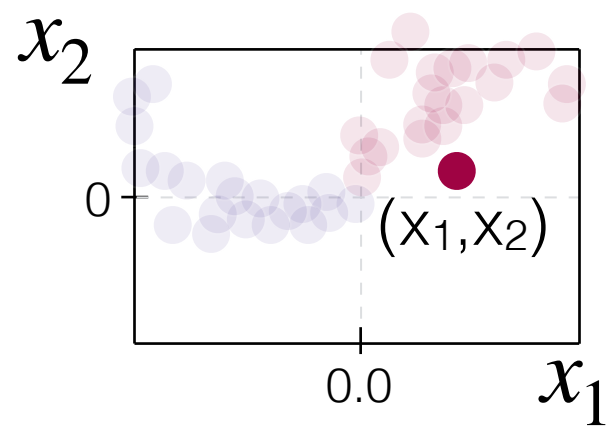
X

x_1	x_2
-------	-------



$$X * W + b$$

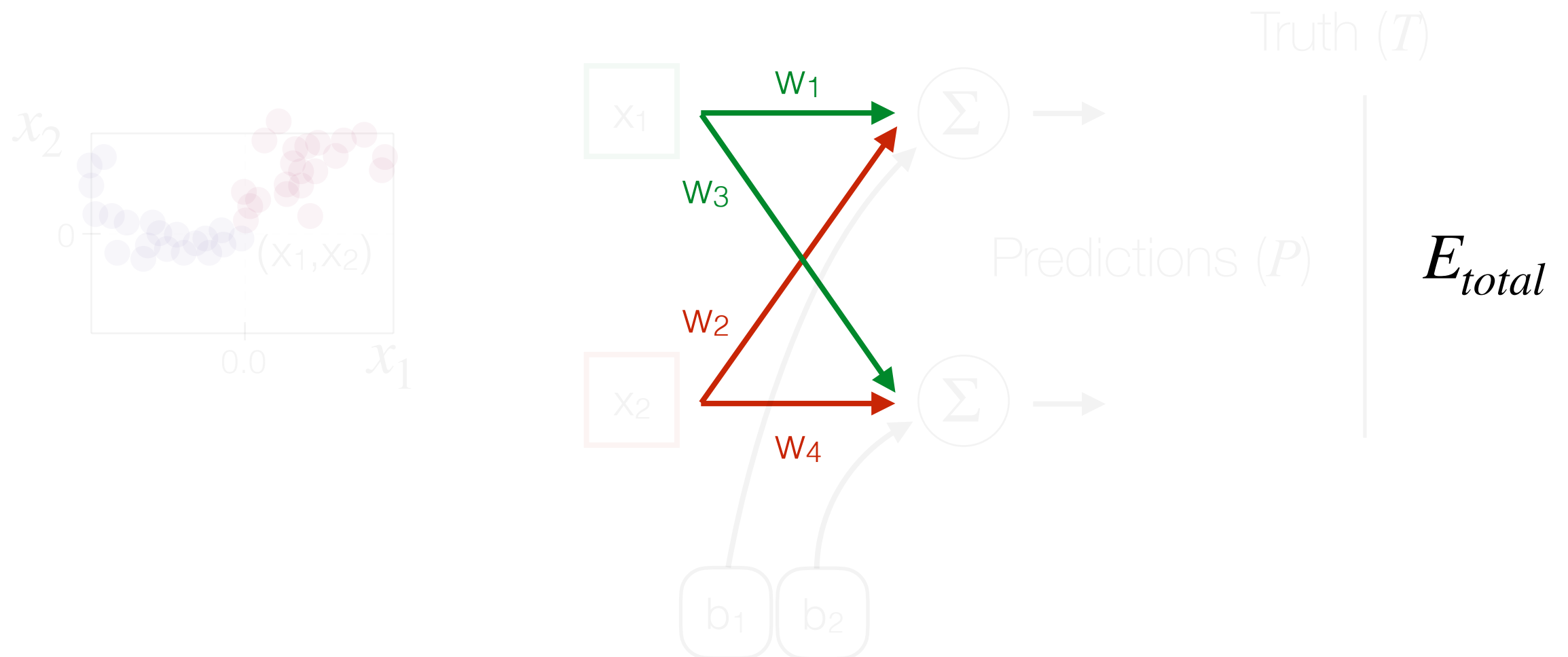




$$X * W + b = P \xrightarrow{T} E_{total}$$

x_1	x_2	w_1	w_3	b_1	b_2	P_1	P_2	E_T
		w_2	w_4					

How does W influence E_{total} ?

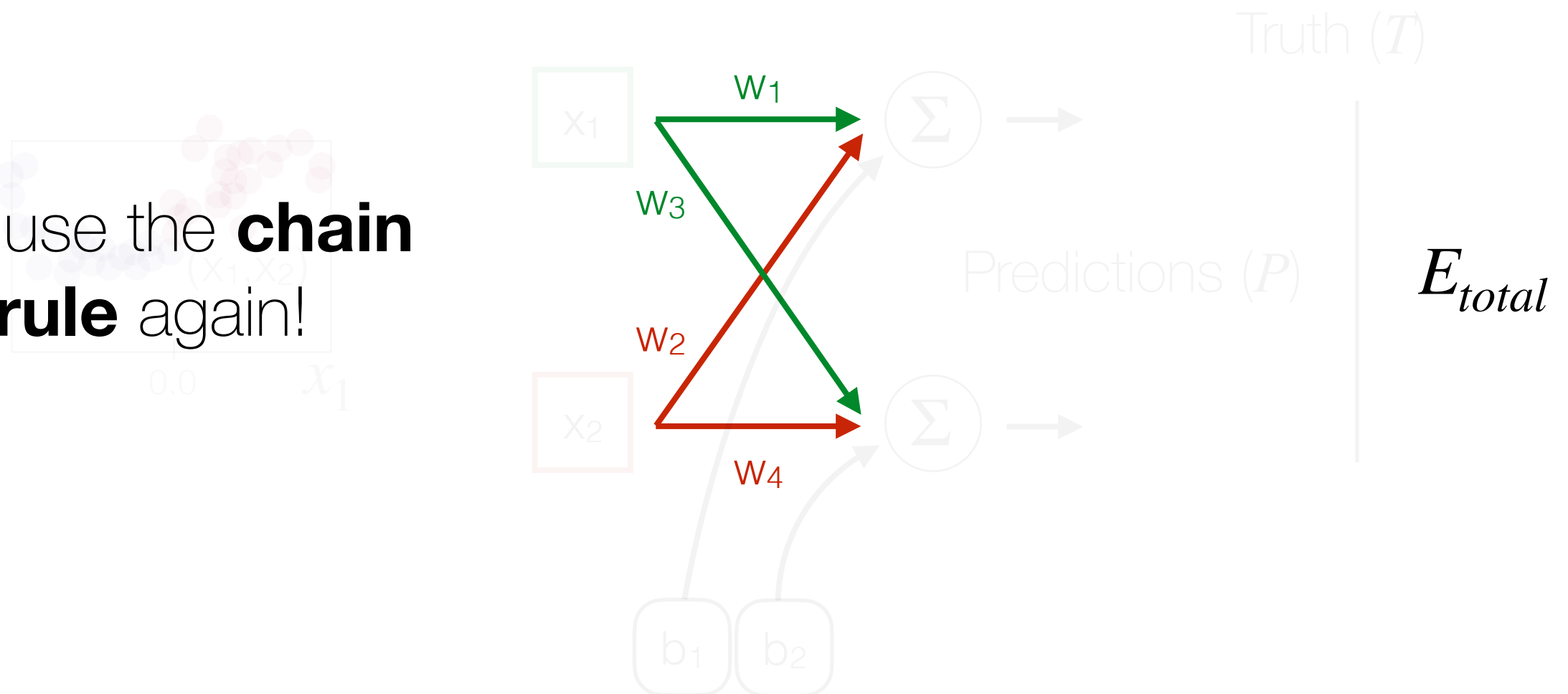


$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

We use the **chain rule** again!



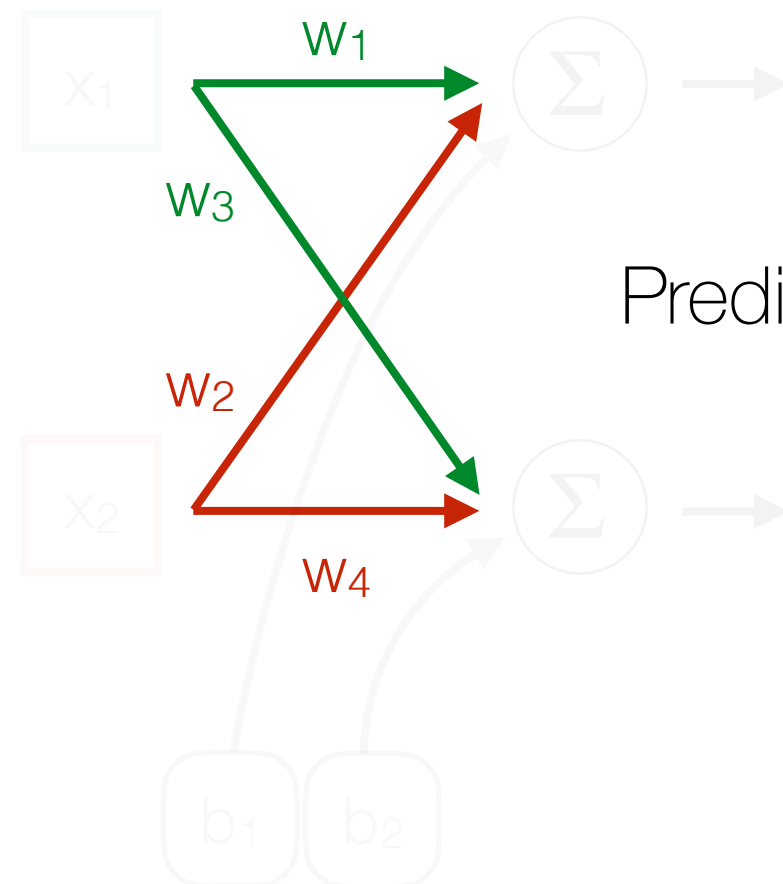
$$X * W + b = P \xrightarrow{T} E_{total}$$

x_1	x_2	w_1	w_3	b_1	b_2	P_1	P_2	E_T
		w_2	w_4					

How does P influence E_{total} ?

We use the **chain rule** again!

Truth (T)



Predictions (P)

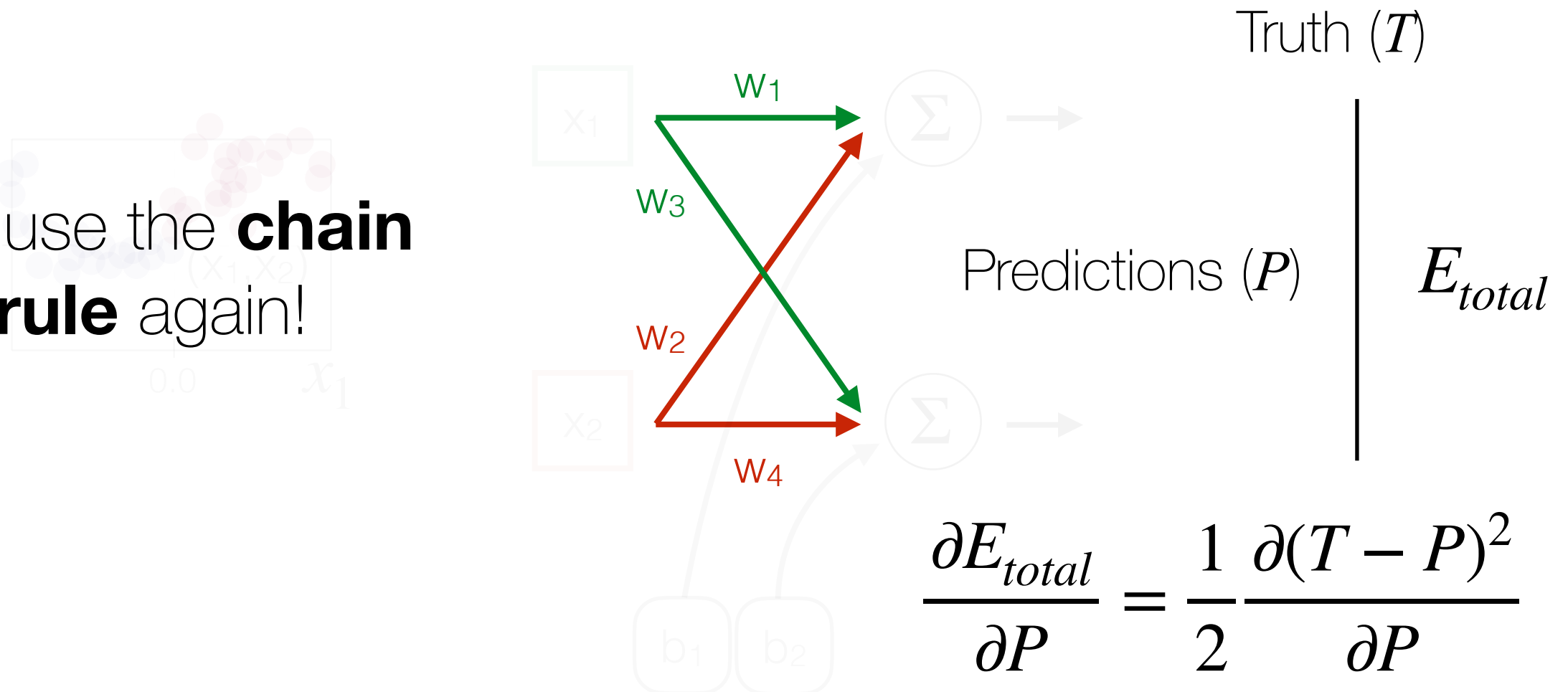
E_{total}

$$X * W + b = P \xrightarrow{T} E_{total}$$

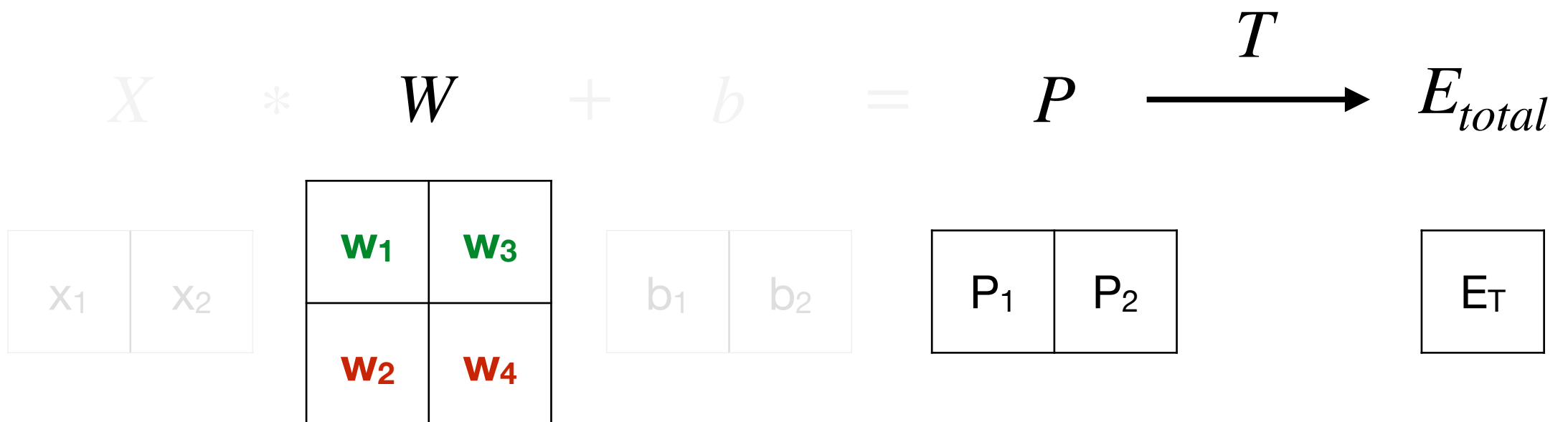


How does P influence E_{total} ?

We use the **chain rule** again!



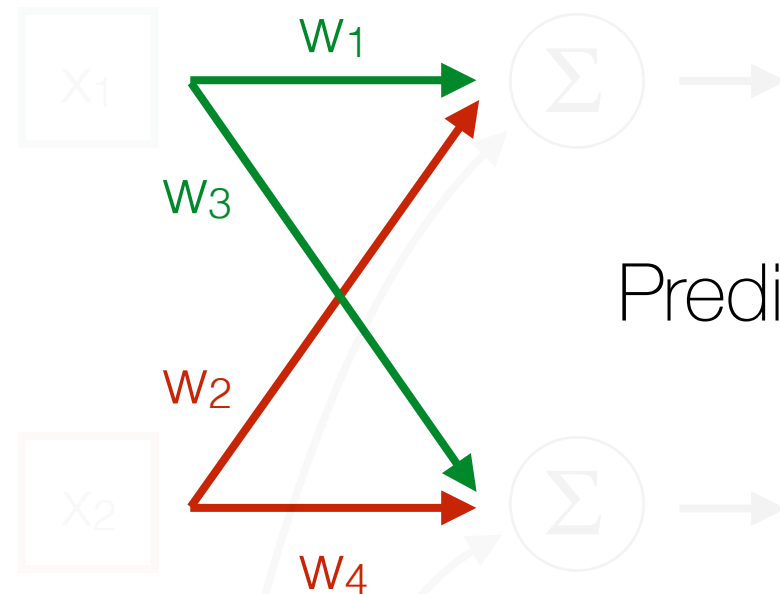
$$\frac{\partial E_{total}}{\partial P} = \frac{1}{2} \frac{\partial (T - P)^2}{\partial P}$$



How does P influence E_{total} ?

We use the **chain rule** again!

Truth (T)



Predictions (P)

E_{total}

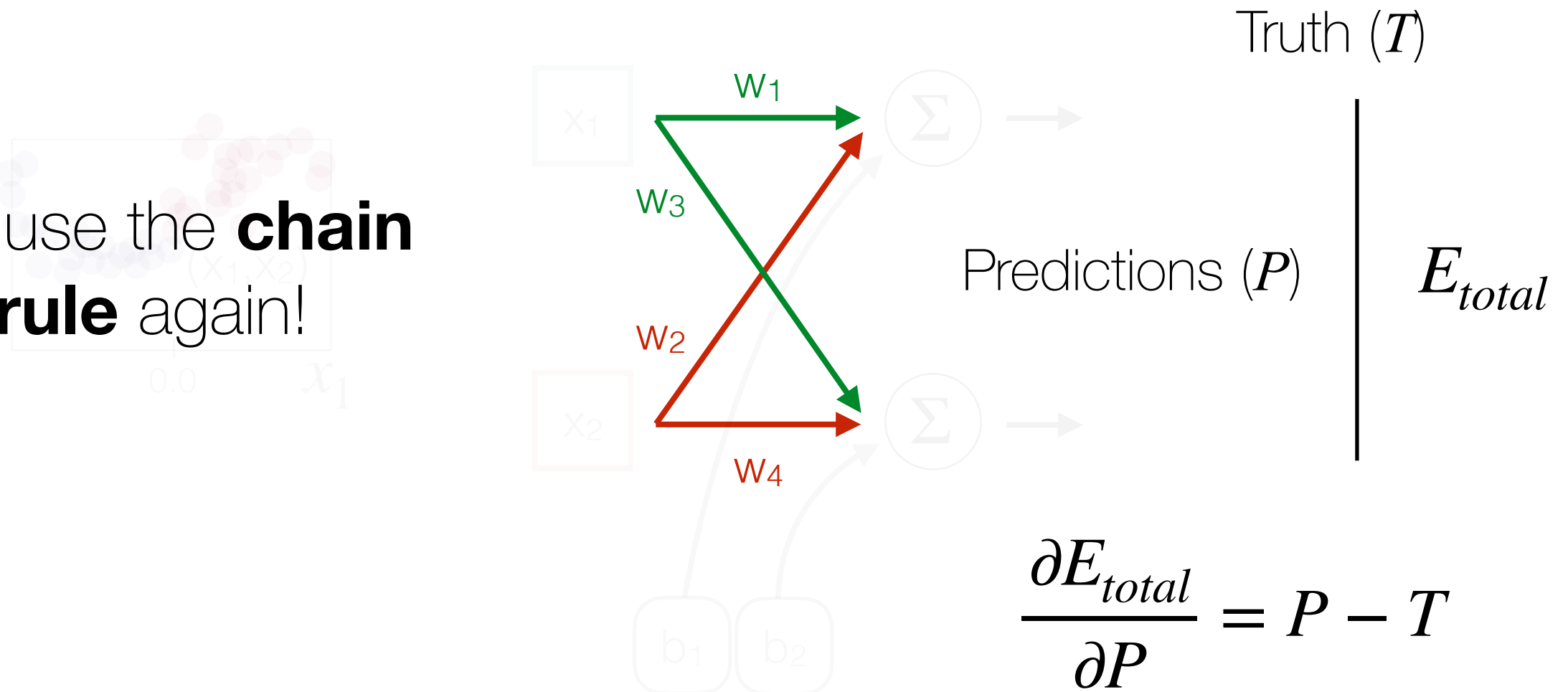
$$\frac{\partial E_{total}}{\partial P} = \frac{1}{2} * 2 * (-1) * (T - P)$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does P influence E_{total} ?

We use the **chain rule** again!



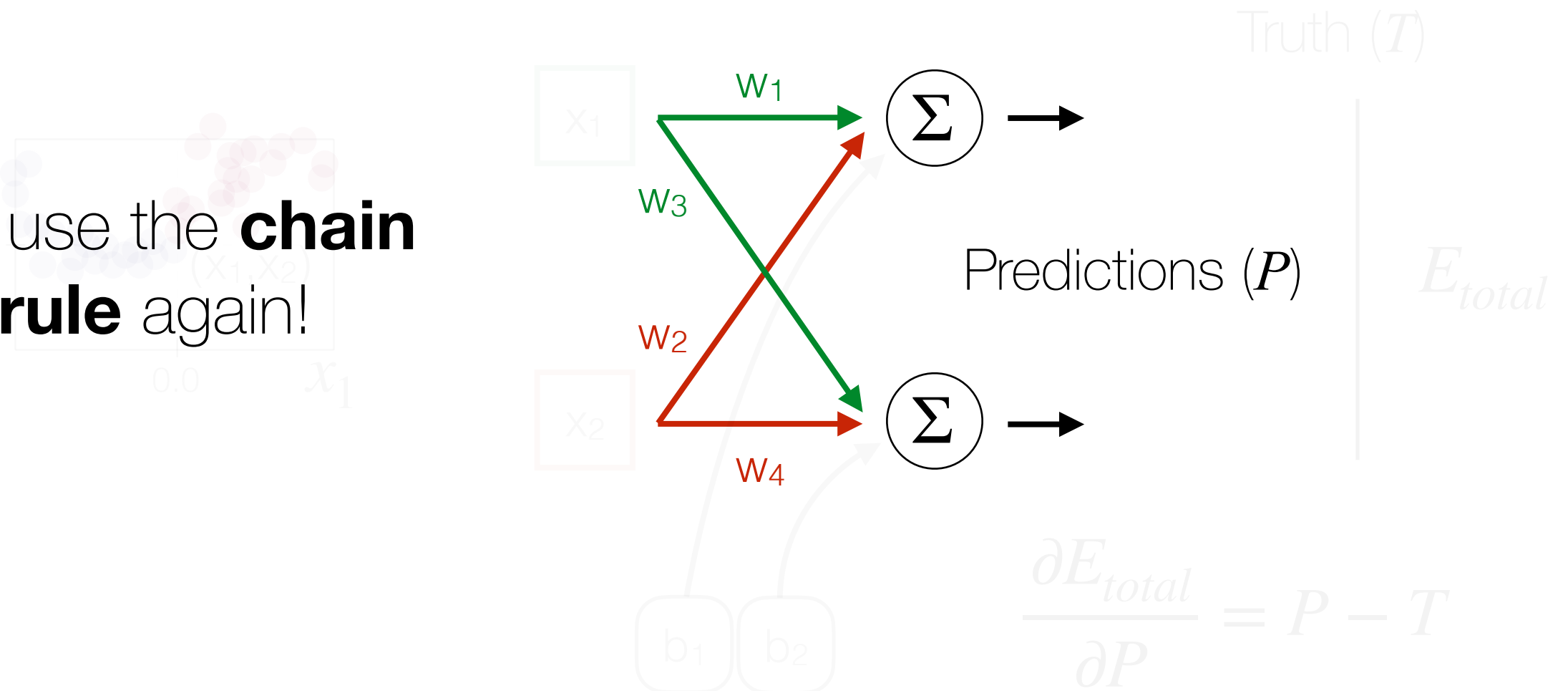
$$\frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence P ?

We use the **chain rule** again!



$$X * W + b = P \xrightarrow{T} E_{total}$$

x_1	x_2
-------	-------

w_1	w_3
w_2	w_4

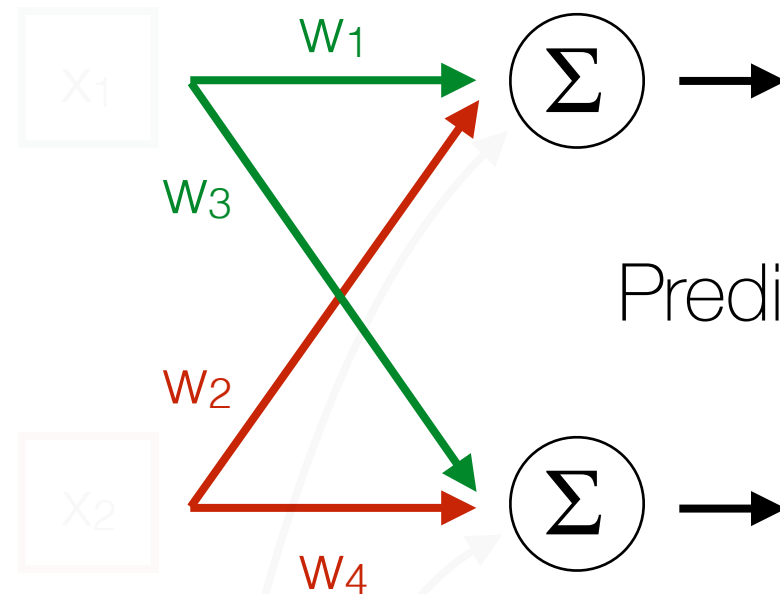
b_1	b_2
-------	-------

P_1	P_2
-------	-------

E_T

How does W influence P ?

We use the **chain rule** again!

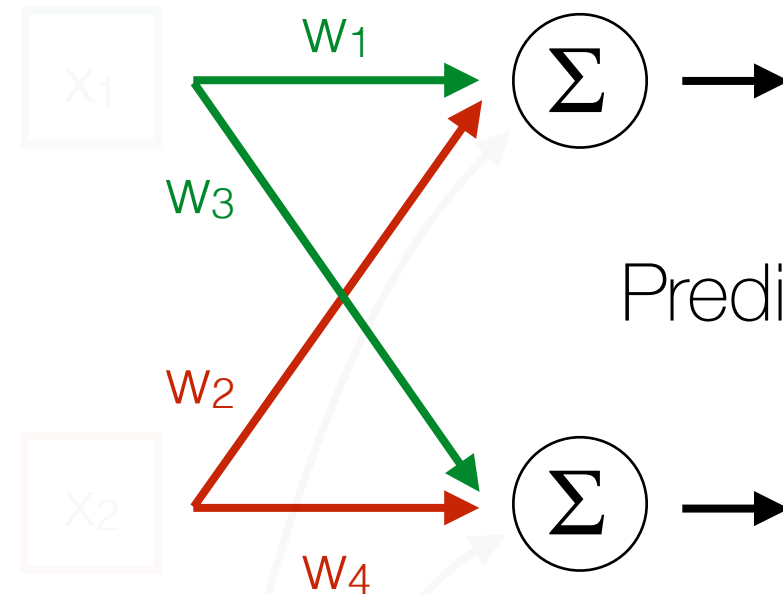


$$\frac{\partial P}{\partial W} = \frac{\partial (X * W + b)}{\partial W} \frac{\partial E_{total}}{\partial P} = P - T$$



How does W influence P ?

We use the **chain rule** again!



$$\frac{\partial P}{\partial W} = X^T$$

$$\frac{\partial E_{total}}{\partial P} = P - T$$

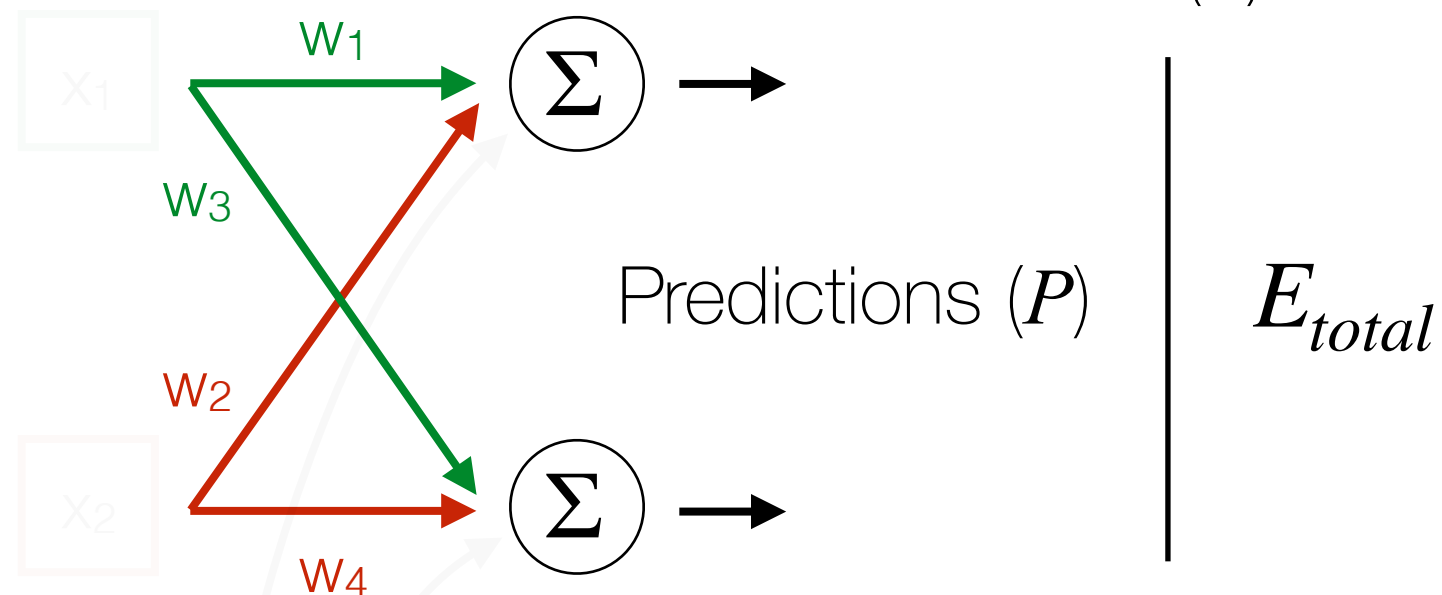
$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

We use the **chain rule** again!

Truth (T)



$$\frac{\partial E_{total}}{\partial W} = ?$$

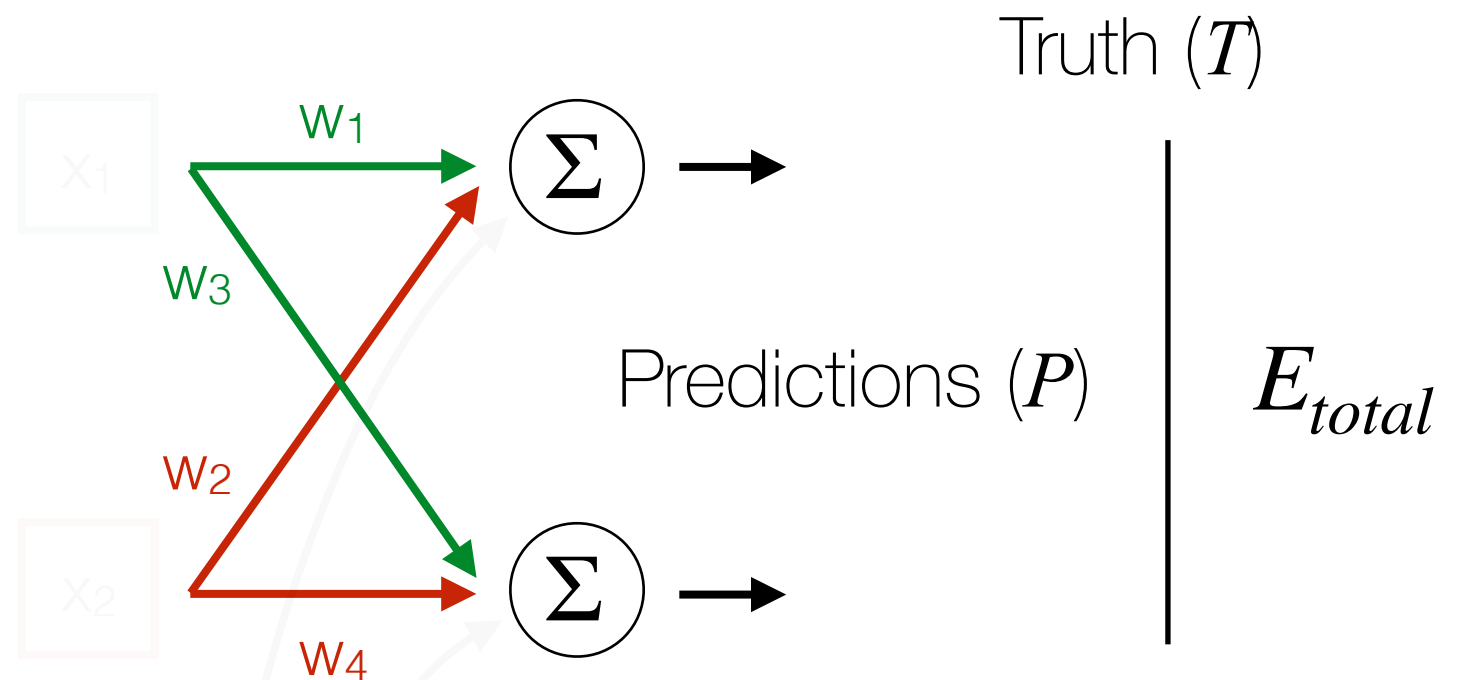
$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

We use the **chain rule** again!



$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

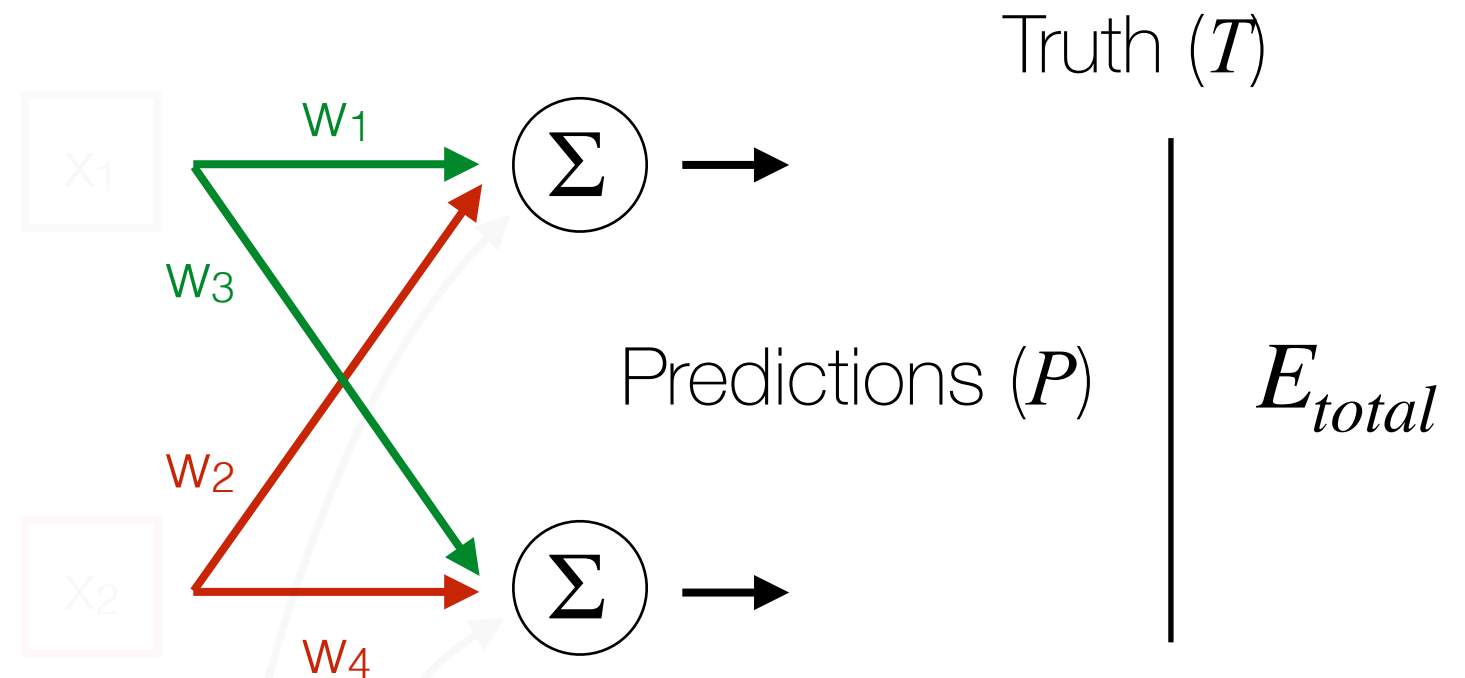
$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

What is the **problem** here?



$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

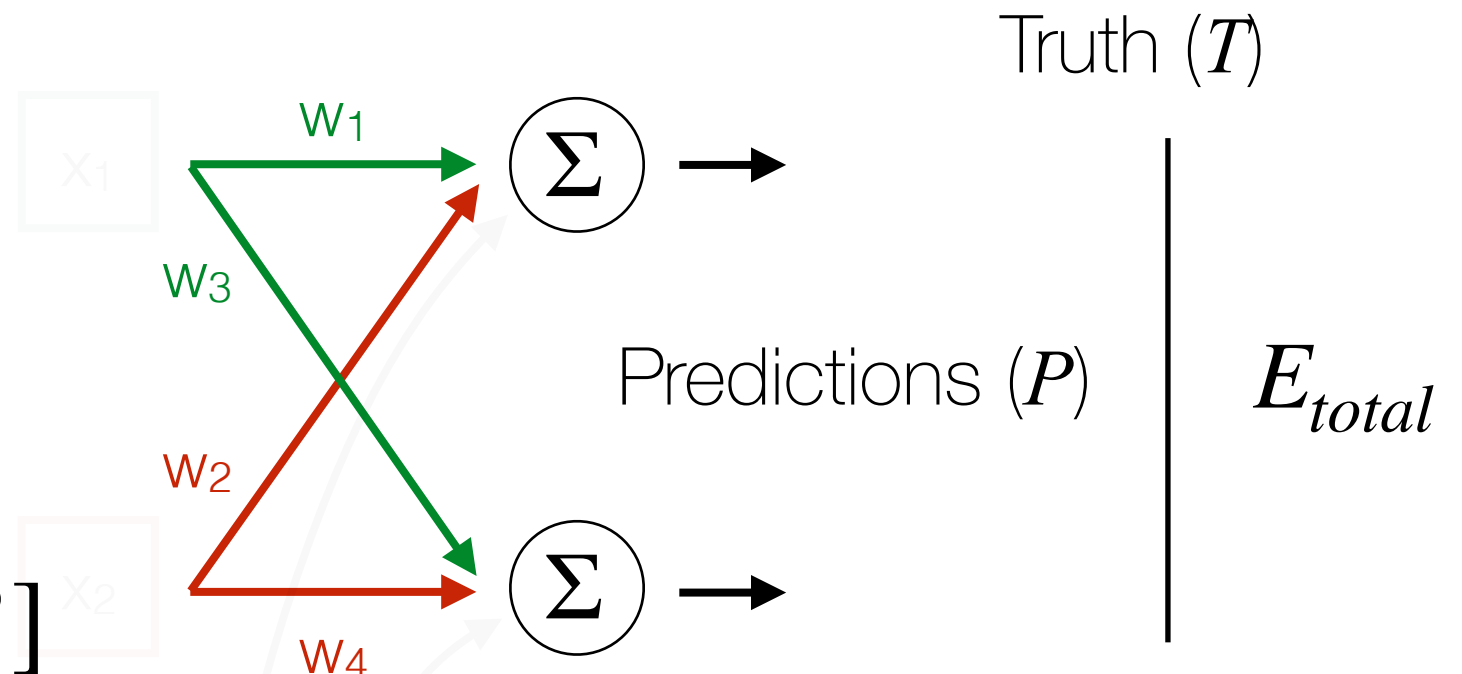
What is the **problem** here?

$$[?; ?] = [?; ?] * [?; ?]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

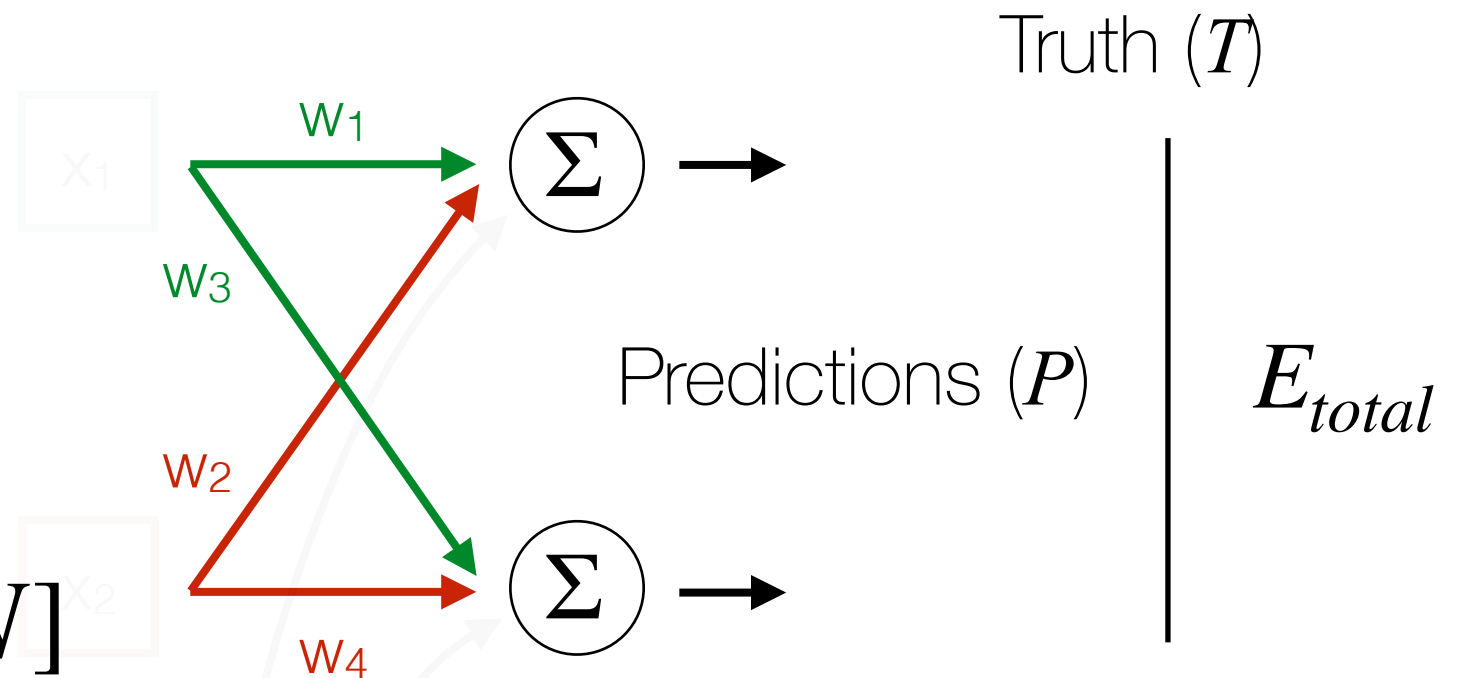
What is the **problem** here?

$$[?; ?] = [?; ?] * [2; N]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

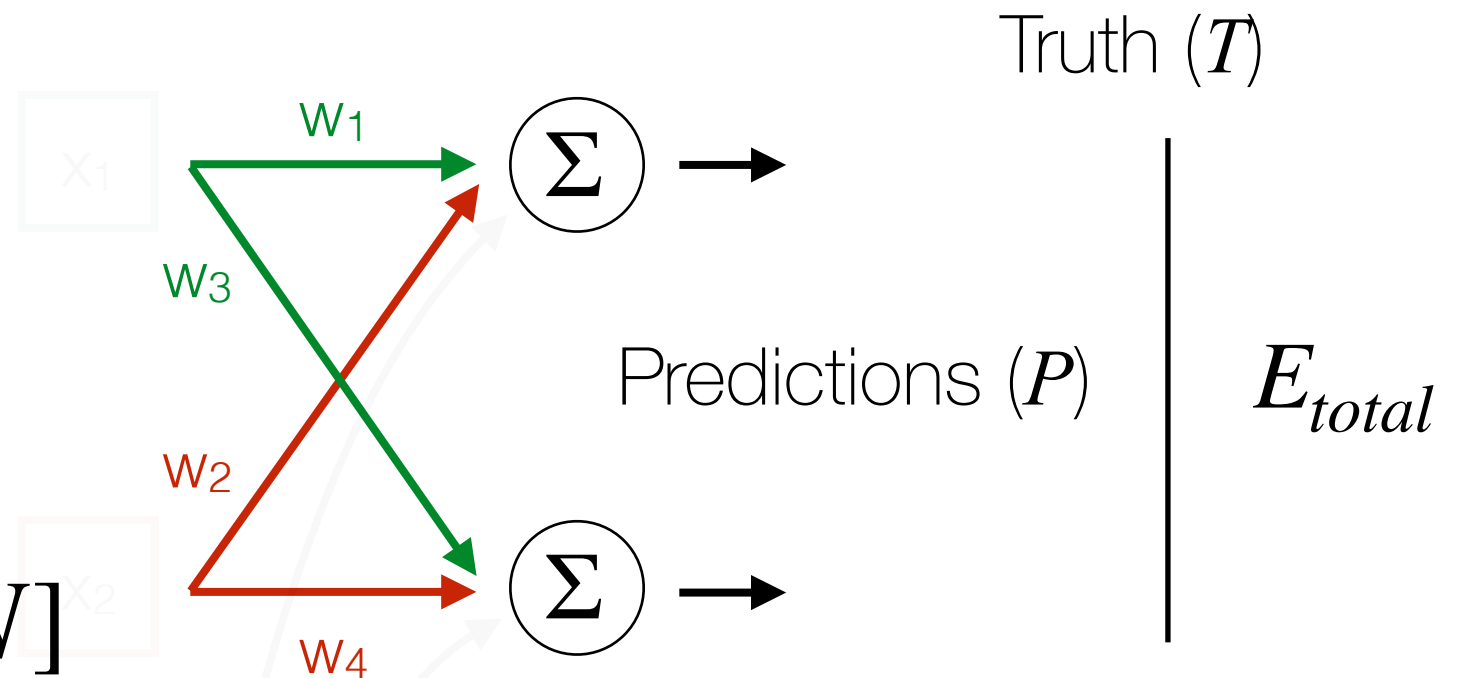
What is the **problem** here?

$$[?; ?] = [?; ?] * [2; N]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

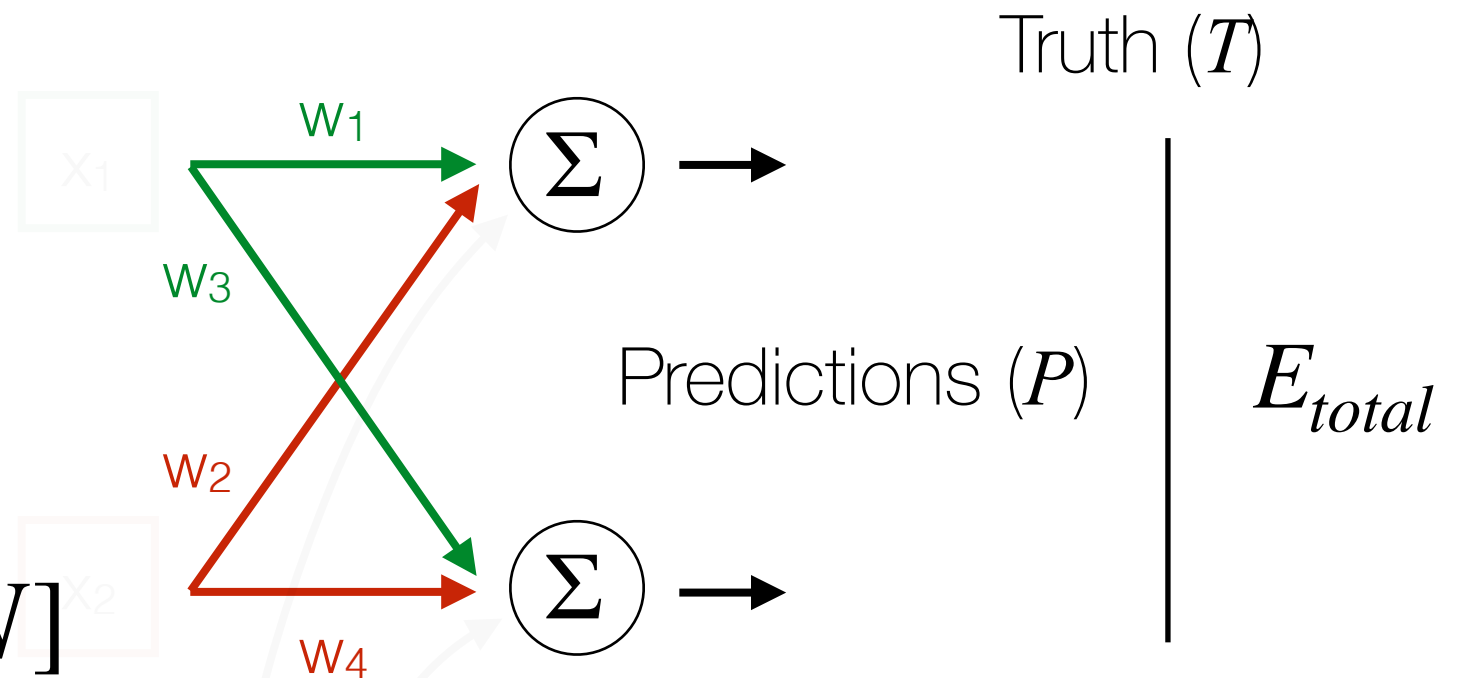
What is the **problem** here?

$$[?; ?] = [?; 2] * [2; N]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

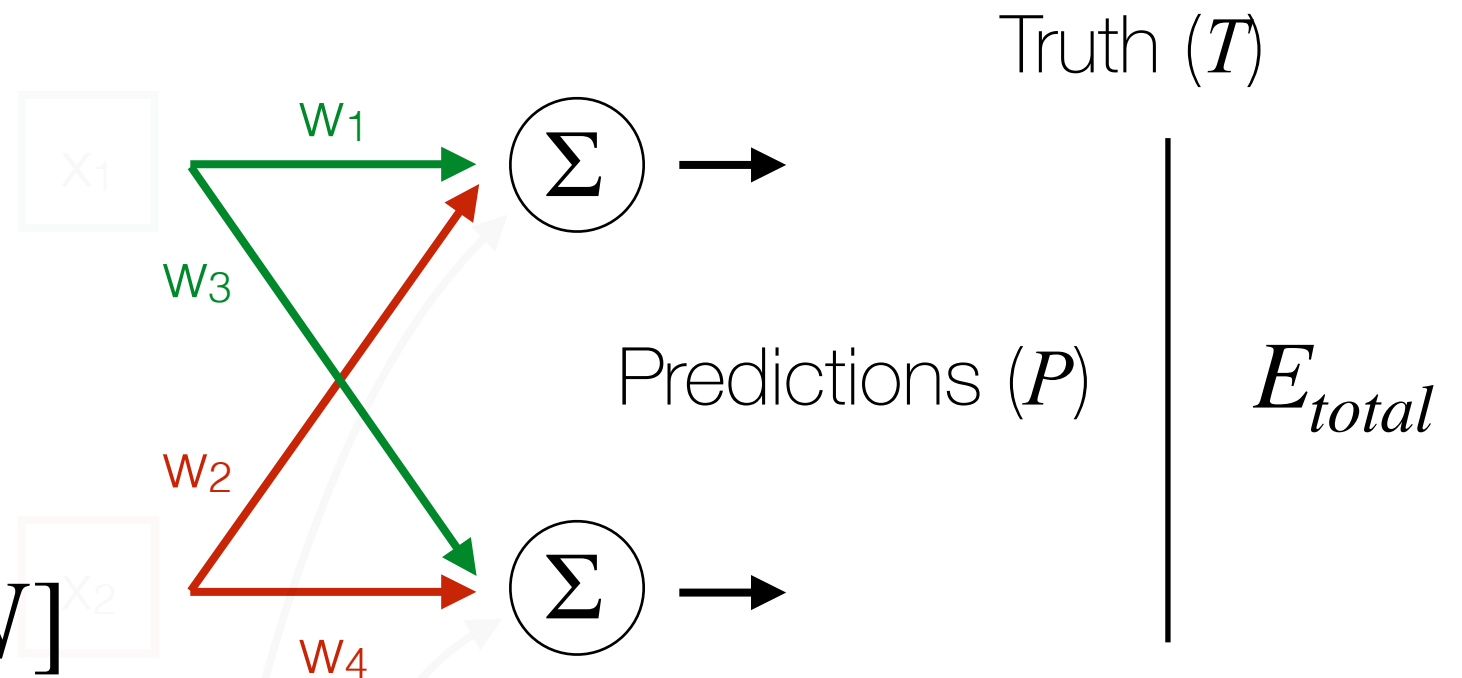
What is the **problem** here?

$$[?; ?] = [N; 2] * [2; N]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence E_{total} ?

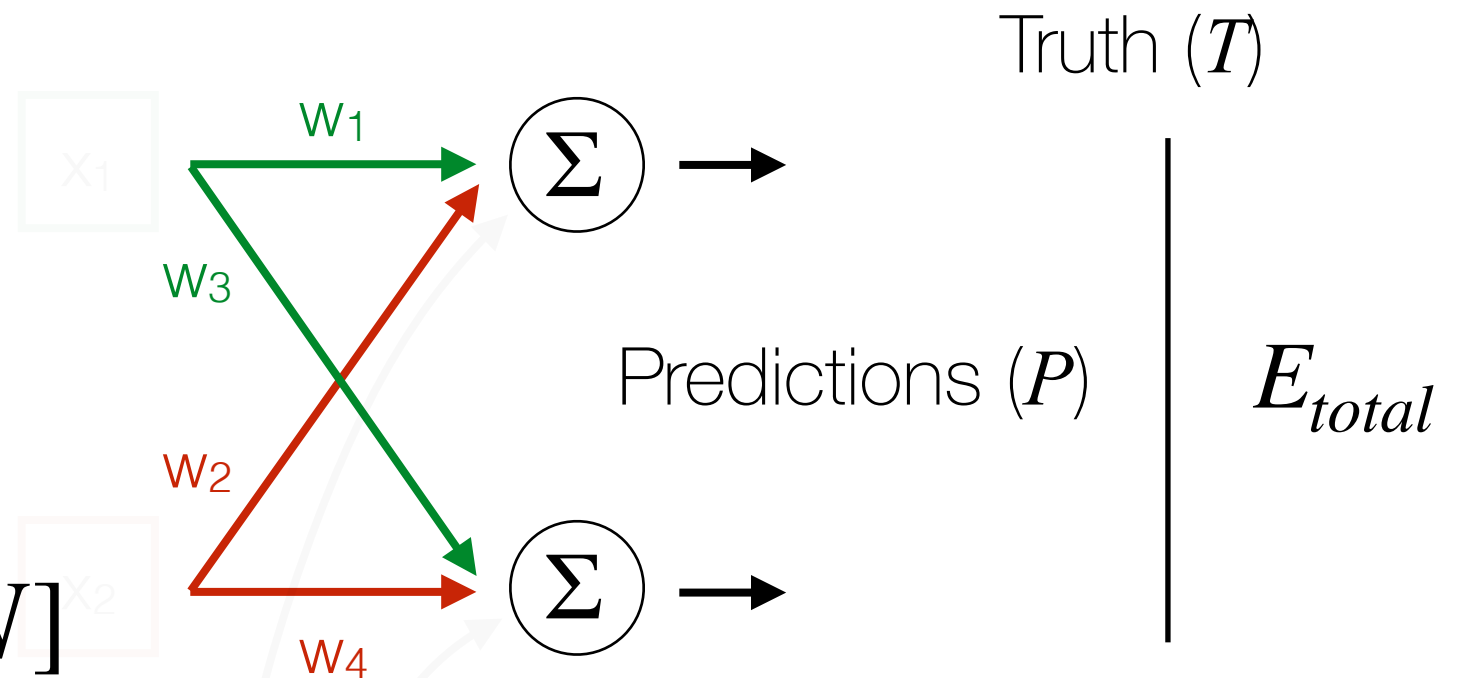
What is the **problem** here?

$$[2; 2] = [N; 2] * [2; N]$$

$$\frac{\partial E_{total}}{\partial W} = \frac{\partial E_{total}}{\partial P} * \frac{\partial P}{\partial W}$$

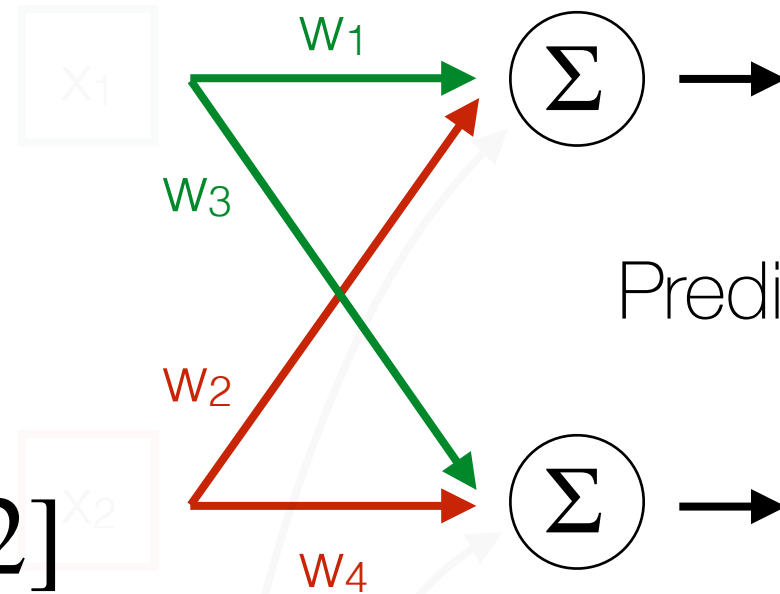
$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



We finally know how W influence E_{total} !

Truth (T)



Predictions (P)

E_{total}

$$[2; 2] = [2; N] * [N; 2]$$

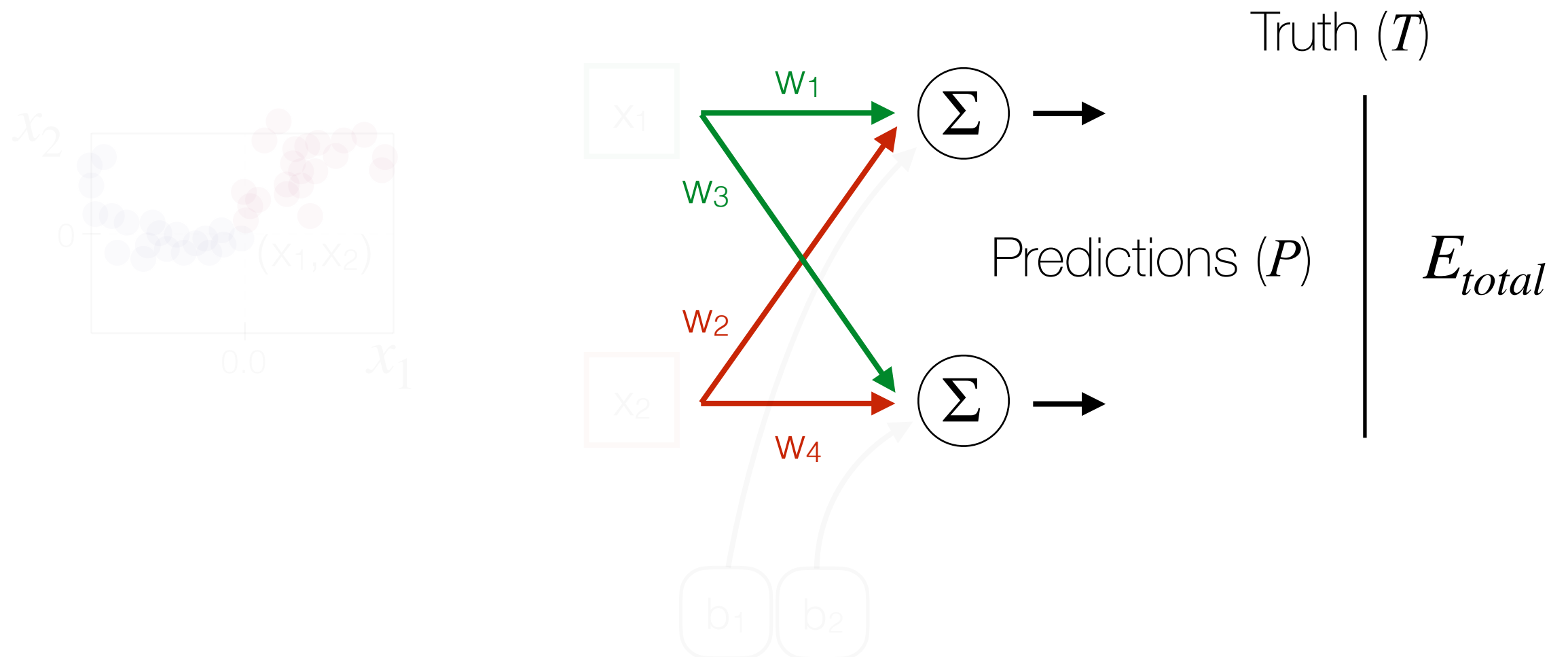
$$\frac{\partial E_{total}}{\partial W} = \frac{\partial P}{\partial W} * \frac{\partial E_{total}}{\partial P}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



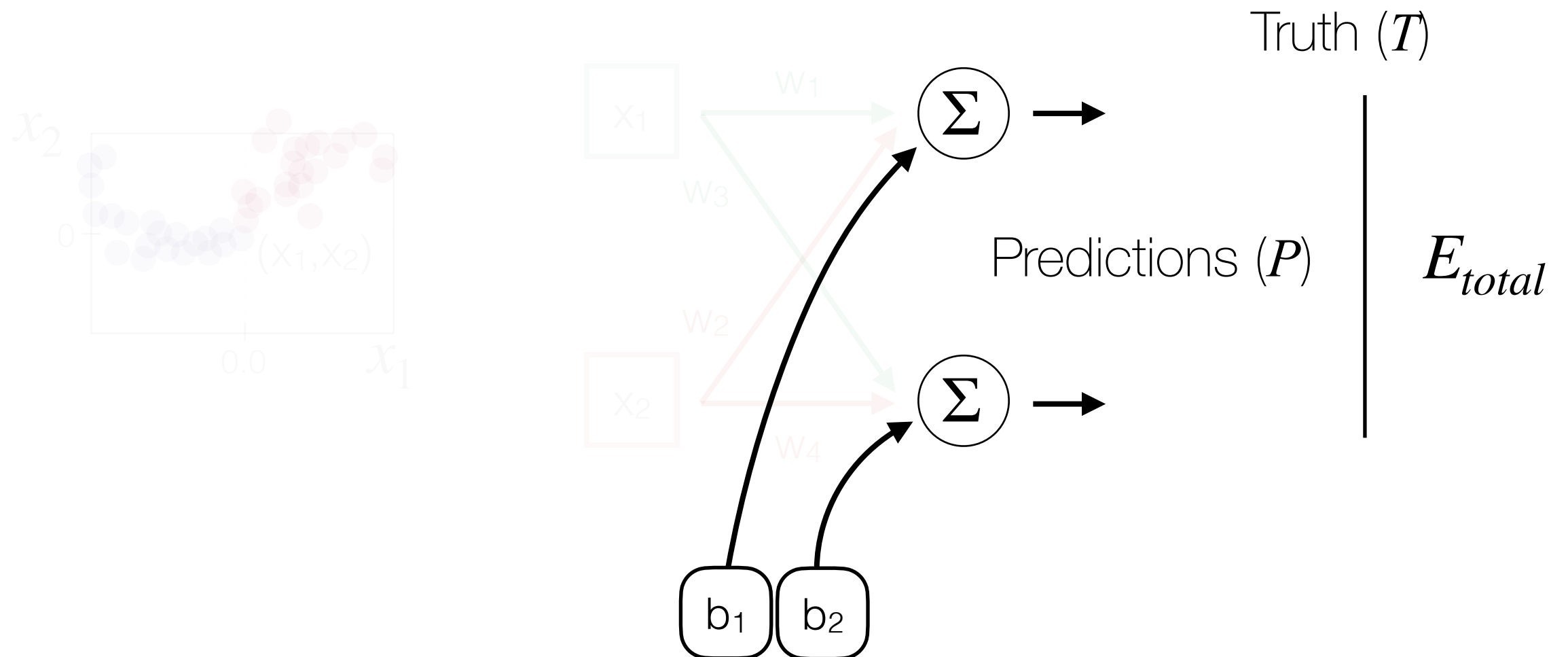
What else is there?



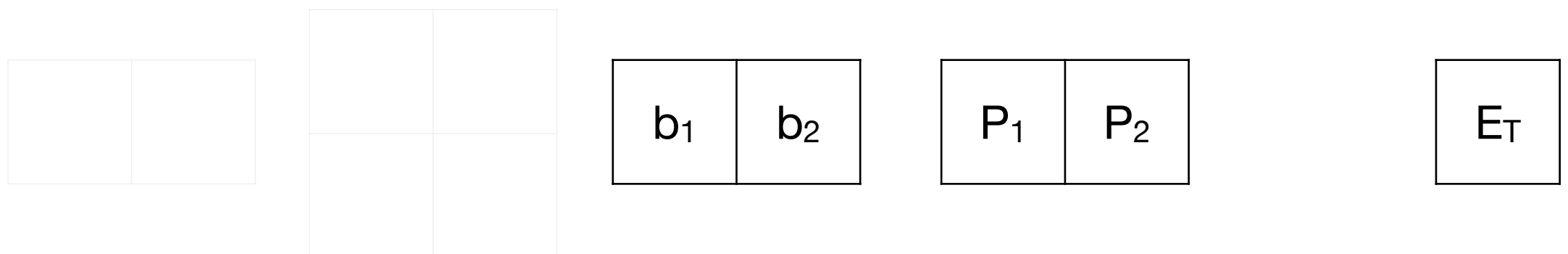
$$X * W + b = P \xrightarrow{T} E_{total}$$



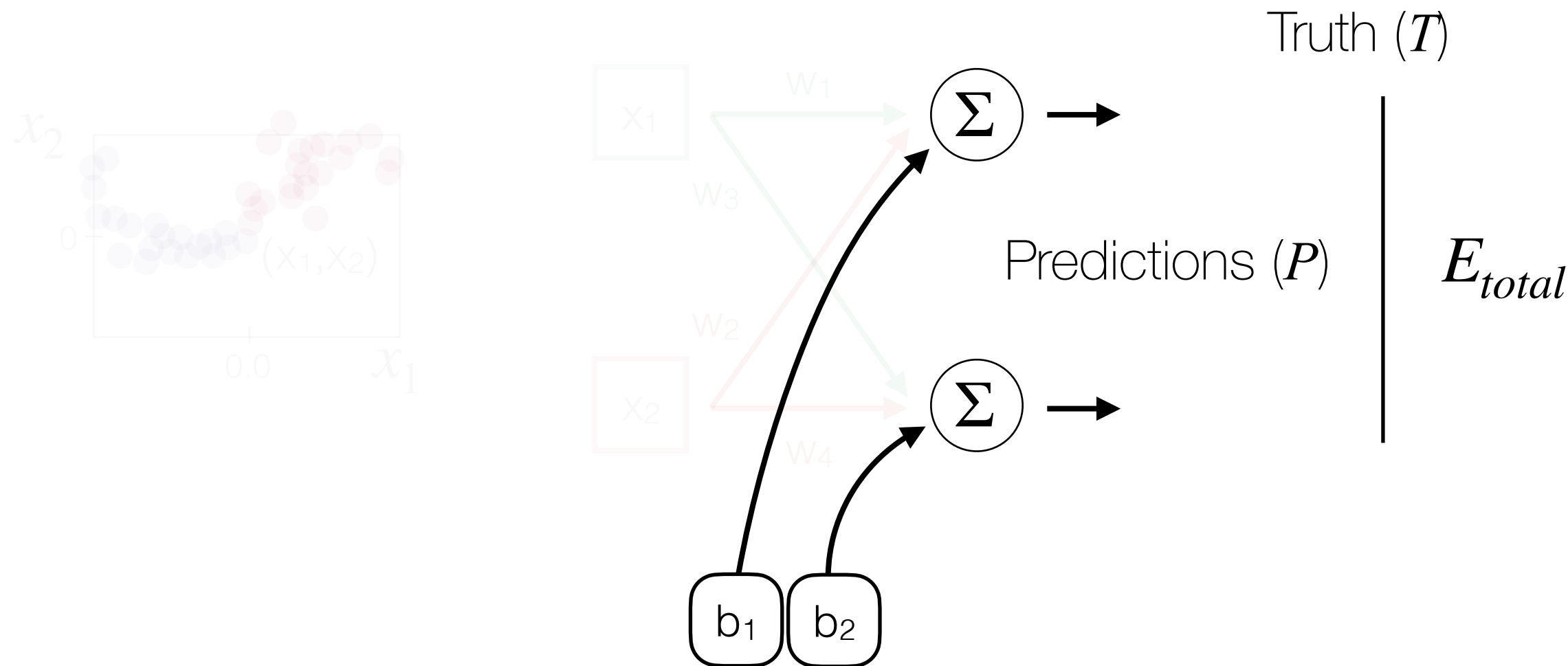
What else is there? **Biases!**



$$X * W + b = P \xrightarrow{T} E_{total}$$



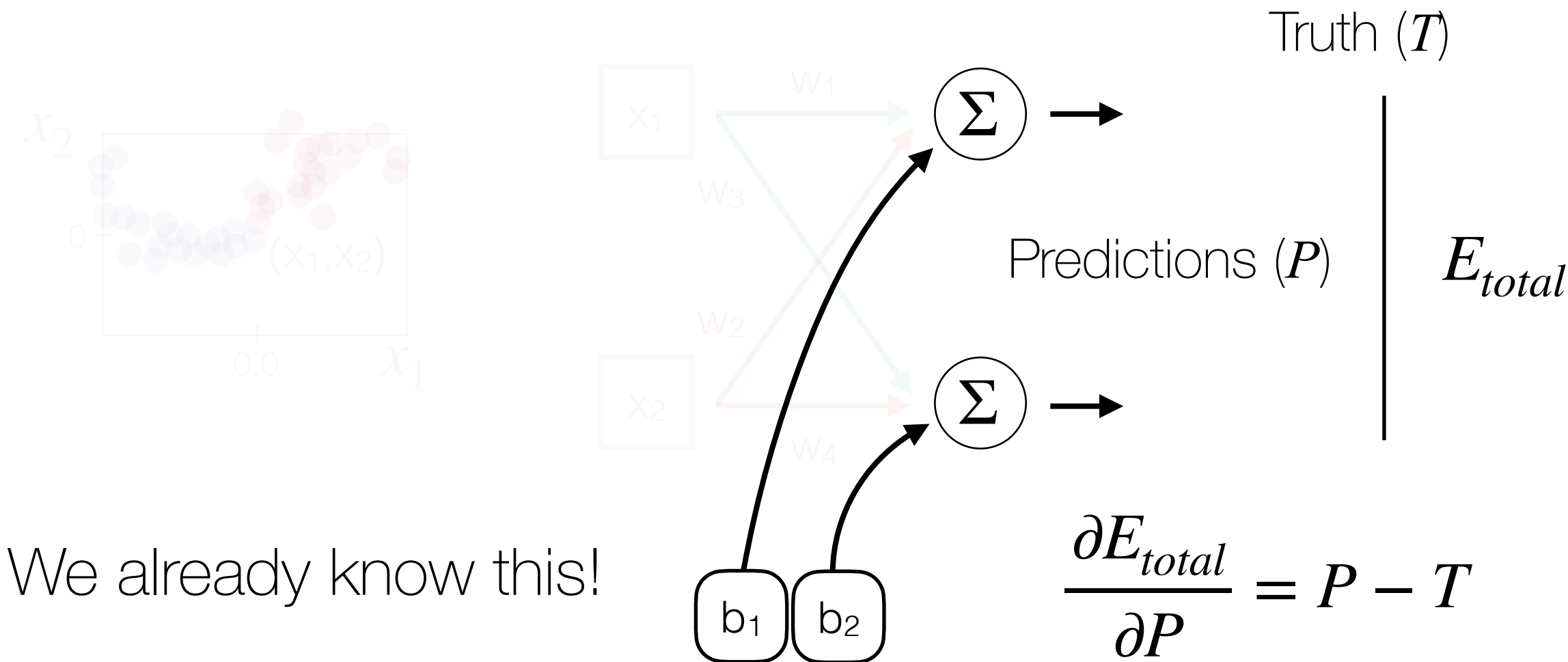
In order to estimate how much $\mathbf{b}$ influence E_{total} we again need to know how much E_{total} is influenced by $\mathbf{P}$



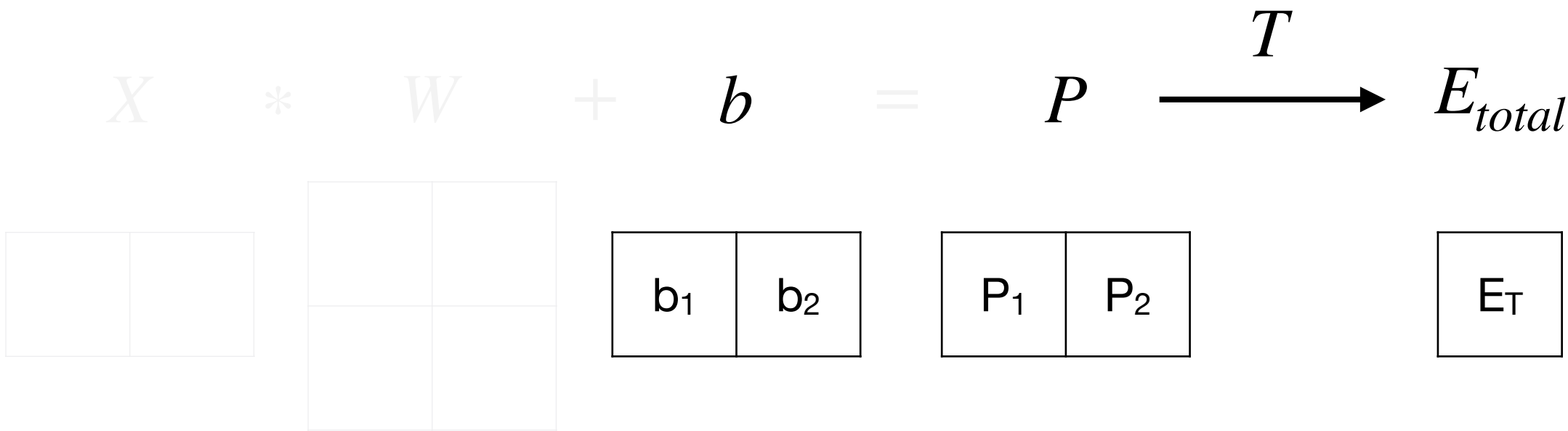
$$X * W + b = P \xrightarrow{T} E_{total}$$

		b_1	b_2	P_1	P_2	E_T
--	--	-------	-------	-------	-------	-------

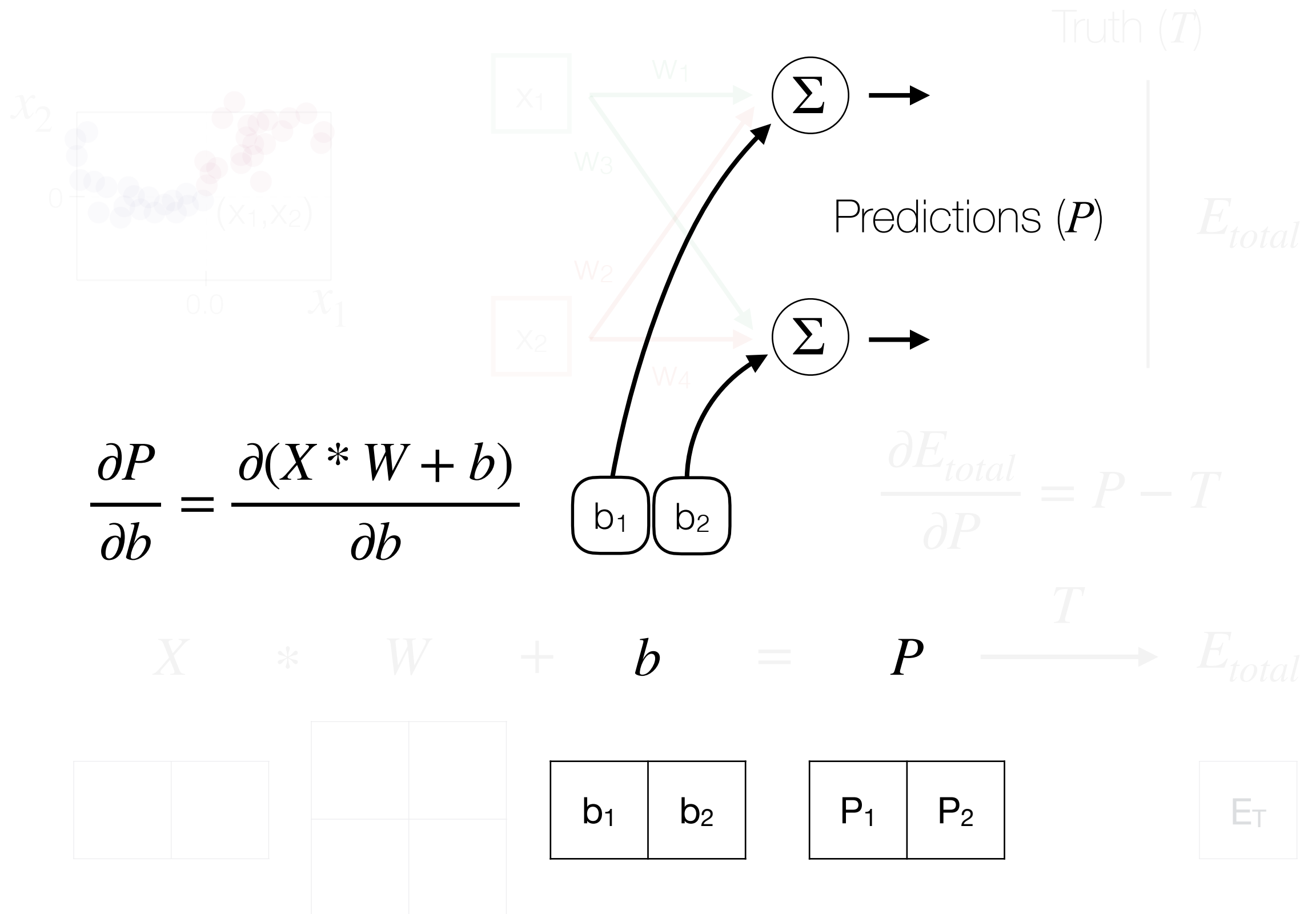
In order to estimate how much $\mathbf{b}$ influence E_{total} we again need to know how much E_{total} is influenced by $\mathbf{P}$



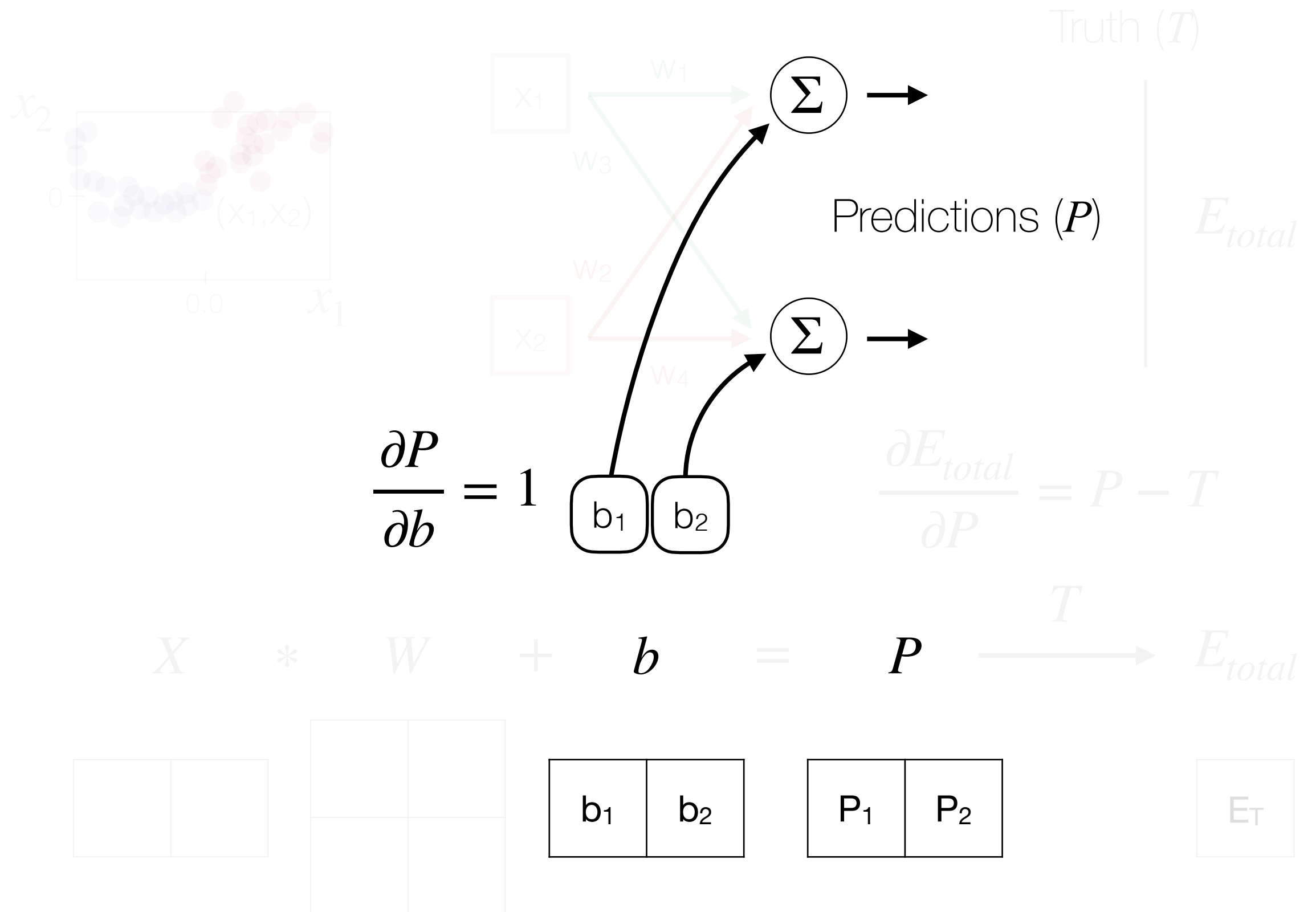
We already know this!



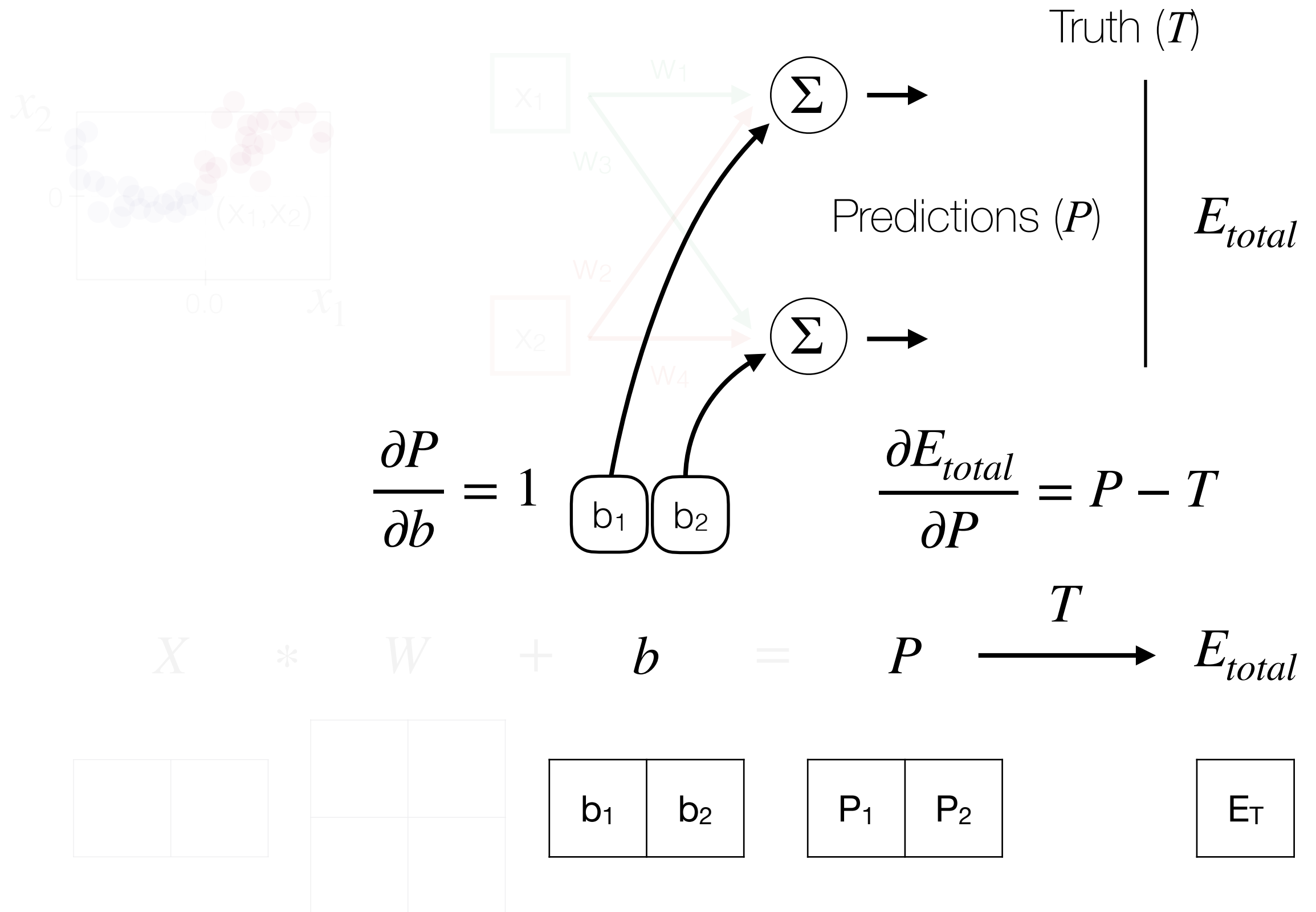
How does b influence P ?



How does b influence P ?



Estimate how much b influence E_{total}



Estimate how much b influence E_{total}

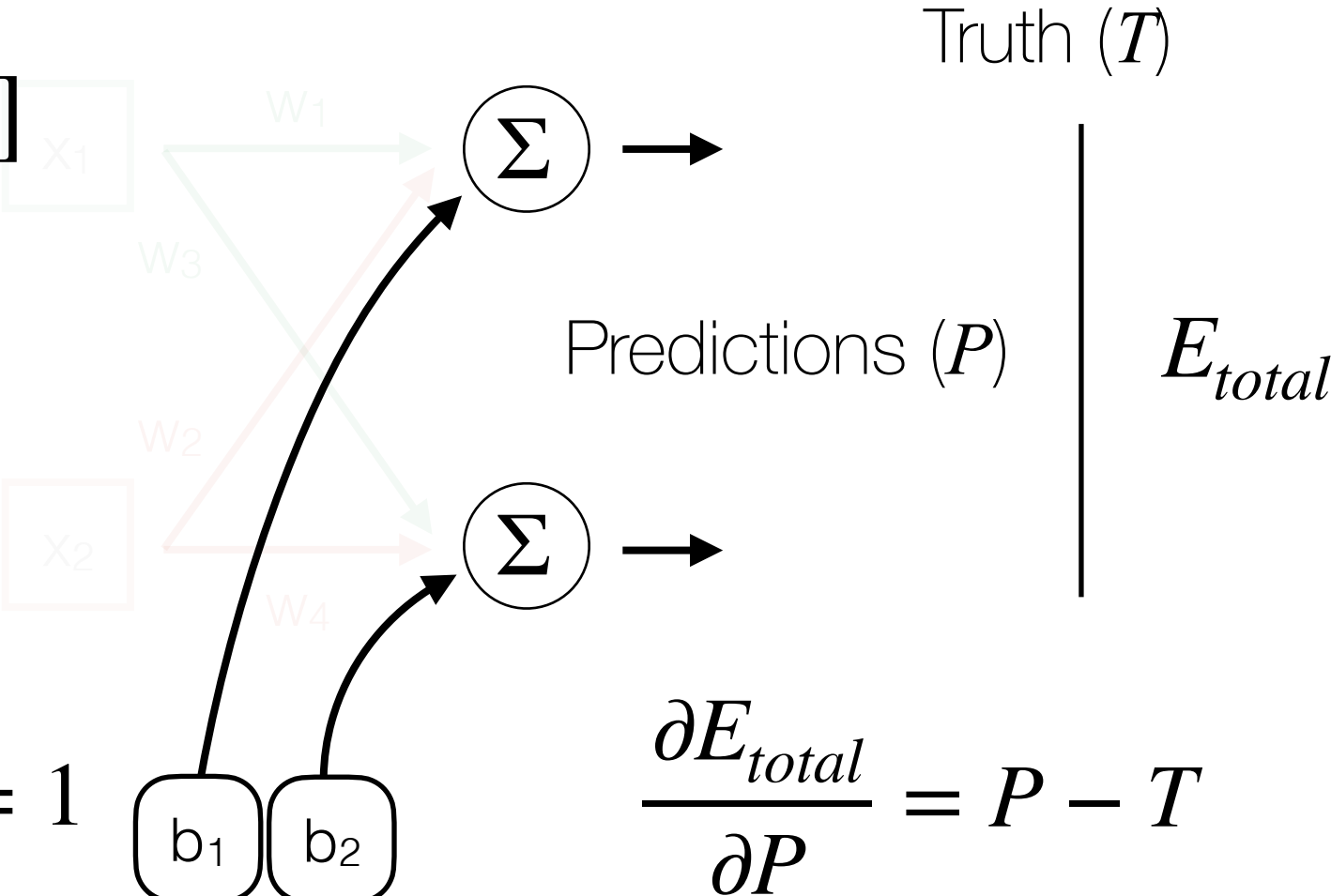
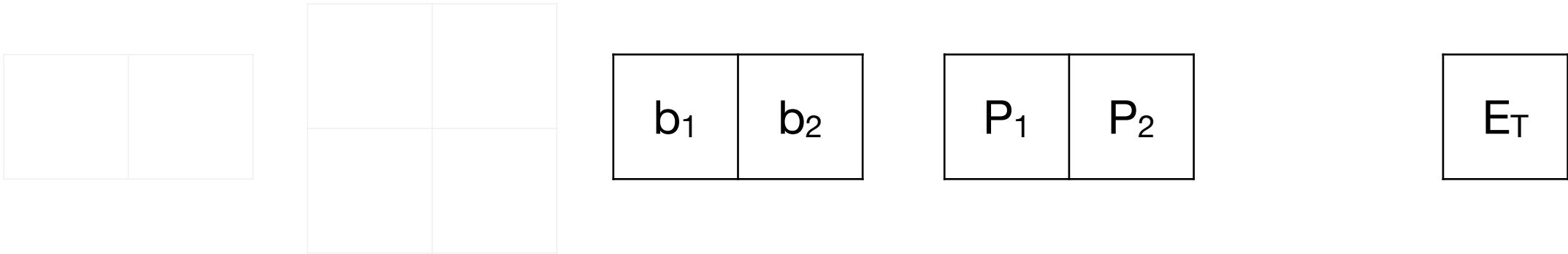
$[?; ?] = [?; ?] * [?; ?]$

$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$

$\frac{\partial P}{\partial b} = 1$

$\frac{\partial E_{total}}{\partial P} = P - T$

$X * W + b = P \xrightarrow{T} E_{total}$



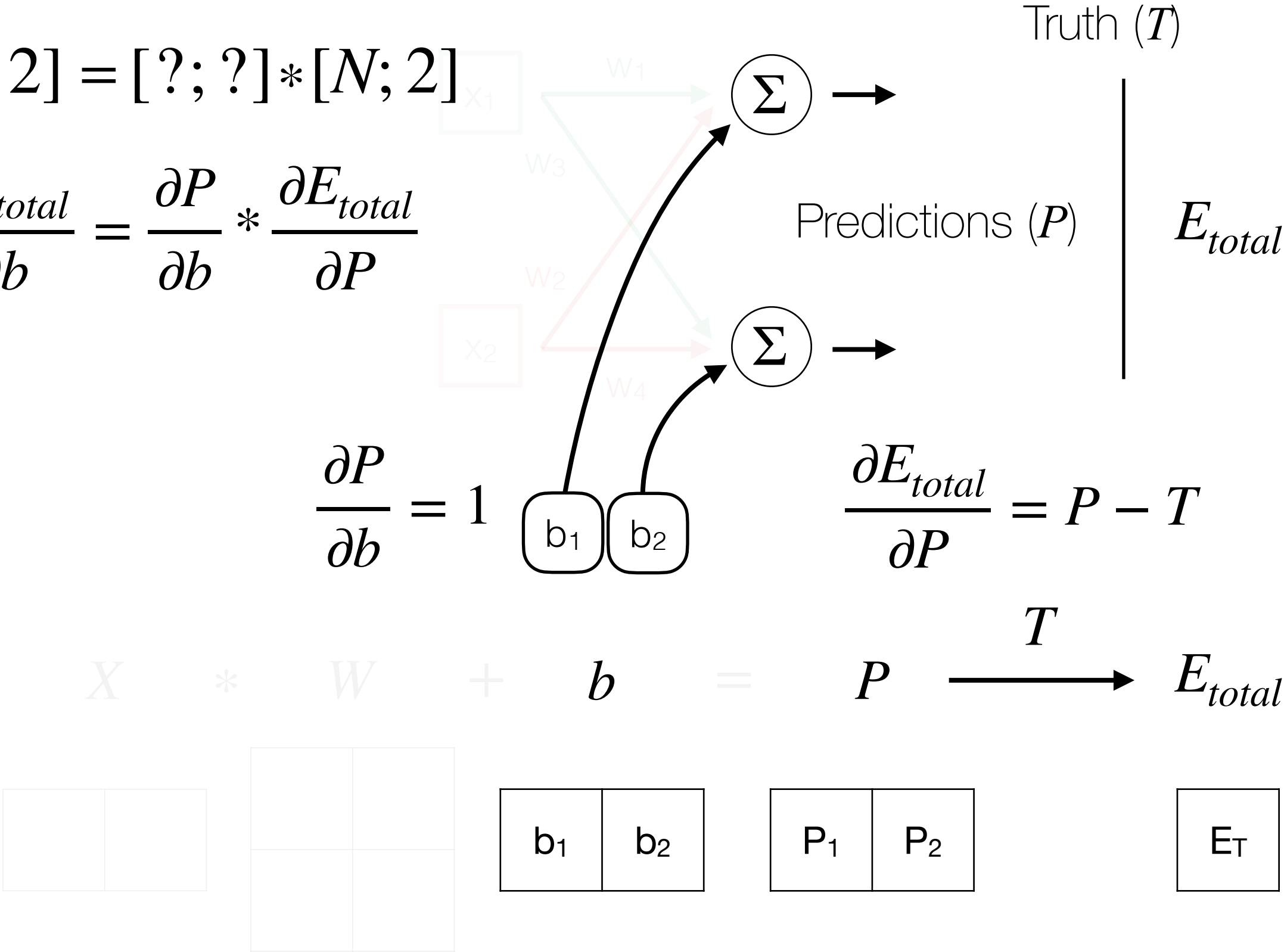
Estimate how much b influence E_{total}

$[1; 2] = [?; ?] * [N; 2]$

$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$

$\frac{\partial P}{\partial b} = 1$

$\frac{\partial E_{total}}{\partial P} = P - T$



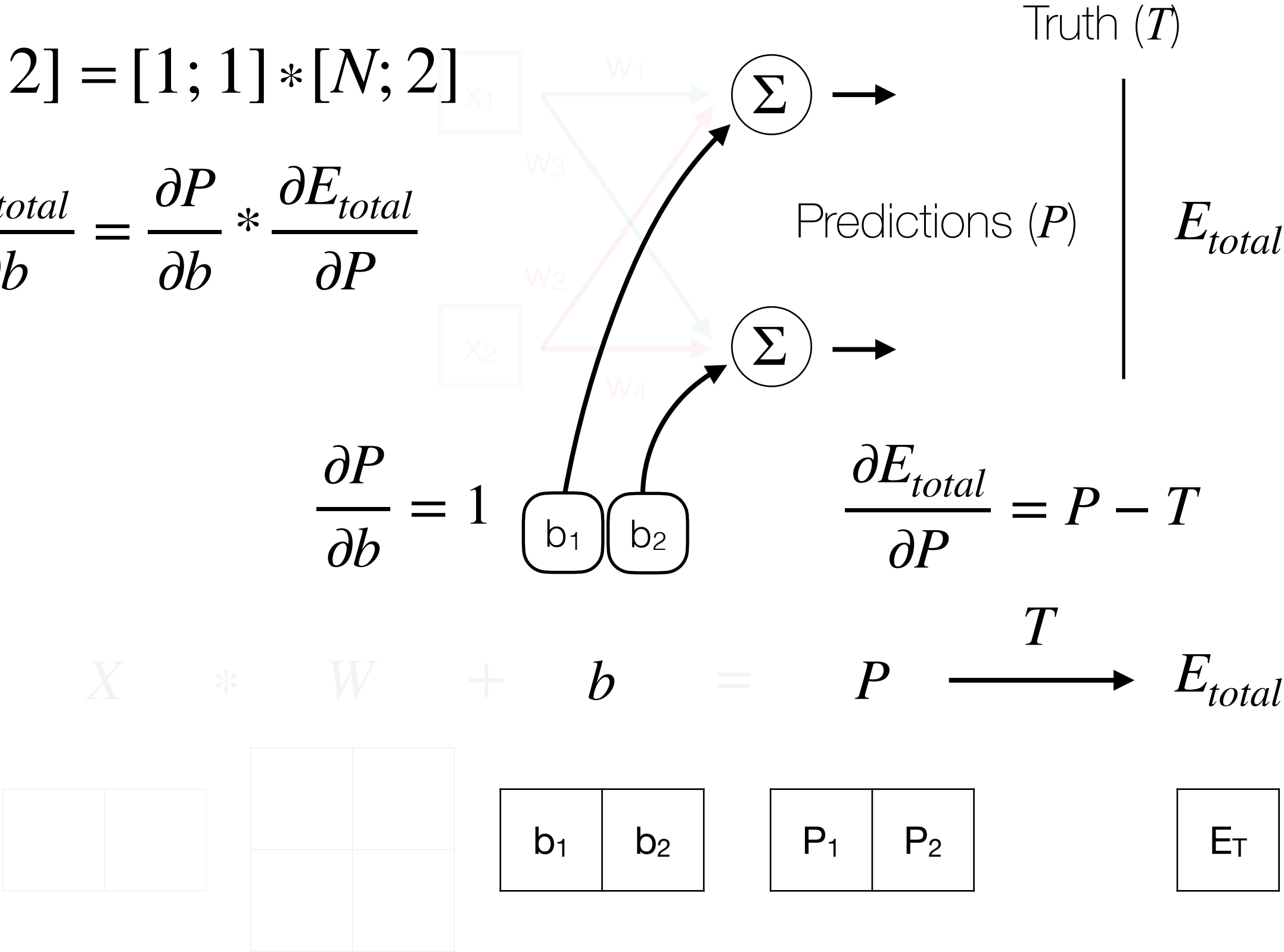
Estimate how much b influence E_{total}

$[1; 2] = [1; 1] * [N; 2]$

$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$

$\frac{\partial P}{\partial b} = 1$

$\frac{\partial E_{total}}{\partial P} = P - T$



Estimate how much b influence E_{total}

$$[1; 2] = [1; 1] * [N; 2]$$

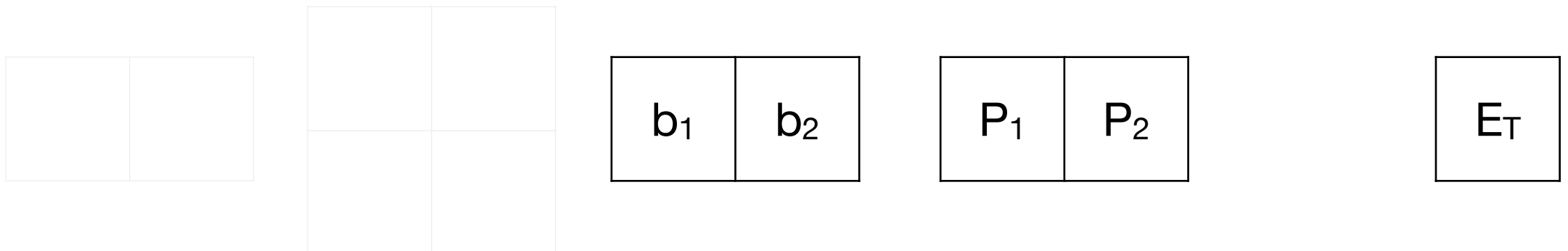
$$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$$

How is this
possible?

$$\frac{\partial P}{\partial b} = 1$$

$$\frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



Estimate how much b influence E_{total}

$$[1; 2] = [1; 1] * [N; 2]$$

$$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$$

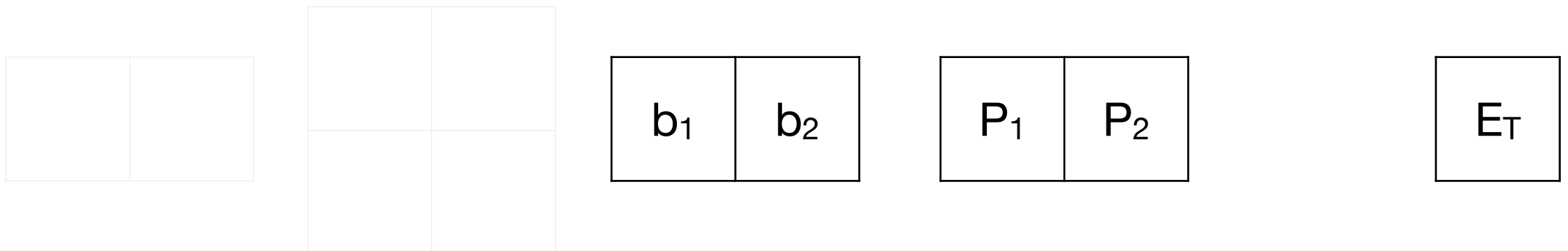
How is this
possible?

It is not.

$$\frac{\partial P}{\partial b} = 1$$

$$\frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



Estimate how much b influence E_{total}

$$[1; 2] = [1; 1] * [N; 2]$$

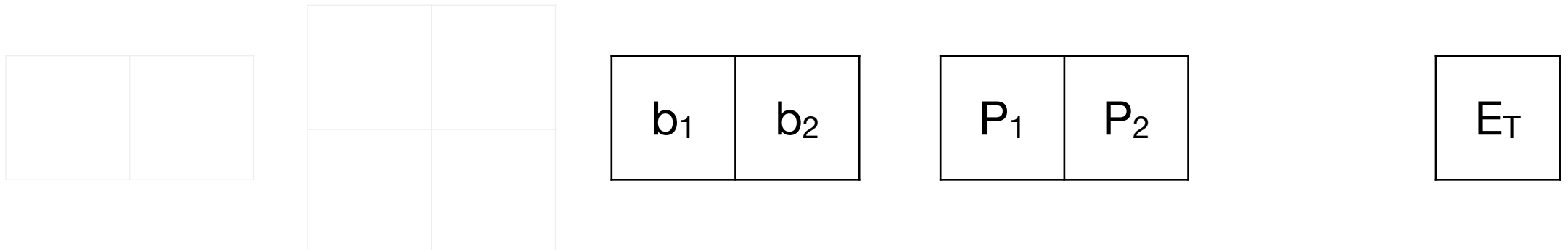
$$\frac{\partial E_{total}}{\partial b} = \frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P}$$

How is this
possible?

It is not.

How can we
make it
possible?

$$X * W + b = P \xrightarrow{T} E_{total}$$



Estimate how much b influence E_{total}

$$[1; 2] = \sum [1; 1] * [N; 2]$$

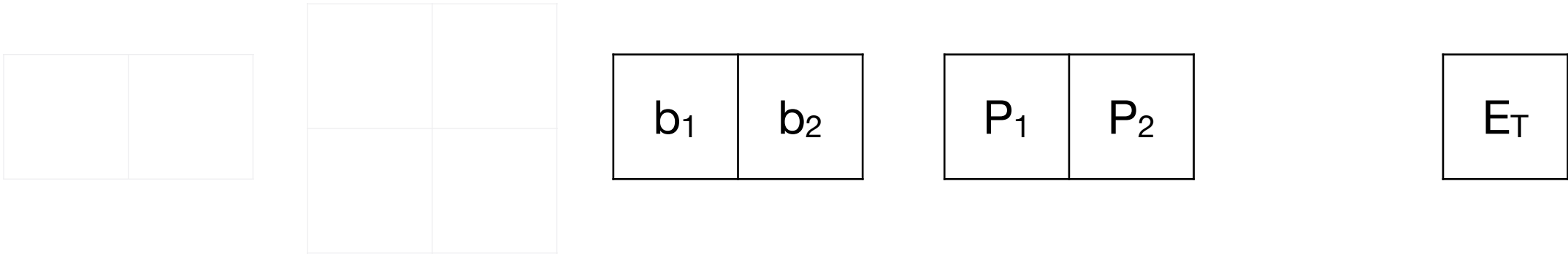
$$\frac{\partial E_{total}}{\partial b} = \left(\sum \left(\frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P} \right) \right)$$

Sum across
rows

$$\frac{\partial P}{\partial b} = 1$$

$$\frac{\partial E_{total}}{\partial P} = P - T$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



Estimate how much b influence E_{total}

$$[1; 2] = \sum [1; 1] * [N; 2] = [1; 2]$$

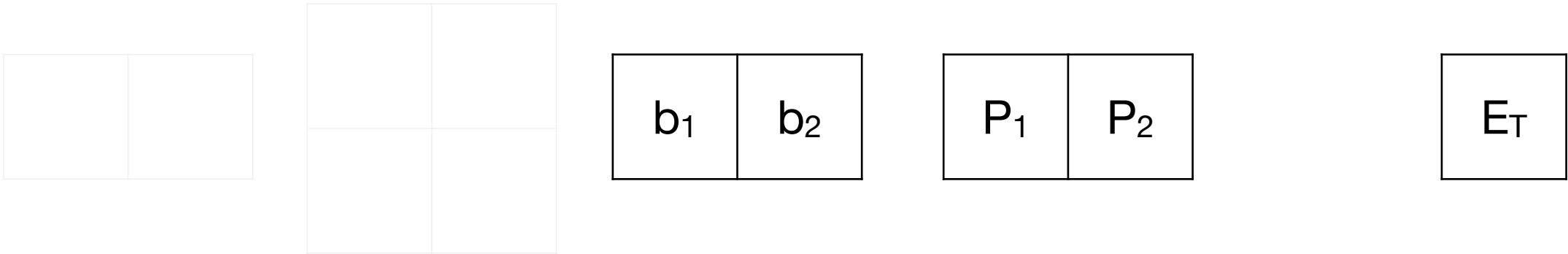
$$\frac{\partial E_{total}}{\partial b} = \left(\sum \left(\frac{\partial P}{\partial b} * \frac{\partial E_{total}}{\partial P} \right) \right)$$

Sum across
rows

$$\frac{\partial P}{\partial b} = 1$$

$$\frac{\partial E_{total}}{\partial P} = T - P$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



Now that we finally know db and dW we can update original weights and biases

Updated

w_1	w_3
w_2	w_4

$=$

Now that we finally know db and dW we can update original weights and biases

Updated

$$\begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array} = \begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array} -$$

Now that we finally know db and dW we can update original weights and biases

$$\begin{array}{c} \text{Updated} \end{array}
 \begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array}
 =
 \begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array}
 -
 \begin{array}{c} \frac{\partial E_{total}}{\partial W} \\ \begin{array}{|c|c|} \hline dw_1 & dw_3 \\ \hline dw_2 & dw_4 \\ \hline \end{array} \end{array}
 *
 \begin{array}{c} \frac{\partial E_{total}}{\partial W} \\ \begin{array}{|c|c|} \hline dw_1 & dw_3 \\ \hline dw_2 & dw_4 \\ \hline \end{array} \end{array}$$

Now that we finally know db and dW we can update original weights and biases

$$\begin{array}{c} \text{Updated} \end{array}
 \begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array}
 =
 \begin{array}{c} W \\ \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array} \end{array}
 -
 \underset{\substack{\uparrow \\ \text{Learning} \\ \text{rate}}}{\eta}
 *
 \begin{array}{c} \frac{\partial E_{total}}{\partial W} \\ \begin{array}{|c|c|} \hline dw_1 & dw_3 \\ \hline dw_2 & dw_4 \\ \hline \end{array} \end{array}$$

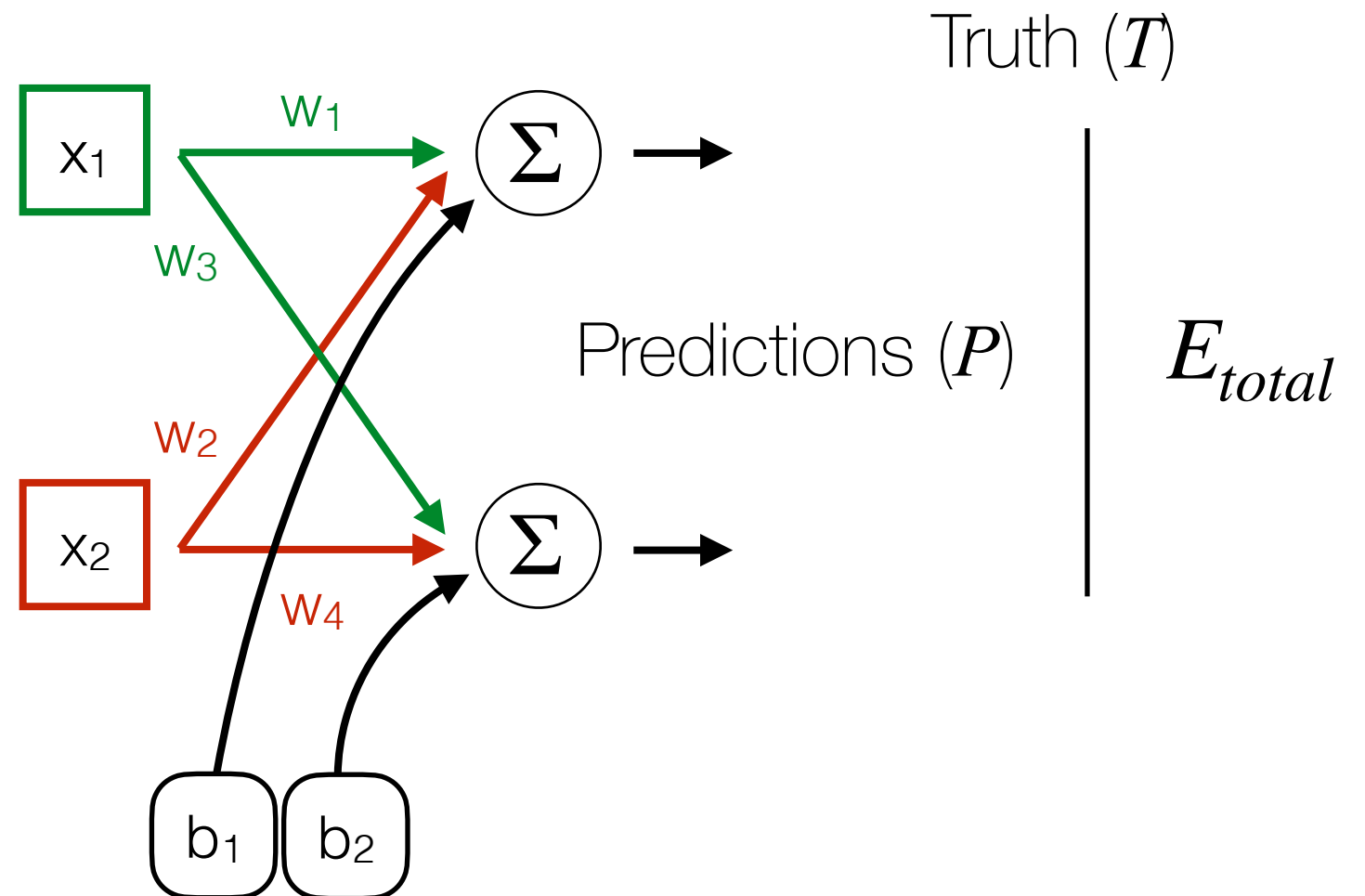
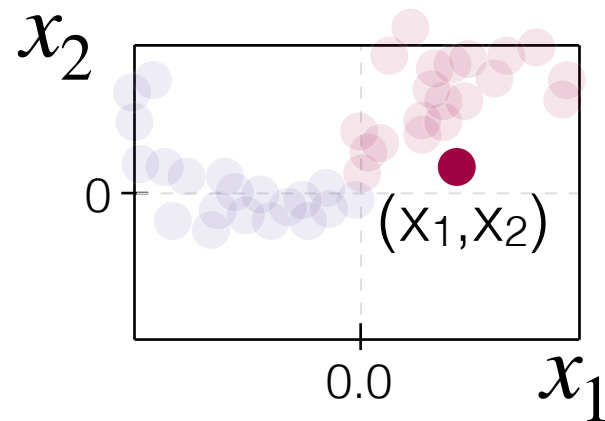
Now that we finally know db and dW we can update original weights and biases

$$\begin{array}{c}
 \text{Updated} \\
 W \\
 \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 W \\
 \begin{array}{|c|c|} \hline w_1 & w_3 \\ \hline w_2 & w_4 \\ \hline \end{array}
 \end{array}
 -
 \begin{array}{c}
 \eta \\
 \uparrow \\
 \text{Learning rate} \\
 \downarrow
 \end{array}
 *
 \begin{array}{c}
 \frac{\partial E_{total}}{\partial W} \\
 \begin{array}{|c|c|} \hline dw_1 & dw_3 \\ \hline dw_2 & dw_4 \\ \hline \end{array}
 \end{array}$$

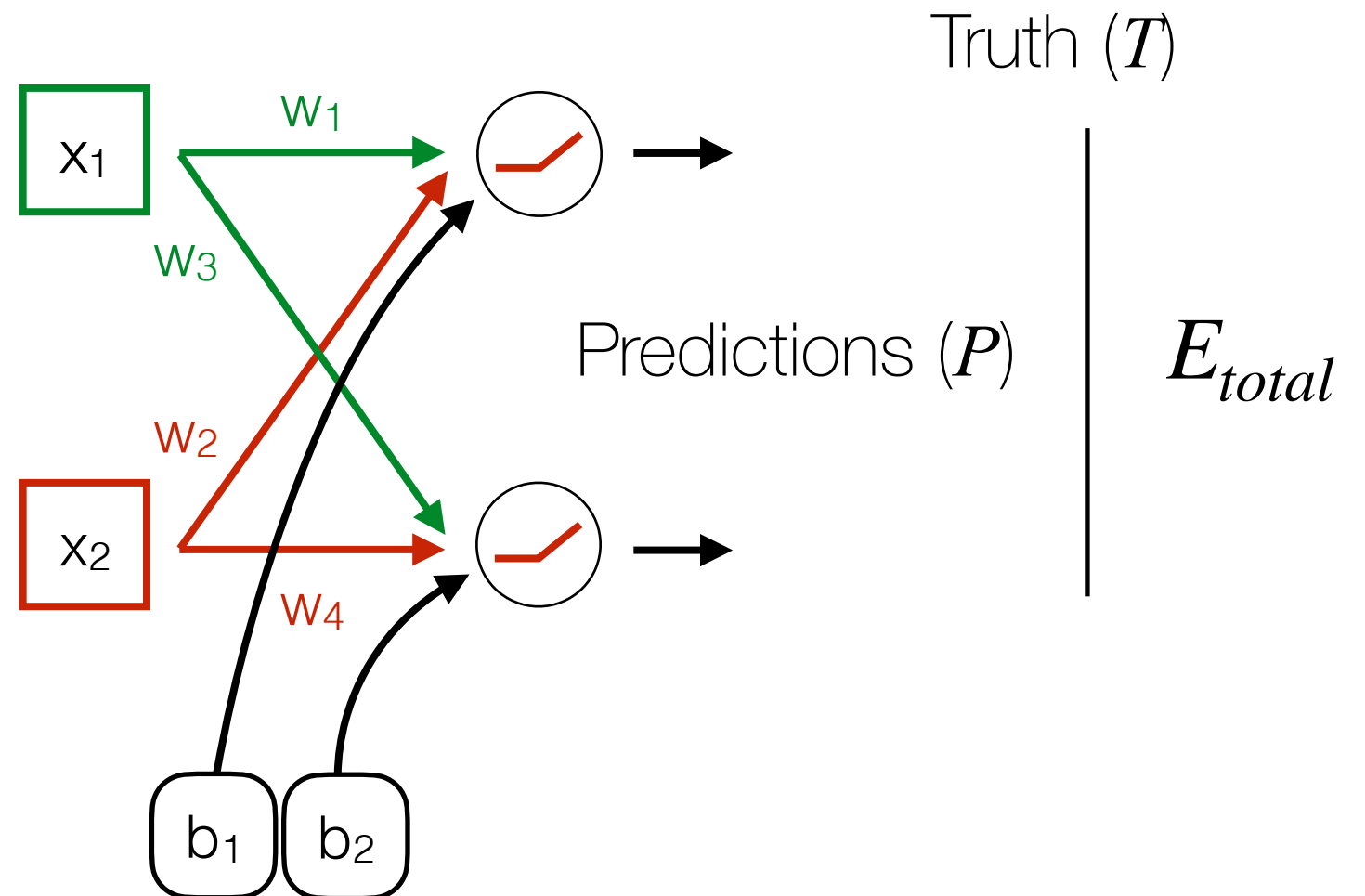
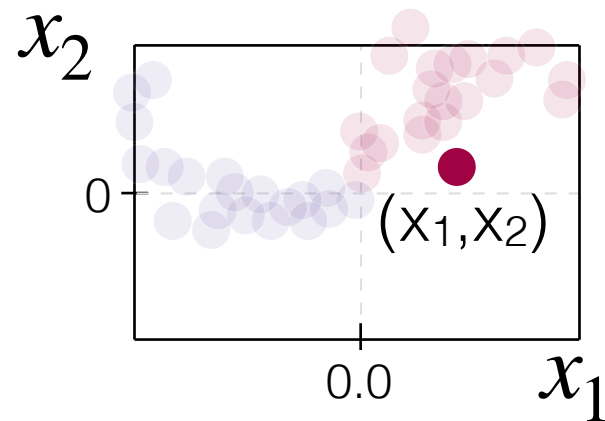
$$\begin{array}{c}
 \text{Updated} \\
 b \\
 \begin{array}{|c|c|} \hline b_1 & b_2 \\ \hline \end{array}
 \end{array}
 =
 \begin{array}{c}
 b \\
 \begin{array}{|c|c|} \hline b_1 & b_2 \\ \hline \end{array}
 \end{array}
 -
 \begin{array}{c}
 \eta \\
 \downarrow \\
 \text{Learning rate} \\
 \uparrow
 \end{array}
 *
 \begin{array}{c}
 \frac{\partial E_{total}}{\partial b} \\
 \begin{array}{|c|c|} \hline db_1 & db_2 \\ \hline \end{array}
 \end{array}$$

What would change if we had an **activation function**? (e.g. **ReLu**)

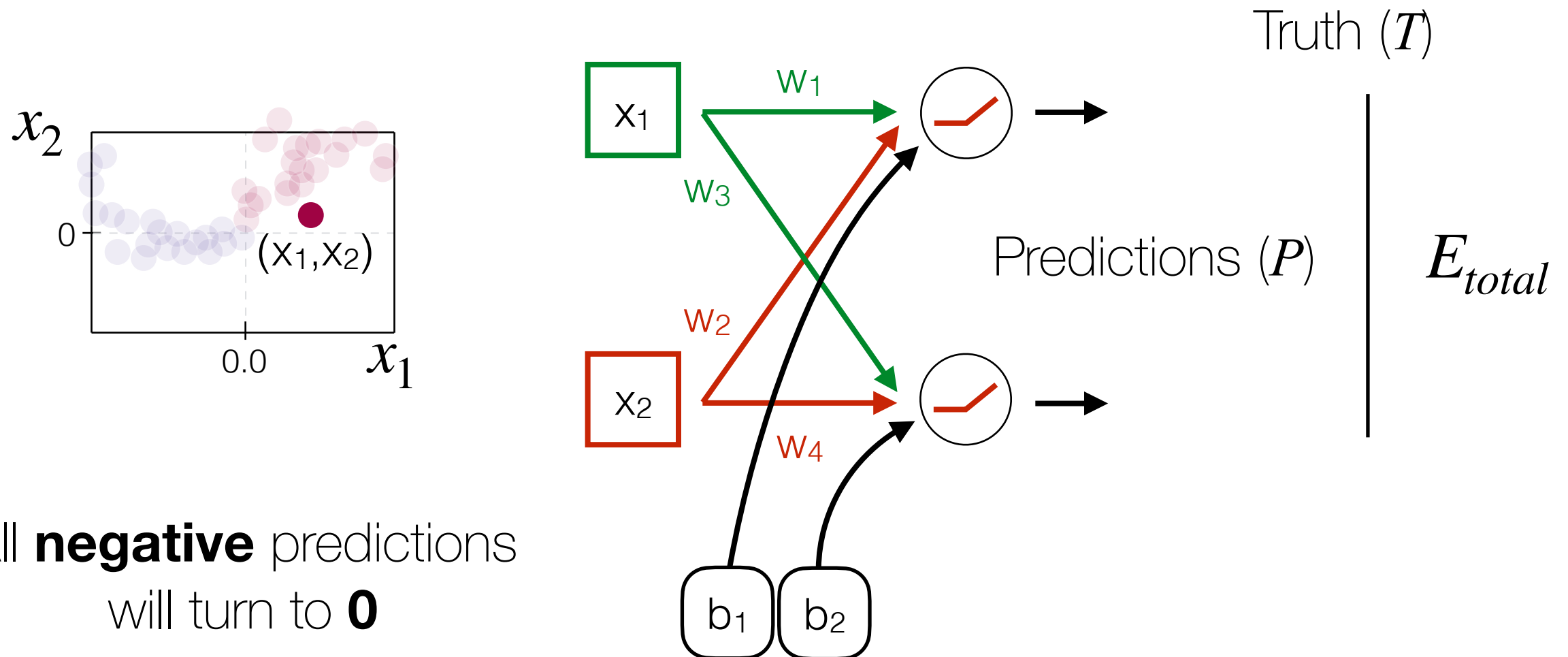
What would change if we had an **activation function**? (e.g. **ReLU**)



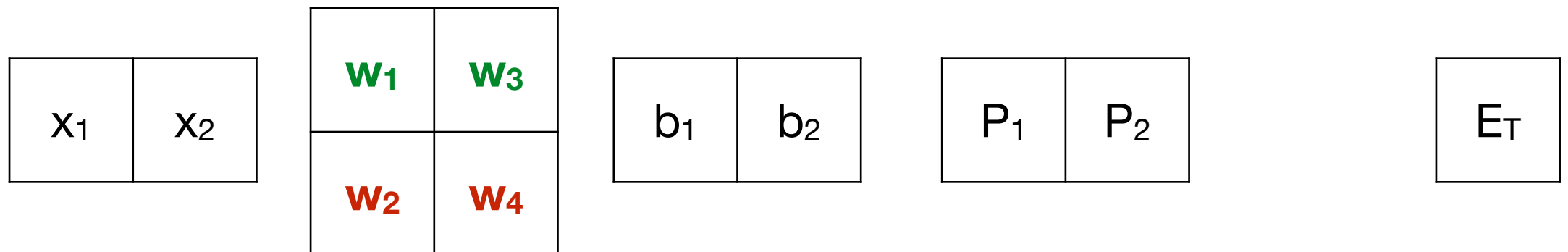
What would change if we had an **activation function**? (e.g. **ReLU**)



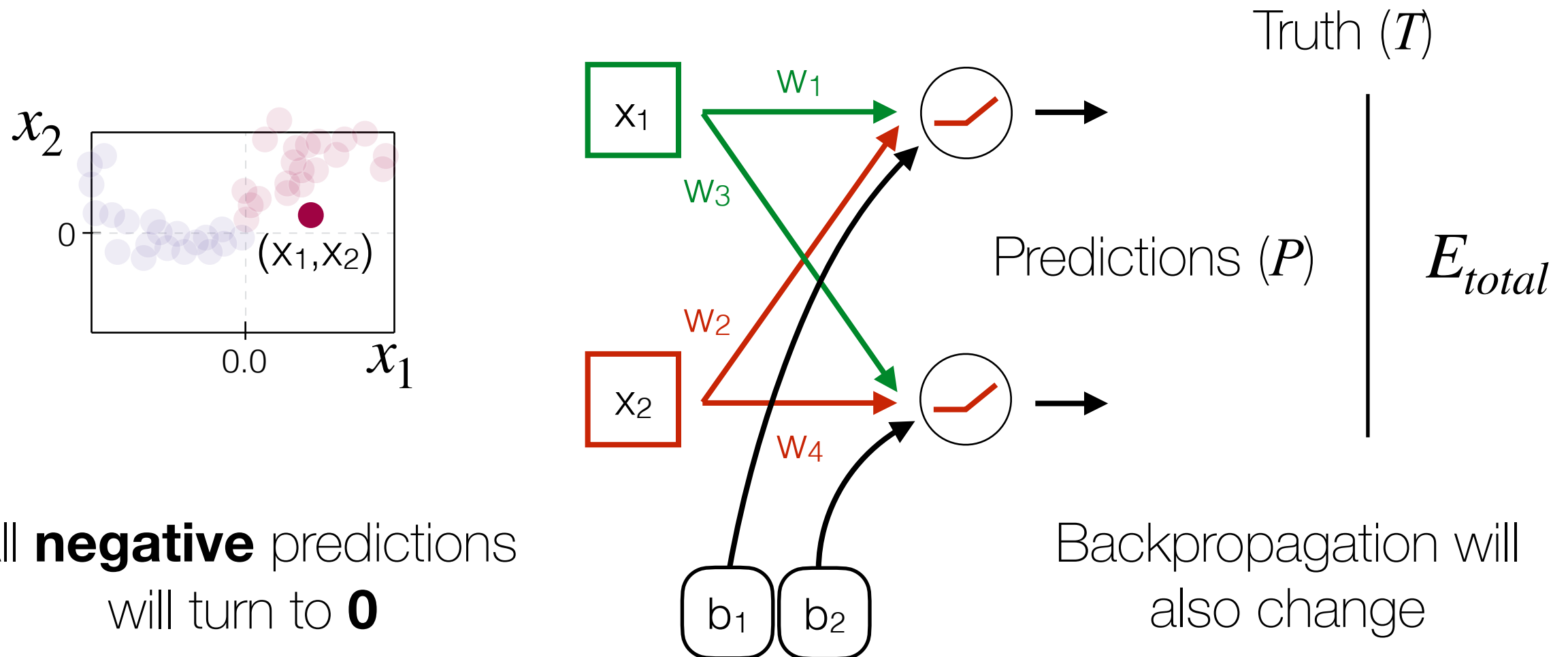
What would change if we had an **activation function**? (e.g. **ReLu**)



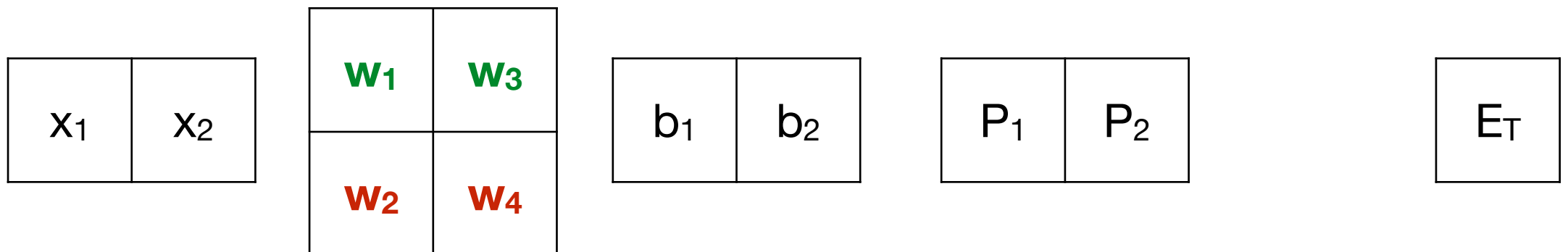
$$ReLU (X * W + b) = P \xrightarrow{T} E_{total}$$



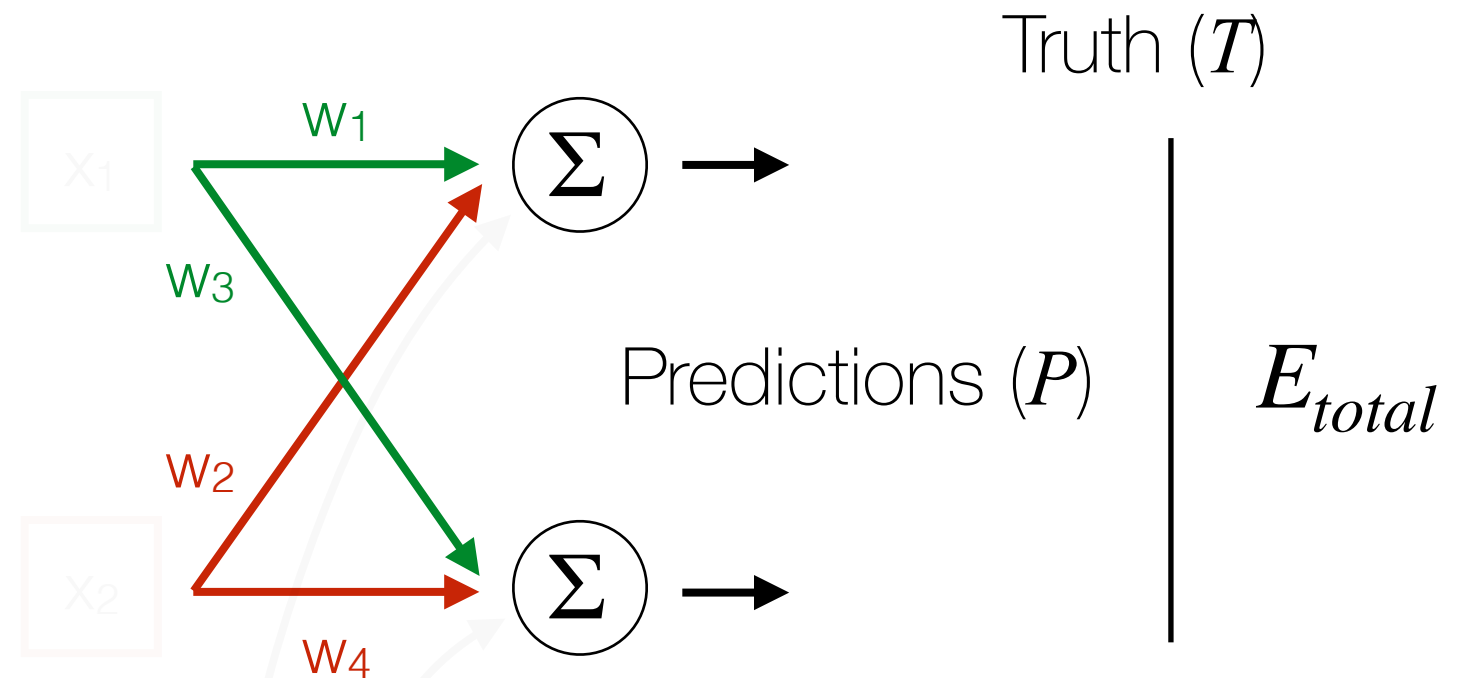
What would change if we had an **activation function**? (e.g. **ReLu**)



$$ReLU (X * W + b) = P \xrightarrow{T} E_{total}$$



We finally know how W influence E_{total} !



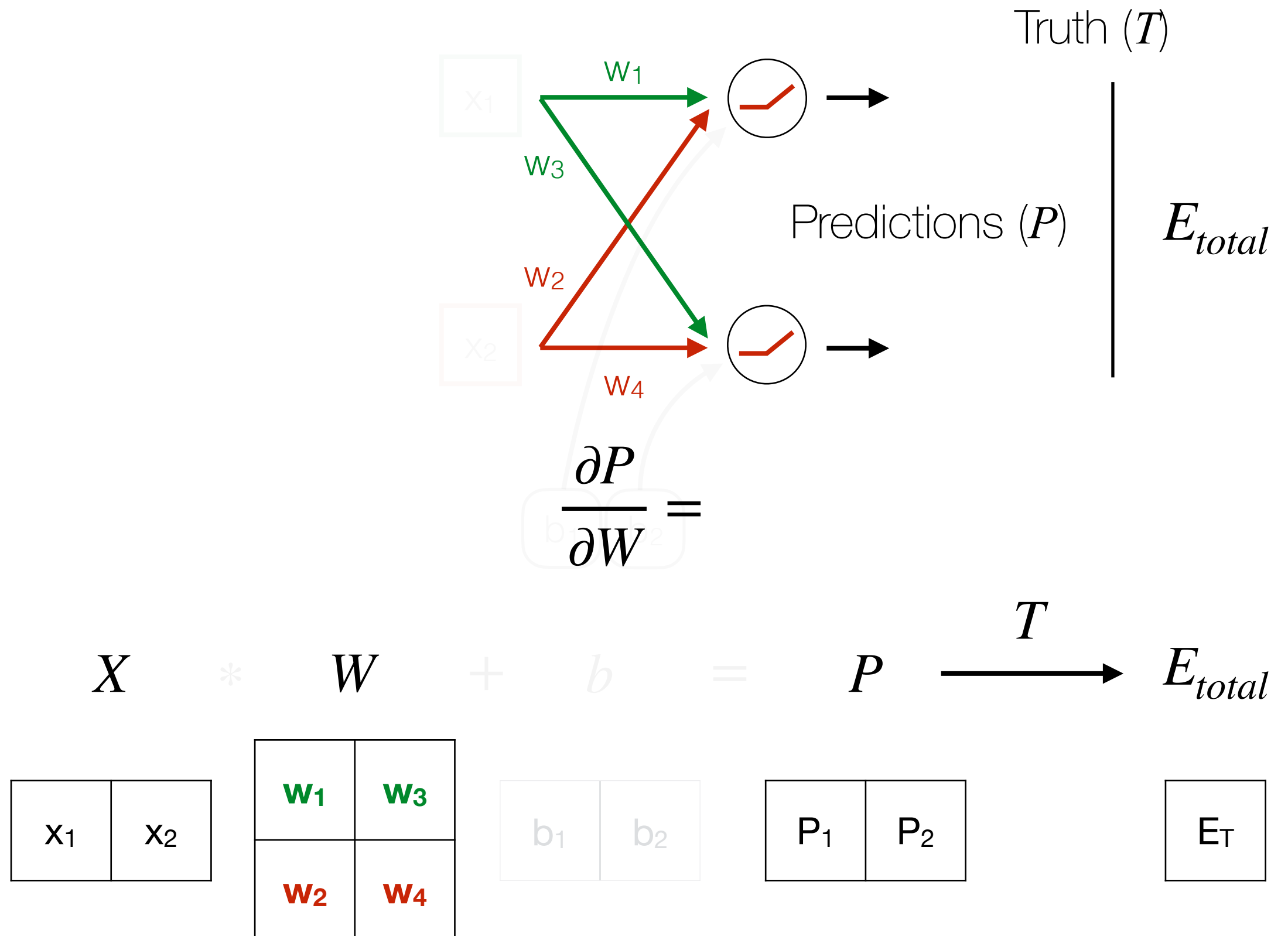
$$\frac{\partial E_{total}}{\partial W} = \frac{\partial P}{\partial W} * \frac{\partial E_{total}}{\partial P}$$

$$\frac{\partial P}{\partial W} = X^T \quad \frac{\partial E_{total}}{\partial P} = P - T$$

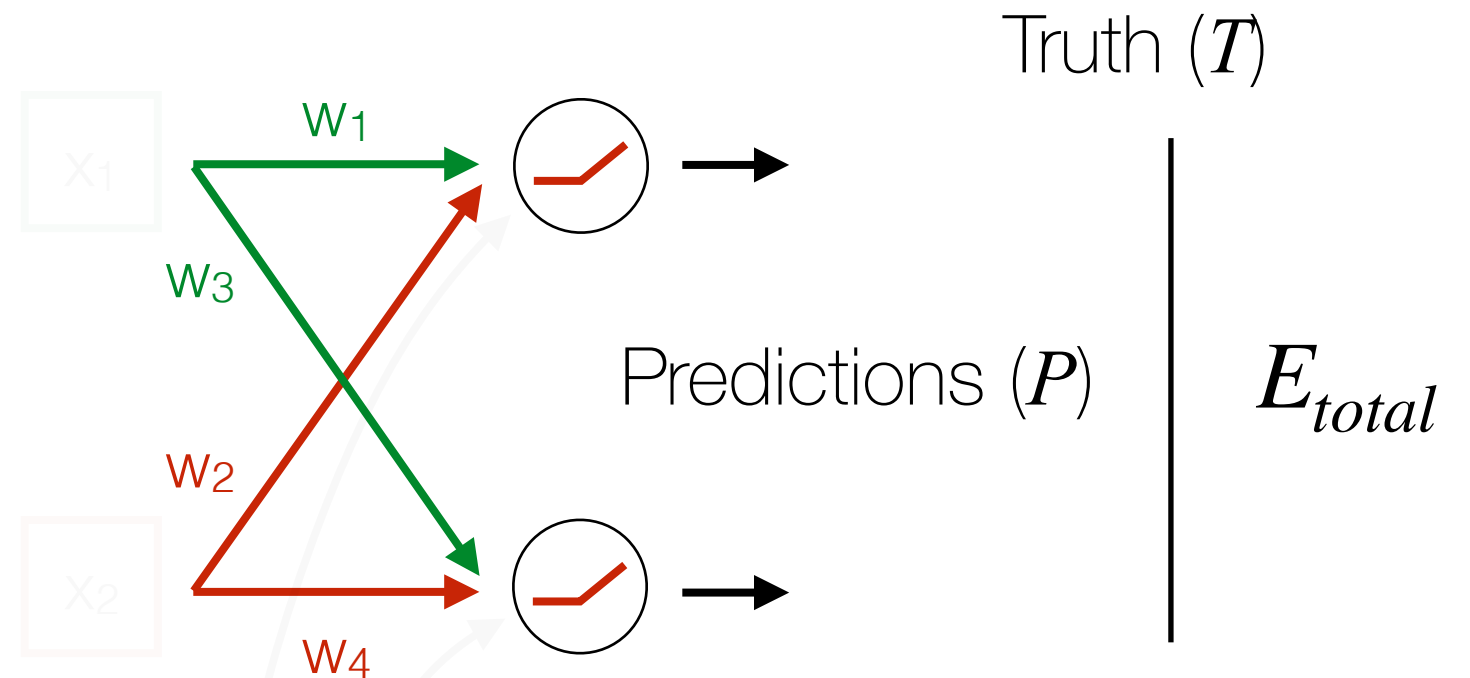
$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence P ?



How does W influence P ?

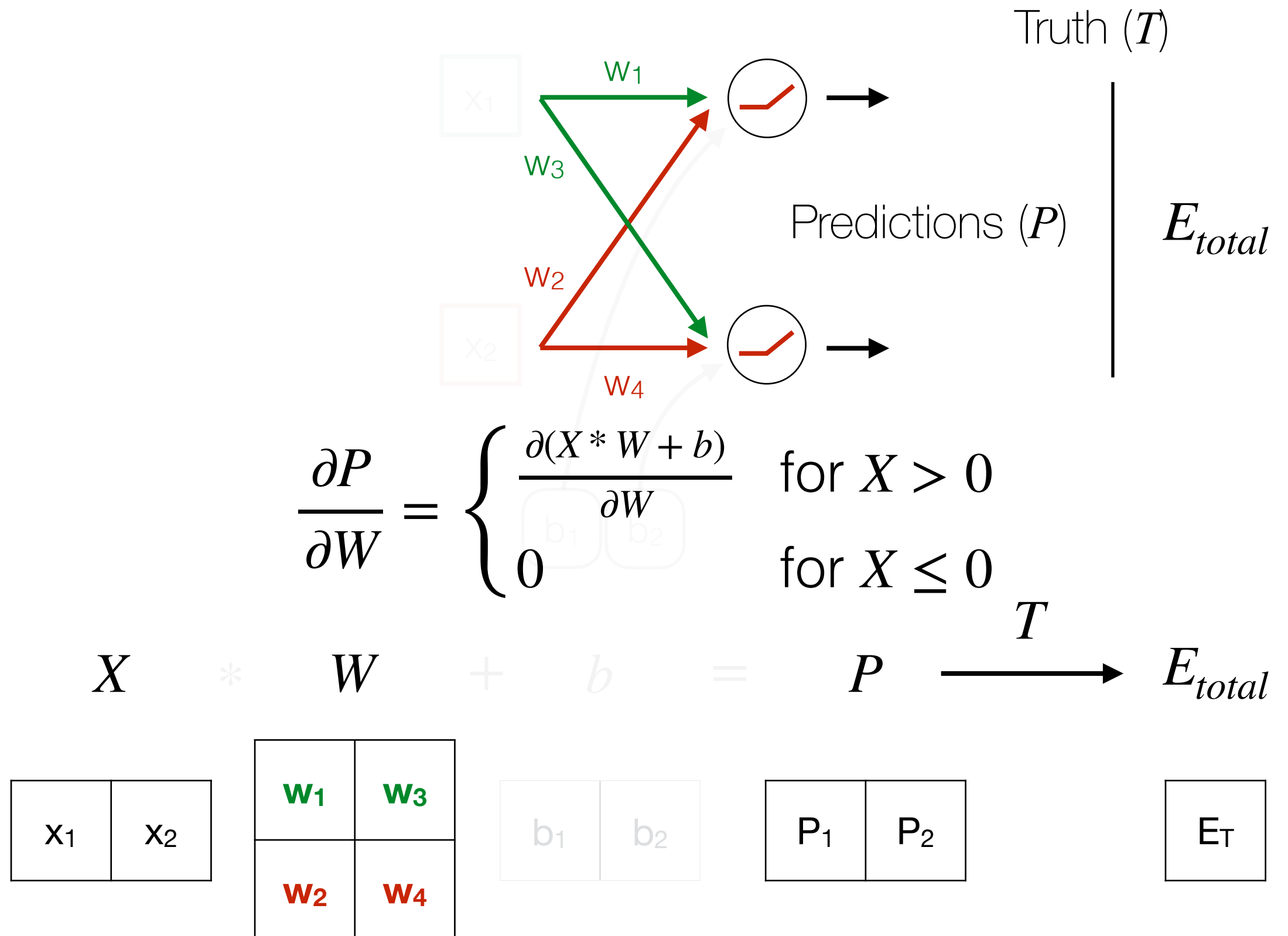


$$\frac{\partial P}{\partial W} = \frac{\partial(\max(0, (X * W + b)))}{\partial W}$$

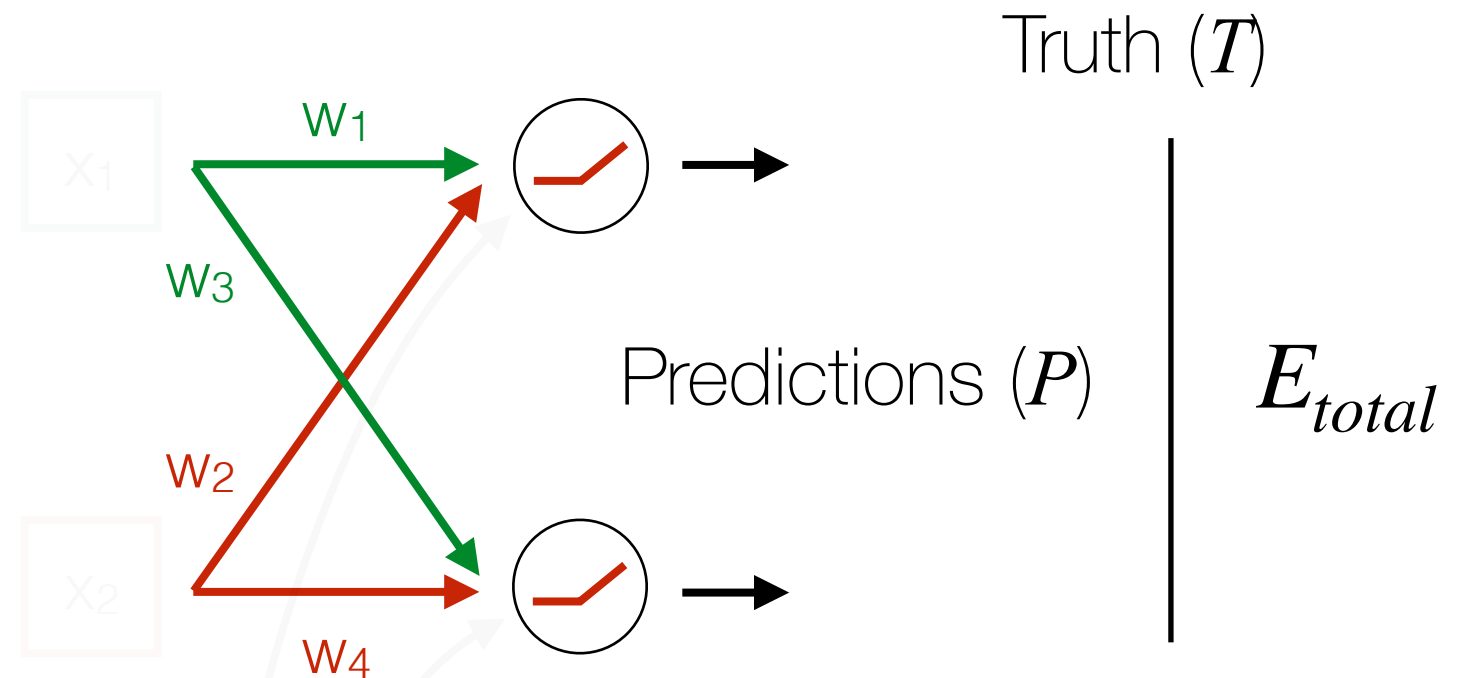
$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence P ?



How does W influence P ?

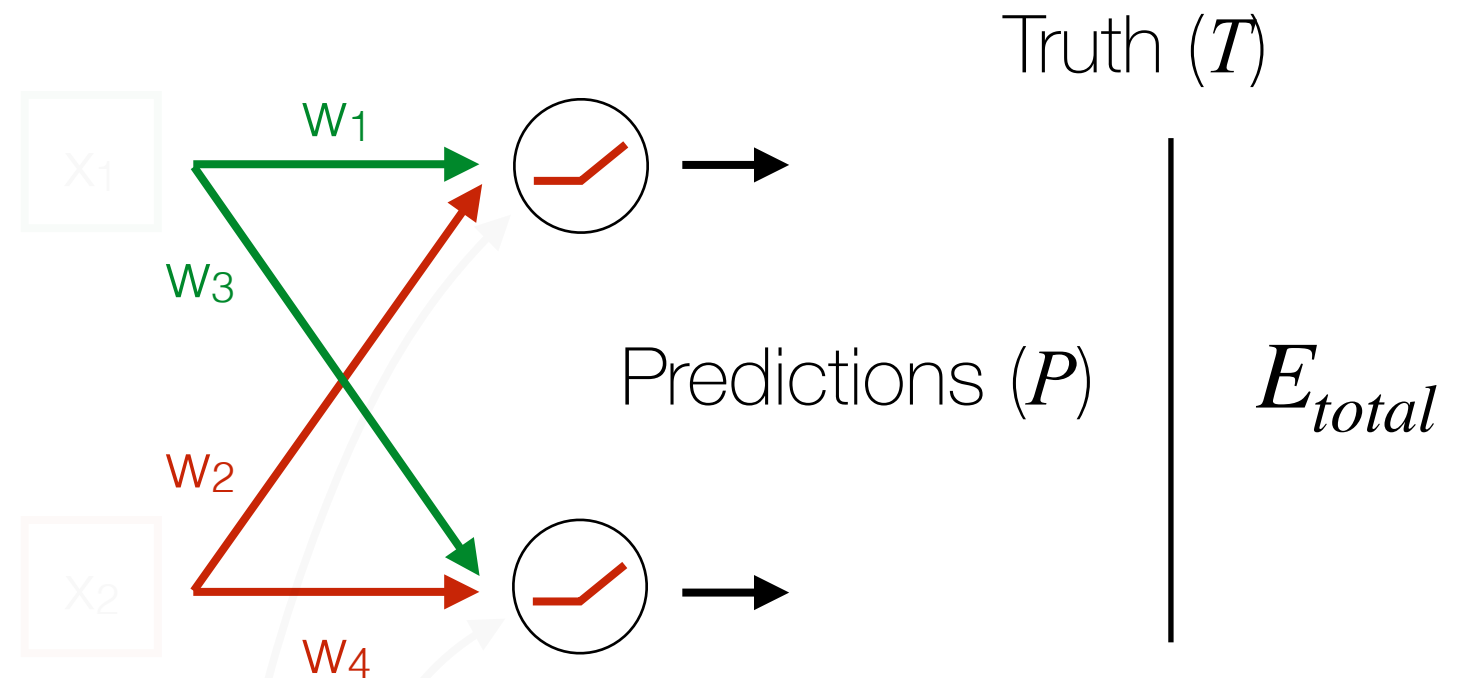


$$\frac{\partial P}{\partial W} = \begin{cases} X^T & \text{for } X > 0 \\ 0 & \text{for } X \leq 0 \end{cases}$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



How does W influence P ?



This would be X with **0**s instead of **negative** values

$$\frac{\partial P}{\partial W} = \begin{cases} X^T & \text{for } X > 0 \\ 0 & \text{for } X \leq 0 \end{cases}$$

$$X * W + b = P \xrightarrow{T} E_{total}$$



Training Neural Networks

(part I)



<http://playground.tensorflow.org/>

Resources:

Brandon Rohrer's youtube video: How Deep Neural Networks Work (<https://youtu.be/ILsA4nyG7I0>)

Stanford University **CS231n** Convolutional Neural Networks for Visual Recognition (github.io version): <http://cs231n.github.io/>

Matt Mazur's: A Step by Step Backpropagation Example (<https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>)

Andrej Karpathy's blog post: Yes you should understand backprop (<https://medium.com/@karpathy/yes-you-should-understand-backprop-e2f06eab496b>)

Raul Vicente's lecture: From brain to Deep Learning and back (<https://www.uttv.ee/naita?id=23585>)

Mayank Agarwal's blog: Back Propagation in Convolutional Neural Networks — Intuition and Code (<https://becominghuman.ai/back-propagation-in-convolutional-neural-networks-intuition-and-code-714ef1c38199>)

That's all Folks!

BIIT



www.slideshare.net/DimaFishman